agent_framework Package

Packages

Expand table

a2a
ag_ui
anthropic
azure
chatkit
declarative
devui
lab
mem0
microsoft
ollama
openai
redis

Modules

Expand table

exceptions
observability

Classes

Expand table

AIFunction	A tool that wraps a Python function to make it callable by AI models.

	This class wraps a Python function to make it callable by AI models with automatic parameter validation and JSON schema generation. Initialize the AIFunction.
AgentExecutor	built-in executor that wraps an agent for handling messages. AgentExecutor adapts its behavior based on the workflow execution mode: • run_stream(): Emits incremental AgentRunUpdateEvent events as the agent produces tokens • run(): Emits a single AgentRunEvent containing the complete response The executor automatically detects the mode via WorkflowContext.is_streaming(). Initialize the executor with a unique identifier.
AgentExecutorRequest	A request to an agent executor.
AgentExecutorResponse	A response from an agent executor.
AgentInputRequest	Request for human input before an agent runs in high-level builder workflows. Emitted via RequestInfoEvent when a workflow pauses before an agent executes. The response is injected into the conversation as a user message to steer the agent's behavior. This is the standard request type used by *.with_request_info()* on SequentialBuilder, ConcurrentBuilder, GroupChatBuilder, and HandoffBuilder.
AgentMiddleware	Abstract base class for agent middleware that can intercept agent invocations. Agent middleware allows you to intercept and modify agent invocations before and after execution. You can inspect messages, modify context, override results, or terminate execution early. **ⓘ Note** AgentMiddleware is an abstract base class. You must subclass it and implement the process() method to create custom agent middleware.
AgentProtocol	A protocol for an agent that can be invoked.

	This protocol defines the interface that all agents must implement, including properties for identification and methods for execution. **Note** Protocols use structural subtyping (duck typing). Classes don't need to explicitly inherit from this protocol to be considered compatible. This allows you to create completely custom agents without using any Agent Framework base classes.
AgentRunContext	Context object for agent middleware invocations. This context is passed through the agent middleware pipeline and contains all information about the agent invocation. Initialize the AgentRunContext.
AgentRunEvent	Event triggered when an agent run is completed. Initialize the agent run event.
AgentRunResponse	Represents the response to an Agent run request. Provides one or more response messages and metadata about the response. A typical response will contain a single message, but may contain multiple messages in scenarios involving function calls, RAG retrievals, or complex logic. Initialize an AgentRunResponse.
AgentRunResponseUpdate	Represents a single streaming response chunk from an Agent. Initialize an AgentRunResponseUpdate.
AgentRunUpdateEvent	Event triggered when an agent is streaming messages. Initialize the agent streaming event.
AgentThread	The Agent thread class, this can represent both a locally managed thread or a thread managed by the service. An `AgentThread` maintains the conversation state and message history for an agent interaction. It can either use a service-managed thread (via `service_thread_id`) or a local message store (via `message_store`), but not both.

	Initialize an AgentThread, do not use this method manually, always use: `agent.get_new_thread()`. **Note** Either service_thread_id or message_store may be set, but not both.
AggregateContextProvider	A ContextProvider that contains multiple context providers. It delegates events to multiple context providers and aggregates responses from those events before returning. This allows you to combine multiple context providers into a single provider. **Note** An AggregateContextProvider is created automatically when you pass a single context provider or a sequence of context providers to the agent constructor. Initialize the AggregateContextProvider with context providers.
BaseAgent	Base class for all Agent Framework agents. This class provides core functionality for agent implementations, including context providers, middleware support, and thread management. **Note** BaseAgent cannot be instantiated directly as it doesn't implement the run(), run_stream(), and other methods required by AgentProtocol. Use a concrete implementation like ChatAgent or create a subclass. Initialize a BaseAgent instance.
BaseAnnotation	Base class for all AI Annotation types.

	Initialize BaseAnnotation.
BaseChatClient	Base class for chat clients. This abstract base class provides core functionality for chat client implementations, including middleware support, message preparation, and tool normalization. **Note** BaseChatClient cannot be instantiated directly as it's an abstract base class. Subclasses must implement _inner_get_response() and _inner_get_streaming_response(). Initialize a BaseChatClient instance.
BaseContent	Represents content used by AI services. Initialize BaseContent.
Case	Runtime wrapper combining a switch-case predicate with its target. Each *Case* couples a boolean predicate with the executor that should handle the message when the predicate evaluates to *True*. The runtime keeps this lightweight container separate from the serialisable *SwitchCaseEdgeGroupCase* so that execution can operate with live callables without polluting persisted state.
ChatAgent	A Chat Client Agent. This is the primary agent implementation that uses a chat client to interact with language models. It supports tools, context providers, middleware, and both streaming and non-streaming responses. Initialize a ChatAgent instance. **Note** The set of parameters from frequency_penalty to request_kwargs are used to call the chat client. They can also be passed to both run methods. When both are set, the ones passed to the run methods take precedence.

ChatClientProtocol	A protocol for a chat client that can generate responses. This protocol defines the interface that all chat clients must implement, including methods for generating both streaming and non-streaming responses. **Note** Protocols use structural subtyping (duck typing). Classes don't need to explicitly inherit from this protocol to be considered compatible.
ChatContext	Context object for chat middleware invocations. This context is passed through the chat middleware pipeline and contains all information about the chat request. Initialize the ChatContext.
ChatMessage	Represents a chat message. Initialize ChatMessage.
ChatMessageStore	An in-memory implementation of ChatMessageStoreProtocol that stores messages in a list. This implementation provides a simple, list-based storage for chat messages with support for serialization and deserialization. It implements all the required methods of the `ChatMessageStoreProtocol` protocol. The store maintains messages in memory and provides methods to serialize and deserialize the state for persistence purposes. Create a ChatMessageStore for use in a thread.
ChatMessageStoreProtocol	Defines methods for storing and retrieving chat messages associated with a specific thread. Implementations of this protocol are responsible for managing the storage of chat messages, including handling large volumes of data by truncating or summarizing messages as necessary.
ChatMiddleware	Abstract base class for chat middleware that can intercept chat client requests. Chat middleware allows you to intercept and modify chat client requests before and after execution. You can modify messages, add system prompts, log requests, or override chat responses.

	Note ChatMiddleware is an abstract base class. You must subclass it and implement the process() method to create custom chat middleware.
ChatOptions	Common request settings for AI services. Initialize ChatOptions.
ChatResponse	Represents the response to a chat request. Initializes a ChatResponse with the provided parameters.
ChatResponseUpdate	Represents a single streaming response chunk from a *ChatClient*. Initializes a ChatResponseUpdate with the provided parameters.
CheckpointStorage	Protocol for checkpoint storage backends.
CitationAnnotation	Represents a citation annotation. Initialize CitationAnnotation.

ConcurrentBuilder	High-level builder for concurrent agent workflows.
	• *participants([...])* accepts a list of AgentProtocol (recommended) or Executor.
	• *register_participants([...])* accepts a list of factories for AgentProtocol (recommended)
	or Executor factories
	• *build()* wires: dispatcher -> fan-out -> participants -> fan-in -> aggregator.
	• *with_aggregator(...)* overrides the default aggregator with an Executor or callback.
	• *register_aggregator(...)* accepts a factory for an Executor as custom aggregator.
	Usage:
	Python
	`from agent_framework import ConcurrentBuilder`
	`# Minimal: use default aggregator (returns list[ChatMessage])`
	`workflow =`

```
ConcurrentBuilder().participants([agent1, agent2,
agent3]).build()

   # With agent factories
   workflow =
ConcurrentBuilder().register_participants([create_age
nt1, create_agent2, create_agent3]).build()


   # Custom aggregator via callback (sync or async).
The callback receives
   # list[AgentExecutorResponse] and its return value
becomes the workflow's output.
   def summarize(results:
list[AgentExecutorResponse]) -> str:
       return " |
".join(r.agent_run_response.messages[-1].text for r
in results)


   workflow =
ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_aggregator(summarize).build()


   # Custom aggregator via a factory
   class MyAggregator(Executor):
       @handler
       async def aggregate(self, results:
list[AgentExecutorResponse], ctx:
WorkflowContext[Never, str]) -> None:
           await ctx.yield_output(" |
".join(r.agent_run_response.messages[-1].text for r
in results))


   workflow = (
       ConcurrentBuilder()
       .register_participants([create_agent1,
create_agent2, create_agent3])
       .register_aggregator(lambda:
MyAggregator(id="my_aggregator"))
       .build()
   )


   # Enable checkpoint persistence so runs can resume
   workflow =
ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_checkpointing(storage).build()

   # Enable request info before aggregation
   workflow =
ConcurrentBuilder().participants([agent1,
agent2]).with_request_info().build()
```

Context	A class containing any context that should be provided to the AI model as supplied by a ContextProvider. Each ContextProvider has the ability to provide its own context for each invocation. The Context class contains the additional context supplied by the ContextProvider. This context will be combined with context supplied by other providers before being passed to the AI model. This context is per invocation, and will not be stored as part of the chat history. Create a new Context object.
ContextProvider	Base class for all context providers. A context provider is a component that can be used to enhance the AI's context management. It can listen to changes in the conversation and provide additional context to the AI model just before invocation. **ⓘ Note** ContextProvider is an abstract base class. You must subclass it and implement the invoking() method to create a custom context provider. Ideally, you should also implement the invoked() and thread_created() methods to track conversation state, but these are optional.
DataContent	Represents binary data content with an associated media type (also known as a MIME type). **ⓘ Important** This is for binary data that is represented as a data URI, not for online resources. Use UriContent for online resources. Initializes a DataContent instance. **ⓘ Important**

	This is for binary data that is represented as a data URI, not for online resources. Use UriContent for online resources.
Default	Runtime representation of the default branch in a switch-case group. The default branch is invoked only when no other case predicates match. In practice it is guaranteed to exist so that routing never produces an empty target.
Edge	Model a directed, optionally-conditional hand-off between two executors. Each *Edge* captures the minimal metadata required to move a message from one executor to another inside the workflow graph. It optionally embeds a boolean predicate that decides if the edge should be taken at runtime. By serialising the edge down to primitives we can reconstruct the topology of a workflow irrespective of the original Python process. Initialize a fully-specified edge between two workflow executors.
EdgeDuplicationError	Exception raised when duplicate edges are detected in the workflow.
ErrorContent	Represents an error. Remarks: Typically used for non-fatal errors, where something went wrong as part of the operation, but the operation was still able to continue. Initializes an ErrorContent instance.
Executor	Base class for all workflow executors that process messages and perform computations. # Overview Executors are the fundamental building blocks of workflows, representing individual processing units that receive messages, perform operations, and produce outputs. Each executor is uniquely identified and can handle specific message types through decorated handler methods. # Type System Executors have a rich type system that defines their capabilities: ## Input Types

The types of messages an executor can process, discovered from handler method signatures:

Python

```
    class MyExecutor(Executor):
        @handler
        async def handle_string(self, message: str,
ctx: WorkflowContext) -> None:
            # This executor can handle 'str' input
types
```

Access via the *input_types* property.

Output Types

The types of messages an executor can send to other executors via *ctx.send_message()*:

Python

```
    class MyExecutor(Executor):
        @handler
        async def handle_data(self, message: str, ctx:
WorkflowContext[int | bool]) -> None:
            # This executor can send 'int' or 'bool'
messages
```

Access via the *output_types* property.

Workflow Output Types

The types of data an executor can emit as workflow-level outputs via *ctx.yield_output()*:

Python

```
    class MyExecutor(Executor):
        @handler
        async def process(self, message: str, ctx:
WorkflowContext[int, str]) -> None:
            # Can send 'int' messages AND yield 'str'
workflow outputs
```

Access via the *workflow_output_types* property.

Handler Discovery

Executors discover their capabilities through decorated methods:

@handler Decorator

Marks methods that process incoming messages:

Python

```
   class MyExecutor(Executor):
       @handler
       async def handle_text(self, message: str, ctx:
WorkflowContext[str]) -> None:
           await ctx.send_message(message.upper())
```

Sub-workflow Request Interception

Use @handler methods to intercept sub-workflow requests:

Python

```
   class ParentExecutor(Executor):
       @handler
       async def handle_subworkflow_request(
           self,
           request: SubWorkflowRequestMessage,
           ctx:
WorkflowContext[SubWorkflowResponseMessage],
       ) -> None:
           if self.is_allowed(request.domain):
               response =
request.create_response(data=True)
               await ctx.send_message(response,
target_id=request.executor_id)
           else:
               await
ctx.request_info(request.source_event,
response_type=request.source_event.response_type)
```

Context Types

Handler methods receive different WorkflowContext variants based on their type annotations:

WorkflowContext (no type parameters)

For handlers that only perform side effects without sending messages or yielding outputs:

Python

```
   class LoggingExecutor(Executor):
       @handler
       async def log_message(self, msg: str, ctx:
WorkflowContext) -> None:
           print(f"Received: {msg}")  # Only logging,
no outputs
```

WorkflowContext[T_Out]

Enables sending messages of type T_Out via *ctx.send_message()*:

Python

```
   class ProcessorExecutor(Executor):
       @handler
       async def handler(self, msg: str, ctx:
WorkflowContext[int]) -> None:
           await ctx.send_message(42)  # Can send int
messages
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```
   class DualOutputExecutor(Executor):
       @handler
       async def handler(self, msg: str, ctx:
WorkflowContext[int, str]) -> None:
           await ctx.send_message(42)  # Send int
message
           await ctx.yield_output("done")  # Yield
str workflow output
```

Function Executors

Simple functions can be converted to executors using the *@executor* decorator:

Python

```
   @executor
   async def process_text(text: str, ctx:
WorkflowContext[str]) -> None:
       await ctx.send_message(text.upper())


   # Or with custom ID:
   @executor(id="text_processor")
   def sync_process(text: str, ctx:
WorkflowContext[str]) -> None:
       ctx.send_message(text.lower())  # Sync
functions run in thread pool
```

Sub-workflow Composition

Executors can contain sub-workflows using WorkflowExecutor. Sub-workflows can make requests that parent workflows can intercept. See WorkflowExecutor documentation for details on workflow composition patterns and request/response handling.

State Management

Executors can contain states that persist across workflow runs and checkpoints. Override the *on_checkpoint_save* and *on_checkpoint_restore* methods to implement custom state serialization and restoration logic.

Implementation Notes

- Do not call *execute()* directly - it's invoked by the workflow engine
- Do not override *execute()* - define handlers using decorators instead
- Each executor must have at least one *@handler* method
- Handler method signatures are validated at initialization time

Initialize the executor with a unique identifier.

ExecutorCompletedEvent	Event triggered when an executor handler is completed. Initialize the executor event with an executor ID and optional data.
ExecutorEvent	Base class for executor events.

	Initialize the executor event with an executor ID and optional data.
ExecutorFailedEvent	Event triggered when an executor handler raises an error.
ExecutorInvokedEvent	Event triggered when an executor handler is invoked. Initialize the executor event with an executor ID and optional data.
FanInEdgeGroup	Represent a converging set of edges that feed a single downstream executor. Fan-in groups are typically used when multiple upstream stages independently produce messages that should all arrive at the same downstream processor. Build a fan-in mapping that merges several sources into one target.
FanOutEdgeGroup	Represent a broadcast-style edge group with optional selection logic. A fan-out forwards a message produced by a single source executor to one or more downstream executors. At runtime we may further narrow the targets by executing a *selection_func* that inspects the payload and returns the subset of ids that should receive the message. Create a fan-out mapping from a single source to many targets.
FileCheckpointStorage	File-based checkpoint storage for persistence. Initialize the file storage.
FinishReason	Represents the reason a chat response completed. Initialize FinishReason with a value.
FunctionApprovalRequestContent	Represents a request for user approval of a function call. Initializes a FunctionApprovalRequestContent instance.
FunctionApprovalResponseContent	Represents a response for user approval of a function call. Initializes a FunctionApprovalResponseContent instance.
FunctionCallContent	Represents a function call request. Initializes a FunctionCallContent instance.
FunctionExecutor	Executor that wraps a user-defined function. This executor allows users to define simple functions (both sync and async) and use them as workflow executors without needing to create full executor classes. Synchronous functions are executed in a thread pool using asyncio.to_thread() to avoid blocking the event loop.

	Initialize the FunctionExecutor with a user-defined function.
FunctionInvocationConfiguration	Configuration for function invocation in chat clients. This class is created automatically on every chat client that supports function invocation. This means that for most cases you can just alter the attributes on the instance, rather then creating a new one. Initialize FunctionInvocationConfiguration.
FunctionInvocationContext	Context object for function middleware invocations. This context is passed through the function middleware pipeline and contains all information about the function invocation. Initialize the FunctionInvocationContext.
FunctionMiddleware	Abstract base class for function middleware that can intercept function invocations. Function middleware allows you to intercept and modify function/tool invocations before and after execution. You can validate arguments, cache results, log invocations, or override function execution. **Note** FunctionMiddleware is an abstract base class. You must subclass it and implement the process() method to create custom function middleware.
FunctionResultContent	Represents the result of a function call. Initializes a FunctionResultContent instance.
GraphConnectivityError	Exception raised when graph connectivity issues are detected.
GroupChatBuilder	High-level builder for manager-directed group chat workflows with dynamic orchestration. GroupChat coordinates multi-agent conversations using a manager that selects which participant speaks next. The manager can be a simple Python function (set_select_speakers_func) or an agent-based selector via set_manager. These two approaches are mutually exclusive. **Core Workflow:** 1. Define participants: list of agents (uses their .name) or dict mapping names to agents

2. Configure speaker selection: set_select_speakers_func OR set_manager (not both)
3. Optional: set round limits, checkpointing, termination conditions
4. Build and run the workflow

Speaker Selection Patterns:

Pattern 1: Simple function-based selection (recommended)

Python

```
from agent_framework import GroupChatBuilder, GroupChatStateSnapshot


def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:
    # state contains: task, participants, conversation, history, round_index
    if state["round_index"] >= 5:
        return None  # Finish
    last_speaker = state["history"][-1].speaker if state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"


workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher_agent, writer_agent])  # Uses agent.name
    .build()
)
```

Pattern 2: LLM-based selection

Python

```
from agent_framework import ChatAgent
from agent_framework.azure import AzureOpenAIChatClient

manager_agent = AzureOpenAIChatClient().create_agent(
    instructions="Coordinate the conversation and pick the next speaker.",
    name="Coordinator",
    temperature=0.3,
```

```
        seed=42,
        max_tokens=500,
    )

    workflow = (
        GroupChatBuilder()
        .set_manager(manager_agent,
display_name="Coordinator")
        .participants([researcher, writer])  # Or use
dict: researcher=r, writer=w
        .with_max_rounds(10)
        .build()
    )
```

Pattern 3: Request info for mid-conversation feedback

Python

```
    from agent_framework import GroupChatBuilder

    # Pause before all participants
    workflow = (
        GroupChatBuilder()
        .set_select_speakers_func(select_next_speaker)
        .participants([researcher, writer])
        .with_request_info()
        .build()
    )

    # Pause only before specific participants
    workflow = (
        GroupChatBuilder()
        .set_select_speakers_func(select_next_speaker)
        .participants([researcher, writer, editor])
        .with_request_info(agents=[editor])  # Only
pause before editor responds
        .build()
    )
```

Participant Specification:

Two ways to specify participants:

- List form: *[agent1, agent2]* - uses *agent.name* attribute for participant names
- Dict form: *{name1: agent1, name2: agent2}* - explicit name control
- Keyword form: *participants(name1=agent1, name2=agent2)* - explicit name control

State Snapshot Structure:

	The GroupChatStateSnapshot passed to set_select_speakers_func contains: • *task*: ChatMessage - Original user task • *participants*: dict[str, str] - Mapping of participant names to descriptions • *conversation*: tuple[ChatMessage, ...] - Full conversation history • *history*: tuple[GroupChatTurn, ...] - Turn-by-turn record with speaker attribution • *round_index*: int - Number of manager selection rounds so far • *pending_agent*: str | None - Name of agent currently processing (if any) **Important Constraints:** • Cannot combine set_select_speakers_func and set_manager • Participant names must be unique • When using list form, agents must have a non-empty *name* attribute Initialize the GroupChatBuilder.
GroupChatDirective	Instruction emitted by a group chat manager implementation.
HandoffBuilder	Fluent builder for conversational handoff workflows with coordinator and specialist agents. The handoff pattern enables a coordinator agent to route requests to specialist agents. Interaction mode controls whether the workflow requests user input after each agent response or completes autonomously once agents finish responding. A termination condition determines when the workflow should stop requesting input and complete. Routing Patterns: **Single-Tier (Default):** Only the coordinator can hand off to specialists. By default, after any specialist responds, control returns to the user for more input. This creates a cyclical flow: user -> coordinator -> [optional specialist] -> user -> coordinator -> ... Use *with_interaction_mode("autonomous")* to skip requesting additional user input and yield the final conversation when an agent responds without delegating. **Multi-Tier (Advanced):** Specialists can hand off to other specialists using *.add_handoff()*. This provides more flexibility for complex workflows but is less controllable than the single-tier pattern. Users lose real-time visibility into intermediate steps during specialist-to-specialist handoffs (though the full conversation history including all handoffs is preserved and can be inspected afterward). Key Features:

- **Automatic handoff detection**: The coordinator invokes a handoff tool whose

  arguments (for example *{"handoff_to": "shipping_agent"}*) identify the specialist to receive control.

- **Auto-generated tools**: By default the builder synthesizes *handoff_to_<agent>* tools for the coordinator, so you don't manually define placeholder functions.
- **Full conversation history**: The entire conversation (including any *ChatMessage.additional_properties*) is preserved and passed to each agent.
- **Termination control**: By default, terminates after 10 user messages. Override with *.with_termination_condition(lambda conv: ...)* for custom logic (e.g., detect "goodbye").
- **Interaction modes**: Choose *human_in_loop* (default) to prompt users between agent turns, or *autonomous* to continue routing back to agents without prompting for user input until a handoff occurs or a termination/turn limit is reached (default autonomous turn limit: 50).
- **Checkpointing**: Optional persistence for resumable workflows.

Usage (Single-Tier):

Python

```
   from agent_framework import HandoffBuilder
   from agent_framework.openai import
OpenAIChatClient

   chat_client = OpenAIChatClient()

   # Create coordinator and specialist agents
   coordinator = chat_client.create_agent(
       instructions=(
           "You are a frontline support agent. Assess
the user's issue and decide "
           "whether to hand off to 'refund_agent' or
'shipping_agent'. When delegation is "
           "required, call the matching handoff tool
(for example `handoff_to_refund_agent`)."
       ),
       name="coordinator_agent",
   )

   refund = chat_client.create_agent(
       instructions="You handle refund requests. Ask
for order details and process refunds.",
       name="refund_agent",
   )

   shipping = chat_client.create_agent(
```

```
        instructions="You resolve shipping issues.
Track packages and update delivery status.",
        name="shipping_agent",
    )

    # Build the handoff workflow - default single-tier
routing
    workflow = (
        HandoffBuilder(
            name="customer_support",
            participants=[coordinator, refund,
shipping],
        )
        .set_coordinator(coordinator)
        .build()
    )

    # Run the workflow
    events = await workflow.run_stream("My package
hasn't arrived yet")
    async for event in events:
        if isinstance(event, RequestInfoEvent):
            # Request user input
            user_response = input("You: ")
            await
workflow.send_response(event.data.request_id,
user_response)
```

Multi-Tier Routing with .add_handoff():

Python

```
    # Enable specialist-to-specialist handoffs with
fluent API
    workflow = (
        HandoffBuilder(participants=[coordinator,
replacement, delivery, billing])
        .set_coordinator(coordinator)
        .add_handoff(coordinator, [replacement,
delivery, billing])  # Coordinator routes to all
        .add_handoff(replacement, [delivery, billing])
# Replacement delegates to delivery/billing
        .add_handoff(delivery, billing)  # Delivery
escalates to billing
        .build()
    )

    # Flow: User → Coordinator → Replacement →
Delivery → Back to User
    # (Replacement hands off to Delivery without
returning to user)
```

Use Participant Factories for State Isolation:

Custom Termination Condition:

Python

```
   # Terminate when user says goodbye or after 5
exchanges
   workflow = (
       HandoffBuilder(participants=[coordinator,
refund, shipping])
       .set_coordinator(coordinator)
       .with_termination_condition(
           lambda conv: (
               sum(1 for msg in conv if
msg.role.value == "user") >= 5
               or any("goodbye" in msg.text.lower()
for msg in conv[-2:])
           )
       )
       .build()
   )
```

Checkpointing:

Python

```
   from agent_framework import
InMemoryCheckpointStorage

   storage = InMemoryCheckpointStorage()
   workflow = (
       HandoffBuilder(participants=[coordinator,
refund, shipping])
       .set_coordinator(coordinator)
       .with_checkpointing(storage)
       .build()
   )
```

Initialize a HandoffBuilder for creating conversational handoff workflows.

The builder starts in an unconfigured state and requires you to call:

1. *.participants([...])* - Register agents
2. or *.participant_factories({...})* - Register agent/executor factories
3. *.set_coordinator(...)* - Designate which agent receives initial user input
4. *.build()* - Construct the final Workflow

	Optional configuration methods allow you to customize context management, termination logic, and persistence. **Note** Participants must have stable names/ids because the workflow maps the handoff tool arguments to these identifiers. Agent names should match the strings emitted by the coordinator's handoff tool (e.g., a tool that outputs {"handoff_to": "billing"} requires an agent named billing).
HandoffUserInputRequest	Request message emitted when the workflow needs fresh user input. Note: The conversation field is intentionally excluded from checkpoint serialization to prevent duplication. The conversation is preserved in the coordinator's state and will be reconstructed on restore. See issue #2667.
HostedCodeInterpreterTool	Represents a hosted tool that can be specified to an AI service to enable it to execute generated code. This tool does not implement code interpretation itself. It serves as a marker to inform a service that it is allowed to execute generated code if the service is capable of doing so. Initialize the HostedCodeInterpreterTool.
HostedFileContent	Represents a hosted file content. Initializes a HostedFileContent instance.
HostedFileSearchTool	Represents a file search tool that can be specified to an AI service to enable it to perform file searches. Initialize a FileSearchTool.
HostedMCPSpecificApproval	Represents the specific mode for a hosted tool. When using this mode, the user must specify which tools always or never require approval. This is represented as a dictionary with two optional keys:
HostedMCPTool	Represents a MCP tool that is managed and executed by the service.

	Create a hosted MCP tool.
HostedVectorStoreContent	Represents a hosted vector store content. Initializes a HostedVectorStoreContent instance.
HostedWebSearchTool	Represents a web search tool that can be specified to an AI service to enable it to perform web searches. Initialize a HostedWebSearchTool.
InMemoryCheckpointStorage	In-memory checkpoint storage for testing and development. Initialize the memory storage.
InProcRunnerContext	In-process execution context for local execution and optional checkpointing. Initialize the in-process execution context.
MCPStdioTool	MCP tool for connecting to stdio-based MCP servers. This class connects to MCP servers that communicate via standard input/output, typically used for local processes. Initialize the MCP stdio tool. **Note** The arguments are used to create a StdioServerParameters object, which is then used to create a stdio client. See mcp.client.stdio.stdio_client and mcp.client.stdio.stdio_server_parameters for more details.
MCPStreamableHTTPTool	MCP tool for connecting to HTTP-based MCP servers. This class connects to MCP servers that communicate via streamable HTTP/SSE. Initialize the MCP streamable HTTP tool. **Note** The arguments are used to create a streamable HTTP client.

	See mcp.client.streamable_http.streamablehttp_client for more details. Any extra arguments passed to the constructor will be passed to the streamable HTTP client constructor.
MCPWebsocketTool	MCP tool for connecting to WebSocket-based MCP servers. This class connects to MCP servers that communicate via WebSocket. Initialize the MCP WebSocket tool. **Note** The arguments are used to create a WebSocket client. See mcp.client.websocket.websocket_client for more details. Any extra arguments passed to the constructor will be passed to the WebSocket client constructor.
MagenticBuilder	Fluent builder for creating Magentic One multi-agent orchestration workflows. Magentic One workflows use an LLM-powered manager to coordinate multiple agents through dynamic task planning, progress tracking, and adaptive replanning. The manager creates plans, selects agents, monitors progress, and determines when to replan or complete. The builder provides a fluent API for configuring participants, the manager, optional plan review, checkpointing, and event callbacks. Human-in-the-loop Support: Magentic provides specialized HITL mechanisms via: • *.with_plan_review()* - Review and approve/revise plans before execution • *.with_human_input_on_stall()* - Intervene when workflow stalls • Tool approval via *FunctionApprovalRequestContent* - Approve individual tool calls These emit *MagenticHumanInterventionRequest* events that provide structured decision options (APPROVE, REVISE, CONTINUE, REPLAN, GUIDANCE) appropriate for Magentic's planning-based orchestration.

Usage:

Python

```
   from agent_framework import MagenticBuilder,
StandardMagenticManager
   from azure.ai.projects.aio import AIProjectClient

   # Create manager with LLM client
   project_client =
AIProjectClient.from_connection_string(...)
   chat_client =
project_client.inference.get_chat_completions_client(
)

   # Build Magentic workflow with agents
   workflow = (
       MagenticBuilder()
       .participants(researcher=research_agent,
writer=writing_agent, coder=coding_agent)

.with_standard_manager(chat_client=chat_client,
max_round_count=20, max_stall_count=3)
       .with_plan_review(enable=True)
       .with_checkpointing(checkpoint_storage)
       .build()
   )

   # Execute workflow
   async for message in workflow.run("Research and
write article about AI agents"):
       print(message.text)
```

With custom manager:

Python

```
   # Create custom manager subclass
   class MyCustomManager(MagenticManagerBase):
       async def plan(self, context: MagenticContext)
-> ChatMessage:
           # Custom planning logic
           ...


   manager = MyCustomManager()
   workflow =
MagenticBuilder().participants(agent1=agent1,
agent2=agent2).with_standard_manager(manager).build()
```

MagenticContext	Context for the Magentic manager.
MagenticManagerBase	Base class for the Magentic One manager.
ManagerDirectiveModel	Pydantic model for structured manager directive output. Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
ManagerSelectionRequest	Request sent to manager agent for next speaker selection. This dataclass packages the full conversation state and task context for the manager agent to analyze and make a speaker selection decision.
ManagerSelectionResponse	Response from manager agent with speaker selection decision. The manager agent must produce this structure (or compatible dict/JSON) to communicate its decision back to the orchestrator. Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
Message	A class representing a message in the workflow.
OrchestrationState	Unified state container for orchestrator checkpointing. This dataclass standardizes checkpoint serialization across all three group chat patterns while allowing pattern-specific extensions via metadata. Common attributes cover shared orchestration concerns (task, conversation, round tracking). Pattern-specific state goes in the metadata dict.
RequestInfoEvent	Event triggered when a workflow executor requests external information. Initialize the request info event.
RequestInfoInterceptor	Internal executor that pauses workflow for human input before agent runs.

	This executor is inserted into the workflow graph by builders when *.with_request_info()* is called. It intercepts AgentExecutorRequest messages BEFORE the agent runs and pauses the workflow via *ctx.request_info()* with an AgentInputRequest. When a response is received, the response handler injects the input as a user message into the conversation and forwards the request to the agent. The optional *agent_filter* parameter allows limiting which agents trigger the pause. If the target agent's ID is not in the filter set, the request is forwarded without pausing. Initialize the request info interceptor executor.
Role	Describes the intended purpose of a message within a chat interaction. Properties: SYSTEM: The role that instructs or sets the behavior of the AI system. USER: The role that provides user input for chat interactions. ASSISTANT: The role that provides responses to system-instructed, user-prompted input. TOOL: The role that provides additional information and references in response to tool use requests. Initialize Role with a value.
Runner	A class to run a workflow in Pregel supersteps. Initialize the runner with edges, shared state, and context.
RunnerContext	Protocol for the execution context used by the runner. A single context that supports messaging, events, and optional checkpointing. If checkpoint storage is not configured, checkpoint methods may raise.
SequentialBuilder	High-level builder for sequential agent/executor workflows with shared context. • *participants([...])* accepts a list of AgentProtocol (recommended) or Executor instances • *register_participants([...])* accepts a list of factories for AgentProtocol (recommended) or Executor factories • Executors must define a handler that consumes list[ChatMessage] and sends out a list[ChatMessage] • The workflow wires participants in order, passing a list[ChatMessage] down the chain • Agents append their assistant messages to the conversation

- Custom executors can transform/summarize and return a list[ChatMessage]
- The final output is the conversation produced by the last participant

Usage:

Python

```
   from agent_framework import SequentialBuilder

   # With agent instances
   workflow =
SequentialBuilder().participants([agent1, agent2,
summarizer_exec]).build()

   # With agent factories
   workflow = (

SequentialBuilder().register_participants([create_age
nt1, create_agent2, create_summarizer_exec]).build()
   )

   # Enable checkpoint persistence
   workflow =
SequentialBuilder().participants([agent1,
agent2]).with_checkpointing(storage).build()

   # Enable request info for mid-workflow feedback
(pauses before each agent)
   workflow =
SequentialBuilder().participants([agent1,
agent2]).with_request_info().build()

   # Enable request info only for specific agents
   workflow = (
       SequentialBuilder()
       .participants([agent1, agent2, agent3])
       .with_request_info(agents=[agent2])  # Only
pause before agent2
       .build()
   )
```

SharedState	A class to manage shared state in a workflow. SharedState provides thread-safe access to workflow state data that needs to be shared across executors during workflow execution. Reserved Keys: The following keys are reserved for internal framework use and should not be modified by user code:

	• *_executor_state*: Stores executor state for checkpointing (managed by Runner) **⚠ Warning** Do not use keys starting with underscore (_) as they may be reserved for internal framework operations. Initialize the shared state.
SingleEdgeGroup	Convenience wrapper for a solitary edge, keeping the group API uniform. Create a one-to-one edge group between two executors.
StandardMagenticManager	Standard Magentic manager that performs real LLM calls via a ChatAgent. The manager constructs prompts that mirror the original Magentic One orchestration: • Facts gathering • Plan creation • Progress ledger in JSON • Facts update and plan update on reset • Final answer synthesis Initialize the Standard Magentic Manager.
SubWorkflowRequestMessage	Message sent from a sub-workflow to an executor in the parent workflow to request information. This message wraps a RequestInfoEvent emitted by the executor in the sub-workflow.
SubWorkflowResponseMessage	Message sent from a parent workflow to a sub-workflow via WorkflowExecutor to provide requested information. This message wraps the response data along with the original RequestInfoEvent emitted by the sub-workflow executor.
SuperStepCompletedEvent	Event triggered when a superstep ends. Initialize the superstep event.
SuperStepStartedEvent	Event triggered when a superstep starts. Initialize the superstep event.

SwitchCaseEdgeGroup	Fan-out variant that mimics a traditional switch/case control flow. Each case inspects the message payload and decides whether it should handle the message. Exactly one case-or the default branch-returns a target at runtime, preserving single-dispatch semantics. Configure a switch/case routing structure for a single source executor.
SwitchCaseEdgeGroupCase	Persistable description of a single conditional branch in a switch-case. Unlike the runtime *Case* object this serialisable variant stores only the target identifier and a descriptive name for the predicate. When the underlying callable is unavailable during deserialisation we substitute a proxy placeholder that fails loudly, ensuring the missing dependency is immediately visible. Record the routing metadata for a conditional case branch.
SwitchCaseEdgeGroupDefault	Persistable descriptor for the fallback branch of a switch-case group. The default branch is guaranteed to exist and is invoked when every other case predicate fails to match the payload. Point the default branch toward the given executor identifier.
TextContent	Represents text content in a chat. Initializes a TextContent instance.
TextReasoningContent	Represents text reasoning content in a chat. Remarks: This class and *TextContent* are superficially similar, but distinct. Initializes a TextReasoningContent instance.
TextSpanRegion	Represents a region of text that has been annotated. Initialize TextSpanRegion.
ToolMode	Defines if and how tools are used in a chat request. Initialize ToolMode.
ToolProtocol	Represents a generic tool. This protocol defines the interface that all tools must implement to be compatible with the agent framework. It is implemented by various tool classes such as HostedMCPTool, HostedWebSearchTool, and AIFunction's. A AIFunction is usually created by the *ai_function* decorator.

	Since each connector needs to parse tools differently, users can pass a dict to specify a service-specific tool when no abstraction is available.
TypeCompatibilityError	Exception raised when type incompatibility is detected between connected executors.
UriContent	Represents a URI content. **ⓘ Important** This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use DataContent instead. Initializes a UriContent instance. Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use *DataContent* instead.
UsageContent	Represents usage information associated with a chat request and response. Initializes a UsageContent instance.
UsageDetails	Provides usage details about a request/response. Initializes the UsageDetails instance.
Workflow	A graph-based execution engine that orchestrates connected executors. # Overview A workflow executes a directed graph of executors connected via edge groups using a Pregel-like model, running in supersteps until the graph becomes idle. Workflows are created using the WorkflowBuilder class - do not instantiate this class directly. # Execution Model Executors run in synchronized supersteps where each executor: • Is invoked when it receives messages from connected edge groups • Can send messages to downstream executors via ctx.send_message() • Can yield workflow-level outputs via ctx.yield_output() • Can emit custom events via ctx.add_event()

Messages between executors are delivered at the end of each superstep and are not visible in the event stream. Only workflow-level events (outputs, custom events) and status events are observable to callers.

Input/Output Types

Workflow types are discovered at runtime by inspecting:

- Input types: From the start executor's input types
- Output types: Union of all executors' workflow output types
  Access these via the input_types and output_types properties.

Execution Methods

The workflow provides two primary execution APIs, each supporting multiple scenarios:

- **run()**: Execute to completion, returns WorkflowRunResult with all events
- **run_stream()**: Returns async generator yielding events as they occur

Both methods support:

- Initial workflow runs: Provide *message* parameter
- Checkpoint restoration: Provide *checkpoint_id* (and optionally *checkpoint_storage*)
- HIL continuation: Provide *responses* to continue after RequestInfoExecutor requests
- Runtime checkpointing: Provide *checkpoint_storage* to enable/override checkpointing for this run

State Management

Workflow instances contain states and states are preserved across calls to *run* and *run_stream*. To execute multiple independent runs, create separate Workflow instances via WorkflowBuilder.

External Input Requests

Executors within a workflow can request external input using *ctx.request_info()*:

1. Executor calls *ctx.request_info()* to request input
2. Executor implements *response_handler()* to process the response
3. Requests are emitted as RequestInfoEvent instances in the event stream

4. Workflow enters IDLE_WITH_PENDING_REQUESTS state
5. Caller handles requests and provides responses via the *send_responses* or *send_responses_streaming* methods
6. Responses are routed to the requesting executors and response handlers are invoked

Checkpointing

Checkpointing can be configured at build time or runtime:

Build-time (via WorkflowBuilder): workflow = WorkflowBuilder().with_checkpointing(storage).build()

Runtime (via run/run_stream parameters): result = await workflow.run(message, checkpoint_storage=runtime_storage)

When enabled, checkpoints are created at the end of each superstep, capturing:

- Executor states
- Messages in transit
- Shared state Workflows can be paused and resumed across process restarts using checkpoint storage.

Composition

Workflows can be nested using WorkflowExecutor, which wraps a child workflow as an executor. The nested workflow's input/output types become part of the WorkflowExecutor's types. When invoked, the WorkflowExecutor runs the nested workflow to completion and processes its outputs.

Initialize the workflow with a list of edges.

WorkflowAgent	An *Agent* subclass that wraps a workflow and exposes it as an agent. Initialize the WorkflowAgent.
WorkflowBuilder	A builder class for constructing workflows. This class provides a fluent API for defining workflow graphs by connecting executors with edges and configuring execution parameters. Call build to create an immutable Workflow instance. Initialize the WorkflowBuilder with an empty list of edges and no starting executor.
WorkflowCheckpoint	Represents a complete checkpoint of workflow state. Checkpoints capture the full execution state of a workflow at a specific point, enabling workflows to be paused and resumed.

	> **Note** The shared_state dict may contain reserved keys managed by the framework. See SharedState class documentation for details on reserved keys.
WorkflowCheckpointSummary	Human-readable summary of a workflow checkpoint.

WorkflowContext	Execution context that enables executors to interact with workflows and other executors.
	# Overview
	WorkflowContext provides a controlled interface for executors to send messages, yield outputs, manage state, and interact with the broader workflow ecosystem. It enforces type safety through generic parameters while preventing direct access to internal runtime components.
	# Type Parameters
	The context is parameterized to enforce type safety for different operations:
	## WorkflowContext (no parameters)
	For executors that only perform side effects without sending messages or yielding outputs:
	Python
	`async def log_handler(message: str, ctx: WorkflowContext) -> None:`
	`print(f"Received: {message}")  # Only side effects`
	## WorkflowContext[T_Out]
	Enables sending messages of type T_Out to other executors:
	Python

```python
    async def processor(message: str, ctx:
WorkflowContext[int]) -> None:
        result = len(message)
        await ctx.send_message(result)  # Send int to
downstream executors
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```python
    async def dual_output(message: str, ctx:
WorkflowContext[int, str]) -> None:
        await ctx.send_message(42)  # Send int message
        await ctx.yield_output("complete")  # Yield
str workflow output
```

Union Types

Multiple types can be specified using union notation:

Python

```python
    async def flexible(message: str, ctx:
WorkflowContext[int | str, bool | dict]) -> None:
        await ctx.send_message("text")  # or send 42
        await ctx.yield_output(True)  # or yield
{"status": "done"}
```

Initialize the executor context with the given workflow context.

WorkflowErrorDetails	Structured error information to surface in error events/results.
WorkflowEvent	Base class for workflow events. Initialize the workflow event with optional data.
WorkflowExecutor	An executor that wraps a workflow to enable hierarchical workflow composition.

Overview

WorkflowExecutor makes a workflow behave as a single executor within a parent workflow, enabling nested workflow architectures. It handles the complete lifecycle of sub-workflow execution including event processing, output forwarding, and request/response coordination between parent and child workflows.

Execution Model

When invoked, WorkflowExecutor:

1. Starts the wrapped workflow with the input message
2. Runs the sub-workflow to completion or until it needs external input
3. Processes the sub-workflow's complete event stream after execution
4. Forwards outputs to the parent workflow as messages
5. Handles external requests by routing them to the parent workflow
6. Accumulates responses and resumes sub-workflow execution

Event Stream Processing

WorkflowExecutor processes events after sub-workflow completion:

Output Forwarding

All outputs from the sub-workflow are automatically forwarded to the parent:

When *allow_direct_output* is False (default):

Python

```
   # An executor in the sub-workflow yields outputs
   await ctx.yield_output("sub-workflow result")

   # WorkflowExecutor forwards to parent via
ctx.send_message()
   # Parent receives the output as a regular message
```

When *allow_direct_output* is True:

Request/Response Coordination

When sub-workflows need external information:

Python

```
   # An executor in the sub-workflow makes request
   request = MyDataRequest(query="user info")

   # WorkflowExecutor captures RequestInfoEvent and
wraps it in a SubWorkflowRequestMessage
   # then send it to the receiving executor in parent
workflow. The executor in parent workflow
   # can handle the request locally or forward it to
an external source.
   # The WorkflowExecutor tracks the pending request,
and implements a response handler.
   # When the response is received, it executes the
response handler to accumulate responses
   # and resume the sub-workflow when all expected
responses are received.
   # The response handler expects a
SubWorkflowResponseMessage wrapping the response
data.
```

State Management

WorkflowExecutor maintains execution state across request/response cycles:

- Tracks pending requests by request_id
- Accumulates responses until all expected responses are received
- Resumes sub-workflow execution with complete response batch
- Handles concurrent executions and multiple pending requests

Type System Integration

WorkflowExecutor inherits its type signature from the wrapped workflow:

Input Types

Matches the wrapped workflow's start executor input types:

Python

```
   # If sub-workflow accepts str, WorkflowExecutor
accepts str
   workflow_executor = WorkflowExecutor(my_workflow,
id="wrapper")
   assert workflow_executor.input_types ==
my_workflow.input_types
```

Output Types

Combines sub-workflow outputs with request coordination types:

Python

```
   # Includes all sub-workflow output types
   # Plus SubWorkflowRequestMessage if sub-workflow
can make requests
   output_types = workflow.output_types +
[SubWorkflowRequestMessage]  # if applicable
```

Error Handling

WorkflowExecutor propagates sub-workflow failures:

- Captures WorkflowFailedEvent from sub-workflow
- Converts to WorkflowErrorEvent in parent context
- Provides detailed error information including sub-workflow ID

Concurrent Execution Support

WorkflowExecutor fully supports multiple concurrent sub-workflow executions:

Per-Execution State Isolation

Each sub-workflow invocation creates an isolated ExecutionContext:

Python

```
   # Multiple concurrent invocations are supported
   workflow_executor = WorkflowExecutor(my_workflow,
id="concurrent_executor")

   # Each invocation gets its own execution context
   # Execution 1: processes input_1 independently
   # Execution 2: processes input_2 independently
   # No state interference between executions
```

Request/Response Coordination

Responses are correctly routed to the originating execution:

- Each execution tracks its own pending requests and expected responses
- Request-to-execution mapping ensures responses reach the correct sub-workflow
- Response accumulation is isolated per execution
- Automatic cleanup when execution completes

Memory Management

- Unlimited concurrent executions supported
- Each execution has unique UUID-based identification
- Cleanup of completed execution contexts
- Thread-safe state management for concurrent access

Important Considerations

Shared Workflow Instance: All concurrent executions use the same underlying workflow instance. For proper isolation, ensure that the wrapped workflow and its executors are stateless.

Python

```
   # Avoid: Stateful executor with instance variables
   class StatefulExecutor(Executor):
       def __init__(self):
           super().__init__(id="stateful")
           self.data = []  # This will be shared
across concurrent executions!
```

Integration with Parent Workflows

Parent workflows can intercept sub-workflow requests:

Implementation Notes

- Sub-workflows run to completion before processing their results
- Event processing is atomic - all outputs are forwarded before requests
- Response accumulation ensures sub-workflows receive complete response batches
- Execution state is maintained for proper resumption after external requests
- Concurrent executions are fully isolated and do not interfere with each other

Initialize the WorkflowExecutor.

WorkflowFailedEvent	Built-in lifecycle event emitted when a workflow run terminates with an error.
WorkflowOutputEvent	Event triggered when a workflow executor yields output. Initialize the workflow output event.
WorkflowRunResult	Container for events generated during non-streaming workflow execution. # Overview Represents the complete execution results of a workflow run, containing all events generated from start to idle state. Workflows produce outputs incrementally through ctx.yield_output() calls during execution. # Event Structure Maintains separation between data-plane and control-plane events: • Data-plane events: Executor invocations, completions, outputs, and requests (in main list) • Control-plane events: Status timeline accessible via status_timeline() method # Key Methods • get_outputs(): Extract all workflow outputs from the execution • get_request_info_events(): Retrieve external input requests made during execution • get_final_state(): Get the final workflow state (IDLE, IDLE_WITH_PENDING_REQUESTS, etc.) • status_timeline(): Access the complete status event history
WorkflowStartedEvent	Built-in lifecycle event emitted when a workflow run begins. Initialize the workflow event with optional data.
WorkflowStatusEvent	Built-in lifecycle event emitted for workflow run state transitions. Initialize the workflow status event with a new state and optional data.
WorkflowValidationError	Base exception for workflow validation errors.
WorkflowViz	A class for visualizing workflows using graphviz and Mermaid. Initialize the WorkflowViz with a workflow.

Enums

Expand table

MagenticHumanInterventionDecision	Decision options for human intervention responses.
MagenticHumanInterventionKind	The kind of human intervention being requested.
ValidationTypeEnum	Enumeration of workflow validation types.
WorkflowEventSource	Identifies whether a workflow event came from the framework or an executor. Use *FRAMEWORK* for events emitted by built-in orchestration paths—even when the code that raises them lives in runner-related modules—and *EXECUTOR* for events surfaced by developer-provided executor implementations.
WorkflowRunState	Run-level state of a workflow execution. Semantics: • STARTED: Run has been initiated and the workflow context has been created. This is an initial state before any meaningful work is performed. In this codebase we emit a dedicated *WorkflowStartedEvent* for telemetry, and typically advance the status directly to *IN_PROGRESS*. Consumers may still rely on *STARTED* for state machines that need an explicit pre-work phase. • IN_PROGRESS: The workflow is actively executing (e.g., the initial message has been delivered to the start executor or a superstep is running). This status is emitted at the beginning of a run and can be followed by other statuses as the run progresses. • IN_PROGRESS_PENDING_REQUESTS: Active execution while one or more request-for-information operations are outstanding. New work may still be scheduled while requests are in flight. • IDLE: The workflow is quiescent with no outstanding requests and no more work to do. This is the normal terminal state for workflows that have finished executing, potentially having produced outputs along the way. • IDLE_WITH_PENDING_REQUESTS: The workflow is paused awaiting external input (e.g., emitted a *RequestInfoEvent*). This is a non-terminal state; the workflow can resume when responses are supplied. • FAILED: Terminal state indicating an error surfaced. Accompanied by a *WorkflowFailedEvent* with structured error details.

- CANCELLED: Terminal state indicating the run was cancelled by a caller or orchestrator. Not currently emitted by default runner paths but included for integrators/orchestrators that support cancellation.

Functions

agent_middleware

Decorator to mark a function as agent middleware.

This decorator explicitly identifies a function as agent middleware, which processes AgentRunContext objects.

Python

```
agent_middleware(func: Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]) -> Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]
```

Parameters

Expand table

Name	Description
func Required*	Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] The middleware function to mark as agent middleware.

Returns

Expand table

Type	Description
Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]	The same function with agent middleware marker.

Examples

Python

```
    from agent_framework import agent_middleware, AgentRunContext, ChatAgent


    @agent_middleware
    async def logging_middleware(context: AgentRunContext, next):
        print(f"Before: {context.agent.name}")
        await next(context)
        print(f"After: {context.result}")


    # Use with an agent
    agent = ChatAgent(chat_client=client, name="assistant",
middleware=logging_middleware)
```

ai_function

Decorate a function to turn it into a AIFunction that can be passed to models and executed automatically.

This decorator creates a Pydantic model from the function's signature, which will be used to validate the arguments passed to the function and to generate the JSON schema for the function's parameters.

To add descriptions to parameters, use the `Annotated` type from `typing` with a string description as the second argument. You can also use Pydantic's `Field` class for more advanced configuration.

> ⓘ **Note**
>
> When approval_mode is set to "always_require", the function will not be executed
>
> until explicit approval is given, this only applies to the auto-invocation flow.
>
> It is also important to note that if the model returns multiple function calls, some that require approval
>
> and others that do not, it will ask approval for all of them.

Python

```
ai_function(func: Callable[[...], ReturnT | Awaitable[ReturnT]] | None = None, *,
name: str | None = None, description: str | None = None, approval_mode:
Literal['always_require', 'never_require'] | None = None, max_invocations: int |
None = None, max_invocation_exceptions: int | None = None, additional_properties:
```

```
dict[str, Any] | None = None) -> AIFunction[Any, ReturnT] |
Callable[[Callable[[...], ReturnT | Awaitable[ReturnT]]], AIFunction[Any,
ReturnT]]
```

Parameters

Expand table

Name	Description
func	Callable[[...], <xref:agent_framework._tools.ReturnT>	Awaitable[<xref:agent_framework._tools.ReturnT>]]	None The function to decorate. Default value: None
name Required*	str	None
description Required*	str	None
approval_mode Required*	Literal['always_require', 'never_require']	None
max_invocations Required*	int	None
max_invocation_exceptions Required*	int	None
additional_properties Required*	dict[str, Any]	None

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the function. If not provided, the function's `__name__` attribute will be used. Default value: None
description	A description of the function. If not provided, the function's docstring will be used. Default value: None
approval_mode	Whether or not approval is required to run this tool. Default is that approval is not needed.

Name	Description
	Default value: None
max_invocations	The maximum number of times this function can be invoked. If None, there is no limit, should be at least 1. Default value: None
max_invocation_exceptions	The maximum number of exceptions allowed during invocations. If None, there is no limit, should be at least 1. Default value: None
additional_properties	Additional properties to set on the function. Default value: None

Returns

Expand table

Type	Description
AIFunction[Any, <xref:agent_framework._tools.ReturnT>] | Callable[[Callable[[...], <xref:agent_framework._tools.ReturnT> | Awaitable[<xref:agent_framework._tools.ReturnT>]]], AIFunction[Any, <xref:agent_framework._tools.ReturnT>]]	

Examples

Python

```python
from agent_framework import ai_function
from typing import Annotated


@ai_function
def ai_function_example(
    arg1: Annotated[str, "The first argument"],
    arg2: Annotated[int, "The second argument"],
) -> str:
    # An example function that takes two arguments and returns a string.
    return f"arg1: {arg1}, arg2: {arg2}"


# the same function but with approval required to run
@ai_function(approval_mode="always_require")
def ai_function_example(
    arg1: Annotated[str, "The first argument"],
    arg2: Annotated[int, "The second argument"],
) -> str:
```

```python
    # An example function that takes two arguments and returns a string.
    return f"arg1: {arg1}, arg2: {arg2}"


# With custom name and description
@ai_function(name="custom_weather", description="Custom weather function")
def another_weather_func(location: str) -> str:
    return f"Weather in {location}"


# Async functions are also supported
@ai_function
async def async_get_weather(location: str) -> str:
    '''Get weather asynchronously.'''
    # Simulate async operation
    return f"Weather in {location}"
```

chat_middleware

Decorator to mark a function as chat middleware.

This decorator explicitly identifies a function as chat middleware, which processes ChatContext objects.

Python

```
chat_middleware(func: Callable[[ChatContext, Callable[[ChatContext],
Awaitable[None]]], Awaitable[None]]) -> Callable[[ChatContext,
Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]
```

Parameters

Expand table

Name	Description
func Required*	Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] The middleware function to mark as chat middleware.

Returns

Expand table

Type	Description
Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]	The same function with chat middleware marker.

Examples

```Python
from agent_framework import chat_middleware, ChatContext, ChatAgent


@chat_middleware
async def logging_middleware(context: ChatContext, next):
    print(f"Messages: {len(context.messages)}")
    await next(context)
    print(f"Response: {context.result}")


# Use with an agent
agent = ChatAgent(chat_client=client, name="assistant",
middleware=logging_middleware)
```

create_edge_runner

Factory function to create the appropriate edge runner for an edge group.

```Python
create_edge_runner(edge_group: EdgeGroup, executors: dict[str, Executor]) ->
EdgeRunner
```

Parameters

Expand table

Name	Description
edge_group Required*	<xref:agent_framework._workflows._edge.EdgeGroup> The edge group to create a runner for.
executors Required*	dict[str, Executor] Map of executor IDs to executor instances.

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._edge_runner.EdgeRunner>	The appropriate EdgeRunner instance.

executor

Decorator that converts a standalone function into a FunctionExecutor instance.

The @executor decorator is designed for **standalone module-level functions only**. For class-based executors, use the Executor base class with @handler on instance methods.

Supports both synchronous and asynchronous functions. Synchronous functions are executed in a thread pool to avoid blocking the event loop.

> ⓘ **Important**
>
> Use @executor for standalone functions (module-level or local functions)
>
> Do NOT use @executor with staticmethod or classmethod
>
> For class-based executors, subclass Executor and use @handler on instance methods

Usage:

Python

```
# Standalone async function (RECOMMENDED):
@executor(id="upper_case")
async def to_upper(text: str, ctx: WorkflowContext[str]):
    await ctx.send_message(text.upper())


# Standalone sync function (runs in thread pool):
@executor
def process_data(data: str):
    return data.upper()


# For class-based executors, use @handler instead:
class MyExecutor(Executor):
    def __init__(self):
        super().__init__(id="my_executor")

    @handler
    async def process(self, data: str, ctx: WorkflowContext[str]):
        await ctx.send_message(data.upper())
```

Python

```
executor(func: Callable[[...], Any] | None = None, *, id: str | None = None) ->
Callable[[Callable[[...], Any]], FunctionExecutor] | FunctionExecutor
```

Parameters

Expand table

Name	Description
func	Callable[[...], Any] | None The function to decorate (when used without parentheses) Default value: None
id Required*	str | None Optional custom ID for the executor. If None, uses the function name.

Keyword-Only Parameters

Expand table

Name	Description
id	Default value: None

Returns

Expand table

Type	Description
Callable[[Callable[[...], Any]], FunctionExecutor] | FunctionExecutor	A FunctionExecutor instance that can be wired into a Workflow.

Exceptions

Expand table

Type	Description
ValueError	If used with staticmethod or classmethod (unsupported pattern)

function_middleware

Decorator to mark a function as function middleware.

This decorator explicitly identifies a function as function middleware, which processes FunctionInvocationContext objects.

Python

```
function_middleware(func: Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]) ->
Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext],
Awaitable[None]]], Awaitable[None]]
```

Parameters

Expand table

Name	Description
func Required*	Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] The middleware function to mark as function middleware.

Returns

Expand table

Type	Description
Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]	The same function with function middleware marker.

Examples

Python

```
   from agent_framework import function_middleware, FunctionInvocationContext,
ChatAgent


   @function_middleware
   async def logging_middleware(context: FunctionInvocationContext, next):
       print(f"Calling: {context.function.name}")
       await next(context)
       print(f"Result: {context.result}")


   # Use with an agent
```

```
    agent = ChatAgent(chat_client=client, name="assistant",
middleware=logging_middleware)
```

get_checkpoint_summary

Python

```
get_checkpoint_summary(checkpoint: WorkflowCheckpoint) ->
WorkflowCheckpointSummary
```

Parameters

Expand table

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

Expand table

Type	Description
WorkflowCheckpointSummary	

get_logger

Get a logger with the specified name, defaulting to 'agent_framework'.

Python

```
get_logger(name: str = 'agent_framework') -> Logger
```

Parameters

Expand table

Name	Description
name	str The name of the logger. Defaults to 'agent_framework'. Default value: "agent_framework"

Returns

Expand table

Type	Description
Logger	The configured logger instance.

handler

Decorator to register a handler for an executor.

Python

```
handler(func: Callable[[ExecutorT, Any, ContextT], Awaitable[Any]]) ->
Callable[[ExecutorT, Any, ContextT], Awaitable[Any]]
```

Parameters

Expand table

Name	Description
func Required*	Callable[[<xref:agent_framework._workflows._executor.ExecutorT>, Any, <xref:agent_framework._workflows._executor.ContextT>], Awaitable[Any]] The function to decorate. Can be None when used without parameters.

Returns

Expand table

Type	Description
Callable[[<xref:agent_framework._workflows._executor.ExecutorT>, Any, <xref:agent_framework._workflows._executor.ContextT>], Awaitable[Any]]	The decorated function with handler metadata.

Examples

@handler async def handle_string(self, message: str, ctx: WorkflowContext[str]) -> None:

...

@handler async def handle_data(self, message: dict, ctx: WorkflowContext[str | int]) -> None:

...

prepare_function_call_results

Prepare the values of the function call results.

Python

```
prepare_function_call_results(content: TextContent | DataContent |
TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent |
ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent |
FunctionApprovalRequestContent | FunctionApprovalResponseContent | Any |
list[TextContent | DataContent | TextReasoningContent | UriContent |
FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent |
HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent |
FunctionApprovalResponseContent | Any]) -> str
```

Parameters

Expand table

Name	Description
content Required*	TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent | Any | list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent | Any]

Returns

Expand table

Type	Description
str	

prepend_agent_framework_to_user_agent

Prepend "agent-framework" to the User-Agent in the headers.

When user agent telemetry is disabled through the `AGENT_FRAMEWORK_USER_AGENT_DISABLED` environment variable, the User-Agent header will not include the agent-framework

information. It will be sent back as is, or as an empty dict when None is passed.

Python

```
prepend_agent_framework_to_user_agent(headers: dict[str, Any] | None = None) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
headers	dict[str, Any] | None The existing headers dictionary. Default value: None

Returns

Expand table

Type	Description
dict[str, Any]	A new dict with "User-Agent" set to "agent-framework-python/{version}" if headers is None. The modified headers dictionary with "agent-framework-python/{version}" prepended to the User-Agent.

Examples

Python

```
from agent_framework import prepend_agent_framework_to_user_agent

# Add agent-framework to new headers
headers = prepend_agent_framework_to_user_agent()
print(headers["User-Agent"])  # "agent-framework-python/0.1.0"

# Prepend to existing headers
existing = {"User-Agent": "my-app/1.0"}
headers = prepend_agent_framework_to_user_agent(existing)
print(headers["User-Agent"])  # "agent-framework-python/0.1.0 my-app/1.0"
```

response_handler

Decorator to register a handler to handle responses for a request.

Python

```
response_handler(func: Callable[[ExecutorT, Any, Any, ContextT], Awaitable[None]])
-> Callable[[ExecutorT, Any, Any, ContextT], Awaitable[None]]
```

Parameters

Expand table

Name	Description
func Required*	Callable[[<xref:agent_framework._workflows._request_info_mixin.ExecutorT>, Any, Any, <xref:agent_framework._workflows._request_info_mixin.ContextT>], Awaitable[None]] The function to decorate.

Returns

Expand table

Type	Description
Callable[[<xref:agent_framework._workflows._request_info_mixin.ExecutorT>, Any, Any, <xref:agent_framework._workflows._request_info_mixin.ContextT>], Awaitable[None]]	The decorated function with handler metadata.

Examples

Python

```
    @handler
    async def run(self, message: int, context: WorkflowContext[str]) -> None:
        # Example of a handler that sends a request
        ...
        # Send a request with a `CustomRequest` payload and expect a `str`
response.
        await context.request_info(CustomRequest(...), str)


    @response_handler
    async def handle_response(
        self,
        original_request: CustomRequest,
        response: str,
        context: WorkflowContext[str],
    ) -> None:
        # Example of a response handler for the above request
        ...
```

```python
@response_handler
async def handle_response(
    self,
    original_request: CustomRequest,
    response: dict,
    context: WorkflowContext[int],
) -> None:
    # Example of a response handler for a request expecting a dict response
    ...
```

setup_logging

Setup the logging configuration for the agent framework.

Python

```python
setup_logging() -> None
```

Returns

Expand table

Type	Description
None	

use_agent_middleware

Class decorator that adds middleware support to an agent class.

This decorator adds middleware functionality to any agent class. It wraps the `run()` and `run_stream()` methods to provide middleware execution.

The middleware execution can be terminated at any point by setting the `context.terminate` property to True. Once set, the pipeline will stop executing further middleware as soon as control returns to the pipeline.

> **Note**
>
> This decorator is already applied to built-in agent classes. You only need to use
>
> it if you're creating custom agent implementations.

Python

```
use_agent_middleware(agent_class: type[TAgent]) -> type[TAgent]
```

Parameters

Expand table

Name	Description
agent_class Required*	type[<xref:TAgent>] The agent class to add middleware support to.

Returns

Expand table

Type	Description
type[~<xref:TAgent>]	The modified agent class with middleware support.

Examples

Python

```
from agent_framework import use_agent_middleware


@use_agent_middleware
class CustomAgent:
    async def run(self, messages, **kwargs):
        # Agent implementation
        pass

    async def run_stream(self, messages, **kwargs):
        # Streaming implementation
        pass
```

use_chat_middleware

Class decorator that adds middleware support to a chat client class.

This decorator adds middleware functionality to any chat client class. It wraps the `get_response()` and `get_streaming_response()` methods to provide middleware execution.

> [!NOTE]
> This decorator is already applied to built-in chat client classes. You only need to use it if you're creating custom chat client implementations.

Python

```
use_chat_middleware(chat_client_class: type[TChatClient]) -> type[TChatClient]
```

Parameters

Expand table

Name	Description
chat_client_class Required*	type[<xref:TChatClient>] The chat client class to add middleware support to.

Returns

Expand table

Type	Description
type[~<xref:TChatClient>]	The modified chat client class with middleware support.

Examples

Python

```python
from agent_framework import use_chat_middleware


@use_chat_middleware
class CustomChatClient:
    async def get_response(self, messages, **kwargs):
        # Chat client implementation
        pass

    async def get_streaming_response(self, messages, **kwargs):
        # Streaming implementation
        pass
```

use_function_invocation

Class decorator that enables tool calling for a chat client.

This decorator wraps the `get_response` and `get_streaming_response` methods to automatically handle function calls from the model, execute them, and return the results back to the model for further processing.

Python

```
use_function_invocation(chat_client: type[TChatClient]) -> type[TChatClient]
```

Parameters

Expand table

Name	Description
chat_client Required*	type[<xref:TChatClient>] The chat client class to decorate.

Returns

Expand table

Type	Description
type[~<xref:TChatClient>]	The decorated chat client class with function invocation enabled.

Exceptions

Expand table

Type	Description
ChatClientInitializationError	If the chat client does not have the required methods.

Examples

Python

```
from agent_framework import use_function_invocation, BaseChatClient
```

```python
@use_function_invocation
class MyCustomClient(BaseChatClient):
    async def get_response(self, messages, **kwargs):
        # Implementation here
        pass

    async def get_streaming_response(self, messages, **kwargs):
        # Implementation here
        pass


# The client now automatically handles function calls
client = MyCustomClient()
```

validate_workflow_graph

Convenience function to validate a workflow graph.

Python

```python
validate_workflow_graph(edge_groups: Sequence[EdgeGroup], executors: dict[str, Executor], start_executor: Executor) -> None
```

Parameters

Expand table

Name	Description
edge_groups Required*	Sequence[<xref:agent_framework._workflows._edge.EdgeGroup>] list of edge groups in the workflow
executors Required*	dict[str, Executor] Map of executor IDs to executor instances
start_executor Required*	Executor The starting executor (can be instance or ID)

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
WorkflowValidationError	If any validation fails

a2a Package

Classes

Expand table

A2AAgent	Agent2Agent (A2A) protocol implementation. Wraps an A2A Client to connect the Agent Framework with external A2A-compliant agents via HTTP/JSON-RPC. Converts framework ChatMessages to A2A Messages on send, and converts A2A responses (Messages/Tasks) back to framework types. Inherits BaseAgent capabilities while managing the underlying A2A protocol communication. Can be initialized with a URL, AgentCard, or existing A2A Client instance. Initialize the A2AAgent.

A2AAgent Class

Agent2Agent (A2A) protocol implementation.

Wraps an A2A Client to connect the Agent Framework with external A2A-compliant agents via HTTP/JSON-RPC. Converts framework ChatMessages to A2A Messages on send, and converts A2A responses (Messages/Tasks) back to framework types. Inherits BaseAgent capabilities while managing the underlying A2A protocol communication.

Can be initialized with a URL, AgentCard, or existing A2A Client instance.

Initialize the A2AAgent.

Constructor

Python

```
A2AAgent()
```

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the agent. Default value: None
id	The unique identifier for the agent, will be created automatically if not provided. Default value: None
description	A brief description of the agent's purpose. Default value: None
agent_card	The agent card for the agent. Default value: None
url	The URL for the A2A server. Default value: None
client	The A2A client for the agent. Default value: None
http_client	Optional httpx.AsyncClient to use.

Name	Description
	Default value: None
auth_interceptor	Optional authentication interceptor for secured endpoints. Default value: None
timeout	Request timeout configuration. Can be a float (applied to all timeout components), httpx.Timeout object (for full control), or None (uses 10.0s connect, 60.0s read, 10.0s write, 5.0s pool - optimized for A2A operations). Default value: None
kwargs	any additional properties, passed to BaseAgent.

Methods

Expand table

__init__	Initialize the A2AAgent.
__new__	
as_tool	Create an AIFunction tool that wraps this agent.
deserialize_thread	Deserialize a thread from its serialized state.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_new_thread	Return a new AgentThread instance that is compatible with the agent.
run	Get a response from the agent. This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.
run_stream	Run the agent as a stream. This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the A2AAgent.

Python

```
__init__(*, name: str | None = None, id: str | None = None, description: str | None = None, agent_card: AgentCard | None = None, url: str | None = None, client: Client | None = None, http_client: AsyncClient | None = None, auth_interceptor: AuthInterceptor | None = None, timeout: float | Timeout | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
name Required*	str | None
id Required*	str | None
description Required*	str | None
agent_card Required*	<xref:a2a.types.AgentCard> | None
url Required*	str | None

Name	Description
client Required*	<xref:a2a.client.client.Client> | None
http_client Required*	<xref:httpx.AsyncClient> | None
auth_interceptor Required*	<xref:a2a.client.auth.interceptor.AuthInterceptor> | None
timeout Required*	float | <xref:httpx.Timeout> | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the agent. Default value: None
id	The unique identifier for the agent, will be created automatically if not provided. Default value: None
description	A brief description of the agent's purpose. Default value: None
agent_card	The agent card for the agent. Default value: None
url	The URL for the A2A server. Default value: None
client	The A2A client for the agent. Default value: None
http_client	Optional httpx.AsyncClient to use. Default value: None
auth_interceptor	Optional authentication interceptor for secured endpoints. Default value: None
timeout	Request timeout configuration. Can be a float (applied to all timeout components), httpx.Timeout object (for full control), or None (uses 10.0s connect, 60.0s read, 10.0s write, 5.0s pool - optimized for A2A operations).

Name	Description
	Default value: None
kwargs	any additional properties, passed to BaseAgent.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

as_tool

Create an AIFunction tool that wraps this agent.

```
Python
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str
= 'task', arg_description: str | None = None, stream_callback:
Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate],
Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

Expand table

Name	Description
name Required*	str | None
description Required*	str | None
arg_name Required*	str

Name	Description
arg_description Required*	str | None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None

Keyword-Only Parameters

Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

Expand table

Type	Description
AIFunction[<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

Expand table

Type	Description
TypeError	If the agent does not implement AgentProtocol.

Type	Description
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
   from agent_framework import ChatAgent

   # Create an agent
   agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

   # Convert the agent to a tool
   research_tool = agent.as_tool()

   # Use the tool with another agent
   coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```
   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

run

Get a response from the agent.

This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentRunResponse	An agent response item.

run_stream

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

Python

```
async run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] |
None = None, *, thread: AgentThread | None = None, **kwargs: Any) ->
AsyncIterable[AgentRunResponseUpdate]
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AsyncIterable[AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Name	Description
**kwargs	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

AGENT_PROVIDER_NAME

Python

```
AGENT_PROVIDER_NAME: Final[str] = 'A2A'
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[Middleware] | None
```

ag_ui Package

Classes

Expand table

AGUIChatClient	Chat client for communicating with AG-UI compliant servers. This client implements the BaseChatClient interface and automatically handles: • Thread ID management for conversation continuity • State synchronization between client and server • Server-Sent Events (SSE) streaming • Event conversion to Agent Framework types Important: Message History Management This client sends exactly the messages it receives to the server. It does NOT automatically maintain conversation history. The server must handle history via thread_id. For stateless servers: Use ChatAgent wrapper which will send full message history on each request. However, even with ChatAgent, the server must echo back all context for the agent to maintain history across turns. Important: Tool Handling (Hybrid Execution - matches .NET) 1. Client tool metadata sent to server - LLM knows about both client and server tools 2. Server has its own tools that execute server-side 3. When LLM calls a client tool, @use_function_invocation executes it locally 4. Both client and server tools work together (hybrid pattern) The wrapping ChatAgent's @use_function_invocation handles client tool execution automatically when the server's LLM decides to call them. Initialize the AG-UI chat client.
AGUIEventConverter	Converter for AG-UI events to Agent Framework types. Handles conversion of AG-UI protocol events to ChatResponseUpdate objects while maintaining state, aggregating content, and tracking metadata. Initialize the converter with fresh state.

AGUIHttpService	HTTP service for AG-UI protocol communication. Handles HTTP POST requests and Server-Sent Events (SSE) stream parsing for the AG-UI protocol. Initialize the HTTP service.
AgentFrameworkAgent	Wraps Agent Framework agents for AG-UI protocol compatibility. Translates between Agent Framework's AgentProtocol and AG-UI's event-based protocol. Uses orchestrators to handle different execution flows (standard execution, human-in-the-loop, etc.). Orchestrators are checked in order; the first matching orchestrator handles the request. Supports predictive state updates for agentic generative UI, with optional confirmation requirements configurable per use case. Initialize the AG-UI compatible agent wrapper.
ConfirmationStrategy	Strategy for generating confirmation messages during human-in-the-loop flows.
DefaultConfirmationStrategy	Generic confirmation messages suitable for most agents. This preserves the original behavior from v1.
DocumentWriterConfirmationStrategy	Domain-specific confirmation messages for document writing agents.
RecipeConfirmationStrategy	Domain-specific confirmation messages for recipe agents.
TaskPlannerConfirmationStrategy	Domain-specific confirmation messages for task planning agents.

Functions

add_agent_framework_fastapi_endpoint

Add an AG-UI endpoint to a FastAPI app.

Python

```
add_agent_framework_fastapi_endpoint(app: FastAPI, agent: AgentProtocol |
AgentFrameworkAgent, path: str = '/', state_schema: Any | None = None,
predict_state_config: dict[str, dict[str, str]] | None = None, allow_origins:
list[str] | None = None, default_state: dict[str, Any] | None = None) -> None
```

Parameters

Expand table

Name	Description
app Required*	<xref:fastapi.applications.FastAPI> The FastAPI application
agent Required*	AgentProtocol | AgentFrameworkAgent The agent to expose (can be raw AgentProtocol or wrapped)
path	str The endpoint path Default value: "/"
state_schema	Any | None Optional state schema for shared state management; accepts dict or Pydantic model/class Default value: None
predict_state_config	dict[str, dict[str, str]] | None Optional predictive state update configuration. Format: {"state_key": {"tool": "tool_name", "tool_argument": "arg_name"}} Default value: None
allow_origins	list[str] | None CORS origins (not yet implemented) Default value: None
default_state	dict[str, Any] | None Optional initial state to seed when the client does not provide state keys Default value: None

Returns

Expand table

Type	Description
None	

AGUIChatClient Class

Chat client for communicating with AG-UI compliant servers.

This client implements the BaseChatClient interface and automatically handles:

- Thread ID management for conversation continuity
- State synchronization between client and server
- Server-Sent Events (SSE) streaming
- Event conversion to Agent Framework types

Important: Message History Management This client sends exactly the messages it receives to the server. It does NOT automatically maintain conversation history. The server must handle history via thread_id.

For stateless servers: Use ChatAgent wrapper which will send full message history on each request. However, even with ChatAgent, the server must echo back all context for the agent to maintain history across turns.

Important: Tool Handling (Hybrid Execution - matches .NET)

1. Client tool metadata sent to server - LLM knows about both client and server tools
2. Server has its own tools that execute server-side
3. When LLM calls a client tool, @use_function_invocation executes it locally
4. Both client and server tools work together (hybrid pattern)

The wrapping ChatAgent's @use_function_invocation handles client tool execution automatically when the server's LLM decides to call them.

Initialize the AG-UI chat client.

Constructor

Python

```
AGUIChatClient()
```

Parameters

Expand table

Name	Description
endpoint Required*	The AG-UI server endpoint URL (e.g., "http://localhost:8888/")
http_client Required*	Optional httpx.AsyncClient instance. If None, one will be created.
timeout Required*	Request timeout in seconds (default: 60.0)
additional_properties Required*	Additional properties to store
****kwargs** Required*	Additional arguments passed to BaseChatClient

Keyword-Only Parameters

Expand table

Name	Description
endpoint Required*	
http_client	Default value: None
timeout	Default value: 60.0
additional_properties	Default value: None

Examples

Direct usage (server manages thread history):

Python

```python
from agent_framework.ag_ui import AGUIChatClient

client = AGUIChatClient(endpoint="http://localhost:8888/")

# First message - thread ID auto-generated
response = await client.get_response("Hello!")
thread_id = response.additional_properties.get("thread_id")

# Second message - server retrieves history using thread_id
response2 = await client.get_response(
```

```
    "How are you?",
    metadata={"thread_id": thread_id}
)
```

Recommended usage with ChatAgent (client manages history):

Python

```python
from agent_framework import ChatAgent
from agent_framework.ag_ui import AGUIChatClient

client = AGUIChatClient(endpoint="http://localhost:8888/")
agent = ChatAgent(name="assistant", client=client)
thread = await agent.get_new_thread()

# ChatAgent automatically maintains history and sends full context
response = await agent.run("Hello!", thread=thread)
response2 = await agent.run("How are you?", thread=thread)
```

Streaming usage:

Python

```python
async for update in client.get_streaming_response("Tell me a story"):
    if update.contents:
        for content in update.contents:
            if hasattr(content, "text"):
                print(content.text, end="", flush=True)
```

Context manager:

Python

```python
async with AGUIChatClient(endpoint="http://localhost:8888/") as client:
    response = await client.get_response("Hello!")
    print(response.messages[0].text)
```

Methods

Expand table

__init__	Initialize the AG-UI chat client.
__new__	

close	Close the HTTP client.
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the AG-UI chat client.

Python

```
__init__(*, endpoint: str, http_client: AsyncClient | None = None, timeout: float = 60.0, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
endpoint Required*	str The AG-UI server endpoint URL (e.g., "http://localhost:8888/")
http_client Required*	<xref:httpx.AsyncClient> | None Optional httpx.AsyncClient instance. If None, one will be created.
timeout Required*	float Request timeout in seconds (default: 60.0)
additional_properties Required*	dict[str, Any] | None Additional properties to store
****kwargs** Required*	Any Additional arguments passed to BaseChatClient

Keyword-Only Parameters

Expand table

Name	Description
endpoint Required*	
http_client	Default value: None
timeout	Default value: 60.0
additional_properties	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python
`__new__(**kwargs)`

close

Close the HTTP client.

Python
`async close() -> None`

Returns

Expand table

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python
`create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext,`

```
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]],

Name	Description
	Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p	float | None

Name	Description
Required*	
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
   from agent_framework.clients import OpenAIChatClient

   # Create a client
   client = OpenAIChatClient(model_id="gpt-4")

   # Create an agent using the convenience method
   agent = client.create_agent(
       name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
   )

   # Run the agent
   response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages	str	ChatMessage	list[str]	list[ChatMessage]

Name	Description
Required*	The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None

Name	Description
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None

Name	Description
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
```

```
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None

Name	Description
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True.

Name	Description
	Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'agui'
```

AGUIEventConverter Class

Converter for AG-UI events to Agent Framework types.

Handles conversion of AG-UI protocol events to ChatResponseUpdate objects while maintaining state, aggregating content, and tracking metadata.

Initialize the converter with fresh state.

Constructor

```
Python
AGUIEventConverter()
```

Methods

Expand table

__init__	Initialize the converter with fresh state.
__new__	
convert_event	Convert a single AG-UI event to ChatResponseUpdate.

__init__

Initialize the converter with fresh state.

```
Python
__init__() -> None
```

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

convert_event

Convert a single AG-UI event to ChatResponseUpdate.

```
Python
convert_event(event: dict[str, Any]) -> ChatResponseUpdate | None
```

Parameters

Expand table

Name	Description
event Required*	dict[str, Any] AG-UI event dictionary

Returns

Expand table

Type	Description
ChatResponseUpdate | None	ChatResponseUpdate if event produces content, None otherwise

Examples

RUN_STARTED event:

```
Python
converter = AGUIEventConverter()
event = {"type": "RUN_STARTED", "threadId": "t1", "runId": "r1"}
update = converter.convert_event(event)
assert update.additional_properties["thread_id"] == "t1"
```

TEXT_MESSAGE_CONTENT event:

Python

```
event = {"type": "TEXT_MESSAGE_CONTENT", "messageId": "m1", "delta": "Hello"}
update = converter.convert_event(event)
assert update.contents[0].text == "Hello"
```

AGUIHttpService Class

HTTP service for AG-UI protocol communication.

Handles HTTP POST requests and Server-Sent Events (SSE) stream parsing for the AG-UI protocol.

Initialize the HTTP service.

Constructor

Python

```
AGUIHttpService()
```

Parameters

Expand table

Name	Description
endpoint Required*	AG-UI server endpoint URL (e.g., "http://localhost:8888/")
http_client	Optional httpx AsyncClient. If None, creates a new one. Default value: None
timeout	Request timeout in seconds (default: 60.0) Default value: 60.0

Examples

Basic usage:

Python

```
service = AGUIHttpService("http://localhost:8888/")
async for event in service.post_run(
    thread_id="thread_123",
    run_id="run_456",
    messages=[{"role": "user", "content": "Hello"}]
):
    print(event["type"])
```

With context manager:

```python
async with AGUIHttpService("http://localhost:8888/") as service:
    async for event in service.post_run(...):
        print(event)
```

Methods

Expand table

__init__	Initialize the HTTP service.
__new__	
close	Close the HTTP client if owned by this service. Only closes the client if it was created by this service instance. If an external client was provided, it remains the caller's responsibility to close it.
post_run	Post a run request and stream AG-UI events.

__init__

Initialize the HTTP service.

```python
__init__(endpoint: str, http_client: AsyncClient | None = None, timeout: float = 60.0) -> None
```

Parameters

Expand table

Name	Description
endpoint Required*	str AG-UI server endpoint URL (e.g., "http://localhost:8888/")
http_client	<xref:httpx.AsyncClient> | None Optional httpx AsyncClient. If None, creates a new one. Default value: None

Name	Description
`timeout`	float Request timeout in seconds (default: 60.0) Default value: 60.0

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

close

Close the HTTP client if owned by this service.

Only closes the client if it was created by this service instance. If an external client was provided, it remains the caller's responsibility to close it.

```
Python
async close() -> None
```

Returns

Expand table

Type	Description
None	

post_run

Post a run request and stream AG-UI events.

Python

```
async post_run(thread_id: str, run_id: str, messages: list[dict[str, Any]], state: dict[str, Any] | None = None, tools: list[dict[str, Any]] | None = None) -> AsyncIterable[dict[str, Any]]
```

Parameters

Expand table

Name	Description
thread_id Required*	str Thread identifier for conversation continuity
run_id Required*	str Unique run identifier
messages Required*	list[dict[str, Any]] List of messages in AG-UI format
state	dict[str, Any] | None Optional state object to send to server Default value: None
tools	list[dict[str, Any]] | None Optional list of tools available to the agent Default value: None

Returns

Expand table

Type	Description
AsyncIterable[dict[str, Any]]	

Exceptions

Expand table

Type	Description
[<xref:HTTPStatusError>](<xref:httpx.HTTPStatusError>)	If the HTTP request fails
ValueError	If SSE parsing encounters invalid data

Examples

Python

```
service = AGUIHttpService("http://localhost:8888/")
async for event in service.post_run(
    thread_id="thread_abc",
    run_id="run_123",
    messages=[{"role": "user", "content": "Hello"}],
    state={"user_context": {"name": "Alice"}}
):
    if event["type"] == "TEXT_MESSAGE_CONTENT":
        print(event["delta"])
```

AgentFrameworkAgent Class

Wraps Agent Framework agents for AG-UI protocol compatibility.

Translates between Agent Framework's AgentProtocol and AG-UI's event-based protocol. Uses orchestrators to handle different execution flows (standard execution, human-in-the-loop, etc.). Orchestrators are checked in order; the first matching orchestrator handles the request.

Supports predictive state updates for agentic generative UI, with optional confirmation requirements configurable per use case.

Initialize the AG-UI compatible agent wrapper.

Constructor

Python

```
AgentFrameworkAgent()
```

Parameters

Expand table

Name	Description
agent Required*	The Agent Framework agent to wrap
name	Optional name for the agent Default value: None
description	Optional description Default value: None
state_schema	Optional state schema for state management; accepts dict or Pydantic model/class Default value: None
predict_state_config	Configuration for predictive state updates. Format: {"state_key": {"tool": "tool_name", "tool_argument": "arg_name"}} Default value: None
require_confirmation	Whether predictive updates require confirmation. Set to False for agentic generative UI that updates automatically.

Name	Description
	Default value: True
orchestrators	Custom orchestrators (auto-configured if None). Orchestrators are checked in order; first match handles the request. Default value: None
confirmation_strategy	Strategy for generating confirmation messages. Defaults to DefaultConfirmationStrategy if None. Default value: None

Methods

Expand table

__init__	Initialize the AG-UI compatible agent wrapper.
__new__	
run_agent	Run the agent and yield AG-UI events. This is the ONLY public method - much simpler than the original 376-line implementation. All orchestration logic has been extracted into dedicated Orchestrator classes. The method creates an ExecutionContext with all needed data, then finds the first orchestrator that can handle the request and delegates to it.

__init__

Initialize the AG-UI compatible agent wrapper.

Python

```
__init__(agent: AgentProtocol, name: str | None = None, description: str | None = None, state_schema: Any | None = None, predict_state_config: dict[str, dict[str, str]] | None = None, require_confirmation: bool = True, orchestrators: list[Orchestrator] | None = None, confirmation_strategy: ConfirmationStrategy | None = None)
```

Parameters

Expand table

Name	Description
agent	AgentProtocol

Name	Description
Required*	The Agent Framework agent to wrap
name	str | None Optional name for the agent Default value: None
description	str | None Optional description Default value: None
state_schema	Any | None Optional state schema for state management; accepts dict or Pydantic model/class Default value: None
predict_state_config	dict[str, dict[str, str]] | None Configuration for predictive state updates. Format: {"state_key": {"tool": "tool_name", "tool_argument": "arg_name"}} Default value: None
require_confirmation	bool Whether predictive updates require confirmation. Set to False for agentic generative UI that updates automatically. Default value: True
orchestrators	list[<xref:agent_framework_ag_ui._orchestrators.Orchestrator>] | None Custom orchestrators (auto-configured if None). Orchestrators are checked in order; first match handles the request. Default value: None
confirmation_strategy	ConfirmationStrategy | None Strategy for generating confirmation messages. Defaults to DefaultConfirmationStrategy if None. Default value: None

__new__

Python

```
__new__(**kwargs)
```

run_agent

Run the agent and yield AG-UI events.

This is the ONLY public method - much simpler than the original 376-line implementation. All orchestration logic has been extracted into dedicated Orchestrator classes.

The method creates an ExecutionContext with all needed data, then finds the first orchestrator that can handle the request and delegates to it.

Python

```
async run_agent(input_data: dict[str, Any]) -> AsyncGenerator[BaseEvent, None]
```

Parameters

Expand table

Name	Description
input_data Required*	dict[str, Any] The AG-UI run input containing messages, state, etc.

Returns

Expand table

Type	Description
AsyncGenerator[<xref:ag_ui.core.events.BaseEvent>, None]	

Exceptions

Expand table

Type	Description
RuntimeError	If no orchestrator matches (should never happen if DefaultOrchestrator is last in the chain)

ConfirmationStrategy Class

Strategy for generating confirmation messages during human-in-the-loop flows.

Constructor

```Python
ConfirmationStrategy()
```

Methods

Expand table

__init__	
__new__	
on_approval_accepted	Generate message when user approves function execution.
on_approval_rejected	Generate message when user rejects function execution.
on_state_confirmed	Generate message when user confirms predictive state changes.
on_state_rejected	Generate message when user rejects predictive state changes.

__init__

```Python
__init__()
```

__new__

```Python
__new__(**kwargs)
```

on_approval_accepted

Generate message when user approves function execution.

Python

```
abstract on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]] List of approved steps with 'description', 'status', etc.

Returns

Expand table

Type	Description
str	Message to display to user

on_approval_rejected

Generate message when user rejects function execution.

Python

```
abstract on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]] List of rejected steps

Returns

Expand table

Type	Description
str	Message to display to user

on_state_confirmed

Generate message when user confirms predictive state changes.

```
Python
abstract on_state_confirmed() -> str
```

Returns

Expand table

Type	Description
str	Message to display to user

on_state_rejected

Generate message when user rejects predictive state changes.

```
Python
abstract on_state_rejected() -> str
```

Returns

Expand table

Type	Description
str	Message to display to user

DefaultConfirmationStrategy Class

Generic confirmation messages suitable for most agents.

This preserves the original behavior from v1.

Constructor

```
Python
DefaultConfirmationStrategy()
```

Methods

Expand table

__init__	
__new__	
on_approval_accepted	Generate generic approval message with step list.
on_approval_rejected	Generate generic rejection message.
on_state_confirmed	Generate generic state confirmation message.
on_state_rejected	Generate generic state rejection message.

__init__

```
Python
__init__()
```

__new__

```
Python
__new__(**kwargs)
```

on_approval_accepted

Generate generic approval message with step list.

```
Python
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_approval_rejected

Generate generic rejection message.

```
Python
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_state_confirmed

Generate generic state confirmation message.

```
Python
on_state_confirmed() -> str
```

Returns

Expand table

Type	Description
str	

on_state_rejected

Generate generic state rejection message.

```
Python
on_state_rejected() -> str
```

Returns

Expand table

Type	Description
str	

DocumentWriterConfirmationStrategy Class

Domain-specific confirmation messages for document writing agents.

Constructor

```
Python
DocumentWriterConfirmationStrategy()
```

Methods

Expand table

__init__	
__new__	
on_approval_accepted	Generate document-specific approval message.
on_approval_rejected	Generate document-specific rejection message.
on_state_confirmed	Generate document-specific state confirmation message.
on_state_rejected	Generate document-specific state rejection message.

__init__

```
Python
__init__()
```

__new__

```
Python
__new__(**kwargs)
```

on_approval_accepted

Generate document-specific approval message.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_approval_rejected

Generate document-specific rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_state_confirmed

Generate document-specific state confirmation message.

```
Python
on_state_confirmed() -> str
```

Returns

Expand table

Type	Description
str	

on_state_rejected

Generate document-specific state rejection message.

```
Python
on_state_rejected() -> str
```

Returns

Expand table

Type	Description
str	

RecipeConfirmationStrategy Class

Domain-specific confirmation messages for recipe agents.

Constructor

```
Python
RecipeConfirmationStrategy()
```

Methods

Expand table

__init__	
__new__	
on_approval_accepted	Generate recipe-specific approval message.
on_approval_rejected	Generate recipe-specific rejection message.
on_state_confirmed	Generate recipe-specific state confirmation message.
on_state_rejected	Generate recipe-specific state rejection message.

__init__

```
Python
__init__()
```

__new__

```
Python
__new__(**kwargs)
```

on_approval_accepted

Generate recipe-specific approval message.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_approval_rejected

Generate recipe-specific rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_state_confirmed

Generate recipe-specific state confirmation message.

```
Python
on_state_confirmed() -> str
```

Returns

Expand table

Type	Description
str	

on_state_rejected

Generate recipe-specific state rejection message.

```
Python
on_state_rejected() -> str
```

Returns

Expand table

Type	Description
str	

TaskPlannerConfirmationStrategy Class

Domain-specific confirmation messages for task planning agents.

Constructor

```
Python
TaskPlannerConfirmationStrategy()
```

Methods

Expand table

__init__	
__new__	
on_approval_accepted	Generate task-specific approval message.
on_approval_rejected	Generate task-specific rejection message.
on_state_confirmed	Task planners typically don't use state confirmation.
on_state_rejected	Task planners typically don't use state confirmation.

__init__

```
Python
__init__()
```

__new__

```
Python
__new__(**kwargs)
```

on_approval_accepted

Generate task-specific approval message.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_approval_rejected

Generate task-specific rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

Expand table

Name	Description
steps Required*	list[dict[str, Any]]

Returns

Expand table

Type	Description
str	

on_state_confirmed

Task planners typically don't use state confirmation.

Python
`on_state_confirmed() -> str`

Returns

Expand table

Type	Description
str	

on_state_rejected

Task planners typically don't use state confirmation.

Python
`on_state_rejected() -> str`

Returns

Expand table

Type	Description
str	

anthropic Package

Classes

Expand table

AnthropicClient	Anthropic Chat client. Initialize an Anthropic Agent client.

AnthropicClient Class

Anthropic Chat client.

Initialize an Anthropic Agent client.

Constructor

Python

```
AnthropicClient()
```

Keyword-Only Parameters

Expand table

Name	Description
api_key	The Anthropic API key to use for authentication. Default value: None
model_id	The ID of the model to use. Default value: None
anthropic_client	An existing Anthropic client to use. If not provided, one will be created. This can be used to further configure the client before passing it in. For instance if you need to set a different base_url for testing or private deployments. Default value: None
additional_beta_flags	Additional beta flags to enable on the client. Default flags are: "mcp-client-2025-04-04", "code-execution-2025-08-25". Default value: None
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Examples

Python

```python
from agent_framework.anthropic import AnthropicClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set ANTHROPIC_API_KEY=your_anthropic_api_key
# ANTHROPIC_CHAT_MODEL_ID=claude-sonnet-4-5-20250929

# Or passing parameters directly
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    api_key="your_anthropic_api_key",
)

# Or loading from a .env file
client = AnthropicClient(env_file_path="path/to/.env")

# Or passing in an existing client
from anthropic import AsyncAnthropic

anthropic_client = AsyncAnthropic(
    api_key="your_anthropic_api_key", base_url="https://custom-anthropic-endpoint.com"
)
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    anthropic_client=anthropic_client,
)
```

Methods

Expand table

__init__	Initialize an Anthropic Agent client.
__new__	
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string.

	This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the service URL for the chat client.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Anthropic Agent client.

Python

```
__init__(*, api_key: str | None = None, model_id: str | None = None,
anthropic_client: AsyncAnthropic | None = None, additional_beta_flags: list[str] |
None = None, env_file_path: str | None = None, env_file_encoding: str | None =
None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
api_key Required*	str | None

Name	Description
model_id Required*	str | None
anthropic_client Required*	<xref:anthropic.AsyncAnthropic> | None
additional_beta_flags Required*	list[str] | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
api_key	The Anthropic API key to use for authentication. Default value: None
model_id	The ID of the model to use. Default value: None
anthropic_client	An existing Anthropic client to use. If not provided, one will be created. This can be used to further configure the client before passing it in. For instance if you need to set a different base_url for testing or private deployments. Default value: None
additional_beta_flags	Additional beta flags to enable on the client. Default flags are: "mcp-client-2025-04-04", "code-execution-2025-08-25". Default value: None
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.anthropic import AnthropicClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set ANTHROPIC_API_KEY=your_anthropic_api_key
# ANTHROPIC_CHAT_MODEL_ID=claude-sonnet-4-5-20250929

# Or passing parameters directly
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    api_key="your_anthropic_api_key",
)

# Or loading from a .env file
client = AnthropicClient(env_file_path="path/to/.env")

# Or passing in an existing client
from anthropic import AsyncAnthropic

anthropic_client = AsyncAnthropic(
    api_key="your_anthropic_api_key", base_url="https://custom-anthropic-
endpoint.com"
)
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    anthropic_client=anthropic_client,
)
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None

Name	Description
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None

Name	Description
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None

Name	Description
`chat_message_store_factory`	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
`context_providers`	Context providers to include during agent invocation. Default value: None
`middleware`	List of middleware to intercept agent and function invocations. Default value: None
`allow_multiple_tool_calls`	Whether to allow multiple tool calls per agent turn. Default value: None
`conversation_id`	The conversation ID to associate with the agent's messages. Default value: None
`frequency_penalty`	The frequency penalty to use. Default value: None
`logit_bias`	The logit bias to use. Default value: None
`max_tokens`	The maximum number of tokens to generate. Default value: None
`metadata`	Additional metadata to include in the request. Default value: None
`model_id`	The model_id to use for the agent. Default value: None
`presence_penalty`	The presence penalty to use. Default value: None
`response_format`	The format of the response. Default value: None
`seed`	The random seed to use. Default value: None
`stop`	The stop sequence(s) for the request. Default value: None
`store`	Whether to store the response. Default value: None
`temperature`	The sampling temperature to use. Default value: None
`tool_choice`	The tool choice for the request.

Name	Description
	Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}`

Name	Description
	• **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None

Name	Description
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None

Name	Description
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id	str | None

Name	Description
Required*	
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use.

Name	Description
	Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the service URL for the chat client.

```
Python
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL for the chat client, or None if not set.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

```
Python
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None

Name	Description
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'anthropic'
```

azure Package

Classes

Expand table

AgentCallbackContext	Context supplied to callback invocations.

AgentFunctionApp	Main application class for creating durable agent function apps using Durable Entities.
	
	This class uses Durable Entities pattern for agent execution, providing:
	
	• Stateful agent conversations
	• Conversation history management
	• Signal-based operation invocation
	• Better state management than orchestrations
	
	# Example:
	
	Python
	
	``
	from agent_framework.azure import AgentFunctionApp, AzureOpenAIChatClient
	
	# Create agents with unique names
	weather_agent = AzureOpenAIChatClient(...).create_agent(
	name="WeatherAgent",
	instructions="You are a helpful weather agent.",
	tools=[get_weather],
	)
	
	math_agent = AzureOpenAIChatClient(...).create_agent(
	name="MathAgent",
	instructions="You are a helpful math assistant.",
	tools=[calculate],
	)
	
	# Option 1: Pass list of agents during initialization
	app = AgentFunctionApp(agents=[weather_agent, math_agent])
	
	# Option 2: Add agents after initialization
	``

```
    app = AgentFunctionApp()
    app.add_agent(weather_agent)
    app.add_agent(math_agent)


    @app.orchestration_trigger(context_name="context")
    def my_orchestration(context):
        writer = app.get_agent(context, "WeatherAgent")
        thread = writer.get_new_thread()
        forecast_task = writer.run("What's the
forecast?", thread=thread)
        forecast = yield forecast_task
        return forecast
```

This creates:

- HTTP trigger endpoint for each agent's requests (if enabled)
- Durable entity for each agent's state management and execution
- Full access to all Azure Functions capabilities

agents

Dictionary of agent name to AgentProtocol instance

enable_health_check

Whether health check endpoint is enabled

Type: bool

enable_http_endpoints

Whether HTTP endpoints are created for agents

Type: bool

enable_mcp_tool_trigger

Whether MCP tool triggers are created for agents

Type: bool

max_poll_retries

Maximum polling attempts when waiting for responses

poll_interval_seconds

Delay (seconds) between polling attempts

Initialize the AgentFunctionApp.

param agents: List of agent instances to register.

param http_auth_level: HTTP authentication level (default: `func.AuthLevel.FUNCTION`).

param enable_health_check: Enable the built-in health check endpoint (default: `True`).

param enable_http_endpoints: Enable HTTP endpoints for agents (default: `True`).

param enable_mcp_tool_trigger: Enable MCP tool triggers for agents (default: `False`). When enabled, agents will be exposed as MCP tools that can be invoked by MCP-compatible clients.

param max_poll_retries: Maximum polling attempts when waiting for a response. Defaults to `DEFAULT_MAX_POLL_RETRIES`.

param poll_interval_seconds: Delay in seconds between polling attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`.

param default_callback: Optional callback invoked for agents without specific callbacks.

note: If no agents are provided, they can be added later using add_agent.

init(agents: list[AgentProtocol] | None = None, http_auth_level: AuthLevel = func.AuthLevel.FUNCTION, enable_health_check: bool = True, enable_http_endpoints: bool = True, max_poll_retries: int = DEFAULT_MAX_POLL_RETRIES, poll_interval_seconds: float = DEFAULT_POLL_INTERVAL_SECONDS, enable_mcp_tool_trigger: bool = False, default_callback: AgentResponseCallbackProtocol | None = None)

Initialize the AgentFunctionApp.

Parameters: * **agents** (list[*AgentProtocol*] **| *None*) – List of agent instances to register.

```
  * **http_auth_level**
(<xref:azure.functions.decorators.core.AuthLevel>) –
HTTP authentication level (default:
`func.AuthLevel.FUNCTION`).

  * **enable_health_check** (<xref:bool>) – Enable the
built-in health check endpoint (default: `True`).

  * **enable_http_endpoints** (<xref:bool>) – Enable
HTTP endpoints for agents (default: `True`).

  * **enable_mcp_tool_trigger** (<xref:bool>) – Enable
MCP tool triggers for agents (default: `False`). When
enabled, agents will be exposed as MCP tools that can
be invoked by MCP-compatible clients.
```

```
  * **max_poll_retries** (<xref:int>) – Maximum polling
attempts when waiting for a response. Defaults to
`DEFAULT_MAX_POLL_RETRIES`.

  * **poll_interval_seconds** (<xref:float>) – Delay in
seconds between polling attempts. Defaults to
`DEFAULT_POLL_INTERVAL_SECONDS`.

  * **default_callback**
(<xref:agent_framework_azurefunctions._callbacks.AgentR
esponseCallbackProtocol>* | *<xref:None>) – Optional
callback invoked for agents without specific callbacks.
```

Note: If no agents are provided, they can be added later using add_agent.

new(**kwargs)

activity_trigger(input_name: str, activity: str | None = None)

Register an Activity Function.

Parameters: * **input_name** (str) – Parameter name of the Activity input.

```
  * **activity**
(<xref:agent_framework.azure.Optional[str]>) – Name of
Activity Function. The value is None by default, in
which case the name of the method is used.
```

add_agent(agent: AgentProtocol, callback: AgentResponseCallbackProtocol | None = None, enable_http_endpoint: bool | None = None, enable_mcp_tool_trigger: bool | None = None) -> None

Add an agent to the function app after initialization.

Parameters: * **agent** (AgentProtocol) – The Microsoft Agent Framework agent instance (must implement AgentProtocol) The agent must have a 'name' attribute.

```
  * **callback**
(<xref:agent_framework_azurefunctions._callbacks.AgentR
esponseCallbackProtocol>* | *<xref:None>) – Optional
callback invoked during agent execution

  * **enable_http_endpoint** (<xref:bool>* | *
<xref:None>) – Optional flag to enable/disable HTTP
endpoint for this agent. The app level
```

```
enable_http_endpoints setting will override this
setting.

  * **enable_mcp_tool_trigger** (<xref:bool>* | *
<xref:None>) – Optional flag to enable/disable MCP tool
trigger for this agent. The app level
enable_mcp_tool_trigger setting will override this
setting.
```

Raises: ValueError – If the agent doesn't have a 'name' attribute or if an agent with the same name is already registered

Return type: None

property agents: dict[str, AgentProtocol]

Returns dict of agent names to agent instances.

Returns: Dictionary mapping agent names to their AgentProtocol instances.

property app_script_file: str

Name of function app script file in which all the functions are defined.

```
  Script file defined here is for placeholder purpose,
please refer to
  worker defined script file path as the single point
of truth.
```

Returns: Script file name.

assistant_create_output(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *assistantCreate* output binding creates a new assistant with a specified system prompt.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
```

```
specifying extra binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

assistant_post_input(arg_name: str, id: str, user_message: str, ai_connection_name: str | None = '', chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, chat_storage_connection_setting: str | None = 'AzureWebJobsStorage', collection_name: str | None = 'ChatState', temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```

  * **id** (<xref:str>) – The ID of the assistant to
update.

  * **user_message** (<xref:str>) – The message entered
by the user for the assistant to respond to.

  * **ai_connection_name** (<xref:str>* | *<xref:None>)
–

    The name of the configuration section for AI
service
    connectivity.

    * **Azure OpenAI**: If specified, looks for
"Endpoint" and/or "Key" in the section.

    * If not specified or missing, falls back to the
*AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*
environment variables.

    * **Managed Identity**: Required for user-assigned
identity auth.

    * **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY*
environment variable.

  * **model** – *Deprecated.* Use *chat_model* instead.
This parameter is unused and will be removed in future
versions.
```

```
  * **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>*
| *<xref:None>) – The deployment or model name of the
OpenAI Chat Completion API. Default is *"gpt-3.5-
turbo"*.

  * **chat_storage_connection_setting** (<xref:str>* |
*<xref:None>) – The config section name for assistant
chat table storage. Default is *"AzureWebJobsStorage"*.

  * **collection_name** (<xref:str>* | *<xref:None>) –
The collection or table name used for assistant chat
storage. Default is *"ChatState"*.

  * **temperature** (<xref:str>* | *<xref:None>) –
Sampling temperature, between 0 and 2. Higher values
(e.g., 0.8) increase randomness; lower values (e.g.,
0.2) make output more deterministic.

  * **top_p** (<xref:str>* | *<xref:None>) –
Alternative to temperature sampling. Uses nucleus
sampling to consider only the tokens comprising the top
*top_p* probability mass. Recommended to use either
this or *temperature*.

  * **max_tokens** (<xref:str>* | *<xref:None>) –
Maximum tokens to generate. Total tokens (prompt +
output) must not exceed the model's context length.

  * **is_reasoning_model** (<xref:bool>* | *
<xref:None>) – Whether the chat model is a reasoning
model. If true, *max_tokens* and *temperature* are not
supported.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying extra binding fields in *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

assistant_query_input(arg_name: str, id: str, timestamp_utc: str, chat_storage_connection_setting: str | None = 'AzureWebJobsStorage', collection_name: str | None = 'ChatState', data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *assistantQuery* input binding retrieves assistant chat history and passes it to the function.

This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **timestamp_utc** (<xref:str>) – The earliest
timestamp (in UTC) for the messages to retrieve from
the chat history. Must be in ISO 8601 format, e.g.,
*"2023-08-01T00:00:00Z"*.

  * **chat_storage_connection_setting** (<xref:str>* |
*<xref:None>) – The name of the configuration section
containing the connection settings for assistant chat
storage. Defaults to *"AzureWebJobsStorage"*.

  * **collection_name** (<xref:str>* | *<xref:None>) –
The name of the table or collection used for assistant
chat storage. Defaults to *"ChatState"*.

  * **id** (<xref:str>) – The unique identifier of the
assistant whose history is being queried.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

assistant_skill_trigger(arg_name: str, function_description: str, function_name: str | None = None, parameter_description_json: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions to invoke and when.

Ref: https://platform.openai.com/docs/guides/function-calling

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the

extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

Parameters: * **arg_name** (str) – The name of the trigger parameter in the function code.

```
  * **function_description** (<xref:str>) – A
description of the assistant function, which is
provided to the model.

  * **function_name** (<xref:str>* | *<xref:None>) –
The name of the assistant function, which is passed to
the language model.

  * **parameter_description_json** (<xref:str>* | *
<xref:None>) – A JSON-formatted description of the
function parameters, provided to the model. If omitted,
the description is autogenerated.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Additional keyword arguments for specifying binding
fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

property auth_level: AuthLevel

Authorization level of the function app. Will be applied to the http trigger functions which do not have authorization level specified.

Returns: Authorization level of the function app.

blob_input(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *blob_input* decorator adds <xref:agent_framework.azure.BlobInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Parameters: * **arg_name** (str) – The name of the variable that represents the blob in the function code.

```
  * **path** (<xref:str>) - The path to the blob.

  * **connection** (<xref:str>) - The name of an app
setting or setting collection that specifies how to
connect to Azure Blobs.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) - Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** - Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

blob_output(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *blob_output* decorator adds <xref:agent_framework.azure.BlobOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Parameters: * **arg_name** (str) – The name of the variable that represents the blob in the function code.

```
  * **path** (<xref:str>) – The path to the blob.

  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Azure Blobs.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** – Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

blob_trigger(arg_name: str, path: str, connection: str, source: BlobSource | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The blob_change_trigger decorator adds <xref:agent_framework.azure.BlobTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building <xref:agent_framework.azure.Function> object used in worker function indexing model. This is equivalent to defining BlobTrigger in the function.json which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.InputStream> object in function code.

```
  * **path** (<xref:str>) – The path to the blob.

  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Azure Blobs.

  * **source**
(<xref:azure.functions.decorators.core.BlobSource>* | *
<xref:None>) – Sets the source of the triggering event.
Use "EventGrid" for an Event Grid-based blob trigger,
which provides much lower latency. The default is
```

```
"LogsAndContainerScan", which uses the standard polling
mechanism to detect changes in the container.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how Functions runtime should
treat the parameter value.

  * **kwargs** – Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_input(arg_name: str, connection: str, database_name: str, container_name: str, partition_key: str | None = None, id: str | None = None, sql_query: str | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_input* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-cosmosdb-v4 Additional details: https://aka.ms/cosmosdb-v4-update

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting container that specifies how to
connect to the Azure Cosmos DB account being monitored.

  * **database_name** (<xref:str>) – The name of the
Cosmos DB database that contains the document.

  * **container_name** (<xref:str>) – The name of the
```

```
container that holds the document.

  * **partition_key** (<xref:str>* | *<xref:None>) –
The partition key value for the document lookup.

  * **id** (<xref:str>* | *<xref:None>) – The ID of the
document to retrieve.

  * **sql_query** (<xref:str>* | *<xref:None>) – An
Azure Cosmos DB SQL query used to retrieve multiple
documents.

  * **preferred_locations** (<xref:str>* | *
<xref:None>) – (Optional) Preferred geo-replicated
regions. Values should be comma-separated, e.g., "East
US,South Central US,North Europe".

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_input_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, id: str | None = None, sql_query: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_input_v3* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.

```
  * **database_name** (<xref:str>) – The name of the
Azure Cosmos DB database containing the target
collection.

  * **collection_name** (<xref:str>) – The name of the
collection that contains the document(s).

  * **connection_string_setting** (<xref:str>) – The
name of the app setting that contains the Azure Cosmos
DB connection string.

  * **id** (<xref:str>* | *<xref:None>) – The ID of a
single document to retrieve.

  * **sql_query** (<xref:str>* | *<xref:None>) – An
Azure Cosmos DB SQL query used to retrieve multiple
documents.

  * **partition_key** (<xref:str>* | *<xref:None>) –
Specifies the partition key value for the lookup.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_output(arg_name: str, connection: str, database_name: str, container_name: str, create_if_not_exists: bool | None = None, partition_key: str | None = None, container_throughput: int | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_output* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4

Parameters: * **arg_name** (str) – The name of the variable that represents the Cosmos DB output object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to the Azure Cosmos DB account.

  * **database_name** (<xref:str>) – The name of the
Azure Cosmos DB database containing the target
container.

  * **container_name** (<xref:str>) – The name of the
container to which documents are written.

  * **create_if_not_exists** (<xref:bool>* | *
<xref:None>) – A boolean indicating whether the
container should be created automatically if it does
not exist.

  * **partition_key** (<xref:str>* | *<xref:None>) –
When *create_if_not_exists* is *True*, this defines the
partition key path for the created container.

  * **container_throughput** (<xref:int>* | *
<xref:None>) – When *create_if_not_exists* is *True*,
this defines the throughput (Request Units) for the
created container.

  * **preferred_locations** (<xref:str>* | *
<xref:None>) – Preferred geographic regions for geo-
replicated database accounts in Azure Cosmos DB. When
set along with *use_multiple_write_locations*, it
enables multi-region writes.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_output_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, create_if_not_exists: bool | None = None, partition_key: str | None = None, collection_throughput: int | None = None, use_multiple_write_locations: bool | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_output_v3* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Parameters: * **arg_name** (str) – The name of the variable that represents the Cosmos DB output object in the function code.

```
  * **database_name** (<xref:str>) – The name of the
Azure Cosmos DB database containing the monitored
collection.

  * **collection_name** (<xref:str>) – The name of the
collection to which documents are written.

  * **connection_string_setting** (<xref:str>) – The
name of an app setting or setting collection that
specifies how to connect to the Azure Cosmos DB
account.

  * **create_if_not_exists** (<xref:bool>* | *
<xref:None>) – A boolean indicating whether the
collection should be created automatically if it does
not exist.

  * **partition_key** (<xref:str>* | *<xref:None>) –
When *create_if_not_exists* is *True*, this defines the
partition key path for the created collection.

  * **collection_throughput** (<xref:int>* | *
<xref:None>) – When *create_if_not_exists* is *True*,
this defines the throughput (Request Units) for the
created collection.

  * **use_multiple_write_locations** (<xref:bool>* | *
```

```
<xref:None>) - When set to *True* and used in
conjunction with *preferred_locations*, this enables
multi-region writes in the Azure Cosmos DB service.

  * **preferred_locations** (<xref:str>* | *
<xref:None>) - Preferred geographic regions for geo-
replicated database accounts in Azure Cosmos DB.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) - Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_trigger(arg_name: str, connection: str, database_name: str, container_name: str, lease_connection: str | None = None, lease_database_name: str | None = None, lease_container_name: str | None = None, create_lease_container_if_not_exists: bool | None = None, leases_container_throughput: int | None = None, lease_container_prefix: str | None = None, feed_poll_delay: int | None = None, lease_acquire_interval: int | None = None, lease_expiration_interval: int | None = None, lease_renew_interval: int | None = None, max_items_per_invocation: int | None = None, start_from_beginning: time | None = None, start_from_time: time | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *cosmos_db_trigger* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to the monitored Azure Cosmos DB account.

  * **database_name** (<xref:str>) – The name of the
Azure Cosmos DB database containing the monitored
collection.

  * **container_name** (<xref:str>) – The name of the
container being monitored.

  * **lease_connection** (<xref:str>* | *<xref:None>) –
(Optional) The name of an app setting or setting
collection that specifies how to connect to the Cosmos
DB account that holds the lease container.

  * **lease_database_name** (<xref:str>* | *
<xref:None>) – The name of the database that holds the
collection used to store leases.

  * **lease_container_name** (<xref:str>* | *
<xref:None>) – (Optional) The name of the container
used to store leases. If not set, the default value
"leases" is used.

  * **create_lease_container_if_not_exists**
(<xref:bool>* | *<xref:None>) – (Optional) If *True*,
the leases container is created automatically if it
does not exist. Defaults to *False*. Note: When using
Azure AD identities, container creation is not allowed,
and the function will not start if this is *True*.

  * **leases_container_throughput** (<xref:int>* | *
<xref:None>) – (Optional) The number of Request Units
(RUs) to assign when the leases container is created.
This is used only when
*create_lease_container_if_not_exists* is *True*. It is
automatically set when configured through the Azure
Portal.

  * **lease_container_prefix** (<xref:str>* | *
<xref:None>) – (Optional) A prefix added to leases
created in the lease container for this function. Use
this to allow multiple functions to share the same
lease container with different prefixes.

  * **feed_poll_delay** (<xref:int>* | *<xref:None>) –
The delay (in milliseconds) between polling a partition
for new changes after draining current changes.

  * **lease_acquire_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) to
```

```
trigger a task to check if partitions are evenly
distributed across host instances.

  * **lease_expiration_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) for which
a lease is held for a partition.

  * **lease_renew_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) to renew
all leases for partitions currently held by an
instance.

  * **max_items_per_invocation** (<xref:int>* | *
<xref:None>) – Maximum number of items received per
function call.

  * **start_from_beginning** (<xref:datetime.time>* | *
<xref:None>) – If *True*, the trigger starts reading
changes from the beginning of the collection's change
history instead of the current time.

  * **start_from_time** (<xref:datetime.time>* | *
<xref:None>) – (Optional) The date and time from which
to begin reading the change feed. Use ISO 8601 format
with a UTC designator, e.g., *2021-02-16T14:19:29Z*.
Only used to set the initial trigger state; has no
effect once the trigger has a lease state.

  * **preferred_locations** (<xref:str>* | *
<xref:None>) – Preferred locations (regions) for geo-
replicated Cosmos DB accounts.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Additional keyword arguments for specifying binding
fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_trigger_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, lease_collection_name: str | None = None, lease_connection_string_setting: str | None = None, lease_database_name: str | None = None, create_lease_collection_if_not_exists: bool | None = None, leases_collection_throughput: int | None = None, lease_collection_prefix: str | None = None, checkpoint_interval: int | None = None,

checkpoint_document_count: int | None = None, feed_poll_delay: int | None = None, lease_renew_interval: int | None = None, lease_acquire_interval: int | None = None, lease_expiration_interval: int | None = None, max_items_per_invocation: int | None = None, start_from_beginning: bool | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *cosmos_db_trigger_v3* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed.

See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.

```
  * **database_name** (<xref:str>) – The name of the
Azure Cosmos DB database containing the monitored
collection.

  * **collection_name** (<xref:str>) – The name of the
collection being monitored.

  * **connection_string_setting** (<xref:str>) – The
name of an app setting or setting collection specifying
how to connect to the monitored Azure Cosmos DB
account.

  * **lease_collection_name** (<xref:str>* | *
<xref:None>) – The name of the collection used to store
leases.

  * **lease_connection_string_setting** (<xref:str>* |
*<xref:None>) – The name of an app setting or setting
collection that specifies how to connect to the Azure
Cosmos DB account that holds the lease collection.

  * **lease_database_name** (<xref:str>* | *
<xref:None>) – The name of the database that holds the
lease collection.
```

```
  * **create_lease_collection_if_not_exists**
(<xref:bool>* | *<xref:None>) – If *True*, the lease
collection is automatically created when it does not
already exist.

  * **leases_collection_throughput** (<xref:int>* | *
<xref:None>) – The number of Request Units (RUs)
assigned when the lease collection is created.

  * **lease_collection_prefix** (<xref:str>* | *
<xref:None>) – A prefix added to leases created in the
lease collection for this Function.

  * **checkpoint_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) between
lease checkpoints. The default behavior is to
checkpoint after every function call.

  * **checkpoint_document_count** (<xref:int>* | *
<xref:None>) – Number of documents processed between
lease checkpoints. Default is after every function
call.

  * **feed_poll_delay** (<xref:int>* | *<xref:None>) –
The delay (in milliseconds) between polling a partition
for new changes after all current changes are drained.

  * **lease_renew_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) to renew
leases for partitions currently held by an instance.

  * **lease_acquire_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) to
trigger a task that checks whether partitions are
evenly distributed among host instances.

  * **lease_expiration_interval** (<xref:int>* | *
<xref:None>) – The interval (in milliseconds) for which
a lease is held for a partition.

  * **max_items_per_invocation** (<xref:int>* | *
<xref:None>) – Maximum number of items received per
function call.

  * **start_from_beginning** (<xref:bool>* | *
<xref:None>) – If *True*, the trigger starts reading
changes from the beginning of the collection's change
history rather than from the current time.

  * **preferred_locations** (<xref:str>* | *
<xref:None>) – Preferred locations (regions) for geo-
replicated database accounts in Azure Cosmos DB.

  * **data_type**
```

```
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Additional keyword arguments for specifying binding
fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_binding_output(arg_name: str, binding_name: str, operation: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_binding_output* decorator adds <xref:agent_framework.azure.DaprBindingOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-binding-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```

  * **binding_name** (<xref:str>) – The configured name
of the binding.

  * **operation** (<xref:str>) – The configured
operation.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_binding_trigger(arg_name: str, binding_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *dapr_binding_trigger* decorator adds <xref:agent_framework.azure.DaprBindingTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the trigger input in the function code.

```
  * **binding_name** (<xref:str>) – The name of the
Dapr trigger. If not specified, the function name is
used as the trigger name.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Keyword arguments for specifying additional binding
fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_invoke_output(arg_name: str, app_id: str, method_name: str, http_verb: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_invoke_output* decorator adds <xref:agent_framework.azure.DaprInvokeOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprInvokeOutput* in the *function.json*, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-invoke-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **app_id** (<xref:str>) – The Dapr app ID to
invoke.

  * **method_name** (<xref:str>) – The method name of
the app to invoke.

  * **http_verb** (<xref:str>) – The HTTP verb to use
for the invocation (e.g., GET, POST).

  * **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_publish_output(arg_name: str, pub_sub_name: str, topic: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_publish_output* decorator adds <xref:agent_framework.azure.DaprPublishOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-publish-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **pub_sub_name** (<xref:str>) – The pub/sub
component name to publish to.

  * **topic** (<xref:str>) – The name of the topic to
publish to.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_secret_input(arg_name: str, secret_store_name: str, key: str, metadata: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_secret_input* decorator adds <xref:agent_framework.azure.DaprSecretInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-secret-input-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state input object in the function code.

```
  * **secret_store_name** (<xref:str>) – The name of
the secret store from which to retrieve the secret.

  * **key** (<xref:str>) – The key identifying the name
of the secret to retrieve.

  * **metadata** (<xref:str>) – Metadata properties in
the form "key1=value1&key2=value2".

  * **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; Defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_service_invocation_trigger(arg_name: str, method_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *dapr_service_invocation_trigger* decorator adds <xref:agent_framework.azure.DaprServiceInvocationTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-service-invocation

Parameters: * **arg_name** (str) – The name of the variable that represents the service invocation input in the function code.

```
  * **method_name** (<xref:str>) – The name of the
method on a remote Dapr App. If not specified, the name
of the function is used as the method name.
```

```
  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Keyword arguments for specifying additional binding
fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_state_input(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_state_input* decorator adds <xref:agent_framework.azure.DaprStateInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateInput* in the *function.json*, which enables the function to read state from the underlying state store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-state-input-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state input object in the function code.

```
  * **state_store** (<xref:str>) – The state store
containing the state.

  * **key** (<xref:str>) – The name of the key to
retrieve.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; Defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.
```

```
  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_state_output(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_state_output* decorator adds <xref:agent_framework.azure.DaprStateOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-state-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **state_store** (<xref:str>) – The state store
containing the state for the specified keys.

  * **key** (<xref:str>) – The name of the key used to
store the state.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_topic_trigger(arg_name: str, pub_sub_name: str, topic: str, route: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *dapr_topic_trigger* decorator adds <xref:agent_framework.azure.DaprTopicTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-topic

Parameters: * **arg_name** (str) – The name of the variable that represents the trigger input in the function code.

```
  * **pub_sub_name** (<xref:str>) – The name of the
Dapr pub/sub component.

  * **topic** (<xref:str>) – The topic name. If
unspecified, the function name is used.

  * **route** (<xref:str>* | *<xref:None>) – The route
for the trigger. If unspecified, the topic name is
used.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Additional keyword arguments for specifying extra
binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

durable_client_input(client_name: str, task_hub: str | None = None, connection_name: str | None = None)

Register a Durable-client Function.

Parameters: * **client_name** (str) – Parameter name of durable client.

```
  * **task_hub**
(<xref:agent_framework.azure.Optional[str]>) – Used in
scenarios where multiple function apps share the same
storage account but need to be isolated from each
other. If not specified, the default value from
host.json is used. This value must match the value used
by the target orchestrator functions.

  * **connection_name**
(<xref:agent_framework.azure.Optional[str]>) – The name
of an app setting that contains a storage account
connection string. The storage account represented by
this connection string must be the same one used by the
target orchestrator functions. If not specified, the
default storage account connection string for the
function app is used.
```

durable_openai_agent_orchestrator(_func=None, *, model_provider=None, model_retry_options: RetryOptions | None = RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5))

Decorate Azure Durable Functions orchestrators that use OpenAI Agents.

Parameters: * **model_provider** (<xref:agent_framework.azure.Optional[ModelProvider]>) – Use a non-default ModelProvider instead of the default OpenAIProvider, such as when testing.

```
  * **model_retry_options**
(<xref:azure.durable_functions.models.RetryOptions.Retr
yOptions>* | *<xref:None>) –
```

embeddings_input(arg_name: str, input: str, input_type: InputType, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The embeddings input decorator generates embeddings used to measure the relatedness of text strings.

Ref: https://platform.openai.com/docs/guides/embeddings

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **input** (<xref:str>) – The input source
containing the data to generate embeddings for.

  * **input_type**
(<xref:azure.functions.decorators.openai.InputType>) –
The type of the input (e.g., string, list, file
reference).

  * **ai_connection_name** (<xref:str>* | *<xref:None>)
–

    The name of the configuration section for AI
service
    connectivity settings.

    * **Azure OpenAI**: If specified, looks for
"Endpoint" and/or "Key" in this section.

    * If not specified or not found, falls back to the
environment variables *AZURE_OPENAI_ENDPOINT* and
*AZURE_OPENAI_KEY*.

    * For user-assigned managed identity
authentication, this is required.

    * **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY*
environment variable.

  * **model** – *(Deprecated)* Use *embeddings_model*
instead. This parameter is unused and will be removed
in future versions.

  * **embeddings_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>*
| *<xref:None>) – The deployment or model name for
OpenAI Embeddings. Default is *"text-embedding-ada-
002"*.

  * **max_chunk_length** (<xref:int>* | *<xref:None>) –
The maximum number of characters into which the input
should be chunked. Default is *8 * 1024*.

  * **max_overlap** (<xref:int>* | *<xref:None>) – The
maximum number of characters to overlap between input
chunks. Default is *128*.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Optional. Defines how the
Functions runtime should interpret the parameter value.

  * **kwargs** – Additional keyword arguments to
include in the *function.json* binding configuration.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

embeddings_store_output(arg_name: str, input: str, input_type: InputType, store_connection_name: str, collection: str, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

Add an embeddings input binding to the function.

The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions:

- Azure AI Search
- Azure Data Explorer
- Azure Cosmos DB using MongoDB

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **input** (<xref:str>) – The input data for which
embeddings should be generated.

  * **input_type**
(<xref:azure.functions.decorators.openai.InputType>) –
The type of the input (e.g., string, list).

  * **store_connection_name** (<xref:str>) – The name
of an app setting or environment variable containing
the vector store connection value.

  * **collection** (<xref:str>) – The collection or
table to search within.

  * **ai_connection_name** (<xref:str>* | *<xref:None>)
–

    The name of the configuration section for AI
service
    connectivity settings.

    * For Azure OpenAI: If specified, looks for
"Endpoint" and/or "Key" in this configuration section.

    * If not specified, or if the section is missing,
falls back to environment variables:
```

```
*AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.

    * For user-assigned managed identity, this property
is required.

    * For OpenAI (non-Azure), set the *OPENAI_API_KEY*
environment variable.

  * **model** – Deprecated. Use *embeddings_model*
instead. This parameter is unused and will be removed
in future versions.

  * **embeddings_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>*
| *<xref:None>) – The deployment or model name for
OpenAI Embeddings. The default is "text-embedding-ada-
002".

  * **max_chunk_length** (<xref:int>* | *<xref:None>) –
The maximum number of characters to chunk the input
into.

  * **max_overlap** (<xref:int>* | *<xref:None>) – The
maximum number of characters to overlap between chunks.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Optional. Defines how the
Functions runtime should treat the parameter value.
Defaults to None.

  * **kwargs** – Additional keyword arguments to
include in the *function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

enable_health_check: bool

enable_http_endpoints: bool

enable_mcp_tool_trigger: bool

entity_trigger(context_name: str, entity_name: str | None = None)

Register an Entity Function.

Parameters: * **context_name** (str) – Parameter name of the Entity input.

```

  * **entity_name**
(<xref:agent_framework.azure.Optional[str]>) – Name of
```

```
Entity Function. The value is None by default, in which
case the name of the method is used.
```

event_grid_output(arg_name: str, topic_endpoint_uri: str | None = None, topic_key_setting: str | None = None, connection: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *event_grid_output* decorator adds <xref:agent_framework.azure.EventGridOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining the output binding in the *function.json*, which enables the function to write events to a custom topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/eventgridtrigger

Parameters: * **arg_name** (str) – The variable name used in the function code that represents the event.

```

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) - Defines how the Functions
runtime should treat the parameter value.

  * **topic_endpoint_uri** (<xref:str>* | *<xref:None>)
- The name of an app setting that contains the URI for
the custom topic.

  * **topic_key_setting** (<xref:str>* | *<xref:None>)
- The name of an app setting that contains an access
key for the custom topic.

  * **connection** (<xref:str>* | *<xref:None>) - The
value of the common prefix for the setting that
contains the topic endpoint URI.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

event_grid_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *event_grid_trigger* decorator adds <xref:agent_framework.azure.EventGridTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/eventgridtrigger

Parameters: * **arg_name** (str) – The variable name used in the function code for the parameter that receives the event data.

```
  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

event_hub_message_trigger(arg_name: str, connection: str, event_hub_name: str, data_type: DataType | str | None = None, cardinality: Cardinality | str | None = None, consumer_group: str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *event_hub_message_trigger* decorator adds <xref:agent_framework.azure.EventHubTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-event-hubs

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.EventHubEvent> object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Event Hubs.

  * **event_hub_name** (<xref:str>) – The name of the
Event Hub.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **cardinality**
(<xref:azure.functions.decorators.core.Cardinality>* |
*<xref:str>* | *<xref:None>) – Set to "many" to enable
batching.

  * **consumer_group** (<xref:str>* | *<xref:None>) –
Optional. The consumer group used to subscribe to
events in the hub.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
– Additional keyword arguments to include in the
binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

event_hub_output(arg_name: str, connection: str, event_hub_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *event_hub_output* decorator adds <xref:agent_framework.azure.EventHubOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-event-hubs

Parameters: * **arg_name** (str) – The name of the variable representing the Event Hub output object in the function code.

```
```

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Event Hub.

  * **event_hub_name** (<xref:str>) – The name of the
Event Hub to send messages to.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

function_name(name: str, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]

Optional: Sets name of the <xref:agent_framework.azure.Function> object. If not set, it will default to the name of the method name.

Parameters: * **name** (str) – Name of the function.

```

  * **setting_extra_fields** (<xref:typing.Dict>*[*
<xref:str>*, *<xref:agent_framework.exceptions.Any>*]
**| *<xref:None>) – Keyword arguments for specifying
```

Return type: Callable[[...], Any]

additional setting fields :return: Decorator function.

generic_input_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *generic_input_binding* decorator adds <xref:agent_framework.azure.GenericInputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-custom

Parameters: * **arg_name** (str) – The name of the input parameter in the function code.

```
  * **type** (<xref:str>) – The type of the binding.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

generic_output_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *generic_output_binding* decorator adds <xref:agent_framework.azure.GenericOutputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-custom

Parameters: * **arg_name** (str) – The name of the output parameter in the function code.

```
  * **type** (<xref:str>) – The type of the binding.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
```

```
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

generic_trigger(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *generic_trigger* decorator adds <xref:agent_framework.azure.GenericTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-custom

Parameters: * **arg_name** (str) – The name of the trigger parameter in the function code.

```

  * **type** (<xref:str>) – The type of the binding.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

get_agent(context: Any, agent_name: str) -> <xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>

Return a DurableAIAgent proxy for a registered agent.

Parameters: * **context** (Any) – Durable Functions orchestration context invoking the agent.

```
  * **agent_name** (<xref:str>) – Name of the agent
registered on this app.
```

Raises: ValueError – If the requested agent has not been registered.

Returns: DurableAIAgent wrapper bound to the orchestration context.

Return type: <xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>

get_functions() -> List[Function]

Get the function objects in the function app.

Returns: A list of <xref:agent_framework.azure.Function> objects defined in the app.

Return type: List[Function]

http_type(http_type: str) -> Callable[[...], Any]

Set http type of the <xref:agent_framework.azure.Function> object.

Parameters: **http_type** (str) – Http type of the function.

Returns: Decorator function.

Return type: Callable[[...], Any]

kafka_output(arg_name: str, topic: str, broker_list: str, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None = None, o_auth_bearer_extensions: str | None = None, max_message_bytes: int = 1_000_000, batch_size: int = 10_000, enable_idempotence: bool = False, message_timeout_ms: int = 300_000, request_timeout_ms: int = 5_000, max_retries: int = 2_147_483_647, authentication_mode: BrokerAuthenticationMode | str | None = 'NOTSET', protocol: BrokerProtocol | str | None = 'NOTSET', linger_ms: int = 5, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *kafka_output* decorator adds <xref:agent_framework.azure.KafkaOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a Kafka output binding in *function.json*, which enables the function to write events to a Kafka topic. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/kafkaoutput

Parameters: * **arg_name** (str) – The variable name used in the function code that represents the output event.

```
  * **topic** (<xref:str>) – The Kafka topic to which
messages are published.

  * **broker_list** (<xref:str>) – The list of Kafka
brokers to which the producer connects.

  * **avro_schema** (<xref:str>* | *<xref:None>) –
Optional. Avro schema to generate a generic record.

  * **username** (<xref:str>* | *<xref:None>) – SASL
username for use with the PLAIN and SASL-SCRAM
mechanisms. Equivalent to *'sasl.username'* in
librdkafka.

  * **password** (<xref:str>* | *<xref:None>) – SASL
password for use with the PLAIN and SASL-SCRAM
mechanisms. Equivalent to *'sasl.password'* in
librdkafka.

  * **ssl_key_location** (<xref:str>* | *<xref:None>) –
Path to the client's private key (PEM) for
authentication. Equivalent to *'ssl.key.location'* in
librdkafka.

  * **ssl_ca_location** (<xref:str>* | *<xref:None>) –
Path to the CA certificate for verifying the broker's
certificate. Equivalent to *'ssl.ca.location'* in
librdkafka.

  * **ssl_certificate_location** (<xref:str>* | *
<xref:None>) – Path to the client's certificate file.
Equivalent to *'ssl.certificate.location'* in
librdkafka.

  * **ssl_key_password** (<xref:str>* | *<xref:None>) –
Password for the client's SSL key. Equivalent to
*'ssl.key.password'* in librdkafka.
```

```
  * **schema_registry_url** (<xref:str>* | *
<xref:None>) – URL of the Avro Schema Registry.

  * **schema_registry_username** (<xref:str>* | *
<xref:None>) – Username for accessing the Schema
Registry.

  * **schema_registry_password** (<xref:str>* | *
<xref:None>) – Password for accessing the Schema
Registry.

  * **o_auth_bearer_method**
(<xref:azure.functions.decorators.kafka.OAuthBearerMeth
od>* | *<xref:str>* | *<xref:None>) – OAuth bearer
method to use, e.g., *'default'* or *'oidc'*.
Equivalent to *'sasl.oauthbearer.method'* in
librdkafka.

  * **o_auth_bearer_client_id** (<xref:str>* | *
<xref:None>) – Used with *oidc* method. Equivalent to
*'sasl.oauthbearer.client.id'* in librdkafka.

  * **o_auth_bearer_client_secret** (<xref:str>* | *
<xref:None>) – Used with *oidc* method. Equivalent to
*'sasl.oauthbearer.client.secret'* in librdkafka.

  * **o_auth_bearer_scope** (<xref:str>* | *
<xref:None>) – Scope of the access request sent to the
broker. Equivalent to *'sasl.oauthbearer.scope'* in
librdkafka.

  * **o_auth_bearer_token_endpoint_url** (<xref:str>* |
*<xref:None>) – OAuth token endpoint. Equivalent to
*'sasl.oauthbearer.token.endpoint.url'* in librdkafka.

  * **o_auth_bearer_extensions** (<xref:str>* | *
<xref:None>) – Additional metadata sent to the broker.
Comma-separated key=value pairs, e.g.,
*"supportFeatureX=true,organizationId=sales-emea"*.
Equivalent to *'sasl.oauthbearer.extensions'* in
librdkafka.

  * **max_message_bytes** (<xref:int>) – Maximum size
(in bytes) of a transmitted message. Default is 1MB.

  * **batch_size** (<xref:int>) – Maximum number of
messages batched in one MessageSet. Default is 10,000.

  * **enable_idempotence** (<xref:bool>) – If *True*,
ensures messages are delivered exactly once and in
order. Default is *False*.

  * **message_timeout_ms** (<xref:int>) – Local timeout
for message delivery. Default is 300000 ms.
```

```
  * **request_timeout_ms** (<xref:int>) – Timeout for
producer request acknowledgment. Default is 5000 ms.

  * **max_retries** (<xref:int>) – Maximum number of
retry attempts for failed messages. Default is
2147483647. Retrying may cause reordering unless
*enable_idempotence=True*.

  * **authentication_mode**
(<xref:azure.functions.decorators.kafka.BrokerAuthentic
ationMode>* | *<xref:str>* | *<xref:None>) – SASL
mechanism used for authentication. Allowed values:
*Gssapi*, *Plain*, *ScramSha256*, *ScramSha512*.
Default is *Plain*. Equivalent to *'sasl.mechanism'* in
librdkafka.

  * **protocol**
(<xref:azure.functions.decorators.kafka.BrokerProtocol>
* | *<xref:str>* | *<xref:None>) – Security protocol
used for broker communication. Default is plaintext.
Equivalent to *'security.protocol'* in librdkafka.

  * **linger_ms** (<xref:int>) – Time to wait between
sending batches of messages to allow for better
throughput via batching.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

kafka_trigger(arg_name: str, topic: str, broker_list: str, event_hub_connection_string: str | None = None, consumer_group: str | None = None, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None = None, o_auth_bearer_extensions: str | None = None, authentication_mode:

BrokerAuthenticationMode | str | None = 'NotSet', protocol: BrokerProtocol | str | None = 'NotSet', cardinality: Cardinality | str | None = 'One', lag_threshold: int = 1000, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *kafka_trigger* decorator adds <xref:agent_framework.azure.KafkaTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/kafkatrigger

Parameters: * **arg_name** (str) – The variable name used in the function code for the parameter that receives the Kafka event data.

```
  * **topic** (<xref:str>) – The topic monitored by the
trigger.

  * **broker_list** (<xref:str>) – The list of Kafka
brokers monitored by the trigger.

  * **event_hub_connection_string** (<xref:str>* | *
<xref:None>) – The name of an app setting that contains
the connection string for Event Hub (when using Kafka
protocol headers with Azure Event Hubs).

  * **consumer_group** (<xref:str>* | *<xref:None>) –
Kafka consumer group used by the trigger.

  * **avro_schema** (<xref:str>* | *<xref:None>) – Used
only if a generic Avro record should be generated.

  * **username** (<xref:str>* | *<xref:None>) – SASL
username for use with the PLAIN or SASL-SCRAM
mechanisms. Equivalent to 'sasl.username' in
librdkafka. Default is empty string.

  * **password** (<xref:str>* | *<xref:None>) – SASL
password for use with the PLAIN or SASL-SCRAM
mechanisms. Equivalent to 'sasl.password' in
librdkafka. Default is empty string.

  * **ssl_key_location** (<xref:str>* | *<xref:None>) –
```

```
Path to the client's private key (PEM) used for
authentication. Equivalent to 'ssl.key.location' in
librdkafka.

  * **ssl_ca_location** (<xref:str>* | *<xref:None>) –
Path to the CA certificate file for verifying the
broker's certificate. Equivalent to 'ssl.ca.location'
in librdkafka.

  * **ssl_certificate_location** (<xref:str>* | *
<xref:None>) – Path to the client's certificate.
Equivalent to 'ssl.certificate.location' in librdkafka.

  * **ssl_key_password** (<xref:str>* | *<xref:None>) –
Password for the client's certificate. Equivalent to
'ssl.key.password' in librdkafka.

  * **schema_registry_url** (<xref:str>* | *
<xref:None>) – URL of the Avro Schema Registry.

  * **schema_registry_username** (<xref:str>* | *
<xref:None>) – Username for the Schema Registry.

  * **schema_registry_password** (<xref:str>* | *
<xref:None>) – Password for the Schema Registry.

  * **o_auth_bearer_method**
(<xref:azure.functions.decorators.kafka.OAuthBearerMeth
od>* | *<xref:str>* | *<xref:None>) – Either 'default'
or 'oidc'. Equivalent to 'sasl.oauthbearer.method' in
librdkafka.

  * **o_auth_bearer_client_id** (<xref:str>* | *
<xref:None>) – Specify only if *o_auth_bearer_method*
is 'oidc'. Equivalent to 'sasl.oauthbearer.client.id'
in librdkafka.

  * **o_auth_bearer_client_secret** (<xref:str>* | *
<xref:None>) – Specify only if *o_auth_bearer_method*
is 'oidc'. Equivalent to
'sasl.oauthbearer.client.secret' in librdkafka.

  * **o_auth_bearer_scope** (<xref:str>* | *
<xref:None>) – Specify only if *o_auth_bearer_method*
is 'oidc'. Used to specify access scope. Equivalent to
'sasl.oauthbearer.scope'.

  * **o_auth_bearer_token_endpoint_url** (<xref:str>* |
*<xref:None>) – Specify only if *o_auth_bearer_method*
is 'oidc'. Equivalent to
'sasl.oauthbearer.token.endpoint.url' in librdkafka.

  * **o_auth_bearer_extensions** (<xref:str>* | *
<xref:None>) – Additional information for the broker,
in the form of a comma-separated list of key=value
```

```
pairs (e.g., "orgId=abc,flag=true"). Equivalent to
'sasl.oauthbearer.extensions' in librdkafka.

  * **authentication_mode**
(<xref:azure.functions.decorators.kafka.BrokerAuthentic
ationMode>* | *<xref:str>* | *<xref:None>) – SASL
mechanism to use. Allowed: Gssapi, Plain, ScramSha256,
ScramSha512. Default: Plain. Equivalent to
'sasl.mechanism'.

  * **protocol**
(<xref:azure.functions.decorators.kafka.BrokerProtocol>
* | *<xref:str>* | *<xref:None>) – Security protocol
used to communicate with brokers. Default: plaintext.
Equivalent to 'security.protocol'.

  * **lag_threshold** (<xref:int>) – Max number of
unprocessed messages per worker instance. Used in
scaling logic to estimate needed worker instances.
Default is 1000.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for extra
binding fields in the binding JSON.

  * **cardinality**
(<xref:azure.functions.decorators.core.Cardinality>* |
*<xref:str>* | *<xref:None>) –
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mcp_tool_trigger(arg_name: str, tool_name: str, description: str | None = None, tool_properties: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *mcp_tool_trigger* decorator adds <xref:agent_framework.azure.MCPToolTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/remote-mcp-functions-python

Parameters: * **arg_name** (str) – The name of the trigger parameter in the function code.

```
  * **tool_name** (<xref:str>) – The logical tool name
exposed to the host.

  * **description** (<xref:str>* | *<xref:None>) –
Optional human-readable description of the tool.

  * **tool_properties** (<xref:str>* | *<xref:None>) –
JSON-serialized tool properties/parameters list.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mysql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *mysql_input* decorator adds <xref:agent_framework.azure.MySqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> input object in the function code.

```
```

```
  * **command_text** (<xref:str>) – The SQL query
string or the name of the stored procedure to execute.

  * **connection_string_setting** (<xref:str>) – The
name of the app setting that contains the connection
string to the MySQL database.

  * **command_type** (<xref:str>* | *<xref:None>) – The
type of command being executed. Accepts *"Text"* for
SQL queries or *"StoredProcedure"* for stored
procedures.

  * **parameters** (<xref:str>* | *<xref:None>) – One
or more parameter values passed to the SQL command.
Should be provided as a comma-separated string in the
format: *"@param1=value1,@param2=value2"*.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** – Additional keyword arguments used to
define fields in the *function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mysql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *mysql_output* decorator adds <xref:agent_framework.azure.MySqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the MySQL output object in the function code.

```
  * **command_text** (<xref:str>) – The SQL command or
the name of the stored procedure executed by the
binding.

  * **connection_string_setting** (<xref:str>) – The
name of the app setting that contains the connection
string for the target MySQL database.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** – Additional keyword arguments used to
define fields in the *function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mysql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *mysql_trigger* decorator adds <xref:agent_framework.azure.MySqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> object in the function code.

```
  * **table_name** (<xref:str>) – The name of the MySQL
table monitored by the trigger.

  * **connection_string_setting** (<xref:str>) – The
name of an app setting that contains the connection
string for the database against which the query or
stored procedure is executed.

  * **leases_table_name** (<xref:str>* | *<xref:None>)
```

```
- The name of the table used to store leases. If not
specified, the default table name is
*Leases_{FunctionId}_{TableId}*.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) - Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

orchestration_trigger(context_name: str, orchestration: str | None = None)

Register an Orchestrator Function.

Parameters: * **context_name** (str) – Parameter name of the DurableOrchestrationContext object.

```

  * **orchestration**
(<xref:agent_framework.azure.Optional[str]>) - Name of
Orchestrator Function. The value is None by default, in
which case the name of the method is used.
```

queue_output(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *queue_output* decorator adds <xref:agent_framework.azure.QueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-queue

Parameters: * **arg_name** (str) – The name of the variable that represents the storage queue output object in the function code.

```
```

```
  * **queue_name** (<xref:str>) – The name of the queue
to which messages will be written.

  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Azure Queues.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

queue_trigger(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *queue_trigger* decorator adds <xref:agent_framework.azure.QueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-queue

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.QueueMessage> object in the function code.

```
  * **queue_name** (<xref:str>) – The name of the queue
to monitor.

  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Azure Queues.

  * **data_type**
```

```
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** – Additional keyword arguments to
include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

register_blueprint(function_container: DecoratorApi) -> None

Register a list of functions in the function app.

Parameters: **function_container** (DecoratorApi) – Instance extending <xref:agent_framework.azure.DecoratorApi>, which contains a list of functions to be registered.

Raises: TypeError – If the provided container is an instance of <xref:agent_framework.azure.FunctionRegister>.

Return type: None

register_functions(function_container: DecoratorApi) -> None

Register a list of functions in the function app.

Parameters: **function_container** (DecoratorApi) – Instance extending <xref:agent_framework.azure.DecoratorApi>, which contains a list of functions to be registered.

Raises: TypeError – If the provided container is an instance of <xref:agent_framework.azure.FunctionRegister>.

Return type: None

retry(strategy: str, max_retry_count: str, delay_interval: str | None = None, minimum_interval: str | None = None, maximum_interval: str | None = None, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]

The *retry* decorator adds a <xref:agent_framework.azure.RetryPolicy> to the function settings object.

Used for building a <xref:agent_framework.azure.Function> object in the worker function indexing model. This is equivalent to defining a RetryPolicy in *function.json*, which enables the function to automatically retry on failure.

All optional fields are assigned default values by the function host during parsing.

Ref: https://aka.ms/azure_functions_retries

Parameters: * **strategy** (str) – The retry strategy to use.

```
  * **max_retry_count** (<xref:str>) – The maximum
number of retry attempts.

  * **delay_interval** (<xref:str>* | *<xref:None>) –
The delay interval between retry attempts.

  * **minimum_interval** (<xref:str>* | *<xref:None>) –
The minimum delay interval between retry attempts.

  * **maximum_interval** (<xref:str>* | *<xref:None>) –
The maximum delay interval between retry attempts.

  * **setting_extra_fields** (<xref:typing.Dict>*[*
<xref:str>*, *<xref:agent_framework.exceptions.Any>*]
**| *<xref:None>) – Keyword arguments for specifying
additional setting fields.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

route(route: str | None = None, trigger_arg_name: str = 'req', binding_arg_name: str = '$return', methods: Iterable[str] | Iterable[HttpMethod] | None = None, auth_level: AuthLevel | str | None = None, trigger_extra_fields: Dict[str, Any] | None = None, binding_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]

The *route* decorator adds <xref:agent_framework.azure.HttpTrigger> and <xref:agent_framework.azure.HttpOutput> bindings to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-http

Parameters: * **route** (str* | *None) – Route for the HTTP endpoint. If None, it defaults to the function name (if present) or the user-defined Python function name.

```
```

```
  * **trigger_arg_name** (<xref:str>) - Argument name
for <xref:agent_framework.azure.HttpRequest>. Defaults
to *'req'*.

  * **binding_arg_name** (<xref:str>) - Argument name
for <xref:agent_framework.azure.HttpResponse>. Defaults
to *'$return'*.

  * **methods** (<xref:typing.Iterable>*[*<xref:str>*]
**| *<xref:typing.Iterable>*[*[HttpMethod]
(azure.functions.decorators.http.HttpMethod)*] **| *
<xref:None>) - A tuple of the HTTP methods to which the
function responds.

  * **auth_level**
(<xref:azure.functions.decorators.core.AuthLevel>* | *
<xref:str>* | *<xref:None>) - Determines what keys, if
any, need to be present on the request in order to
invoke the function.

  * **trigger_extra_fields** (<xref:typing.Dict>*[*
<xref:str>*, *<xref:agent_framework.exceptions.Any>*]
**| *<xref:None>) - Additional fields to include in the
trigger JSON. For example: >>> data_type='STRING'  #
results in 'dataType': 'STRING' in the trigger JSON

  * **binding_extra_fields** (<xref:typing.Dict>*[*
<xref:str>*, *<xref:agent_framework.exceptions.Any>*]
**| *<xref:None>) - Additional fields to include in the
binding JSON. For example: >>> data_type='STRING'  #
results in 'dataType': 'STRING' in the binding JSON
```

Returns: Decorator function.

Return type: Callable[[...], Any]

schedule(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-timer

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.

```
  * **schedule** (<xref:str>) – A string representing a
CRON expression used to schedule the function
execution.

  * **run_on_startup** (<xref:bool>* | *<xref:None>) –
If True, the function is invoked when the runtime
starts.

  * **use_monitor** (<xref:bool>* | *<xref:None>) – Set
to True or False to indicate whether the schedule
should be monitored.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: Callable[[...], Any]

semantic_search_input(arg_name: str, search_connection_name: str, collection: str, query: str | None = None, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, system_prompt: str | None = semantic_search_system_prompt, max_knowledge_count: int | None = 1, temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: https://platform.openai.com/docs/guides/embeddings

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **search_connection_name** (<xref:str>) – The name
of the app setting or environment variable that
contains the vector search connection value.

  * **collection** (<xref:str>) – The name of the
collection or table to search or store.

  * **query** (<xref:str>* | *<xref:None>) – The
semantic query text used for searching the database.

  * **ai_connection_name** (<xref:str>* | *<xref:None>)
–

    The name of the configuration section for AI
service
    connectivity settings.

    * For Azure OpenAI: If specified, looks for
"Endpoint" and/or "Key" in this configuration section.

    * If not specified or the section is missing,
defaults to the environment variables
*AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.

    * For user-assigned managed identity
authentication, this field is required.

    * For OpenAI (non-Azure), set the *OPENAI_API_KEY*
environment variable.

  * **embeddings_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>*
| *<xref:None>) – The deployment or model name for
OpenAI Embeddings. Defaults to "text-embedding-ada-
002".

  * **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>*
| *<xref:None>) – The deployment or model name for the
OpenAI Chat Completion API. Defaults to "gpt-3.5-
turbo".

  * **system_prompt** (<xref:str>* | *<xref:None>) –
Optional. The system prompt provided to the large
language model.

  * **max_knowledge_count** (<xref:int>* | *
<xref:None>) – Optional. The number of knowledge items
to inject into the system prompt. Default is 1.
```

```
  * **temperature** (<xref:str>* | *<xref:None>) – The
sampling temperature to use (range: 0 to 2). Higher
values like 0.8 yield more random output; lower values
like 0.2 make output more focused.

  * **top_p** (<xref:str>* | *<xref:None>) –
Alternative to temperature sampling (nucleus sampling).
For example, 0.1 considers only the top 10% of tokens
by probability mass. Use this or *temperature*.

  * **max_tokens** (<xref:str>* | *<xref:None>) – The
maximum number of tokens to generate in the completion.
The sum of prompt tokens and *max_tokens* must not
exceed the model's context length (usually 2048 or 4096
for newer models).

  * **is_reasoning_model** (<xref:bool>* | *
<xref:None>) – Indicates whether the chat model is a
reasoning model. If True, *max_tokens* and
*temperature* are not supported.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Optional. Defines how the
Functions runtime should interpret the parameter value.
Default is None.

  * **kwargs** – Additional keyword arguments for
specifying extra fields in the *function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_queue_output(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, **kwargs) -> Callable[[...], Any]

The *service_bus_queue_output* decorator adds <xref:agent_framework.azure.ServiceBusQueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the Service Bus queue output object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

  * **queue_name** (<xref:str>) – The name of the queue
to which messages will be sent.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>) – The access rights
required for the connection string.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_queue_trigger(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *on_service_bus_queue_change* decorator adds <xref:agent_framework.azure.ServiceBusQueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusQueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.

```
```

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

  * **queue_name** (<xref:str>) – The name of the queue
to monitor.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>) – Access rights for the
connection string.

  * **is_sessions_enabled** (<xref:bool>* | *
<xref:None>) – Set to True if connecting to a session-
aware queue or subscription.

  * **cardinality**
(<xref:azure.functions.decorators.core.Cardinality>* |
*<xref:str>* | *<xref:None>) – Set to "many" to enable
batching.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_topic_output(arg_name: str, connection: str, topic_name: str, subscription_name: str | None = None, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, **kwargs) -> Callable[[...], Any]

The *service_bus_topic_output* decorator adds <xref:agent_framework.azure.ServiceBusTopicOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusTopicOutput* in the *function.json*, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the Service Bus topic output object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

  * **topic_name** (<xref:str>) – The name of the topic
to which messages will be sent.

  * **subscription_name** (<xref:str>* | *<xref:None>)
– The name of the subscription (optional for output).

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value. Defaults to
DataType.UNDEFINED.

  * **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>) – The access rights
required for the connection string.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_topic_trigger(arg_name: str, connection: str, topic_name: str, subscription_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *on_service_bus_topic_change* decorator adds <xref:agent_framework.azure.ServiceBusTopicTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

  * **topic_name** (<xref:str>) – The name of the topic
to monitor.

  * **subscription_name** (<xref:str>) – The name of
the subscription to monitor.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>) – Access rights for the
connection string.

  * **is_sessions_enabled** (<xref:bool>* | *
<xref:None>) – Set to True if connecting to a session-
aware queue or subscription.

  * **cardinality**
(<xref:azure.functions.decorators.core.Cardinality>* |
*<xref:str>* | *<xref:None>) – Set to "many" to enable
batching.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: Callable[[...], Any]

sql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *sql_input* decorator adds <xref:agent_framework.azure.SqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the

function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> input object in the function code.

```
  * **command_text** (<xref:str>) - The Transact-SQL
query or the name of the stored procedure executed by
the binding.

  * **connection_string_setting** (<xref:str>) - The
name of the app setting that contains the connection
string to the target SQL database.

  * **command_type** (<xref:str>* | *<xref:None>) - The
command type. Use *"Text"* for a raw SQL query and
*"StoredProcedure"* for a stored procedure.

  * **parameters** (<xref:str>* | *<xref:None>) - A
comma-separated string of parameter assignments to pass
to the SQL command. Format:
*@param1=value1,@param2=value2*

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) - Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

sql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *sql_output* decorator adds <xref:agent_framework.azure.SqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the SQL output object in the function code.

```
  * **command_text** (<xref:str>) - The Transact-SQL
query or the name of the stored procedure executed by
the binding.

  * **connection_string_setting** (<xref:str>) - The
name of the app setting that contains the connection
string to the target SQL database.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) - Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

sql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *sql_trigger* decorator adds <xref:agent_framework.azure.SqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> object in the function code.

```
  * **table_name** (<xref:str>) - The name of the SQL
table monitored by the trigger.
```

```
  * **connection_string_setting** (<xref:str>) – The
name of an app setting that contains the connection
string for the database against which the query or
stored procedure is executed.

  * **leases_table_name** (<xref:str>* | *<xref:None>)
– The name of the table used to store leases. If not
specified, the default name is
*Leases_{FunctionId}_{TableId}*.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

table_input(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, take: int | None = None, filter: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]

The *table_input* decorator adds <xref:agent_framework.azure.TableInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/tablesbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the table or entity in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to the table service.

  * **table_name** (<xref:str>) – The name of the
```

```
table.

  * **row_key** (<xref:str>* | *<xref:None>) – The row
key of the table entity to read.

  * **partition_key** (<xref:str>* | *<xref:None>) –
The partition key of the table entity to read.

  * **take** (<xref:int>* | *<xref:None>) – The maximum
number of entities to return.

  * **filter** (<xref:str>* | *<xref:None>) – An OData
filter expression to apply when retrieving entities.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

table_output(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]

The *table_output* decorator adds <xref:agent_framework.azure.TableOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableOutput* in the *function.json*, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/tablesbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the table or entity in the function code.

```
  * **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to the table service.

  * **table_name** (<xref:str>) – The name of the
table.
```

```
  * **row_key** (<xref:str>* | *<xref:None>) – The row
key of the table entity to read.

  * **partition_key** (<xref:str>* | *<xref:None>) –
The partition key of the table entity to read.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

text_completion_input(arg_name: str, prompt: str, ai_connection_name: str | None = '', chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions

The examples typically define an HTTP-triggered function with a hardcoded prompt such as *"who is {name}?"*, where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **prompt** (<xref:str>) – The text prompt to
generate completions for.

  * **ai_connection_name** (<xref:str>* | *<xref:None>)
–

    The name of the configuration section that defines
    AI service connectivity settings.

    * For **Azure OpenAI**: If specified, it looks for
*"Endpoint"* and/or

    *"Key"* in this section. If not specified or
```

```
missing, it falls back to
    the environment variables *AZURE_OPENAI_ENDPOINT*
and *AZURE_OPENAI_KEY*.
    - For **user-assigned managed identity**, this
setting is required.
    - For **non-Azure OpenAI**, ensure the
*OPENAI_API_KEY* environment
    variable is set.

  * **model** – *(Deprecated)* Use *chat_model*
instead. This parameter is unused and will be removed
in future versions.

  * **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>*
| *<xref:None>) – The deployment or model name to use
for the Chat Completions API. Default is *"gpt-3.5-
turbo"*.

  * **temperature** (<xref:str>* | *<xref:None>) –
Sampling temperature (0–2). Higher values (e.g., 0.8)
produce more random output; lower values (e.g., 0.2)
make output more focused and deterministic.

  * **top_p** (<xref:str>* | *<xref:None>) – Controls
nucleus sampling. For example, *top_p=0.1* considers
only tokens in the top 10% cumulative probability.
Recommended to use either *temperature* or *top_p*, but
not both.

  * **max_tokens** (<xref:str>* | *<xref:None>) – The
maximum number of tokens to generate. The sum of prompt
tokens and *max_tokens* must not exceed the model's
context length (usually 2048 or 4096 for newer models).

  * **is_reasoning_model** (<xref:bool>* | *
<xref:None>) – If *True*, indicates that the configured
chat model is a reasoning model. In this case,
*max_tokens* and *temperature* are not supported.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

timer_trigger(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-timer

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.

```
  * **schedule** (<xref:str>) – A string representing a
CRON expression used to schedule the function
execution.

  * **run_on_startup** (<xref:bool>* | *<xref:None>) –
If True, the function is invoked when the runtime
starts.

  * **use_monitor** (<xref:bool>* | *<xref:None>) – Set
to True or False to indicate whether the schedule
should be monitored.

  * **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: Callable[[...], Any]

validate_function_names(functions: List[Function])

Validate that all functions have unique names within the app.

The *functions_bindings* dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.

	Parameters: **functions** (List[Function]) – A list of <xref:agent_framework.azure.Function> objects to validate. Raises: ValueError – If any function name is duplicated. warm_up_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable The *warm_up* decorator adds <xref:agent_framework.azure.WarmUpTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a *WarmUpTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-warmup Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in the function code. `* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.` Returns: Decorator function. Return type: Callable
AgentResponseCallbackProtocol	Protocol describing the callbacks invoked during agent execution.
AzureAIAgentClient	Azure AI Agent Chat client. Initialize an Azure AI Agent client.
AzureAIClient	Azure AI Agent client. Initialize an Azure AI Agent client.
AzureAISearchContextProvider	Azure AI Search Context Provider with hybrid search and semantic ranking. This provider retrieves relevant documents from Azure AI Search to provide context to the AI agent. It supports two modes:

	• **agentic**: Recommended for most scenarios. Uses Knowledge Bases for query planning and multi-hop reasoning. Slightly slower with more token consumption, but provides more accurate results (up to 36% improvement in response relevance). • **semantic** (default): Fast hybrid search combining vector and keyword search with semantic reranking. Best for simple queries where speed is critical. Initialize Azure AI Search Context Provider.
AzureAISearchSettings	Settings for Azure AI Search Context Provider with auto-loading from environment. The settings are first loaded from environment variables with the prefix '>>AZURE_SEARCH_<<'. If the environment variables are not found, the settings can be loaded from a .env file. Initialize the settings class. JSON { "title": "AzureAISearchSettings", "description": "Settings for Azure AI Search Context Provider with auto-loading from environment.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_SEARCH_'.\nIf the environment variables are not found, the settings can be loaded from a .env file.\n\nKeyword Args:\n endpoint: Azure AI Search endpoint URL.\n Can be set via environment variable AZURE_SEARCH_ENDPOINT.\n index_name: Name of the search index.\n Can be set via environment variable AZURE_SEARCH_INDEX_NAME.\n knowledge_base_name: Name of an existing Knowledge Base (for agentic mode).\n Can be set via environment variable AZURE_SEARCH_KNOWLEDGE_BASE_NAME.\n api_key: API key for authentication (optional, use managed identity if not provided).\n Can be set via environment variable AZURE_SEARCH_API_KEY.\n env_file_path: If provided, the .env settings are read from this file path location.\n env_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\nExamples:\n .. code-block:: python\n\n from agent_framework_aisearch import AzureAISearchSettings\n\n # Using environment variables\n # Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net\n # Set AZURE_SEARCH_INDEX_NAME=my-index\n settings = AzureAISearchSettings()\n\n # Or passing parameters directly\n settings = AzureAISearchSettings(\n

```
endpoint=\"https://mysearch.search.windows.net\",\n
index_name=\"my-index\",\n        )\n\n        # Or
loading from a .env file\n        settings =
AzureAISearchSettings(env_file_path=\"path/to/.env\")",
      "type": "object",
      "properties": {
         "env_file_path": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Env File Path"
         },
         "env_file_encoding": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
         },
         "endpoint": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Endpoint"
         },
         "index_name": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Index Name"
         },
         "knowledge_base_name": {
```

```
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Knowledge Base Name"
         },
         "api_key": {
            "anyOf": [
               {
                  "format": "password",
                  "type": "string",
                  "writeOnly": true
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Api Key"
         }
      }
   }
```


AzureAISettings

Azure AI Project settings.

The settings are first loaded from environment variables with the prefix '>>AZURE_AI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON


```
   {
      "title": "AzureAISettings",
      "description": "Azure AI Project settings.\n\nThe
settings are first loaded from environment variables
with the prefix 'AZURE_AI_'.\nIf the environment
variables are not found, the settings can be loaded
from a .env file\nwith the encoding 'utf-8'. If the
settings are not found in the .env file, the
settings\nare ignored; however, validation will fail
alerting that the settings are missing.\n\nKeyword
Args:\n    project_endpoint: The Azure AI Project
endpoint URL.\n        Can be set via environment
```

```
variable AZURE_AI_PROJECT_ENDPOINT.\n
model_deployment_name: The name of the model deployment
to use.\n        Can be set via environment variable
AZURE_AI_MODEL_DEPLOYMENT_NAME.\n    env_file_path: If
provided, the .env settings are read from this file
path location.\n    env_file_encoding: The encoding of
the .env file, defaults to 'utf-8'.\n\nExamples:\n
.. code-block:: python\n\n        from
agent_framework.azure import AzureAISettings\n\n
# Using environment variables\n        # Set
AZURE_AI_PROJECT_ENDPOINT=https://your-
project.cognitiveservices.azure.com\n        # Set
AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4\n        settings
= AzureAISettings()\n\n        # Or passing parameters
directly\n        settings = AzureAISettings(\n
project_endpoint=\"https://your-
project.cognitiveservices.azure.com\",
model_deployment_name=\"gpt-4\"\n        )\n\n        #
Or loading from a .env file\n        settings =
AzureAISettings(env_file_path=\"path/to/.env\")",
      "type": "object",
      "properties": {
         "env_file_path": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Env File Path"
         },
         "env_file_encoding": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
         },
         "project_endpoint": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
```

```
            "default": null,
            "title": "Project Endpoint"
         },
         "model_deployment_name": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Model Deployment Name"
         }
      }
   }
```


AzureOpenAIAssistantsClient	Azure OpenAI Assistants client. Initialize an Azure OpenAI Assistants client.
AzureOpenAIChatClient	Azure OpenAI Chat completion class. Initialize an Azure OpenAI Chat completion client.
AzureOpenAIResponsesClient	Azure Responses completion class. Initialize an Azure OpenAI Responses client.
AzureOpenAISettings	AzureOpenAI model settings. The settings are first loaded from environment variables with the prefix '>>AZURE_OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing. Initialize the settings class. JSON { "title": "AzureOpenAISettings", "description": "AzureOpenAI model settings.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_OPENAI_'.\nIf the environment variables are not found, the settings can be loaded from a .env file\nwith the encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare ignored; however, validation will fail alerting that the

```
settings are missing.\n\nKeyword Args:\n    endpoint:
The endpoint of the Azure deployment. This value\n
can be found in the Keys & Endpoint section when
examining\n        your resource from the Azure portal,
the endpoint should end in openai.azure.com.\n
If both base_url and endpoint are supplied, base_url
will be used.\n        Can be set via environment
variable AZURE_OPENAI_ENDPOINT.\n
chat_deployment_name: The name of the Azure Chat
deployment. This value\n        will correspond to the
custom name you chose for your deployment\n        when
you deployed a model. This value can be found under\n
Resource Management > Deployments in the Azure portal
or, alternatively,\n        under Management >
Deployments in Azure AI Foundry.\n        Can be set
via environment variable
AZURE_OPENAI_CHAT_DEPLOYMENT_NAME.\n
responses_deployment_name: The name of the Azure
Responses deployment. This value\n        will
correspond to the custom name you chose for your
deployment\n        when you deployed a model. This
value can be found under\n        Resource Management >
Deployments in the Azure portal or, alternatively,\n
under Management > Deployments in Azure AI Foundry.\n
Can be set via environment variable
AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME.\n    api_key:
The API key for the Azure deployment. This value can
be\n        found in the Keys & Endpoint section when
examining your resource in\n        the Azure portal.
You can use either KEY1 or KEY2.\n        Can be set
via environment variable AZURE_OPENAI_API_KEY.\n
api_version: The API version to use. The default value
is `default_api_version`.\n        Can be set via
environment variable AZURE_OPENAI_API_VERSION.\n
base_url: The url of the Azure deployment. This value\n
can be found in the Keys & Endpoint section when
examining\n        your resource from the Azure portal,
the base_url consists of the endpoint,\n
followed by /openai/deployments/{deployment_name}/,\n
use endpoint if you only want to supply the endpoint.\n
Can be set via environment variable
AZURE_OPENAI_BASE_URL.\n    token_endpoint: The token
endpoint to use to retrieve the authentication token.\n
The default value is `default_token_endpoint`.\n
Can be set via environment variable
AZURE_OPENAI_TOKEN_ENDPOINT.\n    default_api_version:
The default API version to use if not specified.\n
The default value is \"2024-10-21\".\n
default_token_endpoint: The default token endpoint to
use if not specified.\n        The default value is
\"https://cognitiveservices.azure.com/.default\".\n
env_file_path: The path to the .env file to load
settings from.\n    env_file_encoding: The encoding of
the .env file, defaults to 'utf-8'.\n\nExamples:\n
.. code-block:: python\n\n        from
```

```
agent_framework.azure import AzureOpenAISettings\n\n
# Using environment variables\n        # Set
AZURE_OPENAI_ENDPOINT=https://your-
endpoint.openai.azure.com\n        # Set
AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4\n        # Set
AZURE_OPENAI_API_KEY=your-key\n        settings =
AzureOpenAISettings()\n\n        # Or passing
parameters directly\n        settings =
AzureOpenAISettings(\n
endpoint=\"https://your-endpoint.openai.azure.com\",
chat_deployment_name=\"gpt-4\", api_key=\"your-key\"\n
)\n\n        # Or loading from a .env file\n
settings =
AzureOpenAISettings(env_file_path=\"path/to/.env\")",
      "type": "object",
      "properties": {
         "env_file_path": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Env File Path"
         },
         "env_file_encoding": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
         },
         "chat_deployment_name": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Chat Deployment Name"
         },
         "responses_deployment_name": {
            "anyOf": [
               {
```

```
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Responses Deployment Name"
    },
    "endpoint": {
        "anyOf": [
            {
                "format": "uri",
                "maxLength": 2083,
                "minLength": 1,
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Endpoint"
    },
    "base_url": {
        "anyOf": [
            {
                "format": "uri",
                "maxLength": 2083,
                "minLength": 1,
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Base Url"
    },
    "api_key": {
        "anyOf": [
            {
                "format": "password",
                "type": "string",
                "writeOnly": true
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Api Key"
    },
    "api_version": {
        "anyOf": [
```

```
                {
                   "type": "string"
                },
                {
                   "type": "null"
                }
             ],
             "default": null,
             "title": "Api Version"
          },
          "token_endpoint": {
             "anyOf": [
                {
                   "type": "string"
                },
                {
                   "type": "null"
                }
             ],
             "default": null,
             "title": "Token Endpoint"
          },
          "default_api_version": {
             "default": "2024-10-21",
             "title": "Default Api Version",
             "type": "string"
          },
          "default_token_endpoint": {
             "default":
"https://cognitiveservices.azure.com/.default",
             "title": "Default Token Endpoint",
             "type": "string"
          }
       }
    }
```


DurableAIAgent

A durable agent implementation that uses entity methods to interact with agent entities.

This class implements AgentProtocol and provides methods to work with Azure Durable Functions orchestrations, which use generators and yield instead of async/await.

Key methods:

- get_new_thread(): Create a new conversation thread
- run(): Execute the agent and return a Task for yielding in orchestrations

Note: The run() method is NOT async. It returns a Task directly that must be yielded in orchestrations to wait for the entity call to complete.

Example usage in orchestration: writer = app.get_agent(context, "WriterAgent") thread = writer.get_new_thread() # NOT yielded - returns

immediately

response = yield writer.run(# Yielded - waits for entity call
message="Write a haiku about coding", thread=thread

)

Initialize the DurableAIAgent.

Functions

get_entra_auth_token

Retrieve a Microsoft Entra Auth Token for a given token endpoint.

The token endpoint may be specified as an environment variable, via the .env file or as an argument. If the token endpoint is not provided, the default is None.

Python

```
get_entra_auth_token(credential: TokenCredential, token_endpoint: str, **kwargs: Any) -> str | None
```

Parameters

Expand table

Name	Description
credential Required*	<xref:TokenCredential> The Azure credential to use for authentication.
token_endpoint Required*	str The token endpoint to use to retrieve the authentication token.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional keyword arguments to pass to the token retrieval method.

Returns

Expand table

Type	Description
str | None	The Azure token or None if the token could not be retrieved.

AgentCallbackContext Class

Context supplied to callback invocations.

Constructor

Python

```
AgentCallbackContext()
```

Parameters

Expand table

Name	Description
agent_name Required*	
correlation_id Required*	
thread_id	Default value: None
request_message	Default value: None

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(agent_name: str, correlation_id: str, thread_id: str | None = None,
request_message: str | None = None) -> None
```

Parameters

Expand table

Name	Description
agent_name Required*	str
correlation_id Required*	str
thread_id	str | None Default value: None
request_message	str | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

Attributes

agent_name

```
Python
agent_name: str
```

correlation_id

```
Python
correlation_id: str
```

request_message

Python

```
request_message: str | None = None
```

thread_id

Python

```
thread_id: str | None = None
```

AgentFunctionApp Class

Main application class for creating durable agent function apps using Durable Entities.

This class uses Durable Entities pattern for agent execution, providing:

- Stateful agent conversations
- Conversation history management
- Signal-based operation invocation
- Better state management than orchestrations

Example:

Python

```
from agent_framework.azure import AgentFunctionApp, AzureOpenAIChatClient

# Create agents with unique names
weather_agent = AzureOpenAIChatClient(...).create_agent(
    name="WeatherAgent",
    instructions="You are a helpful weather agent.",
    tools=[get_weather],
)

math_agent = AzureOpenAIChatClient(...).create_agent(
    name="MathAgent",
    instructions="You are a helpful math assistant.",
    tools=[calculate],
)

# Option 1: Pass list of agents during initialization
app = AgentFunctionApp(agents=[weather_agent, math_agent])

# Option 2: Add agents after initialization
app = AgentFunctionApp()
app.add_agent(weather_agent)
app.add_agent(math_agent)


@app.orchestration_trigger(context_name="context")
def my_orchestration(context):
    writer = app.get_agent(context, "WeatherAgent")
    thread = writer.get_new_thread()
    forecast_task = writer.run("What's the forecast?", thread=thread)
    forecast = yield forecast_task
    return forecast
```

This creates:

- HTTP trigger endpoint for each agent's requests (if enabled)
- Durable entity for each agent's state management and execution
- Full access to all Azure Functions capabilities

agents

Dictionary of agent name to AgentProtocol instance

enable_health_check

Whether health check endpoint is enabled

Type: bool

enable_http_endpoints

Whether HTTP endpoints are created for agents

Type: bool

enable_mcp_tool_trigger

Whether MCP tool triggers are created for agents

Type: bool

max_poll_retries

Maximum polling attempts when waiting for responses

poll_interval_seconds

Delay (seconds) between polling attempts

Initialize the AgentFunctionApp.

param agents: List of agent instances to register.

param http_auth_level: HTTP authentication level (default: `func.AuthLevel.FUNCTION`).

param enable_health_check: Enable the built-in health check endpoint (default: `True`).

param enable_http_endpoints: Enable HTTP endpoints for agents (default: `True`).

param enable_mcp_tool_trigger: Enable MCP tool triggers for agents (default: `False`). When enabled, agents will be exposed as MCP tools that can be invoked by MCP-compatible clients.

param max_poll_retries: Maximum polling attempts when waiting for a response. Defaults to `DEFAULT_MAX_POLL_RETRIES`.

param poll_interval_seconds: Delay in seconds between polling attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`.

param default_callback: Optional callback invoked for agents without specific callbacks.

note: If no agents are provided, they can be added later using add_agent.

init(agents: list[AgentProtocol] | None = None, http_auth_level: AuthLevel = func.AuthLevel.FUNCTION, enable_health_check: bool = True, enable_http_endpoints: bool = True, max_poll_retries: int = DEFAULT_MAX_POLL_RETRIES, poll_interval_seconds: float = DEFAULT_POLL_INTERVAL_SECONDS, enable_mcp_tool_trigger: bool = False, default_callback: AgentResponseCallbackProtocol | None = None)

Initialize the AgentFunctionApp.

Parameters: * **agents** (list[AgentProtocol*] **| *None) – List of agent instances to register.

```
  * **http_auth_level** (<xref:azure.functions.decorators.core.AuthLevel>) – HTTP
authentication level (default: `func.AuthLevel.FUNCTION`).

  * **enable_health_check** (<xref:bool>) – Enable the built-in health check endpoint
(default: `True`).

  * **enable_http_endpoints** (<xref:bool>) – Enable HTTP endpoints for agents
(default: `True`).

  * **enable_mcp_tool_trigger** (<xref:bool>) – Enable MCP tool triggers for agents
(default: `False`). When enabled, agents will be exposed as MCP tools that can be
invoked by MCP-compatible clients.

  * **max_poll_retries** (<xref:int>) – Maximum polling attempts when waiting for a
response. Defaults to `DEFAULT_MAX_POLL_RETRIES`.

  * **poll_interval_seconds** (<xref:float>) – Delay in seconds between polling
attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`.

  * **default_callback**
(<xref:agent_framework_azurefunctions._callbacks.AgentResponseCallbackProtocol>* | *
<xref:None>) – Optional callback invoked for agents without specific callbacks.
```

Note: If no agents are provided, they can be added later using add_agent.

new(**kwargs)

activity_trigger(input_name: str, activity: str | None = None)

Register an Activity Function.

Parameters: * **input_name** (str) – Parameter name of the Activity input.

```
  * **activity** (<xref:agent_framework.azure.Optional[str]>) - Name of Activity
Function. The value is None by default, in which case the name of the method is used.
```

add_agent(agent: AgentProtocol, callback: AgentResponseCallbackProtocol | None = None, enable_http_endpoint: bool | None = None, enable_mcp_tool_trigger: bool | None = None) -> None

Add an agent to the function app after initialization.

Parameters: * **agent** (AgentProtocol) – The Microsoft Agent Framework agent instance (must implement AgentProtocol) The agent must have a 'name' attribute.

```
  * **callback**
(<xref:agent_framework_azurefunctions._callbacks.AgentResponseCallbackProtocol>* | *
<xref:None>) - Optional callback invoked during agent execution

  * **enable_http_endpoint** (<xref:bool>* | *<xref:None>) - Optional flag to
enable/disable HTTP endpoint for this agent. The app level enable_http_endpoints
setting will override this setting.

  * **enable_mcp_tool_trigger** (<xref:bool>* | *<xref:None>) - Optional flag to
enable/disable MCP tool trigger for this agent. The app level enable_mcp_tool_trigger
setting will override this setting.
```

Raises: ValueError – If the agent doesn't have a 'name' attribute or if an agent with the same name is already registered

Return type: None

property agents: dict[str, AgentProtocol]

Returns dict of agent names to agent instances.

Returns: Dictionary mapping agent names to their AgentProtocol instances.

property app_script_file: str

Name of function app script file in which all the functions are defined.

```
  Script file defined here is for placeholder purpose, please refer to
  worker defined script file path as the single point of truth.
```

Returns: Script file name.

assistant_create_output(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *assistantCreate* output binding creates a new assistant with a specified system prompt.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying extra binding fields to
include in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

assistant_post_input(arg_name: str, id: str, user_message: str, ai_connection_name: str | None = '', chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, chat_storage_connection_setting: str | None = 'AzureWebJobsStorage', collection_name: str | None = 'ChatState', temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **id** (<xref:str>) – The ID of the assistant to update.

  * **user_message** (<xref:str>) – The message entered by the user for the assistant
to respond to.

  * **ai_connection_name** (<xref:str>* | *<xref:None>) –

    The name of the configuration section for AI service
    connectivity.

    * **Azure OpenAI**: If specified, looks for "Endpoint" and/or "Key" in the
section.

    * If not specified or missing, falls back to the *AZURE_OPENAI_ENDPOINT* and
*AZURE_OPENAI_KEY* environment variables.

    * **Managed Identity**: Required for user-assigned identity auth.

    * **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY* environment variable.

  * **model** – *Deprecated.* Use *chat_model* instead. This parameter is unused and
will be removed in future versions.

  * **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The
deployment or model name of the OpenAI Chat Completion API. Default is *"gpt-3.5-
turbo"*.

  * **chat_storage_connection_setting** (<xref:str>* | *<xref:None>) – The config
section name for assistant chat table storage. Default is *"AzureWebJobsStorage"*.

  * **collection_name** (<xref:str>* | *<xref:None>) – The collection or table name
used for assistant chat storage. Default is *"ChatState"*.

  * **temperature** (<xref:str>* | *<xref:None>) – Sampling temperature, between 0 and
2. Higher values (e.g., 0.8) increase randomness; lower values (e.g., 0.2) make output
more deterministic.

  * **top_p** (<xref:str>* | *<xref:None>) – Alternative to temperature sampling. Uses
nucleus sampling to consider only the tokens comprising the top *top_p* probability
mass. Recommended to use either this or *temperature*.

  * **max_tokens** (<xref:str>* | *<xref:None>) – Maximum tokens to generate. Total
tokens (prompt + output) must not exceed the model's context length.

  * **is_reasoning_model** (<xref:bool>* | *<xref:None>) – Whether the chat model is a
reasoning model. If true, *max_tokens* and *temperature* are not supported.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying extra binding fields in
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

assistant_query_input(arg_name: str, id: str, timestamp_utc: str, chat_storage_connection_setting: str | None = 'AzureWebJobsStorage', collection_name: str | None = 'ChatState', data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *assistantQuery* input binding retrieves assistant chat history and passes it to the function.

This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **timestamp_utc** (<xref:str>) – The earliest timestamp (in UTC) for the messages
to retrieve from the chat history. Must be in ISO 8601 format, e.g., *"2023-08-
01T00:00:00Z"*.

  * **chat_storage_connection_setting** (<xref:str>* | *<xref:None>) – The name of the
configuration section containing the connection settings for assistant chat storage.
Defaults to *"AzureWebJobsStorage"*.

  * **collection_name** (<xref:str>* | *<xref:None>) – The name of the table or
collection used for assistant chat storage. Defaults to *"ChatState"*.

  * **id** (<xref:str>) – The unique identifier of the assistant whose history is
being queried.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

assistant_skill_trigger(arg_name: str, function_description: str, function_name: str | None = None, parameter_description_json: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions

to invoke and when.

Ref: https://platform.openai.com/docs/guides/function-calling

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

Parameters: * **arg_name** (str) – The name of the trigger parameter in the function code.

```
  * **function_description** (<xref:str>) – A description of the assistant function,
which is provided to the model.

  * **function_name** (<xref:str>* | *<xref:None>) – The name of the assistant
function, which is passed to the language model.

  * **parameter_description_json** (<xref:str>* | *<xref:None>) – A JSON-formatted
description of the function parameters, provided to the model. If omitted, the
description is autogenerated.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) – Additional keyword arguments
for specifying binding fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

property auth_level: AuthLevel

Authorization level of the function app. Will be applied to the http trigger functions which do not have authorization level specified.

Returns: Authorization level of the function app.

blob_input(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *blob_input* decorator adds <xref:agent_framework.azure.BlobInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Parameters: * **arg_name** (str) – The name of the variable that represents the blob in the function code.

```
  * **path** (<xref:str>) - The path to the blob.

  * **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Azure Blobs.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

blob_output(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *blob_output* decorator adds <xref:agent_framework.azure.BlobOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Parameters: * **arg_name** (str) – The name of the variable that represents the blob in the function code.

```
  * **path** (<xref:str>) - The path to the blob.

  * **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Azure Blobs.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

blob_trigger(arg_name: str, path: str, connection: str, source: BlobSource | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The blob_change_trigger decorator adds <xref:agent_framework.azure.BlobTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building <xref:agent_framework.azure.Function> object used in worker function indexing model. This is equivalent to defining BlobTrigger in the function.json which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.InputStream> object in function code.

```
  * **path** (<xref:str>) - The path to the blob.

  * **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Azure Blobs.

  * **source** (<xref:azure.functions.decorators.core.BlobSource>* | *<xref:None>) -
Sets the source of the triggering event. Use "EventGrid" for an Event Grid-based blob
trigger, which provides much lower latency. The default is "LogsAndContainerScan",
which uses the standard polling mechanism to detect changes in the container.
```

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how Functions runtime should treat the parameter value.

  * **kwargs** – Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_input(arg_name: str, connection: str, database_name: str, container_name: str, partition_key: str | None = None, id: str | None = None, sql_query: str | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_input* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-cosmosdb-v4 Additional details: https://aka.ms/cosmosdb-v4-update

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting container that
specifies how to connect to the Azure Cosmos DB account being monitored.

  * **database_name** (<xref:str>) – The name of the Cosmos DB database that contains
the document.

  * **container_name** (<xref:str>) – The name of the container that holds the
document.

  * **partition_key** (<xref:str>* | *<xref:None>) – The partition key value for the
document lookup.

  * **id** (<xref:str>* | *<xref:None>) – The ID of the document to retrieve.
```

```
  * **sql_query** (<xref:str>* | *<xref:None>) – An Azure Cosmos DB SQL query used to
retrieve multiple documents.

  * **preferred_locations** (<xref:str>* | *<xref:None>) – (Optional) Preferred geo-
replicated regions. Values should be comma-separated, e.g., "East US,South Central
US,North Europe".

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_input_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, id: str | None = None, sql_query: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_input_v3* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.

```
  * **database_name** (<xref:str>) – The name of the Azure Cosmos DB database
containing the target collection.

  * **collection_name** (<xref:str>) – The name of the collection that contains the
document(s).

  * **connection_string_setting** (<xref:str>) – The name of the app setting that
contains the Azure Cosmos DB connection string.

  * **id** (<xref:str>* | *<xref:None>) – The ID of a single document to retrieve.
```

```
  * **sql_query** (<xref:str>* | *<xref:None>) – An Azure Cosmos DB SQL query used to
retrieve multiple documents.

  * **partition_key** (<xref:str>* | *<xref:None>) – Specifies the partition key value
for the lookup.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_output(arg_name: str, connection: str, database_name: str, container_name: str, create_if_not_exists: bool | None = None, partition_key: str | None = None, container_throughput: int | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_output* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4

Parameters: * **arg_name** (str) – The name of the variable that represents the Cosmos DB output object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to the Azure Cosmos DB account.

  * **database_name** (<xref:str>) – The name of the Azure Cosmos DB database
containing the target container.

  * **container_name** (<xref:str>) – The name of the container to which documents are
written.

  * **create_if_not_exists** (<xref:bool>* | *<xref:None>) – A boolean indicating
```

```
whether the container should be created automatically if it does not exist.

  * **partition_key** (<xref:str>* | *<xref:None>) – When *create_if_not_exists* is
*True*, this defines the partition key path for the created container.

  * **container_throughput** (<xref:int>* | *<xref:None>) – When
*create_if_not_exists* is *True*, this defines the throughput (Request Units) for the
created container.

  * **preferred_locations** (<xref:str>* | *<xref:None>) – Preferred geographic
regions for geo-replicated database accounts in Azure Cosmos DB. When set along with
*use_multiple_write_locations*, it enables multi-region writes.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_output_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, create_if_not_exists: bool | None = None, partition_key: str | None = None, collection_throughput: int | None = None, use_multiple_write_locations: bool | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *cosmos_db_output_v3* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Parameters: * **arg_name** (str) – The name of the variable that represents the Cosmos DB output object in the function code.

```
  * **database_name** (<xref:str>) – The name of the Azure Cosmos DB database
containing the monitored collection.
```

```
  * **collection_name** (<xref:str>) – The name of the collection to which documents
are written.

  * **connection_string_setting** (<xref:str>) – The name of an app setting or setting
collection that specifies how to connect to the Azure Cosmos DB account.

  * **create_if_not_exists** (<xref:bool>* | *<xref:None>) – A boolean indicating
whether the collection should be created automatically if it does not exist.

  * **partition_key** (<xref:str>* | *<xref:None>) – When *create_if_not_exists* is
*True*, this defines the partition key path for the created collection.

  * **collection_throughput** (<xref:int>* | *<xref:None>) – When
*create_if_not_exists* is *True*, this defines the throughput (Request Units) for the
created collection.

  * **use_multiple_write_locations** (<xref:bool>* | *<xref:None>) – When set to
*True* and used in conjunction with *preferred_locations*, this enables multi-region
writes in the Azure Cosmos DB service.

  * **preferred_locations** (<xref:str>* | *<xref:None>) – Preferred geographic
regions for geo-replicated database accounts in Azure Cosmos DB.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_trigger(arg_name: str, connection: str, database_name: str, container_name: str, lease_connection: str | None = None, lease_database_name: str | None = None, lease_container_name: str | None = None, create_lease_container_if_not_exists: bool | None = None, leases_container_throughput: int | None = None, lease_container_prefix: str | None = None, feed_poll_delay: int | None = None, lease_acquire_interval: int | None = None, lease_expiration_interval: int | None = None, lease_renew_interval: int | None = None, max_items_per_invocation: int | None = None, start_from_beginning: time | None = None, start_from_time: time | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *cosmos_db_trigger* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to the monitored Azure Cosmos DB account.

  * **database_name** (<xref:str>) – The name of the Azure Cosmos DB database
containing the monitored collection.

  * **container_name** (<xref:str>) – The name of the container being monitored.

  * **lease_connection** (<xref:str>* | *<xref:None>) – (Optional) The name of an app
setting or setting collection that specifies how to connect to the Cosmos DB account
that holds the lease container.

  * **lease_database_name** (<xref:str>* | *<xref:None>) – The name of the database
that holds the collection used to store leases.

  * **lease_container_name** (<xref:str>* | *<xref:None>) – (Optional) The name of the
container used to store leases. If not set, the default value "leases" is used.

  * **create_lease_container_if_not_exists** (<xref:bool>* | *<xref:None>) –
(Optional) If *True*, the leases container is created automatically if it does not
exist. Defaults to *False*. Note: When using Azure AD identities, container creation
is not allowed, and the function will not start if this is *True*.

  * **leases_container_throughput** (<xref:int>* | *<xref:None>) – (Optional) The
number of Request Units (RUs) to assign when the leases container is created. This is
used only when *create_lease_container_if_not_exists* is *True*. It is automatically
set when configured through the Azure Portal.

  * **lease_container_prefix** (<xref:str>* | *<xref:None>) – (Optional) A prefix
added to leases created in the lease container for this function. Use this to allow
multiple functions to share the same lease container with different prefixes.

  * **feed_poll_delay** (<xref:int>* | *<xref:None>) – The delay (in milliseconds)
between polling a partition for new changes after draining current changes.

  * **lease_acquire_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) to trigger a task to check if partitions are evenly distributed across
host instances.
```

```
  * **lease_expiration_interval** (<xref:int>* | *<xref:None>) - The interval (in
milliseconds) for which a lease is held for a partition.

  * **lease_renew_interval** (<xref:int>* | *<xref:None>) - The interval (in
milliseconds) to renew all leases for partitions currently held by an instance.

  * **max_items_per_invocation** (<xref:int>* | *<xref:None>) - Maximum number of
items received per function call.

  * **start_from_beginning** (<xref:datetime.time>* | *<xref:None>) - If *True*, the
trigger starts reading changes from the beginning of the collection's change history
instead of the current time.

  * **start_from_time** (<xref:datetime.time>* | *<xref:None>) - (Optional) The date
and time from which to begin reading the change feed. Use ISO 8601 format with a UTC
designator, e.g., *2021-02-16T14:19:29Z*. Only used to set the initial trigger state;
has no effect once the trigger has a lease state.

  * **preferred_locations** (<xref:str>* | *<xref:None>) - Preferred locations
(regions) for geo-replicated Cosmos DB accounts.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) - Additional keyword arguments
for specifying binding fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

cosmos_db_trigger_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, lease_collection_name: str | None = None, lease_connection_string_setting: str | None = None, lease_database_name: str | None = None, create_lease_collection_if_not_exists: bool | None = None, leases_collection_throughput: int | None = None, lease_collection_prefix: str | None = None, checkpoint_interval: int | None = None, checkpoint_document_count: int | None = None, feed_poll_delay: int | None = None, lease_renew_interval: int | None = None, lease_acquire_interval: int | None = None, lease_expiration_interval: int | None = None, max_items_per_invocation: int | None = None, start_from_beginning: bool | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *cosmos_db_trigger_v3* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed.

See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.

```
  * **database_name** (<xref:str>) – The name of the Azure Cosmos DB database
containing the monitored collection.

  * **collection_name** (<xref:str>) – The name of the collection being monitored.

  * **connection_string_setting** (<xref:str>) – The name of an app setting or setting
collection specifying how to connect to the monitored Azure Cosmos DB account.

  * **lease_collection_name** (<xref:str>* | *<xref:None>) – The name of the
collection used to store leases.

  * **lease_connection_string_setting** (<xref:str>* | *<xref:None>) – The name of an
app setting or setting collection that specifies how to connect to the Azure Cosmos DB
account that holds the lease collection.

  * **lease_database_name** (<xref:str>* | *<xref:None>) – The name of the database
that holds the lease collection.

  * **create_lease_collection_if_not_exists** (<xref:bool>* | *<xref:None>) – If
*True*, the lease collection is automatically created when it does not already exist.

  * **leases_collection_throughput** (<xref:int>* | *<xref:None>) – The number of
Request Units (RUs) assigned when the lease collection is created.

  * **lease_collection_prefix** (<xref:str>* | *<xref:None>) – A prefix added to
leases created in the lease collection for this Function.

  * **checkpoint_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) between lease checkpoints. The default behavior is to checkpoint after
every function call.

  * **checkpoint_document_count** (<xref:int>* | *<xref:None>) – Number of documents
processed between lease checkpoints. Default is after every function call.

  * **feed_poll_delay** (<xref:int>* | *<xref:None>) – The delay (in milliseconds)
between polling a partition for new changes after all current changes are drained.

  * **lease_renew_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) to renew leases for partitions currently held by an instance.
```

```
  * **lease_acquire_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) to trigger a task that checks whether partitions are evenly distributed
among host instances.

  * **lease_expiration_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) for which a lease is held for a partition.

  * **max_items_per_invocation** (<xref:int>* | *<xref:None>) – Maximum number of
items received per function call.

  * **start_from_beginning** (<xref:bool>* | *<xref:None>) – If *True*, the trigger
starts reading changes from the beginning of the collection's change history rather
than from the current time.

  * **preferred_locations** (<xref:str>* | *<xref:None>) – Preferred locations
(regions) for geo-replicated database accounts in Azure Cosmos DB.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) – Additional keyword arguments
for specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_binding_output(arg_name: str, binding_name: str, operation: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_binding_output* decorator adds <xref:agent_framework.azure.DaprBindingOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-binding-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **binding_name** (<xref:str>) – The configured name of the binding.
```

```
  * **operation** (<xref:str>) – The configured operation.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_binding_trigger(arg_name: str, binding_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *dapr_binding_trigger* decorator adds <xref:agent_framework.azure.DaprBindingTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the trigger input in the function code.

```
  * **binding_name** (<xref:str>) – The name of the Dapr trigger. If not specified,
the function name is used as the trigger name.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) – Keyword arguments for
specifying additional binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_invoke_output(arg_name: str, app_id: str, method_name: str, http_verb: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_invoke_output* decorator adds <xref:agent_framework.azure.DaprInvokeOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprInvokeOutput* in the *function.json*, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-invoke-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **app_id** (<xref:str>) - The Dapr app ID to invoke.

  * **method_name** (<xref:str>) - The method name of the app to invoke.

  * **http_verb** (<xref:str>) - The HTTP verb to use for the invocation (e.g., GET,
POST).

  * **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_publish_output(arg_name: str, pub_sub_name: str, topic: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_publish_output* decorator adds <xref:agent_framework.azure.DaprPublishOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-publish-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **pub_sub_name** (<xref:str>) – The pub/sub component name to publish to.

  * **topic** (<xref:str>) – The name of the topic to publish to.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_secret_input(arg_name: str, secret_store_name: str, key: str, metadata: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_secret_input* decorator adds <xref:agent_framework.azure.DaprSecretInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-secret-input-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state input object in the function code.

```
  * **secret_store_name** (<xref:str>) – The name of the secret store from which to
retrieve the secret.

  * **key** (<xref:str>) – The key identifying the name of the secret to retrieve.

  * **metadata** (<xref:str>) – Metadata properties in the form
"key1=value1&key2=value2".

  * **dapr_address** (<xref:str>* | *<xref:None>) – The Dapr address. Optional;
Defaults to [http://localhost](http://localhost):{daprHttpPort}.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_service_invocation_trigger(arg_name: str, method_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *dapr_service_invocation_trigger* decorator adds <xref:agent_framework.azure.DaprServiceInvocationTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-service-invocation

Parameters: * **arg_name** (str) – The name of the variable that represents the service invocation input in the function code.

```
  * **method_name** (<xref:str>) – The name of the method on a remote Dapr App. If not
specified, the name of the function is used as the method name.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

```
  * **kwargs** (<xref:agent_framework.exceptions.Any>) – Keyword arguments for
specifying additional binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_state_input(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_state_input* decorator adds <xref:agent_framework.azure.DaprStateInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateInput* in the *function.json*, which enables the function to read state from the underlying state store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-state-input-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state input object in the function code.

```
  * **state_store** (<xref:str>) – The state store containing the state.

  * **key** (<xref:str>) – The name of the key to retrieve.

  * **dapr_address** (<xref:str>* | *<xref:None>) – The Dapr address. Optional;
Defaults to [http://localhost](http://localhost):{daprHttpPort}.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_state_output(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *dapr_state_output* decorator adds <xref:agent_framework.azure.DaprStateOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-state-output-binding

Parameters: * **arg_name** (str) – The name of the variable that represents the Dapr state output object in the function code.

```
  * **state_store** (<xref:str>) - The state store containing the state for the
specified keys.

  * **key** (<xref:str>) - The name of the key used to store the state.

  * **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

dapr_topic_trigger(arg_name: str, pub_sub_name: str, topic: str, route: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *dapr_topic_trigger* decorator adds <xref:agent_framework.azure.DaprTopicTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-topic

Parameters: * **arg_name** (str) – The name of the variable that represents the trigger input in the function code.

```
  * **pub_sub_name** (<xref:str>) – The name of the Dapr pub/sub component.

  * **topic** (<xref:str>) – The topic name. If unspecified, the function name is
used.

  * **route** (<xref:str>* | *<xref:None>) – The route for the trigger. If
unspecified, the topic name is used.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) – Additional keyword arguments
for specifying extra binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

durable_client_input(client_name: str, task_hub: str | None = None, connection_name: str | None = None)

Register a Durable-client Function.

Parameters: * **client_name** (str) – Parameter name of durable client.

```
  * **task_hub** (<xref:agent_framework.azure.Optional[str]>) – Used in scenarios
where multiple function apps share the same storage account but need to be isolated
from each other. If not specified, the default value from host.json is used. This
value must match the value used by the target orchestrator functions.

  * **connection_name** (<xref:agent_framework.azure.Optional[str]>) – The name of an
app setting that contains a storage account connection string. The storage account
represented by this connection string must be the same one used by the target
orchestrator functions. If not specified, the default storage account connection
string for the function app is used.
```

durable_openai_agent_orchestrator(_func=None, *, model_provider=None, model_retry_options: RetryOptions | None = RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5))

Decorate Azure Durable Functions orchestrators that use OpenAI Agents.

Parameters: * **model_provider** (<xref:agent_framework.azure.Optional[ModelProvider]>) – Use a non-default ModelProvider instead of the default OpenAIProvider, such as when testing.

```
  * **model_retry_options**
(<xref:azure.durable_functions.models.RetryOptions.RetryOptions>* | *<xref:None>) –
```

embeddings_input(arg_name: str, input: str, input_type: InputType, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The embeddings input decorator generates embeddings used to measure the relatedness of text strings.

Ref: https://platform.openai.com/docs/guides/embeddings

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **input** (<xref:str>) – The input source containing the data to generate
embeddings for.

  * **input_type** (<xref:azure.functions.decorators.openai.InputType>) – The type of
the input (e.g., string, list, file reference).

  * **ai_connection_name** (<xref:str>* | *<xref:None>) –

    The name of the configuration section for AI service
    connectivity settings.

    * **Azure OpenAI**: If specified, looks for "Endpoint" and/or "Key" in this
section.

    * If not specified or not found, falls back to the environment variables
*AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.

    * For user-assigned managed identity authentication, this is required.

    * **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY* environment variable.

  * **model** – *(Deprecated)* Use *embeddings_model* instead. This parameter is
unused and will be removed in future versions.

  * **embeddings_model** (<xref:str>* | *
```

```
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The
deployment or model name for OpenAI Embeddings. Default is *"text-embedding-ada-002"*.

  * **max_chunk_length** (<xref:int>* | *<xref:None>) – The maximum number of
characters into which the input should be chunked. Default is *8 * 1024*.

  * **max_overlap** (<xref:int>* | *<xref:None>) – The maximum number of characters to
overlap between input chunks. Default is *128*.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Optional. Defines how the Functions runtime should interpret the
parameter value.

  * **kwargs** – Additional keyword arguments to include in the *function.json*
binding configuration.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

embeddings_store_output(arg_name: str, input: str, input_type: InputType, store_connection_name: str, collection: str, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

Add an embeddings input binding to the function.

The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions:

- Azure AI Search
- Azure Data Explorer
- Azure Cosmos DB using MongoDB

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **input** (<xref:str>) – The input data for which embeddings should be generated.

  * **input_type** (<xref:azure.functions.decorators.openai.InputType>) – The type of
the input (e.g., string, list).

  * **store_connection_name** (<xref:str>) – The name of an app setting or environment
variable containing the vector store connection value.
```

```
  * **collection** (<xref:str>) – The collection or table to search within.

  * **ai_connection_name** (<xref:str>* | *<xref:None>) –

    The name of the configuration section for AI service
    connectivity settings.

    * For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this
configuration section.

    * If not specified, or if the section is missing, falls back to environment
variables: *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.

    * For user-assigned managed identity, this property is required.

    * For OpenAI (non-Azure), set the *OPENAI_API_KEY* environment variable.

  * **model** – Deprecated. Use *embeddings_model* instead. This parameter is unused
and will be removed in future versions.

  * **embeddings_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The
deployment or model name for OpenAI Embeddings. The default is "text-embedding-ada-
002".

  * **max_chunk_length** (<xref:int>* | *<xref:None>) – The maximum number of
characters to chunk the input into.

  * **max_overlap** (<xref:int>* | *<xref:None>) – The maximum number of characters to
overlap between chunks.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Optional. Defines how the Functions runtime should treat the parameter
value. Defaults to None.

  * **kwargs** – Additional keyword arguments to include in the *function.json*
binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

enable_health_check: bool

enable_http_endpoints: bool

enable_mcp_tool_trigger: bool

entity_trigger(context_name: str, entity_name: str | None = None)

Register an Entity Function.

Parameters: * **context_name** (str) – Parameter name of the Entity input.

```
  * **entity_name** (<xref:agent_framework.azure.Optional[str]>) – Name of Entity
Function. The value is None by default, in which case the name of the method is used.
```

event_grid_output(arg_name: str, topic_endpoint_uri: str | None = None, topic_key_setting: str | None = None, connection: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *event_grid_output* decorator adds <xref:agent_framework.azure.EventGridOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining the output binding in the *function.json*, which enables the function to write events to a custom topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/eventgridtrigger

Parameters: * **arg_name** (str) – The variable name used in the function code that represents the event.

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **topic_endpoint_uri** (<xref:str>* | *<xref:None>) – The name of an app setting
that contains the URI for the custom topic.

  * **topic_key_setting** (<xref:str>* | *<xref:None>) – The name of an app setting
that contains an access key for the custom topic.

  * **connection** (<xref:str>* | *<xref:None>) – The value of the common prefix for
the setting that contains the topic endpoint URI.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

event_grid_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *event_grid_trigger* decorator adds <xref:agent_framework.azure.EventGridTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/eventgridtrigger

Parameters: * **arg_name** (str) – The variable name used in the function code for the parameter that receives the event data.

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

event_hub_message_trigger(arg_name: str, connection: str, event_hub_name: str, data_type: DataType | str | None = None, cardinality: Cardinality | str | None = None, consumer_group: str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *event_hub_message_trigger* decorator adds <xref:agent_framework.azure.EventHubTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-event-hubs

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.EventHubEvent> object in the function code.

```
```

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Event Hubs.

  * **event_hub_name** (<xref:str>) – The name of the Event Hub.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) – Set to "many" to enable batching.

  * **consumer_group** (<xref:str>* | *<xref:None>) – Optional. The consumer group
used to subscribe to events in the hub.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) – Additional keyword arguments
to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

event_hub_output(arg_name: str, connection: str, event_hub_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *event_hub_output* decorator adds <xref:agent_framework.azure.EventHubOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-event-hubs

Parameters: * **arg_name** (str) – The name of the variable representing the Event Hub output object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Event Hub.

  * **event_hub_name** (<xref:str>) – The name of the Event Hub to send messages to.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

```
  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

function_name(name: str, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]

Optional: Sets name of the <xref:agent_framework.azure.Function> object. If not set, it will default to the name of the method name.

Parameters: * **name** (str) – Name of the function.

```

  * **setting_extra_fields** (<xref:typing.Dict>*[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>) – Keyword arguments for
specifying
```

Return type: Callable[[...], Any]

additional setting fields :return: Decorator function.

generic_input_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *generic_input_binding* decorator adds <xref:agent_framework.azure.GenericInputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-custom

Parameters: * **arg_name** (str) – The name of the input parameter in the function code.

```

  * **type** (<xref:str>) – The type of the binding.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

```
  * **kwargs** – Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

generic_output_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *generic_output_binding* decorator adds <xref:agent_framework.azure.GenericOutputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-custom

Parameters: * **arg_name** (str) – The name of the output parameter in the function code.

```

  * **type** (<xref:str>) – The type of the binding.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

generic_trigger(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *generic_trigger* decorator adds <xref:agent_framework.azure.GenericTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-custom

Parameters: * **arg_name** (str) – The name of the trigger parameter in the function code.

```
  * **type** (<xref:str>) – The type of the binding.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

get_agent(context: Any, agent_name: str) -> <xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>

Return a DurableAIAgent proxy for a registered agent.

Parameters: * **context** (Any) – Durable Functions orchestration context invoking the agent.

```
  * **agent_name** (<xref:str>) – Name of the agent registered on this app.
```

Raises: ValueError – If the requested agent has not been registered.

Returns: DurableAIAgent wrapper bound to the orchestration context.

Return type: <xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>

get_functions() -> List[Function]

Get the function objects in the function app.

Returns: A list of <xref:agent_framework.azure.Function> objects defined in the app.

Return type: List[Function]

http_type(http_type: str) -> Callable[[...], Any]

Set http type of the <xref:agent_framework.azure.Function> object.

Parameters: **http_type** (str) – Http type of the function.

Returns: Decorator function.

Return type: Callable[[...], Any]

kafka_output(arg_name: str, topic: str, broker_list: str, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None = None, o_auth_bearer_extensions: str | None = None, max_message_bytes: int = 1_000_000, batch_size: int = 10_000, enable_idempotence: bool = False, message_timeout_ms: int = 300_000, request_timeout_ms: int = 5_000, max_retries: int = 2_147_483_647, authentication_mode: BrokerAuthenticationMode | str | None = 'NOTSET', protocol: BrokerProtocol | str | None = 'NOTSET', linger_ms: int = 5, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *kafka_output* decorator adds <xref:agent_framework.azure.KafkaOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a Kafka output binding in *function.json*, which enables the function to write events to a Kafka topic. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/kafkaoutput

Parameters: * **arg_name** (str) – The variable name used in the function code that represents the output event.

```
* **topic** (<xref:str>) – The Kafka topic to which messages are published.
```

```
  * **broker_list** (<xref:str>) – The list of Kafka brokers to which the producer
connects.

  * **avro_schema** (<xref:str>* | *<xref:None>) – Optional. Avro schema to generate a
generic record.

  * **username** (<xref:str>* | *<xref:None>) – SASL username for use with the PLAIN
and SASL-SCRAM mechanisms. Equivalent to *'sasl.username'* in librdkafka.

  * **password** (<xref:str>* | *<xref:None>) – SASL password for use with the PLAIN
and SASL-SCRAM mechanisms. Equivalent to *'sasl.password'* in librdkafka.

  * **ssl_key_location** (<xref:str>* | *<xref:None>) – Path to the client's private
key (PEM) for authentication. Equivalent to *'ssl.key.location'* in librdkafka.

  * **ssl_ca_location** (<xref:str>* | *<xref:None>) – Path to the CA certificate for
verifying the broker's certificate. Equivalent to *'ssl.ca.location'* in librdkafka.

  * **ssl_certificate_location** (<xref:str>* | *<xref:None>) – Path to the client's
certificate file. Equivalent to *'ssl.certificate.location'* in librdkafka.

  * **ssl_key_password** (<xref:str>* | *<xref:None>) – Password for the client's SSL
key. Equivalent to *'ssl.key.password'* in librdkafka.

  * **schema_registry_url** (<xref:str>* | *<xref:None>) – URL of the Avro Schema
Registry.

  * **schema_registry_username** (<xref:str>* | *<xref:None>) – Username for accessing
the Schema Registry.

  * **schema_registry_password** (<xref:str>* | *<xref:None>) – Password for accessing
the Schema Registry.

  * **o_auth_bearer_method**
(<xref:azure.functions.decorators.kafka.OAuthBearerMethod>* | *<xref:str>* | *
<xref:None>) – OAuth bearer method to use, e.g., *'default'* or *'oidc'*. Equivalent
to *'sasl.oauthbearer.method'* in librdkafka.

  * **o_auth_bearer_client_id** (<xref:str>* | *<xref:None>) – Used with *oidc*
method. Equivalent to *'sasl.oauthbearer.client.id'* in librdkafka.

  * **o_auth_bearer_client_secret** (<xref:str>* | *<xref:None>) – Used with *oidc*
method. Equivalent to *'sasl.oauthbearer.client.secret'* in librdkafka.

  * **o_auth_bearer_scope** (<xref:str>* | *<xref:None>) – Scope of the access request
sent to the broker. Equivalent to *'sasl.oauthbearer.scope'* in librdkafka.

  * **o_auth_bearer_token_endpoint_url** (<xref:str>* | *<xref:None>) – OAuth token
endpoint. Equivalent to *'sasl.oauthbearer.token.endpoint.url'* in librdkafka.

  * **o_auth_bearer_extensions** (<xref:str>* | *<xref:None>) – Additional metadata
sent to the broker. Comma-separated key=value pairs, e.g.,
*"supportFeatureX=true,organizationId=sales-emea"*. Equivalent to
*'sasl.oauthbearer.extensions'* in librdkafka.
```

```
  * **max_message_bytes** (<xref:int>) – Maximum size (in bytes) of a transmitted
message. Default is 1MB.

  * **batch_size** (<xref:int>) – Maximum number of messages batched in one
MessageSet. Default is 10,000.

  * **enable_idempotence** (<xref:bool>) – If *True*, ensures messages are delivered
exactly once and in order. Default is *False*.

  * **message_timeout_ms** (<xref:int>) – Local timeout for message delivery. Default
is 300000 ms.

  * **request_timeout_ms** (<xref:int>) – Timeout for producer request acknowledgment.
Default is 5000 ms.

  * **max_retries** (<xref:int>) – Maximum number of retry attempts for failed
messages. Default is 2147483647. Retrying may cause reordering unless
*enable_idempotence=True*.

  * **authentication_mode**
(<xref:azure.functions.decorators.kafka.BrokerAuthenticationMode>* | *<xref:str>* | *
<xref:None>) – SASL mechanism used for authentication. Allowed values: *Gssapi*,
*Plain*, *ScramSha256*, *ScramSha512*. Default is *Plain*. Equivalent to
*'sasl.mechanism'* in librdkafka.

  * **protocol** (<xref:azure.functions.decorators.kafka.BrokerProtocol>* | *
<xref:str>* | *<xref:None>) – Security protocol used for broker communication. Default
is plaintext. Equivalent to *'security.protocol'* in librdkafka.

  * **linger_ms** (<xref:int>) – Time to wait between sending batches of messages to
allow for better throughput via batching.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

kafka_trigger(arg_name: str, topic: str, broker_list: str, event_hub_connection_string: str | None = None, consumer_group: str | None = None, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None =

None, o_auth_bearer_extensions: str | None = None, authentication_mode: BrokerAuthenticationMode | str | None = 'NotSet', protocol: BrokerProtocol | str | None = 'NotSet', cardinality: Cardinality | str | None = 'One', lag_threshold: int = 1000, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *kafka_trigger* decorator adds <xref:agent_framework.azure.KafkaTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/kafkatrigger

Parameters: * **arg_name** (str) – The variable name used in the function code for the parameter that receives the Kafka event data.

```
  * **topic** (<xref:str>) – The topic monitored by the trigger.

  * **broker_list** (<xref:str>) – The list of Kafka brokers monitored by the trigger.

  * **event_hub_connection_string** (<xref:str>* | *<xref:None>) – The name of an app
setting that contains the connection string for Event Hub (when using Kafka protocol
headers with Azure Event Hubs).

  * **consumer_group** (<xref:str>* | *<xref:None>) – Kafka consumer group used by the
trigger.

  * **avro_schema** (<xref:str>* | *<xref:None>) – Used only if a generic Avro record
should be generated.

  * **username** (<xref:str>* | *<xref:None>) – SASL username for use with the PLAIN
or SASL-SCRAM mechanisms. Equivalent to 'sasl.username' in librdkafka. Default is
empty string.

  * **password** (<xref:str>* | *<xref:None>) – SASL password for use with the PLAIN
or SASL-SCRAM mechanisms. Equivalent to 'sasl.password' in librdkafka. Default is
empty string.

  * **ssl_key_location** (<xref:str>* | *<xref:None>) – Path to the client's private
key (PEM) used for authentication. Equivalent to 'ssl.key.location' in librdkafka.

  * **ssl_ca_location** (<xref:str>* | *<xref:None>) – Path to the CA certificate file
for verifying the broker's certificate. Equivalent to 'ssl.ca.location' in librdkafka.
```

  * **ssl_certificate_location** (<xref:str>* | *<xref:None>) – Path to the client's certificate. Equivalent to 'ssl.certificate.location' in librdkafka.

  * **ssl_key_password** (<xref:str>* | *<xref:None>) – Password for the client's certificate. Equivalent to 'ssl.key.password' in librdkafka.

  * **schema_registry_url** (<xref:str>* | *<xref:None>) – URL of the Avro Schema Registry.

  * **schema_registry_username** (<xref:str>* | *<xref:None>) – Username for the Schema Registry.

  * **schema_registry_password** (<xref:str>* | *<xref:None>) – Password for the Schema Registry.

  * **o_auth_bearer_method** (<xref:azure.functions.decorators.kafka.OAuthBearerMethod>* | *<xref:str>* | *<xref:None>) – Either 'default' or 'oidc'. Equivalent to 'sasl.oauthbearer.method' in librdkafka.

  * **o_auth_bearer_client_id** (<xref:str>* | *<xref:None>) – Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.client.id' in librdkafka.

  * **o_auth_bearer_client_secret** (<xref:str>* | *<xref:None>) – Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.client.secret' in librdkafka.

  * **o_auth_bearer_scope** (<xref:str>* | *<xref:None>) – Specify only if *o_auth_bearer_method* is 'oidc'. Used to specify access scope. Equivalent to 'sasl.oauthbearer.scope'.

  * **o_auth_bearer_token_endpoint_url** (<xref:str>* | *<xref:None>) – Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.token.endpoint.url' in librdkafka.

  * **o_auth_bearer_extensions** (<xref:str>* | *<xref:None>) – Additional information for the broker, in the form of a comma-separated list of key=value pairs (e.g., "orgId=abc,flag=true"). Equivalent to 'sasl.oauthbearer.extensions' in librdkafka.

  * **authentication_mode** (<xref:azure.functions.decorators.kafka.BrokerAuthenticationMode>* | *<xref:str>* | *<xref:None>) – SASL mechanism to use. Allowed: Gssapi, Plain, ScramSha256, ScramSha512. Default: Plain. Equivalent to 'sasl.mechanism'.

  * **protocol** (<xref:azure.functions.decorators.kafka.BrokerProtocol>* | *<xref:str>* | *<xref:None>) – Security protocol used to communicate with brokers. Default: plaintext. Equivalent to 'security.protocol'.

  * **lag_threshold** (<xref:int>) – Max number of unprocessed messages per worker instance. Used in scaling logic to estimate needed worker instances. Default is 1000.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how Functions runtime should treat the parameter value.

```
  * **kwargs** – Additional keyword arguments for extra binding fields in the binding
JSON.

  * **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) –
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mcp_tool_trigger(arg_name: str, tool_name: str, description: str | None = None, tool_properties: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *mcp_tool_trigger* decorator adds <xref:agent_framework.azure.MCPToolTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/remote-mcp-functions-python

Parameters: * **arg_name** (str) – The name of the trigger parameter in the function code.

```
  * **tool_name** (<xref:str>) – The logical tool name exposed to the host.

  * **description** (<xref:str>* | *<xref:None>) – Optional human-readable description
of the tool.

  * **tool_properties** (<xref:str>* | *<xref:None>) – JSON-serialized tool
properties/parameters list.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mysql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *mysql_input* decorator adds <xref:agent_framework.azure.MySqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> input object in the function code.

```
  * **command_text** (<xref:str>) - The SQL query string or the name of the stored
procedure to execute.

  * **connection_string_setting** (<xref:str>) - The name of the app setting that
contains the connection string to the MySQL database.

  * **command_type** (<xref:str>* | *<xref:None>) - The type of command being
executed. Accepts *"Text"* for SQL queries or *"StoredProcedure"* for stored
procedures.

  * **parameters** (<xref:str>* | *<xref:None>) - One or more parameter values passed
to the SQL command. Should be provided as a comma-separated string in the format:
*"@param1=value1,@param2=value2"*.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Additional keyword arguments used to define fields in the
*function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mysql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *mysql_output* decorator adds <xref:agent_framework.azure.MySqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the MySQL output object in the function code.

```
  * **command_text** (<xref:str>) - The SQL command or the name of the stored
procedure executed by the binding.

  * **connection_string_setting** (<xref:str>) - The name of the app setting that
contains the connection string for the target MySQL database.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Additional keyword arguments used to define fields in the
*function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

mysql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *mysql_trigger* decorator adds <xref:agent_framework.azure.MySqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> object in the function code.

```
  * **table_name** (<xref:str>) - The name of the MySQL table monitored by the
trigger.

  * **connection_string_setting** (<xref:str>) - The name of an app setting that
contains the connection string for the database against which the query or stored
procedure is executed.

  * **leases_table_name** (<xref:str>* | *<xref:None>) - The name of the table used to
store leases. If not specified, the default table name is
*Leases_{FunctionId}_{TableId}*.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** - Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

orchestration_trigger(context_name: str, orchestration: str | None = None)

Register an Orchestrator Function.

Parameters: * **context_name** (str) – Parameter name of the DurableOrchestrationContext object.

```
  * **orchestration** (<xref:agent_framework.azure.Optional[str]>) - Name of
Orchestrator Function. The value is None by default, in which case the name of the
method is used.
```

queue_output(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *queue_output* decorator adds <xref:agent_framework.azure.QueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-queue

Parameters: * **arg_name** (str) – The name of the variable that represents the storage queue output object in the function code.

```
  * **queue_name** (<xref:str>) – The name of the queue to which messages will be
written.

  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Azure Queues.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

queue_trigger(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *queue_trigger* decorator adds <xref:agent_framework.azure.QueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-queue

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.QueueMessage> object in the function code.

```
  * **queue_name** (<xref:str>) – The name of the queue to monitor.
```

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Azure Queues.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments to include in the binding JSON.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

register_blueprint(function_container: DecoratorApi) -> None

Register a list of functions in the function app.

Parameters: **function_container** (DecoratorApi) – Instance extending <xref:agent_framework.azure.DecoratorApi>, which contains a list of functions to be registered.

Raises: TypeError – If the provided container is an instance of <xref:agent_framework.azure.FunctionRegister>.

Return type: None

register_functions(function_container: DecoratorApi) -> None

Register a list of functions in the function app.

Parameters: **function_container** (DecoratorApi) – Instance extending <xref:agent_framework.azure.DecoratorApi>, which contains a list of functions to be registered.

Raises: TypeError – If the provided container is an instance of <xref:agent_framework.azure.FunctionRegister>.

Return type: None

retry(strategy: str, max_retry_count: str, delay_interval: str | None = None, minimum_interval: str | None = None, maximum_interval: str | None = None, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]

The *retry* decorator adds a <xref:agent_framework.azure.RetryPolicy> to the function settings object.

Used for building a <xref:agent_framework.azure.Function> object in the worker function indexing model. This is equivalent to defining a RetryPolicy in *function.json*, which enables the

function to automatically retry on failure.

All optional fields are assigned default values by the function host during parsing.

Ref: https://aka.ms/azure_functions_retries

Parameters: * **strategy** (str) – The retry strategy to use.

```
  * **max_retry_count** (<xref:str>) – The maximum number of retry attempts.

  * **delay_interval** (<xref:str>* | *<xref:None>) – The delay interval between retry
attempts.

  * **minimum_interval** (<xref:str>* | *<xref:None>) – The minimum delay interval
between retry attempts.

  * **maximum_interval** (<xref:str>* | *<xref:None>) – The maximum delay interval
between retry attempts.

  * **setting_extra_fields** (<xref:typing.Dict>*[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>) – Keyword arguments for
specifying additional setting fields.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

route(route: str | None = None, trigger_arg_name: str = 'req', binding_arg_name: str = '$return', methods: Iterable[str] | Iterable[HttpMethod] | None = None, auth_level: AuthLevel | str | None = None, trigger_extra_fields: Dict[str, Any] | None = None, binding_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]

The *route* decorator adds <xref:agent_framework.azure.HttpTrigger> and <xref:agent_framework.azure.HttpOutput> bindings to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-http

Parameters: * **route** (str* | *None) – Route for the HTTP endpoint. If None, it defaults to the function name (if present) or the user-defined Python function name.

```
  * **trigger_arg_name** (<xref:str>) - Argument name for
<xref:agent_framework.azure.HttpRequest>. Defaults to *'req'*.

  * **binding_arg_name** (<xref:str>) - Argument name for
<xref:agent_framework.azure.HttpResponse>. Defaults to *'$return'*.

  * **methods** (<xref:typing.Iterable>*[*<xref:str>*] **| *<xref:typing.Iterable>*[*
[HttpMethod](azure.functions.decorators.http.HttpMethod)*] **| *<xref:None>) - A tuple
of the HTTP methods to which the function responds.

  * **auth_level** (<xref:azure.functions.decorators.core.AuthLevel>* | *<xref:str>* |
*<xref:None>) - Determines what keys, if any, need to be present on the request in
order to invoke the function.

  * **trigger_extra_fields** (<xref:typing.Dict>*[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>) - Additional fields to
include in the trigger JSON. For example: >>> data_type='STRING'  # results in
'dataType': 'STRING' in the trigger JSON

  * **binding_extra_fields** (<xref:typing.Dict>*[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>) - Additional fields to
include in the binding JSON. For example: >>> data_type='STRING'  # results in
'dataType': 'STRING' in the binding JSON
```

Returns: Decorator function.

Return type: Callable[[...], Any]

schedule(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-timer

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.

```
  * **schedule** (<xref:str>) – A string representing a CRON expression used to
schedule the function execution.

  * **run_on_startup** (<xref:bool>* | *<xref:None>) – If True, the function is
invoked when the runtime starts.

  * **use_monitor** (<xref:bool>* | *<xref:None>) – Set to True or False to indicate
whether the schedule should be monitored.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) –
```

Returns: Decorator function.

Return type: Callable[[...], Any]

semantic_search_input(arg_name: str, search_connection_name: str, collection: str, query: str | None = None, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, system_prompt: str | None = semantic_search_system_prompt, max_knowledge_count: int | None = 1, temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: https://platform.openai.com/docs/guides/embeddings

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **search_connection_name** (<xref:str>) – The name of the app setting or
environment variable that contains the vector search connection value.
```

  * **collection** (<xref:str>) – The name of the collection or table to search or store.

  * **query** (<xref:str>* | *<xref:None>) – The semantic query text used for searching the database.

  * **ai_connection_name** (<xref:str>* | *<xref:None>) –

    The name of the configuration section for AI service
    connectivity settings.

    * For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section.

    * If not specified or the section is missing, defaults to the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.

    * For user-assigned managed identity authentication, this field is required.

    * For OpenAI (non-Azure), set the *OPENAI_API_KEY* environment variable.

  * **embeddings_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The deployment or model name for OpenAI Embeddings. Defaults to "text-embedding-ada-002".

  * **chat_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The deployment or model name for the OpenAI Chat Completion API. Defaults to "gpt-3.5-turbo".

  * **system_prompt** (<xref:str>* | *<xref:None>) – Optional. The system prompt provided to the large language model.

  * **max_knowledge_count** (<xref:int>* | *<xref:None>) – Optional. The number of knowledge items to inject into the system prompt. Default is 1.

  * **temperature** (<xref:str>* | *<xref:None>) – The sampling temperature to use (range: 0 to 2). Higher values like 0.8 yield more random output; lower values like 0.2 make output more focused.

  * **top_p** (<xref:str>* | *<xref:None>) – Alternative to temperature sampling (nucleus sampling). For example, 0.1 considers only the top 10% of tokens by probability mass. Use this or *temperature*.

  * **max_tokens** (<xref:str>* | *<xref:None>) – The maximum number of tokens to generate in the completion. The sum of prompt tokens and *max_tokens* must not exceed the model's context length (usually 2048 or 4096 for newer models).

  * **is_reasoning_model** (<xref:bool>* | *<xref:None>) – Indicates whether the chat model is a reasoning model. If True, *max_tokens* and *temperature* are not supported.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Optional. Defines how the Functions runtime should interpret the parameter value. Default is None.

```
  * **kwargs** – Additional keyword arguments for specifying extra fields in the
*function.json* binding.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_queue_output(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, **kwargs) -> Callable[[...], Any]

The *service_bus_queue_output* decorator adds <xref:agent_framework.azure.ServiceBusQueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the Service Bus queue output object in the function code.

```

  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Service Bus.

  * **queue_name** (<xref:str>) – The name of the queue to which messages will be
sent.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) – The access rights required for the connection string.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_queue_trigger(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *on_service_bus_queue_change* decorator adds <xref:agent_framework.azure.ServiceBusQueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusQueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Service Bus.

  * **queue_name** (<xref:str>) – The name of the queue to monitor.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) – Access rights for the connection string.

  * **is_sessions_enabled** (<xref:bool>* | *<xref:None>) – Set to True if connecting
to a session-aware queue or subscription.

  * **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) – Set to "many" to enable batching.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) –
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_topic_output(arg_name: str, connection: str, topic_name: str, subscription_name: str | None = None, data_type: DataType | str | None = None, access_rights: AccessRights | str | None =

None, **kwargs) -> Callable[[...], Any]

The *service_bus_topic_output* decorator adds
<xref:agent_framework.azure.ServiceBusTopicOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusTopicOutput* in the *function.json*, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the Service Bus topic output object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Service Bus.

  * **topic_name** (<xref:str>) – The name of the topic to which messages will be
sent.

  * **subscription_name** (<xref:str>* | *<xref:None>) – The name of the subscription
(optional for output).

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
Defaults to DataType.UNDEFINED.

  * **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) – The access rights required for the connection string.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

service_bus_topic_trigger(arg_name: str, connection: str, topic_name: str, subscription_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *on_service_bus_topic_change* decorator adds
<xref:agent_framework.azure.ServiceBusTopicTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Service Bus.

  * **topic_name** (<xref:str>) – The name of the topic to monitor.

  * **subscription_name** (<xref:str>) – The name of the subscription to monitor.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) – Access rights for the connection string.

  * **is_sessions_enabled** (<xref:bool>* | *<xref:None>) – Set to True if connecting
to a session-aware queue or subscription.

  * **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) – Set to "many" to enable batching.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) –
```

Returns: Decorator function.

Return type: Callable[[...], Any]

sql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *sql_input* decorator adds <xref:agent_framework.azure.SqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> input object in the function code.

```
  * **command_text** (<xref:str>) – The Transact-SQL query or the name of the stored
procedure executed by the binding.

  * **connection_string_setting** (<xref:str>) – The name of the app setting that
contains the connection string to the target SQL database.

  * **command_type** (<xref:str>* | *<xref:None>) – The command type. Use *"Text"* for
a raw SQL query and *"StoredProcedure"* for a stored procedure.

  * **parameters** (<xref:str>* | *<xref:None>) – A comma-separated string of
parameter assignments to pass to the SQL command. Format:
*@param1=value1,@param2=value2*

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

sql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *sql_output* decorator adds <xref:agent_framework.azure.SqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the SQL output object in the function code.

```
  * **command_text** (<xref:str>) – The Transact-SQL query or the name of the stored
procedure executed by the binding.

  * **connection_string_setting** (<xref:str>) – The name of the app setting that
contains the connection string to the target SQL database.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

sql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]

The *sql_trigger* decorator adds <xref:agent_framework.azure.SqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Parameters: * **arg_name** (str) – The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> object in the function code.

```
  * **table_name** (<xref:str>) – The name of the SQL table monitored by the trigger.

  * **connection_string_setting** (<xref:str>) – The name of an app setting that
contains the connection string for the database against which the query or stored
procedure is executed.

  * **leases_table_name** (<xref:str>* | *<xref:None>) – The name of the table used to
store leases. If not specified, the default name is *Leases_{FunctionId}_{TableId}*.
```

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

table_input(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, take: int | None = None, filter: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]

The *table_input* decorator adds <xref:agent_framework.azure.TableInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/tablesbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the table or entity in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to the table service.

  * **table_name** (<xref:str>) – The name of the table.

  * **row_key** (<xref:str>* | *<xref:None>) – The row key of the table entity to
read.

  * **partition_key** (<xref:str>* | *<xref:None>) – The partition key of the table
entity to read.

  * **take** (<xref:int>* | *<xref:None>) – The maximum number of entities to return.

  * **filter** (<xref:str>* | *<xref:None>) – An OData filter expression to apply when
retrieving entities.
```

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

table_output(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]

The *table_output* decorator adds <xref:agent_framework.azure.TableOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableOutput* in the *function.json*, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/tablesbindings

Parameters: * **arg_name** (str) – The name of the variable that represents the table or entity in the function code.

```
  * **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to the table service.

  * **table_name** (<xref:str>) – The name of the table.

  * **row_key** (<xref:str>* | *<xref:None>) – The row key of the table entity to
read.

  * **partition_key** (<xref:str>* | *<xref:None>) – The partition key of the table
entity to read.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

text_completion_input(arg_name: str, prompt: str, ai_connection_name: str | None = '', chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, temperature: str |

None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions

The examples typically define an HTTP-triggered function with a hardcoded prompt such as *"who is {name}?"*, where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
  * **prompt** (<xref:str>) – The text prompt to generate completions for.

  * **ai_connection_name** (<xref:str>* | *<xref:None>) –

    The name of the configuration section that defines
    AI service connectivity settings.

    * For **Azure OpenAI**: If specified, it looks for *"Endpoint"* and/or

    *"Key"* in this section. If not specified or missing, it falls back to
    the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.
    - For **user-assigned managed identity**, this setting is required.
    - For **non-Azure OpenAI**, ensure the *OPENAI_API_KEY* environment
    variable is set.

  * **model** – *(Deprecated)* Use *chat_model* instead. This parameter is unused and
will be removed in future versions.

  * **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The
deployment or model name to use for the Chat Completions API. Default is *"gpt-3.5-
turbo"*.

  * **temperature** (<xref:str>* | *<xref:None>) – Sampling temperature (0–2). Higher
values (e.g., 0.8) produce more random output; lower values (e.g., 0.2) make output
more focused and deterministic.

  * **top_p** (<xref:str>* | *<xref:None>) – Controls nucleus sampling. For example,
*top_p=0.1* considers only tokens in the top 10% cumulative probability. Recommended
to use either *temperature* or *top_p*, but not both.

  * **max_tokens** (<xref:str>* | *<xref:None>) – The maximum number of tokens to
generate. The sum of prompt tokens and *max_tokens* must not exceed the model's
```

```
context length (usually 2048 or 4096 for newer models).

  * **is_reasoning_model** (<xref:bool>* | *<xref:None>) – If *True*, indicates that
the configured chat model is a reasoning model. In this case, *max_tokens* and
*temperature* are not supported.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** – Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: Callable[[...], Any]

timer_trigger(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-timer

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.

```

  * **schedule** (<xref:str>) – A string representing a CRON expression used to
schedule the function execution.

  * **run_on_startup** (<xref:bool>* | *<xref:None>) – If True, the function is
invoked when the runtime starts.

  * **use_monitor** (<xref:bool>* | *<xref:None>) – Set to True or False to indicate
whether the schedule should be monitored.

  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

  * **kwargs** (<xref:agent_framework.exceptions.Any>) –
```

Returns: Decorator function.

Return type: Callable[[...], Any]

validate_function_names(functions: List[Function])

Validate that all functions have unique names within the app.

The *functions_bindings* dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.

Parameters: **functions** (List[Function]) – A list of <xref:agent_framework.azure.Function> objects to validate.

Raises: ValueError – If any function name is duplicated.

warm_up_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable

The *warm_up* decorator adds <xref:agent_framework.azure.WarmUpTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *WarmUpTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-warmup

Parameters: * **arg_name** (str) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in the function code.

```
  * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: Callable

Constructor

Python

```
AgentFunctionApp()
```

Parameters

Expand table

Name	Description
agents	Default value: None
http_auth_level	Default value: FUNCTION
enable_health_check	Default value: True
enable_http_endpoints	Default value: True
max_poll_retries	Default value: 30
poll_interval_seconds	Default value: 1.0
enable_mcp_tool_trigger	Default value: False
default_callback	Default value: None

Methods

Expand table

__init__	Initialize the AgentFunctionApp.
__new__	
activity_trigger	Register an Activity Function.
add_agent	Add an agent to the function app after initialization.
assistant_create_output	The *assistantCreate* output binding creates a new assistant with a specified system prompt.
assistant_post_input	The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.
assistant_query_input	The *assistantQuery* input binding retrieves assistant chat history and passes it to the function. This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

assistant_skill_trigger	Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions to invoke and when. Ref: https://platform.openai.com/docs/guides/function-calling You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the extension when an assistant signals it would like to invoke a function in response to a user prompt. The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.
blob_input	The *blob_input* decorator adds <xref:agent_framework.azure.BlobInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-storage-blob
blob_output	The *blob_output* decorator adds <xref:agent_framework.azure.BlobOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-storage-blob
blob_trigger	The blob_change_trigger decorator adds <xref:agent_framework.azure.BlobTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building <xref:agent_framework.azure.Function> object used in worker function indexing model. This is equivalent to defining BlobTrigger in the function.json which enables the function to be triggered when new message(s) are sent to storage blobs.

	All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-binding-storage-blob
cosmos_db_input	The *cosmos_db_input* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle version 4.x and above. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields will be assigned default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-binding-cosmosdb-v4 Additional details: https://aka.ms/cosmosdb-v4-update
cosmos_db_input_v3	The *cosmos_db_input_v3* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed. For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2
cosmos_db_output	The *cosmos_db_output* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed. For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4
cosmos_db_output_v3	The *cosmos_db_output_v3* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a

	<xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed. For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2
cosmos_db_trigger	The *cosmos_db_trigger* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed. See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4
cosmos_db_trigger_v3	The *cosmos_db_trigger_v3* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x. This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed. See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2
dapr_binding_output	The *dapr_binding_output* decorator adds <xref:agent_framework.azure.DaprBindingOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding. All optional fields will be assigned default values by the function host when parsed.

	Ref: https://aka.ms/azure-function-dapr-binding-output-binding
dapr_binding_trigger	The *dapr_binding_trigger* decorator adds <xref:agent_framework.azure.DaprBindingTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-dapr-trigger-binding
dapr_invoke_output	The *dapr_invoke_output* decorator adds <xref:agent_framework.azure.DaprInvokeOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *DaprInvokeOutput* in the *function.json*, which enables the function to invoke another Dapr app. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-dapr-invoke-output-binding
dapr_publish_output	The *dapr_publish_output* decorator adds <xref:agent_framework.azure.DaprPublishOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-dapr-publish-output-binding
dapr_secret_input	The *dapr_secret_input* decorator adds <xref:agent_framework.azure.DaprSecretInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret

	store component. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-dapr-secret-input-binding
dapr_service_invocation_trigger	The *dapr_service_invocation_trigger* decorator adds <xref:agent_framework.azure.DaprServiceInvocationTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-dapr-trigger-service-invocation
dapr_state_input	The *dapr_state_input* decorator adds <xref:agent_framework.azure.DaprStateInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *DaprStateInput* in the *function.json*, which enables the function to read state from the underlying state store component. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-dapr-state-input-binding
dapr_state_output	The *dapr_state_output* decorator adds <xref:agent_framework.azure.DaprStateOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-dapr-state-output-binding
dapr_topic_trigger	The *dapr_topic_trigger* decorator adds <xref:agent_framework.azure.DaprTopicTrigger> to the

	<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system. All optional fields will be assigned default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-dapr-trigger-topic
durable_client_input	Register a Durable-client Function.
durable_openai_agent_orchestrator	Decorate Azure Durable Functions orchestrators that use OpenAI Agents.
embeddings_input	The embeddings input decorator generates embeddings used to measure the relatedness of text strings. Ref: https://platform.openai.com/docs/guides/embeddings
embeddings_store_output	Add an embeddings input binding to the function. The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions: • Azure AI Search • Azure Data Explorer • Azure Cosmos DB using MongoDB
entity_trigger	Register an Entity Function.
event_grid_output	The *event_grid_output* decorator adds <xref:agent_framework.azure.EventGridOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining the output binding in the *function.json*, which enables the function to write events to a custom topic. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/eventgridtrigger
event_grid_trigger	The *event_grid_trigger* decorator adds <xref:agent_framework.azure.EventGridTrigger> to the

	<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/eventgridtrigger
event_hub_message_trigger	The *event_hub_message_trigger* decorator adds <xref:agent_framework.azure.EventHubTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-binding-event-hubs
event_hub_output	The *event_hub_output* decorator adds <xref:agent_framework.azure.EventHubOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-event-hubs
function_name	Optional: Sets name of the <xref:agent_framework.azure.Function> object. If not set, it will default to the name of the method name. additional setting fields :return: Decorator function.
generic_input_binding	The *generic_input_binding* decorator adds <xref:agent_framework.azure.GenericInputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

	This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-custom
generic_output_binding	The *generic_output_binding* decorator adds <xref:agent_framework.azure.GenericOutputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-custom
generic_trigger	The *generic_trigger* decorator adds <xref:agent_framework.azure.GenericTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-binding-custom
get_agent	Return a DurableAIAgent proxy for a registered agent.
get_functions	Get the function objects in the function app.
http_type	Set http type of the <xref:agent_framework.azure.Function> object.
kafka_output	The *kafka_output* decorator adds <xref:agent_framework.azure.KafkaOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining a Kafka output binding in *function.json*, which enables the function to write events to a Kafka topic. All

	optional fields are assigned default values by the function host when parsed. Ref: https://aka.ms/kafkaoutput
kafka_trigger	The *kafka_trigger* decorator adds <xref:agent_framework.azure.KafkaTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/kafkatrigger
mcp_tool_trigger	The *mcp_tool_trigger* decorator adds <xref:agent_framework.azure.MCPToolTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/remote-mcp-functions-python
mysql_input	The *mysql_input* decorator adds <xref:agent_framework.azure.MySqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed. Ref: https://aka.ms/mysqlbindings
mysql_output	The *mysql_output* decorator adds <xref:agent_framework.azure.MySqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a

	<xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/mysqlbindings
mysql_trigger	The *mysql_trigger* decorator adds <xref:agent_framework.azure.MySqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed. Ref: https://aka.ms/mysqlbindings
orchestration_trigger	Register an Orchestrator Function.
queue_output	The *queue_output* decorator adds <xref:agent_framework.azure.QueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-queue
queue_trigger	The *queue_trigger* decorator adds <xref:agent_framework.azure.QueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

	All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-queue
register_blueprint	Register a list of functions in the function app.
register_functions	Register a list of functions in the function app.
retry	The *retry* decorator adds a <xref:agent_framework.azure.RetryPolicy> to the function settings object. Used for building a <xref:agent_framework.azure.Function> object in the worker function indexing model. This is equivalent to defining a RetryPolicy in *function.json*, which enables the function to automatically retry on failure. All optional fields are assigned default values by the function host during parsing. Ref: https://aka.ms/azure_functions_retries
route	The *route* decorator adds <xref:agent_framework.azure.HttpTrigger> and <xref:agent_framework.azure.HttpOutput> bindings to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-http
schedule	The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-timer
semantic_search_input	Enable semantic search capabilities using vector databases and OpenAI models.

	This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG). Ref: https://platform.openai.com/docs/guides/embeddings
service_bus_queue_output	The *service_bus_queue_output* decorator adds <xref:agent_framework.azure.ServiceBusQueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue. All optional fields will be assigned default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-binding-service-bus
service_bus_queue_trigger	The *on_service_bus_queue_change* decorator adds <xref:agent_framework.azure.ServiceBusQueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a *ServiceBusQueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Service Bus queue. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-service-bus
service_bus_topic_output	The *service_bus_topic_output* decorator adds <xref:agent_framework.azure.ServiceBusTopicOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining *ServiceBusTopicOutput* in the *function.json*, which enables the function to write messages to a Service Bus topic. All optional fields will be assigned default values by the function host when they are parsed.

	Ref: https://aka.ms/azure-function-binding-service-bus
service_bus_topic_trigger	The *on_service_bus_topic_change* decorator adds <xref:agent_framework.azure.ServiceBusTopicTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-service-bus
sql_input	The *sql_input* decorator adds <xref:agent_framework.azure.SqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the function to read from a SQL database. All optional fields are assigned default values by the function host when parsed. Ref: https://aka.ms/sqlbindings
sql_output	The *sql_output* decorator adds <xref:agent_framework.azure.SqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed. Ref: https://aka.ms/sqlbindings
sql_trigger	The *sql_trigger* decorator adds <xref:agent_framework.azure.SqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All

	optional fields are assigned default values by the function host when parsed. Ref: https://aka.ms/sqlbindings
table_input	The *table_input* decorator adds <xref:agent_framework.azure.TableInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read from a table in Azure Storage or a Cosmos DB account. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/tablesbindings
table_output	The *table_output* decorator adds <xref:agent_framework.azure.TableOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining *TableOutput* in the *function.json*, which enables the function to write entities to a table in Azure Storage. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/tablesbindings
text_completion_input	The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function. Ref: https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions The examples typically define an HTTP-triggered function with a hardcoded prompt such as "*who is {name}?*", where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.
timer_trigger	The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

	This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-timer
validate_function_names	Validate that all functions have unique names within the app. The *functions_bindings* dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.
warm_up_trigger	The *warm_up* decorator adds <xref:agent_framework.azure.WarmUpTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining a *WarmUpTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-warmup

__init__

Initialize the AgentFunctionApp.

Python

```
__init__(agents: list[AgentProtocol] | None = None, http_auth_level: AuthLevel =
func.AuthLevel.FUNCTION, enable_health_check: bool = True, enable_http_endpoints:
bool = True, max_poll_retries: int = DEFAULT_MAX_POLL_RETRIES,
poll_interval_seconds: float = DEFAULT_POLL_INTERVAL_SECONDS,
enable_mcp_tool_trigger: bool = False, default_callback:
AgentResponseCallbackProtocol | None = None)
```

Parameters

Expand table

Name	Description
agents	list[AgentProtocol] | None List of agent instances to register. Default value: None

Name	Description
http_auth_level	AuthLevel HTTP authentication level (default: `func.AuthLevel.FUNCTION`). Default value: func.AuthLevel.FUNCTION
enable_health_check	bool Enable the built-in health check endpoint (default: `True`). Default value: True
enable_http_endpoints	bool Enable HTTP endpoints for agents (default: `True`). Default value: True
enable_mcp_tool_trigger	bool Enable MCP tool triggers for agents (default: `False`). When enabled, agents will be exposed as MCP tools that can be invoked by MCP-compatible clients. Default value: False
max_poll_retries	int Maximum polling attempts when waiting for a response. Defaults to `DEFAULT_MAX_POLL_RETRIES`. Default value: DEFAULT_MAX_POLL_RETRIES
poll_interval_seconds	float Delay in seconds between polling attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`. Default value: DEFAULT_POLL_INTERVAL_SECONDS
default_callback	AgentResponseCallbackProtocol | None Optional callback invoked for agents without specific callbacks. Default value: None

__new__

Python

```
__new__(**kwargs)
```

activity_trigger

Register an Activity Function.

Python

```
activity_trigger(input_name: str, activity: str | None = None)
```

Parameters

Expand table

Name	Description
input_name Required*	str Parameter name of the Activity input.
activity	<xref:Optional>[str] Name of Activity Function. The value is None by default, in which case the name of the method is used. Default value: None

add_agent

Add an agent to the function app after initialization.

Python

```
add_agent(agent: AgentProtocol, callback: AgentResponseCallbackProtocol | None = None, enable_http_endpoint: bool | None = None, enable_mcp_tool_trigger: bool | None = None) -> None
```

Parameters

Expand table

Name	Description
agent Required*	AgentProtocol The Microsoft Agent Framework agent instance (must implement AgentProtocol) The agent must have a 'name' attribute.
callback	AgentResponseCallbackProtocol | None Optional callback invoked during agent execution Default value: None
enable_http_endpoint	bool | None Optional flag to enable/disable HTTP endpoint for this agent. The app level enable_http_endpoints setting will override this setting. Default value: None
enable_mcp_tool_trigger	bool | None Optional flag to enable/disable MCP tool trigger for this agent. The app level enable_mcp_tool_trigger setting will override this setting. Default value: None

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ValueError	If the agent doesn't have a 'name' attribute or if an agent with the same name is already registered

assistant_create_output

The *assistantCreate* output binding creates a new assistant with a specified system prompt.

Python

```
assistant_create_output(arg_name: str, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying extra binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

assistant_post_input

The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.

Python

```
assistant_post_input(arg_name: str, id: str, user_message: str,
ai_connection_name: str | None = '', chat_model: str | OpenAIModels | None =
OpenAIModels.DefaultChatModel, chat_storage_connection_setting: str | None =
'AzureWebJobsStorage', collection_name: str | None = 'ChatState', temperature: str
| None = '0.5', top_p: str | None = None, max_tokens: str | None = '100',
is_reasoning_model: bool | None = False, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
id Required*	str The ID of the assistant to update.
user_message Required*	str The message entered by the user for the assistant to respond to.
ai_connection_name	str | None The name of the configuration section for AI service connectivity. • **Azure OpenAI**: If specified, looks for "Endpoint" and/or "Key" in the section. • If not specified or missing, falls back to the *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY* environment variables. • **Managed Identity**: Required for user-assigned identity auth. • **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY* environment variable.

Name	Description
	Default value: ""
model Required*	*Deprecated*. Use *chat_model* instead. This parameter is unused and will be removed in future versions.
chat_model	str | OpenAIModels | None The deployment or model name of the OpenAI Chat Completion API. Default is *"gpt-3.5-turbo"*. Default value: OpenAIModels.DefaultChatModel
chat_storage_connection_setting	str | None The config section name for assistant chat table storage. Default is *"AzureWebJobsStorage"*. Default value: "AzureWebJobsStorage"
collection_name	str | None The collection or table name used for assistant chat storage. Default is *"ChatState"*. Default value: "ChatState"
temperature	str | None Sampling temperature, between 0 and 2. Higher values (e.g., 0.8) increase randomness; lower values (e.g., 0.2) make output more deterministic. Default value: "0.5"
top_p	str | None Alternative to temperature sampling. Uses nucleus sampling to consider only the tokens comprising the top *top_p* probability mass. Recommended to use either this or *temperature*. Default value: None
max_tokens	str | None Maximum tokens to generate. Total tokens (prompt + output) must not exceed the model's context length. Default value: "100"
is_reasoning_model	bool | None Whether the chat model is a reasoning model. If true, *max_tokens* and *temperature* are not supported. Default value: False
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying extra binding fields in *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

assistant_query_input

The *assistantQuery* input binding retrieves assistant chat history and passes it to the function.

This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

Python

```
assistant_query_input(arg_name: str, id: str, timestamp_utc: str,
chat_storage_connection_setting: str | None = 'AzureWebJobsStorage',
collection_name: str | None = 'ChatState', data_type: DataType | str | None =
None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
timestamp_utc Required*	str The earliest timestamp (in UTC) for the messages to retrieve from the chat history. Must be in ISO 8601 format, e.g., "*2023-08-01T00:00:00Z*".
chat_storage_connection_setting	str | None The name of the configuration section containing the connection settings for assistant chat storage. Defaults to "*AzureWebJobsStorage*". Default value: "AzureWebJobsStorage"
collection_name	str | None The name of the table or collection used for assistant chat storage. Defaults to "*ChatState*". Default value: "ChatState"

Name	Description
id Required*	str The unique identifier of the assistant whose history is being queried.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

assistant_skill_trigger

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions to invoke and when.

Ref: https://platform.openai.com/docs/guides/function-calling

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

Python

```
assistant_skill_trigger(arg_name: str, function_description: str, function_name:
str | None = None, parameter_description_json: str | None = None, data_type:
DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the trigger parameter in the function code.
function_description Required*	str A description of the assistant function, which is provided to the model.
function_name	str | None The name of the assistant function, which is passed to the language model. Default value: None
parameter_description_json	str | None A JSON-formatted description of the function parameters, provided to the model. If omitted, the description is autogenerated. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

blob_input

The *blob_input* decorator adds <xref:agent_framework.azure.BlobInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Python

```
blob_input(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the blob in the function code.
path Required*	str The path to the blob.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Blobs.
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

blob_output

The *blob_output* decorator adds <xref:agent_framework.azure.BlobOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Python

```
blob_output(arg_name: str, path: str, connection: str, data_type: DataType | None
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the blob in the function code.
path Required*	str The path to the blob.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Blobs.
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

blob_trigger

The blob_change_trigger decorator adds <xref:agent_framework.azure.BlobTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building <xref:agent_framework.azure.Function> object used in worker function indexing model. This

is equivalent to defining BlobTrigger in the function.json which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-storage-blob

Python

```
blob_trigger(arg_name: str, path: str, connection: str, source: BlobSource | None
= None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.InputStream> object in function code.
path Required*	str The path to the blob.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Blobs.
source	BlobSource | None Sets the source of the triggering event. Use "EventGrid" for an Event Grid-based blob trigger, which provides much lower latency. The default is "LogsAndContainerScan", which uses the standard polling mechanism to detect changes in the container. Default value: None
data_type	DataType | None Defines how Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

cosmos_db_input

The *cosmos_db_input* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-cosmosdb-v4 Additional details: https://aka.ms/cosmosdb-v4-update

Python

```
cosmos_db_input(arg_name: str, connection: str, database_name: str,
container_name: str, partition_key: str | None = None, id: str | None = None,
sql_query: str | None = None, preferred_locations: str | None = None, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.
connection Required*	str The name of an app setting or setting container that specifies how to connect to the Azure Cosmos DB account being monitored.
database_name Required*	str The name of the Cosmos DB database that contains the document.
container_name	str

Name	Description
Required*	The name of the container that holds the document.
partition_key	str | None The partition key value for the document lookup. Default value: None
id	str | None The ID of the document to retrieve. Default value: None
sql_query	str | None An Azure Cosmos DB SQL query used to retrieve multiple documents. Default value: None
preferred_locations	str | None (Optional) Preferred geo-replicated regions. Values should be comma-separated, e.g., "East US,South Central US,North Europe". Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

cosmos_db_input_v3

The *cosmos_db_input_v3* decorator adds <xref:agent_framework.azure.CosmosDBInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Python

```
cosmos_db_input_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, id: str | None = None, sql_query: str | None =
None, partition_key: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.
database_name Required*	str The name of the Azure Cosmos DB database containing the target collection.
collection_name Required*	str The name of the collection that contains the document(s).
connection_string_setting Required*	str The name of the app setting that contains the Azure Cosmos DB connection string.
id	str | None The ID of a single document to retrieve. Default value: None
sql_query	str | None An Azure Cosmos DB SQL query used to retrieve multiple documents. Default value: None
partition_key	str | None Specifies the partition key value for the lookup. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs	

Name	Description
Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

cosmos_db_output

The *cosmos_db_output* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4

Python

```
cosmos_db_output(arg_name: str, connection: str, database_name: str,
container_name: str, create_if_not_exists: bool | None = None, partition_key: str
| None = None, container_throughput: int | None = None, preferred_locations: str |
None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...],
Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Cosmos DB output object in the function code.
connection	str

Name	Description
Required*	The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account.
database_name Required*	str The name of the Azure Cosmos DB database containing the target container.
container_name Required*	str The name of the container to which documents are written.
create_if_not_exists	bool | None A boolean indicating whether the container should be created automatically if it does not exist. Default value: None
partition_key	str | None When *create_if_not_exists* is *True*, this defines the partition key path for the created container. Default value: None
container_throughput	int | None When *create_if_not_exists* is *True*, this defines the throughput (Request Units) for the created container. Default value: None
preferred_locations	str | None Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB. When set along with *use_multiple_write_locations*, it enables multi-region writes. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

cosmos_db_output_v3

The *cosmos_db_output_v3* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Python

```
cosmos_db_output_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, create_if_not_exists: bool | None = None,
partition_key: str | None = None, collection_throughput: int | None = None,
use_multiple_write_locations: bool | None = None, preferred_locations: str | None
= None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Cosmos DB output object in the function code.
database_name Required*	str The name of the Azure Cosmos DB database containing the monitored collection.
collection_name Required*	str The name of the collection to which documents are written.
connection_string_setting Required*	str The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account.
create_if_not_exists	bool | None A boolean indicating whether the collection should be created automatically if it does not exist. Default value: None
partition_key	str | None

Name	Description
	When *create_if_not_exists* is *True*, this defines the partition key path for the created collection. Default value: None
collection_throughput	int | None When *create_if_not_exists* is *True*, this defines the throughput (Request Units) for the created collection. Default value: None
use_multiple_write_locations	bool | None When set to *True* and used in conjunction with *preferred_locations*, this enables multi-region writes in the Azure Cosmos DB service. Default value: None
preferred_locations	str | None Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

cosmos_db_trigger

The *cosmos_db_trigger* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v4

Python

```
cosmos_db_trigger(arg_name: str, connection: str, database_name: str,
container_name: str, lease_connection: str | None = None, lease_database_name: str
| None = None, lease_container_name: str | None = None,
create_lease_container_if_not_exists: bool | None = None,
leases_container_throughput: int | None = None, lease_container_prefix: str | None
= None, feed_poll_delay: int | None = None, lease_acquire_interval: int | None =
None, lease_expiration_interval: int | None = None, lease_renew_interval: int |
None = None, max_items_per_invocation: int | None = None, start_from_beginning:
time | None = None, start_from_time: time | None = None, preferred_locations: str
| None = None, data_type: DataType | str | None = None, **kwargs: Any) ->
Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to the monitored Azure Cosmos DB account.
database_name Required*	str The name of the Azure Cosmos DB database containing the monitored collection.
container_name Required*	str The name of the container being monitored.
lease_connection	str | None (Optional) The name of an app setting or setting collection that specifies how to connect to the Cosmos DB account that holds the lease container. Default value: None
lease_database_name	str | None The name of the database that holds the collection used to store leases.

Name	Description
	Default value: None
lease_container_name	str | None (Optional) The name of the container used to store leases. If not set, the default value "leases" is used. Default value: None
create_lease_container_if_not_exists	bool | None (Optional) If *True*, the leases container is created automatically if it does not exist. Defaults to *False*. Note: When using Azure AD identities, container creation is not allowed, and the function will not start if this is *True*. Default value: None
leases_container_throughput	int | None (Optional) The number of Request Units (RUs) to assign when the leases container is created. This is used only when *create_lease_container_if_not_exists* is *True*. It is automatically set when configured through the Azure Portal. Default value: None
lease_container_prefix	str | None (Optional) A prefix added to leases created in the lease container for this function. Use this to allow multiple functions to share the same lease container with different prefixes. Default value: None
feed_poll_delay	int | None The delay (in milliseconds) between polling a partition for new changes after draining current changes. Default value: None
lease_acquire_interval	int | None The interval (in milliseconds) to trigger a task to check if partitions are evenly distributed across host instances. Default value: None
lease_expiration_interval	int | None The interval (in milliseconds) for which a lease is held for a partition. Default value: None
lease_renew_interval	int | None The interval (in milliseconds) to renew all leases for partitions currently held by an instance. Default value: None

Name	Description
max_items_per_invocation	int | None Maximum number of items received per function call. Default value: None
start_from_beginning	time | None If *True*, the trigger starts reading changes from the beginning of the collection's change history instead of the current time. Default value: None
start_from_time	time | None (Optional) The date and time from which to begin reading the change feed. Use ISO 8601 format with a UTC designator, e.g., *2021-02-16T14:19:29Z*. Only used to set the initial trigger state; has no effect once the trigger has a lease state. Default value: None
preferred_locations	str | None Preferred locations (regions) for geo-replicated Cosmos DB accounts. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

cosmos_db_trigger_v3

The *cosmos_db_trigger_v3* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a

<xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed.

See: https://aka.ms/cosmosdb-v4-update Ref: https://aka.ms/azure-function-binding-cosmosdb-v2

Python

```
cosmos_db_trigger_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, lease_collection_name: str | None = None,
lease_connection_string_setting: str | None = None, lease_database_name: str |
None = None, create_lease_collection_if_not_exists: bool | None = None,
leases_collection_throughput: int | None = None, lease_collection_prefix: str |
None = None, checkpoint_interval: int | None = None, checkpoint_document_count:
int | None = None, feed_poll_delay: int | None = None, lease_renew_interval: int |
None = None, lease_acquire_interval: int | None = None, lease_expiration_interval:
int | None = None, max_items_per_invocation: int | None = None,
start_from_beginning: bool | None = None, preferred_locations: str | None = None,
data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.
database_name Required*	str The name of the Azure Cosmos DB database containing the monitored collection.
collection_name Required*	str The name of the collection being monitored.
connection_string_setting Required*	str The name of an app setting or setting collection specifying how to connect to the monitored Azure Cosmos DB account.
lease_collection_name	str | None

Name	Description
	The name of the collection used to store leases. Default value: None
lease_connection_string_setting	str | None The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account that holds the lease collection. Default value: None
lease_database_name	str | None The name of the database that holds the lease collection. Default value: None
create_lease_collection_if_not_exists	bool | None If *True*, the lease collection is automatically created when it does not already exist. Default value: None
leases_collection_throughput	int | None The number of Request Units (RUs) assigned when the lease collection is created. Default value: None
lease_collection_prefix	str | None A prefix added to leases created in the lease collection for this Function. Default value: None
checkpoint_interval	int | None The interval (in milliseconds) between lease checkpoints. The default behavior is to checkpoint after every function call. Default value: None
checkpoint_document_count	int | None Number of documents processed between lease checkpoints. Default is after every function call. Default value: None
feed_poll_delay	int | None The delay (in milliseconds) between polling a partition for new changes after all current changes are drained. Default value: None
lease_renew_interval	int | None The interval (in milliseconds) to renew leases for partitions currently held by an instance. Default value: None

Name	Description
lease_acquire_interval	int | None The interval (in milliseconds) to trigger a task that checks whether partitions are evenly distributed among host instances. Default value: None
lease_expiration_interval	int | None The interval (in milliseconds) for which a lease is held for a partition. Default value: None
max_items_per_invocation	int | None Maximum number of items received per function call. Default value: None
start_from_beginning	bool | None If *True*, the trigger starts reading changes from the beginning of the collection's change history rather than from the current time. Default value: None
preferred_locations	str | None Preferred locations (regions) for geo-replicated database accounts in Azure Cosmos DB. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_binding_output

The *dapr_binding_output* decorator adds <xref:agent_framework.azure.DaprBindingOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-binding-output-binding

Python

```
dapr_binding_output(arg_name: str, binding_name: str, operation: str,
dapr_address: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
binding_name Required*	str The configured name of the binding.
operation Required*	str The configured operation.
dapr_address	str | None The Dapr address. Optional; defaults to http://localhost:{daprHttpPort}. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_binding_trigger

The *dapr_binding_trigger* decorator adds <xref:agent_framework.azure.DaprBindingTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-binding

Python

```
dapr_binding_trigger(arg_name: str, binding_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the trigger input in the function code.
binding_name Required*	str The name of the Dapr trigger. If not specified, the function name is used as the trigger name.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_invoke_output

The *dapr_invoke_output* decorator adds <xref:agent_framework.azure.DaprInvokeOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprInvokeOutput* in the *function.json*, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-invoke-output-binding

Python

```
dapr_invoke_output(arg_name: str, app_id: str, method_name: str, http_verb: str,
dapr_address: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
app_id Required*	str The Dapr app ID to invoke.
method_name Required*	str The method name of the app to invoke.
http_verb Required*	str The HTTP verb to use for the invocation (e.g., GET, POST).
dapr_address	str | None The Dapr address. Optional; defaults to http://localhost:{daprHttpPort}.

Name	Description
	Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_publish_output

The *dapr_publish_output* decorator adds <xref:agent_framework.azure.DaprPublishOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-publish-output-binding

Python

```
dapr_publish_output(arg_name: str, pub_sub_name: str, topic: str, dapr_address:
str | None = None, data_type: DataType | str | None = None, **kwargs) ->
Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
pub_sub_name Required*	str The pub/sub component name to publish to.
topic Required*	str The name of the topic to publish to.
dapr_address	str | None The Dapr address. Optional; defaults to http://localhost:{daprHttpPort}. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_secret_input

The *dapr_secret_input* decorator adds <xref:agent_framework.azure.DaprSecretInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-secret-input-binding

Python

```
dapr_secret_input(arg_name: str, secret_store_name: str, key: str, metadata: str,
dapr_address: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state input object in the function code.
secret_store_name Required*	str The name of the secret store from which to retrieve the secret.
key Required*	str The key identifying the name of the secret to retrieve.
metadata Required*	str Metadata properties in the form "key1=value1&key2=value2".
dapr_address	str | None The Dapr address. Optional; Defaults to http://localhost:{daprHttpPort}. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_service_invocation_trigger

The *dapr_service_invocation_trigger* decorator adds

<xref:agent_framework.azure.DaprServiceInvocationTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-service-invocation

Python

```
dapr_service_invocation_trigger(arg_name: str, method_name: str, data_type:
DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the service invocation input in the function code.
method_name Required*	str The name of the method on a remote Dapr App. If not specified, the name of the function is used as the method name.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_state_input

The *dapr_state_input* decorator adds <xref:agent_framework.azure.DaprStateInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateInput* in the *function.json*, which enables the function to read state from the underlying state store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-state-input-binding

Python

```
dapr_state_input(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state input object in the function code.
state_store Required*	str The state store containing the state.
key Required*	str The name of the key to retrieve.
dapr_address	str | None The Dapr address. Optional; Defaults to http://localhost :{daprHttpPort}. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_state_output

The *dapr_state_output* decorator adds <xref:agent_framework.azure.DaprStateOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-dapr-state-output-binding

Python

```
dapr_state_output(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
state_store Required*	str The state store containing the state for the specified keys.
key Required*	str The name of the key used to store the state.
dapr_address	str | None The Dapr address. Optional; defaults to http://localhost:{daprHttpPort}. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value.

Name	Description
	Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

dapr_topic_trigger

The *dapr_topic_trigger* decorator adds <xref:agent_framework.azure.DaprTopicTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-dapr-trigger-topic

Python

```
dapr_topic_trigger(arg_name: str, pub_sub_name: str, topic: str, route: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the trigger input in the function code.
pub_sub_name Required*	str The name of the Dapr pub/sub component.

Name	Description
topic Required*	str The topic name. If unspecified, the function name is used.
route	str | None The route for the trigger. If unspecified, the topic name is used. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Additional keyword arguments for specifying extra binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

durable_client_input

Register a Durable-client Function.

Python

```
durable_client_input(client_name: str, task_hub: str | None = None,
connection_name: str | None = None)
```

Parameters

Expand table

Name	Description
client_name Required*	str Parameter name of durable client.
task_hub	<xref:Optional>[str] Used in scenarios where multiple function apps share the same storage account but need to be isolated from each other. If not specified, the default value from

Name	Description
	host.json is used. This value must match the value used by the target orchestrator functions. Default value: None
connection_name	<xref:Optional>[str] The name of an app setting that contains a storage account connection string. The storage account represented by this connection string must be the same one used by the target orchestrator functions. If not specified, the default storage account connection string for the function app is used. Default value: None

durable_openai_agent_orchestrator

Decorate Azure Durable Functions orchestrators that use OpenAI Agents.

Python

```
durable_openai_agent_orchestrator(_func=None, *, model_provider=None,
model_retry_options: RetryOptions | None =
RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5))
```

Parameters

Expand table

Name	Description
model_provider Required*	<xref:Optional>[<xref:ModelProvider>] Use a non-default ModelProvider instead of the default OpenAIProvider, such as when testing.
model_retry_options Required*	RetryOptions | None
_func	Default value: None

Keyword-Only Parameters

Expand table

Name	Description
model_provider	Default value: None
model_retry_options	

Name	Description
	Default value: RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5)

embeddings_input

The embeddings input decorator generates embeddings used to measure the relatedness of text strings.

Ref: https://platform.openai.com/docs/guides/embeddings

Python

```
embeddings_input(arg_name: str, input: str, input_type: InputType,
ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None =
OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024,
max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs)
-> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
input Required*	str The input source containing the data to generate embeddings for.
input_type Required*	InputType The type of the input (e.g., string, list, file reference).
ai_connection_name	str | None The name of the configuration section for AI service connectivity settings. • **Azure OpenAI**: If specified, looks for "Endpoint" and/or "Key" in this section. • If not specified or not found, falls back to the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*. • For user-assigned managed identity authentication, this is required. • **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY* environment variable. Default value: ""
model	

Name	Description
Required*	*(Deprecated)* Use *embeddings_model* instead. This parameter is unused and will be removed in future versions.
embeddings_model	str | OpenAIModels | None The deployment or model name for OpenAI Embeddings. Default is *"text-embedding-ada-002"*. Default value: OpenAIModels.DefaultEmbeddingsModel
max_chunk_length	int | None The maximum number of characters into which the input should be chunked. Default is *8 * 1024*. Default value: 8 * 1024
max_overlap	int | None The maximum number of characters to overlap between input chunks. Default is *128*. Default value: 128
data_type	DataType | str | None Optional. Defines how the Functions runtime should interpret the parameter value. Default value: None
kwargs Required*	Additional keyword arguments to include in the *function.json* binding configuration.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

embeddings_store_output

Add an embeddings input binding to the function.

The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions:

- Azure AI Search
- Azure Data Explorer

- Azure Cosmos DB using MongoDB

Python

```
embeddings_store_output(arg_name: str, input: str, input_type: InputType,
store_connection_name: str, collection: str, ai_connection_name: str | None = '',
embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel,
max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
input Required*	str The input data for which embeddings should be generated.
input_type Required*	InputType The type of the input (e.g., string, list).
store_connection_name Required*	str The name of an app setting or environment variable containing the vector store connection value.
collection Required*	str The collection or table to search within.
ai_connection_name	str | None The name of the configuration section for AI service connectivity settings. • For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section. • If not specified, or if the section is missing, falls back to environment variables: *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*. • For user-assigned managed identity, this property is required. • For OpenAI (non-Azure), set the *OPENAI_API_KEY* environment variable. Default value: ""
model Required*	Deprecated. Use *embeddings_model* instead. This parameter is unused and will be removed in future versions.
embeddings_model	str | OpenAIModels | None

Name	Description
	The deployment or model name for OpenAI Embeddings. The default is "text-embedding-ada-002". Default value: OpenAIModels.DefaultEmbeddingsModel
max_chunk_length	int | None The maximum number of characters to chunk the input into. Default value: 8 * 1024
max_overlap	int | None The maximum number of characters to overlap between chunks. Default value: 128
data_type	DataType | str | None Optional. Defines how the Functions runtime should treat the parameter value. Defaults to None. Default value: None
kwargs Required*	Additional keyword arguments to include in the *function.json* binding.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

entity_trigger

Register an Entity Function.

Python

```
entity_trigger(context_name: str, entity_name: str | None = None)
```

Parameters

Expand table

Name	Description
context_name Required*	str Parameter name of the Entity input.

Name	Description
entity_name	<xref:Optional>[str] Name of Entity Function. The value is None by default, in which case the name of the method is used. Default value: None

event_grid_output

The *event_grid_output* decorator adds <xref:agent_framework.azure.EventGridOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining the output binding in the *function.json*, which enables the function to write events to a custom topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/eventgridtrigger

Python

```
event_grid_output(arg_name: str, topic_endpoint_uri: str | None = None,
topic_key_setting: str | None = None, connection: str | None = None, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The variable name used in the function code that represents the event.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
topic_endpoint_uri	str | None The name of an app setting that contains the URI for the custom topic. Default value: None
topic_key_setting	str | None The name of an app setting that contains an access key for the custom topic. Default value: None

Name	Description
connection	str | None The value of the common prefix for the setting that contains the topic endpoint URI. Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

event_grid_trigger

The *event_grid_trigger* decorator adds <xref:agent_framework.azure.EventGridTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/eventgridtrigger

Python

```
event_grid_trigger(arg_name: str, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The variable name used in the function code for the parameter that receives the event data.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value.

Name	Description
	Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

event_hub_message_trigger

The *event_hub_message_trigger* decorator adds
<xref:agent_framework.azure.EventHubTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-event-hubs

Python

```
event_hub_message_trigger(arg_name: str, connection: str, event_hub_name: str,
data_type: DataType | str | None = None, cardinality: Cardinality | str | None =
None, consumer_group: str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.EventHubEvent> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Event Hubs.

Name	Description
event_hub_name Required*	str The name of the Event Hub.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
cardinality	Cardinality | str | None Set to "many" to enable batching. Default value: None
consumer_group	str | None Optional. The consumer group used to subscribe to events in the hub. Default value: None
kwargs Required*	Any Additional keyword arguments to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

event_hub_output

The *event_hub_output* decorator adds <xref:agent_framework.azure.EventHubOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-event-hubs

Python

```
event_hub_output(arg_name: str, connection: str, event_hub_name: str, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable representing the Event Hub output object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Event Hub.
event_hub_name Required*	str The name of the Event Hub to send messages to.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

function_name

Optional: Sets name of the <xref:agent_framework.azure.Function> object. If not set, it will default to the name of the method name.

additional setting fields :return: Decorator function.

Python

```
function_name(name: str, setting_extra_fields: Dict[str, Any] | None = None) ->
Callable[[...], Any]
```

Parameters

Expand table

Name	Description
name Required*	str Name of the function.
setting_extra_fields	Dict[str, Any] | None Keyword arguments for specifying Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	

generic_input_binding

The *generic_input_binding* decorator adds <xref:agent_framework.azure.GenericInputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-custom

Python

```
generic_input_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the input parameter in the function code.

Name	Description
type Required*	str The type of the binding.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

generic_output_binding

The *generic_output_binding* decorator adds
<xref:agent_framework.azure.GenericOutputBinding> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-custom

Python

```
generic_output_binding(arg_name: str, type: str, data_type: DataType | str | None
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the output parameter in the function code.
type Required*	str The type of the binding.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

generic_trigger

The *generic_trigger* decorator adds <xref:agent_framework.azure.GenericTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-custom

Python

```
generic_trigger(arg_name: str, type: str, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the trigger parameter in the function code.
type Required*	str The type of the binding.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

get_agent

Return a DurableAIAgent proxy for a registered agent.

Python

```
get_agent(context: Any, agent_name: str) -> DurableAIAgent
```

Parameters

Expand table

Name	Description
context Required*	Any Durable Functions orchestration context invoking the agent.
agent_name Required*	str Name of the agent registered on this app.

Returns

Expand table

Type	Description
DurableAIAgent	DurableAIAgent wrapper bound to the orchestration context.

Exceptions

Expand table

Type	Description
ValueError	If the requested agent has not been registered.

get_functions

Get the function objects in the function app.

Python

```
get_functions() -> List[Function]
```

Returns

Expand table

Type	Description
List[Function]	A list of <xref:agent_framework.azure.Function> objects defined in the app.

http_type

Set http type of the <xref:agent_framework.azure.Function> object.

Python

```
http_type(http_type: str) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
http_type Required*	str Http type of the function.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

kafka_output

The *kafka_output* decorator adds <xref:agent_framework.azure.KafkaOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining a Kafka output binding in *function.json*, which enables the function to write events to a Kafka topic. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/kafkaoutput

Python

```
kafka_output(arg_name: str, topic: str, broker_list: str, avro_schema: str | None
= None, username: str | None = None, password: str | None = None,
ssl_key_location: str | None = None, ssl_ca_location: str | None = None,
ssl_certificate_location: str | None = None, ssl_key_password: str | None = None,
schema_registry_url: str | None = None, schema_registry_username: str | None =
None, schema_registry_password: str | None = None, o_auth_bearer_method:
OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None,
o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None =
None, o_auth_bearer_token_endpoint_url: str | None = None,
o_auth_bearer_extensions: str | None = None, max_message_bytes: int = 1_000_000,
batch_size: int = 10_000, enable_idempotence: bool = False, message_timeout_ms:
int = 300_000, request_timeout_ms: int = 5_000, max_retries: int = 2_147_483_647,
authentication_mode: BrokerAuthenticationMode | str | None = 'NOTSET', protocol:
BrokerProtocol | str | None = 'NOTSET', linger_ms: int = 5, data_type: DataType |
str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The variable name used in the function code that represents the output event.
topic Required*	str The Kafka topic to which messages are published.
broker_list Required*	str The list of Kafka brokers to which the producer connects.
avro_schema	str | None Optional. Avro schema to generate a generic record. Default value: None
username	str | None SASL username for use with the PLAIN and SASL-SCRAM mechanisms. Equivalent to *'sasl.username'* in librdkafka. Default value: None
password	str | None SASL password for use with the PLAIN and SASL-SCRAM mechanisms. Equivalent to *'sasl.password'* in librdkafka. Default value: None
ssl_key_location	str | None Path to the client's private key (PEM) for authentication. Equivalent to *'ssl.key.location'* in librdkafka. Default value: None
ssl_ca_location	str | None Path to the CA certificate for verifying the broker's certificate. Equivalent to *'ssl.ca.location'* in librdkafka. Default value: None
ssl_certificate_location	str | None Path to the client's certificate file. Equivalent to *'ssl.certificate.location'* in librdkafka. Default value: None
ssl_key_password	str | None Password for the client's SSL key. Equivalent to *'ssl.key.password'* in librdkafka. Default value: None
schema_registry_url	str | None URL of the Avro Schema Registry. Default value: None

Name	Description
schema_registry_username	str | None Username for accessing the Schema Registry. Default value: None
schema_registry_password	str | None Password for accessing the Schema Registry. Default value: None
o_auth_bearer_method	OAuthBearerMethod | str | None OAuth bearer method to use, e.g., '*default*' or '*oidc*'. Equivalent to '*sasl.oauthbearer.method*' in librdkafka. Default value: None
o_auth_bearer_client_id	str | None Used with *oidc* method. Equivalent to '*sasl.oauthbearer.client.id*' in librdkafka. Default value: None
o_auth_bearer_client_secret	str | None Used with *oidc* method. Equivalent to '*sasl.oauthbearer.client.secret*' in librdkafka. Default value: None
o_auth_bearer_scope	str | None Scope of the access request sent to the broker. Equivalent to '*sasl.oauthbearer.scope*' in librdkafka. Default value: None
o_auth_bearer_token_endpoint_url	str | None OAuth token endpoint. Equivalent to '*sasl.oauthbearer.token.endpoint.url*' in librdkafka. Default value: None
o_auth_bearer_extensions	str | None Additional metadata sent to the broker. Comma-separated key=value pairs, e.g., "*supportFeatureX=true,organizationId=sales-emea*". Equivalent to '*sasl.oauthbearer.extensions*' in librdkafka. Default value: None
max_message_bytes	int Maximum size (in bytes) of a transmitted message. Default is 1MB. Default value: 1_000_000
batch_size	int Maximum number of messages batched in one MessageSet. Default is 10,000. Default value: 10_000

Name	Description
enable_idempotence	bool If *True*, ensures messages are delivered exactly once and in order. Default is *False*. Default value: False
message_timeout_ms	int Local timeout for message delivery. Default is 300000 ms. Default value: 300_000
request_timeout_ms	int Timeout for producer request acknowledgment. Default is 5000 ms. Default value: 5_000
max_retries	int Maximum number of retry attempts for failed messages. Default is 2147483647. Retrying may cause reordering unless *enable_idempotence=True*. Default value: 2_147_483_647
authentication_mode	BrokerAuthenticationMode | str | None SASL mechanism used for authentication. Allowed values: *Gssapi*, *Plain*, *ScramSha256*, *ScramSha512*. Default is *Plain*. Equivalent to *'sasl.mechanism'* in librdkafka. Default value: "NOTSET"
protocol	BrokerProtocol | str | None Security protocol used for broker communication. Default is plaintext. Equivalent to *'security.protocol'* in librdkafka. Default value: "NOTSET"
linger_ms	int Time to wait between sending batches of messages to allow for better throughput via batching. Default value: 5
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

kafka_trigger

The *kafka_trigger* decorator adds <xref:agent_framework.azure.KafkaTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/kafkatrigger

Python

```
kafka_trigger(arg_name: str, topic: str, broker_list: str,
event_hub_connection_string: str | None = None, consumer_group: str | None = None,
avro_schema: str | None = None, username: str | None = None, password: str | None
= None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None,
ssl_certificate_location: str | None = None, ssl_key_password: str | None = None,
schema_registry_url: str | None = None, schema_registry_username: str | None =
None, schema_registry_password: str | None = None, o_auth_bearer_method:
OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None,
o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None =
None, o_auth_bearer_token_endpoint_url: str | None = None,
o_auth_bearer_extensions: str | None = None, authentication_mode:
BrokerAuthenticationMode | str | None = 'NotSet', protocol: BrokerProtocol | str |
None = 'NotSet', cardinality: Cardinality | str | None = 'One', lag_threshold: int
= 1000, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The variable name used in the function code for the parameter that receives the Kafka event data.
topic Required*	str The topic monitored by the trigger.

Name	Description
broker_list Required*	str The list of Kafka brokers monitored by the trigger.
event_hub_connection_string	str | None The name of an app setting that contains the connection string for Event Hub (when using Kafka protocol headers with Azure Event Hubs). Default value: None
consumer_group	str | None Kafka consumer group used by the trigger. Default value: None
avro_schema	str | None Used only if a generic Avro record should be generated. Default value: None
username	str | None SASL username for use with the PLAIN or SASL-SCRAM mechanisms. Equivalent to 'sasl.username' in librdkafka. Default is empty string. Default value: None
password	str | None SASL password for use with the PLAIN or SASL-SCRAM mechanisms. Equivalent to 'sasl.password' in librdkafka. Default is empty string. Default value: None
ssl_key_location	str | None Path to the client's private key (PEM) used for authentication. Equivalent to 'ssl.key.location' in librdkafka. Default value: None
ssl_ca_location	str | None Path to the CA certificate file for verifying the broker's certificate. Equivalent to 'ssl.ca.location' in librdkafka. Default value: None
ssl_certificate_location	str | None Path to the client's certificate. Equivalent to 'ssl.certificate.location' in librdkafka. Default value: None
ssl_key_password	str | None Password for the client's certificate. Equivalent to 'ssl.key.password' in librdkafka. Default value: None

Name	Description
schema_registry_url	str | None URL of the Avro Schema Registry. Default value: None
schema_registry_username	str | None Username for the Schema Registry. Default value: None
schema_registry_password	str | None Password for the Schema Registry. Default value: None
o_auth_bearer_method	OAuthBearerMethod | str | None Either 'default' or 'oidc'. Equivalent to 'sasl.oauthbearer.method' in librdkafka. Default value: None
o_auth_bearer_client_id	str | None Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.client.id' in librdkafka. Default value: None
o_auth_bearer_client_secret	str | None Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.client.secret' in librdkafka. Default value: None
o_auth_bearer_scope	str | None Specify only if *o_auth_bearer_method* is 'oidc'. Used to specify access scope. Equivalent to 'sasl.oauthbearer.scope'. Default value: None
o_auth_bearer_token_endpoint_url	str | None Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.token.endpoint.url' in librdkafka. Default value: None
o_auth_bearer_extensions	str | None Additional information for the broker, in the form of a comma-separated list of key=value pairs (e.g., "orgId=abc,flag=true"). Equivalent to 'sasl.oauthbearer.extensions' in librdkafka. Default value: None
authentication_mode	BrokerAuthenticationMode | str | None SASL mechanism to use. Allowed: Gssapi, Plain, ScramSha256, ScramSha512. Default: Plain. Equivalent to 'sasl.mechanism'. Default value: "NotSet"

Name	Description
protocol	BrokerProtocol | str | None Security protocol used to communicate with brokers. Default: plaintext. Equivalent to 'security.protocol'. Default value: "NotSet"
lag_threshold	int Max number of unprocessed messages per worker instance. Used in scaling logic to estimate needed worker instances. Default is 1000. Default value: 1000
data_type	DataType | str | None Defines how Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for extra binding fields in the binding JSON.
cardinality	Cardinality | str | None Default value: "One"

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

mcp_tool_trigger

The *mcp_tool_trigger* decorator adds <xref:agent_framework.azure.MCPToolTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: https://aka.ms/remote-mcp-functions-python

Python

```
mcp_tool_trigger(arg_name: str, tool_name: str, description: str | None = None,
tool_properties: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the trigger parameter in the function code.
tool_name Required*	str The logical tool name exposed to the host.
description	str | None Optional human-readable description of the tool. Default value: None
tool_properties	str | None JSON-serialized tool properties/parameters list. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

mysql_input

The *mysql_input* decorator adds <xref:agent_framework.azure.MySqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Python

```
mysql_input(arg_name: str, command_text: str, connection_string_setting: str,
command_type: str | None = 'Text', parameters: str | None = None, data_type:
DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> input object in the function code.
command_text Required*	str The SQL query string or the name of the stored procedure to execute.
connection_string_setting Required*	str The name of the app setting that contains the connection string to the MySQL database.
command_type	str | None The type of command being executed. Accepts "*Text*" for SQL queries or "*StoredProcedure*" for stored procedures. Default value: 'Text'
parameters	str | None One or more parameter values passed to the SQL command. Should be provided as a comma-separated string in the format: "*@param1=value1,@param2=value2*". Default value: None
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments used to define fields in the *function.json* binding.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

mysql_output

The *mysql_output* decorator adds <xref:agent_framework.azure.MySqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Python

```
mysql_output(arg_name: str, command_text: str, connection_string_setting: str,
data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the MySQL output object in the function code.
command_text Required*	str The SQL command or the name of the stored procedure executed by the binding.
connection_string_setting Required*	str The name of the app setting that contains the connection string for the target MySQL database.

Name	Description
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments used to define fields in the *function.json* binding.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

mysql_trigger

The *mysql_trigger* decorator adds <xref:agent_framework.azure.MySqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/mysqlbindings

Python

```
mysql_trigger(arg_name: str, table_name: str, connection_string_setting: str,
leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs)
-> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> object in the function

Name	Description
	code.
table_name Required*	str The name of the MySQL table monitored by the trigger.
connection_string_setting Required*	str The name of an app setting that contains the connection string for the database against which the query or stored procedure is executed.
leases_table_name	str | None The name of the table used to store leases. If not specified, the default table name is *Leases_{FunctionId}_{TableId}*. Default value: None
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

orchestration_trigger

Register an Orchestrator Function.

Python

```
orchestration_trigger(context_name: str, orchestration: str | None = None)
```

Parameters

Expand table

Name	Description
context_name Required*	str Parameter name of the DurableOrchestrationContext object.

Name	Description
orchestration	<xref:Optional>[str] Name of Orchestrator Function. The value is None by default, in which case the name of the method is used. Default value: None

queue_output

The *queue_output* decorator adds <xref:agent_framework.azure.QueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-queue

Python

```
queue_output(arg_name: str, queue_name: str, connection: str, data_type: DataType
| None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the storage queue output object in the function code.
queue_name Required*	str The name of the queue to which messages will be written.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Queues.
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None

Name	Description
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

queue_trigger

The *queue_trigger* decorator adds <xref:agent_framework.azure.QueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-queue

Python

```
queue_trigger(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.QueueMessage> object in the function code.
queue_name Required*	str The name of the queue to monitor.
connection	str

Name	Description
Required*	The name of an app setting or setting collection that specifies how to connect to Azure Queues.
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments to include in the binding JSON.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

register_blueprint

Register a list of functions in the function app.

Python

```
register_blueprint(function_container: DecoratorApi) -> None
```

Parameters

Expand table

Name	Description
function_container Required*	DecoratorApi Instance extending <xref:agent_framework.azure.DecoratorApi>, which contains a list of functions to be registered.

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
TypeError	If the provided container is an instance of
agent_framework.azure.FunctionRegister	

register_functions

Register a list of functions in the function app.

Python

```
register_functions(function_container: DecoratorApi) -> None
```

Parameters

Expand table

Name	Description
function_container Required*	DecoratorApi Instance extending <xref:agent_framework.azure.DecoratorApi>, which contains a list of functions to be registered.

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
TypeError	If the provided container is an instance of
agent_framework.azure.FunctionRegister	

retry

The *retry* decorator adds a <xref:agent_framework.azure.RetryPolicy> to the function settings object.

Used for building a <xref:agent_framework.azure.Function> object in the worker function indexing model. This is equivalent to defining a RetryPolicy in *function.json*, which enables the function to automatically retry on failure.

All optional fields are assigned default values by the function host during parsing.

Ref: https://aka.ms/azure_functions_retries

Python

```
retry(strategy: str, max_retry_count: str, delay_interval: str | None = None,
minimum_interval: str | None = None, maximum_interval: str | None = None,
setting_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
strategy Required*	str The retry strategy to use.
max_retry_count Required*	str The maximum number of retry attempts.
delay_interval	str | None The delay interval between retry attempts. Default value: None
minimum_interval	str | None The minimum delay interval between retry attempts. Default value: None
maximum_interval	str | None The maximum delay interval between retry attempts. Default value: None
setting_extra_fields	Dict[str, Any] | None Keyword arguments for specifying additional setting fields. Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

route

The *route* decorator adds <xref:agent_framework.azure.HttpTrigger> and <xref:agent_framework.azure.HttpOutput> bindings to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-http

Python

```
route(route: str | None = None, trigger_arg_name: str = 'req', binding_arg_name:
str = '$return', methods: Iterable[str] | Iterable[HttpMethod] | None = None,
auth_level: AuthLevel | str | None = None, trigger_extra_fields: Dict[str, Any] |
None = None, binding_extra_fields: Dict[str, Any] | None = None) ->
Callable[[...], Any]
```

Parameters

Expand table

Name	Description
route	str | None Route for the HTTP endpoint. If None, it defaults to the function name (if present) or the user-defined Python function name. Default value: None
trigger_arg_name	str Argument name for <xref:agent_framework.azure.HttpRequest>. Defaults to *'req'*. Default value: 'req'

Name	Description
binding_arg_name	str Argument name for <xref:agent_framework.azure.HttpResponse>. Defaults to '*$return*'. Default value: '$return'
methods	Iterable[str] | Iterable[HttpMethod] | None A tuple of the HTTP methods to which the function responds. Default value: None
auth_level	AuthLevel | str | None Determines what keys, if any, need to be present on the request in order to invoke the function. Default value: None
trigger_extra_fields	Dict[str, Any] | None Additional fields to include in the trigger JSON. For example: > data_type='STRING' # results in 'dataType': 'STRING' in the trigger JSON Default value: None
binding_extra_fields	Dict[str, Any] | None Additional fields to include in the binding JSON. For example: > data_type='STRING' # results in 'dataType': 'STRING' in the binding JSON Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

schedule

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-timer

Python

```
schedule(arg_name: str, schedule: str, run_on_startup: bool | None = None,
use_monitor: bool | None = None, data_type: DataType | str | None = None,
**kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.
schedule Required*	str A string representing a CRON expression used to schedule the function execution.
run_on_startup	bool | None If True, the function is invoked when the runtime starts. Default value: None
use_monitor	bool | None Set to True or False to indicate whether the schedule should be monitored. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

semantic_search_input

Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: https://platform.openai.com/docs/guides/embeddings

Python

```
semantic_search_input(arg_name: str, search_connection_name: str, collection: str, query: str | None = None, ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, system_prompt: str | None = semantic_search_system_prompt, max_knowledge_count: int | None = 1, temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
search_connection_name Required*	str The name of the app setting or environment variable that contains the vector search connection value.
collection Required*	str The name of the collection or table to search or store.
query	str | None The semantic query text used for searching the database. Default value: None
ai_connection_name	str | None

Name	Description
	The name of the configuration section for AI service connectivity settings. • For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section. • If not specified or the section is missing, defaults to the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*. • For user-assigned managed identity authentication, this field is required. • For OpenAI (non-Azure), set the *OPENAI_API_KEY* environment variable. Default value: ""
embeddings_model	str | OpenAIModels | None The deployment or model name for OpenAI Embeddings. Defaults to "text-embedding-ada-002". Default value: OpenAIModels.DefaultEmbeddingsModel
chat_model	str | OpenAIModels | None The deployment or model name for the OpenAI Chat Completion API. Defaults to "gpt-3.5-turbo". Default value: OpenAIModels.DefaultChatModel
system_prompt	str | None Optional. The system prompt provided to the large language model. Default value: semantic_search_system_prompt
max_knowledge_count	int | None Optional. The number of knowledge items to inject into the system prompt. Default is 1. Default value: 1
temperature	str | None The sampling temperature to use (range: 0 to 2). Higher values like 0.8 yield more random output; lower values like 0.2 make output more focused. Default value: "0.5"
top_p	str | None Alternative to temperature sampling (nucleus sampling). For example, 0.1 considers only the top 10% of tokens by probability mass. Use this or *temperature*. Default value: None
max_tokens	str | None The maximum number of tokens to generate in the completion. The sum of prompt tokens and *max_tokens* must not exceed the model's context length (usually 2048 or 4096 for newer models).

Name	Description
	Default value: "100"
is_reasoning_model	bool | None Indicates whether the chat model is a reasoning model. If True, *max_tokens* and *temperature* are not supported. Default value: False
data_type	DataType | str | None Optional. Defines how the Functions runtime should interpret the parameter value. Default is None. Default value: None
kwargs Required*	Additional keyword arguments for specifying extra fields in the *function.json* binding.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

service_bus_queue_output

The *service_bus_queue_output* decorator adds
<xref:agent_framework.azure.ServiceBusQueueOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Python

```
service_bus_queue_output(arg_name: str, connection: str, queue_name: str,
data_type: DataType | str | None = None, access_rights: AccessRights | str | None
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Service Bus queue output object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
queue_name Required*	str The name of the queue to which messages will be sent.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
access_rights	AccessRights | str | None The access rights required for the connection string. Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

service_bus_queue_trigger

The *on_service_bus_queue_change* decorator adds
<xref:agent_framework.azure.ServiceBusQueueTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusQueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Python

```
service_bus_queue_trigger(arg_name: str, connection: str, queue_name: str,
data_type: DataType | str | None = None, access_rights: AccessRights | str | None
= None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str |
None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
queue_name Required*	str The name of the queue to monitor.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
access_rights	AccessRights | str | None Access rights for the connection string. Default value: None
is_sessions_enabled	bool | None Set to True if connecting to a session-aware queue or subscription. Default value: None
cardinality	Cardinality | str | None Set to "many" to enable batching. Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

service_bus_topic_output

The *service_bus_topic_output* decorator adds <xref:agent_framework.azure.ServiceBusTopicOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusTopicOutput* in the *function.json*, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Python

```
service_bus_topic_output(arg_name: str, connection: str, topic_name: str,
subscription_name: str | None = None, data_type: DataType | str | None = None,
access_rights: AccessRights | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Service Bus topic output object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
topic_name Required*	str The name of the topic to which messages will be sent.
subscription_name	str | None The name of the subscription (optional for output). Default value: None
data_type	DataType | str | None

Name	Description
	Defines how the Functions runtime should treat the parameter value. Defaults to DataType.UNDEFINED. Default value: None
access_rights	AccessRights | str | None The access rights required for the connection string. Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

service_bus_topic_trigger

The *on_service_bus_topic_change* decorator adds
<xref:agent_framework.azure.ServiceBusTopicTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-service-bus

Python

```
service_bus_topic_trigger(arg_name: str, connection: str, topic_name: str,
subscription_name: str, data_type: DataType | str | None = None, access_rights:
AccessRights | str | None = None, is_sessions_enabled: bool | None = None,
cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...],
Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
topic_name Required*	str The name of the topic to monitor.
subscription_name Required*	str The name of the subscription to monitor.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
access_rights	AccessRights | str | None Access rights for the connection string. Default value: None
is_sessions_enabled	bool | None Set to True if connecting to a session-aware queue or subscription. Default value: None
cardinality	Cardinality | str | None Set to "many" to enable batching. Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

sql_input

The *sql_input* decorator adds <xref:agent_framework.azure.SqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a

<xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Python

```
sql_input(arg_name: str, command_text: str, connection_string_setting: str,
command_type: str | None = 'Text', parameters: str | None = None, data_type:
DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> input object in the function code.
command_text Required*	str The Transact-SQL query or the name of the stored procedure executed by the binding.
connection_string_setting Required*	str The name of the app setting that contains the connection string to the target SQL database.
command_type	str | None The command type. Use "*Text*" for a raw SQL query and "*StoredProcedure*" for a stored procedure. Default value: 'Text'
parameters	str | None A comma-separated string of parameter assignments to pass to the SQL command. Format: *@param1=value1,@param2=value2* Default value: None
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs	

Name	Description
Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

sql_output

The *sql_output* decorator adds <xref:agent_framework.azure.SqlOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Python

```
sql_output(arg_name: str, command_text: str, connection_string_setting: str,
data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the SQL output object in the function code.
command_text Required*	str The Transact-SQL query or the name of the stored procedure executed by the binding.
connection_string_setting	str

Name	Description
Required*	The name of the app setting that contains the connection string to the target SQL database.
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

sql_trigger

The *sql_trigger* decorator adds <xref:agent_framework.azure.SqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All optional fields are assigned default values by the function host when parsed.

Ref: https://aka.ms/sqlbindings

Python

```
sql_trigger(arg_name: str, table_name: str, connection_string_setting: str,
leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs)
-> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> object in the function code.
table_name Required*	str The name of the SQL table monitored by the trigger.
connection_string_setting Required*	str The name of an app setting that contains the connection string for the database against which the query or stored procedure is executed.
leases_table_name	str | None The name of the table used to store leases. If not specified, the default name is *Leases_{FunctionId}_{TableId}*. Default value: None
data_type	DataType | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

table_input

The *table_input* decorator adds <xref:agent_framework.azure.TableInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/tablesbindings

Python

```
table_input(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, take: int | None = None, filter: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the table or entity in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to the table service.
table_name Required*	str The name of the table.
row_key	str | None The row key of the table entity to read. Default value: None
partition_key	str | None The partition key of the table entity to read. Default value: None
take	int | None The maximum number of entities to return. Default value: None
filter	str | None An OData filter expression to apply when retrieving entities. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

table_output

The *table_output* decorator adds <xref:agent_framework.azure.TableOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableOutput* in the *function.json*, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: https://aka.ms/tablesbindings

Python

```
table_output(arg_name: str, connection: str, table_name: str, row_key: str | None
= None, partition_key: str | None = None, data_type: DataType | str | None = None)
-> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the table or entity in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to the table service.
table_name Required*	str The name of the table.
row_key	str | None The row key of the table entity to read. Default value: None
partition_key	str | None The partition key of the table entity to read. Default value: None
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

text_completion_input

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions

The examples typically define an HTTP-triggered function with a hardcoded prompt such as "*who is {name}?*", where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

Python

```
text_completion_input(arg_name: str, prompt: str, ai_connection_name: str | None =
'', chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel,
temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None
= '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
prompt Required*	str The text prompt to generate completions for.
ai_connection_name	str | None The name of the configuration section that defines AI service connectivity settings.

Name	Description
	• For **Azure OpenAI**: If specified, it looks for "*Endpoint*" and/or "*Key*" in this section. If not specified or missing, it falls back to the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*. • For **user-assigned managed identity**, this setting is required. • For **non-Azure OpenAI**, ensure the *OPENAI_API_KEY* environment variable is set. Default value: ""
model Required*	*(Deprecated)* Use *chat_model* instead. This parameter is unused and will be removed in future versions.
chat_model	str | OpenAIModels | None The deployment or model name to use for the Chat Completions API. Default is "*gpt-3.5-turbo*". Default value: OpenAIModels.DefaultChatModel
temperature	str | None Sampling temperature (0–2). Higher values (e.g., 0.8) produce more random output; lower values (e.g., 0.2) make output more focused and deterministic. Default value: "0.5"
top_p	str | None Controls nucleus sampling. For example, *top_p=0.1* considers only tokens in the top 10% cumulative probability. Recommended to use either *temperature* or *top_p*, but not both. Default value: None
max_tokens	str | None The maximum number of tokens to generate. The sum of prompt tokens and *max_tokens* must not exceed the model's context length (usually 2048 or 4096 for newer models). Default value: "100"
is_reasoning_model	bool | None If *True*, indicates that the configured chat model is a reasoning model. In this case, *max_tokens* and *temperature* are not supported. Default value: False
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

timer_trigger

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-timer

Python

```
timer_trigger(arg_name: str, schedule: str, run_on_startup: bool | None = None,
use_monitor: bool | None = None, data_type: DataType | str | None = None,
**kwargs: Any) -> Callable[[...], Any]
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.
schedule Required*	str A string representing a CRON expression used to schedule the function execution.
run_on_startup	bool | None If True, the function is invoked when the runtime starts. Default value: None
use_monitor	bool | None Set to True or False to indicate whether the schedule should be monitored. Default value: None

Name	Description
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
Callable[[...], Any]	Decorator function.

validate_function_names

Validate that all functions have unique names within the app.

The *functions_bindings* dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.

Python

```
validate_function_names(functions: List[Function])
```

Parameters

Expand table

Name	Description
functions Required*	List[Function] A list of <xref:agent_framework.azure.Function> objects to validate.

Exceptions

Expand table

Type	Description
ValueError	If any function name is duplicated.

warm_up_trigger

The *warm_up* decorator adds <xref:agent_framework.azure.WarmUpTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *WarmUpTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: https://aka.ms/azure-function-binding-warmup

Python

```
warm_up_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs)
-> Callable
```

Parameters

Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in the function code.
data_type	DataType | str | None Defines how the Functions runtime should treat the parameter value. Default value: None

Returns

Expand table

Type	Description
Callable	Decorator function.

Attributes

agents

Returns dict of agent names to agent instances.

Returns

Expand table

Type	Description
	Dictionary mapping agent names to their AgentProtocol instances.

app_script_file

Name of function app script file in which all the functions are defined.

Script file defined here is for placeholder purpose, please refer to worker defined script file path as the single point of truth.

Returns

Expand table

Type	Description
	Script file name.

auth_level

Authorization level of the function app. Will be applied to the http trigger functions which do not have authorization level specified.

Returns

Expand table

Type	Description
	Authorization level of the function app.

enable_health_check

Whether health check endpoint is enabled

Python

```
enable_health_check: bool
```

enable_http_endpoints

Whether HTTP endpoints are created for agents

Python

```
enable_http_endpoints: bool
```

enable_mcp_tool_trigger

Whether MCP tool triggers are created for agents

Python

```
enable_mcp_tool_trigger: bool
```

max_poll_retries

Maximum polling attempts when waiting for responses

poll_interval_seconds

Delay (seconds) between polling attempts

AgentResponseCallbackProtocol Class

Protocol describing the callbacks invoked during agent execution.

Constructor

Python

```
AgentResponseCallbackProtocol()
```

Methods

Expand table

__init__	
__new__	
on_agent_response	Handle the final agent response.
on_streaming_response_update	Handle a streaming response update emitted by the agent.

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

on_agent_response

Handle the final agent response.

Python

```
async on_agent_response(response: AgentRunResponse, context: AgentCallbackContext)
```

```
-> None
```

Parameters

Expand table

Name	Description
response Required*	AgentRunResponse
context Required*	AgentCallbackContext

Returns

Expand table

Type	Description
None	

on_streaming_response_update

Handle a streaming response update emitted by the agent.

Python

```
async on_streaming_response_update(update: AgentRunResponseUpdate, context:
AgentCallbackContext) -> None
```

Parameters

Expand table

Name	Description
update Required*	AgentRunResponseUpdate
context Required*	AgentCallbackContext

Returns

Expand table

Type	Description
None	

AzureAIAgentClient Class

Azure AI Agent Chat client.

Initialize an Azure AI Agent client.

Constructor

Python

```
AzureAIAgentClient()
```

Keyword-Only Parameters

Expand table

Name	Description
agents_client	An existing AgentsClient to use. If not provided, one will be created. Default value: None
agent_id	The ID of an existing agent to use. If not provided and agents_client is provided, a new agent will be created (and deleted after the request). If neither agents_client nor agent_id is provided, both will be created and managed automatically. Default value: None
agent_name	The name to use when creating new agents. Default value: None
agent_description	The description to use when creating new agents. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
project_endpoint	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a agents_client is passed. Default value: None
model_deployment_name	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
credential	Azure async credential to use for authentication.

Name	Description
	Default value: None
should_cleanup_agent	Whether to cleanup (delete) agents created by this client when the client is closed or context is exited. Defaults to True. Only affects agents created by this client instance; existing agents passed via agent_id are never deleted. Default value: True
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Examples

Python

```python
from agent_framework_azure_ai import AzureAIAgentClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIAgentClient(credential=credential)

# Or passing parameters directly
client = AzureAIAgentClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIAgentClient(credential=credential, env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an Azure AI Agent client.
__new__	
close	Close the agents_client and clean up any agents we created.

create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
from_settings	Initialize a AzureAIAgentClient from a dictionary of settings.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the service URL for the chat client.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Azure AI Agent client.

Python

```
__init__(*, agents_client: AgentsClient | None = None, agent_id: str | None =
None, agent_name: str | None = None, agent_description: str | None = None,
thread_id: str | None = None, project_endpoint: str | None = None,
model_deployment_name: str | None = None, credential: AsyncTokenCredential | None
= None, should_cleanup_agent: bool = True, env_file_path: str | None = None,
env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
agents_client Required*	<xref:azure.ai.agents.aio._patch.AgentsClient> | None
agent_id Required*	str | None
agent_name Required*	str | None
agent_description Required*	str | None
thread_id Required*	str | None
project_endpoint Required*	str | None
model_deployment_name Required*	str | None
credential Required*	AsyncTokenCredential | None
should_cleanup_agent Required*	bool
env_file_path Required*	str | None
env_file_encoding Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
agents_client	An existing AgentsClient to use. If not provided, one will be created. Default value: None
agent_id	The ID of an existing agent to use. If not provided and agents_client is provided, a new agent will be created (and deleted after the request). If neither agents_client nor agent_id is provided, both will be created and managed automatically. Default value: None
agent_name	The name to use when creating new agents. Default value: None
agent_description	The description to use when creating new agents. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
project_endpoint	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a agents_client is passed. Default value: None
model_deployment_name	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
credential	Azure async credential to use for authentication. Default value: None
should_cleanup_agent	Whether to cleanup (delete) agents created by this client when the client is closed or context is exited. Defaults to True. Only affects agents created by this client instance; existing agents passed via agent_id are never deleted. Default value: True
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Returns

Expand table

Type	Description
None	

Examples

Python

```
   from agent_framework_azure_ai import AzureAIAgentClient
   from azure.identity.aio import DefaultAzureCredential

   # Using environment variables
   # Set AZURE_AI_PROJECT_ENDPOINT=https://your-
project.cognitiveservices.azure.com
   # Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
   credential = DefaultAzureCredential()
   client = AzureAIAgentClient(credential=credential)

   # Or passing parameters directly
   client = AzureAIAgentClient(
       project_endpoint="https://your-project.cognitiveservices.azure.com",
       model_deployment_name="gpt-4",
       credential=credential,
   )

   # Or loading from a .env file
   client = AzureAIAgentClient(credential=credential,
env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Close the agents_client and clean up any agents we created.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id	str | None

Name	Description
Required*	
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None

Name	Description
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent.

Name	Description
	Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request.

Name	Description
	Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
```

```
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
```

```
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

from_settings

Initialize a AzureAIAgentClient from a dictionary of settings.

Python

```
from_settings(settings: dict[str, Any]) -> TAzureAIAgentClient
```

Parameters

Expand table

Name	Description
settings Required*	dict[str, Any] A dictionary of settings for the service.

Returns

Expand table

Type	Description
<xref:agent_framework_azure_ai._chat_client.TAzureAIAgentClient>	

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None

Name	Description
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None

Name	Description
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
```

```
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature	float | None

Name	Description
Required*	
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response.

Name	Description
	Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the service URL for the chat client.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL for the chat client, or None if not set.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None

Name	Description
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai'
```

AzureAIClient Class

Azure AI Agent client.

Initialize an Azure AI Agent client.

Constructor

Python

```
AzureAIClient()
```

Keyword-Only Parameters

Expand table

Name	Description
project_client	An existing AIProjectClient to use. If not provided, one will be created. Default value: None
agent_name	The name to use when creating new agents or using existing agents. Default value: None
agent_version	The version of the agent to use. Default value: None
agent_description	The description to use when creating new agents. Default value: None
conversation_id	Default conversation ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
project_endpoint	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a project_client is passed. Default value: None
model_deployment_name	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
credential	Azure async credential to use for authentication. Default value: None
use_latest_version	

Name	Description
	Boolean flag that indicates whether to use latest agent version if it exists in the service. Default value: None
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Examples

Python

```python
from agent_framework.azure import AzureAIClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIClient(credential=credential)

# Or passing parameters directly
client = AzureAIClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIClient(credential=credential, env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an Azure AI Agent client.
__new__	
close	Close the project_client.
configure_azure_monitor	Setup observability with Azure Monitor (Azure AI Foundry integration).

	This method configures Azure Monitor for telemetry collection using the connection string from the Azure AI project client. **Note** This method retrieves the Application Insights connection string from the Azure AI project client automatically. You must have Application Insights configured in your Azure AI project for this to work.
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary.

	Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Azure AI Agent client.

Python

```
__init__(*, project_client: AIProjectClient | None = None, agent_name: str | None
= None, agent_version: str | None = None, agent_description: str | None = None,
conversation_id: str | None = None, project_endpoint: str | None = None,
model_deployment_name: str | None = None, credential: AsyncTokenCredential | None
= None, use_latest_version: bool | None = None, env_file_path: str | None = None,
env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
project_client Required*	<xref:azure.ai.projects.aio._patch.AIProjectClient> | None
agent_name Required*	str | None
agent_version Required*	str | None
agent_description Required*	str | None
conversation_id Required*	str | None
project_endpoint Required*	str | None
model_deployment_name Required*	str | None

Name	Description
credential Required*	AsyncTokenCredential | None
use_latest_version Required*	bool | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
project_client	An existing AIProjectClient to use. If not provided, one will be created. Default value: None
agent_name	The name to use when creating new agents or using existing agents. Default value: None
agent_version	The version of the agent to use. Default value: None
agent_description	The description to use when creating new agents. Default value: None
conversation_id	Default conversation ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
project_endpoint	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a project_client is passed. Default value: None
model_deployment_name	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
credential	Azure async credential to use for authentication. Default value: None

Name	Description
use_latest_version	Boolean flag that indicates whether to use latest agent version if it exists in the service. Default value: None
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureAIClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-
project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIClient(credential=credential)

# Or passing parameters directly
client = AzureAIClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIClient(credential=credential, env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Close the project_client.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

configure_azure_monitor

Setup observability with Azure Monitor (Azure AI Foundry integration).

This method configures Azure Monitor for telemetry collection using the connection string from the Azure AI project client.

> **Note**
>
> This method retrieves the Application Insights connection string from the
>
> Azure AI project client automatically. You must have Application Insights
>
> configured in your Azure AI project for this to work.

Python

```
async configure_azure_monitor(enable_sensitive_data: bool = False, **kwargs: Any)
-> None
```

Parameters

Expand table

Name	Description
enable_sensitive_data	bool Enable sensitive data logging (prompts, responses). Should only be enabled in development/test environments. Default is False. Default value: False
****kwargs** Required*	Any Additional arguments passed to configure_azure_monitor(). Common options include: • enable_live_metrics (bool): Enable Azure Monitor Live Metrics • credential (TokenCredential): Azure credential for Entra ID auth • resource (Resource): Custom OpenTelemetry resource See https://learn.microsoft.com/python/api/azure-monitor-opentelemetry/azure.monitor.opentelemetry.configure_azure_monitor for full list of options.

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ImportError	If azure-monitor-opentelemetry-exporter is not installed.

Examples

Python

```
from agent_framework.azure import AzureAIClient
from azure.ai.projects.aio import AIProjectClient
from azure.identity.aio import DefaultAzureCredential

async with (
    DefaultAzureCredential() as credential,
```

```
        AIProjectClient(
            endpoint="https://your-project.api.azureml.ms", credential=credential
        ) as project_client,
        AzureAIClient(project_client=project_client) as client,
    ):
        # Setup observability with defaults
        await client.configure_azure_monitor()

        # With live metrics enabled
        await client.configure_azure_monitor(enable_live_metrics=True)

        # With sensitive data logging (dev/test only)
        await client.configure_azure_monitor(enable_sensitive_data=True)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None

Name	Description
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient
```

```
    # Create a client
    client = OpenAIChatClient(model_id="gpt-4")

    # Create an agent using the convenience method
    agent = client.create_agent(
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
```

```
    client_data = {
        "type": "open_ai_chat_client",
        "model_id": "gpt-4o-mini",
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
```

```python
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value	

Name	Description
Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None

Name	Description
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None

Name	Description
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`allow_multiple_tool_calls`	Whether to allow multiple tool calls in a single response. Default value: None
`frequency_penalty`	The frequency penalty to use. Default value: None
`logit_bias`	The logit bias to use. Default value: None
`max_tokens`	The maximum number of tokens to generate. Default value: None
`metadata`	Additional metadata to include in the request. Default value: None
`model_id`	The model_id to use for the agent. Default value: None
`presence_penalty`	The presence penalty to use. Default value: None
`response_format`	The format of the response. Default value: None
`seed`	The random seed to use. Default value: None
`stop`	The stop sequence(s) for the request. Default value: None
`store`	Whether to store the response. Default value: None
`temperature`	The sampling temperature to use. Default value: None
`tool_choice`	The tool choice for the request. Default is *auto*. Default value: None
`tools`	The tools to use for the request. Default value: None
`top_p`	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

FILE_SEARCH_MAX_RESULTS

Python

```
FILE_SEARCH_MAX_RESULTS: int = 50
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai'
```

AzureAISearchContextProvider Class

Azure AI Search Context Provider with hybrid search and semantic ranking.

This provider retrieves relevant documents from Azure AI Search to provide context to the AI agent. It supports two modes:

- **agentic**: Recommended for most scenarios. Uses Knowledge Bases for query planning and multi-hop reasoning. Slightly slower with more token consumption, but provides more accurate results (up to 36% improvement in response relevance).
- **semantic** (default): Fast hybrid search combining vector and keyword search with semantic reranking. Best for simple queries where speed is critical.

Initialize Azure AI Search Context Provider.

Constructor

Python

```
AzureAISearchContextProvider()
```

Parameters

Expand table

Name	Description
endpoint	Azure AI Search endpoint URL. Can also be set via environment variable AZURE_SEARCH_ENDPOINT. Default value: None
index_name	Name of the search index to query. Can also be set via environment variable AZURE_SEARCH_INDEX_NAME. Default value: None
api_key	API key for authentication (string or AzureKeyCredential). Can also be set via environment variable AZURE_SEARCH_API_KEY. Default value: None
credential	AsyncTokenCredential for managed identity authentication. Use this for Entra ID authentication instead of api_key. Default value: None
mode	

Name	Description
Required*	Search mode - "semantic" for hybrid search with semantic ranking (fast) or "agentic" for multi-hop reasoning (slower). Default: "semantic".
top_k Required*	Maximum number of documents to retrieve. Only applies to semantic mode. In agentic mode, the server-side Knowledge Base determines retrieval based on query complexity and reasoning effort. Default: 5.
semantic_configuration_name Required*	Name of semantic configuration in the index. Required for semantic ranking. If None, uses index default.
vector_field_name Required*	Name of the vector field in the index for hybrid search. Required if using vector search. Default: None (keyword search only).
embedding_function Required*	Async function to generate embeddings for vector search. Signature: async def embed(text: str) -> list[float] Required if vector_field_name is specified and no server-side vectorization.
context_prompt Required*	Custom prompt to prepend to retrieved context. Default: "Use the following context to answer the question:"
azure_openai_resource_url Required*	Azure OpenAI resource URL for Knowledge Base model calls. Required when using agentic mode with index_name (to auto-create Knowledge Base). Not required when using an existing knowledge_base_name. Example: "https://myresource.openai.azure.com "
model_deployment_name Required*	Model deployment name in Azure OpenAI for Knowledge Base. Required when using agentic mode with index_name (to auto-create Knowledge Base). Not required when using an existing knowledge_base_name.
model_name Required*	The underlying model name (e.g., "gpt-4o", "gpt-4o-mini"). If not provided, defaults to model_deployment_name. Used for Knowledge Base configuration.
knowledge_base_name Required*	Name of an existing Knowledge Base to use. Required for agentic mode if not providing index_name. Supports KBs with any source type (web, blob, index, etc.).
retrieval_instructions Required*	Custom instructions for the Knowledge Base's retrieval planning. Only used in agentic mode.
azure_openai_api_key Required*	Azure OpenAI API key for Knowledge Base to call the model. Only needed when using API key authentication instead of managed identity.
knowledge_base_output_mode Required*	Output mode for Knowledge Base retrieval. Only used in agentic mode. "extractive_data": Returns raw chunks without synthesis (default, recommended for agent integration). "answer_synthesis":

Name	Description
	Returns synthesized answer from the LLM. Some knowledge sources require answer_synthesis mode. Default: "extractive_data".
retrieval_reasoning_effort Required*	Reasoning effort for Knowledge Base query planning. Only used in agentic mode. "minimal": Fastest, basic query planning. "medium": Moderate reasoning with some query decomposition. "low": Lower reasoning effort than medium. Default: "minimal".
agentic_message_history_count Required*	Number of recent messages from conversation history to send to the Knowledge Base. This context helps with query planning in agentic mode, allowing the Knowledge Base to understand the conversation flow and generate better retrieval queries. There is no technical limit - adjust based on your use case. Default: 10.
env_file_path Required*	Path to environment file for loading settings.
env_file_encoding Required*	Encoding of the environment file.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: semantic
top_k	Default value: 5
semantic_configuration_name	Default value: None
vector_field_name	Default value: None
embedding_function	Default value: None
context_prompt	Default value: None
azure_openai_resource_url	Default value: None
model_deployment_name	Default value: None
model_name	Default value: None
knowledge_base_name	Default value: None
retrieval_instructions	Default value: None
azure_openai_api_key	Default value: None

Name	Description
knowledge_base_output_mode	Default value: extractive_data
retrieval_reasoning_effort	Default value: minimal
agentic_message_history_count	Default value: 10
env_file_path	Default value: None
env_file_encoding	Default value: None

Examples

Python

```
from agent_framework_aisearch import AzureAISearchContextProvider
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net
# Set AZURE_SEARCH_INDEX_NAME=my-index
credential = DefaultAzureCredential()
provider = AzureAISearchContextProvider(credential=credential)

# Or passing parameters directly
provider = AzureAISearchContextProvider(
    endpoint="https://mysearch.search.windows.net",
    index_name="my-index",
    credential=credential,
)

# Or loading from a .env file
provider = AzureAISearchContextProvider(credential=credential,
env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize Azure AI Search Context Provider.
__new__	
invoked	Called after the agent has received a response from the underlying inference service. You can inspect the request and response messages, and update the state of the context provider.

invoking	Retrieve relevant context from Azure AI Search before model invocation.
thread_created	Called just after a new thread is created. Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

__init__

Initialize Azure AI Search Context Provider.

Python

```
__init__(endpoint: str | None = None, index_name: str | None = None, api_key: str
| AzureKeyCredential | None = None, credential: AsyncTokenCredential | None =
None, *, mode: Literal['semantic', 'agentic'] = 'semantic', top_k: int = 5,
semantic_configuration_name: str | None = None, vector_field_name: str | None =
None, embedding_function: Callable[[str], Awaitable[list[float]]] | None = None,
context_prompt: str | None = None, azure_openai_resource_url: str | None = None,
model_deployment_name: str | None = None, model_name: str | None = None,
knowledge_base_name: str | None = None, retrieval_instructions: str | None = None,
azure_openai_api_key: str | None = None, knowledge_base_output_mode:
Literal['extractive_data', 'answer_synthesis'] = 'extractive_data',
retrieval_reasoning_effort: Literal['minimal', 'medium', 'low'] = 'minimal',
agentic_message_history_count: int = _DEFAULT_AGENTIC_MESSAGE_HISTORY_COUNT,
env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

Expand table

Name	Description
endpoint	str | None Azure AI Search endpoint URL. Can also be set via environment variable AZURE_SEARCH_ENDPOINT. Default value: None
index_name	str | None Name of the search index to query. Can also be set via environment variable AZURE_SEARCH_INDEX_NAME. Default value: None
api_key	str | AzureKeyCredential | None API key for authentication (string or AzureKeyCredential). Can also be set via environment variable AZURE_SEARCH_API_KEY. Default value: None

Name	Description
credential	AsyncTokenCredential | None AsyncTokenCredential for managed identity authentication. Use this for Entra ID authentication instead of api_key. Default value: None
mode Required*	Literal['semantic', 'agentic'] Search mode - "semantic" for hybrid search with semantic ranking (fast) or "agentic" for multi-hop reasoning (slower). Default: "semantic".
top_k Required*	int Maximum number of documents to retrieve. Only applies to semantic mode. In agentic mode, the server-side Knowledge Base determines retrieval based on query complexity and reasoning effort. Default: 5.
semantic_configuration_name Required*	str | None Name of semantic configuration in the index. Required for semantic ranking. If None, uses index default.
vector_field_name Required*	str | None Name of the vector field in the index for hybrid search. Required if using vector search. Default: None (keyword search only).
embedding_function Required*	Callable[[str], Awaitable[list[float]]] | None Async function to generate embeddings for vector search. Signature: async def embed(text: str) -> list[float] Required if vector_field_name is specified and no server-side vectorization.
context_prompt Required*	str | None Custom prompt to prepend to retrieved context. Default: "Use the following context to answer the question:"
azure_openai_resource_url Required*	str | None Azure OpenAI resource URL for Knowledge Base model calls. Required when using agentic mode with index_name (to auto-create Knowledge Base). Not required when using an existing knowledge_base_name. Example: "https://myresource.openai.azure.com "
model_deployment_name Required*	str | None Model deployment name in Azure OpenAI for Knowledge Base. Required when using agentic mode with index_name (to auto-create Knowledge Base). Not required when using an existing knowledge_base_name.
model_name Required*	str | None The underlying model name (e.g., "gpt-4o", "gpt-4o-mini"). If not provided, defaults to model_deployment_name. Used for

Name	Description
	Knowledge Base configuration.
knowledge_base_name Required*	str | None Name of an existing Knowledge Base to use. Required for agentic mode if not providing index_name. Supports KBs with any source type (web, blob, index, etc.).
retrieval_instructions Required*	str | None Custom instructions for the Knowledge Base's retrieval planning. Only used in agentic mode.
azure_openai_api_key Required*	str | None Azure OpenAI API key for Knowledge Base to call the model. Only needed when using API key authentication instead of managed identity.
knowledge_base_output_mode Required*	Literal['extractive_data', 'answer_synthesis'] Output mode for Knowledge Base retrieval. Only used in agentic mode. "extractive_data": Returns raw chunks without synthesis (default, recommended for agent integration). "answer_synthesis": Returns synthesized answer from the LLM. Some knowledge sources require answer_synthesis mode. Default: "extractive_data".
retrieval_reasoning_effort Required*	Literal['minimal', 'medium', 'low'] Reasoning effort for Knowledge Base query planning. Only used in agentic mode. "minimal": Fastest, basic query planning. "medium": Moderate reasoning with some query decomposition. "low": Lower reasoning effort than medium. Default: "minimal".
agentic_message_history_count Required*	int Number of recent messages from conversation history to send to the Knowledge Base. This context helps with query planning in agentic mode, allowing the Knowledge Base to understand the conversation flow and generate better retrieval queries. There is no technical limit - adjust based on your use case. Default: 10.
env_file_path Required*	str | None Path to environment file for loading settings.
env_file_encoding Required*	str | None Encoding of the environment file.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: "semantic"
top_k	Default value: 5
semantic_configuration_name	Default value: None
vector_field_name	Default value: None
embedding_function	Default value: None
context_prompt	Default value: None
azure_openai_resource_url	Default value: None
model_deployment_name	Default value: None
model_name	Default value: None
knowledge_base_name	Default value: None
retrieval_instructions	Default value: None
azure_openai_api_key	Default value: None
knowledge_base_output_mode	Default value: "extractive_data"
retrieval_reasoning_effort	Default value: "minimal"
agentic_message_history_count	Default value: _DEFAULT_AGENTIC_MESSAGE_HISTORY_COUNT
env_file_path	Default value: None
env_file_encoding	Default value: None

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework_aisearch import AzureAISearchContextProvider
from azure.identity.aio import DefaultAzureCredential
```

```
    # Using environment variables
    # Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net
    # Set AZURE_SEARCH_INDEX_NAME=my-index
    credential = DefaultAzureCredential()
    provider = AzureAISearchContextProvider(credential=credential)

    # Or passing parameters directly
    provider = AzureAISearchContextProvider(
        endpoint="https://mysearch.search.windows.net",
        index_name="my-index",
        credential=credential,
    )

    # Or loading from a .env file
    provider = AzureAISearchContextProvider(credential=credential,
env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

invoked

Called after the agent has received a response from the underlying inference service.

You can inspect the request and response messages, and update the state of the context provider.

Python

```
async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
request_messages Required*	ChatMessage | Sequence[ChatMessage] The messages that were sent to the model/agent.
response_messages	ChatMessage | Sequence[ChatMessage] | None

Name	Description
	The messages that were returned by the model/agent. Default value: None
invoke_exception	Exception | None The exception that was thrown, if any. Default value: None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments (not used at present).

Returns

Expand table

Type	Description
None	

invoking

Retrieve relevant context from Azure AI Search before model invocation.

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs:
Any) -> Context
```

Parameters

Expand table

Name	Description
messages Required*	ChatMessage | MutableSequence[ChatMessage] User messages to use for context retrieval.

Name	Description
**kwargs Required*	Any Additional arguments (unused).

Returns

Expand table

Type	Description
Context	Context object with retrieved documents as messages.

thread_created

Called just after a new thread is created.

Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

Python

```
async thread_created(thread_id: str | None) -> None
```

Parameters

Expand table

Name	Description
thread_id Required*	str | None The ID of the new thread.

Returns

Expand table

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories
when answering user questions:'
```

AzureAISearchSettings Class

Settings for Azure AI Search Context Provider with auto-loading from environment.

The settings are first loaded from environment variables with the prefix '>>AZURE_SEARCH_<<'. If the environment variables are not found, the settings can be loaded from a .env file.

Initialize the settings class.

JSON


```
{
   "title": "AzureAISearchSettings",
   "description": "Settings for Azure AI Search Context Provider with auto-loading
from environment.\n\nThe settings are first loaded from environment variables with the
prefix 'AZURE_SEARCH_'.\nIf the environment variables are not found, the settings can
be loaded from a .env file.\n\nKeyword Args:\n    endpoint: Azure AI Search endpoint
URL.\n        Can be set via environment variable AZURE_SEARCH_ENDPOINT.\n
index_name: Name of the search index.\n        Can be set via environment variable
AZURE_SEARCH_INDEX_NAME.\n    knowledge_base_name: Name of an existing Knowledge Base
(for agentic mode).\n        Can be set via environment variable
AZURE_SEARCH_KNOWLEDGE_BASE_NAME.\n    api_key: API key for authentication (optional,
use managed identity if not provided).\n        Can be set via environment variable
AZURE_SEARCH_API_KEY.\n    env_file_path: If provided, the .env settings are read from
this file path location.\n    env_file_encoding: The encoding of the .env file,
defaults to 'utf-8'.\n\nExamples:\n    .. code-block:: python\n\n        from
agent_framework_aisearch import AzureAISearchSettings\n\n        # Using environment
variables\n        # Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net\n
# Set AZURE_SEARCH_INDEX_NAME=my-index\n        settings = AzureAISearchSettings()\n\n
# Or passing parameters directly\n        settings = AzureAISearchSettings(\n
endpoint=\"https://mysearch.search.windows.net\",\n            index_name=\"my-
index\",\n        )\n\n        # Or loading from a .env file\n        settings =
AzureAISearchSettings(env_file_path=\"path/to/.env\")",
   "type": "object",
   "properties": {
      "env_file_path": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Env File Path"
      },
      "env_file_encoding": {
         "anyOf": [
```

```
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": "utf-8",
         "title": "Env File Encoding"
      },
      "endpoint": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Endpoint"
      },
      "index_name": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Index Name"
      },
      "knowledge_base_name": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Knowledge Base Name"
      },
      "api_key": {
         "anyOf": [
            {
               "format": "password",
               "type": "string",
               "writeOnly": true
            },
            {
               "type": "null"
```

```
            }
        ],
        "default": null,
        "title": "Api Key"
    }
  }
}
```

Constructor

Python

```
pydantic settings AzureAISearchSettings
```

Keyword-Only Parameters

Expand table

Name	Description
endpoint	Azure AI Search endpoint URL. Can be set via environment variable AZURE_SEARCH_ENDPOINT. Default value: None
index_name	Name of the search index. Can be set via environment variable AZURE_SEARCH_INDEX_NAME. Default value: None
knowledge_base_name	Name of an existing Knowledge Base (for agentic mode). Can be set via environment variable AZURE_SEARCH_KNOWLEDGE_BASE_NAME. Default value: None
api_key	API key for authentication (optional, use managed identity if not provided). Can be set via environment variable AZURE_SEARCH_API_KEY. Default value: None
env_file_path	If provided, the .env settings are read from this file path location. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python

```
from agent_framework_aisearch import AzureAISearchSettings
```

```python
# Using environment variables
# Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net
# Set AZURE_SEARCH_INDEX_NAME=my-index
settings = AzureAISearchSettings()

# Or passing parameters directly
settings = AzureAISearchSettings(
    endpoint="https://mysearch.search.windows.net",
    index_name="my-index",
)

# Or loading from a .env file
settings = AzureAISearchSettings(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize the settings class.
__new__	Override the **new** method to set the env_prefix.
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed

	values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	

schema
schema_json
update_forward_refs
validate

__init__

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

Expand table

Name	Description
cls Required*	type[<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].

Name	Description
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • '*any_of*': Use the *anyOf* keyword to combine schemas (the default). • '*primitive_type_array*': Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
`union_format`	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
`params` Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias	bool | None

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
`json_data` Required*	str | bytes | bytearray The JSON data to validate.
`strict` Required*	bool | None Whether to enforce types strictly.
`extra` Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`context` Required*	Any | None Extra variables to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name	bool | None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj	Any

Name	Description
Required*	

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

```
Python
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
Python
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

api_key

Python

```
field api_key: SecretStr | None = None
```

endpoint

Python

```
field endpoint: str | None = None
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'AZURE_SEARCH_'
```

index_name

Python

```
field index_name: str | None = None
```

knowledge_base_name

Python

```
field knowledge_base_name: str | None = None
```

AzureAISettings Class

Azure AI Project settings.

The settings are first loaded from environment variables with the prefix '>>AZURE_AI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON


```
{
   "title": "AzureAISettings",
   "description": "Azure AI Project settings.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_AI_'.\nIf the environment variables are not found, the settings can be loaded from a .env file\nwith the encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare ignored; however, validation will fail alerting that the settings are missing.\n\nKeyword Args:\n project_endpoint: The Azure AI Project endpoint URL.\n        Can be set via environment variable AZURE_AI_PROJECT_ENDPOINT.\n    model_deployment_name: The name of the model deployment to use.\n        Can be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME.\n    env_file_path: If provided, the .env settings are read from this file path location.\n    env_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\nExamples:\n    .. code-block:: python\n\n        from agent_framework.azure import AzureAISettings\n\n        # Using environment variables\n        # Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com\n        # Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4\n        settings = AzureAISettings()\n\n        # Or passing parameters directly\n settings = AzureAISettings(\n            project_endpoint=\"https://your-project.cognitiveservices.azure.com\", model_deployment_name=\"gpt-4\"\n        )\n\n # Or loading from a .env file\n        settings = AzureAISettings(env_file_path=\"path/to/.env\")",
   "type": "object",
   "properties": {
      "env_file_path": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Env File Path"
      },
```

```
        "env_file_encoding": {
           "anyOf": [
              {
                 "type": "string"
              },
              {
                 "type": "null"
              }
           ],
           "default": "utf-8",
           "title": "Env File Encoding"
        },
        "project_endpoint": {
           "anyOf": [
              {
                 "type": "string"
              },
              {
                 "type": "null"
              }
           ],
           "default": null,
           "title": "Project Endpoint"
        },
        "model_deployment_name": {
           "anyOf": [
              {
                 "type": "string"
              },
              {
                 "type": "null"
              }
           ],
           "default": null,
           "title": "Model Deployment Name"
        }
     }
  }
```


Constructor

Python

```
pydantic settings AzureAISettings
```

Keyword-Only Parameters

Expand table

Name	Description
project_endpoint	The Azure AI Project endpoint URL. Can be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Default value: None
model_deployment_name	The name of the model deployment to use. Can be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
env_file_path	If provided, the .env settings are read from this file path location. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python

```
from agent_framework.azure import AzureAISettings

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
settings = AzureAISettings()

# Or passing parameters directly
settings = AzureAISettings(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
model_deployment_name="gpt-4"
)

# Or loading from a .env file
settings = AzureAISettings(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize the settings class.
__new__	Override the **new** method to set the env_prefix.
construct	
copy	Returns a copy of the model.

	!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.

model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

Expand table

Name	Description
cls Required*	type[<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None

Name	Description
Required*	Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Name	Description
`encoder`	Default value: PydanticUndefined
`models_as_dict`	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
`_fields_set`	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None

Name	Description
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`mode`	Default value: 'python'
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults	bool

Name	Description
Required*	Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format	Literal['any_of', 'primitive_type_array']

Name	Description
Required*	The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default). • *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors	bool

Name	Description
Required*	Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str	bytes	bytearray The JSON data to validate.
strict Required*	bool	None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid']	None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any	None Extra variables to pass to the validator.
by_alias Required*	bool	None Whether to use the field's alias when validating against the provided input data.
by_name	bool	None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

```
Python
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

```
Python
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes

Name	Description
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True

Name	Description
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

Python

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
`value` Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'AZURE_AI_'
```

model_deployment_name

Python

```
field model_deployment_name: str | None = None
```

project_endpoint

Python

```
field project_endpoint: str | None = None
```

AzureOpenAIAssistantsClient Class

Azure OpenAI Assistants client.

Initialize an Azure OpenAI Assistants client.

Constructor

Python

```
AzureOpenAIAssistantsClient()
```

Keyword-Only Parameters

Expand table

Name	Description
deployment_name	The Azure OpenAI deployment name for the model to use. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
assistant_id	The ID of an Azure OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None
assistant_name	The name to use when creating new assistants. Default value: None
assistant_description	The description to use when creating new assistants. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	

Name	Description
	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential to use for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None

Examples

Python

```python
from agent_framework.azure import AzureOpenAIAssistantsClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIAssistantsClient()

# Or passing parameters directly
client = AzureOpenAIAssistantsClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
```

```
api_key="your-key"
    )

    # Or loading from a .env file
    client = AzureOpenAIAssistantsClient(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an Azure OpenAI Assistants client.
__new__	
close	Clean up any assistants we created.
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Azure OpenAI Assistants client.

Python

```
__init__(*, deployment_name: str | None = None, assistant_id: str | None = None,
assistant_name: str | None = None, assistant_description: str | None = None,
thread_id: str | None = None, api_key: str | None = None, endpoint: str | None =
None, base_url: str | None = None, api_version: str | None = None, ad_token: str |
None = None, ad_token_provider: Callable[[], str | Awaitable[str]] | None = None,
token_endpoint: str | None = None, credential: TokenCredential | None = None,
default_headers: Mapping[str, str] | None = None, async_client: AsyncAzureOpenAI |
None = None, env_file_path: str | None = None, env_file_encoding: str | None =
None) -> None
```

Parameters

Expand table

Name	Description
deployment_name Required*	str | None
assistant_id Required*	str | None
assistant_name Required*	str | None
assistant_description Required*	str | None
thread_id Required*	str | None
api_key	str | None

Name	Description
Required*	
endpoint Required*	str | None
base_url Required*	str | None
api_version Required*	str | None
ad_token Required*	str | None
ad_token_provider Required*	Callable[[], str | Awaitable[str]] | None
token_endpoint Required*	str | None
credential Required*	<xref:TokenCredential> | None
default_headers Required*	Mapping[str, str] | None
async_client Required*	<xref:openai.lib.azure.AsyncAzureOpenAI> | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None

Keyword-Only Parameters

Expand table

Name	Description
deployment_name	The Azure OpenAI deployment name for the model to use. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
assistant_id	The ID of an Azure OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None

Name	Description
assistant_name	The name to use when creating new assistants. Default value: None
assistant_description	The description to use when creating new assistants. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential to use for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None

Name	Description
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureOpenAIAssistantsClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIAssistantsClient()

# Or passing parameters directly
client = AzureOpenAIAssistantsClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIAssistantsClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Clean up any assistants we created.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None

Name	Description
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient
```

```
    # Create a client
    client = OpenAIChatClient(model_id="gpt-4")

    # Create an agent using the convenience method
    agent = client.create_agent(
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
```

```
    client_data = {
        "type": "open_ai_chat_client",
        "model_id": "gpt-4o-mini",
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
```

```python
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value	

Name	Description
Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None

Name	Description
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None

Name	Description
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`allow_multiple_tool_calls`	Whether to allow multiple tool calls in a single response. Default value: None
`frequency_penalty`	The frequency penalty to use. Default value: None
`logit_bias`	The logit bias to use. Default value: None
`max_tokens`	The maximum number of tokens to generate. Default value: None
`metadata`	Additional metadata to include in the request. Default value: None
`model_id`	The model_id to use for the agent. Default value: None
`presence_penalty`	The presence penalty to use. Default value: None
`response_format`	The format of the response. Default value: None
`seed`	The random seed to use. Default value: None
`stop`	The stop sequence(s) for the request. Default value: None
`store`	Whether to store the response. Default value: None
`temperature`	The sampling temperature to use. Default value: None
`tool_choice`	The tool choice for the request. Default is *auto*. Default value: None
`tools`	The tools to use for the request. Default value: None
`top_p`	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_AZURE_API_VERSION

Python

```
DEFAULT_AZURE_API_VERSION: ClassVar[str] = '2024-05-01-preview'
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

AzureOpenAIChatClient Class

Azure OpenAI Chat completion class.

Initialize an Azure OpenAI Chat completion client.

Constructor

Python

```
AzureOpenAIChatClient()
```

Keyword-Only Parameters

Expand table

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (chat_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	

Name	Description
	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use *developer* or *system*. Default value: None
kwargs	Other keyword parameters.

Examples

Python

```
from agent_framework.azure import AzureOpenAIChatClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIChatClient()

# Or passing parameters directly
client = AzureOpenAIChatClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIChatClient(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an Azure OpenAI Chat completion client.
__new__	
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Azure OpenAI Chat completion client.

Python

```
__init__(*, api_key: str | None = None, deployment_name: str | None = None,
endpoint: str | None = None, base_url: str | None = None, api_version: str | None
= None, ad_token: str | None = None, ad_token_provider: Callable[[], str |
Awaitable[str]] | None = None, token_endpoint: str | None = None, credential:
TokenCredential | None = None, default_headers: Mapping[str, str] | None = None,
async_client: AsyncAzureOpenAI | None = None, env_file_path: str | None = None,
env_file_encoding: str | None = None, instruction_role: str | None = None,
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
api_key Required*	str | None
deployment_name Required*	str | None
endpoint Required*	str | None
base_url Required*	str | None
api_version Required*	str | None
ad_token Required*	str | None
ad_token_provider Required*	Callable[[], str | Awaitable[str]] | None
token_endpoint Required*	str | None
credential Required*	TokenCredential | None
default_headers	Mapping[str, str] | None

Name	Description
Required*	
async_client Required*	<xref:openai.lib.azure.AsyncAzureOpenAI> | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None
instruction_role Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (chat_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider.

Name	Description
	Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use *developer* or *system*. Default value: None
kwargs	Other keyword parameters.

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureOpenAIChatClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIChatClient()
```

```
    # Or passing parameters directly
    client = AzureOpenAIChatClient(
        endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
api_key="your-key"
    )

    # Or loading from a .env file
    client = AzureOpenAIChatClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str |
None = None, instructions: str | None = None, chat_message_store_factory:
Callable[[], ChatMessageStoreProtocol] | None = None, context_providers:
ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None,
middleware: AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
```

```
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias	dict[str | int, float] | None

Name	Description
Required*	
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None

Name	Description
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
    from agent_framework.clients import OpenAIChatClient

    # Create a client
    client = OpenAIChatClient(model_id="gpt-4")

    # Create an agent using the convenience method
    agent = client.create_agent(
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any]

Name	Description
	The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```python
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens	int | None

Name	Description
Required*	
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters.

Name	Description
	Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.

Name	Description
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties	dict[str, Any] | None

Name	Description
Required*	
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None

Name	Description
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai.openai'
```

AzureOpenAIResponsesClient Class

Azure Responses completion class.

Initialize an Azure OpenAI Responses client.

Constructor

Python

```
AzureOpenAIResponsesClient()
```

Keyword-Only Parameters

Expand table

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (responses_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Currently, the base_url must end with "/openai/v1/". Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Currently, the api_version must be "preview". Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None

Name	Description
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use *developer* or *system*. Default value: None
kwargs	Additional keyword arguments.

Examples

Python

```
from agent_framework.azure import AzureOpenAIResponsesClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME=gpt-4o
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIResponsesClient()

# Or passing parameters directly
client = AzureOpenAIResponsesClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4o",
api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIResponsesClient(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an Azure OpenAI Responses client.
__new__	
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Azure OpenAI Responses client.

Python

```
__init__(*, api_key: str | None = None, deployment_name: str | None = None,
endpoint: str | None = None, base_url: str | None = None, api_version: str | None
= None, ad_token: str | None = None, ad_token_provider: Callable[[], str |
Awaitable[str]] | None = None, token_endpoint: str | None = None, credential:
TokenCredential | None = None, default_headers: Mapping[str, str] | None = None,
async_client: AsyncAzureOpenAI | None = None, env_file_path: str | None = None,
env_file_encoding: str | None = None, instruction_role: str | None = None,
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
api_key Required*	str | None
deployment_name Required*	str | None
endpoint Required*	str | None
base_url Required*	str | None
api_version Required*	str | None
ad_token Required*	str | None
ad_token_provider Required*	Callable[[], str | Awaitable[str]] | None
token_endpoint Required*	str | None
credential Required*	TokenCredential | None
default_headers	Mapping[str, str] | None

Name	Description
Required*	
async_client Required*	<xref:openai.lib.azure.AsyncAzureOpenAI> | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None
instruction_role Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (responses_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Currently, the base_url must end with "/openai/v1/". Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Currently, the api_version must be "preview". Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None

Name	Description
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use *developer* or *system*. Default value: None
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureOpenAIResponsesClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME=gpt-4o
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIResponsesClient()
```

```
   # Or passing parameters directly
   client = AzureOpenAIResponsesClient(
       endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-
4o", api_key="your-key"
   )

   # Or loading from a .env file
   client = AzureOpenAIResponsesClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str |
None = None, instructions: str | None = None, chat_message_store_factory:
Callable[[], ChatMessageStoreProtocol] | None = None, context_providers:
ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None,
middleware: AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
```

```
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias	dict[str | int, float] | None

Name	Description
Required*	
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None

Name	Description
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
    from agent_framework.clients import OpenAIChatClient

    # Create a client
    client = OpenAIChatClient(model_id="gpt-4")

    # Create an agent using the convenience method
    agent = client.create_agent(
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any]

Name	Description
	The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```python
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens	int | None

Name	Description
Required*	
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters.

Name	Description
	Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.

Name	Description
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties	dict[str, Any] | None

Name	Description
Required*	
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None

Name	Description
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

FILE_SEARCH_MAX_RESULTS

Python

```
FILE_SEARCH_MAX_RESULTS: int = 50
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai.openai'
```

AzureOpenAISettings Class

AzureOpenAI model settings.

The settings are first loaded from environment variables with the prefix '>>AZURE_OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON


```
{
   "title": "AzureOpenAISettings",
   "description": "AzureOpenAI model settings.\n\nThe settings are first loaded
from environment variables with the prefix 'AZURE_OPENAI_'.\nIf the environment
variables are not found, the settings can be loaded from a .env file\nwith the
encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare
ignored; however, validation will fail alerting that the settings are
missing.\n\nKeyword Args:\n    endpoint: The endpoint of the Azure deployment. This
value\n        can be found in the Keys & Endpoint section when examining\n
your resource from the Azure portal, the endpoint should end in openai.azure.com.\n
If both base_url and endpoint are supplied, base_url will be used.\n        Can be set
via environment variable AZURE_OPENAI_ENDPOINT.\n    chat_deployment_name: The name of
the Azure Chat deployment. This value\n        will correspond to the custom name you
chose for your deployment\n        when you deployed a model. This value can be found
under\n        Resource Management > Deployments in the Azure portal or,
alternatively,\n        under Management > Deployments in Azure AI Foundry.\n
Can be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME.\n
responses_deployment_name: The name of the Azure Responses deployment. This value\n
will correspond to the custom name you chose for your deployment\n        when you
deployed a model. This value can be found under\n        Resource Management >
Deployments in the Azure portal or, alternatively,\n        under Management >
Deployments in Azure AI Foundry.\n        Can be set via environment variable
AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME.\n    api_key: The API key for the Azure
deployment. This value can be\n        found in the Keys & Endpoint section when
examining your resource in\n        the Azure portal. You can use either KEY1 or
KEY2.\n        Can be set via environment variable AZURE_OPENAI_API_KEY.\n
api_version: The API version to use. The default value is `default_api_version`.\n
Can be set via environment variable AZURE_OPENAI_API_VERSION.\n    base_url: The url
of the Azure deployment. This value\n        can be found in the Keys & Endpoint
section when examining\n        your resource from the Azure portal, the base_url
consists of the endpoint,\n        followed by
/openai/deployments/{deployment_name}/,\n        use endpoint if you only want to
supply the endpoint.\n        Can be set via environment variable
AZURE_OPENAI_BASE_URL.\n    token_endpoint: The token endpoint to use to retrieve the
authentication token.\n        The default value is `default_token_endpoint`.\n
```

```
Can be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT.\n
default_api_version: The default API version to use if not specified.\n        The
default value is \"2024-10-21\".\n    default_token_endpoint: The default token
endpoint to use if not specified.\n        The default value is
\"https://cognitiveservices.azure.com/.default\".\n    env_file_path: The path to the
.env file to load settings from.\n    env_file_encoding: The encoding of the .env
file, defaults to 'utf-8'.\n\nExamples:\n    .. code-block:: python\n\n        from
agent_framework.azure import AzureOpenAISettings\n\n        # Using environment
variables\n        # Set AZURE_OPENAI_ENDPOINT=https://your-
endpoint.openai.azure.com\n        # Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4\n
# Set AZURE_OPENAI_API_KEY=your-key\n        settings = AzureOpenAISettings()\n\n
# Or passing parameters directly\n        settings = AzureOpenAISettings(\n
endpoint=\"https://your-endpoint.openai.azure.com\", chat_deployment_name=\"gpt-4\",
api_key=\"your-key\"\n        )\n\n        # Or loading from a .env file\n
settings = AzureOpenAISettings(env_file_path=\"path/to/.env\")",
      "type": "object",
      "properties": {
         "env_file_path": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Env File Path"
         },
         "env_file_encoding": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
         },
         "chat_deployment_name": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Chat Deployment Name"
         },
         "responses_deployment_name": {
            "anyOf": [
```

```
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Responses Deployment Name"
      },
      "endpoint": {
         "anyOf": [
            {
               "format": "uri",
               "maxLength": 2083,
               "minLength": 1,
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Endpoint"
      },
      "base_url": {
         "anyOf": [
            {
               "format": "uri",
               "maxLength": 2083,
               "minLength": 1,
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Base Url"
      },
      "api_key": {
         "anyOf": [
            {
               "format": "password",
               "type": "string",
               "writeOnly": true
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Api Key"
      },
      "api_version": {
```

```
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Api Version"
         },
         "token_endpoint": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Token Endpoint"
         },
         "default_api_version": {
            "default": "2024-10-21",
            "title": "Default Api Version",
            "type": "string"
         },
         "default_token_endpoint": {
            "default": "https://cognitiveservices.azure.com/.default",
            "title": "Default Token Endpoint",
            "type": "string"
         }
      }
   }
```


Constructor

Python

```
pydantic settings AzureOpenAISettings
```

Keyword-Only Parameters

Expand table

Name	Description
endpoint	The endpoint of the Azure deployment. This value can be found in the Keys & Endpoint section when examining your resource from the Azure

Name	Description
	portal, the endpoint should end in openai.azure.com. If both base_url and endpoint are supplied, base_url will be used. Can be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
chat_deployment_name	The name of the Azure Chat deployment. This value will correspond to the custom name you chose for your deployment when you deployed a model. This value can be found under Resource Management > Deployments in the Azure portal or, alternatively, under Management > Deployments in Azure AI Foundry. Can be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
responses_deployment_name	The name of the Azure Responses deployment. This value will correspond to the custom name you chose for your deployment when you deployed a model. This value can be found under Resource Management > Deployments in the Azure portal or, alternatively, under Management > Deployments in Azure AI Foundry. Can be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME. Default value: None
api_key	The API key for the Azure deployment. This value can be found in the Keys & Endpoint section when examining your resource in the Azure portal. You can use either KEY1 or KEY2. Can be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
api_version	The API version to use. The default value is *default_api_version*. Can be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
base_url	The url of the Azure deployment. This value can be found in the Keys & Endpoint section when examining your resource from the Azure portal, the base_url consists of the endpoint, followed by /openai/deployments/{deployment_name}/, use endpoint if you only want to supply the endpoint. Can be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
token_endpoint	The token endpoint to use to retrieve the authentication token. The default value is *default_token_endpoint*. Can be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
default_api_version	The default API version to use if not specified. The default value is "2024-10-21". Default value: 2024-10-21
default_token_endpoint	

Name	Description
	The default token endpoint to use if not specified. The default value is "https://cognitiveservices.azure.com/.default ". Default value: https://cognitiveservices.azure.com/.default
env_file_path	The path to the .env file to load settings from. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python

```
from agent_framework.azure import AzureOpenAISettings

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
settings = AzureOpenAISettings()

# Or passing parameters directly
settings = AzureOpenAISettings(
    endpoint="https://your-endpoint.openai.azure.com", chat_deployment_name="gpt-4", api_key="your-key"
)

# Or loading from a .env file
settings = AzureOpenAISettings(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize the settings class.
__new__	Override the **new** method to set the env_prefix.
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use:

	`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
get_azure_auth_token	Retrieve a Microsoft Entra Auth Token for a given token endpoint for the use with Azure OpenAI. The required role for the token is *Cognitive Services OpenAI Contributor*. The token endpoint may be specified as an environment variable, via the .env file or as an argument. If the token endpoint is not provided, the default is None. The *token_endpoint* argument takes precedence over the *token_endpoint* attribute.
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.

model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

Expand table

Name	Description
cls Required*	type[<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
```

```
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

```
Python
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

get_azure_auth_token

Retrieve a Microsoft Entra Auth Token for a given token endpoint for the use with Azure OpenAI.

The required role for the token is *Cognitive Services OpenAI Contributor*. The token endpoint may be specified as an environment variable, via the .env file or as an argument. If the token endpoint is not provided, the default is None. The *token_endpoint* argument takes precedence over the *token_endpoint* attribute.

```
Python
get_azure_auth_token(credential: TokenCredential, token_endpoint: str | None = None, **kwargs: Any) -> str | None
```

Parameters

Expand table

Name	Description
credential Required*	TokenCredential The Azure AD credential to use.
token_endpoint	str | None The token endpoint to use. Defaults to *https://cognitiveservices.azure.com/.default* . Default value: None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional keyword arguments to pass to the token retrieval method.

Returns

Expand table

Type	Description
str | None	The Azure token or None if the token could not be retrieved.

Exceptions

Expand table

Type	Description
ServiceInitializationError	If the token endpoint is not provided.

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
```

```
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
`_fields_set`	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
`values` Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
`update` Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
`deep`	bool

Name	Description
Required*	Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback	Callable[[Any], Any] | None

Name	Description
Required*	A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
`serialize_as_any` Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`mode`	Default value: 'python'
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context	Any | None

Name	Description
Required*	Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None

Name	Description
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool

Name	Description
	Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default). • *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python
`model_parametrized_name(params: tuple[type[Any], ...]) -> str`

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
`model_post_init(context: Any, /) -> None`

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.

Name	Description
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name	bool | None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python
`parse_obj(obj: Any) -> Self`

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python
`parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self`

Parameters

Expand table

Name	Description
b Required*	str | bytes

Name	Description
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True

Name	Description
`ref_template`	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
`localns` Required*	Any

Returns

Expand table

Type	Description
None	

validate

Python

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
`value` Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

api_key

Python

```
field api_key: SecretStr | None = None
```

api_version

Python

```
field api_version: str | None = None
```

base_url

Python

```
field base_url: AnyUrl | None = None
```

chat_deployment_name

Python

```
field chat_deployment_name: str | None = None
```

default_api_version

Python

```
field default_api_version: str = '2024-10-21'
```

default_token_endpoint

Python

```
field default_token_endpoint: str = 'https://cognitiveservices.azure.com/.default'
```

endpoint

Python

```
field endpoint: AnyUrl | None = None
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'AZURE_OPENAI_'
```

responses_deployment_name

Python

```
field responses_deployment_name: str | None = None
```

token_endpoint

Python

```
field token_endpoint: str | None = None
```

DurableAIAgent Class

A durable agent implementation that uses entity methods to interact with agent entities.

This class implements AgentProtocol and provides methods to work with Azure Durable Functions orchestrations, which use generators and yield instead of async/await.

Key methods:

- get_new_thread(): Create a new conversation thread
- run(): Execute the agent and return a Task for yielding in orchestrations

Note: The run() method is NOT async. It returns a Task directly that must be yielded in orchestrations to wait for the entity call to complete.

Example usage in orchestration: writer = app.get_agent(context, "WriterAgent") thread = writer.get_new_thread() # NOT yielded - returns immediately

response = yield writer.run(# Yielded - waits for entity call message="Write a haiku about coding", thread=thread

)

Initialize the DurableAIAgent.

Constructor

Python
`DurableAIAgent()`

Parameters

Expand table

Name	Description
`context` Required*	The orchestration context
`agent_name` Required*	Name of the agent (used to construct entity ID)

Methods

Expand table

__init__	Initialize the DurableAIAgent.
__new__	
get_new_thread	Create a new agent thread for this orchestration instance. Each call creates a unique thread with its own conversation context. The session ID is deterministic (uses context.new_uuid()) to ensure orchestration replay works correctly.
run	Execute the agent with messages and return an AgentTask for orchestrations. This method implements AgentProtocol and returns an AgentTask (subclass of TaskBase) that can be yielded in Durable Functions orchestrations. The task's result will be a typed AgentRunResponse.
run_stream	Run the agent with streaming (not supported for durable agents).

__init__

Initialize the DurableAIAgent.

Python

```
__init__(context: Any, agent_name: str)
```

Parameters

Expand table

Name	Description
context Required*	Any The orchestration context
agent_name Required*	str Name of the agent (used to construct entity ID)

__new__

Python

```
__new__(**kwargs)
```

get_new_thread

Create a new agent thread for this orchestration instance.

Each call creates a unique thread with its own conversation context. The session ID is deterministic (uses context.new_uuid()) to ensure orchestration replay works correctly.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance with a unique session ID

run

Execute the agent with messages and return an AgentTask for orchestrations.

This method implements AgentProtocol and returns an AgentTask (subclass of TaskBase) that can be yielded in Durable Functions orchestrations. The task's result will be a typed AgentRunResponse.

Python

```
run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *,
thread: AgentThread | None = None, response_format: type[BaseModel] | None = None,
**kwargs: Any) -> AgentTask
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the agent Default value: None
thread Required*	AgentThread | None Optional agent thread for conversation context
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None Optional Pydantic model for response parsing
****kwargs** Required*	Any Additional arguments (enable_tool_calls)

Keyword-Only Parameters

Expand table

Name	Description
thread	Default value: None
response_format	Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework_azurefunctions._orchestration.AgentTask>	An AgentTask that resolves to an AgentRunResponse when yielded

Examples

@app.orchestration_trigger(context_name="context") def my_orchestration(context):

agent = app.get_agent(context, "MyAgent") thread = agent.get_new_thread() response = yield agent.run("Hello", thread=thread)

response is typed as AgentRunResponse

run_stream

Run the agent with streaming (not supported for durable agents).

Python

```
run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AsyncIterator[AgentRunResponseUpdate]
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None Default value: None
thread Required*	AgentThread | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	Default value: None

Returns

Expand table

Type	Description
AsyncIterator[AgentRunResponseUpdate]	

Exceptions

Expand table

Type	Description
NotImplementedError	Streaming is not supported for durable agents.

Attributes

description

Get the description of the agent.

display_name

Get the display name of the agent.

id

Get the unique identifier for this agent.

name

Get the name of the agent.

chatkit Package

Classes

Expand table

ThreadItemConverter	Helper class to convert ChatKit thread items to Agent Framework ChatMessage objects. This class provides a base implementation for converting ChatKit thread items to Agent Framework messages. It can be extended to handle attachments, @-mentions, hidden context items, and custom thread item formats. Initialize the converter.

Functions

simple_to_agent_input

Helper function that uses the default ThreadItemConverter.

This function provides a quick way to get started with ChatKit integration without needing to create a custom ThreadItemConverter instance.

Python

```
async simple_to_agent_input(thread_items: Sequence[UserMessageItem | AssistantMessageItem | ClientToolCallItem | WidgetItem | GeneratedImageItem | WorkflowItem | TaskItem | HiddenContextItem | SDKHiddenContextItem | EndOfTurnItem] | UserMessageItem | AssistantMessageItem | ClientToolCallItem | WidgetItem | GeneratedImageItem | WorkflowItem | TaskItem | HiddenContextItem | SDKHiddenContextItem | EndOfTurnItem) -> list[ChatMessage]
```

Parameters

Expand table

Name	Description
thread_items Required*	Sequence[<xref:chatkit.types.UserMessageItem> | <xref:chatkit.types.AssistantMessageItem> | <xref:chatkit.types.ClientToolCallItem> | <xref:chatkit.types.WidgetItem> | <xref:chatkit.types.GeneratedImageItem> | <xref:chatkit.types.WorkflowItem> | <xref:chatkit.types.TaskItem> | <xref:chatkit.types.HiddenContextItem> | <xref:chatkit.types.SDKHiddenContextItem>

Name	Description
	| <xref:chatkit.types.EndOfTurnItem>] | <xref:chatkit.types.UserMessageItem> | <xref:chatkit.types.AssistantMessageItem> | <xref:chatkit.types.ClientToolCallItem> | <xref:chatkit.types.WidgetItem> | <xref:chatkit.types.GeneratedImageItem> | <xref:chatkit.types.WorkflowItem> | <xref:chatkit.types.TaskItem> | <xref:chatkit.types.HiddenContextItem> | <xref:chatkit.types.SDKHiddenContextItem> | <xref:chatkit.types.EndOfTurnItem> A single ThreadItem or a sequence of ThreadItems to convert.

Returns

Expand table

Type	Description
list[ChatMessage]	A list of ChatMessage objects that can be sent to an Agent Framework agent.

Examples

Python

```
from agent_framework_chatkit import simple_to_agent_input

# Convert a single item
messages = await simple_to_agent_input(user_message_item)

# Convert multiple items
messages = await simple_to_agent_input([user_message_item,
assistant_message_item, task_item])
```

stream_agent_response

Convert a streamed AgentRunResponseUpdate from Agent Framework to ChatKit events.

This helper function takes a stream of AgentRunResponseUpdate objects from a Microsoft Agent Framework agent and converts them to ChatKit ThreadStreamEvent objects that can be consumed by the ChatKit UI.

The function supports real-time token-by-token streaming by emitting ThreadItemUpdated events with AssistantMessageContentPartTextDelta for each text chunk as it arrives from the agent.

Python

```
async stream_agent_response(response_stream:
AsyncIterable[AgentRunResponseUpdate], thread_id: str, generate_id:
Callable[[str], str] | None = None) -> AsyncIterator[ThreadCreatedEvent |
ThreadUpdatedEvent | ThreadItemDoneEvent | ThreadItemAddedEvent |
ThreadItemUpdatedEvent | ThreadItemRemovedEvent | ThreadItemReplacedEvent |
StreamOptionsEvent | ProgressUpdateEvent | ClientEffectEvent | ErrorEvent |
NoticeEvent]
```

Parameters

Expand table

Name	Description
response_stream Required*	AsyncIterable[AgentRunResponseUpdate] An async iterable of AgentRunResponseUpdate objects from an Agent Framework agent.
thread_id Required*	str The ChatKit thread ID for the conversation.
generate_id	Callable[[str], str] | None Optional function to generate IDs for ChatKit items. If not provided, simple incremental IDs will be used. Default value: None

Returns

Expand table

Type	Description
AsyncIterator[<xref:chatkit.types.ThreadCreatedEvent> | <xref:chatkit.types.ThreadUpdatedEvent> | <xref:chatkit.types.ThreadItemDoneEvent> | <xref:chatkit.types.ThreadItemAddedEvent> | <xref:chatkit.types.ThreadItemUpdatedEvent> | <xref:chatkit.types.ThreadItemRemovedEvent> | <xref:chatkit.types.ThreadItemReplacedEvent> | <xref:chatkit.types.StreamOptionsEvent> | <xref:chatkit.types.ProgressUpdateEvent> | <xref:chatkit.types.ClientEffectEvent> | <xref:chatkit.types.ErrorEvent> | <xref:chatkit.types.NoticeEvent>]	

ThreadItemConverter Class

Helper class to convert ChatKit thread items to Agent Framework ChatMessage objects.

This class provides a base implementation for converting ChatKit thread items to Agent Framework messages. It can be extended to handle attachments, @-mentions, hidden context items, and custom thread item formats.

Initialize the converter.

Constructor

Python

```
ThreadItemConverter()
```

Parameters

Expand table

Name	Description
attachment_data_fetcher	Optional async function to fetch attachment binary data. If provided, it should take an attachment ID and return the binary data as bytes. If not provided, attachments will be converted to UriContent using available URLs. Default value: None
attachment_data_fetcher Required*	Optional async function to fetch attachment data by ID.

Methods

Expand table

__init__	Initialize the converter.
__new__	
assistant_message_to_input	Convert a ChatKit AssistantMessageItem to Agent Framework ChatMessage(s). The default implementation extracts text from all content parts and creates an assistant message.

	Note Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.
attachment_to_message_content	Convert a ChatKit attachment to Agent Framework content. This method is called internally by *user_message_to_input()* to handle attachments. Override this method to customize attachment handling for your storage backend. The default implementation provides two strategies: 1. If an attachment_data_fetcher was provided, it fetches the binary data and creates a DataContent object 2. Otherwise, for ImageAttachment with preview_url, it creates a UriContent object For FileAttachment without a data fetcher, returns None (attachment is skipped). **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types including attachments within user messages.
client_tool_call_to_input	Convert a ChatKit ClientToolCallItem to Agent Framework ChatMessage(s). The default implementation converts completed tool calls into function call and result content. **Note** Instead of calling this method directly, use to_agent_input() which handles

	all ThreadItem types and provides proper message ordering.
end_of_turn_to_input	Convert a ChatKit EndOfTurnItem to Agent Framework ChatMessage(s). The default implementation skips end-of-turn markers as they are only UI hints. **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.
hidden_context_to_input	Convert a ChatKit HiddenContextItem or SDKHiddenContextItem to Agent Framework ChatMessage(s). This method is called internally by *to_agent_input()*. Override this method to customize how hidden context is converted. The default implementation wraps the hidden context in XML tags and returns a system message. This allows the model to distinguish hidden context from regular conversation. **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.
tag_to_message_content	Convert a ChatKit tag (@-mention) to Agent Framework content. This method is called internally by *user_message_to_input()* to handle tags. Override this method to customize tag conversion for your application. The default implementation extracts the tag's display name and wraps it in XML tags to provide context to the model about the @-mention. **Note**

	Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types including tags within user messages.
task_to_input	Convert a ChatKit TaskItem to Agent Framework ChatMessage(s). This method is called internally by *to_agent_input()*. Override this method to customize how tasks are converted. The default implementation converts custom tasks with title/content into a user message explaining what task was displayed to the user. **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.
to_agent_input	Convert ChatKit thread items to Agent Framework ChatMessages. This is the main entry point for converting ChatKit thread items. It handles all ThreadItem types (UserMessageItem, AssistantMessageItem, TaskItem, etc.) and calls the appropriate conversion method for each.
user_message_to_input	Convert a ChatKit UserMessageItem to Agent Framework ChatMessage(s). This method is called internally by *to_agent_input()*. Override this method to customize how user messages are converted. **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.
widget_to_input	Convert a ChatKit WidgetItem to Agent Framework ChatMessage(s). This method is called internally by *to_agent_input()*. Override this method to customize how widgets are converted.

	The default implementation converts the widget to a JSON representation and includes it in a user message, allowing the model to understand what UI element was displayed to the user. **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.
workflow_to_input	Convert a ChatKit WorkflowItem to Agent Framework ChatMessage(s). This method is called internally by *to_agent_input()*. Override this method to customize how workflows are converted. The default implementation converts each custom task in the workflow into a separate user message explaining what tasks were performed. **Note** Instead of calling this method directly, use to_agent_input() which handles all ThreadItem types and provides proper message ordering.

__init__

Initialize the converter.

Python

```
__init__(attachment_data_fetcher: Callable[[str], Awaitable[bytes]] | None = None)
-> None
```

Parameters

Expand table

Name	Description
attachment_data_fetcher	Callable[[str], Awaitable[bytes]] | None Optional async function to fetch attachment data by ID.

Name	Description
	Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

assistant_message_to_input

Convert a ChatKit AssistantMessageItem to Agent Framework ChatMessage(s).

The default implementation extracts text from all content parts and creates an assistant message.

> **Note**
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

```
Python
async assistant_message_to_input(item: AssistantMessageItem) -> ChatMessage |
list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item	<xref:chatkit.types.AssistantMessageItem>

Name	Description
Required*	The ChatKit assistant message item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A ChatMessage with assistant role, or None to skip.

attachment_to_message_content

Convert a ChatKit attachment to Agent Framework content.

This method is called internally by *user_message_to_input()* to handle attachments. Override this method to customize attachment handling for your storage backend.

The default implementation provides two strategies:

1. If an attachment_data_fetcher was provided, it fetches the binary data

   and creates a DataContent object

2. Otherwise, for ImageAttachment with preview_url, it creates a UriContent object

For FileAttachment without a data fetcher, returns None (attachment is skipped).

> ⓘ Note
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types including attachments within user messages.

Python

```
async attachment_to_message_content(attachment: FileAttachment | ImageAttachment)
-> DataContent | UriContent | None
```

Parameters

Expand table

Name	Description
attachment Required*	<xref:chatkit.types.FileAttachment> | <xref:chatkit.types.ImageAttachment> The ChatKit attachment to convert (FileAttachment or ImageAttachment).

Returns

Expand table

Type	Description
DataContent | UriContent | None	DataContent if binary data is available, UriContent if only URL is available, or None if the attachment cannot be converted.

Examples

Python

```
# With data fetcher
async def fetch_data(attachment_id: str) -> bytes:
    return await my_storage.get_file(attachment_id)


converter = ThreadItemConverter(attachment_data_fetcher=fetch_data)
messages = await converter.to_agent_input(thread_items)

# Without data fetcher (uses URLs for images)
converter = ThreadItemConverter()
messages = await converter.to_agent_input(thread_items)
```

client_tool_call_to_input

Convert a ChatKit ClientToolCallItem to Agent Framework ChatMessage(s).

The default implementation converts completed tool calls into function call and result content.

> **Note**
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

Python

```
async client_tool_call_to_input(item: ClientToolCallItem) -> ChatMessage |
list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.ClientToolCallItem> The ChatKit client tool call item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A list containing function call and result messages, or None for pending calls.

end_of_turn_to_input

Convert a ChatKit EndOfTurnItem to Agent Framework ChatMessage(s).

The default implementation skips end-of-turn markers as they are only UI hints.

> **Note**
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

Python

```
async end_of_turn_to_input(item: EndOfTurnItem) -> ChatMessage | list[ChatMessage]
| None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.EndOfTurnItem> The ChatKit end-of-turn item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	None (end-of-turn items are not converted).

hidden_context_to_input

Convert a ChatKit HiddenContextItem or SDKHiddenContextItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how hidden context is converted.

The default implementation wraps the hidden context in XML tags and returns a system message. This allows the model to distinguish hidden context from regular conversation.

> **Note**
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

Python

```
hidden_context_to_input(item: HiddenContextItem | SDKHiddenContextItem) ->
ChatMessage | list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.HiddenContextItem> | <xref:chatkit.types.SDKHiddenContextItem> The ChatKit hidden context item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A ChatMessage with system role, a list of messages, or None to skip.

Examples

Python

```
    # Default behavior
    converter = ThreadItemConverter()
    hidden_item = HiddenContextItem(
        id="ctx_1",
        thread_id="thread_1",
        created_at=datetime.now(),
        content="User's email: user@example.com",
    )
    message = converter.hidden_context_to_input(hidden_item)
    # Returns: ChatMessage(role=SYSTEM, text="<HIDDEN_CONTEXT>User's email: ...
</HIDDEN_CONTEXT>")
```

tag_to_message_content

Convert a ChatKit tag (@-mention) to Agent Framework content.

This method is called internally by *user_message_to_input()* to handle tags. Override this method to customize tag conversion for your application.

The default implementation extracts the tag's display name and wraps it in XML tags to provide context to the model about the @-mention.

> ⓘ Note
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types including tags within user messages.

Python

```
tag_to_message_content(tag: UserMessageTagContent) -> TextContent
```

Parameters

Expand table

Name	Description
tag Required*	<xref:chatkit.types.UserMessageTagContent> The ChatKit tag content to convert.

Returns

Expand table

Type	Description
TextContent	TextContent with the tag information.

Examples

Python

```
   # Default behavior
   converter = ThreadItemConverter()
   tag = UserMessageTagContent(
       type="input_tag", id="tag_1", text="john", data={"name": "John Doe"},
interactive=False
   )
   content = converter.tag_to_message_content(tag)
   # Returns: TextContent(text="<TAG>Name:John Doe</TAG>")
```

task_to_input

Convert a ChatKit TaskItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how tasks are converted.

The default implementation converts custom tasks with title/content into a user message explaining what task was displayed to the user.

> **Note**

Instead of calling this method directly, use to_agent_input() which handles

all ThreadItem types and provides proper message ordering.

Python

```
task_to_input(item: TaskItem) -> ChatMessage | list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.TaskItem> The ChatKit task item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A ChatMessage, a list of messages, or None to skip the task.

Examples

Python

```
   # Task with both title and content
   from chatkit.types import Task

   task_item = TaskItem(
       id="task_1",
       thread_id="thread_1",
       created_at=datetime.now(),
       task=Task(type="custom", title="Data Analysis", content="Analyzed sales
data"),
   )
   message = converter.task_to_input(task_item)
   # Returns message explaining the task was performed
```

to_agent_input

Convert ChatKit thread items to Agent Framework ChatMessages.

This is the main entry point for converting ChatKit thread items. It handles all ThreadItem types (UserMessageItem, AssistantMessageItem, TaskItem, etc.) and calls the appropriate conversion method for each.

Python

```
async to_agent_input(thread_items: Sequence[UserMessageItem | AssistantMessageItem | ClientToolCallItem | WidgetItem | GeneratedImageItem | WorkflowItem | TaskItem | HiddenContextItem | SDKHiddenContextItem | EndOfTurnItem] | UserMessageItem | AssistantMessageItem | ClientToolCallItem | WidgetItem | GeneratedImageItem | WorkflowItem | TaskItem | HiddenContextItem | SDKHiddenContextItem | EndOfTurnItem) -> list[ChatMessage]
```

Parameters

Expand table

Name	Description
thread_items Required*	Sequence[<xref:chatkit.types.UserMessageItem> | <xref:chatkit.types.AssistantMessageItem> | <xref:chatkit.types.ClientToolCallItem> | <xref:chatkit.types.WidgetItem> | <xref:chatkit.types.GeneratedImageItem> | <xref:chatkit.types.WorkflowItem> | <xref:chatkit.types.TaskItem> | <xref:chatkit.types.HiddenContextItem> | <xref:chatkit.types.SDKHiddenContextItem> | <xref:chatkit.types.EndOfTurnItem>] | <xref:chatkit.types.UserMessageItem> | <xref:chatkit.types.AssistantMessageItem> | <xref:chatkit.types.ClientToolCallItem> | <xref:chatkit.types.WidgetItem> | <xref:chatkit.types.GeneratedImageItem> | <xref:chatkit.types.WorkflowItem> | <xref:chatkit.types.TaskItem> | <xref:chatkit.types.HiddenContextItem> | <xref:chatkit.types.SDKHiddenContextItem> | <xref:chatkit.types.EndOfTurnItem> A single ThreadItem or a sequence of ThreadItems to convert.

Returns

Expand table

Type	Description
list[ChatMessage]	A list of ChatMessage objects that can be sent to an Agent Framework agent.

Examples

Python

```
   from agent_framework_chatkit import ThreadItemConverter

   converter = ThreadItemConverter()

   # Convert a single thread item
   messages = await converter.to_agent_input(user_message_item)

   # Convert multiple thread items
   messages = await converter.to_agent_input([user_message_item,
assistant_message_item, task_item])

   # Use with agent
   from agent_framework import ChatAgent

   agent = ChatAgent(...)
   response = await agent.run_stream(messages)
```

user_message_to_input

Convert a ChatKit UserMessageItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how user messages are converted.

> ⓘ Note
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

Python

```
async user_message_to_input(item: UserMessageItem, is_last_message: bool = True) -
> ChatMessage | list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.UserMessageItem> The ChatKit user message item to convert.
is_last_message	bool

Name	Description
	Whether this is the last message in the thread (used for quoted_text handling). Default value: True

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A ChatMessage, list of messages, or None to skip.

widget_to_input

Convert a ChatKit WidgetItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how widgets are converted.

The default implementation converts the widget to a JSON representation and includes it in a user message, allowing the model to understand what UI element was displayed to the user.

> ⓘ Note
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

Python

```
widget_to_input(item: WidgetItem) -> ChatMessage | list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.WidgetItem> The ChatKit widget item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A ChatMessage describing the widget, or None to skip.

Examples

Python

```
# Widget item
from chatkit.widgets import Card, Text

widget_item = WidgetItem(
    id="widget_1",
    thread_id="thread_1",
    created_at=datetime.now(),
    widget=Card(children=[Text(value="Hello")]),
)
message = converter.widget_to_input(widget_item)
# Returns message with JSON representation of the widget
```

workflow_to_input

Convert a ChatKit WorkflowItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how workflows are converted.

The default implementation converts each custom task in the workflow into a separate user message explaining what tasks were performed.

> ⓘ Note
>
> Instead of calling this method directly, use to_agent_input() which handles
>
> all ThreadItem types and provides proper message ordering.

Python

```
workflow_to_input(item: WorkflowItem) -> ChatMessage | list[ChatMessage] | None
```

Parameters

Expand table

Name	Description
item Required*	<xref:chatkit.types.WorkflowItem> The ChatKit workflow item to convert.

Returns

Expand table

Type	Description
ChatMessage | list[ChatMessage] | None	A list of ChatMessages (one per task), a single message, or None to skip.

Examples

Python

```
# Workflow with multiple tasks
from chatkit.types import Workflow, Task

workflow_item = WorkflowItem(
    id="wf_1",
    thread_id="thread_1",
    created_at=datetime.now(),
    workflow=Workflow(
        type="custom",
        tasks=[
            Task(type="custom", title="Step 1", content="Gathered data"),
            Task(type="custom", title="Step 2", content="Analyzed results"),
        ],
    ),
)
messages = converter.workflow_to_input(workflow_item)
# Returns list of messages for each task
```

declarative Package

devui Package

Classes

Expand table

AgentFrameworkRequest	OpenAI ResponseCreateParams with Agent Framework routing. This properly extends the real OpenAI API request format. • Uses 'model' field as entity_id (agent/workflow name) • Uses 'conversation' field for conversation context (OpenAI standard) Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
DevServer	Development Server - OpenAI compatible API server for debugging agents. Initialize the development server.
DiscoveryResponse	Response model for entity discovery. Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
EntityInfo	Entity information for discovery and detailed views. Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
OpenAIError	OpenAI standard error response model. Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Functions

main

CLI entry point for devui command.

Python

```
main() -> None
```

Returns

Expand table

Type	Description
None	

serve

Launch Agent Framework DevUI with simple API.

Python

```
serve(entities: list[Any] | None = None, entities_dir: str | None = None, port: int = 8080, host: str = '127.0.0.1', auto_open: bool = False, cors_origins: list[str] | None = None, ui_enabled: bool = True, instrumentation_enabled: bool = False, mode: str = 'developer', auth_enabled: bool = False, auth_token: str | None = None) -> None
```

Parameters

Expand table

Name	Description
entities	list[Any] | None List of entities for in-memory registration (IDs auto-generated) Default value: None

Name	Description
entities_dir	str | None Directory to scan for entities Default value: None
port	int Port to run server on Default value: 8080
host	str Host to bind server to Default value: "127.0.0.1"
auto_open	bool Whether to automatically open browser Default value: False
cors_origins	list[str] | None List of allowed CORS origins Default value: None
ui_enabled	bool Whether to enable the UI Default value: True
instrumentation_enabled	bool Whether to enable OpenTelemetry instrumentation Default value: False
mode	str Server mode - 'developer' (full access, verbose errors) or 'user' (restricted APIs, generic errors) Default value: "developer"
auth_enabled	bool Whether to enable Bearer token authentication Default value: False
auth_token	str | None Custom authentication token (auto-generated if not provided with auth_enabled=True) Default value: None

Returns

Expand table

Type	Description
None	

AgentFrameworkRequest Class

OpenAI ResponseCreateParams with Agent Framework routing.

This properly extends the real OpenAI API request format.

- Uses 'model' field as entity_id (agent/workflow name)
- Uses 'conversation' field for conversation context (OpenAI standard)

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python

```
pydantic model AgentFrameworkRequest
```

Keyword-Only Parameters

Expand table

Name	Description
`model`	Default value: None
`input` Required*	
`stream`	Default value: False
`conversation`	Default value: None
`instructions`	Default value: None
`metadata`	Default value: None
`temperature`	Default value: None
`max_output_tokens`	Default value: None

Name	Description
top_p	Default value: None
tools	Default value: None
reasoning	Default value: None
extra_body	Default value: None

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
__new__	
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
get_entity_id	Get entity_id from metadata.entity_id. In DevUI, entity_id is specified in metadata for routing.
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is

	performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.

parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
to_openai_params	Convert to dict for OpenAI client compatibility.
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```python
__new__(**kwargs)
```

construct

```python
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False

Name	Description
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

```
Python
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

get_entity_id

Get entity_id from metadata.entity_id.

In DevUI, entity_id is specified in metadata for routing.

Python

```
get_entity_id() -> str | None
```

Returns

Expand table

Type	Description
str | None	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults	bool

Name	Description
Required*	
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
`_fields_set`	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
`values` Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].

Name	Description
`fallback` Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
`serialize_as_any` Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`indent`	Default value: None
`ensure_ascii`	Default value: False
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • '*any_of*': Use the *anyOf* keyword to combine schemas (the default). • '*primitive_type_array*': Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
`union_format`	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
`params` Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias	bool | None

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
`json_data` Required*	str | bytes | bytearray The JSON data to validate.
`strict` Required*	bool | None Whether to enforce types strictly.
`extra` Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`context` Required*	Any | None Extra variables to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name	bool | None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj	Any

Name	Description
Required*	

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

to_openai_params

Convert to dict for OpenAI client compatibility.

Python

```
to_openai_params() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

update_forward_refs

Python

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

Python

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

conversation

Python

```
field conversation: str | dict[str, Any] | None = None
```

extra_body

Python

```
field extra_body: dict[str, Any] | None = None
```

input

Python

```
field input: str | list[Any] | dict[str, Any] [Required]
```

instructions

Python

```
field instructions: str | None = None
```

max_output_tokens

Python

```
field max_output_tokens: int | None = None
```

metadata

Python

```
field metadata: dict[str, Any] | None = None
```

model

Python

```
field model: str | None = None
```

reasoning

Python

```
field reasoning: dict[str, Any] | None = None
```

stream

Python

```
field stream: bool | None = False
```

temperature

Python

```
field temperature: float | None = None
```

tools

Python

```
field tools: list[dict[str, Any]] | None = None
```

top_p

Python

```
field top_p: float | None = None
```

DevServer Class

Development Server - OpenAI compatible API server for debugging agents.

Initialize the development server.

Constructor

```
Python
DevServer()
```

Parameters

Expand table

Name	Description
entities_dir	Directory to scan for entities Default value: None
port	Port to run server on Default value: 8080
host	Host to bind server to Default value: 127.0.0.1
cors_origins	List of allowed CORS origins Default value: None
ui_enabled	Whether to enable the UI Default value: True
mode	Server mode - 'developer' (full access, verbose errors) or 'user' (restricted APIs, generic errors) Default value: developer

Methods

Expand table

__init__	Initialize the development server.
__new__	

create_app	Create the FastAPI application.
get_app	Get the FastAPI application instance.
register_entities	Register entities to be discovered when server starts.

__init__

Initialize the development server.

Python

```
__init__(entities_dir: str | None = None, port: int = 8080, host: str = '127.0.0.1', cors_origins: list[str] | None = None, ui_enabled: bool = True, mode: str = 'developer') -> None
```

Parameters

Expand table

Name	Description
entities_dir	str | None Directory to scan for entities Default value: None
port	int Port to run server on Default value: 8080
host	str Host to bind server to Default value: "127.0.0.1"
cors_origins	list[str] | None List of allowed CORS origins Default value: None
ui_enabled	bool Whether to enable the UI Default value: True
mode	str Server mode - 'developer' (full access, verbose errors) or 'user' (restricted APIs, generic errors) Default value: "developer"

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

create_app

Create the FastAPI application.

```
Python
create_app() -> FastAPI
```

Returns

Expand table

Type	Description
<xref:fastapi.applications.FastAPI>	

get_app

Get the FastAPI application instance.

```
Python
get_app() -> FastAPI
```

Returns

Expand table

Type	Description
<xref:fastapi.applications.FastAPI>	

register_entities

Register entities to be discovered when server starts.

Python

```
register_entities(entities: list[Any]) -> None
```

Parameters

Expand table

Name	Description
entities Required*	list[Any] List of entity objects to register

Returns

Expand table

Type	Description
None	

DiscoveryResponse Class

Response model for entity discovery.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python

```
pydantic model DiscoveryResponse
```

Keyword-Only Parameters

Expand table

Name	Description
entities	Default value: <factory>

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
__new__	
construct	
copy	Returns a copy of the model.

	!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.

model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

construct

```
Python
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].

Name	Description
`fallback` Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
`serialize_as_any` Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`indent`	Default value: None
`ensure_ascii`	Default value: False
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • '*any_of*': Use the *anyOf* keyword to combine schemas (the default). • '*primitive_type_array*': Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias	bool | None

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
`json_data` Required*	str | bytes | bytearray The JSON data to validate.
`strict` Required*	bool | None Whether to enforce types strictly.
`extra` Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`context` Required*	Any | None Extra variables to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name	bool | None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj	Any

Name	Description
Required*	

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

```
Python
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
Python
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

entities

Python

```
field entities: list[EntityInfo] [Optional]
```

EntityInfo Class

Entity information for discovery and detailed views.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

```
Python
pydantic model EntityInfo
```

Keyword-Only Parameters

Expand table

Name	Description
id Required*	
type Required*	
name Required*	
description	Default value: None
framework Required*	
tools	Default value: None
metadata	Default value: <factory>
source	Default value: directory
required_env_vars	Default value: None
deployment_supported	Default value: False

Name	Description
deployment_reason	Default value: None
instructions	Default value: None
model_id	Default value: None
chat_client_type	Default value: None
context_providers	Default value: None
middleware	Default value: None
executors	Default value: None
workflow_dump	Default value: None
input_schema	Default value: None
input_type_name	Default value: None
start_executor_id	Default value: None

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
__new__	
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and

	automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

construct

```
Python
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`include`	Default value: None
`exclude`	Default value: None
`by_alias`	Default value: False
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`encoder`	Default value: PydanticUndefined
`models_as_dict`	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int]
| set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] |
Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
```

```
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings	bool | Literal['none', 'warn', 'error']

Name	Description
Required*	How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
`fallback` Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
`serialize_as_any` Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`mode`	Default value: 'python'
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] |

Name	Description
	Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None

Name	Description
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • '*any_of*': Use the *anyOf* keyword to combine schemas (the default). • '*primitive_type_array*': Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context:
```

```
Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) ->
Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.

Name	Description
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None

Name	Description
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
Python
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
Python
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
`value` Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

chat_client_type

Python

```
field chat_client_type: str | None = None
```

context_providers

Python

```
field context_providers: list[str] | None = None
```

deployment_reason

Python

```
field deployment_reason: str | None = None
```

deployment_supported

Python

```
field deployment_supported: bool = False
```

description

Python

```
field description: str | None = None
```

executors

Python

```
field executors: list[str] | None = None
```

framework

Python

```
field framework: str [Required]
```

id

Python

```
field id: str [Required]
```

input_schema

Python

```
field input_schema: dict[str, Any] | None = None
```

input_type_name

Python

```
field input_type_name: str | None = None
```

instructions

Python

```
field instructions: str | None = None
```

metadata

Python

```
field metadata: dict[str, Any] [Optional]
```

middleware

Python

```
field middleware: list[str] | None = None
```

model_id

Python

```
field model_id: str | None = None
```

name

Python

```
field name: str [Required]
```

required_env_vars

Python

```
field required_env_vars: list[EnvVarRequirement] | None = None
```

source

Python

```
field source: str = 'directory'
```

start_executor_id

Python

```
field start_executor_id: str | None = None
```

tools

Python

```
field tools: list[str | dict[str, Any]] | None = None
```

type

Python

```
field type: str [Required]
```

workflow_dump

Python

```
field workflow_dump: dict[str, Any] | None = None
```

OpenAIError Class

OpenAI standard error response model.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

```
Python
pydantic model OpenAIError
```

Keyword-Only Parameters

Expand table

Name	Description
error Required*	

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
__new__	
construct	
copy	Returns a copy of the model.

	!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
create	Create a standard OpenAI error response.
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes.

	This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
to_dict	Return the error payload as a plain mapping.
to_json	Return the error payload serialized to JSON.
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```
__new__(**kwargs)
```

construct

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

create

Create a standard OpenAI error response.

Python

```
create(message: str, type: str = 'invalid_request_error', code: str | None = None)
-> OpenAIError
```

Parameters

Expand table

Name	Description
message Required*	str
type	str Default value: "invalid_request_error"
code	str | None Default value: None

Returns

Expand table

Type	Description
OpenAIError	

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`include`	Default value: None
`exclude`	Default value: None
`by_alias`	Default value: False
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`encoder`	Default value: PydanticUndefined
`models_as_dict`	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int]
| set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] |
Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
```

```
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings	bool | Literal['none', 'warn', 'error']

Name	Description
Required*	How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
`fallback` Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
`serialize_as_any` Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`mode`	Default value: 'python'
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] |

Name	Description
	Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None

Name	Description
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • '*any_of*': Use the *anyOf* keyword to combine schemas (the default). • '*primitive_type_array*': Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context:
```

```
Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) ->
Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.

Name	Description
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None

Name	Description
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`by_alias`	Default value: True
`ref_template`	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

to_dict

Return the error payload as a plain mapping.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

Return the error payload serialized to JSON.

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

update_forward_refs

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

error

Python

```
field error: dict[str, Any] [Required]
```

lab Package

mem0 Package

Classes

Expand table

Mem0Provider	Mem0 Context Provider. Initializes a new instance of the Mem0Provider class.

Mem0Provider Class

Mem0 Context Provider.

Initializes a new instance of the Mem0Provider class.

Constructor

```
Python
Mem0Provider()
```

Parameters

Expand table

Name	Description
mem0_client	A pre-created Mem0 MemoryClient or None to create a default client. Default value: None
api_key	The API key for authenticating with the Mem0 API. If not provided, it will attempt to use the MEM0_API_KEY environment variable. Default value: None
application_id	The application ID for scoping memories or None. Default value: None
agent_id	The agent ID for scoping memories or None. Default value: None
thread_id	The thread ID for scoping memories or None. Default value: None
user_id	The user ID for scoping memories or None. Default value: None
scope_to_per_operation_thread_id	Whether to scope memories to per-operation thread ID. Default value: False
context_prompt	The prompt to prepend to retrieved memories. Default value: ## Memories Consider the following memories when answering user questions:

Methods

Expand table

__init__	Initializes a new instance of the Mem0Provider class.
__new__	
invoked	
invoking	Called before invoking the AI model to provide context.
thread_created	Called when a new thread is created.

__init__

Initializes a new instance of the Mem0Provider class.

Python

```
__init__(mem0_client: AsyncMemory | AsyncMemoryClient | None = None, api_key: str | None = None, application_id: str | None = None, agent_id: str | None = None, thread_id: str | None = None, user_id: str | None = None, scope_to_per_operation_thread_id: bool = False, context_prompt: str = ContextProvider.DEFAULT_CONTEXT_PROMPT) -> None
```

Parameters

Expand table

Name	Description
mem0_client	<xref:mem0.memory.main.AsyncMemory> | <xref:mem0.client.main.AsyncMemoryClient> | None A pre-created Mem0 MemoryClient or None to create a default client. Default value: None
api_key	str | None The API key for authenticating with the Mem0 API. If not provided, it will attempt to use the MEM0_API_KEY environment variable. Default value: None
application_id	str | None The application ID for scoping memories or None. Default value: None

Name	Description
agent_id	str | None The agent ID for scoping memories or None. Default value: None
thread_id	str | None The thread ID for scoping memories or None. Default value: None
user_id	str | None The user ID for scoping memories or None. Default value: None
scope_to_per_operation_thread_id	bool Whether to scope memories to per-operation thread ID. Default value: False
context_prompt	str The prompt to prepend to retrieved memories. Default value: ContextProvider.DEFAULT_CONTEXT_PROMPT

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

invoked

Python

```
async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
request_messages Required*	ChatMessage | Sequence[ChatMessage]
response_messages	ChatMessage | Sequence[ChatMessage] | None Default value: None
invoke_exception	Exception | None Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

invoking

Called before invoking the AI model to provide context.

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs: Any) -> Context
```

Parameters

Expand table

Name	Description
messages Required*	ChatMessage | MutableSequence[ChatMessage] List of new messages in the thread.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
**kwargs	not used at present.

Returns

Expand table

Type	Description
Context	Context object containing instructions with memories.

thread_created

Called when a new thread is created.

Python

```
async thread_created(thread_id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
thread_id	str | None The ID of the thread or None. Default value: None

Returns

Expand table

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories when answering user questions:'
```

microsoft Package

Classes

Expand table

CacheProvider	Protocol for cache providers used by Purview integration.
CopilotStudioAgent	A Copilot Studio Agent. Initialize the Copilot Studio Agent.
PurviewAppLocation	Identifier representing the app's location for Purview policy evaluation. Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
PurviewAuthenticationError	Authentication / authorization failure (401/403). Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewChatPolicyMiddleware	Chat middleware variant for Purview policy evaluation. This allows users to attach Purview enforcement directly to a chat client Behavior: • Pre-chat: evaluates outgoing (user + context) messages as an upload activity and can terminate execution if blocked. • Post-chat: evaluates the received response messages (streaming is not presently supported) and can replace them with a blocked message. Uses the same user_id from the request to ensure consistent user identity throughout the evaluation. Usage:
PurviewPaymentRequiredError	Payment required (402). Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewPolicyMiddleware	Agent middleware that enforces Purview policies on prompt and response.

	Accepts either a synchronous TokenCredential or an AsyncTokenCredential. Usage:
PurviewRateLimitError	Rate limiting or throttling (429). Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewRequestError	Other non-success HTTP errors. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewServiceError	Base exception for Purview errors. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
PurviewSettings	Settings for Purview integration mirroring .NET PurviewSettings. Initialize the settings class. JSON { "title": "PurviewSettings", "description": "Settings for Purview integration mirroring .NET PurviewSettings.\n\nAttributes:\n app_name: Public app name.\n app_version: Optional version string of the application.\n tenant_id: Optional tenant id (guid) of the user making the request.\n purview_app_location: Optional app location for policy evaluation.\n graph_base_uri: Base URI for Microsoft Graph.\n blocked_prompt_message: Custom message to return when a prompt is blocked by policy.\n blocked_response_message: Custom message to return when a response is blocked by policy.\n ignore_exceptions: If True, all Purview exceptions will be logged but not thrown in middleware.\n ignore_payment_required: If True, 402 payment required errors will be logged but not thrown.\n cache_ttl_seconds: Time to live for cache entries in seconds (default 14400 = 4 hours).\n max_cache_size_bytes: Maximum cache size in bytes (default 200MB).", "type": "object", "properties": { "env_file_path": { "anyOf": [{ "type": "string"

```
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Env File Path"
      },
      "env_file_encoding": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": "utf-8",
         "title": "Env File Encoding"
      },
      "app_name": {
         "title": "App Name",
         "type": "string"
      },
      "app_version": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "App Version"
      },
      "tenant_id": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Tenant Id"
      },
      "purview_app_location": {
         "anyOf": [
            {
               "$ref": "#/$defs/PurviewAppLocation"
            },
            {
               "type": "null"
```

```
                }
             ],
             "default": null
          },
          "graph_base_uri": {
             "default":
"https://graph.microsoft.com/v1.0/",
             "title": "Graph Base Uri",
             "type": "string"
          },
          "blocked_prompt_message": {
             "default": "Prompt blocked by policy",
             "description": "Message to return when a
prompt is blocked by policy.",
             "title": "Blocked Prompt Message",
             "type": "string"
          },
          "blocked_response_message": {
             "default": "Response blocked by policy",
             "description": "Message to return when a
response is blocked by policy.",
             "title": "Blocked Response Message",
             "type": "string"
          },
          "ignore_exceptions": {
             "default": false,
             "description": "If True, all Purview
exceptions will be logged but not thrown in middleware.",
             "title": "Ignore Exceptions",
             "type": "boolean"
          },
          "ignore_payment_required": {
             "default": false,
             "description": "If True, 402 payment required
errors will be logged but not thrown.",
             "title": "Ignore Payment Required",
             "type": "boolean"
          },
          "cache_ttl_seconds": {
             "default": 14400,
             "description": "Time to live for cache
entries in seconds (default 14400 = 4 hours).",
             "title": "Cache Ttl Seconds",
             "type": "integer"
          },
          "max_cache_size_bytes": {
             "default": 209715200,
             "description": "Maximum cache size in bytes
(default 200MB).",
             "title": "Max Cache Size Bytes",
             "type": "integer"
          }
       },
       "$defs": {
          "PurviewAppLocation": {
```

```
            "description": "Identifier representing the
app's location for Purview policy evaluation.",
            "properties": {
               "location_type": {
                  "$ref": "#/$defs/PurviewLocationType",
                  "description": "The location type."
               },
               "location_value": {
                  "description": "The location value.",
                  "title": "Location Value",
                  "type": "string"
               }
            },
            "required": [
               "location_type",
               "location_value"
            ],
            "title": "PurviewAppLocation",
            "type": "object"
         },
         "PurviewLocationType": {
            "description": "The type of location for
Purview policy evaluation.",
            "enum": [
               "application",
               "uri",
               "domain"
            ],
            "title": "PurviewLocationType",
            "type": "string"
         }
      },
      "required": [
         "app_name"
      ]
   }
```


Enums

Expand table

PurviewLocationType	The type of location for Purview policy evaluation.

Functions

acquire_token

Acquire an authentication token using MSAL Public Client Application.

This function attempts to acquire a token silently first (using cached tokens), and falls back to interactive authentication if needed.

Python

```
acquire_token(*, client_id: str, tenant_id: str, username: str | None = None,
token_cache: Any | None = None, scopes: list[str] | None = None) -> str
```

Parameters

Expand table

Name	Description
client_id Required*	str
tenant_id Required*	str
username Required*	str | None
token_cache Required*	Any | None
scopes Required*	list[str] | None

Keyword-Only Parameters

Expand table

Name	Description
client_id	The client ID of the application.
tenant_id	The tenant ID for authentication.
username	Optional username to filter accounts. Default value: None
token_cache	Optional token cache for storing tokens. Default value: None
scopes	Optional list of scopes. Defaults to Power Platform API scopes. Default value: None

Returns

Expand table

Type	Description
str	The access token string.

Exceptions

Expand table

Type	Description
ServiceException	If authentication token cannot be acquired.

CacheProvider Class

Protocol for cache providers used by Purview integration.

Constructor

```Python
CacheProvider()
```

Methods

Expand table

__init__	
__new__	
get	Get a value from the cache.
remove	Remove a value from the cache.
set	Set a value in the cache.

__init__

```Python
__init__(*args, **kwargs)
```

__new__

```Python
__new__(**kwargs)
```

get

Get a value from the cache.

```Python
```

```
async get(key: str) -> Any | None
```

Parameters

Expand table

Name	Description
key Required*	str The cache key.

Returns

Expand table

Type	Description
Any | None	The cached value or None if not found or expired.

remove

Remove a value from the cache.

Python

```
async remove(key: str) -> None
```

Parameters

Expand table

Name	Description
key Required*	str The cache key.

Returns

Expand table

Type	Description
None	

set

Set a value in the cache.

Python

```
async set(key: str, value: Any, ttl_seconds: int | None = None) -> None
```

Parameters

Expand table

Name	Description
key Required*	str The cache key.
value Required*	Any The value to cache.
ttl_seconds	int | None Time to live in seconds. If None, uses provider default. Default value: None

Returns

Expand table

Type	Description
None	

CopilotStudioAgent Class

A Copilot Studio Agent.

Initialize the Copilot Studio Agent.

Constructor

```
Python
CopilotStudioAgent()
```

Parameters

Expand table

Name	Description
`client`	Optional pre-configured CopilotClient instance. If not provided, a new client will be created using the other parameters. Default value: None
`settings`	Optional pre-configured ConnectionSettings. If not provided, settings will be created from the other parameters. Default value: None

Keyword-Only Parameters

Expand table

Name	Description
`id`	id of the CopilotAgent Default value: None
`name`	Name of the CopilotAgent Default value: None
`description`	Description of the CopilotAgent Default value: None
`context_providers`	Context Providers, to be used by the copilot agent. Default value: None

Name	Description
middleware	Agent middlewares used by the agent. Default value: None
environment_id	Environment ID of the Power Platform environment containing the Copilot Studio app. Can also be set via COPILOTSTUDIOAGENT__ENVIRONMENTID environment variable. Default value: None
agent_identifier	The agent identifier or schema name of the Copilot to use. Can also be set via COPILOTSTUDIOAGENT__SCHEMANAME environment variable. Default value: None
client_id	The app ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT__AGENTAPPID environment variable. Default value: None
tenant_id	The tenant ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT__TENANTID environment variable. Default value: None
token	Optional pre-acquired authentication token. If not provided, token acquisition will be attempted using MSAL. Default value: None
cloud	The Power Platform cloud to use (Public, GCC, etc.). Default value: None
agent_type	The type of Copilot Studio agent (Copilot, Agent, etc.). Default value: None
custom_power_platform_cloud	Custom Power Platform cloud URL if using a custom environment. Default value: None
username	Optional username for token acquisition. Default value: None
token_cache	Optional token cache for storing authentication tokens. Default value: None
scopes	Optional list of authentication scopes. Defaults to Power Platform API scopes if not provided. Default value: None
env_file_path	Optional path to .env file for loading configuration. Default value: None
env_file_encoding	Encoding of the .env file, defaults to 'utf-8'. Default value: None

Methods

Expand table

__init__	Initialize the Copilot Studio Agent.
__new__	
as_tool	Create an AIFunction tool that wraps this agent.
deserialize_thread	Deserialize a thread from its serialized state.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_new_thread	Return a new AgentThread instance that is compatible with the agent.
run	Get a response from the agent. This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available. Note: For streaming responses, use the run_stream method, which returns intermediate steps and the final result as a stream of AgentRunResponseUpdate objects. Streaming only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.
run_stream	Run the agent as a stream. This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller. Note: An AgentRunResponseUpdate object contains a chunk of a message.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

	Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the Copilot Studio Agent.

Python

```
__init__(client: CopilotClient | None = None, settings: ConnectionSettings | None = None, *, id: str | None = None, name: str | None = None, description: str | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]] | None = None, environment_id: str | None = None, agent_identifier: str | None = None, client_id: str | None = None, tenant_id: str | None = None, token: str | None = None, cloud: PowerPlatformCloud | None = None, agent_type: AgentType | None = None, custom_power_platform_cloud: str | None = None, username: str | None = None, token_cache: Any | None = None, scopes: list[str] | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

Expand table

Name	Description
client	CopilotClient | None Optional pre-configured CopilotClient instance. If not provided, a new client will be created using the other parameters. Default value: None
settings	ConnectionSettings | None Optional pre-configured ConnectionSettings. If not provided, settings will be created from the other parameters. Default value: None
id Required*	str | None
name	str | None

Name	Description
Required*	
description Required*	str | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | None
environment_id Required*	str | None
agent_identifier Required*	str | None
client_id Required*	str | None
tenant_id Required*	str | None
token Required*	str | None
cloud Required*	PowerPlatformCloud | None
agent_type Required*	AgentType | None
custom_power_platform_cloud Required*	str | None
username Required*	str | None
token_cache Required*	Any | None
scopes Required*	list[str] | None
env_file_path Required*	str | None

Name	Description
env_file_encoding Required*	str | None

Keyword-Only Parameters

Expand table

Name	Description
id	id of the CopilotAgent Default value: None
name	Name of the CopilotAgent Default value: None
description	Description of the CopilotAgent Default value: None
context_providers	Context Providers, to be used by the copilot agent. Default value: None
middleware	Agent middlewares used by the agent. Default value: None
environment_id	Environment ID of the Power Platform environment containing the Copilot Studio app. Can also be set via COPILOTSTUDIOAGENT__ENVIRONMENTID environment variable. Default value: None
agent_identifier	The agent identifier or schema name of the Copilot to use. Can also be set via COPILOTSTUDIOAGENT__SCHEMANAME environment variable. Default value: None
client_id	The app ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT__AGENTAPPID environment variable. Default value: None
tenant_id	The tenant ID of the App Registration used for authentication. Can also be set via COPILOTSTUDIOAGENT__TENANTID environment variable. Default value: None
token	Optional pre-acquired authentication token. If not provided, token acquisition will be attempted using MSAL. Default value: None

Name	Description
cloud	The Power Platform cloud to use (Public, GCC, etc.). Default value: None
agent_type	The type of Copilot Studio agent (Copilot, Agent, etc.). Default value: None
custom_power_platform_cloud	Custom Power Platform cloud URL if using a custom environment. Default value: None
username	Optional username for token acquisition. Default value: None
token_cache	Optional token cache for storing authentication tokens. Default value: None
scopes	Optional list of authentication scopes. Defaults to Power Platform API scopes if not provided. Default value: None
env_file_path	Optional path to .env file for loading configuration. Default value: None
env_file_encoding	Encoding of the .env file, defaults to 'utf-8'. Default value: None

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ServiceInitializationError	If required configuration is missing or invalid.

__new__

Python

```
__new__(**kwargs)
```

as_tool

Create an AIFunction tool that wraps this agent.

Python

```
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str
= 'task', arg_description: str | None = None, stream_callback:
Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate],
Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

Expand table

Name	Description
name Required*	str | None
description Required*	str | None
arg_name Required*	str
arg_description Required*	str | None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None

Keyword-Only Parameters

Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task").

Name	Description
	Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

Expand table

Type	Description
AIFunction[<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

Expand table

Type	Description
TypeError	If the agent does not implement AgentProtocol.
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
   from agent_framework import ChatAgent

   # Create an agent
   agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

   # Convert the agent to a tool
   research_tool = agent.as_tool()

   # Use the tool with another agent
   coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated
```

```python
    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
    context_data = {
        "type": "agent_run_context",
        "messages": [{"role": "user", "text": "Hello"}],
        "is_streaming": False,
        "metadata": {"session_id": "abc123"},
        # agent and result are excluded from serialization
    }

    # Inject agent and result during middleware processing
    my_agent = BaseAgent(name="test-agent")
    dependencies = {
        "agent_run_context": {
            "agent": my_agent,
            "result": None,  # Will be populated during execution
        }
    }

    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

run

Get a response from the agent.

This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.

Note: For streaming responses, use the run_stream method, which returns intermediate steps and the final result as a stream of AgentRunResponseUpdate objects. Streaming only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread | None
kwargs	Any

Name	Description
Required*	

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentRunResponse	An agent response item.

run_stream

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

Note: An AgentRunResponseUpdate object contains a chunk of a message.

Python

```
async run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] |
None = None, *, thread: AgentThread | None = None, **kwargs: Any) ->
AsyncIterable[AgentRunResponseUpdate]
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None

Name	Description
	The message(s) to send to the agent. Default value: None
thread Required*	AgentThread | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AsyncIterable[AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[Middleware] | None
```

PurviewAppLocation Class

Identifier representing the app's location for Purview policy evaluation.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python

```
pydantic model PurviewAppLocation
```

Keyword-Only Parameters

Expand table

Name	Description
`location_type` Required*	
`location_value` Required*	

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
__new__	

construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
get_policy_location	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.

model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```
__new__(**kwargs)
```

construct

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None

Name	Description
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

get_policy_location

Python

```
get_policy_location() -> dict[str, str]
```

Returns

Expand table

Type	Description
dict[str, str]	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Name	Description
`encoder`	Default value: PydanticUndefined
`models_as_dict`	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
`_fields_set`	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None

Name	Description
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`mode`	Default value: 'python'
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults	bool

Name	Description
Required*	Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format	Literal['any_of', 'primitive_type_array']

Name	Description
Required*	The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default). • *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors	bool

Name	Description
Required*	Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name	bool | None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

```
Python
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

```
Python
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes

Name	Description
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True

Name	Description
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

Python

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
`value` Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

location_type

The location type.

Python

```
field location_type: PurviewLocationType [Required]
```

location_value

The location value.

Python

```
field location_value: str [Required]
```

PurviewAuthenticationError Class

Authentication / authorization failure (401/403).

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
PurviewAuthenticationError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

PurviewChatPolicyMiddleware Class

Chat middleware variant for Purview policy evaluation.

This allows users to attach Purview enforcement directly to a chat client

Behavior:

- Pre-chat: evaluates outgoing (user + context) messages as an upload activity and can terminate execution if blocked.
- Post-chat: evaluates the received response messages (streaming is not presently supported) and can replace them with a blocked message. Uses the same user_id from the request to ensure consistent user identity throughout the evaluation.

Usage:

Constructor

Python
`PurviewChatPolicyMiddleware()`

Parameters

Expand table

Name	Description
`credential` Required*	
`settings` Required*	
`cache_provider`	Default value: None

Methods

Expand table

__init__

__new__
process

__init__

Python

```
__init__(credential: TokenCredential | AsyncTokenCredential, settings:
PurviewSettings, cache_provider: CacheProvider | None = None) -> None
```

Parameters

Expand table

Name	Description
credential Required*	TokenCredential | AsyncTokenCredential
settings Required*	PurviewSettings
cache_provider	CacheProvider | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

process

Python

```
async process(context: ChatContext, next: Callable[[ChatContext],
Awaitable[None]]) -> None
```

Parameters

Expand table

Name	Description
context Required*	ChatContext
next Required*	Callable[[ChatContext], Awaitable[None]]

Returns

Expand table

Type	Description
None	

PurviewLocationType Enum

The type of location for Purview policy evaluation.

Fields

Expand table

APPLICATION
DOMAIN
URI

PurviewPaymentRequiredError Class

Payment required (402).

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
PurviewPaymentRequiredError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

PurviewPolicyMiddleware Class

Agent middleware that enforces Purview policies on prompt and response.

Accepts either a synchronous TokenCredential or an AsyncTokenCredential.

Usage:

Constructor

Python

```
PurviewPolicyMiddleware()
```

Parameters

Expand table

Name	Description
credential Required*	
settings Required*	
cache_provider	Default value: None

Methods

Expand table

__init__
__new__
process

__init__

Python

```
__init__(credential: TokenCredential | AsyncTokenCredential, settings:
PurviewSettings, cache_provider: CacheProvider | None = None) -> None
```

Parameters

Expand table

Name	Description
credential Required*	TokenCredential | AsyncTokenCredential
settings Required*	PurviewSettings
cache_provider	CacheProvider | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

process

Python

```
async process(context: AgentRunContext, next: Callable[[AgentRunContext],
Awaitable[None]]) -> None
```

Parameters

Expand table

Name	Description
context Required*	AgentRunContext
next Required*	Callable[[AgentRunContext], Awaitable[None]]

Returns

Expand table

Type	Description
None	

PurviewRateLimitError Class

Rate limiting or throttling (429).

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
PurviewRateLimitError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

PurviewRequestError Class

Other non-success HTTP errors.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`PurviewRequestError()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

PurviewServiceError Class

Base exception for Purview errors.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
PurviewServiceError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

PurviewSettings Class

Settings for Purview integration mirroring .NET PurviewSettings.

Initialize the settings class.

JSON


```
{
   "title": "PurviewSettings",
   "description": "Settings for Purview integration mirroring .NET
PurviewSettings.\n\nAttributes:\n    app_name: Public app name.\n    app_version:
Optional version string of the application.\n    tenant_id: Optional tenant id (guid)
of the user making the request.\n    purview_app_location: Optional app location for
policy evaluation.\n    graph_base_uri: Base URI for Microsoft Graph.\n
blocked_prompt_message: Custom message to return when a prompt is blocked by policy.\n
blocked_response_message: Custom message to return when a response is blocked by
policy.\n    ignore_exceptions: If True, all Purview exceptions will be logged but not
thrown in middleware.\n    ignore_payment_required: If True, 402 payment required
errors will be logged but not thrown.\n    cache_ttl_seconds: Time to live for cache
entries in seconds (default 14400 = 4 hours).\n    max_cache_size_bytes: Maximum cache
size in bytes (default 200MB).",
   "type": "object",
   "properties": {
      "env_file_path": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Env File Path"
      },
      "env_file_encoding": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": "utf-8",
         "title": "Env File Encoding"
      },
      "app_name": {
         "title": "App Name",
```

```
            "type": "string"
         },
         "app_version": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "App Version"
         },
         "tenant_id": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Tenant Id"
         },
         "purview_app_location": {
            "anyOf": [
               {
                  "$ref": "#/$defs/PurviewAppLocation"
               },
               {
                  "type": "null"
               }
            ],
            "default": null
         },
         "graph_base_uri": {
            "default": "https://graph.microsoft.com/v1.0/",
            "title": "Graph Base Uri",
            "type": "string"
         },
         "blocked_prompt_message": {
            "default": "Prompt blocked by policy",
            "description": "Message to return when a prompt is blocked by policy.",
            "title": "Blocked Prompt Message",
            "type": "string"
         },
         "blocked_response_message": {
            "default": "Response blocked by policy",
            "description": "Message to return when a response is blocked by policy.",
            "title": "Blocked Response Message",
            "type": "string"
         },
         "ignore_exceptions": {
```

```
            "default": false,
            "description": "If True, all Purview exceptions will be logged but not
thrown in middleware.",
            "title": "Ignore Exceptions",
            "type": "boolean"
         },
         "ignore_payment_required": {
            "default": false,
            "description": "If True, 402 payment required errors will be logged but
not thrown.",
            "title": "Ignore Payment Required",
            "type": "boolean"
         },
         "cache_ttl_seconds": {
            "default": 14400,
            "description": "Time to live for cache entries in seconds (default 14400 =
4 hours).",
            "title": "Cache Ttl Seconds",
            "type": "integer"
         },
         "max_cache_size_bytes": {
            "default": 209715200,
            "description": "Maximum cache size in bytes (default 200MB).",
            "title": "Max Cache Size Bytes",
            "type": "integer"
         }
      },
      "$defs": {
         "PurviewAppLocation": {
            "description": "Identifier representing the app's location for Purview
policy evaluation.",
            "properties": {
               "location_type": {
                  "$ref": "#/$defs/PurviewLocationType",
                  "description": "The location type."
               },
               "location_value": {
                  "description": "The location value.",
                  "title": "Location Value",
                  "type": "string"
               }
            },
            "required": [
               "location_type",
               "location_value"
            ],
            "title": "PurviewAppLocation",
            "type": "object"
         },
         "PurviewLocationType": {
            "description": "The type of location for Purview policy evaluation.",
            "enum": [
               "application",
               "uri",
               "domain"
```

```
            ],
            "title": "PurviewLocationType",
            "type": "string"
         }
      },
      "required": [
         "app_name"
      ]
   }
```

Constructor

Python

```
pydantic settings PurviewSettings
```

Keyword-Only Parameters

Expand table

Name	Description
env_file_path	Default value: None
env_file_encoding	Default value: utf-8
app_name Required*	
app_version	Default value: None
tenant_id	Default value: None
purview_app_location	Default value: None
graph_base_uri	Default value: https://graph.microsoft.com/v1.0/
blocked_prompt_message	Default value: Prompt blocked by policy
blocked_response_message	Default value: Response blocked by policy
ignore_exceptions	Default value: False
ignore_payment_required	Default value: False
cache_ttl_seconds	Default value: 14400
max_cache_size_bytes	Default value: 209715200

Methods

Expand table

__init__	Initialize the settings class.
__new__	Override the **new** method to set the env_prefix.
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
get_scopes	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python

```
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

Expand table

Name	Description
cls Required*	type[<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False

Name	Description
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

```
Python
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

get_scopes

```
Python
get_scopes() -> list[str]
```

Returns

Expand table

Type	Description
list[str]	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Name	Description
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int]
| set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] |
Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.

Name	Description
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True

Name	Description
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool

Name	Description
	If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.

Name	Description
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default). • *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`from_attributes`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
`json_data` Required*	str | bytes | bytearray The JSON data to validate.
`strict` Required*	bool | None Whether to enforce types strictly.
`extra` Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`context` Required*	Any | None Extra variables to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None

Name	Description
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

```
Python
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

```
Python
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
Python
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

app_name

Public app name.

Python

```
field app_name: str [Required]
```

app_version

Optional version string of the application.

Python

```
field app_version: str | None = None
```

tenant_id

Optional tenant id (guid) of the user making the request.

Python

```
field tenant_id: str | None = None
```

purview_app_location

Optional app location for policy evaluation.

Python

```
field purview_app_location: PurviewAppLocation | None = None
```

graph_base_uri

Base URI for Microsoft Graph.

Python

```
field graph_base_uri: str = 'https://graph.microsoft.com/v1.0/'
```

blocked_prompt_message

Custom message to return when a prompt is blocked by policy.

Python

```
field blocked_prompt_message: str = 'Prompt blocked by policy'
```

blocked_response_message

Custom message to return when a response is blocked by policy.

Python

```
field blocked_response_message: str = 'Response blocked by policy'
```

ignore_exceptions

If True, all Purview exceptions will be logged but not thrown in middleware.

Python

```
field ignore_exceptions: bool = False
```

ignore_payment_required

If True, 402 payment required errors will be logged but not thrown.

Python

```
field ignore_payment_required: bool = False
```

cache_ttl_seconds

Time to live for cache entries in seconds (default 14400 = 4 hours).

Python

```
field cache_ttl_seconds: int = 14400
```

max_cache_size_bytes

Maximum cache size in bytes (default 200MB).

Python

```
field max_cache_size_bytes: int = 209715200
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = ''
```

ollama Package

Classes

Expand table

OllamaChatClient	Ollama Chat completion class. Initialize an Ollama Chat client.
OllamaSettings	Ollama settings. Initialize the settings class.

JSON


```
{
   "title": "OllamaSettings",
   "description": "Ollama settings.",
   "type": "object",
   "properties": {
      "env_file_path": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Env File Path"
      },
      "env_file_encoding": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": "utf-8",
         "title": "Env File Encoding"
      },
      "host": {
         "anyOf": [
            {
               "type": "string"
```

```
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Host"
        },
        "model_id": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Model Id"
        }
    }
}
```

OllamaChatClient Class

Ollama Chat completion class.

Initialize an Ollama Chat client.

Constructor

```python
OllamaChatClient()
```

Keyword-Only Parameters

Expand table

Name	Description
host	Ollama server URL, if none *http://localhost:11434* is used. Can be set via the OLLAMA_HOST env variable. Default value: None
client	An optional Ollama Client instance. If not provided, a new instance will be created. Default value: None
model_id	The Ollama chat model ID to use. Can be set via the OLLAMA_MODEL_ID env variable. Default value: None
env_file_path	An optional path to a dotenv (.env) file to load environment variables from. Default value: None
env_file_encoding	The encoding to use when reading the dotenv (.env) file. Defaults to 'utf-8'. Default value: None
****kwargs**	Additional keyword arguments passed to BaseChatClient.

Methods

Expand table

__init__	Initialize an Ollama Chat client.
__new__	

create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an Ollama Chat client.

Python

```
__init__(*, host: str | None = None, client: AsyncClient | None = None, model_id: str | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
host Required*	str | None
client Required*	<xref:ollama._client.AsyncClient> | None
model_id Required*	str | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
host	Ollama server URL, if none *http://localhost:11434* is used. Can be set via the OLLAMA_HOST env variable. Default value: None
client	An optional Ollama Client instance. If not provided, a new instance will be created. Default value: None
model_id	The Ollama chat model ID to use. Can be set via the OLLAMA_MODEL_ID env variable. Default value: None
env_file_path	An optional path to a dotenv (.env) file to load environment variables from.

Name	Description
	Default value: None
env_file_encoding	The encoding to use when reading the dotenv (.env) file. Defaults to 'utf-8'. Default value: None
****kwargs**	Additional keyword arguments passed to BaseChatClient.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None,
```

```
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None

Name	Description
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options	dict[str, Any] | None

Name	Description
Required*	
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None

Name	Description
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None

Name	Description
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None

Name	Description
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None

Name	Description
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'ollama'
```

OllamaSettings Class

Ollama settings.

Initialize the settings class.

JSON


```
{
   "title": "OllamaSettings",
   "description": "Ollama settings.",
   "type": "object",
   "properties": {
      "env_file_path": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Env File Path"
      },
      "env_file_encoding": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": "utf-8",
         "title": "Env File Encoding"
      },
      "host": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
         "default": null,
         "title": "Host"
      },
      "model_id": {
```

```
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Model Id"
         }
      }
   }
```

Constructor

Python

```
pydantic settings OllamaSettings
```

Keyword-Only Parameters

Expand table

Name	Description
env_file_path	Default value: None
env_file_encoding	Default value: utf-8
host	Default value: None
model_id	Default value: None

Methods

Expand table

__init__	Initialize the settings class.
__new__	Override the **new** method to set the env_prefix.
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

	If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires

	the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python
`static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings`

Parameters

Expand table

Name	Description
cls Required*	type[<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python
`construct(_fields_set: set[str] | None = None, **values: Any) -> Self`

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None

Name	Description
Required*	Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined

Name	Description
`models_as_dict`	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
`_fields_set`	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
`values` Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.

Name	Description
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
```

```
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.

Name	Description
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False

Name	Description
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default).

Name	Description
	• *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra	Literal['allow', 'ignore', 'forbid'] | None

Name	Description
Required*	Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`from_attributes` Required*	bool | None Whether to extract data from object attributes.
`context` Required*	Any | None Additional context to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`from_attributes`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.

Name	Description
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
```

```
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None

Name	Description
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'OLLAMA_'
```

host

Python

```
field host: str | None = None
```

model_id

Python

```
field model_id: str | None = None
```

openai Package

Classes

Expand table

OpenAIAssistantsClient	OpenAI Assistants client. Initialize an OpenAI Assistants client.
OpenAIChatClient	OpenAI Chat completion class. Initialize an OpenAI Chat completion client.
OpenAIContentFilterException	AI exception for an error from Azure OpenAI's content filter. Initializes a new instance of the ContentFilterAIException class.
OpenAIResponsesClient	OpenAI Responses client class. Initialize an OpenAI Responses client.
OpenAISettings	OpenAI environment settings. The settings are first loaded from environment variables with the prefix '>>OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing. Initialize the settings class. JSON { "title": "OpenAISettings", "description": "OpenAI environment settings.\n\nThe settings are first loaded from environment variables with the prefix 'OPENAI_'.\nIf the environment variables are not found, the settings can be loaded from a .env file with the\nencoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored;\nhowever, validation will fail alerting that the settings are missing.\n\nKeyword Args:\n api_key: OpenAI API key, see https://platform.openai.com/account/api-keys.\n Can be set via environment variable OPENAI_API_KEY.\n base_url: The base URL for the OpenAI API.\n Can be set via environment variable OPENAI_BASE_URL.\n org_id: This is usually optional unless your account

```
belongs to multiple organizations.\n        Can be set
via environment variable OPENAI_ORG_ID.\n
chat_model_id: The OpenAI chat model ID to use, for
example, gpt-3.5-turbo or gpt-4.\n        Can be set via
environment variable OPENAI_CHAT_MODEL_ID.\n
responses_model_id: The OpenAI responses model ID to use,
for example, gpt-4o or o1.\n        Can be set via
environment variable OPENAI_RESPONSES_MODEL_ID.\n
env_file_path: The path to the .env file to load settings
from.\n    env_file_encoding: The encoding of the .env
file, defaults to 'utf-8'.\n\nExamples:\n    .. code-
block:: python\n\n        from agent_framework.openai
import OpenAISettings\n\n        # Using environment
variables\n        # Set OPENAI_API_KEY=sk-...\n        #
Set OPENAI_CHAT_MODEL_ID=gpt-4\n        settings =
OpenAISettings()\n\n        # Or passing parameters
directly\n        settings =
OpenAISettings(api_key=\"sk-...\", chat_model_id=\"gpt-
4\")\n\n        # Or loading from a .env file\n
settings =
OpenAISettings(env_file_path=\"path/to/.env\")",
      "type": "object",
      "properties": {
         "env_file_path": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Env File Path"
         },
         "env_file_encoding": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
         },
         "api_key": {
            "anyOf": [
               {
                  "format": "password",
                  "type": "string",
                  "writeOnly": true
               },
               {
```

```
                "type": "null"
            }
        ],
        "default": null,
        "title": "Api Key"
    },
    "base_url": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Base Url"
    },
    "org_id": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Org Id"
    },
    "chat_model_id": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Chat Model Id"
    },
    "responses_model_id": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Responses Model Id"
    }
```

```
        }
    }
```

Enums

Expand table

ContentFilterResultSeverity	The severity of the content filter result.

ContentFilterResultSeverity Enum

The severity of the content filter result.

Fields

Expand table

HIGH
LOW
MEDIUM
SAFE

OpenAIAssistantsClient Class

OpenAI Assistants client.

Initialize an OpenAI Assistants client.

Constructor

Python

```
OpenAIAssistantsClient()
```

Keyword-Only Parameters

Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models. Can also be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
assistant_id	The ID of an OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None
assistant_name	The name to use when creating new assistants. Default value: None
assistant_description	The description to use when creating new assistants. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
base_url	

Name	Description
	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Examples

Python

```python
from agent_framework.openai import OpenAIAssistantsClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIAssistantsClient()

# Or passing parameters directly
client = OpenAIAssistantsClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIAssistantsClient(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an OpenAI Assistants client.
__new__	
close	Clean up any assistants we created.
create_agent	Create a ChatAgent with this client.

	This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an OpenAI Assistants client.

Python

```
__init__(*, model_id: str | None = None, assistant_id: str | None = None,
assistant_name: str | None = None, assistant_description: str | None = None,
thread_id: str | None = None, api_key: str | Callable[[], str | Awaitable[str]] |
None = None, org_id: str | None = None, base_url: str | None = None,
default_headers: Mapping[str, str] | None = None, async_client: AsyncOpenAI | None
= None, env_file_path: str | None = None, env_file_encoding: str | None = None,
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
model_id Required*	str | None
assistant_id Required*	str | None
assistant_name Required*	str | None
assistant_description Required*	str | None
thread_id Required*	str | None
api_key Required*	str | Callable[[], str | Awaitable[str]] | None
org_id Required*	str | None
base_url Required*	str | None
default_headers Required*	Mapping[str, str] | None
async_client Required*	<xref:openai.AsyncOpenAI> | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
assistant_id	The ID of an OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None
assistant_name	The name to use when creating new assistants. Default value: None
assistant_description	The description to use when creating new assistants. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
base_url	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables.

Name	Description
	Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.openai import OpenAIAssistantsClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIAssistantsClient()

# Or passing parameters directly
client = OpenAIAssistantsClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIAssistantsClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Clean up any assistants we created.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata	dict[str, Any] | None

Name	Description
Required*	
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	

Name	Description
	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient
```

```
    # Create a client
    client = OpenAIChatClient(model_id="gpt-4")

    # Create an agent using the convenience method
    agent = client.create_agent(
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
```

```
    client_data = {
        "type": "open_ai_chat_client",
        "model_id": "gpt-4o-mini",
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
```

```python
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value	

Name	Description
Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None

Name	Description
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None

Name	Description
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`allow_multiple_tool_calls`	Whether to allow multiple tool calls in a single response. Default value: None
`frequency_penalty`	The frequency penalty to use. Default value: None
`logit_bias`	The logit bias to use. Default value: None
`max_tokens`	The maximum number of tokens to generate. Default value: None
`metadata`	Additional metadata to include in the request. Default value: None
`model_id`	The model_id to use for the agent. Default value: None
`presence_penalty`	The presence penalty to use. Default value: None
`response_format`	The format of the response. Default value: None
`seed`	The random seed to use. Default value: None
`stop`	The stop sequence(s) for the request. Default value: None
`store`	Whether to store the response. Default value: None
`temperature`	The sampling temperature to use. Default value: None
`tool_choice`	The tool choice for the request. Default is *auto*. Default value: None
`tools`	The tools to use for the request. Default value: None
`top_p`	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

OpenAIChatClient Class

OpenAI Chat completion class.

Initialize an OpenAI Chat completion client.

Constructor

Python

```
OpenAIChatClient()
```

Keyword-Only Parameters

Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
base_url	The base URL to use. If provided will override the standard value for an OpenAI connector, the env vars or .env file value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables.

Name	Description
	Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None

Examples

Python

```python
from agent_framework.openai import OpenAIChatClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIChatClient()

# Or passing parameters directly
client = OpenAIChatClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIChatClient(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an OpenAI Chat completion client.
__new__	
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string.

	This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an OpenAI Chat completion client.

Python

```
__init__(*, model_id: str | None = None, api_key: str | Callable[[], str |
Awaitable[str]] | None = None, org_id: str | None = None, default_headers:
Mapping[str, str] | None = None, async_client: AsyncOpenAI | None = None,
instruction_role: str | None = None, base_url: str | None = None, env_file_path:
str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

Expand table

Name	Description
model_id Required*	str | None
api_key Required*	str | Callable[[], str | Awaitable[str]] | None
org_id Required*	str | None
default_headers Required*	Mapping[str, str] | None
async_client Required*	<xref:openai.AsyncOpenAI> | None
instruction_role Required*	str | None
base_url Required*	str | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None

Keyword-Only Parameters

Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None

Name	Description
async_client	An existing client to use. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
base_url	The base URL to use. If provided will override the standard value for an OpenAI connector, the env vars or .env file value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None

Returns

Expand table

Type	Description
None	

Examples

Python

```python
from agent_framework.openai import OpenAIChatClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
client = OpenAIChatClient()

# Or passing parameters directly
client = OpenAIChatClient(model_id="gpt-4", api_key="sk-...")

# Or loading from a .env file
client = OpenAIChatClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None

Name	Description
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None

Name	Description
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent.

Name	Description
	Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request.

Name	Description
	Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
```

```
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
```

```
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None

Name	Description
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None

Name	Description
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None

Name	Description
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None

Name	Description
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

OpenAIContentFilterException Class

AI exception for an error from Azure OpenAI's content filter.

Initializes a new instance of the ContentFilterAIException class.

Constructor

Python

```
OpenAIContentFilterException()
```

Parameters

Expand table

Name	Description
`message` Required*	The error message.
`inner_exception` Required*	The inner exception.

Methods

Expand table

__init__	Initializes a new instance of the ContentFilterAIException class.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Initializes a new instance of the ContentFilterAIException class.

Python

```
__init__(message: str, inner_exception: BadRequestError) -> None
```

Parameters

Expand table

Name	Description
message Required*	str The error message.
inner_exception Required*	<xref:openai.BadRequestError> The inner exception.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

content_filter_code

Python

```
content_filter_code: ContentFilterCodes
```

content_filter_result

Python

```
content_filter_result: dict[str, ContentFilterResult]
```

param

Python

```
param: str | None
```

OpenAIResponsesClient Class

OpenAI Responses client class.

Initialize an OpenAI Responses client.

Constructor

```python
OpenAIResponsesClient()
```

Keyword-Only Parameters

Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable OPENAI_RESPONSES_MODEL_ID. Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
base_url	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None

Name	Description
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Examples

Python

```python
from agent_framework.openai import OpenAIResponsesClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_RESPONSES_MODEL_ID=gpt-4o
client = OpenAIResponsesClient()

# Or passing parameters directly
client = OpenAIResponsesClient(model_id="gpt-4o", api_key="sk-...")

# Or loading from a .env file
client = OpenAIResponsesClient(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize an OpenAI Responses client.
__new__	
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection

	capabilities are available through the `dependencies` parameter.
get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an OpenAI Responses client.

Python

```
__init__(*, model_id: str | None = None, api_key: str | Callable[[], str | Awaitable[str]] | None = None, org_id: str | None = None, base_url: str | None = None, default_headers: Mapping[str, str] | None = None, async_client: AsyncOpenAI | None = None, instruction_role: str | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
model_id	str | None

Name	Description
Required*	
api_key Required*	str | Callable[[], str | Awaitable[str]] | None
org_id Required*	str | None
base_url Required*	str | None
default_headers Required*	Mapping[str, str] | None
async_client Required*	<xref:openai.AsyncOpenAI> | None
instruction_role Required*	str | None
env_file_path Required*	str | None
env_file_encoding Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
model_id	OpenAI model name, see https://platform.openai.com/docs/models . Can also be set via environment variable OPENAI_RESPONSES_MODEL_ID. Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_API_KEY. Default value: None
org_id	The org ID to use. If provided will override the env vars or .env file value. Can also be set via environment variable OPENAI_ORG_ID. Default value: None
base_url	The base URL to use. If provided will override the standard value. Can also be set via environment variable OPENAI_BASE_URL.

Name	Description
	Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, "system" or "developer". If not provided, the default is "system". Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None
kwargs	Other keyword parameters.

Returns

Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.openai import OpenAIResponsesClient

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_RESPONSES_MODEL_ID=gpt-4o
client = OpenAIResponsesClient()

# Or passing parameters directly
client = OpenAIResponsesClient(model_id="gpt-4o", api_key="sk-...")

# Or loading from a .env file
client = OpenAIResponsesClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None

Name	Description
name Required*	str | None
description Required*	str | None
instructions Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None

Name	Description
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent.

Name	Description
	Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request.

Name	Description
	Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
```

```
        name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
    )

    # Run the agent
    response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
```

```
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None

Name	Description
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None

Name	Description
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None

Name	Description
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None

Name	Description
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

FILE_SEARCH_MAX_RESULTS

Python

```
FILE_SEARCH_MAX_RESULTS: int = 50
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

OpenAISettings Class

OpenAI environment settings.

The settings are first loaded from environment variables with the prefix '>>OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON


```
{
   "title": "OpenAISettings",
   "description": "OpenAI environment settings.\n\nThe settings are first loaded
from environment variables with the prefix 'OPENAI_'.\nIf the environment variables
are not found, the settings can be loaded from a .env file with the\nencoding 'utf-8'.
If the settings are not found in the .env file, the settings are ignored;\nhowever,
validation will fail alerting that the settings are missing.\n\nKeyword Args:\n
api_key: OpenAI API key, see https://platform.openai.com/account/api-keys.\n
Can be set via environment variable OPENAI_API_KEY.\n    base_url: The base URL for
the OpenAI API.\n        Can be set via environment variable OPENAI_BASE_URL.\n
org_id: This is usually optional unless your account belongs to multiple
organizations.\n        Can be set via environment variable OPENAI_ORG_ID.\n
chat_model_id: The OpenAI chat model ID to use, for example, gpt-3.5-turbo or gpt-4.\n
Can be set via environment variable OPENAI_CHAT_MODEL_ID.\n    responses_model_id: The
OpenAI responses model ID to use, for example, gpt-4o or o1.\n        Can be set via
environment variable OPENAI_RESPONSES_MODEL_ID.\n    env_file_path: The path to the
.env file to load settings from.\n    env_file_encoding: The encoding of the .env
file, defaults to 'utf-8'.\n\nExamples:\n    .. code-block:: python\n\n        from
agent_framework.openai import OpenAISettings\n\n        # Using environment
variables\n        # Set OPENAI_API_KEY=sk-...\n        # Set
OPENAI_CHAT_MODEL_ID=gpt-4\n        settings = OpenAISettings()\n\n        # Or
passing parameters directly\n        settings = OpenAISettings(api_key=\"sk-...\",
chat_model_id=\"gpt-4\")\n\n        # Or loading from a .env file\n        settings =
OpenAISettings(env_file_path=\"path/to/.env\")",
   "type": "object",
   "properties": {
      "env_file_path": {
         "anyOf": [
            {
               "type": "string"
            },
            {
               "type": "null"
            }
         ],
```

```
            "default": null,
            "title": "Env File Path"
         },
         "env_file_encoding": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
         },
         "api_key": {
            "anyOf": [
               {
                  "format": "password",
                  "type": "string",
                  "writeOnly": true
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Api Key"
         },
         "base_url": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Base Url"
         },
         "org_id": {
            "anyOf": [
               {
                  "type": "string"
               },
               {
                  "type": "null"
               }
            ],
            "default": null,
            "title": "Org Id"
         },
         "chat_model_id": {
            "anyOf": [
```

```
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Chat Model Id"
        },
        "responses_model_id": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Responses Model Id"
        }
    }
}
```

Constructor

Python

```
pydantic settings OpenAISettings
```

Keyword-Only Parameters

Expand table

Name	Description
api_key	OpenAI API key, see https://platform.openai.com/account/api-keys. Can be set via environment variable OPENAI_API_KEY. Default value: None
base_url	The base URL for the OpenAI API. Can be set via environment variable OPENAI_BASE_URL. Default value: None
org_id	This is usually optional unless your account belongs to multiple organizations. Can be set via environment variable OPENAI_ORG_ID. Default value: None

Name	Description
chat_model_id	The OpenAI chat model ID to use, for example, gpt-3.5-turbo or gpt-4. Can be set via environment variable OPENAI_CHAT_MODEL_ID. Default value: None
responses_model_id	The OpenAI responses model ID to use, for example, gpt-4o or o1. Can be set via environment variable OPENAI_RESPONSES_MODEL_ID. Default value: None
env_file_path	The path to the .env file to load settings from. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python

```python
from agent_framework.openai import OpenAISettings

# Using environment variables
# Set OPENAI_API_KEY=sk-...
# Set OPENAI_CHAT_MODEL_ID=gpt-4
settings = OpenAISettings()

# Or passing parameters directly
settings = OpenAISettings(api_key="sk-...", chat_model_id="gpt-4")

# Or loading from a .env file
settings = OpenAISettings(env_file_path="path/to/.env")
```

Methods

Expand table

__init__	Initialize the settings class.
__new__	Override the **new** method to set the env_prefix.
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use:

	`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

```
Python
static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings
```

Parameters

Expand table

Name	Description
cls Required*	type[<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

```
Python
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None

Name	Description
Required*	Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined

Name	Description
`models_as_dict`	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
`_fields_set`	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
`values` Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.

Name	Description
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
```

```
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.

Name	Description
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False

Name	Description
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default).

Name	Description
	• *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra	Literal['allow', 'ignore', 'forbid'] | None

Name	Description
Required*	Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`from_attributes` Required*	bool | None Whether to extract data from object attributes.
`context` Required*	Any | None Additional context to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`from_attributes`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.

Name	Description
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
```

```
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None

Name	Description
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
Python
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
Python
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

api_key

Python

```
field api_key: SecretStr | Callable[[], str | Awaitable[str]] | None = None
```

base_url

Python

```
field base_url: str | None = None
```

chat_model_id

Python

```
field chat_model_id: str | None = None
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'OPENAI_'
```

org_id

Python

```
field org_id: str | None = None
```

responses_model_id

Python

```
field responses_model_id: str | None = None
```

redis Package

Classes

Expand table

RedisChatMessageStore	Redis-backed implementation of ChatMessageStoreProtocol using Redis Lists. This implementation provides persistent, thread-safe chat message storage using Redis Lists. Messages are stored as JSON-serialized strings in chronological order, with each conversation thread isolated by a unique Redis key. # Key Features: - **Persistent Storage**: Messages survive application restarts and crashes - **Thread Isolation**: Each conversation thread has its own Redis key namespace - **Auto Message Limits**: Configurable automatic trimming of old messages using LTRIM - **Performance Optimized**: Uses native Redis operations for efficiency - **State Serialization**: Full compatibility with Agent Framework thread serialization - **Initial Message Support**: Pre-load conversations with existing message history - **Production Ready**: Atomic operations, error handling, connection pooling Redis Operations: - RPUSH: Add messages to the end of the list (chronological order) - LRANGE: Retrieve messages in chronological order - LTRIM: Maintain message limits by trimming old messages - DELETE: Clear all messages for a thread Initialize the Redis chat message store. Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider. param redis_url: Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider. param credential_provider: Redis credential provider (redis.credentials.CredentialProvider) for Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url.

param host: Redis host name (e.g., "myredis.redis.cache.windows.net"). Required when using credential_provider.

param port: Redis port number. Defaults to 6380 (Azure Redis SSL port).

param ssl: Enable SSL/TLS connection. Defaults to True.

param username: Redis username. Defaults to None.

param thread_id: Unique identifier for this conversation thread. If not provided, a UUID will be auto-generated. This becomes part of the Redis key: {key_prefix}: {thread_id}

param key_prefix: Prefix for Redis keys to namespace different applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios.

param max_messages: Maximum number of messages to retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM. None means unlimited storage.

param messages: Initial messages to pre-populate the conversation. These are added to Redis on first access if the Redis key is empty. Useful for resuming conversations or seeding with context.

raises ValueError: If neither redis_url nor credential_provider is provided.

raises ValueError: If both redis_url and credential_provider are provided.

raises ValueError: If credential_provider is used without host parameter.

Examples

Traditional connection: store = RedisChatMessageStore(redis_url="redis://localhost:6379", thread_id="conversation_123"

)

Azure Managed Redis with credential provider: from redis.credentials import CredentialProvider from azure.identity.aio import DefaultAzureCredential

store = RedisChatMessageStore(credential_provider=CredentialProvider(DefaultAzureCredential()), host="myredis.redis.cache.windows.net", thread_id="conversation_123"

)

init(redis_url: str | None = None, credential_provider: CredentialProvider | None = None, host: str | None = None, port: int = 6380, ssl: bool = True, username: str | None = None, thread_id: str | None = None, key_prefix: str = 'chat_messages', max_messages: int | None = None, messages: Sequence[ChatMessage] | None = None) -> None

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

Parameters: * **redis_url** (str* | *None) – Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider.

```
  * **credential_provider**
(<xref:redis.credentials.CredentialProvider>* | *<xref:None>)
– Redis credential provider
(redis.credentials.CredentialProvider) for Azure AD
authentication. Requires host parameter. Mutually exclusive
with redis_url.

  * **host** (<xref:str>* | *<xref:None>) – Redis host name
(e.g., "myredis.redis.cache.windows.net"). Required when using
credential_provider.

  * **port** (<xref:int>) – Redis port number. Defaults to
6380 (Azure Redis SSL port).

  * **ssl** (<xref:bool>) – Enable SSL/TLS connection.
Defaults to True.

  * **username** (<xref:str>* | *<xref:None>) – Redis
username. Defaults to None.

  * **thread_id** (<xref:str>* | *<xref:None>) – Unique
identifier for this conversation thread. If not provided, a
UUID will be auto-generated. This becomes part of the Redis
key: {key_prefix}:{thread_id}

  * **key_prefix** (<xref:str>) – Prefix for Redis keys to
namespace different applications. Defaults to 'chat_messages'.
Useful for multi-tenant scenarios.

  * **max_messages** (<xref:int>* | *<xref:None>) – Maximum
number of messages to retain in Redis. When exceeded, oldest
messages are automatically trimmed using LTRIM. None means
unlimited storage.

  * **messages** (<xref:collections.abc.Sequence>*[*
<xref:agent_framework.ChatMessage>*] **| *<xref:None>) –
Initial messages to pre-populate the conversation. These are
added to Redis on first access if the Redis key is empty.
Useful for resuming conversations or seeding with context.
```

Raises: * ValueError – If neither redis_url nor credential_provider is provided.

```
  * <xref:ValueError> - If both redis_url and
credential_provider are provided.

  * <xref:ValueError> - If credential_provider is used without
host parameter.
```

Return type: None

Examples

Traditional connection: store = RedisChatMessageStore(redis_url="redis://localhost:6379", thread_id="conversation_123"

```

  )
```

Azure Managed Redis with credential provider: from redis.credentials import CredentialProvider from azure.identity.aio import DefaultAzureCredential

```
  store = RedisChatMessageStore(

credential_provider=CredentialProvider(DefaultAzureCredential(
)),
     host="myredis.redis.cache.windows.net",
     thread_id="conversation_123"

  )
```

new(**kwargs)

async aclose() -> None

Close the Redis connection.

This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.

Return type: None

async add_messages(messages: Sequence[ChatMessage]) -> None

Add messages to the Redis store (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured.

Parameters: **messages** (Sequence[ChatMessage]) – Sequence of ChatMessage objects to add to the store. Can be empty (no-op) or contain multiple messages.

Return type: None

Thread Safety:

- Atomic pipeline ensures all messages are added together
- LTRIM operation is atomic for consistent message limits

Example

Python

```
   messages = [ChatMessage(role="user", text="Hello"),
ChatMessage(role="assistant", text="Hi there!")]
   await store.add_messages(messages)
   ````

async append(item: ChatMessage) -> <xref:None>

Append a message to the end of the store.

Parameters:
   item (<xref:agent_framework.ChatMessage>) – The
ChatMessage to append.

Return type:
   <xref:None>

async clear() -> <xref:None>

Remove all messages from the store.

Permanently deletes all messages for this conversation thread
by removing
the Redis key. This operation cannot be undone.

Warning:
- This permanently deletes all conversation history
- Consider exporting messages before clearing if backup is
needed

Example

<!-- literal_block {"ids": [], "classes": [], "names": [],
"dupnames": [], "backrefs": [], "xml:space": "preserve",
"force": false, "language": "python", "highlight_args": {},
"linenos": false} -->
```
```

````python

   # Clear conversation history
   await store.clear()

   # Verify messages are gone
   messages = await store.list_messages()
   assert len(messages) == 0
   ````

Return type:
   <xref:None>

async count() -> <xref:int>

Return the number of messages in the Redis store.

Returns:
   The count of messages currently stored in Redis.

Return type:
   <xref:int>

async classmethod deserialize(serialized_store_state: Any,
**kwargs: Any) ->
<xref:agent_framework_redis._chat_message_store.RedisChatMessa
geStore>

Deserialize state data into a new store instance
(ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol
protocol for state deserialization.
Creates a new RedisChatMessageStore instance from previously
serialized data,
allowing the store to reconnect to the same conversation data
in Redis.

Parameters:
   * **serialized_store_state**
(<xref:agent_framework.exceptions.Any>) – Previously
serialized state data from serialize_state(). Should be a
dictionary with thread_id, redis_url, etc.

   * **kwargs** (<xref:agent_framework.exceptions.Any>) –

Keyword Arguments:
   ****kwargs** – Additional arguments passed to Pydantic
model validation.

Returns:
   A new RedisChatMessageStore instance configured from the

```
serialized state.

Raises:
   <xref:ValueError> – If required fields are missing or
invalid in the serialized state.

Return type:

<xref:agent_framework_redis._chat_message_store.RedisChatMessa
geStore>

async extend(items: Sequence[ChatMessage]) -> <xref:None>


Extend the store by appending all messages from the iterable.

Parameters:
   **items** (<xref:collections.abc.Sequence>*[*
<xref:agent_framework.ChatMessage>*]*) – Sequence of
ChatMessage objects to append.

Return type:
   <xref:None>

async getitem(index: int) ->
<xref:agent_framework.ChatMessage>


Get a message by index using Redis LINDEX.

Parameters:
   **index** (<xref:int>) – The index of the message to
retrieve.

Returns:
   The ChatMessage at the specified index.

Raises:
   <xref:IndexError> – If the index is out of range.

Return type:
   <xref:agent_framework.ChatMessage>

async index(item: ChatMessage) -> <xref:int>


Return the index of the first occurrence of the specified
message.

Uses Redis LINDEX to iterate through the list without loading
all messages.
Still O(N) but more memory efficient for large lists.

Parameters:
   **item** (<xref:agent_framework.ChatMessage>) – The
```

```
ChatMessage to find.

Returns:
   The index of the first occurrence of the message.

Raises:
   <xref:ValueError> – If the message is not found in the
store.

Return type:
   <xref:int>

async list_messages() -> <xref:list>
[<xref:agent_framework.ChatMessage>]


Get all messages from the store in chronological order
(ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol
protocol for retrieving messages.
Returns all messages stored in Redis, ordered from oldest
(index 0) to newest (index -1).

Returns:
   List of ChatMessage objects in chronological order (oldest
first).
   Returns empty list if no messages exist or if Redis
connection fails.

Return type:
   <xref:list>[<xref:agent_framework.ChatMessage>]



#### Example

<!-- literal_block {"ids": [], "classes": [], "names": [],
"dupnames": [], "backrefs": [], "xml:space": "preserve",
"force": false, "language": "python", "highlight_args": {},
"linenos": false} -->

````python

   # Get all conversation history
   messages = await store.list_messages()
   ````

async ping() -> <xref:bool>

Test the Redis connection.

Returns:
   True if the connection is successful, False otherwise.
```

```
Return type:
   <xref:bool>

property redis_key: <xref:str>


Get the Redis key for this thread's messages.

The key format is: {key_prefix}:{thread_id}

Returns:
   Redis key string used for storing this thread's messages.



#### ExampleFor key_prefix="chat_messages" and
thread_id="user_123_session_456":
Returns "chat_messages:user_123_session_456"

async remove(item: ChatMessage) -> <xref:None>


Remove the first occurrence of the specified message from the
store.

Uses Redis LREM command for efficient removal by value.
O(N) but performed natively in Redis without data transfer.

Parameters:
   **item** (<xref:agent_framework.ChatMessage>) – The
ChatMessage to remove.

Raises:
   <xref:ValueError> – If the message is not found in the
store.

Return type:
   <xref:None>

async serialize(**kwargs: Any) ->
<xref:agent_framework.exceptions.Any>


Serialize the current store state for persistence
(ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol
protocol for state serialization.
Captures the Redis connection configuration and thread
information needed to
reconstruct the store and reconnect to the same conversation
data.

Keyword Arguments:
```

```
   ****kwargs** – Additional arguments passed to Pydantic
model_dump() for serialization.
   Common options: exclude_none=True, by_alias=True

Returns:
   Dictionary containing serialized store configuration that
can be persisted
   to databases, files, or other storage mechanisms.

Parameters:
   **kwargs** (<xref:agent_framework.exceptions.Any>) –

Return type:
   <xref:agent_framework.exceptions.Any>

async setitem(index: int, item: ChatMessage) -> <xref:None>


Set a message at the specified index using Redis LSET.

Parameters:
   * **index** (<xref:int>) – The index at which to set the
message.

   * **item** (<xref:agent_framework.ChatMessage>) – The
ChatMessage to set at the specified index.

Raises:
   <xref:IndexError> – If the index is out of range.

Return type:
   <xref:None>

async update_from_state(serialized_store_state: Any, **kwargs:
Any) -> <xref:None>


Deserialize state data into this store instance
(ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol
protocol for state deserialization.
Restores the store configuration from previously serialized
data, allowing the store
to reconnect to the same conversation data in Redis.

Parameters:
   * **serialized_store_state**
(<xref:agent_framework.exceptions.Any>) – Previously
serialized state data from serialize_state(). Should be a
dictionary with thread_id, redis_url, etc.

   * **kwargs** (<xref:agent_framework.exceptions.Any>) –

Keyword Arguments:
```

	``` ****kwargs** – Additional arguments passed to Pydantic model validation. Return type: <xref:None> ```
RedisProvider	Redis context provider with dynamic, filterable schema. Stores context in Redis and retrieves scoped context. Uses full-text or optional hybrid vector search to ground model responses. Create a Redis Context Provider.

RedisChatMessageStore Class

Redis-backed implementation of ChatMessageStoreProtocol using Redis Lists.

This implementation provides persistent, thread-safe chat message storage using Redis Lists. Messages are stored as JSON-serialized strings in chronological order, with each conversation thread isolated by a unique Redis key.

Key Features:

- **Persistent Storage**: Messages survive application restarts and crashes
- **Thread Isolation**: Each conversation thread has its own Redis key namespace
- **Auto Message Limits**: Configurable automatic trimming of old messages using LTRIM
- **Performance Optimized**: Uses native Redis operations for efficiency
- **State Serialization**: Full compatibility with Agent Framework thread serialization
- **Initial Message Support**: Pre-load conversations with existing message history
- **Production Ready**: Atomic operations, error handling, connection pooling

Redis Operations:

- RPUSH: Add messages to the end of the list (chronological order)
- LRANGE: Retrieve messages in chronological order
- LTRIM: Maintain message limits by trimming old messages
- DELETE: Clear all messages for a thread

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

param redis_url: Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider.

param credential_provider: Redis credential provider (redis.credentials.CredentialProvider) for Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url.

param host: Redis host name (e.g., "myredis.redis.cache.windows.net"). Required when using credential_provider.

param port: Redis port number. Defaults to 6380 (Azure Redis SSL port).

param ssl: Enable SSL/TLS connection. Defaults to True.

param username: Redis username. Defaults to None.

param thread_id: Unique identifier for this conversation thread. If not provided, a UUID will be auto-generated. This becomes part of the Redis key: {key_prefix}:{thread_id}

param key_prefix: Prefix for Redis keys to namespace different applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios.

param max_messages: Maximum number of messages to retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM. None means unlimited storage.

param messages: Initial messages to pre-populate the conversation. These are added to Redis on first access if the Redis key is empty. Useful for resuming conversations or seeding with context.

raises ValueError: If neither redis_url nor credential_provider is provided.

raises ValueError: If both redis_url and credential_provider are provided.

raises ValueError: If credential_provider is used without host parameter.

Examples

Traditional connection: store = RedisChatMessageStore(redis_url="redis://localhost:6379", thread_id="conversation_123"

)

Azure Managed Redis with credential provider: from redis.credentials import CredentialProvider from azure.identity.aio import DefaultAzureCredential

store = RedisChatMessageStore(
credential_provider=CredentialProvider(DefaultAzureCredential()),
host="myredis.redis.cache.windows.net", thread_id="conversation_123"

)

init(redis_url: str | None = None, credential_provider: CredentialProvider | None = None, host: str | None = None, port: int = 6380, ssl: bool = True, username: str | None = None, thread_id: str | None = None, key_prefix: str = 'chat_messages', max_messages: int | None = None, messages: Sequence[ChatMessage] | None = None) -> None

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

Parameters: * **redis_url** (str* | *None) – Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider.

```
  * **credential_provider** (<xref:redis.credentials.CredentialProvider>* | *
<xref:None>) – Redis credential provider (redis.credentials.CredentialProvider) for
Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url.

  * **host** (<xref:str>* | *<xref:None>) – Redis host name (e.g.,
"myredis.redis.cache.windows.net"). Required when using credential_provider.

  * **port** (<xref:int>) – Redis port number. Defaults to 6380 (Azure Redis SSL
port).

  * **ssl** (<xref:bool>) – Enable SSL/TLS connection. Defaults to True.

  * **username** (<xref:str>* | *<xref:None>) – Redis username. Defaults to None.

  * **thread_id** (<xref:str>* | *<xref:None>) – Unique identifier for this
conversation thread. If not provided, a UUID will be auto-generated. This becomes part
of the Redis key: {key_prefix}:{thread_id}

  * **key_prefix** (<xref:str>) – Prefix for Redis keys to namespace different
applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios.

  * **max_messages** (<xref:int>* | *<xref:None>) – Maximum number of messages to
retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM.
None means unlimited storage.

  * **messages** (<xref:collections.abc.Sequence>*[*
<xref:agent_framework.ChatMessage>*] **| *<xref:None>) – Initial messages to pre-
populate the conversation. These are added to Redis on first access if the Redis key
is empty. Useful for resuming conversations or seeding with context.
```

Raises: * ValueError – If neither redis_url nor credential_provider is provided.

```
  * <xref:ValueError> – If both redis_url and credential_provider are provided.

  * <xref:ValueError> – If credential_provider is used without host parameter.
```

Return type: None

Examples

Traditional connection: store = RedisChatMessageStore(redis_url="redis://localhost:6379", thread_id="conversation_123"

```
)
```

Azure Managed Redis with credential provider: from redis.credentials import CredentialProvider from azure.identity.aio import DefaultAzureCredential

```
store = RedisChatMessageStore(
   credential_provider=CredentialProvider(DefaultAzureCredential()),
   host="myredis.redis.cache.windows.net",
   thread_id="conversation_123"

)
```

new(**kwargs)

async aclose() -> None

Close the Redis connection.

This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.

Return type: None

async add_messages(messages: Sequence[ChatMessage]) -> None

Add messages to the Redis store (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured.

Parameters: **messages** (Sequence[ChatMessage]) – Sequence of ChatMessage objects to add to the store. Can be empty (no-op) or contain multiple messages.

Return type: None

Thread Safety:

- Atomic pipeline ensures all messages are added together
- LTRIM operation is atomic for consistent message limits

Example

Python

```
   messages = [ChatMessage(role="user", text="Hello"), ChatMessage(role="assistant",
text="Hi there!")]
   await store.add_messages(messages)
   ````

async append(item: ChatMessage) -> <xref:None>

Append a message to the end of the store.

Parameters:
   item (<xref:agent_framework.ChatMessage>) – The ChatMessage to append.

Return type:
   <xref:None>

async clear() -> <xref:None>

Remove all messages from the store.

Permanently deletes all messages for this conversation thread by removing
the Redis key. This operation cannot be undone.

Warning:
- This permanently deletes all conversation history
- Consider exporting messages before clearing if backup is needed

Example

<!-- literal_block {"ids": [], "classes": [], "names": [], "dupnames": [],
"backrefs": [], "xml:space": "preserve", "force": false, "language": "python",
"highlight_args": {}, "linenos": false} -->

````python

   # Clear conversation history
   await store.clear()

   # Verify messages are gone
   messages = await store.list_messages()
```

```
    assert len(messages) == 0
    ````

Return type:
    <xref:None>

async count() -> <xref:int>

Return the number of messages in the Redis store.

Returns:
    The count of messages currently stored in Redis.

Return type:
    <xref:int>

async classmethod deserialize(serialized_store_state: Any, **kwargs: Any) ->
<xref:agent_framework_redis._chat_message_store.RedisChatMessageStore>

Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol
method).

This method implements the required ChatMessageStoreProtocol protocol for state
deserialization.
Creates a new RedisChatMessageStore instance from previously serialized data,
allowing the store to reconnect to the same conversation data in Redis.

Parameters:
    * **serialized_store_state** (<xref:agent_framework.exceptions.Any>) – Previously
serialized state data from serialize_state(). Should be a dictionary with thread_id,
redis_url, etc.

    * **kwargs** (<xref:agent_framework.exceptions.Any>) –

Keyword Arguments:
    ****kwargs** – Additional arguments passed to Pydantic model validation.

Returns:
    A new RedisChatMessageStore instance configured from the serialized state.

Raises:
    <xref:ValueError> – If required fields are missing or invalid in the serialized
state.

Return type:
    <xref:agent_framework_redis._chat_message_store.RedisChatMessageStore>

async extend(items: Sequence[ChatMessage]) -> <xref:None>

Extend the store by appending all messages from the iterable.

Parameters:
```
```

```
   **items** (<xref:collections.abc.Sequence>*[*<xref:agent_framework.ChatMessage>*]*)
– Sequence of ChatMessage objects to append.

Return type:
   <xref:None>

async getitem(index: int) -> <xref:agent_framework.ChatMessage>


Get a message by index using Redis LINDEX.

Parameters:
   **index** (<xref:int>) – The index of the message to retrieve.

Returns:
   The ChatMessage at the specified index.

Raises:
   <xref:IndexError> – If the index is out of range.

Return type:
   <xref:agent_framework.ChatMessage>

async index(item: ChatMessage) -> <xref:int>


Return the index of the first occurrence of the specified message.

Uses Redis LINDEX to iterate through the list without loading all messages.
Still O(N) but more memory efficient for large lists.

Parameters:
   **item** (<xref:agent_framework.ChatMessage>) – The ChatMessage to find.

Returns:
   The index of the first occurrence of the message.

Raises:
   <xref:ValueError> – If the message is not found in the store.

Return type:
   <xref:int>

async list_messages() -> <xref:list>[<xref:agent_framework.ChatMessage>]


Get all messages from the store in chronological order (ChatMessageStoreProtocol
protocol method).

This method implements the required ChatMessageStoreProtocol protocol for retrieving
messages.
Returns all messages stored in Redis, ordered from oldest (index 0) to newest (index
-1).

Returns:
```

```
   List of ChatMessage objects in chronological order (oldest first).
   Returns empty list if no messages exist or if Redis connection fails.

Return type:
   <xref:list>[<xref:agent_framework.ChatMessage>]



#### Example

<!-- literal_block {"ids": [], "classes": [], "names": [], "dupnames": [],
"backrefs": [], "xml:space": "preserve", "force": false, "language": "python",
"highlight_args": {}, "linenos": false} -->

````python

   # Get all conversation history
   messages = await store.list_messages()
   ````

async ping() -> <xref:bool>

Test the Redis connection.

Returns:
   True if the connection is successful, False otherwise.

Return type:
   <xref:bool>

property redis_key: <xref:str>

Get the Redis key for this thread's messages.

The key format is: {key_prefix}:{thread_id}

Returns:
   Redis key string used for storing this thread's messages.

ExampleFor key_prefix="chat_messages" and thread_id="user_123_session_456":
Returns "chat_messages:user_123_session_456"

async remove(item: ChatMessage) -> <xref:None>

Remove the first occurrence of the specified message from the store.

Uses Redis LREM command for efficient removal by value.
O(N) but performed natively in Redis without data transfer.

Parameters:
```

```
   **item** (<xref:agent_framework.ChatMessage>) – The ChatMessage to remove.

Raises:
   <xref:ValueError> – If the message is not found in the store.

Return type:
   <xref:None>

async serialize(**kwargs: Any) -> <xref:agent_framework.exceptions.Any>


Serialize the current store state for persistence (ChatMessageStoreProtocol protocol
method).

This method implements the required ChatMessageStoreProtocol protocol for state
serialization.
Captures the Redis connection configuration and thread information needed to
reconstruct the store and reconnect to the same conversation data.

Keyword Arguments:
   ****kwargs** – Additional arguments passed to Pydantic model_dump() for
serialization.
   Common options: exclude_none=True, by_alias=True

Returns:
   Dictionary containing serialized store configuration that can be persisted
   to databases, files, or other storage mechanisms.

Parameters:
   **kwargs** (<xref:agent_framework.exceptions.Any>) –

Return type:
   <xref:agent_framework.exceptions.Any>

async setitem(index: int, item: ChatMessage) -> <xref:None>


Set a message at the specified index using Redis LSET.

Parameters:
   * **index** (<xref:int>) – The index at which to set the message.

   * **item** (<xref:agent_framework.ChatMessage>) – The ChatMessage to set at the
specified index.

Raises:
   <xref:IndexError> – If the index is out of range.

Return type:
   <xref:None>

async update_from_state(serialized_store_state: Any, **kwargs: Any) -> <xref:None>


Deserialize state data into this store instance (ChatMessageStoreProtocol protocol
```

```
method).

This method implements the required ChatMessageStoreProtocol protocol for state
deserialization.
Restores the store configuration from previously serialized data, allowing the store
to reconnect to the same conversation data in Redis.

Parameters:
   * **serialized_store_state** (<xref:agent_framework.exceptions.Any>) – Previously
serialized state data from serialize_state(). Should be a dictionary with thread_id,
redis_url, etc.

   * **kwargs** (<xref:agent_framework.exceptions.Any>) –

Keyword Arguments:
   ****kwargs** – Additional arguments passed to Pydantic model validation.

Return type:
   <xref:None>
```

Constructor

Python

```
RedisChatMessageStore()
```

Parameters

Expand table

Name	Description
redis_url	Default value: None
credential_provider	Default value: None
host	Default value: None
port	Default value: 6380
ssl	Default value: True
username	Default value: None
thread_id	Default value: None
key_prefix	Default value: chat_messages
max_messages	Default value: None

Name	Description
messages	Default value: None

Methods

Expand table

__init__	Initialize the Redis chat message store. Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.
__new__	
aclose	Close the Redis connection. This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.
add_messages	Add messages to the Redis store (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured. Thread Safety: • Atomic pipeline ensures all messages are added together • LTRIM operation is atomic for consistent message limits
append	Append a message to the end of the store.
clear	Remove all messages from the store. Permanently deletes all messages for this conversation thread by removing the Redis key. This operation cannot be undone. Warning: • This permanently deletes all conversation history • Consider exporting messages before clearing if backup is needed
count	Return the number of messages in the Redis store.
deserialize	Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol method).

	This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Creates a new RedisChatMessageStore instance from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.
extend	Extend the store by appending all messages from the iterable.
getitem	Get a message by index using Redis LINDEX.
index	Return the index of the first occurrence of the specified message. Uses Redis LINDEX to iterate through the list without loading all messages. Still O(N) but more memory efficient for large lists.
list_messages	Get all messages from the store in chronological order (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for retrieving messages. Returns all messages stored in Redis, ordered from oldest (index 0) to newest (index -1).
ping	Test the Redis connection.
remove	Remove the first occurrence of the specified message from the store. Uses Redis LREM command for efficient removal by value. O(N) but performed natively in Redis without data transfer.
serialize	Serialize the current store state for persistence (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for state serialization. Captures the Redis connection configuration and thread information needed to reconstruct the store and reconnect to the same conversation data.
setitem	Set a message at the specified index using Redis LSET.
update_from_state	Deserialize state data into this store instance (ChatMessageStoreProtocol protocol method). This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Restores the store configuration from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

__init__

Initialize the Redis chat message store.

Creates a Redis-backed chat message store for a specific conversation thread. Supports both traditional URL-based authentication and Azure Managed Redis with credential provider.

Python

```
__init__(redis_url: str | None = None, credential_provider: CredentialProvider | None = None, host: str | None = None, port: int = 6380, ssl: bool = True, username: str | None = None, thread_id: str | None = None, key_prefix: str = 'chat_messages', max_messages: int | None = None, messages: Sequence[ChatMessage] | None = None) -> None
```

Parameters

Expand table

Name	Description
redis_url	str | None Redis connection URL (e.g., "redis://localhost:6379"). Used for traditional authentication. Mutually exclusive with credential_provider. Default value: None
credential_provider	<xref:redis.credentials.CredentialProvider> | None Redis credential provider (redis.credentials.CredentialProvider) for Azure AD authentication. Requires host parameter. Mutually exclusive with redis_url. Default value: None
host	str | None Redis host name (e.g., "myredis.redis.cache.windows.net"). Required when using credential_provider. Default value: None
port	int Redis port number. Defaults to 6380 (Azure Redis SSL port). Default value: 6380
ssl	bool Enable SSL/TLS connection. Defaults to True. Default value: True
username	str | None Redis username. Defaults to None. Default value: None
thread_id	str | None Unique identifier for this conversation thread. If not provided, a UUID will be auto-generated. This becomes part of the Redis key: {key_prefix}:{thread_id} Default value: None
key_prefix	str Prefix for Redis keys to namespace different applications. Defaults to 'chat_messages'. Useful for multi-tenant scenarios. Default value: "chat_messages"

Name	Description
`max_messages`	int | None Maximum number of messages to retain in Redis. When exceeded, oldest messages are automatically trimmed using LTRIM. None means unlimited storage. Default value: None
`messages`	Sequence[ChatMessage] | None Initial messages to pre-populate the conversation. These are added to Redis on first access if the Redis key is empty. Useful for resuming conversations or seeding with context. Default value: None

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ValueError	If neither redis_url nor credential_provider is provided.
ValueError	If both redis_url and credential_provider are provided.
ValueError	If credential_provider is used without host parameter.

Examples

Traditional connection: store = RedisChatMessageStore(redis_url="redis://localhost:6379", thread_id="conversation_123"

)

Azure Managed Redis with credential provider: from redis.credentials import CredentialProvider from azure.identity.aio import DefaultAzureCredential

store = RedisChatMessageStore(
credential_provider=CredentialProvider(DefaultAzureCredential()),

host="myredis.redis.cache.windows.net", thread_id="conversation_123"

)

__new__

```
Python
__new__(**kwargs)
```

aclose

Close the Redis connection.

This method provides a clean way to close the underlying Redis connection when the store is no longer needed. This is particularly useful in samples and applications where explicit resource cleanup is desired.

```
Python
async aclose() -> None
```

Returns

Expand table

Type	Description
None	

add_messages

Add messages to the Redis store (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for adding messages. Messages are appended to the Redis list in chronological order, with automatic trimming if message limits are configured.

Thread Safety:

- Atomic pipeline ensures all messages are added together
- LTRIM operation is atomic for consistent message limits

Python

```
async add_messages(messages: Sequence[ChatMessage]) -> None
```

Parameters

Expand table

Name	Description
messages Required*	Sequence[ChatMessage] Sequence of ChatMessage objects to add to the store. Can be empty (no-op) or contain multiple messages.

Returns

Expand table

Type	Description
None	

Examples

Python

```
   messages = [ChatMessage(role="user", text="Hello"),
ChatMessage(role="assistant", text="Hi there!")]
   await store.add_messages(messages)
```

append

Append a message to the end of the store.

Python

```
async append(item: ChatMessage) -> None
```

Parameters

Expand table

Name	Description
`item` Required*	ChatMessage The ChatMessage to append.

Returns

Expand table

Type	Description
None	

clear

Remove all messages from the store.

Permanently deletes all messages for this conversation thread by removing the Redis key. This operation cannot be undone.

Warning:

- This permanently deletes all conversation history
- Consider exporting messages before clearing if backup is needed

Python

```
async clear() -> None
```

Returns

Expand table

Type	Description
None	

Examples

Python

```
# Clear conversation history
await store.clear()
```

```
    # Verify messages are gone
    messages = await store.list_messages()
    assert len(messages) == 0
```

count

Return the number of messages in the Redis store.

Python

```
async count() -> int
```

Returns

Expand table

Type	Description
int	The count of messages currently stored in Redis.

deserialize

Deserialize state data into a new store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Creates a new RedisChatMessageStore instance from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

Python

```
async classmethod deserialize(serialized_store_state: Any, **kwargs: Any) ->
RedisChatMessageStore
```

Parameters

Expand table

Name	Description
serialized_store_state Required*	Any Previously serialized state data from serialize_state(). Should be a dictionary with thread_id, redis_url, etc.
kwargs	Any

Name	Description
Required*	

Keyword-Only Parameters

Expand table

Name	Description
**kwargs	Additional arguments passed to Pydantic model validation.

Returns

Expand table

Type	Description
RedisChatMessageStore	A new RedisChatMessageStore instance configured from the serialized state.

Exceptions

Expand table

Type	Description
ValueError	If required fields are missing or invalid in the serialized state.

extend

Extend the store by appending all messages from the iterable.

Python

```
async extend(items: Sequence[ChatMessage]) -> None
```

Parameters

Expand table

Name	Description
items Required*	Sequence[ChatMessage] Sequence of ChatMessage objects to append.

Returns

Expand table

Type	Description
None	

getitem

Get a message by index using Redis LINDEX.

```
Python
async getitem(index: int) -> ChatMessage
```

Parameters

Expand table

Name	Description
index Required*	int The index of the message to retrieve.

Returns

Expand table

Type	Description
ChatMessage	The ChatMessage at the specified index.

Exceptions

Expand table

Type	Description
IndexError	If the index is out of range.

index

Return the index of the first occurrence of the specified message.

Uses Redis LINDEX to iterate through the list without loading all messages. Still O(N) but more memory efficient for large lists.

Python

```
async index(item: ChatMessage) -> int
```

Parameters

Expand table

Name	Description
item Required*	ChatMessage The ChatMessage to find.

Returns

Expand table

Type	Description
int	The index of the first occurrence of the message.

Exceptions

Expand table

Type	Description
ValueError	If the message is not found in the store.

list_messages

Get all messages from the store in chronological order (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for retrieving messages. Returns all messages stored in Redis, ordered from oldest (index 0) to newest (index -1).

Python

```
async list_messages() -> list[ChatMessage]
```

Returns

Expand table

Type	Description
list[ChatMessage]	List of ChatMessage objects in chronological order (oldest first). Returns empty list if no messages exist or if Redis connection fails.

Examples

Python

```
# Get all conversation history
messages = await store.list_messages()
```

ping

Test the Redis connection.

Python

```
async ping() -> bool
```

Returns

Expand table

Type	Description
bool	True if the connection is successful, False otherwise.

remove

Remove the first occurrence of the specified message from the store.

Uses Redis LREM command for efficient removal by value. O(N) but performed natively in Redis without data transfer.

Python

```
async remove(item: ChatMessage) -> None
```

Parameters

Expand table

Name	Description
item Required*	ChatMessage The ChatMessage to remove.

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ValueError	If the message is not found in the store.

serialize

Serialize the current store state for persistence (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state serialization. Captures the Redis connection configuration and thread information needed to reconstruct the store and reconnect to the same conversation data.

Python

```
async serialize(**kwargs: Any) -> Any
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional arguments passed to Pydantic model_dump() for serialization. Common options: exclude_none=True, by_alias=True

Returns

Expand table

Type	Description
Any	Dictionary containing serialized store configuration that can be persisted to databases, files, or other storage mechanisms.

setitem

Set a message at the specified index using Redis LSET.

Python

```
async setitem(index: int, item: ChatMessage) -> None
```

Parameters

Expand table

Name	Description
index Required*	int The index at which to set the message.
item Required*	ChatMessage The ChatMessage to set at the specified index.

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
IndexError	If the index is out of range.

update_from_state

Deserialize state data into this store instance (ChatMessageStoreProtocol protocol method).

This method implements the required ChatMessageStoreProtocol protocol for state deserialization. Restores the store configuration from previously serialized data, allowing the store to reconnect to the same conversation data in Redis.

Python

```
async update_from_state(serialized_store_state: Any, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
serialized_store_state Required*	Any Previously serialized state data from serialize_state(). Should be a dictionary with thread_id, redis_url, etc.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
**kwargs	Additional arguments passed to Pydantic model validation.

Returns

Expand table

Type	Description
None	

Attributes

redis_key

Get the Redis key for this thread's messages.

The key format is: {key_prefix}:{thread_id}

Returns

Expand table

Type	Description
	Redis key string used for storing this thread's messages.

Examples

For key_prefix="chat_messages" and thread_id="user_123_session_456": Returns "chat_messages:user_123_session_456"

RedisProvider Class

Redis context provider with dynamic, filterable schema.

Stores context in Redis and retrieves scoped context. Uses full-text or optional hybrid vector search to ground model responses.

Create a Redis Context Provider.

Constructor

```python
RedisProvider()
```

Parameters

Expand table

Name	Description
redis_url	The Redis server URL. Default value: redis://localhost:6379
index_name	The name of the Redis index. Default value: context
prefix	The prefix for all keys in the Redis database. Default value: context
redis_vectorizer	The vectorizer to use for Redis. Default value: None
vector_field_name	The name of the vector field in Redis. Default value: None
vector_algorithm	The algorithm to use for vector search. Default value: None
vector_distance_metric	The distance metric to use for vector search. Default value: None
application_id	The application ID to scope the context. Default value: None

Name	Description
agent_id	The agent ID to scope the context. Default value: None
user_id	The user ID to scope the context. Default value: None
thread_id	The thread ID to scope the context. Default value: None
scope_to_per_operation_thread_id	Whether to scope to the per-operation thread ID. Default value: False
context_prompt	The context prompt to use for the provider. Default value: ## Memories Consider the following memories when answering user questions:
redis_index	The Redis index to use for the provider. Default value: None
overwrite_index	Whether to overwrite the existing Redis index. Default value: False

Methods

Expand table

__init__	Create a Redis Context Provider.
__new__	
invoked	
invoking	Called before invoking the model to provide scoped context. Concatenates recent messages into a query, fetches matching memories from Redis. Prepends them as instructions.
search_all	Returns all documents in the index. Streams results via pagination to avoid excessive memory and response sizes.
thread_created	Called when a new thread is created. Captures the per-operation thread id when scoping is enabled to enforce single-thread usage.

__init__

Create a Redis Context Provider.

Python

```
__init__(redis_url: str = 'redis://localhost:6379', index_name: str = 'context',
prefix: str = 'context', redis_vectorizer: BaseVectorizer | None = None,
vector_field_name: str | None = None, vector_algorithm: Literal['flat', 'hnsw'] |
None = None, vector_distance_metric: Literal['cosine', 'ip', 'l2'] | None = None,
application_id: str | None = None, agent_id: str | None = None, user_id: str |
None = None, thread_id: str | None = None, scope_to_per_operation_thread_id: bool
= False, context_prompt: str = ContextProvider.DEFAULT_CONTEXT_PROMPT,
redis_index: Any = None, overwrite_index: bool = False)
```

Parameters

Expand table

Name	Description
redis_url	str The Redis server URL. Default value: "redis://localhost:6379"
index_name	str The name of the Redis index. Default value: "context"
prefix	str The prefix for all keys in the Redis database. Default value: "context"
redis_vectorizer	<xref:redisvl.utils.vectorize.base.BaseVectorizer> | None The vectorizer to use for Redis. Default value: None
vector_field_name	str | None The name of the vector field in Redis. Default value: None
vector_algorithm	Literal['flat', 'hnsw'] | None The algorithm to use for vector search. Default value: None
vector_distance_metric	Literal['cosine', 'ip', 'l2'] | None The distance metric to use for vector search. Default value: None
application_id	str | None The application ID to scope the context. Default value: None

Name	Description
agent_id	str | None The agent ID to scope the context. Default value: None
user_id	str | None The user ID to scope the context. Default value: None
thread_id	str | None The thread ID to scope the context. Default value: None
scope_to_per_operation_thread_id	bool Whether to scope to the per-operation thread ID. Default value: False
context_prompt	str The context prompt to use for the provider. Default value: ContextProvider.DEFAULT_CONTEXT_PROMPT
redis_index	Any The Redis index to use for the provider. Default value: None
overwrite_index	bool Whether to overwrite the existing Redis index. Default value: False

__new__

Python

```
__new__(**kwargs)
```

invoked

Python

```
async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
request_messages Required*	ChatMessage | Sequence[ChatMessage]
response_messages	ChatMessage | Sequence[ChatMessage] | None Default value: None
invoke_exception	Exception | None Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

invoking

Called before invoking the model to provide scoped context.

Concatenates recent messages into a query, fetches matching memories from Redis. Prepends them as instructions.

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs:
Any) -> Context
```

Parameters

Expand table

Name	Description
messages Required*	ChatMessage | MutableSequence[ChatMessage] List of new messages in the thread.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
**kwargs	not used at present at present.

Returns

Expand table

Type	Description
Context	Context object containing instructions with memories.

search_all

Returns all documents in the index.

Streams results via pagination to avoid excessive memory and response sizes.

Python

```
async search_all(page_size: int = 200) -> list[dict[str, Any]]
```

Parameters

Expand table

Name	Description
page_size	int Page size used for pagination under the hood. Default value: 200

Returns

Expand table

Type	Description
list[dict[str, Any]]	List of all documents.

thread_created

Called when a new thread is created.

Captures the per-operation thread id when scoping is enabled to enforce single-thread usage.

Python

```
async thread_created(thread_id: str | None) -> None
```

Parameters

Expand table

Name	Description
thread_id Required*	str | None The ID of the thread or None.

Returns

Expand table

Type	Description
None	

Attributes

schema_dict

Get the Redis schema dictionary, computing and caching it on first access.

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories when answering user questions:'
```

exceptions Module

Classes

Expand table

AdditionItemMismatch	An error occurred while adding two types. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentException	Base class for all agent exceptions. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentExecutionException	An error occurred while executing the agent. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentFrameworkException	Base exceptions for the Agent Framework. Automatically logs the message as debug. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentInitializationError	An error occurred while initializing the agent. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
AgentThreadException	An error occurred while managing the agent thread. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.

Any	Special type indicating an unconstrained type. • Any is compatible with every type. • Any assumed to have all methods. • All values assumed to be instances of Any. Note that all the above statements are true from the point of view of static type checkers. At runtime, Any should not be used with instance checks.
ChatClientException	An error occurred while dealing with a chat client. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ChatClientInitializationError	An error occurred while initializing the chat client. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ContentError	An error occurred while processing content. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
MiddlewareException	An error occurred during middleware execution. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceContentFilterException	An error was raised by the content filter of the service. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceException	Base class for all service exceptions. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceInitializationError	An error occurred while initializing the service.

	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceInvalidAuthError	An error occurred while authenticating the service. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceInvalidExecutionSettingsError	An error occurred while validating the execution settings of the service. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceInvalidRequestError	An error occurred while validating the request to the service. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceInvalidResponseError	An error occurred while validating the response from the service. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ServiceResponseException	Base class for all service response exceptions. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ToolException	An error occurred while executing a tool. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
ToolExecutionException	An error occurred while executing a tool. Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.

AdditionItemMismatch Class

An error occurred while adding two types.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
AdditionItemMismatch()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

AgentException Class

Base class for all agent exceptions.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`AgentException()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

AgentExecutionException Class

An error occurred while executing the agent.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
AgentExecutionException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

AgentFrameworkException Class

Base exceptions for the Agent Framework.

Automatically logs the message as debug.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python

```
AgentFrameworkException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level: Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] | None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

AgentInitializationError Class

An error occurred while initializing the agent.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`AgentInitializationError()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

AgentThreadException Class

An error occurred while managing the agent thread.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`AgentThreadException()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

Any Class

Special type indicating an unconstrained type.

- Any is compatible with every type.
- Any assumed to have all methods.
- All values assumed to be instances of Any.

Note that all the above statements are true from the point of view of static type checkers. At runtime, Any should not be used with instance checks.

Constructor

Python

```
Any()
```

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__()
```

__new__

Python

```
static __new__(cls, *args, **kwargs)
```

ChatClientException Class

An error occurred while dealing with a chat client.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`ChatClientException()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ChatClientInitializationError Class

An error occurred while initializing the chat client.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ChatClientInitializationError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ContentError Class

An error occurred while processing content.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`ContentError()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

MiddlewareException Class

An error occurred during middleware execution.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
MiddlewareException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceContentFilterException Class

An error was raised by the content filter of the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceContentFilterException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceException Class

Base class for all service exceptions.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

Python
`ServiceException()`

Parameters

Expand table

Name	Description
`message` Required*	
`inner_exception`	Default value: None
`log_level`	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceInitializationError Class

An error occurred while initializing the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceInitializationError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceInvalidAuthError Class

An error occurred while authenticating the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceInvalidAuthError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceInvalidExecutionSettingsError Class

An error occurred while validating the execution settings of the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceInvalidExecutionSettingsError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceInvalidRequestError Class

An error occurred while validating the request to the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceInvalidRequestError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceInvalidResponseError Class

An error occurred while validating the response from the service.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceInvalidResponseError()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ServiceResponseException Class

Base class for all service response exceptions.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ServiceResponseException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ToolException Class

An error occurred while executing a tool.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ToolException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

ToolExecutionException Class

An error occurred while executing a tool.

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Constructor

```
Python
ToolExecutionException()
```

Parameters

Expand table

Name	Description
message Required*	
inner_exception	Default value: None
log_level	Default value: 10

Methods

Expand table

__init__	Create an AgentFrameworkException. This emits a debug log (by default), with the inner_exception if provided.
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Create an AgentFrameworkException.

This emits a debug log (by default), with the inner_exception if provided.

Python

```
__init__(message: str, inner_exception: Exception | None = None, log_level:
Literal[0] | Literal[10] | Literal[20] | Literal[30] | Literal[40] | Literal[50] |
None = 10, *args: Any, **kwargs: Any)
```

Parameters

Expand table

Name	Description
message Required*	str
inner_exception	Exception | None Default value: None
log_level	Literal[0] | typing.Literal[10] | typing.Literal[20] | typing.Literal[30] | typing.Literal[40] | typing.Literal[50] | None Default value: 10
args Required*	Any
kwargs Required*	Any

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```Python
with_traceback()
```

Attributes

args

observability Module

Enums

Expand table

OtelAttr	Enum to capture the attributes used in OpenTelemetry for Generative AI. Based on: https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/ and https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/

Functions

configure_otel_providers

Configure otel providers and enable instrumentation for the application with OpenTelemetry.

This method creates the exporters and providers for the application based on the provided values and environment variables and enables instrumentation.

Call this method once during application startup, before any telemetry is captured. DO NOT call this method multiple times, as it may lead to unexpected behavior.

The function automatically reads standard OpenTelemetry environment variables:

- OTEL_EXPORTER_OTLP_ENDPOINT: Base OTLP endpoint for all signals
- OTEL_EXPORTER_OTLP_TRACES_ENDPOINT: OTLP endpoint for traces
- OTEL_EXPORTER_OTLP_METRICS_ENDPOINT: OTLP endpoint for metrics
- OTEL_EXPORTER_OTLP_LOGS_ENDPOINT: OTLP endpoint for logs
- OTEL_EXPORTER_OTLP_PROTOCOL: Protocol (grpc/http)
- OTEL_EXPORTER_OTLP_HEADERS: Headers for all signals
- ENABLE_CONSOLE_EXPORTERS: Enable console output for telemetry

> **Note**
>
> Since you can only setup one provider per signal type (logs, traces, metrics),
>
> you can choose to use this method and take the exporter and provider that we created.
>
> Alternatively, you can setup the providers yourself, or through another library

(e.g., Azure Monitor) and just call enable_instrumentation() to enable instrumentation.

> **Note**
>
> By default, the Agent Framework emits metrics with the prefixes agent_framework and gen_ai (OpenTelemetry GenAI semantic conventions). You can use the views parameter to filter which metrics are collected and exported. You can also use the create_metric_views() helper function to get default views.

-[References]-

- https://opentelemetry.io/docs/languages/sdk-configuration/general/
- https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/

Python

```
configure_otel_providers(*, enable_sensitive_data: bool | None = None, exporters:
list['LogRecordExporter | SpanExporter | MetricExporter'] | None = None, views:
list['View'] | None = None, vs_code_extension_port: int | None = None,
env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

Expand table

Name	Description
enable_sensitive_data Required*	bool | None
exporters Required*	list[| <xref:SpanExporter> | <xref:MetricExporter'>] | None
views Required*	list[<xref:'View'>] | None
vs_code_extension_port Required*	int | None
env_file_path Required*	str | None

Name	Description
env_file_encoding Required*	str | None

Keyword-Only Parameters

Expand table

Name	Description
enable_sensitive_data	Enable OpenTelemetry sensitive events. Overrides the environment variable ENABLE_SENSITIVE_DATA if set. Default is None. Default value: None
exporters	A list of custom exporters for logs, metrics or spans, or any combination. These will be added in addition to exporters configured via environment variables. Default is None. Default value: None
views	Optional list of OpenTelemetry views for metrics configuration. Views allow filtering and customizing which metrics are collected. Default is None (empty list). Default value: None
vs_code_extension_port	The port the AI Toolkit or Azure AI Foundry VS Code extensions are listening on. When set, additional OTEL exporters will be created with endpoint *http://localhost:{vs_code_extension_port}*. Overrides the environment variable VS_CODE_EXTENSION_PORT if set. Default is None. Default value: None
env_file_path	An optional path to a .env file to load environment variables from. Default is None. Default value: None
env_file_encoding	The encoding to use when loading the .env file. Default is None which uses the system default encoding. Default value: None

Returns

Expand table

Type	Description
None	

Examples

Python

```python
from agent_framework.observability import configure_otel_providers

# Using environment variables (recommended)
# Set ENABLE_INSTRUMENTATION=true
# Set OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
configure_otel_providers()

# Enable console output for debugging
# Set ENABLE_CONSOLE_EXPORTERS=true
configure_otel_providers()

# With custom exporters
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.exporter.otlp.proto.grpc._log_exporter import
OTLPLogExporter

configure_otel_providers(
    exporters=[
        OTLPSpanExporter(endpoint="http://custom:4317"),
        OTLPLogExporter(endpoint="http://custom:4317"),
    ],
)

# VS Code extension integration
configure_otel_providers(
    vs_code_extension_port=4317,  # Connects to AI Toolkit
)

# Enable sensitive data logging (development only)
configure_otel_providers(
    enable_sensitive_data=True,
)

# With custom metrics views
from opentelemetry.sdk.metrics.view import View

configure_otel_providers(
    views=[
        View(instrument_name="agent_framework*"),
        View(instrument_name="gen_ai*"),
    ],
)
```

This example shows how to first setup your providers, and then ensure Agent Framework emits traces, logs and metrics

Python

```
# when azure monitor is installed
from agent_framework.observability import enable_instrumentation
from azure.monitor.opentelemetry import configure_azure_monitor

connection_string = "InstrumentationKey=your_instrumentation_key_here;..."
configure_azure_monitor(connection_string=connection_string)
enable_instrumentation()
```

create_metric_views

Create the default OpenTelemetry metric views for Agent Framework.

Python

```
create_metric_views() -> list['View']
```

Returns

Expand table

Type	Description
list[‘<xref:View>’]	

create_resource

Create an OpenTelemetry Resource from environment variables and parameters.

This function reads standard OpenTelemetry environment variables to configure the resource, which identifies your service in telemetry backends.

The following environment variables are read:

- OTEL_SERVICE_NAME: The name of the service (defaults to "agent_framework")
- OTEL_SERVICE_VERSION: The version of the service (defaults to package version)
- OTEL_RESOURCE_ATTRIBUTES: Additional resource attributes as key=value pairs

Python

```
create_resource(service_name: str | None = None, service_version: str | None =
None, env_file_path: str | None = None, env_file_encoding: str | None = None,
**attributes: Any) -> Resource
```

Parameters

Expand table

Name	Description
service_name	str | None Override the service name. If not provided, reads from OTEL_SERVICE_NAME environment variable or defaults to "agent_framework". Default value: None
service_version	str | None Override the service version. If not provided, reads from OTEL_SERVICE_VERSION environment variable or defaults to the package version. Default value: None
env_file_path	str | None Path to a .env file to load environment variables from. Default is None, which loads from '.env' if present. Default value: None
env_file_encoding	str | None Encoding to use when reading the .env file. Default is None, which uses the system default encoding. Default value: None
****attributes** Required*	Any Additional resource attributes to include. These will be merged with attributes from OTEL_RESOURCE_ATTRIBUTES environment variable.

Returns

Expand table

Type	Description
<xref:opentelemetry.sdk.resources.Resource>	A configured OpenTelemetry Resource instance.

Examples

Python

```
from agent_framework.observability import create_resource

# Use defaults from environment variables
resource = create_resource()

# Override service name
```

```
    resource = create_resource(service_name="my_service")

    # Add custom attributes
    resource = create_resource(
        service_name="my_service", service_version="1.0.0",
deployment_environment="production"
    )

    # Load from custom .env file
    resource = create_resource(env_file_path="config/.env")
```

enable_instrumentation

Enable instrumentation for your application.

Calling this method implies you want to enable observability in your application.

This method does not configure exporters or providers. It only updates the global variables that trigger the instrumentation code. If you have already set the environment variable ENABLE_INSTRUMENTATION=true, calling this method has no effect, unless you want to enable or disable sensitive data events.

Python

```
enable_instrumentation(*, enable_sensitive_data: bool | None = None) -> None
```

Parameters

Expand table

Name	Description
enable_sensitive_data Required*	bool | None

Keyword-Only Parameters

Expand table

Name	Description
enable_sensitive_data	Enable OpenTelemetry sensitive events. Overrides the environment variable ENABLE_SENSITIVE_DATA if set. Default is None. Default value: None

Returns

Expand table

Type	Description
None	

get_meter

Returns a Meter for Agent Framework.

This is a convenience wrapper for metrics.get_meter() replicating the behavior of opentelemetry.metrics.get_meter().

Python

```
get_meter(name: str = 'agent_framework', version: str = version_info, schema_url: str | None = None, attributes: dict[str, Any] | None = None) -> Meter
```

Parameters

Expand table

Name	Description
name	str The name of the instrumenting library. Default is "agent_framework". Default value: "agent_framework"
version	str The version of agent_framework. Default is the current version of the package. Default value: version_info
schema_url	str | None Optional schema URL of the emitted telemetry. Default value: None
attributes	dict[str, Any] | None Optional attributes associated with the emitted telemetry. Default value: None

Returns

Expand table

Type	Description
<xref:opentelemetry.metrics.Meter>	A Meter instance for recording metrics.

Examples

Python

```python
from agent_framework import get_meter

# Get default meter
meter = get_meter()

# Create a counter metric
request_counter = meter.create_counter(
    name="requests",
    description="Number of requests",
    unit="1",
)
request_counter.add(1, {"endpoint": "/api/chat"})

# Create a histogram metric
duration_histogram = meter.create_histogram(
    name="request_duration",
    description="Request duration in seconds",
    unit="s",
)
duration_histogram.record(0.125, {"status": "success"})
```

get_tracer

Returns a Tracer for use by the given instrumentation library.

This function is a convenience wrapper for trace.get_tracer() replicating the behavior of opentelemetry.trace.TracerProvider.get_tracer. If tracer_provider is omitted the current configured one is used.

Python

```python
get_tracer(instrumenting_module_name: str = 'agent_framework',
instrumenting_library_version: str = version_info, schema_url: str | None = None,
attributes: dict[str, Any] | None = None) -> Tracer
```

Parameters

Expand table

Name	Description
instrumenting_module_name	str The name of the instrumenting library. Default is "agent_framework". Default value: "agent_framework"
instrumenting_library_version	str The version of the instrumenting library. Default is the current agent_framework version. Default value: version_info
schema_url	str | None Optional schema URL for the emitted telemetry. Default value: None
attributes	dict[str, Any] | None Optional attributes associated with the emitted telemetry. Default value: None

Returns

Expand table

Type	Description
<xref:opentelemetry.trace.Tracer>	A Tracer instance for creating spans.

Examples

Python

```python
from agent_framework import get_tracer

# Get default tracer
tracer = get_tracer()

# Use tracer to create spans
with tracer.start_as_current_span("my_operation") as span:
    span.set_attribute("custom.attribute", "value")
    # Your operation here
    pass

# Get tracer with custom module name
custom_tracer = get_tracer(
    instrumenting_module_name="my_custom_module",
    instrumenting_library_version="1.0.0",
)
```

use_agent_instrumentation

Class decorator that enables OpenTelemetry observability for an agent.

This decorator automatically traces agent run requests, captures events, and logs interactions for the decorated agent class.

> **Note**
>
> This decorator must be applied to the agent class itself, not an instance.
>
> The agent class should have a class variable AGENT_PROVIDER_NAME to set the
>
> proper system name for telemetry.

Python

```
use_agent_instrumentation(agent: type[TAgent] | None = None, *, capture_usage:
bool = True) -> type[TAgent] | Callable[[type[TAgent]], type[TAgent]]
```

Parameters

Expand table

Name	Description
agent	type[<xref:TAgent>] | None The agent class to enable observability for. Default value: None
capture_usage Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
capture_usage	Whether to capture token usage as a span attribute. Defaults to True, set to False when the agent has underlying traces that already capture token usage to avoid double counting. Default value: True

Returns

Expand table

Type	Description
type[~<xref:TAgent>] | Callable[[type[~<xref:TAgent>]], type[~<xref:TAgent>]]	The decorated agent class with observability enabled.

Exceptions

Expand table

Type	Description
AgentInitializationError	If the agent does not have required methods (run, run_stream).

Examples

Python

```
from agent_framework import use_agent_instrumentation, configure_otel_providers
from agent_framework._agents import AgentProtocol


# Decorate a custom agent class
@use_agent_instrumentation
class MyCustomAgent:
    AGENT_PROVIDER_NAME = "my_agent_system"

    async def run(self, messages=None, *, thread=None, **kwargs):
        # Your implementation
        pass

    async def run_stream(self, messages=None, *, thread=None, **kwargs):
        # Your implementation
        pass


# Setup observability
configure_otel_providers(otlp_endpoint="http://localhost:4317")

# Now all agent runs will be traced
agent = MyCustomAgent()
response = await agent.run("Perform a task")
```

use_instrumentation

Class decorator that enables OpenTelemetry observability for a chat client.

This decorator automatically traces chat completion requests, captures metrics, and logs events for the decorated chat client class.

> **Note**
>
> This decorator must be applied to the class itself, not an instance.
>
> The chat client class should have a class variable OTEL_PROVIDER_NAME to
>
> set the proper provider name for telemetry.

Python

```
use_instrumentation(chat_client: type[TChatClient]) -> type[TChatClient]
```

Parameters

Expand table

Name	Description
chat_client Required*	type[<xref:TChatClient>] The chat client class to enable observability for.

Returns

Expand table

Type	Description
type[~<xref:TChatClient>]	The decorated chat client class with observability enabled.

Exceptions

Expand table

Type	Description
ChatClientInitializationError	If the chat client does not have required methods (get_response, get_streaming_response).

Examples

Python

```python
from agent_framework import use_instrumentation, configure_otel_providers
from agent_framework import ChatClientProtocol


# Decorate a custom chat client class
@use_instrumentation
class MyCustomChatClient:
    OTEL_PROVIDER_NAME = "my_provider"

    async def get_response(self, messages, **kwargs):
        # Your implementation
        pass

    async def get_streaming_response(self, messages, **kwargs):
        # Your implementation
        pass


# Setup observability
configure_otel_providers(otlp_endpoint="http://localhost:4317")

# Now all calls will be traced
client = MyCustomChatClient()
response = await client.get_response("Hello")
```

OtelAttr Enum

Enum to capture the attributes used in OpenTelemetry for Generative AI.

Based on: https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-spans/ and https://opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans/

Fields

Expand table

ADDRESS
AGENT_CREATE_OPERATION
AGENT_DESCRIPTION
AGENT_FRAMEWORK_GEN_AI_SYSTEM
AGENT_ID
AGENT_INVOKE_OPERATION
AGENT_NAME
ASSISTANT_MESSAGE
BUILD_COMPLETED
BUILD_ERROR
BUILD_ERROR_MESSAGE
BUILD_ERROR_TYPE
BUILD_STARTED
BUILD_VALIDATION_COMPLETED
CHAT_COMPLETION_OPERATION
CHOICE
CHOICE_COUNT
CONVERSATION_ID
DATA_SOURCE_ID

DURATION_UNIT
EDGE_GROUP_DELIVERED
EDGE_GROUP_DELIVERY_STATUS
EDGE_GROUP_ID
EDGE_GROUP_PROCESS_SPAN
EDGE_GROUP_TYPE
ENCODING_FORMATS
ERROR_TYPE
EVENT_NAME
EXECUTOR_ID
EXECUTOR_PROCESS_SPAN
EXECUTOR_TYPE
FINISH_REASONS
FREQUENCY_PENALTY
INPUT_MESSAGES
INPUT_TOKENS
MEASUREMENT_FUNCTION_INVOCATION_DURATION
MEASUREMENT_FUNCTION_TAG_NAME
MESSAGE_DESTINATION_EXECUTOR_ID
MESSAGE_PAYLOAD_TYPE
MESSAGE_SEND_SPAN
MESSAGE_SOURCE_ID
MESSAGE_TARGET_ID
MESSAGE_TYPE
OPERATION
OUTPUT_MESSAGES
OUTPUT_TOKENS

OUTPUT_TYPE
PORT
PRESENCE_PENALTY
PROVIDER_NAME
RESPONSE_ID
SEED
SPAN_ID
STOP_SEQUENCES
SYSTEM_INSTRUCTIONS
SYSTEM_MESSAGE
TOOL_ARGUMENTS
TOOL_CALL_ID
TOOL_DEFINITIONS
TOOL_DESCRIPTION
TOOL_EXECUTION_OPERATION
TOOL_MESSAGE
TOOL_NAME
TOOL_RESULT
TOOL_TYPE
TOP_K
TRACE_ID
T_TYPE
T_TYPE_INPUT
T_TYPE_OUTPUT
T_UNIT
USER_MESSAGE
WORKFLOW_BUILD_SPAN

WORKFLOW_COMPLETED
WORKFLOW_DEFINITION
WORKFLOW_DESCRIPTION
WORKFLOW_ERROR
WORKFLOW_ID
WORKFLOW_NAME
WORKFLOW_RUN_SPAN
WORKFLOW_STARTED

AIFunction Class

A tool that wraps a Python function to make it callable by AI models.

This class wraps a Python function to make it callable by AI models with automatic parameter validation and JSON schema generation.

Initialize the AIFunction.

Constructor

Python

```
AIFunction()
```

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the function.
description	A description of the function.
approval_mode	Whether or not approval is required to run this tool. Default is that approval is not needed. Default value: None
max_invocations	The maximum number of times this function can be invoked. If None, there is no limit. Should be at least 1. Default value: None
max_invocation_exceptions	The maximum number of exceptions allowed during invocations. If None, there is no limit. Should be at least 1. Default value: None
additional_properties	Additional properties to set on the function. Default value: None
func	The function to wrap. Default value: None
input_model	The Pydantic model that defines the input parameters for the function. This can also be a JSON schema dictionary. If not provided, it will be inferred from the function signature.

Name	Description
	Default value: None
**kwargs	Additional keyword arguments.

Examples

Python

```
from typing import Annotated
from pydantic import BaseModel, Field
from agent_framework import AIFunction, ai_function


# Using the decorator with string annotations
@ai_function
def get_weather(
    location: Annotated[str, "The city name"],
    unit: Annotated[str, "Temperature unit"] = "celsius",
) -> str:
    '''Get the weather for a location.'''
    return f"Weather in {location}: 22°{unit[0].upper()}"


# Using direct instantiation with Field
class WeatherArgs(BaseModel):
    location: Annotated[str, Field(description="The city name")]
    unit: Annotated[str, Field(description="Temperature unit")] = "celsius"


weather_func = AIFunction(
    name="get_weather",
    description="Get the weather for a location",
    func=lambda location, unit="celsius": f"Weather in {location}: 22°
{unit[0].upper()}",
    approval_mode="never_require",
    input_model=WeatherArgs,
)

# Invoke the function
result = await weather_func.invoke(arguments=WeatherArgs(location="Seattle"))
```

Methods

Expand table

__init__	Initialize the AIFunction.

__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
invoke	Run the AI function with the provided arguments as a Pydantic model.
parameters	Create the JSON schema of the parameters.
to_dict	
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.
to_json_schema_spec	Convert a AIFunction to the JSON Schema function specification format.

__init__

Initialize the AIFunction.

Python

```
__init__(*, name: str, description: str = '', approval_mode:
Literal['always_require', 'never_require'] | None = None, max_invocations: int |
None = None, max_invocation_exceptions: int | None = None, additional_properties:
dict[str, Any] | None = None, func: Callable[[...], Awaitable[ReturnT] | ReturnT]
| None = None, input_model: type[ArgsT] | Mapping[str, Any] | None = None,
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
name Required*	str
description Required*	str
approval_mode Required*	Literal['always_require', 'never_require'] | None
max_invocations Required*	int | None
max_invocation_exceptions Required*	int | None
additional_properties Required*	dict[str, Any] | None
func Required*	Callable[[...], Awaitable[<xref:agent_framework._tools.ReturnT>] | <xref:agent_framework._tools.ReturnT>] | None
input_model Required*	type[<xref:ArgsT>] | Mapping[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the function.
description	A description of the function. Default value: ""
approval_mode	Whether or not approval is required to run this tool. Default is that approval is not needed. Default value: None
max_invocations	The maximum number of times this function can be invoked. If None, there is no limit. Should be at least 1. Default value: None
max_invocation_exceptions	The maximum number of exceptions allowed during invocations. If None, there is no limit. Should be at least 1.

Name	Description
	Default value: None
additional_properties	Additional properties to set on the function. Default value: None
func	The function to wrap. Default value: None
input_model	The Pydantic model that defines the input parameters for the function. This can also be a JSON schema dictionary. If not provided, it will be inferred from the function signature. Default value: None
****kwargs**	Additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"
```

```python
# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

invoke

Run the AI function with the provided arguments as a Pydantic model.

Python

```
async invoke(*, arguments: ArgsT | None = None, **kwargs: Any) -> ReturnT
```

Parameters

Expand table

Name	Description
arguments Required*	<xref:agent_framework._tools.ArgsT> | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
arguments	A Pydantic model instance containing the arguments for the function. Default value: None

Name	Description
kwargs	Keyword arguments to pass to the function, will not be used if `arguments` is provided.

Returns

Expand table

Type	Description
<xref:agent_framework._tools.ReturnT>	The result of the function execution.

Exceptions

Expand table

Type	Description
TypeError	If arguments is not an instance of the expected input model.

parameters

Create the JSON schema of the parameters.

Python

```
parameters() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary containing the JSON schema for the function's parameters. The result is cached after the first call for performance.

to_dict

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Default value: None
exclude_none	Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

to_json_schema_spec

Convert a AIFunction to the JSON Schema function specification format.

Python

```
to_json_schema_spec() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary containing the function specification in JSON Schema format.

Attributes

declaration_only

Indicate whether the function is declaration only (i.e., has no implementation).

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'_cached_parameters',
'_invocation_duration_histogram', 'input_model'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'func'}
```

AgentExecutor Class

built-in executor that wraps an agent for handling messages.

AgentExecutor adapts its behavior based on the workflow execution mode:

- run_stream(): Emits incremental AgentRunUpdateEvent events as the agent produces tokens
- run(): Emits a single AgentRunEvent containing the complete response

The executor automatically detects the mode via WorkflowContext.is_streaming().

Initialize the executor with a unique identifier.

Constructor

Python

```
AgentExecutor()
```

Parameters

Expand table

Name	Description
agent Required*	The agent to be wrapped by this executor.
agent_thread Required*	The thread to use for running the agent. If None, a new thread will be created.
output_response Required*	Whether to yield an AgentRunResponse as a workflow output when the agent completes.
id Required*	A unique identifier for the executor. If None, the agent's name will be used if available.

Keyword-Only Parameters

Expand table

Name	Description
agent_thread	Default value: None
output_response	Default value: False
id	Default value: None

Methods

Expand table

__init__	Initialize the executor with a unique identifier.
__new__	
can_handle	Check if the executor can handle a given message type.
clone	
execute	Execute the executor with a given message and context parameters. • Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using @handler decorator.
from_dict	
from_json	
from_message	Accept a single ChatMessage as input.
from_messages	Accept a list of chat inputs (strings or ChatMessage) as conversation context.
from_response	Enable seamless chaining: accept a prior AgentExecutorResponse as input. Strategy: treat the prior response's messages as the conversation state and immediately run the agent to produce a new response.
from_str	Accept a raw user prompt string and run the agent (one-shot).
handle_user_input_response	Handle user input responses for function approvals during agent execution. This will hold the executor's execution until all pending user input requests are resolved.
is_request_supported	Check if the executor supports request of the given type and handling a response of the given type.
on_checkpoint_restore	Restore executor state from checkpoint.

on_checkpoint_save	Capture current executor state for checkpointing. NOTE: if the thread storage is on the server side, the full thread state may not be serialized locally. Therefore, we are relying on the server-side to ensure the thread state is preserved and immutable across checkpoints. This is not the case for AzureAI Agents, but works for the Responses API.
reset	Reset the internal cache of the executor.
run	Handle an AgentExecutorRequest (canonical input). This is the standard path: extend cache with provided messages; if should_respond run the agent and emit an AgentExecutorResponse downstream.
to_dict	Serialize executor definition for workflow topology export.
to_json	

__init__

Initialize the executor with a unique identifier.

Python

```
__init__(agent: AgentProtocol, *, agent_thread: AgentThread | None = None,
output_response: bool = False, id: str | None = None)
```

Parameters

Expand table

Name	Description
agent Required*	AgentProtocol The agent to be wrapped by this executor.
agent_thread Required*	AgentThread | None The thread to use for running the agent. If None, a new thread will be created.
output_response Required*	bool Whether to yield an AgentRunResponse as a workflow output when the agent completes.
id Required*	str | None A unique identifier for the executor. If None, the agent's name will be used if available.

Keyword-Only Parameters

Expand table

Name	Description
`agent_thread`	Default value: None
`output_response`	Default value: False
`id`	Default value: None

__new__

```
Python
__new__(**kwargs)
```

can_handle

Check if the executor can handle a given message type.

```
Python
can_handle(message: Message) -> bool
```

Parameters

Expand table

Name	Description
`message` Required*	Message The message to check.

Returns

Expand table

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

Expand table

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] | None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] | None Optional source span IDs from multiple sources for linking. Default value: None

Returns

Expand table

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_json

Python
`from_json(raw: str) -> TModel`

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_message

Accept a single ChatMessage as input.

Python
`async from_message(message: ChatMessage, ctx: WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None`

Parameters

Expand table

Name	Description
message Required*	ChatMessage
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

Expand table

Type	Description
None	

from_messages

Accept a list of chat inputs (strings or ChatMessage) as conversation context.

Python

```
async from_messages(messages: list[str | ChatMessage], ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

Expand table

Name	Description
messages Required*	list[str | ChatMessage]
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

Expand table

Type	Description
None	

from_response

Enable seamless chaining: accept a prior AgentExecutorResponse as input.

Strategy: treat the prior response's messages as the conversation state and immediately run the agent to produce a new response.

Python

```
async from_response(prior: AgentExecutorResponse, ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

Expand table

Name	Description
prior Required*	AgentExecutorResponse
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

Expand table

Type	Description
None	

from_str

Accept a raw user prompt string and run the agent (one-shot).

Python

```
async from_str(text: str, ctx: WorkflowContext[AgentExecutorResponse,
AgentRunResponse]) -> None
```

Parameters

Expand table

Name	Description
text Required*	str
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

Expand table

Type	Description
None	

handle_user_input_response

Handle user input responses for function approvals during agent execution.

This will hold the executor's execution until all pending user input requests are resolved.

Python

```
async handle_user_input_response(original_request: FunctionApprovalRequestContent,
response: FunctionApprovalResponseContent, ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

Expand table

Name	Description
original_request Required*	FunctionApprovalRequestContent The original function approval request sent by the agent.
response Required*	FunctionApprovalResponseContent The user's response to the function approval request.
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse] The workflow context for emitting events and outputs.

Returns

Expand table

Type	Description
None	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

Expand table

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

Expand table

Type	Description
bool	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Restore executor state from checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any] Checkpoint data dict

Returns

Expand table

Type	Description
None	

on_checkpoint_save

Capture current executor state for checkpointing.

NOTE: if the thread storage is on the server side, the full thread state may not be serialized locally. Therefore, we are relying on the server-side to ensure the thread state is preserved and immutable across checkpoints. This is not the case for AzureAI Agents, but works for the Responses API.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	Dict containing serialized cache and thread state

reset

Reset the internal cache of the executor.

Python

```
reset() -> None
```

Returns

Expand table

Type	Description
None	

run

Handle an AgentExecutorRequest (canonical input).

This is the standard path: extend cache with provided messages; if should_respond run the agent and emit an AgentExecutorResponse downstream.

Python

```
async run(request: AgentExecutorRequest, ctx:
WorkflowContext[AgentExecutorResponse, AgentRunResponse]) -> None
```

Parameters

Expand table

Name	Description
request Required*	AgentExecutorRequest
ctx Required*	WorkflowContext[AgentExecutorResponse, AgentRunResponse]

Returns

Expand table

Type	Description
None	

to_dict

Serialize executor definition for workflow topology export.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

description

Get the description of the underlying agent.

input_types

Get the list of input types that this executor can handle.

Returns

Expand table

Type	Description
	A list of the message types that this executor's handlers can process.

output_response

Whether this executor yields AgentRunResponse as workflow output when complete.

output_types

Get the list of output types that this executor can produce via send_message().

Returns

Expand table

Type	Description
	A list of the output types inferred from the handlers' WorkflowContext[T] annotations.

workflow_output_types

AgentExecutorRequest Class

A request to an agent executor.

Constructor

Python

```
AgentExecutorRequest()
```

Parameters

Expand table

Name	Description
messages Required*	
should_respond	Default value: True

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(messages: list[ChatMessage], should_respond: bool = True) -> None
```

Parameters

Expand table

Name	Description
messages Required*	list[ChatMessage]
should_respond	bool Default value: True

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

Attributes

messages

A list of chat messages to be processed by the agent.

```
Python
messages: list[ChatMessage]
```

should_respond

A flag indicating whether the agent should respond to the messages. If False, the messages will be saved to the executor's cache but not sent to the agent.

```
Python
should_respond: bool = True
```

AgentExecutorResponse Class

A response from an agent executor.

Constructor

Python

```
AgentExecutorResponse()
```

Parameters

Expand table

Name	Description
executor_id Required*	
agent_run_response Required*	
full_conversation	Default value: None

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(executor_id: str, agent_run_response: AgentRunResponse,
full_conversation: list[ChatMessage] | None = None) -> None
```

Parameters

Expand table

Name	Description
executor_id Required*	str
agent_run_response Required*	AgentRunResponse
full_conversation	list[ChatMessage] | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

Attributes

executor_id

The ID of the executor that generated the response.

```
Python
executor_id: str
```

agent_run_response

The underlying agent run response (unaltered from client).

```
Python
agent_run_response: AgentRunResponse
```

full_conversation

The full conversation context (prior inputs + all assistant/tool outputs) that should be used when chaining to another AgentExecutor. This prevents downstream agents losing user prompts while keeping the emitted AgentRunEvent text faithful to the raw agent output.

Python

```
full_conversation: list[ChatMessage] | None = None
```

AgentInputRequest Class

Request for human input before an agent runs in high-level builder workflows.

Emitted via RequestInfoEvent when a workflow pauses before an agent executes. The response is injected into the conversation as a user message to steer the agent's behavior.

This is the standard request type used by *.with_request_info()* on SequentialBuilder, ConcurrentBuilder, GroupChatBuilder, and HandoffBuilder.

Constructor

Python

```
AgentInputRequest()
```

Parameters

Expand table

Name	Description
target_agent_id Required*	
conversation	Default value: <factory>
instruction	Default value: None
metadata	Default value: <factory>

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(target_agent_id: str | None, conversation:
list[agent_framework._types.ChatMessage] = <factory>, instruction: str | None =
None, metadata: dict[str, typing.Any] = <factory>) -> None
```

Parameters

Expand table

Name	Description
target_agent_id Required*	str | None
conversation	list[ChatMessage] Default value: <factory>
instruction	str | None Default value: None
metadata	dict[str, Any] Default value: <factory>

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

Attributes

target_agent_id

ID of the agent that is about to run

Python

```
target_agent_id: str | None
```

conversation

Current conversation history the agent will receive

Python

```
conversation: list[ChatMessage]
```

instruction

Optional instruction from the orchestrator (e.g., manager in GroupChat)

Python

```
instruction: str | None = None
```

metadata

Builder-specific context (stores internal state for resume)

Python

```
metadata: dict[str, Any]
```

AgentMiddleware Class

Abstract base class for agent middleware that can intercept agent invocations.

Agent middleware allows you to intercept and modify agent invocations before and after execution. You can inspect messages, modify context, override results, or terminate execution early.

> ⓘ Note
>
> AgentMiddleware is an abstract base class. You must subclass it and implement
>
> the process() method to create custom agent middleware.

Constructor

Python

```
AgentMiddleware()
```

Examples

Python

```
from agent_framework import AgentMiddleware, AgentRunContext, ChatAgent


class RetryMiddleware(AgentMiddleware):
    def __init__(self, max_retries: int = 3):
        self.max_retries = max_retries

    async def process(self, context: AgentRunContext, next):
        for attempt in range(self.max_retries):
            await next(context)
            if context.result and not context.result.is_error:
                break
            print(f"Retry {attempt + 1}/{self.max_retries}")


# Use with an agent
agent = ChatAgent(chat_client=client, name="assistant",
middleware=RetryMiddleware())
```

Methods

Expand table

__init__	
__new__	
process	Process an agent invocation. **Note** Middleware should not return anything. All data manipulation should happen within the context object. Set context.result to override execution, or observe context.result after calling next() for actual results.

__init__

Python

```
__init__()
```

__new__

Python

```
__new__(**kwargs)
```

process

Process an agent invocation.

Note

Middleware should not return anything. All data manipulation should happen within the context object. Set context.result to override execution, or observe context.result after calling next() for actual results.

Python

```
abstract async process(context: AgentRunContext, next: Callable[[AgentRunContext], Awaitable[None]]) -> None
```

Parameters

Expand table

Name	Description
context Required*	AgentRunContext Agent invocation context containing agent, messages, and metadata. Use context.is_streaming to determine if this is a streaming call. Middleware can set context.result to override execution, or observe the actual execution result after calling next(). For non-streaming: AgentRunResponse For streaming: AsyncIterable[AgentRunResponseUpdate]
next Required*	Callable[[AgentRunContext], Awaitable[None]] Function to call the next middleware or final agent execution. Does not return anything - all data flows through the context.

Returns

Expand table

Type	Description
None	

AgentProtocol Class

A protocol for an agent that can be invoked.

This protocol defines the interface that all agents must implement, including properties for identification and methods for execution.

> ⓘ Note
>
> Protocols use structural subtyping (duck typing). Classes don't need
>
> to explicitly inherit from this protocol to be considered compatible.
>
> This allows you to create completely custom agents without using
>
> any Agent Framework base classes.

Constructor

Python

```
AgentProtocol()
```

Examples

Python

```
from agent_framework import AgentProtocol


# Any class implementing the required methods is compatible
# No need to inherit from AgentProtocol or use any framework classes
class CustomAgent:
    def __init__(self):
        self._id = "custom-agent-001"
        self._name = "Custom Agent"

    @property
    def id(self) -> str:
        return self._id

    @property
    def name(self) -> str | None:
```

```
            return self._name

        @property
        def display_name(self) -> str:
            return self.name or self.id

        @property
        def description(self) -> str | None:
            return "A fully custom agent implementation"

        async def run(self, messages=None, *, thread=None, **kwargs):
            # Your custom implementation
            from agent_framework import AgentRunResponse

            return AgentRunResponse(messages=[], response_id="custom-response")

        def run_stream(self, messages=None, *, thread=None, **kwargs):
            # Your custom streaming implementation
            async def _stream():
                from agent_framework import AgentRunResponseUpdate

                yield AgentRunResponseUpdate()

            return _stream()

        def get_new_thread(self, **kwargs):
            # Return your own thread implementation
            return {"id": "custom-thread", "messages": []}


    # Verify the instance satisfies the protocol
    instance = CustomAgent()
    assert isinstance(instance, AgentProtocol)
```

Methods

Expand table

__init__	
__new__	
get_new_thread	Creates a new conversation thread for the agent.
run	Get a response from the agent. This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available. Note: For streaming responses, use the run_stream method, which returns intermediate steps and the final result as a stream of AgentRunResponseUpdate objects. Streaming

	only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.
run_stream	Run the agent as a stream. This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller. Note: An AgentRunResponseUpdate object contains a chunk of a message.

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

get_new_thread

Creates a new conversation thread for the agent.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Returns

Expand table

Type	Description
AgentThread	

run

Get a response from the agent.

This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.

Note: For streaming responses, use the run_stream method, which returns intermediate steps and the final result as a stream of AgentRunResponseUpdate objects. Streaming only the final result is not feasible because the timing of the final result's availability is unknown, and blocking the caller until then is undesirable in streaming scenarios.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None

Name	Description
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentRunResponse	An agent response item.

run_stream

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

Note: An AgentRunResponseUpdate object contains a chunk of a message.

Python

```
run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AsyncIterable[AgentRunResponseUpdate]
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AsyncIterable[AgentRunResponseUpdate]	

Attributes

description

Returns the description of the agent.

display_name

Returns the display name of the agent.

id

Returns the ID of the agent.

name

Returns the name of the agent.

AgentRunContext Class

Context object for agent middleware invocations.

This context is passed through the agent middleware pipeline and contains all information about the agent invocation.

Initialize the AgentRunContext.

Constructor

```
Python
AgentRunContext()
```

Parameters

Expand table

Name	Description
agent Required*	The agent being invoked.
messages Required*	The messages being sent to the agent.
thread	The agent thread for this invocation, if any. Default value: None
is_streaming	Whether this is a streaming invocation. Default value: False
metadata	Metadata dictionary for sharing data between agent middleware. Default value: None
result	Agent execution result. Default value: None
terminate	A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	Additional keyword arguments passed to the agent run method. Default value: None

Examples

Python

```python
from agent_framework import AgentMiddleware, AgentRunContext


class LoggingMiddleware(AgentMiddleware):
    async def process(self, context: AgentRunContext, next):
        print(f"Agent: {context.agent.name}")
        print(f"Messages: {len(context.messages)}")
        print(f"Thread: {context.thread}")
        print(f"Streaming: {context.is_streaming}")

        # Store metadata
        context.metadata["start_time"] = time.time()

        # Continue execution
        await next(context)

        # Access result after execution
        print(f"Result: {context.result}")
```

Methods

Expand table

__init__	Initialize the AgentRunContext.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

	Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the AgentRunContext.

Python

```
__init__(agent: AgentProtocol, messages: list[ChatMessage], thread: AgentThread | None = None, is_streaming: bool = False, metadata: dict[str, Any] | None = None, result: AgentRunResponse | AsyncIterable[AgentRunResponseUpdate] | None = None, terminate: bool = False, kwargs: dict[str, Any] | None = None) -> None
```

Parameters

Expand table

Name	Description
agent Required*	AgentProtocol The agent being invoked.
messages Required*	list[ChatMessage] The messages being sent to the agent.
thread	AgentThread | None The agent thread for this invocation, if any. Default value: None
is_streaming	bool Whether this is a streaming invocation. Default value: False
metadata	dict[str, Any] | None Metadata dictionary for sharing data between agent middleware. Default value: None
result	AgentRunResponse | AsyncIterable[AgentRunResponseUpdate] | None Agent execution result. Default value: None

Name	Description
terminate	bool A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	dict[str, Any] | None Additional keyword arguments passed to the agent run method. Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value	

Name	Description
Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None

Name	Description
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

agent

The agent being invoked.

messages

The messages being sent to the agent.

thread

The agent thread for this invocation, if any.

is_streaming

Whether this is a streaming invocation.

metadata

Metadata dictionary for sharing data between agent middleware.

result

Agent execution result. Can be observed after calling `next()` to see the actual execution result or can be set to override the execution result. For non-streaming: should be AgentRunResponse. For streaming: should be AsyncIterable[AgentRunResponseUpdate].

terminate

A flag indicating whether to terminate execution after current middleware. When set to True, execution will stop as soon as control returns to framework.

kwargs

Additional keyword arguments passed to the agent run method.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'agent', 'result', 'thread'}
```

AgentRunEvent Class

Event triggered when an agent run is completed.

Initialize the agent run event.

Constructor

Python

```
AgentRunEvent()
```

Parameters

Expand table

Name	Description
executor_id Required*	
data	Default value: None

Methods

Expand table

__init__	Initialize the agent run event.
__new__	

__init__

Initialize the agent run event.

Python

```
__init__(executor_id: str, data: AgentRunResponse | None = None)
```

Parameters

Expand table

Name	Description
executor_id Required*	str
data	AgentRunResponse | None Default value: None

__new__

```
Python
__new__(**kwargs)
```

Attributes

data

```
Python
data: AgentRunResponse | None
```

AgentRunResponse Class

Represents the response to an Agent run request.

Provides one or more response messages and metadata about the response. A typical response will contain a single message, but may contain multiple messages in scenarios involving function calls, RAG retrievals, or complex logic.

Initialize an AgentRunResponse.

Constructor

Python

```
AgentRunResponse()
```

Keyword-Only Parameters

Expand table

Name	Description
messages	The list of chat messages in the response. Default value: None
response_id	The ID of the chat response. Default value: None
created_at	A timestamp for the chat response. Default value: None
usage_details	The usage details for the chat response. Default value: None
value	The structured output of the agent run response, if applicable. Default value: None
additional_properties	Any additional properties associated with the chat response. Default value: None
raw_representation	The raw representation of the chat response from an underlying implementation. Default value: None
****kwargs**	Additional properties to set on the response.

Examples

Python

```python
from agent_framework import AgentRunResponse, ChatMessage

# Create agent response
msg = ChatMessage(role="assistant", text="Task completed successfully.")
response = AgentRunResponse(messages=[msg], response_id="run_123")
print(response.text)  # "Task completed successfully."

# Access user input requests
user_requests = response.user_input_requests
print(len(user_requests))  # 0

# Combine streaming updates
updates = [...]  # List of AgentRunResponseUpdate objects
response = AgentRunResponse.from_agent_run_response_updates(updates)

# Serialization - to_dict and from_dict
response_dict = response.to_dict()
# {'type': 'agent_run_response', 'messages': [...], 'response_id': 'run_123',
#  'additional_properties': {}}
restored_response = AgentRunResponse.from_dict(response_dict)
print(restored_response.response_id)  # "run_123"

# Serialization - to_json and from_json
response_json = response.to_json()
# '{"type": "agent_run_response", "messages": [...], "response_id": "run_123",
...}'
restored_from_json = AgentRunResponse.from_json(response_json)
print(restored_from_json.text)  # "Task completed successfully."
```

Methods

Expand table

__init__	Initialize an AgentRunResponse.
__new__	
from_agent_response_generator	Joins multiple updates into a single AgentRunResponse.
from_agent_run_response_updates	Joins multiple updates into a single AgentRunResponse.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It

	supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.
try_parse_value	If there is a value, does nothing, otherwise tries to parse the text into the value.

__init__

Initialize an AgentRunResponse.

Python

```
__init__(*, messages: ChatMessage | list[ChatMessage] | MutableMapping[str, Any] | list[MutableMapping[str, Any]] | None = None, response_id: str | None = None, created_at: str | None = None, usage_details: UsageDetails | MutableMapping[str, Any] | None = None, value: Any | None = None, raw_representation: Any | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
messages Required*	ChatMessage | list[ChatMessage] | MutableMapping[str, Any] | list[MutableMapping[str, Any]] | None
response_id Required*	str | None
created_at Required*	str | None
usage_details Required*	UsageDetails | MutableMapping[str, Any] | None
value Required*	Any | None
raw_representation Required*	Any | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
messages	The list of chat messages in the response. Default value: None
response_id	The ID of the chat response. Default value: None
created_at	A timestamp for the chat response. Default value: None
usage_details	The usage details for the chat response. Default value: None
value	The structured output of the agent run response, if applicable. Default value: None
additional_properties	Any additional properties associated with the chat response. Default value: None

Name	Description
raw_representation	The raw representation of the chat response from an underlying implementation. Default value: None
****kwargs**	Additional properties to set on the response.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_agent_response_generator

Joins multiple updates into a single AgentRunResponse.

Python

```
async classmethod from_agent_response_generator(updates:
AsyncIterable[AgentRunResponseUpdate], *, output_format_type: type[BaseModel] |
None = None) -> TAgentRunResponse
```

Parameters

Expand table

Name	Description
updates Required*	AsyncIterable[AgentRunResponseUpdate] An async iterable of AgentRunResponseUpdate objects to combine.
output_format_type Required*	type[<xref:pydantic.main.BaseModel>] | None

Keyword-Only Parameters

Expand table

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._types.TAgentRunResponse>	

from_agent_run_response_updates

Joins multiple updates into a single AgentRunResponse.

Python

```
from_agent_run_response_updates(updates: Sequence[AgentRunResponseUpdate], *,
output_format_type: type[BaseModel] | None = None) -> TAgentRunResponse
```

Parameters

Expand table

Name	Description
updates Required*	Sequence[AgentRunResponseUpdate] A sequence of AgentRunResponseUpdate objects to combine.
output_format_type Required*	type[<xref:pydantic.main.BaseModel>] | None

Keyword-Only Parameters

Expand table

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data. Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._types.TAgentRunResponse>	

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
```

```
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	

Name	Description
	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

try_parse_value

If there is a value, does nothing, otherwise tries to parse the text into the value.

Python

```
try_parse_value(output_format_type: type[BaseModel]) -> None
```

Parameters

Expand table

Name	Description
output_format_type Required*	type[<xref:pydantic.main.BaseModel>]

Returns

Expand table

Type	Description
None	

Attributes

text

Get the concatenated text of all messages.

user_input_requests

Get all BaseUserInputRequest messages from the response.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

AgentRunResponseUpdate Class

Represents a single streaming response chunk from an Agent.

Initialize an AgentRunResponseUpdate.

Constructor

Python

```
AgentRunResponseUpdate()
```

Keyword-Only Parameters

Expand table

Name	Description
contents	Optional list of BaseContent items or dicts to include in the update. Default value: None
text	Optional text content of the update. Default value: None
role	The role of the author of the response update (Role, string, or dict Default value: None
author_name	Optional name of the author of the response update. Default value: None
response_id	Optional ID of the response of which this update is a part. Default value: None
message_id	Optional ID of the message of which this update is a part. Default value: None
created_at	Optional timestamp for the chat response update. Default value: None
additional_properties	Optional additional properties associated with the chat response update. Default value: None
raw_representation	Optional raw representation of the chat response update. Default value: None
kwargs	will be combined with additional_properties if provided.

Examples

Python

```
from agent_framework import AgentRunResponseUpdate, TextContent

# Create an agent run update
update = AgentRunResponseUpdate(
    contents=[TextContent(text="Processing...")],
    role="assistant",
    response_id="run_123",
)
print(update.text)  # "Processing..."

# Check for user input requests
user_requests = update.user_input_requests

# Serialization - to_dict and from_dict
update_dict = update.to_dict()
# {'type': 'agent_run_response_update', 'contents': [{'type': 'text', 'text':
'Processing...'}],
#  'role': {'type': 'role', 'value': 'assistant'}, 'response_id': 'run_123'}
restored_update = AgentRunResponseUpdate.from_dict(update_dict)
print(restored_update.response_id)  # "run_123"

# Serialization - to_json and from_json
update_json = update.to_json()
# '{"type": "agent_run_response_update", "contents": [{"type": "text", "text":
"Processing..."}], ...}'
restored_from_json = AgentRunResponseUpdate.from_json(update_json)
print(restored_from_json.text)  # "Processing..."
```

Methods

Expand table

__init__	Initialize an AgentRunResponseUpdate.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string.

	This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize an AgentRunResponseUpdate.

Python

```
__init__(*, contents: Sequence[TextContent | DataContent | TextReasoningContent |
UriContent | FunctionCallContent | FunctionResultContent | ErrorContent |
UsageContent | HostedFileContent | HostedVectorStoreContent |
FunctionApprovalRequestContent | FunctionApprovalResponseContent |
MutableMapping[str, Any]] | None = None, text: TextContent | str | None = None,
role: Role | MutableMapping[str, Any] | str | None = None, author_name: str | None
= None, response_id: str | None = None, message_id: str | None = None, created_at:
str | None = None, additional_properties: MutableMapping[str, Any] | None = None,
raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
contents Required*	Sequence[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent | MutableMapping[str, Any]] | None
text	TextContent | str | None

Name	Description
Required*	
role Required*	Role | MutableMapping[str, Any] | str | None
author_name Required*	str | None
response_id Required*	str | None
message_id Required*	str | None
created_at Required*	str | None
additional_properties Required*	MutableMapping[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
contents	Optional list of BaseContent items or dicts to include in the update. Default value: None
text	Optional text content of the update. Default value: None
role	The role of the author of the response update (Role, string, or dict Default value: None
author_name	Optional name of the author of the response update. Default value: None
response_id	Optional ID of the response of which this update is a part. Default value: None
message_id	Optional ID of the message of which this update is a part. Default value: None

Name	Description
created_at	Optional timestamp for the chat response update. Default value: None
additional_properties	Optional additional properties associated with the chat response update. Default value: None
raw_representation	Optional raw representation of the chat response update. Default value: None
kwargs	will be combined with additional_properties if provided.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None

Name	Description
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

text

Get the concatenated text of all TextContent objects in contents.

user_input_requests

Get all BaseUserInputRequest messages from the response.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

AgentRunUpdateEvent Class

Event triggered when an agent is streaming messages.

Initialize the agent streaming event.

Constructor

```
Python
AgentRunUpdateEvent()
```

Parameters

Expand table

Name	Description
executor_id Required*	
data	Default value: None

Methods

Expand table

__init__	Initialize the agent streaming event.
__new__	

__init__

Initialize the agent streaming event.

```
Python
__init__(executor_id: str, data: AgentRunResponseUpdate | None = None)
```

Parameters

Expand table

Name	Description
executor_id Required*	str
data	AgentRunResponseUpdate | None Default value: None

__new__

```
Python
__new__(**kwargs)
```

Attributes

data

```
Python
data: AgentRunResponseUpdate | None
```

AgentThread Class

The Agent thread class, this can represent both a locally managed thread or a thread managed by the service.

An `AgentThread` maintains the conversation state and message history for an agent interaction. It can either use a service-managed thread (via `service_thread_id`) or a local message store (via `message_store`), but not both.

Initialize an AgentThread, do not use this method manually, always use: `agent.get_new_thread()`.

> **Note**
>
> Either service_thread_id or message_store may be set, but not both.

Constructor

Python

```
AgentThread()
```

Parameters

Expand table

Name	Description
`service_thread_id` Required*	The optional ID of the thread managed by the agent service.
`message_store` Required*	The optional ChatMessageStore implementation for managing chat messages.
`context_provider` Required*	The optional ContextProvider for the thread.

Keyword-Only Parameters

Expand table

Name	Description
service_thread_id	Default value: None
message_store	Default value: None
context_provider	Default value: None

Examples

Python

```
   from agent_framework import ChatAgent, ChatMessageStore
   from agent_framework.openai import OpenAIChatClient

   client = OpenAIChatClient(model="gpt-4o")

   # Create agent with service-managed threads using a service_thread_id
   service_agent = ChatAgent(name="assistant", client=client)
   service_thread = await
service_agent.get_new_thread(service_thread_id="thread_abc123")

   # Create agent with service-managed threads using conversation_id
   conversation_agent = ChatAgent(name="assistant", client=client,
conversation_id="thread_abc123")
   conversation_thread = await conversation_agent.get_new_thread()

   # Create agent with custom message store factory
   local_agent = ChatAgent(name="assistant", client=client,
chat_message_store_factory=ChatMessageStore)
   local_thread = await local_agent.get_new_thread()

   # Serialize and restore thread state
   state = await local_thread.serialize()
   restored_thread = await local_agent.deserialize_thread(state)
```

Methods

Expand table

__init__	Initialize an AgentThread, do not use this method manually, always use: `agent.get_new_thread()`. **Note**

	Either service_thread_id or message_store may be set, but not both.
__new__	
deserialize	Deserializes the state from a dictionary into a new AgentThread instance.
on_new_messages	Invoked when a new message has been contributed to the chat by any participant.
serialize	Serializes the current object's state.
update_from_thread_state	Deserializes the state from a dictionary into the thread properties.

__init__

Initialize an AgentThread, do not use this method manually, always use: `agent.get_new_thread()`.

> [!NOTE]
> Either service_thread_id or message_store may be set, but not both.

Python

```
__init__(*, service_thread_id: str | None = None, message_store:
ChatMessageStoreProtocol | None = None, context_provider: AggregateContextProvider
| None = None) -> None
```

Parameters

Expand table

Name	Description
service_thread_id Required*	str | None The optional ID of the thread managed by the agent service.
message_store Required*	ChatMessageStoreProtocol | None The optional ChatMessageStore implementation for managing chat messages.
context_provider Required*	AggregateContextProvider | None The optional ContextProvider for the thread.

Keyword-Only Parameters

Expand table

Name	Description
service_thread_id	Default value: None
message_store	Default value: None
context_provider	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

deserialize

Deserializes the state from a dictionary into a new AgentThread instance.

Python

```
async classmethod deserialize(serialized_thread_state: MutableMapping[str, Any],
*, message_store: ChatMessageStoreProtocol | None = None, **kwargs: Any) ->
TAgentThread
```

Parameters

Expand table

Name	Description
serialized_thread_state Required*	MutableMapping[str, Any] The serialized thread state as a dictionary.
message_store Required*	ChatMessageStoreProtocol | None

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
message_store	Optional ChatMessageStoreProtocol to use for managing messages. If not provided, a new ChatMessageStore will be created if needed. Default value: None
****kwargs**	Additional arguments for deserialization.

Returns

Expand table

Type	Description
<xref:agent_framework._threads.TAgentThread>	A new AgentThread instance with properties set from the serialized state.

on_new_messages

Invoked when a new message has been contributed to the chat by any participant.

Python

```
async on_new_messages(new_messages: ChatMessage | Sequence[ChatMessage]) -> None
```

Parameters

Expand table

Name	Description
new_messages Required*	ChatMessage | Sequence[ChatMessage] The new ChatMessage or sequence of ChatMessage objects to add to the thread.

Returns

Expand table

Type	Description
None	

serialize

Serializes the current object's state.

Python

```
async serialize(**kwargs: Any) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Arguments for serialization.

Returns

Expand table

Type	Description
dict[str, Any]	

update_from_thread_state

Deserializes the state from a dictionary into the thread properties.

Python

```
async update_from_thread_state(serialized_thread_state: MutableMapping[str, Any],
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
serialized_thread_state Required*	MutableMapping[str, Any] The serialized thread state as a dictionary.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional arguments for deserialization.

Returns

Expand table

Type	Description
None	

Attributes

is_initialized

Indicates if the thread is initialized.

This means either the `service_thread_id` or the `message_store` is set.

message_store

Gets the `ChatMessageStoreProtocol` used by this thread.

service_thread_id

Gets the ID of the current thread to support cases where the thread is owned by the agent service.

AggregateContextProvider Class

A ContextProvider that contains multiple context providers.

It delegates events to multiple context providers and aggregates responses from those events before returning. This allows you to combine multiple context providers into a single provider.

> ⓘ Note
>
> An AggregateContextProvider is created automatically when you pass a single context provider or a sequence of context providers to the agent constructor.

Initialize the AggregateContextProvider with context providers.

Constructor

Python

```
AggregateContextProvider()
```

Parameters

Expand table

Name	Description
context_providers	The context provider(s) to add. Default value: None

Examples

Python

```
from agent_framework import AggregateContextProvider, ChatAgent

# Create multiple context providers
provider1 = CustomContextProvider1()
provider2 = CustomContextProvider2()
provider3 = CustomContextProvider3()
```

```
    # Pass them to the agent - AggregateContextProvider is created automatically
    agent = ChatAgent(chat_client=client, name="assistant", context_providers=
[provider1, provider2, provider3])

    # Verify that an AggregateContextProvider was created
    assert isinstance(agent.context_providers, AggregateContextProvider)

    # Add additional providers to the agent
    provider4 = CustomContextProvider4()
    agent.context_providers.add(provider4)
```

Methods

Expand table

__init__	Initialize the AggregateContextProvider with context providers.
__new__	
add	Add a new context provider.
invoked	
invoking	
thread_created	

__init__

Initialize the AggregateContextProvider with context providers.

Python

```
__init__(context_providers: ContextProvider | Sequence[ContextProvider] | None =
None) -> None
```

Parameters

Expand table

Name	Description
context_providers	ContextProvider | Sequence[ContextProvider] | None The context provider(s) to add. Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

add

Add a new context provider.

```
Python
add(context_provider: ContextProvider) -> None
```

Parameters

Expand table

Name	Description
context_provider Required*	ContextProvider The context provider to add.

Returns

Expand table

Type	Description
None	

invoked

```
Python
async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
```

```
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
request_messages Required*	ChatMessage | Sequence[ChatMessage]
response_messages	ChatMessage | Sequence[ChatMessage] | None Default value: None
invoke_exception	Exception | None Default value: None
kwargs Required*	Any

Returns

Expand table

Type	Description
None	

invoking

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs:
Any) -> Context
```

Parameters

Expand table

Name	Description
messages Required*	ChatMessage | MutableSequence[ChatMessage]
kwargs Required*	Any

Returns

Expand table

Type	Description
Context	

thread_created

```
Python
async thread_created(thread_id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
thread_id	str | None Default value: None

Returns

Expand table

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

```
Python
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories when answering user questions:'
```

BaseAgent Class

Base class for all Agent Framework agents.

This class provides core functionality for agent implementations, including context providers, middleware support, and thread management.

> ⓘ Note
>
> BaseAgent cannot be instantiated directly as it doesn't implement the
>
> run(), run_stream(), and other methods required by AgentProtocol.
>
> Use a concrete implementation like ChatAgent or create a subclass.

Initialize a BaseAgent instance.

Constructor

Python

```
BaseAgent()
```

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier of the agent. If no id is provided, a new UUID will be generated. Default value: None
name	The name of the agent, can be None. Default value: None
description	The description of the agent. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations.

Name	Description
	Default value: None
additional_properties	Additional properties set on the agent. Default value: None
kwargs	Additional keyword arguments (merged into additional_properties).

Examples

Python

```python
from agent_framework import BaseAgent, AgentThread, AgentRunResponse


# Create a concrete subclass that implements the protocol
class SimpleAgent(BaseAgent):
    async def run(self, messages=None, *, thread=None, **kwargs):
        # Custom implementation
        return AgentRunResponse(messages=[], response_id="simple-response")

    def run_stream(self, messages=None, *, thread=None, **kwargs):
        async def _stream():
            # Custom streaming implementation
            yield AgentRunResponseUpdate()

        return _stream()


# Now instantiate the concrete subclass
agent = SimpleAgent(name="my-agent", description="A simple agent implementation")

# Create with specific ID and additional properties
agent = SimpleAgent(
    id="custom-id-123",
    name="configured-agent",
    description="An agent with custom configuration",
    additional_properties={"version": "1.0", "environment": "production"},
)

# Access agent properties
print(agent.id)  # Custom or auto-generated UUID
print(agent.display_name)  # Returns name or id
```

Methods

Expand table

__init__	Initialize a BaseAgent instance.
__new__	
as_tool	Create an AIFunction tool that wraps this agent.
deserialize_thread	Deserialize a thread from its serialized state.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_new_thread	Return a new AgentThread instance that is compatible with the agent.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize a BaseAgent instance.

Python

```
__init__(*, id: str | None = None, name: str | None = None, description: str | None = None, context_providers: ContextProvider | Sequence[ContextProvider] | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
```

```
Awaitable[None]] | Sequence[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, additional_properties: MutableMapping[str, Any] |
None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
context_providers Required*	ContextProvider | Sequence[ContextProvider] | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | Sequence[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
additional_properties Required*	MutableMapping[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier of the agent. If no id is provided, a new UUID will be generated. Default value: None
name	The name of the agent, can be None. Default value: None
description	The description of the agent. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
additional_properties	Additional properties set on the agent. Default value: None
kwargs	Additional keyword arguments (merged into additional_properties).

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

as_tool

Create an AIFunction tool that wraps this agent.

```
Python
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str
= 'task', arg_description: str | None = None, stream_callback:
```

```
Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate],
Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

Expand table

Name	Description
name Required*	str | None
description Required*	str | None
arg_name Required*	str
arg_description Required*	str | None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None

Keyword-Only Parameters

Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

Expand table

Type	Description
AIFunction[<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

Expand table

Type	Description
TypeError	If the agent does not implement AgentProtocol.
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
   from agent_framework import ChatAgent

   # Create an agent
   agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

   # Convert the agent to a tool
   research_tool = agent.as_tool()

   # Use the tool with another agent
   coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
`serialized_thread` Required*	Any The serialized thread data.
`kwargs` Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
`kwargs`	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
`value`	

Name	Description
Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

BaseAnnotation Class

Base class for all AI Annotation types.

Initialize BaseAnnotation.

Constructor

```
Python
BaseAnnotation()
```

Keyword-Only Parameters

Expand table

Name	Description
annotated_regions	A list of regions that have been annotated. Can be region objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
****kwargs**	Additional keyword arguments (merged into additional_properties).

Methods

Expand table

__init__	Initialize BaseAnnotation.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize BaseAnnotation.

Python

```
__init__(*, annotated_regions: list[TextSpanRegion] | list[MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
annotated_regions Required*	list[TextSpanRegion] | list[MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
annotated_regions	A list of regions that have been annotated. Can be region objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
****kwargs**	Additional keyword arguments (merged into additional_properties).

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

BaseChatClient Class

Base class for chat clients.

This abstract base class provides core functionality for chat client implementations, including middleware support, message preparation, and tool normalization.

> ⓘ Note
>
> BaseChatClient cannot be instantiated directly as it's an abstract base class.
>
> Subclasses must implement _inner_get_response() and _inner_get_streaming_response().

Initialize a BaseChatClient instance.

Constructor

Python

```
BaseChatClient()
```

Keyword-Only Parameters

Expand table

Name	Description
middleware	Middleware for the client. Default value: None
additional_properties	Additional properties for the client. Default value: None
kwargs	Additional keyword arguments (merged into additional_properties).

Examples

Python

```
from agent_framework import BaseChatClient, ChatResponse, ChatMessage
from collections.abc import AsyncIterable
```

```
   class CustomChatClient(BaseChatClient):
       async def _inner_get_response(self, *, messages, chat_options, **kwargs):
           # Your custom implementation
           return ChatResponse(
               messages=[ChatMessage(role="assistant", text="Hello!")],
response_id="custom-response"
           )

       async def _inner_get_streaming_response(self, *, messages, chat_options,
**kwargs):
           # Your custom streaming implementation
           from agent_framework import ChatResponseUpdate

           yield ChatResponseUpdate(role="assistant", contents=[{"type": "text",
"text": "Hello!"}])


   # Create an instance of your custom client
   client = CustomChatClient()

   # Use the client to get responses
   response = await client.get_response("Hello, how are you?")
```

Methods

Expand table

__init__	Initialize a BaseChatClient instance.
__new__	
create_agent	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

get_response	Get a response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
get_streaming_response	Get a streaming response from a chat client. When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.
service_url	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize a BaseChatClient instance.

Python

```
__init__(*, middleware: ChatMiddleware | Callable[[ChatContext,
Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware |
Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext],
Awaitable[None]]], Awaitable[None]] | list[ChatMiddleware | Callable[[ChatContext,
Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware |
Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext],
Awaitable[None]]], Awaitable[None]]] | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
middleware Required*	ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | list[ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]] | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
middleware	Middleware for the client. Default value: None
additional_properties	Additional properties for the client. Default value: None
kwargs	Additional keyword arguments (merged into additional_properties).

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

Expand table

Name	Description
id Required*	str | None
name Required*	str | None
description Required*	str | None
instructions	str | None

Name	Description
Required*	
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None

Name	Description
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None

Name	Description
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request.

Name	Description
	Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}`

Name	Description
	• **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None

Name	Description
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None

Name	Description
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id	str | None

Name	Description
Required*	
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use.

Name	Description
	Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is *auto*. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include `chat_options` which provides base values that can be overridden by direct parameters.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

Expand table

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'unknown'
```

BaseContent Class

Represents content used by AI services.

Initialize BaseContent.

Constructor

```
Python
BaseContent()
```

Keyword-Only Parameters

Expand table

Name	Description
annotations	Optional annotations associated with the content. Can be annotation objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
****kwargs**	Additional keyword arguments (merged into additional_properties).

Methods

Expand table

__init__	Initialize BaseContent.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection

	to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize BaseContent.

Python

```
__init__(*, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
annotations	Optional annotations associated with the content. Can be annotation objects or dicts. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
****kwargs**	Additional keyword arguments (merged into additional_properties).

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"
```

```python
# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

annotations

Optional annotations associated with the content.

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

Case Class

Runtime wrapper combining a switch-case predicate with its target.

Each *Case* couples a boolean predicate with the executor that should handle the message when the predicate evaluates to *True*. The runtime keeps this lightweight container separate from the serialisable *SwitchCaseEdgeGroupCase* so that execution can operate with live callables without polluting persisted state.

Constructor

Python

```
Case()
```

Parameters

Expand table

Name	Description
condition Required*	
target Required*	

Examples

Python

```
class JsonExecutor(Executor):
    def __init__(self) -> None:
        super().__init__(id="json", defer_discovery=True)


processor = JsonExecutor()
case = Case(condition=lambda payload: payload["kind"] == "json", target=processor)
assert case.target.id == "json"
```

Methods

Expand table

__init__
__new__

__init__

```
Python
__init__(condition: Callable[[Any], bool], target: Executor | str) -> None
```

Parameters

Expand table

Name	Description
condition Required*	Callable[[Any], bool]
target Required*	Executor | str

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

Attributes

condition

```
Python
```

```
condition: Callable[[Any], bool]
```

target

Python

```
target: Executor | str
```

ChatAgent Class

A Chat Client Agent.

This is the primary agent implementation that uses a chat client to interact with language models. It supports tools, context providers, middleware, and both streaming and non-streaming responses.

Initialize a ChatAgent instance.

> **Note**
>
> The set of parameters from frequency_penalty to request_kwargs are used to
>
> call the chat client. They can also be passed to both run methods.
>
> When both are set, the ones passed to the run methods take precedence.

Constructor

Python

```
ChatAgent()
```

Parameters

Expand table

Name	Description
chat_client Required*	The chat client to use for the agent.
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
conversation_id	The conversation ID for service-managed threads. Cannot be used together with chat_message_store_factory. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. This overrides the model_id set in the chat client if it contains one. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: auto
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Examples

Basic usage:

Python

```
from agent_framework import ChatAgent
from agent_framework.clients import OpenAIChatClient

# Create a basic chat agent
client = OpenAIChatClient(model_id="gpt-4")
agent = ChatAgent(chat_client=client, name="assistant", description="A helpful
assistant")

# Run the agent with a simple message
```

```
response = await agent.run("Hello, how are you?")
print(response.text)
```

With tools and streaming:

Python

```
# Create an agent with tools and instructions
def get_weather(location: str) -> str:
    return f"The weather in {location} is sunny."


agent = ChatAgent(
    chat_client=client,
    name="weather-agent",
    instructions="You are a weather assistant.",
    tools=get_weather,
    temperature=0.7,
    max_tokens=500,
)

# Use streaming responses
async for update in agent.run_stream("What's the weather in Paris?"):
    print(update.text, end="")
```

With additional provider specific options:

Python

```
agent = ChatAgent(
    chat_client=client,
    name="reasoning-agent",
    instructions="You are a reasoning assistant.",
    model_id="gpt-5",
    temperature=0.7,
    max_tokens=500,
    additional_chat_options={
        "reasoning": {"effort": "high", "summary": "concise"}
    },  # OpenAI Responses specific.
)

# Use streaming responses
async for update in agent.run_stream("How do you prove the pythagorean theorem?"):
    print(update.text, end="")
```

Methods

Expand table

__init__	Initialize a ChatAgent instance. **Note** The set of parameters from frequency_penalty to request_kwargs are used to call the chat client. They can also be passed to both run methods. When both are set, the ones passed to the run methods take precedence.
__new__	
as_mcp_server	Create an MCP server from an agent instance. This function automatically creates a MCP server from an agent instance, it uses the provided arguments to configure the server and exposes the agent as a single MCP tool.
as_tool	Create an AIFunction tool that wraps this agent.
deserialize_thread	Deserialize a thread from its serialized state.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_new_thread	Get a new conversation thread for the agent. If you supply a service_thread_id, the thread will be marked as service managed. If you don't supply a service_thread_id but have a conversation_id configured on the agent, that conversation_id will be used to create a service-managed thread. If you don't supply a service_thread_id but have a chat_message_store_factory configured on the agent, that factory will be used to create a message store for the thread and the thread will be managed locally.

	When neither is present, the thread will be created without a service ID or message store. This will be updated based on usage when you run the agent with this thread. If you run with `store=True`, the response will include a thread_id and that will be set. Otherwise a message store is created from the default factory.
run	Run the agent with the given messages and options. **ⓘ Note** Since you won't always call agent.run() directly (it gets called through workflows), it is advised to set your default values for all the chat client parameters in the agent constructor. If both parameters are used, the ones passed to the run methods take precedence.
run_stream	Stream the agent with the given messages and options. **ⓘ Note** Since you won't always call agent.run_stream() directly (it gets called through orchestration), it is advised to set your default values for all the chat client parameters in the agent constructor. If both parameters are used, the ones passed to the run methods take precedence.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize a ChatAgent instance.

> **Note**
>
> The set of parameters from frequency_penalty to request_kwargs are used to
>
> call the chat client. They can also be passed to both run methods.
>
> When both are set, the ones passed to the run methods take precedence.

Python

```
__init__(chat_client: ChatClientProtocol, instructions: str | None = None, *, id:
str | None = None, name: str | None = None, description: str | None = None,
chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None,
context_providers: ContextProvider | list[ContextProvider] |
AggregateContextProvider | None = None, middleware: AgentMiddleware |
Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]],
Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
chat_client Required*	ChatClientProtocol The chat client to use for the agent.

Name	Description
instructions	str | None Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
id Required*	str | None
name Required*	str | None
description Required*	str | None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] | None
context_providers Required*	ContextProvider | list[ContextProvider] | AggregateContextProvider | None
middleware Required*	AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata	dict[str, Any] | None

Name	Description
Required*	
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	

Name	Description
	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	The collection of multiple context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
conversation_id	The conversation ID for service-managed threads. Cannot be used together with chat_message_store_factory. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. This overrides the model_id set in the chat client if it contains one. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client `get_response` and `get_streaming_response` methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as `additional_properties`.

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
AgentInitializationError	If both conversation_id and chat_message_store_factory are provided.

__new__

```
Python
__new__(**kwargs)
```

as_mcp_server

Create an MCP server from an agent instance.

This function automatically creates a MCP server from an agent instance, it uses the provided arguments to configure the server and exposes the agent as a single MCP tool.

```
Python
as_mcp_server(*, server_name: str = 'Agent', version: str | None = None,
instructions: str | None = None, lifespan: Callable[[Server[Any, Any]],
AbstractAsyncContextManager[Any]] | None = None, **kwargs: Any) -> Server[Any,
Any]
```

Parameters

Expand table

Name	Description
server_name Required*	str
version Required*	str | None
instructions Required*	str | None
lifespan Required*	Callable[[<xref:mcp.server.lowlevel.server.Server> [Any, Any]], AbstractAsyncContextManager[Any]] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
server_name	The name of the server. Default value: "Agent"
version	The version of the server. Default value: None
instructions	The instructions to use for the server. Default value: None
lifespan	The lifespan of the server. Default value: None
****kwargs**	Any extra arguments to pass to the server creation.

Returns

Expand table

Type	Description
<xref:mcp.server.lowlevel.server.Server>[Any, Any]	The MCP server instance.

as_tool

Create an AIFunction tool that wraps this agent.

Python

```
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str = 'task', arg_description: str | None = None, stream_callback: Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

Expand table

Name	Description
name Required*	str | None
description Required*	str | None

Name	Description
arg_name Required*	str
arg_description Required*	str | None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None

Keyword-Only Parameters

Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

Expand table

Type	Description
AIFunction[<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

Expand table

Type	Description
TypeError	If the agent does not implement AgentProtocol.
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
   from agent_framework import ChatAgent

   # Create an agent
   agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

   # Convert the agent to a tool
   research_tool = agent.as_tool()

   # Use the tool with another agent
   coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any]

Name	Description
	The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```python
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Get a new conversation thread for the agent.

If you supply a service_thread_id, the thread will be marked as service managed.

If you don't supply a service_thread_id but have a conversation_id configured on the agent, that conversation_id will be used to create a service-managed thread.

If you don't supply a service_thread_id but have a chat_message_store_factory configured on the agent, that factory will be used to create a message store for the thread and the thread will be managed locally.

When neither is present, the thread will be created without a service ID or message store. This will be updated based on usage when you run the agent with this thread. If you run with `store=True`, the response will include a thread_id and that will be set. Otherwise a message store is created from the default factory.

Python

```
get_new_thread(*, service_thread_id: str | None = None, **kwargs: Any) ->
AgentThread
```

Parameters

Expand table

Name	Description
service_thread_id Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
service_thread_id	Optional service managed thread ID. Default value: None
kwargs	Not used at present.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance.

run

Run the agent with the given messages and options.

> **Note**
>
> Since you won't always call agent.run() directly (it gets called
>
> through workflows), it is advised to set your default values for
>
> all the chat client parameters in the agent constructor.
>
> If both parameters are used, the ones passed to the run methods take precedence.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | list[ToolProtocol | Callable[..., Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The messages to process. Default value: None
thread Required*	AgentThread | None
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None

Name	Description
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | list[ToolProtocol | Callable[..., Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The thread to use for the agent. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	Additional properties to include in the request. Use this field for provider-specific parameters. Default value: None
kwargs	Additional keyword arguments for the agent. Will only be passed to functions that are called.

Returns

Expand table

Type	Description
AgentRunResponse	An AgentRunResponse containing the agent's response.

run_stream

Stream the agent with the given messages and options.

> **Note**
>
> Since you won't always call agent.run_stream() directly (it gets called

through orchestration), it is advised to set your default values for

all the chat client parameters in the agent constructor.

If both parameters are used, the ones passed to the run methods take precedence.

Python

```
run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | list[ToolProtocol | Callable[..., Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> AsyncIterable[AgentRunResponseUpdate]
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The messages to process. Default value: None
thread Required*	AgentThread | None
allow_multiple_tool_calls Required*	bool | None
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None

Name	Description
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | list[ToolProtocol | Callable[..., Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_chat_options Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The thread to use for the agent. Default value: None

Name	Description
`allow_multiple_tool_calls`	Whether to allow multiple tool calls in a single response. Default value: None
`frequency_penalty`	The frequency penalty to use. Default value: None
`logit_bias`	The logit bias to use. Default value: None
`max_tokens`	The maximum number of tokens to generate. Default value: None
`metadata`	Additional metadata to include in the request. Default value: None
`model_id`	The model_id to use for the agent. Default value: None
`presence_penalty`	The presence penalty to use. Default value: None
`response_format`	The format of the response. Default value: None
`seed`	The random seed to use. Default value: None
`stop`	The stop sequence(s) for the request. Default value: None
`store`	Whether to store the response. Default value: None
`temperature`	The sampling temperature to use. Default value: None
`tool_choice`	The tool choice for the request. Default value: None
`tools`	The tools to use for the request. Default value: None
`top_p`	The nucleus sampling probability to use. Default value: None
`user`	The user to associate with the request. Default value: None
`additional_chat_options`	Additional properties to include in the request. Use this field for provider-specific parameters.

Name	Description
	Default value: None
kwargs	Any additional keyword arguments. Will only be passed to functions that are called.

Returns

Expand table

Type	Description
AsyncIterable[AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

AGENT_PROVIDER_NAME

Python

```
AGENT_PROVIDER_NAME: ClassVar[str] = 'microsoft.agent_framework'
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None
```

ChatClientProtocol Class

A protocol for a chat client that can generate responses.

This protocol defines the interface that all chat clients must implement, including methods for generating both streaming and non-streaming responses.

> ⓘ Note
>
> Protocols use structural subtyping (duck typing). Classes don't need
>
> to explicitly inherit from this protocol to be considered compatible.

Constructor

Python

```
ChatClientProtocol()
```

Examples

Python

```
from agent_framework import ChatClientProtocol, ChatResponse, ChatMessage


# Any class implementing the required methods is compatible
class CustomChatClient:
    @property
    def additional_properties(self) -> dict[str, Any]:
        return {}

    async def get_response(self, messages, **kwargs):
        # Your custom implementation
        return ChatResponse(messages=[], response_id="custom")

    def get_streaming_response(self, messages, **kwargs):
        async def _stream():
            from agent_framework import ChatResponseUpdate

            yield ChatResponseUpdate()

        return _stream()
```

```
# Verify the instance satisfies the protocol
client = CustomChatClient()
assert isinstance(client, ChatClientProtocol)
```

Methods

Expand table

__init__	
__new__	
get_response	Send input and return the response.
get_streaming_response	Send input messages and stream the response.

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

get_response

Send input and return the response.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
```

```
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> ChatResponse
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The sequence of input messages to send.
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None

Name	Description
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None

Name	Description
`temperature`	The sampling temperature to use. Default value: None
`tool_choice`	The tool choice for the request. Default value: None
`tools`	The tools to use for the request. Default value: None
`top_p`	The nucleus sampling probability to use. Default value: None
`user`	The user to associate with the request. Default value: None
`additional_properties`	Additional properties to include in the request. Default value: None
`kwargs`	Any additional keyword arguments. Will only be passed to functions that are called.

Returns

Expand table

Type	Description
ChatResponse	The response messages generated by the client.

Exceptions

Expand table

Type	Description
ValueError	If the input message sequence is `None`

get_streaming_response

Send input messages and stream the response.

Python

```
get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, frequency_penalty: float | None = None, logit_bias: dict[str
```

```
| int, float] | None = None, max_tokens: int | None = None, metadata: dict[str,
Any] | None = None, model_id: str | None = None, presence_penalty: float | None =
None, response_format: type[BaseModel] | None = None, seed: int | None = None,
stop: str | Sequence[str] | None = None, store: bool | None = None, temperature:
float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

Expand table

Name	Description
messages Required*	str | ChatMessage | list[str] | list[ChatMessage] The sequence of input messages to send.
frequency_penalty Required*	float | None
logit_bias Required*	dict[str | int, float] | None
max_tokens Required*	int | None
metadata Required*	dict[str, Any] | None
model_id Required*	str | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature	float | None

Name	Description
Required*	
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | dict[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use.

Name	Description
	Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Default value: None
kwargs	Any additional keyword arguments. Will only be passed to functions that are called.

Returns

Expand table

Type	Description
AsyncIterable[ChatResponseUpdate]	

Exceptions

Expand table

Type	Description
ValueError	If the input message sequence is `None`

Attributes

additional_properties

Get additional properties associated with the client.

ChatContext Class

Context object for chat middleware invocations.

This context is passed through the chat middleware pipeline and contains all information about the chat request.

Initialize the ChatContext.

Constructor

```
Python
ChatContext()
```

Parameters

Expand table

Name	Description
chat_client Required*	The chat client being invoked.
messages Required*	The messages being sent to the chat client.
chat_options Required*	The options for the chat request.
is_streaming	Whether this is a streaming invocation. Default value: False
metadata	Metadata dictionary for sharing data between chat middleware. Default value: None
result	Chat execution result. Default value: None
terminate	A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	Additional keyword arguments passed to the chat client. Default value: None

Examples

Python

```python
from agent_framework import ChatMiddleware, ChatContext


class TokenCounterMiddleware(ChatMiddleware):
    async def process(self, context: ChatContext, next):
        print(f"Chat client: {context.chat_client.__class__.__name__}")
        print(f"Messages: {len(context.messages)}")
        print(f"Model: {context.chat_options.model_id}")

        # Store metadata
        context.metadata["input_tokens"] = self.count_tokens(context.messages)

        # Continue execution
        await next(context)

        # Access result and count output tokens
        if context.result:
            context.metadata["output_tokens"] = self.count_tokens(context.result)
```

Methods

Expand table

__init__	Initialize the ChatContext.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

	Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the ChatContext.

Python

```
__init__(chat_client: ChatClientProtocol, messages: MutableSequence[ChatMessage], chat_options: ChatOptions, is_streaming: bool = False, metadata: dict[str, Any] | None = None, result: ChatResponse | AsyncIterable[ChatResponseUpdate] | None = None, terminate: bool = False, kwargs: dict[str, Any] | None = None) -> None
```

Parameters

Expand table

Name	Description
chat_client Required*	ChatClientProtocol The chat client being invoked.
messages Required*	MutableSequence[ChatMessage] The messages being sent to the chat client.
chat_options Required*	ChatOptions The options for the chat request.
is_streaming	bool Whether this is a streaming invocation. Default value: False
metadata	dict[str, Any] | None Metadata dictionary for sharing data between chat middleware. Default value: None
result	ChatResponse | AsyncIterable[ChatResponseUpdate] | None Chat execution result. Default value: None
terminate	bool A flag indicating whether to terminate execution after current middleware.

Name	Description
	Default value: False
kwargs	dict[str, Any] | None Additional keyword arguments passed to the chat client. Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python
`__new__(**kwargs)`

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	

Name	Description
	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

chat_client

The chat client being invoked.

messages

The messages being sent to the chat client.

chat_options

The options for the chat request.

is_streaming

Whether this is a streaming invocation.

metadata

Metadata dictionary for sharing data between chat middleware.

result

Chat execution result. Can be observed after calling `next()` to see the actual execution result or can be set to override the execution result. For non-streaming: should be ChatResponse. For streaming: should be AsyncIterable[ChatResponseUpdate].

terminate

A flag indicating whether to terminate execution after current middleware. When set to True, execution will stop as soon as control returns to framework.

kwargs

Additional keyword arguments passed to the chat client.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'chat_client', 'result'}
```

ChatMessage Class

Represents a chat message.

Initialize ChatMessage.

Constructor

```python
ChatMessage()
```

Parameters

Expand table

Name	Description
role Required*	The role of the author of the message (Role, string, or dict).

Keyword-Only Parameters

Expand table

Name	Description
text	Optional text content of the message. Default value: None
contents	Optional list of BaseContent items or dicts to include in the message. Default value: None
author_name	Optional name of the author of the message. Default value: None
message_id	Optional ID of the chat message. Default value: None
additional_properties	Optional additional properties associated with the chat message. Additional properties are used within Agent Framework, they are not sent to services. Default value: None
raw_representation	Optional raw representation of the chat message.

Name	Description
	Default value: None
kwargs	will be combined with additional_properties if provided.

Examples

Python

```
from agent_framework import ChatMessage, TextContent

# Create a message with text
user_msg = ChatMessage(role="user", text="What's the weather?")
print(user_msg.text)  # "What's the weather?"

# Create a message with role string
system_msg = ChatMessage(role="system", text="You are a helpful assistant.")

# Create a message with contents
assistant_msg = ChatMessage(
    role="assistant",
    contents=[TextContent(text="The weather is sunny!")],
)
print(assistant_msg.text)  # "The weather is sunny!"

# Serialization - to_dict and from_dict
msg_dict = user_msg.to_dict()
# {'type': 'chat_message', 'role': {'type': 'role', 'value': 'user'},
#  'contents': [{'type': 'text', 'text': "What's the weather?"}],
'additional_properties': {}}
restored_msg = ChatMessage.from_dict(msg_dict)
print(restored_msg.text)  # "What's the weather?"

# Serialization - to_json and from_json
msg_json = user_msg.to_json()
# '{"type": "chat_message", "role": {"type": "role", "value": "user"}, "contents":
[...], ...}'
restored_from_json = ChatMessage.from_json(msg_json)
print(restored_from_json.role.value)  # "user"
```

Methods

Expand table

__init__	Initialize ChatMessage.
__new__	

from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize ChatMessage.

Python

```
__init__(role: Role | Literal['system', 'user', 'assistant', 'tool'], *, text:
str, author_name: str | None = None, message_id: str | None = None,
additional_properties: MutableMapping[str, Any] | None = None, raw_representation:
Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
role Required*	The role of the author of the message (Role, string, or dict).

Keyword-Only Parameters

Expand table

Name	Description
text	Optional text content of the message. Default value: None
contents	Optional list of BaseContent items or dicts to include in the message. Default value: None
author_name	Optional name of the author of the message. Default value: None
message_id	Optional ID of the chat message. Default value: None
additional_properties	Optional additional properties associated with the chat message. Additional properties are used within Agent Framework, they are not sent to services. Default value: None
raw_representation	Optional raw representation of the chat message. Default value: None
kwargs	will be combined with additional_properties if provided.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
```

```
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution

contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific).

Name	Description
	Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCl ass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

text

Returns the text content of the message.

Remarks: This property concatenates the text of all TextContent objects in Contents.

role

The role of the author of the message.

contents

The chat message content items.

author_name

The name of the author of the message.

message_id

The ID of the chat message.

additional_properties

Any additional properties associated with the chat message. Additional properties are used within Agent Framework, they are not sent to services.

raw_representation

The raw representation of the chat message from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ChatMessageStore Class

An in-memory implementation of ChatMessageStoreProtocol that stores messages in a list.

This implementation provides a simple, list-based storage for chat messages with support for serialization and deserialization. It implements all the required methods of the `ChatMessageStoreProtocol` protocol.

The store maintains messages in memory and provides methods to serialize and deserialize the state for persistence purposes.

Create a ChatMessageStore for use in a thread.

Constructor

Python

```
ChatMessageStore()
```

Parameters

Expand table

Name	Description
messages	The messages to store. Default value: None

Examples

Python

```
from agent_framework import ChatMessageStore, ChatMessage

# Create an empty store
store = ChatMessageStore()

# Add messages
message = ChatMessage(role="user", content="Hello")
await store.add_messages([message])

# Retrieve messages
messages = await store.list_messages()
```

```
# Serialize for persistence
state = await store.serialize()

# Deserialize from saved state
restored_store = await ChatMessageStore.deserialize(state)
```

Methods

Expand table

__init__	Create a ChatMessageStore for use in a thread.
__new__	
add_messages	Add messages to the store.
deserialize	Create a new ChatMessageStore instance from serialized state data.
list_messages	Get all messages from the store in chronological order.
serialize	Serialize the current store state for persistence.
update_from_state	Update the current ChatMessageStore instance from serialized state data.

__init__

Create a ChatMessageStore for use in a thread.

Python

```
__init__(messages: Sequence[ChatMessage] | None = None)
```

Parameters

Expand table

Name	Description
messages	Sequence[ChatMessage] | None The messages to store. Default value: None

__new__

Python

```
__new__(**kwargs)
```

add_messages

Add messages to the store.

Python

```
async add_messages(messages: Sequence[ChatMessage]) -> None
```

Parameters

Expand table

Name	Description
messages Required*	Sequence[ChatMessage] Sequence of ChatMessage objects to add to the store.

Returns

Expand table

Type	Description
None	

deserialize

Create a new ChatMessageStore instance from serialized state data.

Python

```
async classmethod deserialize(serialized_store_state: MutableMapping[str, Any],
**kwargs: Any) -> TChatMessageStore
```

Parameters

Expand table

Name	Description
serialized_store_state	MutableMapping[str, Any]

Name	Description
Required*	Previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional arguments for deserialization.

Returns

Expand table

Type	Description
<xref:agent_framework._threads.TChatMessageStore>	A new ChatMessageStore instance populated with messages from the serialized state.

list_messages

Get all messages from the store in chronological order.

Python

```
async list_messages() -> list[ChatMessage]
```

Returns

Expand table

Type	Description
list[ChatMessage]	List of ChatMessage objects, ordered from oldest to newest.

serialize

Serialize the current store state for persistence.

Python

```
async serialize(**kwargs: Any) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional arguments for serialization.

Returns

Expand table

Type	Description
dict[str, Any]	Serialized state data that can be used with deserialize_state.

update_from_state

Update the current ChatMessageStore instance from serialized state data.

Python

```
async update_from_state(serialized_store_state: MutableMapping[str, Any],
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
serialized_store_state	MutableMapping[str, Any]

Name	Description
Required*	Previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional arguments for deserialization.

Returns

Expand table

Type	Description
None	

ChatMessageStoreProtocol Class

Defines methods for storing and retrieving chat messages associated with a specific thread.

Implementations of this protocol are responsible for managing the storage of chat messages, including handling large volumes of data by truncating or summarizing messages as necessary.

Constructor

Python

```
ChatMessageStoreProtocol()
```

Examples

Python

```
from agent_framework import ChatMessage


class MyMessageStore:
    def __init__(self):
        self._messages = []

    async def list_messages(self) -> list[ChatMessage]:
        return self._messages

    async def add_messages(self, messages: Sequence[ChatMessage]) -> None:
        self._messages.extend(messages)

    @classmethod
    async def deserialize(cls, serialized_store_state, **kwargs):
        store = cls()
        store._messages = serialized_store_state.get("messages", [])
        return store

    async def update_from_state(self, serialized_store_state, **kwargs) -> None:
        self._messages = serialized_store_state.get("messages", [])

    async def serialize(self, **kwargs):
        return {"messages": self._messages}


# Use the custom store
store = MyMessageStore()
```

Methods

Expand table

__init__	
__new__	
add_messages	Adds messages to the store.
deserialize	Creates a new instance of the store from previously serialized state. This method, together with `serialize()` can be used to save and load messages from a persistent store if this store only has messages in memory.
list_messages	Gets all the messages from the store that should be used for the next agent invocation. Messages are returned in ascending chronological order, with the oldest message first. If the messages stored in the store become very large, it is up to the store to truncate, summarize or otherwise limit the number of messages returned. When using implementations of `ChatMessageStoreProtocol`, a new one should be created for each thread since they may contain state that is specific to a thread.
serialize	Serializes the current object's state. This method, together with `deserialize()` can be used to save and load messages from a persistent store if this store only has messages in memory.
update_from_state	Update the current ChatMessageStore instance from serialized state data.

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

add_messages

Adds messages to the store.

Python

```
async add_messages(messages: Sequence[ChatMessage]) -> None
```

Parameters

Expand table

Name	Description
messages Required*	Sequence[ChatMessage] The sequence of ChatMessage objects to add to the store.

Returns

Expand table

Type	Description
None	

deserialize

Creates a new instance of the store from previously serialized state.

This method, together with `serialize()` can be used to save and load messages from a persistent store if this store only has messages in memory.

Python

```
async classmethod deserialize(serialized_store_state: MutableMapping[str, Any],
**kwargs: Any) -> ChatMessageStoreProtocol
```

Parameters

Expand table

Name	Description
serialized_store_state Required*	MutableMapping[str, Any] The previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
**kwargs	Additional arguments for deserialization.

Returns

Expand table

Type	Description
ChatMessageStoreProtocol	A new instance of the store populated with messages from the serialized state.

list_messages

Gets all the messages from the store that should be used for the next agent invocation.

Messages are returned in ascending chronological order, with the oldest message first.

If the messages stored in the store become very large, it is up to the store to truncate, summarize or otherwise limit the number of messages returned.

When using implementations of `ChatMessageStoreProtocol`, a new one should be created for each thread since they may contain state that is specific to a thread.

Python

```
async list_messages() -> list[ChatMessage]
```

Returns

Expand table

Type	Description
list[ChatMessage]	

serialize

Serializes the current object's state.

This method, together with `deserialize()` can be used to save and load messages from a persistent store if this store only has messages in memory.

Python

```
async serialize(**kwargs: Any) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional arguments for serialization.

Returns

Expand table

Type	Description
dict[str, Any]	The serialized state data that can be used with `deserialize()`.

update_from_state

Update the current ChatMessageStore instance from serialized state data.

Python

```
async update_from_state(serialized_store_state: MutableMapping[str, Any],
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
serialized_store_state Required*	MutableMapping[str, Any] Previously serialized state data containing messages.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional arguments for deserialization.

Returns

Expand table

Type	Description
None	

ChatMiddleware Class

Abstract base class for chat middleware that can intercept chat client requests.

Chat middleware allows you to intercept and modify chat client requests before and after execution. You can modify messages, add system prompts, log requests, or override chat responses.

> ⓘ Note
>
> ChatMiddleware is an abstract base class. You must subclass it and implement
>
> the process() method to create custom chat middleware.

Constructor

Python

```
ChatMiddleware()
```

Examples

Python

```
from agent_framework import ChatMiddleware, ChatContext, ChatAgent


class SystemPromptMiddleware(ChatMiddleware):
    def __init__(self, system_prompt: str):
        self.system_prompt = system_prompt

    async def process(self, context: ChatContext, next):
        # Add system prompt to messages
        from agent_framework import ChatMessage

        context.messages.insert(0, ChatMessage(role="system",
content=self.system_prompt))

        # Continue execution
        await next(context)


# Use with an agent
```

```
    agent = ChatAgent(
        chat_client=client, name="assistant", middleware=SystemPromptMiddleware("You
are a helpful assistant.")
    )
```

Methods

Expand table

__init__	
__new__	
process	Process a chat client request.

> **Note**
>
> Middleware should not return anything. All data manipulation should happen within the context object. Set context.result to override execution, or observe context.result after calling next() for actual results.

__init__

Python

```
__init__()
```

__new__

Python

```
__new__(**kwargs)
```

process

Process a chat client request.

> **Note**
>
> Middleware should not return anything. All data manipulation should happen

within the context object. Set context.result to override execution,

or observe context.result after calling next() for actual results.

Python

```
abstract async process(context: ChatContext, next: Callable[[ChatContext],
Awaitable[None]]) -> None
```

Parameters

Expand table

Name	Description
context Required*	ChatContext Chat invocation context containing chat client, messages, options, and metadata. Use context.is_streaming to determine if this is a streaming call. Middleware can set context.result to override execution, or observe the actual execution result after calling next(). For non-streaming: ChatResponse For streaming: AsyncIterable[ChatResponseUpdate]
next Required*	Callable[[ChatContext], Awaitable[None]] Function to call the next middleware or final chat execution. Does not return anything - all data flows through the context.

Returns

Expand table

Type	Description
None	

ChatOptions Class

Common request settings for AI services.

Initialize ChatOptions.

Constructor

```python
ChatOptions()
```

Keyword-Only Parameters

Expand table

Name	Description
model_id	The AI model ID to use. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls. Default value: None
conversation_id	The conversation ID. Default value: None
frequency_penalty	The frequency penalty (must be between -2.0 and 2.0). Default value: None
instructions	the instructions, will be turned into a system or equivalent message. Default value: None
logit_bias	The logit bias mapping. Default value: None
max_tokens	The maximum number of tokens (must be > 0). Default value: None
metadata	Metadata mapping. Default value: None
presence_penalty	The presence penalty (must be between -2.0 and 2.0). Default value: None
response_format	Structured output response format schema. Must be a valid Pydantic model.

Name	Description
	Default value: None
seed	Random seed for reproducibility. Default value: None
stop	Stop sequences. Default value: None
store	Whether to store the conversation. Default value: None
temperature	The temperature (must be between 0.0 and 2.0). Default value: None
tool_choice	The tool choice mode. Default value: None
tools	List of available tools. Default value: None
top_p	The top-p value (must be between 0.0 and 1.0). Default value: None
user	The user ID. Default value: None
additional_properties	Provider-specific additional properties, can also be passed as kwargs. Default value: None
****kwargs**	Additional properties to include in additional_properties.

Examples

Python

```python
from agent_framework import ChatOptions, ai_function

# Create basic chat options
options = ChatOptions(
    model_id="gpt-4",
    temperature=0.7,
    max_tokens=1000,
)


# With tools
@ai_function
def get_weather(location: str) -> str:
    '''Get weather for a location.'''
```

```
        return f"Weather in {location}"


    options = ChatOptions(
        model_id="gpt-4",
        tools=get_weather,
        tool_choice="auto",
    )

    # Require a specific tool to be called
    options_required = ChatOptions(
        model_id="gpt-4",
        tools=get_weather,
        tool_choice=ToolMode.REQUIRED(function_name="get_weather"),
    )

    # Combine options
    base_options = ChatOptions(temperature=0.5)
    extended_options = ChatOptions(max_tokens=500, tools=get_weather)
    combined = base_options & extended_options
```

Methods

Expand table

__init__	Initialize ChatOptions.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize ChatOptions.

Python

```
__init__(*, model_id: str | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, instructions: str | None = None, logit_bias: MutableMapping[str | int, float] | None = None, max_tokens: int | None = None, metadata: MutableMapping[str, str] | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | Mapping[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_properties: MutableMapping[str, Any] | None = None, **kwargs: Any)
```

Parameters

Expand table

Name	Description
model_id Required*	str | None
allow_multiple_tool_calls Required*	bool | None
conversation_id Required*	str | None
frequency_penalty Required*	float | None
instructions Required*	str | None
logit_bias Required*	MutableMapping[str | int, float] | None

Name	Description
max_tokens Required*	int | None
metadata Required*	MutableMapping[str, str] | None
presence_penalty Required*	float | None
response_format Required*	type[<xref:pydantic.main.BaseModel>] | None
seed Required*	int | None
stop Required*	str | Sequence[str] | None
store Required*	bool | None
temperature Required*	float | None
tool_choice Required*	ToolMode | Literal['auto', 'required', 'none'] | collections.abc.Mapping[str, typing.Any] | None
tools Required*	ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None
top_p Required*	float | None
user Required*	str | None
additional_properties Required*	MutableMapping[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
model_id	The AI model ID to use. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls. Default value: None
conversation_id	The conversation ID. Default value: None
frequency_penalty	The frequency penalty (must be between -2.0 and 2.0). Default value: None
instructions	the instructions, will be turned into a system or equivalent message. Default value: None
logit_bias	The logit bias mapping. Default value: None
max_tokens	The maximum number of tokens (must be > 0). Default value: None
metadata	Metadata mapping. Default value: None
presence_penalty	The presence penalty (must be between -2.0 and 2.0). Default value: None
response_format	Structured output response format schema. Must be a valid Pydantic model. Default value: None
seed	Random seed for reproducibility. Default value: None
stop	Stop sequences. Default value: None
store	Whether to store the conversation. Default value: None
temperature	The temperature (must be between 0.0 and 2.0). Default value: None
tool_choice	The tool choice mode. Default value: None
tools	List of available tools. Default value: None
top_p	The top-p value (must be between 0.0 and 1.0).

Name	Description
	Default value: None
user	The user ID. Default value: None
additional_properties	Provider-specific additional properties, can also be passed as kwargs. Default value: None
****kwargs**	Additional properties to include in additional_properties.

__new__

Python
`__new__(**kwargs)`

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```python
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

tools

Return the tools that are specified.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'_tools'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ChatResponse Class

Represents the response to a chat request.

Initializes a ChatResponse with the provided parameters.

Constructor

Python

```
ChatResponse()
```

Keyword-Only Parameters

Expand table

Name	Description
messages	A single ChatMessage or a sequence of ChatMessage objects to include in the response. Default value: None
text	The text content to include in the response. If provided, it will be added as a ChatMessage. Default value: None
response_id	Optional ID of the chat response. Default value: None
conversation_id	Optional identifier for the state of the conversation. Default value: None
model_id	Optional model ID used in the creation of the chat response. Default value: None
created_at	Optional timestamp for the chat response. Default value: None
finish_reason	Optional reason for the chat response. Default value: None
usage_details	Optional usage details for the chat response. Default value: None
value	Optional value of the structured output. Default value: None

Name	Description
response_format	Optional response format for the chat response. Default value: None
additional_properties	Optional additional properties associated with the chat response. Default value: None
raw_representation	Optional raw representation of the chat response from an underlying implementation. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import ChatResponse, ChatMessage

# Create a simple text response
response = ChatResponse(text="Hello, how can I help you?")
print(response.text)  # "Hello, how can I help you?"

# Create a response with messages
msg = ChatMessage(role="assistant", text="The weather is sunny.")
response = ChatResponse(
    messages=[msg],
    finish_reason="stop",
    model_id="gpt-4",
)

# Combine streaming updates
updates = [...]  # List of ChatResponseUpdate objects
response = ChatResponse.from_chat_response_updates(updates)

# Serialization - to_dict and from_dict
response_dict = response.to_dict()
# {'type': 'chat_response', 'messages': [...], 'model_id': 'gpt-4',
#  'finish_reason': {'type': 'finish_reason', 'value': 'stop'}}
restored_response = ChatResponse.from_dict(response_dict)
print(restored_response.model_id)  # "gpt-4"

# Serialization - to_json and from_json
response_json = response.to_json()
# '{"type": "chat_response", "messages": [...], "model_id": "gpt-4", ...}'
restored_from_json = ChatResponse.from_json(response_json)
print(restored_from_json.text)  # "The weather is sunny."
```

Methods

Expand table

__init__	Initializes a ChatResponse with the provided parameters.
__new__	
from_chat_response_generator	Joins multiple updates into a single ChatResponse.
from_chat_response_updates	Joins multiple updates into a single ChatResponse.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.
try_parse_value	If there is a value, does nothing, otherwise tries to parse the text into the value.

__init__

Initializes a ChatResponse with the provided parameters.

Python

```
__init__(*, messages: ChatMessage | MutableSequence[ChatMessage], response_id: str
| None = None, conversation_id: str | None = None, model_id: str | None = None,
created_at: str | None = None, finish_reason: FinishReason | None = None,
usage_details: UsageDetails | None = None, value: Any | None = None,
response_format: type[BaseModel] | None = None, additional_properties: dict[str,
Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Keyword-Only Parameters

Expand table

Name	Description
messages	A single ChatMessage or a sequence of ChatMessage objects to include in the response. Default value: None
text	The text content to include in the response. If provided, it will be added as a ChatMessage. Default value: None
response_id	Optional ID of the chat response. Default value: None
conversation_id	Optional identifier for the state of the conversation. Default value: None
model_id	Optional model ID used in the creation of the chat response. Default value: None
created_at	Optional timestamp for the chat response. Default value: None
finish_reason	Optional reason for the chat response. Default value: None
usage_details	Optional usage details for the chat response. Default value: None
value	Optional value of the structured output. Default value: None
response_format	Optional response format for the chat response. Default value: None
additional_properties	Optional additional properties associated with the chat response. Default value: None
raw_representation	Optional raw representation of the chat response from an underlying implementation.

Name	Description
	Default value: None
**kwargs	Any additional keyword arguments.

__new__

```
Python
__new__(**kwargs)
```

from_chat_response_generator

Joins multiple updates into a single ChatResponse.

```
Python
async classmethod from_chat_response_generator(updates:
AsyncIterable[ChatResponseUpdate], *, output_format_type: type[BaseModel] | None =
None) -> TChatResponse
```

Parameters

Expand table

Name	Description
updates Required*	AsyncIterable[ChatResponseUpdate] An async iterable of ChatResponseUpdate objects to combine.
output_format_type Required*	type[<xref:pydantic.main.BaseModel>] | None

Keyword-Only Parameters

Expand table

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data. Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._types.TChatResponse>	

Examples

Python

```
from agent_framework import ChatResponse, ChatResponseUpdate, ChatClient

client = ChatClient()  # should be a concrete implementation
response = await ChatResponse.from_chat_response_generator(
    client.get_streaming_response("Hello, how are you?")
)
print(response.text)
```

from_chat_response_updates

Joins multiple updates into a single ChatResponse.

Python

```
from_chat_response_updates(updates: Sequence[ChatResponseUpdate], *,
output_format_type: type[BaseModel] | None = None) -> TChatResponse
```

Parameters

Expand table

Name	Description
updates Required*	Sequence[ChatResponseUpdate] A sequence of ChatResponseUpdate objects to combine.
output_format_type Required*	type[<xref:pydantic.main.BaseModel>] | None

Keyword-Only Parameters

Expand table

Name	Description
output_format_type	Optional Pydantic model type to parse the response text into structured data. Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._types.TChatResponse>	

Examples

Python

```
from agent_framework import ChatResponse, ChatResponseUpdate

# Create some response updates
updates = [
    ChatResponseUpdate(role="assistant", text="Hello"),
    ChatResponseUpdate(text=" How can I help you?"),
]

# Combine updates into a single ChatResponse
response = ChatResponse.from_chat_response_updates(updates)
print(response.text)  # "Hello How can I help you?"
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"
```

```python
# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None

Name	Description
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

try_parse_value

If there is a value, does nothing, otherwise tries to parse the text into the value.

Python

```
try_parse_value(output_format_type: type[BaseModel]) -> None
```

Parameters

Expand table

Name	Description
`output_format_type` Required*	type[<xref:pydantic.main.BaseModel>]

Returns

Expand table

Type	Description
None	

Attributes

text

Returns the concatenated text of all messages in the response.

messages

The list of chat messages in the response.

response_id

The ID of the chat response.

conversation_id

An identifier for the state of the conversation.

model_id

The model ID used in the creation of the chat response.

created_at

A timestamp for the chat response.

finish_reason

The reason for the chat response.

usage_details

The usage details for the chat response.

structured_output

The structured output of the chat response, if applicable.

additional_properties

Any additional properties associated with the chat response.

raw_representation

The raw representation of the chat response from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ChatResponseUpdate Class

Represents a single streaming response chunk from a *ChatClient*.

Initializes a ChatResponseUpdate with the provided parameters.

Constructor

Python

```
ChatResponseUpdate()
```

Keyword-Only Parameters

Expand table

Name	Description
contents	Optional list of BaseContent items or dicts to include in the update. Default value: None
text	Optional text content to include in the update. Default value: None
role	Optional role of the author of the response update (Role, string, or dict Default value: None
author_name	Optional name of the author of the response update. Default value: None
response_id	Optional ID of the response of which this update is a part. Default value: None
message_id	Optional ID of the message of which this update is a part. Default value: None
conversation_id	Optional identifier for the state of the conversation of which this update is a part Default value: None
model_id	Optional model ID associated with this response update. Default value: None
created_at	Optional timestamp for the chat response update. Default value: None

Name	Description
finish_reason	Optional finish reason for the operation. Default value: None
additional_properties	Optional additional properties associated with the chat response update. Default value: None
raw_representation	Optional raw representation of the chat response update from an underlying implementation. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import ChatResponseUpdate, TextContent

# Create a response update
update = ChatResponseUpdate(
    contents=[TextContent(text="Hello")],
    role="assistant",
    message_id="msg_123",
)
print(update.text)  # "Hello"

# Create update with text shorthand
update = ChatResponseUpdate(text="World!", role="assistant")

# Serialization - to_dict and from_dict
update_dict = update.to_dict()
# {'type': 'chat_response_update', 'contents': [{'type': 'text', 'text': 'Hello'}],
#  'role': {'type': 'role', 'value': 'assistant'}, 'message_id': 'msg_123'}
restored_update = ChatResponseUpdate.from_dict(update_dict)
print(restored_update.text)  # "Hello"

# Serialization - to_json and from_json
update_json = update.to_json()
# '{"type": "chat_response_update", "contents": [{"type": "text", "text":
"Hello"}], ...}'
restored_from_json = ChatResponseUpdate.from_json(update_json)
print(restored_from_json.message_id)  # "msg_123"
```

Methods

Expand table

__init__	Initializes a ChatResponseUpdate with the provided parameters.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a ChatResponseUpdate with the provided parameters.

Python

```
__init__(*, contents: Sequence[TextContent | DataContent | TextReasoningContent |
UriContent | FunctionCallContent | FunctionResultContent | ErrorContent |
UsageContent | HostedFileContent | HostedVectorStoreContent |
FunctionApprovalRequestContent | FunctionApprovalResponseContent | dict[str, Any]]
| None = None, text: TextContent | str | None = None, role: Role |
Literal['system', 'user', 'assistant', 'tool'] | dict[str, Any] | None = None,
author_name: str | None = None, response_id: str | None = None, message_id: str |
None = None, conversation_id: str | None = None, model_id: str | None = None,
created_at: str | None = None, finish_reason: FinishReason | dict[str, Any] | None
= None, additional_properties: dict[str, Any] | None = None, raw_representation:
Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
contents Required*	Sequence[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent | dict[str, Any]] | None
text Required*	TextContent | str | None
role Required*	Role | Literal['system', 'user', 'assistant', 'tool'] | dict[str, typing.Any] | None
author_name Required*	str | None
response_id Required*	str | None
message_id Required*	str | None
conversation_id Required*	str | None
model_id Required*	str | None
created_at Required*	str | None
finish_reason Required*	FinishReason | dict[str, Any] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
contents	Optional list of BaseContent items or dicts to include in the update. Default value: None
text	Optional text content to include in the update. Default value: None
role	Optional role of the author of the response update (Role, string, or dict Default value: None
author_name	Optional name of the author of the response update. Default value: None
response_id	Optional ID of the response of which this update is a part. Default value: None
message_id	Optional ID of the message of which this update is a part. Default value: None
conversation_id	Optional identifier for the state of the conversation of which this update is a part Default value: None
model_id	Optional model ID associated with this response update. Default value: None
created_at	Optional timestamp for the chat response update. Default value: None
finish_reason	Optional finish reason for the operation. Default value: None
additional_properties	Optional additional properties associated with the chat response update. Default value: None
raw_representation	Optional raw representation of the chat response update from an underlying implementation. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

Python
`__new__(**kwargs)`

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies	MutableMapping[str, Any] | None

Name	Description
Required*	

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```
   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.

Type	Description
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None

Name	Description
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

text

Returns the concatenated text of all contents in the update.

contents

The chat response update content items.

role

The role of the author of the response update.

author_name

The name of the author of the response update.

response_id

The ID of the response of which this update is a part.

message_id

The ID of the message of which this update is a part.

conversation_id

An identifier for the state of the conversation of which this update is a part.

model_id

The model ID associated with this response update.

created_at

A timestamp for the chat response update.

finish_reason

The finish reason for the operation.

additional_properties

Any additional properties associated with the chat response update.

raw_representation

The raw representation of the chat response update from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

CheckpointStorage Class

Protocol for checkpoint storage backends.

Constructor

```python
CheckpointStorage()
```

Methods

Expand table

__init__	
__new__	
delete_checkpoint	Delete a checkpoint by ID.
list_checkpoint_ids	List checkpoint IDs. If workflow_id is provided, filter by that workflow.
list_checkpoints	List checkpoint objects. If workflow_id is provided, filter by that workflow.
load_checkpoint	Load a checkpoint by ID.
save_checkpoint	Save a checkpoint and return its ID.

__init__

```python
__init__(*args, **kwargs)
```

__new__

```python
__new__(**kwargs)
```

delete_checkpoint

Delete a checkpoint by ID.

```
Python
async delete_checkpoint(checkpoint_id: str) -> bool
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
bool	

list_checkpoint_ids

List checkpoint IDs. If workflow_id is provided, filter by that workflow.

```
Python
async list_checkpoint_ids(workflow_id: str | None = None) -> list[str]
```

Parameters

Expand table

Name	Description
workflow_id	str | None Default value: None

Returns

Expand table

Type	Description
list[str]	

list_checkpoints

List checkpoint objects. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoints(workflow_id: str | None = None) -> list[WorkflowCheckpoint]
```

Parameters

Expand table

Name	Description
workflow_id	str | None Default value: None

Returns

Expand table

Type	Description
list[WorkflowCheckpoint]	

load_checkpoint

Load a checkpoint by ID.

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
WorkflowCheckpoint | None	

save_checkpoint

Save a checkpoint and return its ID.

Python

```
async save_checkpoint(checkpoint: WorkflowCheckpoint) -> str
```

Parameters

Expand table

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

Expand table

Type	Description
str	

CitationAnnotation Class

Represents a citation annotation.

Initialize CitationAnnotation.

Constructor

Python

```
CitationAnnotation()
```

Keyword-Only Parameters

Expand table

Name	Description
title	The title of the cited content. Default value: None
url	The URL of the cited content. Default value: None
file_id	The file identifier of the cited content, if applicable. Default value: None
tool_name	The name of the tool that generated the citation, if applicable. Default value: None
snippet	A snippet of the cited content, if applicable. Default value: None
annotated_regions	A list of regions that have been annotated with this citation. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content from an underlying implementation. Default value: None
****kwargs**	Additional keyword arguments.

Examples

Python

```python
from agent_framework import CitationAnnotation, TextSpanRegion

# Create a citation annotation
citation = CitationAnnotation(
    title="Agent Framework Documentation",
    url="https://example.com/docs",
    snippet="This is a relevant excerpt...",
    annotated_regions=[TextSpanRegion(start_index=0, end_index=25)],
)
print(citation.title)  # "Agent Framework Documentation"
```

Methods

Expand table

__init__	Initialize CitationAnnotation.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize CitationAnnotation.

Python

```
__init__(*, title: str | None = None, url: str | None = None, file_id: str | None
= None, tool_name: str | None = None, snippet: str | None = None,
annotated_regions: list[TextSpanRegion] | None = None, additional_properties:
dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs:
Any) -> None
```

Parameters

Expand table

Name	Description
title Required*	str | None
url Required*	str | None
file_id Required*	str | None
tool_name Required*	str | None
snippet Required*	str | None
annotated_regions Required*	list[TextSpanRegion] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
title	The title of the cited content. Default value: None

Name	Description
`url`	The URL of the cited content. Default value: None
`file_id`	The file identifier of the cited content, if applicable. Default value: None
`tool_name`	The name of the tool that generated the citation, if applicable. Default value: None
`snippet`	A snippet of the cited content, if applicable. Default value: None
`annotated_regions`	A list of regions that have been annotated with this citation. Default value: None
`additional_properties`	Optional additional properties associated with the content. Default value: None
`raw_representation`	Optional raw representation of the content from an underlying implementation. Default value: None
`**kwargs`	Additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection

to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```python
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}
```

```
    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

type

The type of content, which is always "citation" for this class.

title

The title of the cited content.

url

The URL of the cited content.

file_id

The file identifier of the cited content, if applicable.

tool_name

The name of the tool that generated the citation, if applicable.

snippet

A snippet of the cited content, if applicable.

annotated_regions

A list of regions that have been annotated with this citation.

Python

```
annotated_regions: list[TextSpanRegion] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content from an underlying implementation.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ConcurrentBuilder Class

High-level builder for concurrent agent workflows.

- *participants([...])* accepts a list of AgentProtocol (recommended) or Executor.
- *register_participants([...])* accepts a list of factories for AgentProtocol (recommended)

  or Executor factories
- *build()* wires: dispatcher -> fan-out -> participants -> fan-in -> aggregator.
- *with_aggregator(...)* overrides the default aggregator with an Executor or callback.
- *register_aggregator(...)* accepts a factory for an Executor as custom aggregator.

Usage:

Python

```
   from agent_framework import ConcurrentBuilder

   # Minimal: use default aggregator (returns list[ChatMessage])
   workflow = ConcurrentBuilder().participants([agent1, agent2, agent3]).build()

   # With agent factories
   workflow = ConcurrentBuilder().register_participants([create_agent1, create_agent2,
create_agent3]).build()


   # Custom aggregator via callback (sync or async). The callback receives
   # list[AgentExecutorResponse] and its return value becomes the workflow's output.
   def summarize(results: list[AgentExecutorResponse]) -> str:
       return " | ".join(r.agent_run_response.messages[-1].text for r in results)


   workflow = ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_aggregator(summarize).build()


   # Custom aggregator via a factory
   class MyAggregator(Executor):
       @handler
       async def aggregate(self, results: list[AgentExecutorResponse], ctx:
WorkflowContext[Never, str]) -> None:
           await ctx.yield_output(" | ".join(r.agent_run_response.messages[-1].text
for r in results))


   workflow = (
       ConcurrentBuilder()
```

```
        .register_participants([create_agent1, create_agent2, create_agent3])
        .register_aggregator(lambda: MyAggregator(id="my_aggregator"))
        .build()
    )


    # Enable checkpoint persistence so runs can resume
    workflow = ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_checkpointing(storage).build()

    # Enable request info before aggregation
    workflow = ConcurrentBuilder().participants([agent1,
agent2]).with_request_info().build()
```

Constructor

Python

```
ConcurrentBuilder()
```

Methods

Expand table

__init__	
__new__	
build	Build and validate the concurrent workflow. Wiring pattern: • Dispatcher (internal) fans out the input to all *participants* • Fan-in collects *AgentExecutorResponse* objects from all participants • If request info is enabled, the orchestration emits a request info event with outputs from all participants before sending the outputs to the aggregator • Aggregator yields output and the workflow becomes idle. The output is either: - list[ChatMessage] (default aggregator: one user + one assistant per agent) - custom payload from the provided aggregator Example: **Python** `workflow = ConcurrentBuilder().participants([agent1,`

```
agent2]).build()
```

participants	Define the parallel participants for this concurrent workflow.
	
	Accepts AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant is wired as a parallel branch using fan-out edges from an internal dispatcher.
	
	Example:
	
	Python
	
	`wf = ConcurrentBuilder().participants([researcher_agent, marketer_agent, legal_agent]).build()`
	
	`# Mixing agent(s) and executor(s) is supported`
	`wf2 = ConcurrentBuilder().participants([researcher_agent, my_custom_executor]).build()`

register_aggregator	Define a custom aggregator for this concurrent workflow. Accepts a factory (callable) that returns an Executor instance. The executor should handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)*. Example: .. code-block:: python class MyCustomExecutor(Executor): ... wf = (ConcurrentBuilder() .register_participants([create_researcher, create_marketer, create_legal]) .register_aggregator(lambda: MyCustomExecutor(id="my_aggregator")) .build())

register_participants	Define the parallel participants for this concurrent workflow.
	
	Accepts factories (callables) that return AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant created by a factory is wired as a parallel branch using fan-out edges from an internal dispatcher.
	
	Example:
	
	Python
	
	`def create_researcher() -> ChatAgent:`
	`  return ...`
	
	`def create_marketer() -> ChatAgent:`
	`  return ...`

```
    def create_legal() -> ChatAgent:
        return ...


    class MyCustomExecutor(Executor): ...


    wf =
ConcurrentBuilder().register_participants([create_researcher,
create_marketer, create_legal]).build()

    # Mixing agent(s) and executor(s) is supported
    wf2 =
ConcurrentBuilder().register_participants([create_researcher,
MyCustomExecutor]).build()
```

with_aggregator	Override the default aggregator with an executor or a callback. • Executor: must handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)* • Callback: sync or async callable with one of the signatures: *(results: list[AgentExecutorResponse]) -> Any | None* or *(results: list[AgentExecutorResponse], ctx: WorkflowContext) -> Any | None*. If the callback returns a non-None value, it becomes the workflow's output. Example:
with_checkpointing	Enable checkpoint persistence using the provided storage backend.
with_request_info	Enable request info before aggregation in the workflow. When enabled, the workflow pauses after all parallel agents complete, emitting a RequestInfoEvent that allows the caller to review and optionally modify the combined results before aggregation. The caller provides feedback via the standard response_handler/request_info pattern. **Note** Unlike SequentialBuilder and GroupChatBuilder, ConcurrentBuilder does not support per-agent filtering since all agents run in parallel and results are collected together. The pause occurs once with all agent outputs received.

__init__

Python

```
__init__() -> None
```

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

build

Build and validate the concurrent workflow.

Wiring pattern:

- Dispatcher (internal) fans out the input to all *participants*
- Fan-in collects *AgentExecutorResponse* objects from all participants
- If request info is enabled, the orchestration emits a request info event with outputs from all participants

  before sending the outputs to the aggregator
- Aggregator yields output and the workflow becomes idle. The output is either: - list[ChatMessage] (default aggregator: one user + one assistant per agent) - custom payload from the provided aggregator

Example:

Python

```
workflow = ConcurrentBuilder().participants([agent1, agent2]).build()
```

Python

```
build() -> Workflow
```

Returns

Expand table

Type	Description
Workflow	a ready-to-run workflow instance

Exceptions

Expand table

Type	Description
ValueError	if no participants were defined

participants

Define the parallel participants for this concurrent workflow.

Accepts AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant is wired as a parallel branch using fan-out edges from an internal dispatcher.

Example:

Python

```
   wf = ConcurrentBuilder().participants([researcher_agent, marketer_agent,
legal_agent]).build()

   # Mixing agent(s) and executor(s) is supported
   wf2 = ConcurrentBuilder().participants([researcher_agent,
my_custom_executor]).build()
```

Python

```
participants(participants: Sequence[AgentProtocol | Executor]) ->
ConcurrentBuilder
```

Parameters

Expand table

Name	Description
participants Required*	Sequence[AgentProtocol | Executor] Sequence of AgentProtocol or Executor instances

Returns

Expand table

Type	Description
ConcurrentBuilder	

Exceptions

Expand table

Type	Description
ValueError	if *participants* is empty, contains duplicates, or *.register_participants()* or *.participants()* were already called
TypeError	if any entry is not AgentProtocol or Executor

register_aggregator

Define a custom aggregator for this concurrent workflow.

Accepts a factory (callable) that returns an Executor instance. The executor should handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)*.

Example: .. code-block:: python

class MyCustomExecutor(Executor): ...

wf = (ConcurrentBuilder() .register_participants([create_researcher, create_marketer, create_legal]) .register_aggregator(lambda: MyCustomExecutor(id="my_aggregator")) .build()

)

Python

```
register_aggregator(aggregator_factory: Callable[[], Executor]) ->
ConcurrentBuilder
```

Parameters

Expand table

Name	Description
aggregator_factory Required*	Callable[[], Executor] Callable that returns an Executor instance

Returns

Expand table

Type	Description
ConcurrentBuilder	

register_participants

Define the parallel participants for this concurrent workflow.

Accepts factories (callables) that return AgentProtocol instances (e.g., created by a chat client) or Executor instances. Each participant created by a factory is wired as a parallel branch using fan-out edges from an internal dispatcher.

Example:

Python

```python
def create_researcher() -> ChatAgent:
    return ...


def create_marketer() -> ChatAgent:
    return ...


def create_legal() -> ChatAgent:
    return ...


class MyCustomExecutor(Executor): ...
```

```
   wf = ConcurrentBuilder().register_participants([create_researcher,
create_marketer, create_legal]).build()

   # Mixing agent(s) and executor(s) is supported
   wf2 = ConcurrentBuilder().register_participants([create_researcher,
MyCustomExecutor]).build()
```

Python

```
register_participants(participant_factories: Sequence[Callable[[], AgentProtocol |
Executor]]) -> ConcurrentBuilder
```

Parameters

Expand table

Name	Description
participant_factories Required*	Sequence[Callable[[], AgentProtocol | Executor]] Sequence of callables returning AgentProtocol or Executor instances

Returns

Expand table

Type	Description
ConcurrentBuilder	

Exceptions

Expand table

Type	Description
ValueError	if *participant_factories* is empty or *.participants()* or *.register_participants()* were already called

with_aggregator

Override the default aggregator with an executor or a callback.

- Executor: must handle *list[AgentExecutorResponse]* and yield output using *ctx.yield_output(...)*
- Callback: sync or async callable with one of the signatures: *(results: list[AgentExecutorResponse]) -> Any | None* or *(results: list[AgentExecutorResponse], ctx: WorkflowContext) -> Any | None*. If the callback returns a non-None value, it becomes the workflow's output.

Example:

Python

```
with_aggregator(aggregator: Executor | Callable[[list[AgentExecutorResponse]], Any] | Callable[[list[AgentExecutorResponse], WorkflowContext[Never, Any]], Any]) -> ConcurrentBuilder
```

Parameters

Expand table

Name	Description
aggregator Required*	Executor | Callable[[list[AgentExecutorResponse]], Any] | Callable[[list[AgentExecutorResponse], WorkflowContext[Never, Any]], Any] Executor instance, or callback function

Returns

Expand table

Type	Description
ConcurrentBuilder	

with_checkpointing

Enable checkpoint persistence using the provided storage backend.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> ConcurrentBuilder
```

Parameters

Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage CheckpointStorage instance for persisting workflow state

Returns

Expand table

Type	Description
ConcurrentBuilder	

with_request_info

Enable request info before aggregation in the workflow.

When enabled, the workflow pauses after all parallel agents complete, emitting a RequestInfoEvent that allows the caller to review and optionally modify the combined results before aggregation. The caller provides feedback via the standard response_handler/request_info pattern.

> **Note**
>
> Unlike SequentialBuilder and GroupChatBuilder, ConcurrentBuilder does not
>
> support per-agent filtering since all agents run in parallel and results
>
> are collected together. The pause occurs once with all agent outputs received.

Python

```
with_request_info() -> ConcurrentBuilder
```

Returns

Expand table

Type	Description
<xref:self>	The builder instance for fluent chaining.

Context Class

A class containing any context that should be provided to the AI model as supplied by a ContextProvider.

Each ContextProvider has the ability to provide its own context for each invocation. The Context class contains the additional context supplied by the ContextProvider. This context will be combined with context supplied by other providers before being passed to the AI model. This context is per invocation, and will not be stored as part of the chat history.

Create a new Context object.

Constructor

Python

```
Context()
```

Parameters

Expand table

Name	Description
instructions	The instructions to provide to the AI model. Default value: None
messages	The list of messages to include in the context. Default value: None
tools	The list of tools to provide to this run. Default value: None

Examples

Python

```
from agent_framework import Context, ChatMessage

# Create context with instructions
context = Context(
    instructions="Use a professional tone when responding.",
    messages=[ChatMessage(content="Previous context", role="user")],
```

```
    tools=[my_tool],
)

# Access context properties
print(context.instructions)
print(len(context.messages))
```

Methods

Expand table

__init__	Create a new Context object.
__new__	

__init__

Create a new Context object.

Python

```
__init__(instructions: str | None = None, messages: Sequence[ChatMessage] | None = None, tools: Sequence[ToolProtocol] | None = None)
```

Parameters

Expand table

Name	Description
instructions	str | None The instructions to provide to the AI model. Default value: None
messages	Sequence[ChatMessage] | None The list of messages to include in the context. Default value: None
tools	Sequence[ToolProtocol] | None The list of tools to provide to this run. Default value: None

__new__

Python

```
__new__(**kwargs)
```

ContextProvider Class

Base class for all context providers.

A context provider is a component that can be used to enhance the AI's context management. It can listen to changes in the conversation and provide additional context to the AI model just before invocation.

> ⓘ Note
>
> ContextProvider is an abstract base class. You must subclass it and implement
>
> the invoking() method to create a custom context provider. Ideally, you should
>
> also implement the invoked() and thread_created() methods to track conversation
>
> state, but these are optional.

Constructor

Python

```
ContextProvider()
```

Examples

Python

```
from agent_framework import ContextProvider, Context, ChatMessage


class CustomContextProvider(ContextProvider):
    async def invoking(self, messages, **kwargs):
        # Add custom instructions before each invocation
        return Context(instructions="Always be concise and helpful.", messages=[],
tools=[])


# Use with a chat agent
async with CustomContextProvider() as provider:
    agent = ChatAgent(chat_client=client, name="assistant",
context_providers=provider)
```

Methods

Expand table

__init__	
__new__	
invoked	Called after the agent has received a response from the underlying inference service. You can inspect the request and response messages, and update the state of the context provider.
invoking	Called just before the model/agent is invoked. Implementers can load any additional context required at this time, and they should return any context that should be passed to the agent.
thread_created	Called just after a new thread is created. Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

__init__

Python

```
__init__()
```

__new__

Python

```
__new__(**kwargs)
```

invoked

Called after the agent has received a response from the underlying inference service.

You can inspect the request and response messages, and update the state of the context provider.

Python

```
async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
invoke_exception: Exception | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
request_messages Required*	ChatMessage | Sequence[ChatMessage] The messages that were sent to the model/agent.
response_messages	ChatMessage | Sequence[ChatMessage] | None The messages that were returned by the model/agent. Default value: None
invoke_exception	Exception | None The exception that was thrown, if any. Default value: None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments (not used at present).

Returns

Expand table

Type	Description
None	

invoking

Called just before the model/agent is invoked.

Implementers can load any additional context required at this time, and they should return any context that should be passed to the agent.

Python

```
abstract async invoking(messages: ChatMessage | MutableSequence[ChatMessage],
**kwargs: Any) -> Context
```

Parameters

Expand table

Name	Description
messages Required*	ChatMessage | MutableSequence[ChatMessage] The most recent messages that the agent is being invoked with.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments (not used at present).

Returns

Expand table

Type	Description
Context	A Context object containing instructions, messages, and tools to include.

thread_created

Called just after a new thread is created.

Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

Python

```
async thread_created(thread_id: str | None) -> None
```

Parameters

Expand table

Name	Description
thread_id Required*	str | None The ID of the new thread.

Returns

Expand table

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories when answering user questions:'
```

DataContent Class

Represents binary data content with an associated media type (also known as a MIME type).

> **ⓘ Important**
>
> This is for binary data that is represented as a data URI, not for online resources.
>
> Use UriContent for online resources.

Initializes a DataContent instance.

> **ⓘ Important**
>
> This is for binary data that is represented as a data URI, not for online resources.
>
> Use UriContent for online resources.

Constructor

Python

```
DataContent()
```

Keyword-Only Parameters

Expand table

Name	Description
`uri`	The URI of the data represented by this instance. Should be in the form: "data:{media_type};base64,{base64_data}". Default value: None
`data`	The binary data or base64-encoded string represented by this instance. If bytes, the data is transformed into a base64-encoded data URI. If str, it is assumed to be already base64-encoded and used directly. Default value: None
`media_type`	The media type of the data. Default value: None

Name	Description
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import DataContent

# Create from binary data
image_data = b"raw image bytes"
data_content = DataContent(data=image_data, media_type="image/png")

# Create from base64-encoded string
base64_string = "iVBORw0KGgoAAAANS..."
data_content = DataContent(data=base64_string, media_type="image/png")

# Create from data URI
data_uri = "..."
data_content = DataContent(uri=data_uri)

# Check media type
if data_content.has_top_level_media_type("image"):
    print("This is an image")
```

Methods

Expand table

__init__	Initializes a DataContent instance.

> **Important**
>
> This is for binary data that is represented as a data URI, not for online resources.

	Use UriContent for online resources.
__new__	
create_data_uri_from_base64	Create a data URI and media type from base64 image data.
detect_image_format_from_base64	Detect image format from base64 data by examining the binary header.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_data_bytes	Extracts and returns the binary data from the data URI.
get_data_bytes_as_str	Extracts and returns the base64-encoded data from the data URI.
has_top_level_media_type	
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a DataContent instance.

> **ⓘ Important**
>
> This is for binary data that is represented as a data URI, not for online resources.

Use UriContent for online resources.

Python

```
__init__(*, uri: str, annotations: Sequence[CitationAnnotation |
MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] |
None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Keyword-Only Parameters

Expand table

Name	Description
uri	The URI of the data represented by this instance. Should be in the form: "data:{media_type};base64,{base64_data}". Default value: None
data	The binary data or base64-encoded string represented by this instance. If bytes, the data is transformed into a base64-encoded data URI. If str, it is assumed to be already base64-encoded and used directly. Default value: None
media_type	The media type of the data. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

__new__

Python

```
__new__(**kwargs)
```

create_data_uri_from_base64

Create a data URI and media type from base64 image data.

Python

```
static create_data_uri_from_base64(image_base64: str) -> tuple[str, str]
```

Parameters

Expand table

Name	Description
image_base64 Required*	str Base64 encoded image data

Returns

Expand table

Type	Description
tuple[str, str]	Tuple of (data_uri, media_type)

detect_image_format_from_base64

Detect image format from base64 data by examining the binary header.

Python

```
static detect_image_format_from_base64(image_base64: str) -> str
```

Parameters

Expand table

Name	Description
image_base64 Required*	str Base64 encoded image data

Returns

Expand table

Type	Description
str	Image format as string (png, jpeg, webp, gif) with png as fallback

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}
```

```
   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
```

```
        "messages": [{"role": "user", "text": "Hello"}],
        "is_streaming": False,
        "metadata": {"session_id": "abc123"},
        # agent and result are excluded from serialization
    }

    # Inject agent and result during middleware processing
    my_agent = BaseAgent(name="test-agent")
    dependencies = {
        "agent_run_context": {
            "agent": my_agent,
            "result": None,  # Will be populated during execution
        }
    }

    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_data_bytes

Extracts and returns the binary data from the data URI.

Python

```
get_data_bytes() -> bytes
```

Returns

Expand table

Type	Description
bytes	The binary data as bytes.

get_data_bytes_as_str

Extracts and returns the base64-encoded data from the data URI.

Python

```
get_data_bytes_as_str() -> str
```

Returns

Expand table

Type	Description
str	The binary data as str.

has_top_level_media_type

Python

```
has_top_level_media_type(top_level_media_type: Literal['application', 'audio', 'image', 'text']) -> bool
```

Parameters

Expand table

Name	Description
top_level_media_type Required*	Literal['application', 'audio', 'image', 'text']

Returns

Expand table

Type	Description
bool	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Name	Description
**kwargs	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

uri

The URI of the data represented by this instance, typically in the form of a data URI. Should be in the form: "data:{media_type};base64,{base64_data}".

media_type

The media type of the data.

type

The type of content, which is always "data" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

Default Class

Runtime representation of the default branch in a switch-case group.

The default branch is invoked only when no other case predicates match. In practice it is guaranteed to exist so that routing never produces an empty target.

Constructor

Python

```
Default()
```

Parameters

Expand table

Name	Description
target Required*	

Examples

Python

```
class DeadLetterExecutor(Executor):
    def __init__(self) -> None:
        super().__init__(id="dead_letter", defer_discovery=True)


fallback = Default(target=DeadLetterExecutor())
assert fallback.target.id == "dead_letter"
```

Methods

Expand table

__init__
__new__

__init__

```
Python
__init__(target: Executor | str) -> None
```

Parameters

Expand table

Name	Description
target Required*	Executor | str

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

Attributes

target

```
Python
target: Executor | str
```

Edge Class

Model a directed, optionally-conditional hand-off between two executors.

Each *Edge* captures the minimal metadata required to move a message from one executor to another inside the workflow graph. It optionally embeds a boolean predicate that decides if the edge should be taken at runtime. By serialising the edge down to primitives we can reconstruct the topology of a workflow irrespective of the original Python process.

Initialize a fully-specified edge between two workflow executors.

Constructor

Python

```
Edge()
```

Parameters

Expand table

Name	Description
source_id Required*	str Canonical identifier of the upstream executor instance.
target_id Required*	str Canonical identifier of the downstream executor instance.
condition	Optional predicate that receives the message payload and returns *True* when the edge should be traversed. When omitted, the edge is considered unconditionally active. Default value: None
condition_name Required*	<xref:<xref:agent_framework.str | None> > Optional override that pins a human-friendly name for the condition when the callable cannot be introspected (for example after deserialization).
Examples Required*	**Python** `edge = Edge("fetch", "parse", condition=lambda data: data.is_valid)`

Name	Description

	``
	    assert edge.source_id == "fetch"
	    assert edge.target_id == "parse"
	``

Keyword-Only Parameters

Expand table

Name	Description
condition_name	Default value: None

Examples

Python

```
   edge = Edge(source_id="ingest", target_id="score", condition=lambda payload:
payload["ready"])
   assert edge.should_route({"ready": True}) is True
   assert edge.should_route({"ready": False}) is False
```

Methods

Expand table

__init__	Initialize a fully-specified edge between two workflow executors.
__new__	
clone	
from_dict	Reconstruct an *Edge* from its serialised dictionary form. The deserialised edge will lack the executable predicate because we do not attempt to hydrate Python callables from storage. Instead, the stored *condition_name* is preserved so that downstream consumers can detect missing callables and re-register them where appropriate.
from_json	
should_route	Evaluate the edge predicate against an incoming payload. When the edge was defined without an explicit predicate the method returns *True*, signalling an unconditional routing rule. Otherwise the user-supplied callable decides whether the

	message should proceed along this edge. Any exception raised by the callable is deliberately allowed to surface to the caller to avoid masking logic bugs.
to_dict	Produce a JSON-serialisable view of the edge metadata. The representation includes the source and target executor identifiers plus the condition name when it is known. Serialisation intentionally omits the live callable to keep payloads transport-friendly.
to_json	

__init__

Initialize a fully-specified edge between two workflow executors.

Python

```
__init__(source_id: str, target_id: str, condition: Callable[[Any], bool] | None = None, *, condition_name: str | None = None) -> None
```

Parameters

Expand table

Name	Description
source_id Required*	str Canonical identifier of the upstream executor instance.
target_id Required*	str Canonical identifier of the downstream executor instance.
condition	Callable[[Any], bool] | None Optional predicate that receives the message payload and returns *True* when the edge should be traversed. When omitted, the edge is considered unconditionally active. Default value: None
condition_name Required*	str | None Optional override that pins a human-friendly name for the condition when the callable cannot be introspected (for example after deserialization).
Examples Required*	**Python** `edge = Edge("fetch", "parse", condition=lambda data: data.is_valid)`

Name	Description
	``` assert edge.source_id == "fetch" assert edge.target_id == "parse" ```

Keyword-Only Parameters

Expand table

Name	Description
condition_name	Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
__new__(**kwargs)
```

clone

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Reconstruct an *Edge* from its serialised dictionary form.

The deserialised edge will lack the executable predicate because we do not attempt to hydrate Python callables from storage. Instead, the stored *condition_name* is preserved so that downstream consumers can detect missing callables and re-register them where appropriate.

Python

```
from_dict(data: dict[str, Any]) -> Edge
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
Edge	

Examples

Python

```
   payload = {"source_id": "reader", "target_id": "writer", "condition_name":
"is_ready"}
   edge = Edge.from_dict(payload)
   assert edge.source_id == "reader"
   assert edge.condition_name == "is_ready"
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

should_route

Evaluate the edge predicate against an incoming payload.

When the edge was defined without an explicit predicate the method returns *True*, signalling an unconditional routing rule. Otherwise the user-supplied callable decides whether the message should proceed along this edge. Any exception raised by the callable is deliberately allowed to surface to the caller to avoid masking logic bugs.

Python

```
should_route(data: Any) -> bool
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
bool	

Examples

Python

```
   edge = Edge("stage1", "stage2", condition=lambda payload: payload["score"] >
0.8)
   assert edge.should_route({"score": 0.9}) is True
   assert edge.should_route({"score": 0.4}) is False
```

to_dict

Produce a JSON-serialisable view of the edge metadata.

The representation includes the source and target executor identifiers plus the condition name when it is known. Serialisation intentionally omits the live callable to keep payloads transport-friendly.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

```
Python
```

```
   edge = Edge("reader", "writer", condition=lambda payload: payload["ok"])
   snapshot = edge.to_dict()
   assert snapshot == {"source_id": "reader", "target_id": "writer",
"condition_name": "<lambda>"}
```

to_json

```
Python
```

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

id

Return the stable identifier used to reference this edge.

The identifier combines the source and target executor identifiers with a deterministic separator. This allows other graph structures such as adjacency lists or visualisations to refer to an edge without carrying the full object.

Examples

```
Python
```

```
   edge = Edge("reader", "writer")
```

```
    assert edge.id == "reader->writer"
```

ID_SEPARATOR

Python

```
ID_SEPARATOR: ClassVar[str] = '->'
```

condition_name

Python

```
condition_name: str | None
```

source_id

Python

```
source_id: str
```

target_id

Python

```
target_id: str
```

EdgeDuplicationError Class

Exception raised when duplicate edges are detected in the workflow.

Constructor

```
Python
EdgeDuplicationError()
```

Parameters

Expand table

Name	Description
edge_id Required*	

Methods

Expand table

__init__	
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

```
Python
__init__(edge_id: str)
```

Parameters

Expand table

Name	Description
edge_id Required*	str

__new__

```
Python
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

```
Python
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```
Python
with_traceback()
```

Attributes

args

ErrorContent Class

Represents an error.

Remarks: Typically used for non-fatal errors, where something went wrong as part of the operation, but the operation was still able to continue.

Initializes an ErrorContent instance.

Constructor

Python

```
ErrorContent()
```

Keyword-Only Parameters

Expand table

Name	Description
message	The error message. Default value: None
error_code	The error code associated with the error. Default value: None
details	Additional details about the error. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import ErrorContent

# Create an error content
error = ErrorContent(
    message="Failed to process request",
    error_code="PROCESSING_ERROR",
    details="The input format was invalid",
)
print(str(error))  # "Error PROCESSING_ERROR: Failed to process request"

# Error without code
simple_error = ErrorContent(message="Something went wrong")
print(str(simple_error))  # "Something went wrong"
```

Methods

Expand table

__init__	Initializes an ErrorContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes an ErrorContent instance.

Python

```
__init__(*, message: str | None = None, error_code: str | None = None, details:
str | None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str,
Any]] | None = None, additional_properties: dict[str, Any] | None = None,
raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
message Required*	str | None
error_code Required*	str | None
details Required*	str | None
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
message	The error message. Default value: None
error_code	The error code associated with the error. Default value: None
details	Additional details about the error. Default value: None

Name	Description
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

error_code

The error code associated with the error.

details

Additional details about the error.

message

The error message.

type

The type of content, which is always "error" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

Executor Class

Base class for all workflow executors that process messages and perform computations.

Overview

Executors are the fundamental building blocks of workflows, representing individual processing units that receive messages, perform operations, and produce outputs. Each executor is uniquely identified and can handle specific message types through decorated handler methods.

Type System

Executors have a rich type system that defines their capabilities:

Input Types

The types of messages an executor can process, discovered from handler method signatures:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_string(self, message: str, ctx: WorkflowContext) -> None:
        # This executor can handle 'str' input types
```

Access via the *input_types* property.

Output Types

The types of messages an executor can send to other executors via *ctx.send_message()*:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_data(self, message: str, ctx: WorkflowContext[int | bool]) ->
None:
        # This executor can send 'int' or 'bool' messages
```

Access via the *output_types* property.

Workflow Output Types

The types of data an executor can emit as workflow-level outputs via *ctx.yield_output()*:

Python

```
class MyExecutor(Executor):
    @handler
    async def process(self, message: str, ctx: WorkflowContext[int, str]) -> None:
        # Can send 'int' messages AND yield 'str' workflow outputs
```

Access via the *workflow_output_types* property.

Handler Discovery

Executors discover their capabilities through decorated methods:

@handler Decorator

Marks methods that process incoming messages:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_text(self, message: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(message.upper())
```

Sub-workflow Request Interception

Use @handler methods to intercept sub-workflow requests:

Python

```
class ParentExecutor(Executor):
    @handler
    async def handle_subworkflow_request(
        self,
        request: SubWorkflowRequestMessage,
        ctx: WorkflowContext[SubWorkflowResponseMessage],
    ) -> None:
        if self.is_allowed(request.domain):
            response = request.create_response(data=True)
```

```
                await ctx.send_message(response, target_id=request.executor_id)
            else:
                await ctx.request_info(request.source_event,
response_type=request.source_event.response_type)
```

Context Types

Handler methods receive different WorkflowContext variants based on their type annotations:

WorkflowContext (no type parameters)

For handlers that only perform side effects without sending messages or yielding outputs:

Python

```python
class LoggingExecutor(Executor):
    @handler
    async def log_message(self, msg: str, ctx: WorkflowContext) -> None:
        print(f"Received: {msg}")  # Only logging, no outputs
```

WorkflowContext[T_Out]

Enables sending messages of type T_Out via *ctx.send_message()*:

Python

```python
class ProcessorExecutor(Executor):
    @handler
    async def handler(self, msg: str, ctx: WorkflowContext[int]) -> None:
        await ctx.send_message(42)  # Can send int messages
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```python
class DualOutputExecutor(Executor):
    @handler
    async def handler(self, msg: str, ctx: WorkflowContext[int, str]) -> None:
        await ctx.send_message(42)  # Send int message
        await ctx.yield_output("done")  # Yield str workflow output
```

Function Executors

Simple functions can be converted to executors using the *@executor* decorator:

Python

```python
@executor
async def process_text(text: str, ctx: WorkflowContext[str]) -> None:
    await ctx.send_message(text.upper())


# Or with custom ID:
@executor(id="text_processor")
def sync_process(text: str, ctx: WorkflowContext[str]) -> None:
    ctx.send_message(text.lower())  # Sync functions run in thread pool
```

Sub-workflow Composition

Executors can contain sub-workflows using WorkflowExecutor. Sub-workflows can make requests that parent workflows can intercept. See WorkflowExecutor documentation for details on workflow composition patterns and request/response handling.

State Management

Executors can contain states that persist across workflow runs and checkpoints. Override the *on_checkpoint_save* and *on_checkpoint_restore* methods to implement custom state serialization and restoration logic.

Implementation Notes

- Do not call *execute()* directly - it's invoked by the workflow engine
- Do not override *execute()* - define handlers using decorators instead
- Each executor must have at least one *@handler* method
- Handler method signatures are validated at initialization time

Initialize the executor with a unique identifier.

Constructor

Python

```
Executor()
```

Parameters

Expand table

Name	Description
id Required*	A unique identifier for the executor.

Keyword-Only Parameters

Expand table

Name	Description
type	The executor type name. If not provided, uses class name. Default value: None
type	Alternative parameter name for executor type.
defer_discovery	If True, defer handler method discovery until later. Default value: False
****_**	Additional keyword arguments. Unused in this implementation.
type_	Default value: None

Methods

Expand table

__init__	Initialize the executor with a unique identifier.
__new__	
can_handle	Check if the executor can handle a given message type.
clone	
execute	Execute the executor with a given message and context parameters. • Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using @handler decorator.

from_dict	
from_json	
is_request_supported	Check if the executor supports request of the given type and handling a response of the given type.
on_checkpoint_restore	Hook called when the workflow is restored from a checkpoint. Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.
on_checkpoint_save	Hook called when the workflow is being saved to a checkpoint. Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint. The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.
to_dict	Serialize executor definition for workflow topology export.
to_json	

__init__

Initialize the executor with a unique identifier.

Python

```
__init__(id: str, *, type: str | None = None, type_: str | None = None,
defer_discovery: bool = False, **_: Any) -> None
```

Parameters

Expand table

Name	Description
id Required*	str A unique identifier for the executor.
type Required*	str | None
type_ Required*	str | None

Name	Description
defer_discovery Required*	bool
_ Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
type	The executor type name. If not provided, uses class name. Default value: None
type	Alternative parameter name for executor type.
defer_discovery	If True, defer handler method discovery until later. Default value: False
****_**	Additional keyword arguments. Unused in this implementation.
type_	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

can_handle

Check if the executor can handle a given message type.

Python

```
can_handle(message: Message) -> bool
```

Parameters

Expand table

Name	Description
message Required*	Message The message to check.

Returns

Expand table

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state: SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] | None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

Expand table

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] | None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] | None Optional source span IDs from multiple sources for linking. Default value: None

Returns

Expand table

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python
`from_dict(data: dict[str, Any]) -> TModel`

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_json

Python
`from_json(raw: str) -> TModel`

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

Expand table

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

Expand table

Type	Description
bool	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Hook called when the workflow is restored from a checkpoint.

Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any] The state dictionary that was saved during checkpointing.

Returns

Expand table

Type	Description
None	

on_checkpoint_save

Hook called when the workflow is being saved to a checkpoint.

Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	A state dictionary to be saved during checkpointing.

to_dict

Serialize executor definition for workflow topology export.

Python
`to_dict() -> dict[str, Any]`

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

Python
`to_json() -> str`

Returns

Expand table

Type	Description
str	

Attributes

input_types

Get the list of input types that this executor can handle.

Returns

Expand table

Type	Description
	A list of the message types that this executor's handlers can process.

output_types

Get the list of output types that this executor can produce via send_message().

Returns

Expand table

Type	Description
	A list of the output types inferred from the handlers' WorkflowContext[T] annotations.

workflow_output_types

Get the list of workflow output types that this executor can produce via yield_output().

Returns

Expand table

Type	Description
	A list of the workflow output types inferred from handlers' WorkflowContext[T, U] annotations.

ExecutorCompletedEvent Class

Event triggered when an executor handler is completed.

Initialize the executor event with an executor ID and optional data.

Constructor

Python

```
ExecutorCompletedEvent()
```

Parameters

Expand table

Name	Description
executor_id Required*	
data	Default value: None

Methods

Expand table

__init__	Initialize the executor event with an executor ID and optional data.
__new__	

__init__

Initialize the executor event with an executor ID and optional data.

Python

```
__init__(executor_id: str, data: Any | None = None)
```

Parameters

Expand table

Name	Description
executor_id Required*	str
data	Any | None Default value: None

__new__

Python
`__new__(**kwargs)`

ExecutorEvent Class

Base class for executor events.

Initialize the executor event with an executor ID and optional data.

Constructor

Python

```
ExecutorEvent()
```

Parameters

Expand table

Name	Description
executor_id Required*	
data	Default value: None

Methods

Expand table

__init__	Initialize the executor event with an executor ID and optional data.
__new__	

__init__

Initialize the executor event with an executor ID and optional data.

Python

```
__init__(executor_id: str, data: Any | None = None)
```

Parameters

Expand table

Name	Description
executor_id Required*	str
data	Any | None Default value: None

__new__

Python
`__new__(**kwargs)`

ExecutorFailedEvent Class

Event triggered when an executor handler raises an error.

Constructor

Python

```
ExecutorFailedEvent()
```

Parameters

Expand table

Name	Description
executor_id Required*	
details Required*	

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(executor_id: str, details: WorkflowErrorDetails)
```

Parameters

Expand table

Name	Description
executor_id Required*	str
details Required*	WorkflowErrorDetails

__new__

Python

```
__new__(**kwargs)
```

ExecutorInvokedEvent Class

Event triggered when an executor handler is invoked.

Initialize the executor event with an executor ID and optional data.

Constructor

Python

```
ExecutorInvokedEvent()
```

Parameters

Expand table

Name	Description
executor_id Required*	
data	Default value: None

Methods

Expand table

__init__	Initialize the executor event with an executor ID and optional data.
__new__	

__init__

Initialize the executor event with an executor ID and optional data.

Python

```
__init__(executor_id: str, data: Any | None = None)
```

Parameters

Expand table

Name	Description
executor_id Required*	str
data	Any | None Default value: None

__new__

Python
`__new__(**kwargs)`

FanInEdgeGroup Class

Represent a converging set of edges that feed a single downstream executor.

Fan-in groups are typically used when multiple upstream stages independently produce messages that should all arrive at the same downstream processor.

Build a fan-in mapping that merges several sources into one target.

Constructor

Python

```
FanInEdgeGroup()
```

Parameters

Expand table

Name	Description
source_ids Required*	Sequence of upstream executor identifiers contributing messages.
target_id Required*	Downstream executor that receives every message emitted by the sources.
id Required*	str Optional explicit identifier for the edge group.

Examples
Required*
	Python
	`group = FanInEdgeGroup(["parser", "enricher"], target_id="writer")`
	`assert group.to_dict()["edges"][0]["target_id"] == "writer"`

Keyword-Only Parameters

Expand table

Name	Description
`id`	Default value: None

Methods

Expand table

__init__	Build a fan-in mapping that merges several sources into one target.
__new__	
clone	
from_dict	Hydrate the correct *EdgeGroup* subclass from serialised state. The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
from_json	
register	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.
to_dict	Serialise the group metadata and contained edges into primitives. The payload captures each edge through its own *to_dict* call, enabling round-tripping through formats such as JSON without leaking Python objects.
to_json	

__init__

Build a fan-in mapping that merges several sources into one target.

Python

```
__init__(source_ids: Sequence[str], target_id: str, *, id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
source_ids Required*	Sequence[str] Sequence of upstream executor identifiers contributing messages.
target_id Required*	str Downstream executor that receives every message emitted by the sources.
id Required*	str | None Optional explicit identifier for the edge group.

Examples
Required*
	Python
	`group = FanInEdgeGroup(["parser", "enricher"], target_id="writer")`
	`assert group.to_dict()["edges"][0]["target_id"] == "writer"`

Keyword-Only Parameters

Expand table

Name	Description
id	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._edge.EdgeGroup>	

Examples

Python

```
   payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id":
"b"}]}
   group = EdgeGroup.from_dict(payload)
   assert isinstance(group, EdgeGroup)
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python

```
register(subclass: type[EdgeGroup]) -> type[EdgeGroup]
```

Parameters

Expand table

Name	Description
subclass Required*	type[<xref:agent_framework._workflows._edge.EdgeGroup>]

Returns

Expand table

Type	Description
type[<xref:agent_framework._workflows._edge.EdgeGroup>]	

Examples

Python

```
@EdgeGroup.register
class CustomGroup(EdgeGroup):
    pass


assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup
```

to_dict

Serialise the group metadata and contained edges into primitives.

The payload captures each edge through its own *to_dict* call, enabling round-tripping through formats such as JSON without leaking Python objects.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

Python

```
group = EdgeGroup([Edge("read", "write")])
snapshot = group.to_dict()
assert snapshot["edges"][0]["source_id"] == "read"
```

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

type

Python

```
type: str
```

FanOutEdgeGroup Class

Represent a broadcast-style edge group with optional selection logic.

A fan-out forwards a message produced by a single source executor to one or more downstream executors. At runtime we may further narrow the targets by executing a *selection_func* that inspects the payload and returns the subset of ids that should receive the message.

Create a fan-out mapping from a single source to many targets.

Constructor

Python

```
FanOutEdgeGroup()
```

Parameters

Expand table

Name	Description
source_id Required*	Identifier of the upstream executor broadcasting the message.
target_ids Required*	Ordered set of downstream executor identifiers that may receive the message. At least two targets are required to preserve the fan-out semantics.
selection_func	Optional callable that returns the subset of *target_ids* that should be active for a given payload. The callable receives the original message plus a copy of all configured target ids. Default value: None
selection_func_name Required*	<xref:<xref:agent_framework.str | None>> Static identifier used when persisting the fan-out. Needed when the callable cannot be introspected or is unavailable during deserialisation.
id Required*	str Stable identifier for the group; defaults to an autogenerated UUID.
Examples Required*	Python `def choose_targets(message: dict[str, Any], available:`

Name	Description
	list[str]) -> list[str]: return [target for target in available if message.get(target)] ```` group = FanOutEdgeGroup("sensor", ["db", "cache"], selection_func=choose_targets) assert group.selection_func is choose_targets

Keyword-Only Parameters

Expand table

Name	Description
selection_func_name	Default value: None
id	Default value: None

Methods

Expand table

__init__	Create a fan-out mapping from a single source to many targets.
__new__	
clone	
from_dict	Hydrate the correct *EdgeGroup* subclass from serialised state. The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
from_json	
register	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.
to_dict	Serialise the fan-out group while preserving selection metadata. In addition to the base *EdgeGroup* payload we embed the human-friendly name of the selection function. The callable itself is not persisted.

to_json

__init__

Create a fan-out mapping from a single source to many targets.

Python

```
__init__(source_id: str, target_ids: Sequence[str], selection_func: Callable[[Any, list[str]], list[str]] | None = None, *, selection_func_name: str | None = None, id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
source_id Required*	str Identifier of the upstream executor broadcasting the message.
target_ids Required*	Sequence[str] Ordered set of downstream executor identifiers that may receive the message. At least two targets are required to preserve the fan-out semantics.
selection_func	Callable[[Any, list[str]], list[str]] | None Optional callable that returns the subset of *target_ids* that should be active for a given payload. The callable receives the original message plus a copy of all configured target ids. Default value: None
selection_func_name Required*	str | None Static identifier used when persisting the fan-out. Needed when the callable cannot be introspected or is unavailable during deserialisation.
id Required*	str | None Stable identifier for the group; defaults to an autogenerated UUID.

Examples
Required*
	Python
	`def choose_targets(message: dict[str, Any], available: list[str]) -> list[str]:`
	`  return [target for target in available if message.get(target)]`
	``

Name	Description
	``` group = FanOutEdgeGroup("sensor", ["db", "cache"], selection_func=choose_targets) assert group.selection_func is choose_targets ```

Keyword-Only Parameters

Expand table

Name	Description
selection_func_name	Default value: None
id	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._edge.EdgeGroup>	

Examples

```
Python
```

```
   payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id":
"b"}]}
   group = EdgeGroup.from_dict(payload)
   assert isinstance(group, EdgeGroup)
```

from_json

```
Python
```

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python

```
register(subclass: type[EdgeGroup]) -> type[EdgeGroup]
```

Parameters

Expand table

Name	Description
subclass Required*	type[<xref:agent_framework._workflows._edge.EdgeGroup>]

Returns

Expand table

Type	Description
type[<xref:agent_framework._workflows._edge.EdgeGroup>]	

Examples

Python

```
@EdgeGroup.register
class CustomGroup(EdgeGroup):
    pass


assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup
```

to_dict

Serialise the fan-out group while preserving selection metadata.

In addition to the base *EdgeGroup* payload we embed the human-friendly name of the selection function. The callable itself is not persisted.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

Python

```
group = FanOutEdgeGroup("source", ["a", "b"], selection_func=lambda *_: ["a"])
snapshot = group.to_dict()
assert snapshot["selection_func_name"] == "<lambda>"
```

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

selection_func

Expose the runtime callable used to select active fan-out targets.

When no selection function was supplied the property returns *None*, signalling that all targets must receive the payload.

Examples

Python

```
group = FanOutEdgeGroup("source", ["x", "y"], selection_func=None)
```

```
assert group.selection_func is None
```

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

target_ids

Return a shallow copy of the configured downstream executor ids.

The list is defensively copied to prevent callers from mutating the internal state while still providing deterministic ordering.

Examples

Python

```
group = FanOutEdgeGroup("node", ["alpha", "beta"])
assert group.target_ids == ["alpha", "beta"]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

selection_func_name

Python

```
selection_func_name: str | None
```

type

Python

```
type: str
```

FileCheckpointStorage Class

File-based checkpoint storage for persistence.

Initialize the file storage.

Constructor

```
Python
FileCheckpointStorage()
```

Parameters

Expand table

Name	Description
storage_path Required*	

Methods

Expand table

__init__	Initialize the file storage.
__new__	
delete_checkpoint	Delete a checkpoint by ID.
list_checkpoint_ids	List checkpoint IDs. If workflow_id is provided, filter by that workflow.
list_checkpoints	List checkpoint objects. If workflow_id is provided, filter by that workflow.
load_checkpoint	Load a checkpoint by ID.
save_checkpoint	Save a checkpoint and return its ID.

__init__

Initialize the file storage.

Python

```
__init__(storage_path: str | Path)
```

Parameters

Expand table

Name	Description
storage_path Required*	str | Path

__new__

Python

```
__new__(**kwargs)
```

delete_checkpoint

Delete a checkpoint by ID.

Python

```
async delete_checkpoint(checkpoint_id: str) -> bool
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
bool	

list_checkpoint_ids

List checkpoint IDs. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoint_ids(workflow_id: str | None = None) -> list[str]
```

Parameters

Expand table

Name	Description
workflow_id	str | None Default value: None

Returns

Expand table

Type	Description
list[str]	

list_checkpoints

List checkpoint objects. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoints(workflow_id: str | None = None) -> list[WorkflowCheckpoint]
```

Parameters

Expand table

Name	Description
workflow_id	str | None Default value: None

Returns

Expand table

Type	Description
list[WorkflowCheckpoint]	

load_checkpoint

Load a checkpoint by ID.

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
WorkflowCheckpoint | None	

save_checkpoint

Save a checkpoint and return its ID.

Python

```
async save_checkpoint(checkpoint: WorkflowCheckpoint) -> str
```

Parameters

Expand table

Name	Description
`checkpoint` Required*	WorkflowCheckpoint

Returns

Expand table

Type	Description
str	

FinishReason Class

Represents the reason a chat response completed.

Initialize FinishReason with a value.

Constructor

Python

```
FinishReason()
```

Parameters

Expand table

Name	Description
value Required*	The string representation of the finish reason.

Examples

Python

```
from agent_framework import FinishReason

# Use predefined finish reason constants
stop_reason = FinishReason.STOP  # Normal completion
length_reason = FinishReason.LENGTH  # Max tokens reached
tool_calls_reason = FinishReason.TOOL_CALLS  # Tool calls triggered
filter_reason = FinishReason.CONTENT_FILTER  # Content filter triggered

# Check finish reason
if stop_reason == FinishReason.STOP:
    print("Response completed normally")
```

Methods

Expand table

__init__	Initialize FinishReason with a value.

__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize FinishReason with a value.

Python

```
__init__(value: str) -> None
```

Parameters

Expand table

Name	Description
value Required*	str The string representation of the finish reason.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```
   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

value

The string representation of the finish reason.

CONTENT_FILTER

Python

```
CONTENT_FILTER: FinishReason = FinishReason(value='content_filter')
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

LENGTH

Python

```
LENGTH: FinishReason = FinishReason(value='length')
```

STOP

Python

```
STOP: FinishReason = FinishReason(value='stop')
```

TOOL_CALLS

Python

```
TOOL_CALLS: FinishReason = FinishReason(value='tool_calls')
```

FunctionApprovalRequestContent Class

Represents a request for user approval of a function call.

Initializes a FunctionApprovalRequestContent instance.

Constructor

Python

```
FunctionApprovalRequestContent()
```

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request. Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None
****kwargs**	Additional keyword arguments.

Examples

Python

```
from agent_framework import FunctionApprovalRequestContent, FunctionCallContent

# Create a function approval request
func_call = FunctionCallContent(
    call_id="call_123",
    name="send_email",
    arguments={"to": "user@example.com", "subject": "Hello"},
```

```
)
approval_request = FunctionApprovalRequestContent(
    id="approval_001",
    function_call=func_call,
)

# Create response
approval_response = approval_request.create_response(approved=True)
print(approval_response.approved)  # True
```

Methods

Expand table

__init__	Initializes a FunctionApprovalRequestContent instance.
__new__	
create_response	Create a response for the function approval request.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a FunctionApprovalRequestContent instance.

Python

```
__init__(*, id: str, function_call: FunctionCallContent | MutableMapping[str,
Any], annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
= None, additional_properties: dict[str, Any] | None = None, raw_representation:
Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
id Required*	str
function_call Required*	FunctionCallContent | MutableMapping[str, Any]
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request. Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None

Name	Description
**kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

create_response

Create a response for the function approval request.

```
Python
create_response(approved: bool) -> FunctionApprovalResponseContent
```

Parameters

Expand table

Name	Description
approved Required*	bool

Returns

Expand table

Type	Description
FunctionApprovalResponseContent	

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}`

Name	Description
	• **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}
```

```
   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }
```

```
# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value	str

Name	Description
Required*	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

FunctionApprovalResponseContent Class

Represents a response for user approval of a function call.

Initializes a FunctionApprovalResponseContent instance.

Constructor

Python

```
FunctionApprovalResponseContent()
```

Parameters

Expand table

Name	Description
approved Required*	Whether the function call was approved.

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request. Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None
****kwargs**	Additional keyword arguments.

Examples

Python

```python
from agent_framework import FunctionApprovalResponseContent, FunctionCallContent

# Create a function approval response
func_call = FunctionCallContent(
    call_id="call_123",
    name="send_email",
    arguments={"to": "user@example.com"},
)
response = FunctionApprovalResponseContent(
    approved=False,
    id="approval_001",
    function_call=func_call,
)
print(response.approved)  # False
```

Methods

Expand table

__init__	Initializes a FunctionApprovalResponseContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a FunctionApprovalResponseContent instance.

Python

```
__init__(approved: bool, *, id: str, function_call: FunctionCallContent |
MutableMapping[str, Any], annotations: Sequence[CitationAnnotation |
MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] |
None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
approved Required*	bool Whether the function call was approved.
id Required*	str
function_call Required*	FunctionCallContent | MutableMapping[str, Any]
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	The unique identifier for the request.
function_call	The function call content to be approved. Can be a FunctionCallContent object or dict.
annotations	Optional list of annotations for the request.

Name	Description
	Default value: None
additional_properties	Optional additional properties for the request. Default value: None
raw_representation	Optional raw representation of the request. Default value: None
****kwargs**	Additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

FunctionCallContent Class

Represents a function call request.

Initializes a FunctionCallContent instance.

Constructor

Python

```
FunctionCallContent()
```

Keyword-Only Parameters

Expand table

Name	Description
call_id	The function call identifier.
name	The name of the function requested.
arguments	The arguments requested to be provided to the function, can be a string to allow gradual completion of the args. Default value: None
exception	Any exception that occurred while mapping the original function call data to this representation. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import FunctionCallContent

# Create a function call
func_call = FunctionCallContent(
    call_id="call_123",
    name="get_weather",
    arguments={"location": "Seattle", "unit": "celsius"},
)

# Parse arguments
args = func_call.parse_arguments()
print(args["location"])  # "Seattle"

# Create with string arguments (gradual completion)
func_call_partial_1 = FunctionCallContent(
    call_id="call_124",
    name="search",
    arguments='{"query": ',
)
func_call_partial_2 = FunctionCallContent(
    call_id="call_124",
    name="search",
    arguments='"latest news"}',
)
full_call = func_call_partial_1 + func_call_partial_2
args = full_call.parse_arguments()
print(args["query"])  # "latest news"
```

Methods

Expand table

__init__	Initializes a FunctionCallContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
parse_arguments	Parse the arguments into a dictionary.

	If they cannot be parsed as json or if the resulting json is not a dict, they are returned as a dictionary with a single key "raw".
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a FunctionCallContent instance.

Python

```
__init__(*, call_id: str, name: str, arguments: str | dict[str, Any | None] | None = None, exception: Exception | None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
call_id Required*	str
name Required*	str
arguments Required*	str | dict[str, Any | None] | None
exception Required*	Exception | None
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None

Name	Description
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
call_id	The function call identifier.
name	The name of the function requested.
arguments	The arguments requested to be provided to the function, can be a string to allow gradual completion of the args. Default value: None
exception	Any exception that occurred while mapping the original function call data to this representation. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}
```

```
   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
```

```
        "messages": [{"role": "user", "text": "Hello"}],
        "is_streaming": False,
        "metadata": {"session_id": "abc123"},
        # agent and result are excluded from serialization
    }

    # Inject agent and result during middleware processing
    my_agent = BaseAgent(name="test-agent")
    dependencies = {
        "agent_run_context": {
            "agent": my_agent,
            "result": None,  # Will be populated during execution
        }
    }

    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

parse_arguments

Parse the arguments into a dictionary.

If they cannot be parsed as json or if the resulting json is not a dict, they are returned as a dictionary with a single key "raw".

```
Python
parse_arguments() -> dict[str, Any | None] | None
```

Returns

Expand table

Type	Description
dict[str, Any | None] | None	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

```
Python
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

call_id

The function call identifier.

name

The name of the function requested.

arguments

The arguments requested to be provided to the function.

exception

Any exception that occurred while mapping the original function call data to this representation.

type

The type of content, which is always "function_call" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

FunctionExecutor Class

Executor that wraps a user-defined function.

This executor allows users to define simple functions (both sync and async) and use them as workflow executors without needing to create full executor classes.

Synchronous functions are executed in a thread pool using asyncio.to_thread() to avoid blocking the event loop.

Initialize the FunctionExecutor with a user-defined function.

Constructor

```
Python
FunctionExecutor()
```

Parameters

Expand table

Name	Description
func Required*	The function to wrap as an executor (can be sync or async)
id	Optional executor ID. If None, uses the function name. Default value: None

Methods

Expand table

__init__	Initialize the FunctionExecutor with a user-defined function.
__new__	
can_handle	Check if the executor can handle a given message type.
clone	
execute	Execute the executor with a given message and context parameters.

	• Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using @handler decorator.
from_dict	
from_json	
is_request_supported	Check if the executor supports request of the given type and handling a response of the given type.
on_checkpoint_restore	Hook called when the workflow is restored from a checkpoint. Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.
on_checkpoint_save	Hook called when the workflow is being saved to a checkpoint. Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint. The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.
to_dict	Serialize executor definition for workflow topology export.
to_json	

__init__

Initialize the FunctionExecutor with a user-defined function.

Python

```
__init__(func: Callable[[...], Any], id: str | None = None)
```

Parameters

Expand table

Name	Description
func Required*	Callable[[...], Any] The function to wrap as an executor (can be sync or async)
id	str | None Optional executor ID. If None, uses the function name.

Name	Description
	Default value: None

Exceptions

Expand table

Type	Description
ValueError	If func is a staticmethod or classmethod (use @handler on instance methods instead)

__new__

```
Python
__new__(**kwargs)
```

can_handle

Check if the executor can handle a given message type.

```
Python
can_handle(message: Message) -> bool
```

Parameters

Expand table

Name	Description
message Required*	Message The message to check.

Returns

Expand table

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

Expand table

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] | None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] | None Optional source span IDs from multiple sources for linking. Default value: None

Returns

Expand table

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_json

```
Python
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

```
Python
```

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

Expand table

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

Expand table

Type	Description
bool	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Hook called when the workflow is restored from a checkpoint.

Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any] The state dictionary that was saved during checkpointing.

Returns

Expand table

Type	Description
None	

on_checkpoint_save

Hook called when the workflow is being saved to a checkpoint.

Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	A state dictionary to be saved during checkpointing.

to_dict

Serialize executor definition for workflow topology export.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

```
Python
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

input_types

Get the list of input types that this executor can handle.

Returns

Expand table

Type	Description
	A list of the message types that this executor's handlers can process.

output_types

Get the list of output types that this executor can produce via send_message().

Returns

Expand table

Type	Description
	A list of the output types inferred from the handlers' WorkflowContext[T] annotations.

workflow_output_types

Get the list of workflow output types that this executor can produce via yield_output().

Returns

Expand table

Type	Description
	A list of the workflow output types inferred from handlers' WorkflowContext[T, U] annotations.

FunctionInvocationConfiguration Class

Configuration for function invocation in chat clients.

This class is created automatically on every chat client that supports function invocation. This means that for most cases you can just alter the attributes on the instance, rather then creating a new one.

Initialize FunctionInvocationConfiguration.

Constructor

Python

```
FunctionInvocationConfiguration()
```

Parameters

Expand table

Name	Description
enabled	Whether function invocation is enabled. Default value: True
max_iterations	Maximum number of function invocation iterations. Default value: 40
max_consecutive_errors_per_request	Maximum consecutive errors allowed per request. Default value: 3
terminate_on_unknown_calls	Whether to terminate on unknown function calls. Default value: False
additional_tools	Additional tools to include for function execution. Default value: None
include_detailed_errors	Whether to include detailed error information in function results. Default value: False

Methods

Expand table

__init__	Initialize FunctionInvocationConfiguration.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize FunctionInvocationConfiguration.

Python

```
__init__(enabled: bool = True, max_iterations: int = DEFAULT_MAX_ITERATIONS,
max_consecutive_errors_per_request: int =
DEFAULT_MAX_CONSECUTIVE_ERRORS_PER_REQUEST, terminate_on_unknown_calls: bool =
False, additional_tools: Sequence[ToolProtocol] | None = None,
include_detailed_errors: bool = False) -> None
```

Parameters

Expand table

Name	Description
`enabled`	bool Whether function invocation is enabled. Default value: True
`max_iterations`	int Maximum number of function invocation iterations. Default value: DEFAULT_MAX_ITERATIONS
`max_consecutive_errors_per_request`	int Maximum consecutive errors allowed per request. Default value: DEFAULT_MAX_CONSECUTIVE_ERRORS_PER_REQUEST
`terminate_on_unknown_calls`	bool Whether to terminate on unknown function calls. Default value: False
`additional_tools`	Sequence[ToolProtocol] | None Additional tools to include for function execution. Default value: None
`include_detailed_errors`	bool Whether to include detailed error information in function results. Default value: False

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}`

Name	Description
	Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

enabled

Whether function invocation is enabled. When this is set to False, the client will not attempt to invoke any functions, because the tool mode will be set to None.

max_iterations

Maximum number of function invocation iterations. Each request to this client might end up making multiple requests to the model. Each time the model responds with a function call request, this client might perform that invocation and send the results back to the model in a new request. This property limits the number of times such a roundtrip is performed. The value must be at least one, as it includes the initial request. If you want to fully disable function invocation, use the `enabled` property. The default is 40.

max_consecutive_errors_per_request

Maximum consecutive errors allowed per request. The maximum number of consecutive function call errors allowed before stopping further function calls for the request. The default is 3.

terminate_on_unknown_calls

Whether to terminate on unknown function calls. When False, call requests to any tools that aren't available to the client will result in a response message automatically being created and returned to the inner client stating that the tool couldn't be found. This behavior can help in cases where a model hallucinates a function, but it's problematic if the model has been made aware of the existence of tools outside of the normal mechanisms, and requests one of those. `additional_tools` can be used to help with that. But if instead the consumer wants to know about all function call requests that the client can't handle, this can be set to True. Upon receiving a request to call a function that the client doesn't know about, it will terminate the function calling loop and return the response, leaving the handling of the function call requests to the consumer of the client.

additional_tools

Additional tools to include for function execution. These will not impact the requests sent by the client, which will pass through the `tools` unmodified. However, if the inner client requests the invocation of a tool that was not in `ChatOptions.tools`, this `additional_tools` collection will also be consulted to look for a corresponding tool. This is useful when the service might have been pre-configured to be aware of certain tools that aren't also sent on each individual request. These tools are treated the same as `declaration_only` tools and will be returned to the user.

include_detailed_errors

Whether to include detailed error information in function results. When set to True, detailed error information such as exception type and message will be included in the function result content when a function invocation fails. When False, only a generic error message will be included.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

FunctionInvocationContext Class

Context object for function middleware invocations.

This context is passed through the function middleware pipeline and contains all information about the function invocation.

Initialize the FunctionInvocationContext.

Constructor

Python

```
FunctionInvocationContext()
```

Parameters

Expand table

Name	Description
function Required*	The function being invoked.
arguments Required*	The validated arguments for the function.
metadata	Metadata dictionary for sharing data between function middleware. Default value: None
result	Function execution result. Default value: None
terminate	A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	Additional keyword arguments passed to the chat method that invoked this function. Default value: None

Examples

Python

```
from agent_framework import FunctionMiddleware, FunctionInvocationContext
```

```python
class ValidationMiddleware(FunctionMiddleware):
    async def process(self, context: FunctionInvocationContext, next):
        print(f"Function: {context.function.name}")
        print(f"Arguments: {context.arguments}")

        # Validate arguments
        if not self.validate(context.arguments):
            context.result = {"error": "Validation failed"}
            context.terminate = True
            return

        # Continue execution
        await next(context)
```

Methods

Expand table

__init__	Initialize the FunctionInvocationContext.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the FunctionInvocationContext.

Python

```
__init__(function: AIFunction[Any, Any], arguments: BaseModel, metadata: dict[str, Any] | None = None, result: Any = None, terminate: bool = False, kwargs: dict[str, Any] | None = None) -> None
```

Parameters

Expand table

Name	Description
function Required*	AIFunction[Any, Any] The function being invoked.
arguments Required*	<xref:BaseModel> The validated arguments for the function.
metadata	dict[str, Any] | None Metadata dictionary for sharing data between function middleware. Default value: None
result	Any Function execution result. Default value: None
terminate	bool A flag indicating whether to terminate execution after current middleware. Default value: False
kwargs	dict[str, Any] | None Additional keyword arguments passed to the chat method that invoked this function. Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}
```

```
   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
```

```
        "messages": [{"role": "user", "text": "Hello"}],
        "is_streaming": False,
        "metadata": {"session_id": "abc123"},
        # agent and result are excluded from serialization
    }

    # Inject agent and result during middleware processing
    my_agent = BaseAgent(name="test-agent")
    dependencies = {
        "agent_run_context": {
            "agent": my_agent,
            "result": None,  # Will be populated during execution
        }
    }

    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	

Name	Description
	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

function

The function being invoked.

arguments

The validated arguments for the function.

metadata

Metadata dictionary for sharing data between function middleware.

result

Function execution result. Can be observed after calling `next()` to see the actual execution result or can be set to override the execution result.

terminate

A flag indicating whether to terminate execution after current middleware. When set to True, execution will stop as soon as control returns to framework.

kwargs

Additional keyword arguments passed to the chat method that invoked this function.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'arguments', 'function', 'result'}
```

FunctionMiddleware Class

Abstract base class for function middleware that can intercept function invocations.

Function middleware allows you to intercept and modify function/tool invocations before and after execution. You can validate arguments, cache results, log invocations, or override function execution.

> ⓘ Note
>
> FunctionMiddleware is an abstract base class. You must subclass it and implement
>
> the process() method to create custom function middleware.

Constructor

Python

```
FunctionMiddleware()
```

Examples

Python

```
   from agent_framework import FunctionMiddleware, FunctionInvocationContext,
ChatAgent


   class CachingMiddleware(FunctionMiddleware):
       def __init__(self):
           self.cache = {}

       async def process(self, context: FunctionInvocationContext, next):
           cache_key = f"{context.function.name}:{context.arguments}"

           # Check cache
           if cache_key in self.cache:
               context.result = self.cache[cache_key]
               context.terminate = True
               return

           # Execute function
           await next(context)
```

```
            # Cache result
            if context.result:
                self.cache[cache_key] = context.result


    # Use with an agent
    agent = ChatAgent(chat_client=client, name="assistant",
middleware=CachingMiddleware())
```

Methods

Expand table

__init__	
__new__	
process	Process a function invocation.

> [!NOTE]
> Middleware should not return anything. All data manipulation should happen within the context object. Set context.result to override execution, or observe context.result after calling next() for actual results.

__init__

Python

```
__init__()
```

__new__

Python

```
__new__(**kwargs)
```

process

Process a function invocation.

> **Note**
>
> Middleware should not return anything. All data manipulation should happen
>
> within the context object. Set context.result to override execution,
>
> or observe context.result after calling next() for actual results.

Python

```
abstract async process(context: FunctionInvocationContext, next:
Callable[[FunctionInvocationContext], Awaitable[None]]) -> None
```

Parameters

Expand table

Name	Description
context Required*	FunctionInvocationContext Function invocation context containing function, arguments, and metadata. Middleware can set context.result to override execution, or observe the actual execution result after calling next().
next Required*	Callable[[FunctionInvocationContext], Awaitable[None]] Function to call the next middleware or final function execution. Does not return anything - all data flows through the context.

Returns

Expand table

Type	Description
None	

FunctionResultContent Class

Represents the result of a function call.

Initializes a FunctionResultContent instance.

Constructor

Python

```
FunctionResultContent()
```

Keyword-Only Parameters

Expand table

Name	Description
call_id	The identifier of the function call for which this is the result.
result	The result of the function call, or a generic error message if the function call failed. Default value: None
exception	An exception that occurred if the function call failed. Default value: None
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```
from agent_framework import FunctionResultContent
```

```python
# Create a successful function result
result = FunctionResultContent(
    call_id="call_123",
    result={"temperature": 22, "condition": "sunny"},
)

# Create a failed function result
failed_result = FunctionResultContent(
    call_id="call_124",
    result="Function execution failed",
    exception=ValueError("Invalid location"),
)
```

Methods

Expand table

__init__	Initializes a FunctionResultContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a FunctionResultContent instance.

Python

```
__init__(*, call_id: str, result: Any | None = None, exception: Exception | None =
None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
= None, additional_properties: dict[str, Any] | None = None, raw_representation:
Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
call_id Required*	str
result Required*	Any | None
exception Required*	Exception | None
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
call_id	The identifier of the function call for which this is the result.
result	The result of the function call, or a generic error message if the function call failed. Default value: None
exception	An exception that occurred if the function call failed. Default value: None
annotations	Optional annotations associated with the content. Default value: None

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value	

Name	Description
Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
   from agent_framework.openai import OpenAIChatClient
   from openai import AsyncOpenAI


   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
```

```
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

call_id

The identifier of the function call for which this is the result.

result

The result of the function call, or a generic error message if the function call failed.

exception

An exception that occurred if the function call failed.

type

The type of content, which is always "function_result" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

GraphConnectivityError Class

Exception raised when graph connectivity issues are detected.

Constructor

```
Python
GraphConnectivityError()
```

Parameters

Expand table

Name	Description
message Required*	

Methods

Expand table

__init__	
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

```
Python
__init__(message: str)
```

Parameters

Expand table

Name	Description
message Required*	str

__new__

```
Python
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

```
Python
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```
Python
with_traceback()
```

Attributes

args

GroupChatBuilder Class

High-level builder for manager-directed group chat workflows with dynamic orchestration.

GroupChat coordinates multi-agent conversations using a manager that selects which participant speaks next. The manager can be a simple Python function (set_select_speakers_func) or an agent-based selector via set_manager. These two approaches are mutually exclusive.

Core Workflow:

1. Define participants: list of agents (uses their .name) or dict mapping names to agents
2. Configure speaker selection: set_select_speakers_func OR

   set_manager (not both)
3. Optional: set round limits, checkpointing, termination conditions
4. Build and run the workflow

Speaker Selection Patterns:

Pattern 1: Simple function-based selection (recommended)

Python

```
from agent_framework import GroupChatBuilder, GroupChatStateSnapshot


def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:
    # state contains: task, participants, conversation, history, round_index
    if state["round_index"] >= 5:
        return None  # Finish
    last_speaker = state["history"][-1].speaker if state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"


workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher_agent, writer_agent])  # Uses agent.name
    .build()
)
```

Pattern 2: LLM-based selection

Python

```
from agent_framework import ChatAgent
from agent_framework.azure import AzureOpenAIChatClient
```

```
manager_agent = AzureOpenAIChatClient().create_agent(
    instructions="Coordinate the conversation and pick the next speaker.",
    name="Coordinator",
    temperature=0.3,
    seed=42,
    max_tokens=500,
)

workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent, display_name="Coordinator")
    .participants([researcher, writer])  # Or use dict: researcher=r, writer=w
    .with_max_rounds(10)
    .build()
)
```

Pattern 3: Request info for mid-conversation feedback

Python

```
from agent_framework import GroupChatBuilder

# Pause before all participants
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher, writer])
    .with_request_info()
    .build()
)

# Pause only before specific participants
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher, writer, editor])
    .with_request_info(agents=[editor])  # Only pause before editor responds
    .build()
)
```

Participant Specification:

Two ways to specify participants:

- List form: *[agent1, agent2]* - uses *agent.name* attribute for participant names
- Dict form: *{name1: agent1, name2: agent2}* - explicit name control
- Keyword form: *participants(name1=agent1, name2=agent2)* - explicit name control

State Snapshot Structure:

The GroupChatStateSnapshot passed to set_select_speakers_func contains:

- *task*: ChatMessage - Original user task

- *participants*: dict[str, str] - Mapping of participant names to descriptions
- *conversation*: tuple[ChatMessage, ...] - Full conversation history
- *history*: tuple[GroupChatTurn, ...] - Turn-by-turn record with speaker attribution
- *round_index*: int - Number of manager selection rounds so far
- *pending_agent*: str | None - Name of agent currently processing (if any)

Important Constraints:

- Cannot combine set_select_speakers_func and set_manager
- Participant names must be unique
- When using list form, agents must have a non-empty *name* attribute

Initialize the GroupChatBuilder.

Constructor

Python
`GroupChatBuilder()`

Parameters

Expand table

Name	Description
`_orchestrator_factory` Required*	Internal extension point for custom orchestrator implementations. Used by Magentic. Not part of public API - subject to change.
`_participant_factory` Required*	Internal extension point for custom participant pipelines. Used by Magentic. Not part of public API - subject to change.

Keyword-Only Parameters

Expand table

Name	Description
`_orchestrator_factory`	Default value: None
`_participant_factory`	Default value: None

Methods

Expand table

__init__	Initialize the GroupChatBuilder.

__new__	

build	Build and validate the group chat workflow.
	
	Assembles the orchestrator, participants, and their interconnections into a complete workflow graph. The orchestrator delegates speaker selection to the manager, routes requests to the appropriate participants, and collects their responses to continue or complete the conversation.
	
	Wiring pattern:
	
	• Orchestrator receives initial input (str, ChatMessage, or list[ChatMessage])
	• Orchestrator queries manager for next action (participant selection or finish)
	• If participant selected: request routed directly to participant entry node
	• Participant pipeline: AgentExecutor for agents or custom executor chains
	• Participant response flows back to orchestrator
	• Orchestrator updates state and queries manager again
	• When manager returns finish directive: orchestrator yields final message and becomes idle
	
	Usage:
	
	Python
	
	`from agent_framework import GroupChatBuilder`
	
	`# Execute the workflow`
	`workflow = GroupChatBuilder().set_manager(manager_agent).participants( agent1=agent1, agent2=agent2).build()`
	`async for message in workflow.run("Solve this problem collaboratively"): print(message.text)`

participants	Define participants for this group chat workflow.
	
	Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances. Provide a mapping of name → participant for explicit control, or pass a sequence and names will be inferred from the agent's name attribute (or executor id).
	
	Usage:
	
	Python
	
	`from agent_framework import GroupChatBuilder`
	
	`workflow = ( GroupChatBuilder().set_manager(manager_agent).participants(`

```
[writer_agent, reviewer_agent]).build()
    )
```

set_manager	Configure the manager/coordinator agent for group chat orchestration.
	
	The manager coordinates participants by selecting who speaks next based on conversation state and task requirements. The manager is a full workflow participant with access to all agent infrastructure (tools, context, observability).
	
	The manager agent must produce structured output compatible with ManagerSelectionResponse to communicate its speaker selection decisions. Use response_format for reliable parsing. GroupChatBuilder enforces this when the manager is a ChatAgent and rejects incompatible response formats.
	
	Example:
	
	Python
	
	`from agent_framework import GroupChatBuilder, ChatAgent`
	`from agent_framework.openai import OpenAIChatClient`
	
	`# Coordinator agent - response_format is enforced to ManagerSelectionResponse`
	`coordinator = ChatAgent(`
	`name="Coordinator",`
	`description="Coordinates multi-agent collaboration",`
	`instructions='''`
	`You coordinate a team conversation. Review the conversation history`
	`and select the next participant to speak.`
	
	`When ready to finish, set finish=True and provide a summary in final_message.`
	`''',`
	`chat_client=OpenAIChatClient(),`
	`)`
	
	`workflow = (`
	`GroupChatBuilder()`
	`.set_manager(coordinator, display_name="Orchestrator")`
	`.participants([researcher, writer])`
	`.build()`
	`)`
	
	Note
	
	The manager agent's response_format must be ManagerSelectionResponse for structured output.
	
	Custom response formats raise ValueError instead of being overridden.
	
	The manager can be included in with_request_info to pause before the manager

runs, allowing human steering of orchestration decisions. If no filter is specified,

the manager is included automatically. To filter explicitly:

.with_request_info(agents=[manager, writer]) # Pause before manager and writer

set_select_speakers_func	Configure speaker selection using a pure function that examines group chat state. This is the primary way to control orchestration flow in a GroupChat. Your selector function receives an immutable snapshot of the current conversation state and returns the name of the next participant to speak, or None to finish the conversation. The selector function can implement any logic including: • Simple round-robin or rule-based selection • LLM-based decision making with custom prompts • Conversation summarization before routing to the next agent • Custom metadata or context passing For advanced scenarios, return a GroupChatDirective instead of a string to include custom instructions or metadata for the next participant. The selector function signature: def select_next_speaker(state: GroupChatStateSnapshot) -> str | None: # state contains: task, participants, conversation, history, round_index # Return participant name to continue, or None to finish ... Example (simple): **Python** `def select_next_speaker(state: GroupChatStateSnapshot) -> str	None: if state["round_index"] >= 3: return None # Finish after 3 rounds last_speaker = state["history"][-1].speaker if state["history"] else None if last_speaker == "researcher": return "writer" return "researcher" workflow = (GroupChatBuilder() .set_select_speakers_func(select_next_speaker) .participants(researcher=researcher_agent, writer=writer_agent) .build())` Example (with LLM and custom instructions):

Python

```
from agent_framework import GroupChatDirective


async def llm_based_selector(state:
GroupChatStateSnapshot) -> GroupChatDirective | None:
    if state["round_index"] >= 5:
        return GroupChatDirective(finish=True)

    # Use LLM to decide next speaker and summarize
conversation
    conversation_summary = await
summarize_with_llm(state["conversation"])
    next_agent = await
pick_agent_with_llm(state["participants"], state["task"])

    # Pass custom instruction to the selected agent
    return GroupChatDirective(
        agent_name=next_agent,
        instruction=f"Context summary:
{conversation_summary}",
    )


workflow =
GroupChatBuilder().set_select_speakers_func(llm_based_selec
tor).participants(...).build()
```

> **Note**
>
> Cannot be combined with set_manager. Choose one orchestration strategy.

with_checkpointing	Enable checkpointing for the built workflow using the provided storage.
	
	Checkpointing allows the workflow to persist state and resume from interruption points, enabling long-running conversations and failure recovery.
	
	Usage:
	
	Python
	
	`from agent_framework import GroupChatBuilder, MemoryCheckpointStorage`
	
	`storage = MemoryCheckpointStorage()`
	`workflow = (`
	`  GroupChatBuilder()`
	`  .set_manager(manager_agent)`
	`  .participants(agent1=agent1, agent2=agent2)`
	`  .with_checkpointing(storage)`

```
        .build()
    )
```

with_max_rounds	Set a maximum number of manager rounds to prevent infinite conversations.
	
	When the round limit is reached, the workflow automatically completes with a default completion message. Setting to None allows unlimited rounds.
	
	Usage:
	
	Python
	
	`from agent_framework import GroupChatBuilder`
	
	`# Limit to 15 rounds`
	`workflow = (`
	`GroupChatBuilder()`
	`.set_manager(manager_agent)`
	`.participants(agent1=agent1, agent2=agent2)`
	`.with_max_rounds(15)`
	`.build()`
	`)`
	
	`# Unlimited rounds`
	`workflow = (`
	
	`GroupChatBuilder().set_manager(manager_agent).participants(agent1=agent1).with_max_rounds(None).build()`
	`)`

with_request_handler	Register an interceptor factory that creates executors for special requests.

with_request_info	Enable request info before participants run in the workflow.
	
	When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.
	
	Example:
	
	Python
	
	`# Pause before all participants`
	`workflow = (`
	`GroupChatBuilder()`
	`.set_manager(manager)`
	`.participants([optimist, pragmatist, creative])`
	`.with_request_info()`
	`.build()`
	`)`
	
	`# Pause only before specific participants`
	`workflow = (`
	`GroupChatBuilder()`
	`.set_manager(manager)`

```
        .participants([optimist, pragmatist, creative])
        .with_request_info(agents=[pragmatist])  # Only
pause before pragmatist
        .build()
    )
```

with_termination_condition	Define a custom termination condition for the group chat workflow.
	
	The condition receives the full conversation (including manager and agent messages) and may be async. When it returns True, the orchestrator halts the conversation and emits a completion message authored by the manager.
	
	Example:
	
	Python
	
	`from agent_framework import ChatMessage, GroupChatBuilder, Role`
	
	`def stop_after_two_calls(conversation: list[ChatMessage]) -> bool:`
	`  calls = sum(1 for msg in conversation if msg.role == Role.ASSISTANT and msg.author_name == "specialist")`
	`  return calls >= 2`
	
	`specialist_agent = ...`
	`workflow = (`
	`  GroupChatBuilder()`
	`  .set_select_speakers_func(lambda _: "specialist")`
	`  .participants(specialist=specialist_agent)`
	`  .with_termination_condition(stop_after_two_calls)`
	`  .build()`
	`)`

__init__

Initialize the GroupChatBuilder.

Python

```
__init__(*, _orchestrator_factory: Callable[[_GroupChatConfig], Executor] | None = None, _participant_factory: Callable[[GroupChatParticipantSpec, _GroupChatConfig], Sequence[Executor]] | None = None) -> None
```

Parameters

Expand table

Name	Description
_orchestrator_factory Required*	Callable[[<xref:agent_framework._workflows._group_chat._GroupChatConfig>], Execut None Internal extension point for custom orchestrator implementations. Used by Magentic. part of public API - subject to change.
_participant_factory Required*	Callable[[<xref:agent_framework._workflows._participant_utils.GroupChatParticipantSp <xref:agent_framework._workflows._group_chat._GroupChatConfig>], Sequence[Executor]] | None Internal extension point for custom participant pipelines. Used by Magentic. Not part public API - subject to change.

Keyword-Only Parameters

Expand table

Name	Description
_orchestrator_factory	Default value: None
_participant_factory	Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

build

Build and validate the group chat workflow.

Assembles the orchestrator, participants, and their interconnections into a complete workflow graph. The orchestrator delegates speaker selection to the manager, routes requests to the appropriate participants, and collects their responses to continue or complete the conversation.

Wiring pattern:

- Orchestrator receives initial input (str, ChatMessage, or list[ChatMessage])

- Orchestrator queries manager for next action (participant selection or finish)
- If participant selected: request routed directly to participant entry node
- Participant pipeline: AgentExecutor for agents or custom executor chains
- Participant response flows back to orchestrator
- Orchestrator updates state and queries manager again
- When manager returns finish directive: orchestrator yields final message and becomes idle

Usage:

Python

```
   from agent_framework import GroupChatBuilder

   # Execute the workflow
   workflow =
GroupChatBuilder().set_manager(manager_agent).participants(agent1=agent1,
agent2=agent2).build()
   async for message in workflow.run("Solve this problem collaboratively"):
       print(message.text)
```

Python

```
build() -> Workflow
```

Returns

Expand table

Type	Description
Workflow	Validated Workflow instance ready for execution

Exceptions

Expand table

Type	Description
ValueError	If manager or participants are not configured (when using default factory)

participants

Define participants for this group chat workflow.

Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances. Provide a mapping of name → participant for explicit control, or pass a sequence and names

will be inferred from the agent's name attribute (or executor id).

Usage:

Python

```
from agent_framework import GroupChatBuilder

workflow = (
    GroupChatBuilder().set_manager(manager_agent).participants([writer_agent,
reviewer_agent]).build()
)
```

Python

```
participants(participants: Mapping[str, AgentProtocol | Executor] |
Sequence[AgentProtocol | Executor] | None = None, /, **named_participants:
AgentProtocol | Executor) -> GroupChatBuilder
```

Positional-Only Parameters

Expand table

Name	Description
participants	Default value: None

Parameters

Expand table

Name	Description
participants Required*	Mapping[str, AgentProtocol | Executor] | Sequence[AgentProtocol | Executor] | None Optional mapping or sequence of participant definitions
****named_participants** Required*	AgentProtocol | Executor Keyword arguments mapping names to agent/executor instances

Returns

Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

Exceptions

Expand table

Type	Description
ValueError	If participants are empty, names are duplicated, or names are empty strings

set_manager

Configure the manager/coordinator agent for group chat orchestration.

The manager coordinates participants by selecting who speaks next based on conversation state and task requirements. The manager is a full workflow participant with access to all agent infrastructure (tools, context, observability).

The manager agent must produce structured output compatible with ManagerSelectionResponse to communicate its speaker selection decisions. Use response_format for reliable parsing. GroupChatBuilder enforces this when the manager is a ChatAgent and rejects incompatible response formats.

Example:

Python

```
from agent_framework import GroupChatBuilder, ChatAgent
from agent_framework.openai import OpenAIChatClient

# Coordinator agent - response_format is enforced to ManagerSelectionResponse
coordinator = ChatAgent(
    name="Coordinator",
    description="Coordinates multi-agent collaboration",
    instructions='''
    You coordinate a team conversation. Review the conversation history
    and select the next participant to speak.

    When ready to finish, set finish=True and provide a summary in
final_message.
    ''',
    chat_client=OpenAIChatClient(),
)

workflow = (
    GroupChatBuilder()
    .set_manager(coordinator, display_name="Orchestrator")
    .participants([researcher, writer])
    .build()
)
```

Note

The manager agent's response_format must be ManagerSelectionResponse for structured output.

Custom response formats raise ValueError instead of being overridden.

The manager can be included in with_request_info to pause before the manager runs, allowing human steering of orchestration decisions. If no filter is specified, the manager is included automatically. To filter explicitly:

.with_request_info(agents=[manager, writer]) # Pause before manager and writer

Python

```
set_manager(manager: AgentProtocol | Executor, *, display_name: str | None = None)
-> GroupChatBuilder
```

Parameters

Expand table

Name	Description
manager Required*	AgentProtocol | Executor Agent or executor responsible for speaker selection and coordination. Must return ManagerSelectionResponse or compatible dict/JSON structure.
display_name Required*	str | None Optional name for manager messages in conversation history. If not provided, uses manager.name for AgentProtocol or manager.id for Executor.

Keyword-Only Parameters

Expand table

Name	Description
display_name	Default value: None

Returns

Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining.

Exceptions

Expand table

Type	Description
ValueError	If manager is already configured via
set_select_speakers_func	
TypeError	If manager is not AgentProtocol or Executor instance

set_select_speakers_func

Configure speaker selection using a pure function that examines group chat state.

This is the primary way to control orchestration flow in a GroupChat. Your selector function receives an immutable snapshot of the current conversation state and returns the name of the next participant to speak, or None to finish the conversation.

The selector function can implement any logic including:

- Simple round-robin or rule-based selection
- LLM-based decision making with custom prompts
- Conversation summarization before routing to the next agent
- Custom metadata or context passing

For advanced scenarios, return a GroupChatDirective instead of a string to include custom instructions or metadata for the next participant.

The selector function signature: def select_next_speaker(state: GroupChatStateSnapshot) -> str | None: # state contains: task, participants, conversation, history, round_index # Return participant name to continue, or None to finish ...

Example (simple):

Python

```
def select_next_speaker(state: GroupChatStateSnapshot) -> str | None:
    if state["round_index"] >= 3:
        return None  # Finish after 3 rounds
    last_speaker = state["history"][-1].speaker if state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"


workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants(researcher=researcher_agent, writer=writer_agent)
```

```
        .build()
    )
```

Example (with LLM and custom instructions):

Python

```
    from agent_framework import GroupChatDirective


    async def llm_based_selector(state: GroupChatStateSnapshot) ->
GroupChatDirective | None:
        if state["round_index"] >= 5:
            return GroupChatDirective(finish=True)

        # Use LLM to decide next speaker and summarize conversation
        conversation_summary = await summarize_with_llm(state["conversation"])
        next_agent = await pick_agent_with_llm(state["participants"],
state["task"])

        # Pass custom instruction to the selected agent
        return GroupChatDirective(
            agent_name=next_agent,
            instruction=f"Context summary: {conversation_summary}",
        )


    workflow =
GroupChatBuilder().set_select_speakers_func(llm_based_selector).participants(...).
build()
```

> **Note**
>
> Cannot be combined with set_manager. Choose one orchestration strategy.

Python

```
set_select_speakers_func(selector: Callable[[Mapping[str, Any]], Awaitable[str |
None]] | Callable[[Mapping[str, Any]], str | None], *, display_name: str | None =
None, final_message: ChatMessage | str | Callable[[Mapping[str, Any]], Any] | None
= None) -> GroupChatBuilder
```

Parameters

Expand table

Name	Description
selector Required*	Callable[[Mapping[str, Any]], Awaitable[str | None]] | Callable[[Mapping[str, Any]], str | None] Function that takes GroupChatStateSnapshot and returns the next speaker's name (str) to continue the conversation, or None to finish. May be sync or async. Can also

Name	Description
	return GroupChatDirective for advanced control (instruction, metadata).
display_name Required*	str | None Optional name shown in conversation history for orchestrator messages (defaults to "manager").
final_message Required*	ChatMessage | str | Callable[[Mapping[str, Any]], Any] | None Optional final message (or factory) emitted when selector returns None (defaults to "Conversation completed." authored by the manager).

Keyword-Only Parameters

Expand table

Name	Description
display_name	Default value: None
final_message	Default value: None

Returns

Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining.

with_checkpointing

Enable checkpointing for the built workflow using the provided storage.

Checkpointing allows the workflow to persist state and resume from interruption points, enabling long-running conversations and failure recovery.

Usage:

Python

```
from agent_framework import GroupChatBuilder, MemoryCheckpointStorage

storage = MemoryCheckpointStorage()
workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent)
    .participants(agent1=agent1, agent2=agent2)
    .with_checkpointing(storage)
    .build()
)
```

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> GroupChatBuilder
```

Parameters

Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage Storage implementation for persisting workflow state

Returns

Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

with_max_rounds

Set a maximum number of manager rounds to prevent infinite conversations.

When the round limit is reached, the workflow automatically completes with a default completion message. Setting to None allows unlimited rounds.

Usage:

Python

```
from agent_framework import GroupChatBuilder

# Limit to 15 rounds
workflow = (
    GroupChatBuilder()
    .set_manager(manager_agent)
    .participants(agent1=agent1, agent2=agent2)
    .with_max_rounds(15)
    .build()
)

# Unlimited rounds
workflow = (

GroupChatBuilder().set_manager(manager_agent).participants(agent1=agent1).with_max
_rounds(None).build()
    )
```

Python

```
with_max_rounds(max_rounds: int | None) -> GroupChatBuilder
```

Parameters

Expand table

Name	Description
max_rounds Required*	int | None Maximum number of manager selection rounds, or None for unlimited

Returns

Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

with_request_handler

Register an interceptor factory that creates executors for special requests.

Python

```
with_request_handler(handler: Callable[[_GroupChatConfig], Executor] | Executor,
*, condition: Callable[[Any], bool]) -> GroupChatBuilder
```

Parameters

Expand table

Name	Description
handler Required*	Callable[[<xref:agent_framework._workflows._group_chat._GroupChatConfig>], Executor] | Executor Callable that receives the wiring and returns an executor, or a pre-built executor
condition Required*	Callable[[Any], bool] Filter determining which orchestrator messages the interceptor should process

Keyword-Only Parameters

Expand table

Name	Description
condition Required*	

Returns

Expand table

Type	Description
GroupChatBuilder	Self for fluent chaining

with_request_info

Enable request info before participants run in the workflow.

When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.

Example:

Python

```
# Pause before all participants
workflow = (
    GroupChatBuilder()
    .set_manager(manager)
    .participants([optimist, pragmatist, creative])
    .with_request_info()
    .build()
)

# Pause only before specific participants
workflow = (
    GroupChatBuilder()
    .set_manager(manager)
    .participants([optimist, pragmatist, creative])
    .with_request_info(agents=[pragmatist])  # Only pause before pragmatist
    .build()
)
```

Python

```
with_request_info(*, agents: Sequence[str | AgentProtocol | Executor] | None =
None) -> GroupChatBuilder
```

Parameters

Expand table

Name	Description
agents Required*	Sequence[str | AgentProtocol | Executor] | None Optional filter - only pause before these specific agents/executors. Accepts agent names (str), agent instances, or executor instances. If None (default), pauses before every participant.

Keyword-Only Parameters

Expand table

Name	Description
agents	Default value: None

Returns

Expand table

Type	Description
<xref:self>	The builder instance for fluent chaining.

with_termination_condition

Define a custom termination condition for the group chat workflow.

The condition receives the full conversation (including manager and agent messages) and may be async. When it returns True, the orchestrator halts the conversation and emits a completion message authored by the manager.

Example:

Python

```
from agent_framework import ChatMessage, GroupChatBuilder, Role


def stop_after_two_calls(conversation: list[ChatMessage]) -> bool:
    calls = sum(1 for msg in conversation if msg.role == Role.ASSISTANT and
msg.author_name == "specialist")
    return calls >= 2


specialist_agent = ...
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(lambda _: "specialist")
    .participants(specialist=specialist_agent)
```

```
        .with_termination_condition(stop_after_two_calls)
        .build()
    )
```

Python

```
with_termination_condition(condition: Callable[[list[ChatMessage]], bool |
Awaitable[bool]]) -> GroupChatBuilder
```

Parameters

Expand table

Name	Description
condition Required*	Callable[[list[ChatMessage]], bool | Awaitable[bool]]

Returns

Expand table

Type	Description
GroupChatBuilder	

GroupChatDirective Class

Instruction emitted by a group chat manager implementation.

Constructor

Python

```
GroupChatDirective()
```

Parameters

Expand table

Name	Description
agent_name	Default value: None
instruction	Default value: None
metadata	Default value: None
finish	Default value: False
final_message	Default value: None

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(agent_name: str | None = None, instruction: str | None = None, metadata: dict[str, Any] | None = None, finish: bool = False, final_message: ChatMessage | None = None) -> None
```

Parameters

Expand table

Name	Description
agent_name	str | None Default value: None
instruction	str | None Default value: None
metadata	dict[str, Any] | None Default value: None
finish	bool Default value: False
final_message	ChatMessage | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

Attributes

agent_name

Python

```
agent_name: str | None = None
```

final_message

Python

```python
final_message: ChatMessage | None = None
```

finish

Python

```python
finish: bool = False
```

instruction

Python

```python
instruction: str | None = None
```

metadata

Python

```python
metadata: dict[str, Any] | None = None
```

HandoffBuilder Class

Fluent builder for conversational handoff workflows with coordinator and specialist agents.

The handoff pattern enables a coordinator agent to route requests to specialist agents. Interaction mode controls whether the workflow requests user input after each agent response or completes autonomously once agents finish responding. A termination condition determines when the workflow should stop requesting input and complete.

Routing Patterns:

Single-Tier (Default): Only the coordinator can hand off to specialists. By default, after any specialist responds, control returns to the user for more input. This creates a cyclical flow: user -> coordinator -> [optional specialist] -> user -> coordinator -> ... Use *with_interaction_mode("autonomous")* to skip requesting additional user input and yield the final conversation when an agent responds without delegating.

Multi-Tier (Advanced): Specialists can hand off to other specialists using *.add_handoff()*. This provides more flexibility for complex workflows but is less controllable than the single-tier pattern. Users lose real-time visibility into intermediate steps during specialist-to-specialist handoffs (though the full conversation history including all handoffs is preserved and can be inspected afterward).

Key Features:

- **Automatic handoff detection**: The coordinator invokes a handoff tool whose

  arguments (for example *{"handoff_to": "shipping_agent"}*) identify the specialist to receive control.

- **Auto-generated tools**: By default the builder synthesizes *handoff_to_<agent>* tools for the coordinator, so you don't manually define placeholder functions.
- **Full conversation history**: The entire conversation (including any *ChatMessage.additional_properties*) is preserved and passed to each agent.
- **Termination control**: By default, terminates after 10 user messages. Override with *.with_termination_condition(lambda conv: ...)* for custom logic (e.g., detect "goodbye").
- **Interaction modes**: Choose *human_in_loop* (default) to prompt users between agent turns, or *autonomous* to continue routing back to agents without prompting for user input until a handoff occurs or a termination/turn limit is reached (default autonomous turn limit: 50).

- **Checkpointing**: Optional persistence for resumable workflows.

Usage (Single-Tier):

Python

```python
from agent_framework import HandoffBuilder
from agent_framework.openai import OpenAIChatClient

chat_client = OpenAIChatClient()

# Create coordinator and specialist agents
coordinator = chat_client.create_agent(
    instructions=(
        "You are a frontline support agent. Assess the user's issue and decide "
        "whether to hand off to 'refund_agent' or 'shipping_agent'. When delegation
is "
        "required, call the matching handoff tool (for example
`handoff_to_refund_agent`)."
    ),
    name="coordinator_agent",
)

refund = chat_client.create_agent(
    instructions="You handle refund requests. Ask for order details and process
refunds.",
    name="refund_agent",
)

shipping = chat_client.create_agent(
    instructions="You resolve shipping issues. Track packages and update delivery
status.",
    name="shipping_agent",
)

# Build the handoff workflow - default single-tier routing
workflow = (
    HandoffBuilder(
        name="customer_support",
        participants=[coordinator, refund, shipping],
    )
    .set_coordinator(coordinator)
    .build()
)

# Run the workflow
events = await workflow.run_stream("My package hasn't arrived yet")
async for event in events:
    if isinstance(event, RequestInfoEvent):
        # Request user input
        user_response = input("You: ")
        await workflow.send_response(event.data.request_id, user_response)
```

Multi-Tier Routing with .add_handoff():

Python

```
# Enable specialist-to-specialist handoffs with fluent API
workflow = (
    HandoffBuilder(participants=[coordinator, replacement, delivery, billing])
    .set_coordinator(coordinator)
    .add_handoff(coordinator, [replacement, delivery, billing])  # Coordinator routes to all
    .add_handoff(replacement, [delivery, billing])  # Replacement delegates to delivery/billing
    .add_handoff(delivery, billing)  # Delivery escalates to billing
    .build()
)

# Flow: User → Coordinator → Replacement → Delivery → Back to User
# (Replacement hands off to Delivery without returning to user)
```

Use Participant Factories for State Isolation:

Custom Termination Condition:

Python

```
# Terminate when user says goodbye or after 5 exchanges
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
    .set_coordinator(coordinator)
    .with_termination_condition(
        lambda conv: (
            sum(1 for msg in conv if msg.role.value == "user") >= 5
            or any("goodbye" in msg.text.lower() for msg in conv[-2:])
        )
    )
    .build()
)
```

Checkpointing:

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
```

```
        .set_coordinator(coordinator)
        .with_checkpointing(storage)
        .build()
    )
```

Initialize a HandoffBuilder for creating conversational handoff workflows.

The builder starts in an unconfigured state and requires you to call:

1. *.participants([...])* - Register agents
2. or *.participant_factories({...})* - Register agent/executor factories
3. *.set_coordinator(...)* - Designate which agent receives initial user input
4. *.build()* - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

> **Note**
>
> Participants must have stable names/ids because the workflow maps the
>
> handoff tool arguments to these identifiers. Agent names should match
>
> the strings emitted by the coordinator's handoff tool (e.g., a tool that
>
> outputs {"handoff_to": "billing"} requires an agent named billing).

Constructor

Python
`HandoffBuilder()`

Parameters

Expand table

Name	Description
name Required*	Optional workflow name for identification and logging.
participants	

Name	Description
Required*	List of agents (AgentProtocol) or executors to participate in the handoff. The first agent you specify as coordinator becomes the orchestrating agent.
`participant_factories` Required*	Mapping of factory names to callables that produce agents or executors when invoked. This allows for lazy instantiation and state isolation per workflow instance created by this builder.
`description` Required*	Optional human-readable description of the workflow.
`name` Required*	Optional workflow identifier used in logging and debugging. If not provided, a default name will be generated.
`participants` Required*	Optional list of agents (AgentProtocol) or executors that will participate in the handoff workflow. You can also call *.participants([...])* later. Each participant must have a unique identifier (name for agents, id for executors).
`participant_factories` Required*	Optional mapping of factory names to callables that produce agents or executors when invoked. This allows for lazy instantiation and state isolation per workflow instance created by this builder.
`description` Required*	Optional human-readable description explaining the workflow's purpose. Useful for documentation and observability.

Keyword-Only Parameters

Expand table

Name	Description
`name`	Default value: None
`participants`	Default value: None
`participant_factories`	Default value: None
`description`	Default value: None

Methods

Expand table

__init__	Initialize a HandoffBuilder for creating conversational handoff workflows. The builder starts in an unconfigured state and requires you to call: 1. *.participants([...])* - Register agents

2. or *.participant_factories({...})* - Register agent/executor factories
3. *.set_coordinator(...)* - Designate which agent receives initial user input
4. *.build()* - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

> ⓘ **Note**
>
> Participants must have stable names/ids because the workflow maps the
>
> handoff tool arguments to these identifiers. Agent names should match
>
> the strings emitted by the coordinator's handoff tool (e.g., a tool that
>
> outputs {"handoff_to": "billing"} requires an agent named billing).

__new__	
add_handoff	Add handoff routing from a source agent to one or more target agents. This method enables specialist-to-specialist handoffs by configuring which agents can hand off to which others. Call this method multiple times to build a complete routing graph. By default, only the starting agent can hand off to all other participants; use this method to enable additional routing paths. > ⓘ **Note** > > Handoff tools are automatically registered for each source agent > > If a source agent is configured multiple times via add_handoff, targets are merged
build	Construct the final Workflow instance from the configured builder. This method validates the configuration and assembles all internal components: • Input normalization executor • Starting agent executor • Handoff coordinator • Specialist agent executors • User input gateway

- Request/response handling

Example (Minimal):

Python

```
   workflow = (
       HandoffBuilder(participants=[coordinator, refund,
billing]).set_coordinator("coordinator").build()
   )

   # Run the workflow
   async for event in workflow.run_stream("I need help"):
       # Handle events...
       pass
```

Example (Full Configuration):

Python

```
   from agent_framework import InMemoryCheckpointStorage

   storage = InMemoryCheckpointStorage()
   workflow = (
       HandoffBuilder(
           name="support_workflow",
           participants=[coordinator, refund, billing],
           description="Customer support with specialist
routing",
       )
       .set_coordinator("coordinator")
       .with_termination_condition(lambda conv: len(conv) >
20)
       .request_prompt("How can we help?")
       .with_checkpointing(storage)
       .build()
   )
```

> ⓘ **Note**
>
> After calling build(), the builder instance should not be reused. Create a
>
> new builder if you need to construct another workflow with different configuration.

enable_return_to_previous	Enable direct return to the current agent after user input, bypassing the coordinator.

When enabled, after a specialist responds without requesting another handoff, user input routes directly back to that same specialist instead of always routing back to the coordinator agent for re-evaluation.

This is useful when a specialist needs multiple turns with the user to gather information or resolve an issue, avoiding unnecessary coordinator involvement while maintaining context.

Flow Comparison:

Default (disabled): User -> Coordinator -> Specialist -> User -> Coordinator -> Specialist -> ...

With return_to_previous (enabled): User -> Coordinator -> Specialist -> User -> Specialist -> ...

Example:

Python

```
   workflow = (
       HandoffBuilder(participants=[triage,
technical_support, billing])
       .set_coordinator("triage")
       .add_handoff(triage, [technical_support, billing])
       .enable_return_to_previous()  # Enable direct return
routing
       .build()
   )

   # Flow: User asks question
   # -> Triage routes to Technical Support
   # -> Technical Support asks clarifying question
   # -> User provides more info
   # -> Routes back to Technical Support (not Triage)
   # -> Technical Support continues helping
```

Multi-tier handoff example:

Python

```
   workflow = (
       HandoffBuilder(participants=[triage, specialist_a,
specialist_b])
       .set_coordinator("triage")
       .add_handoff(triage, [specialist_a, specialist_b])
       .add_handoff(specialist_a, specialist_b)
       .enable_return_to_previous()
       .build()
   )
```

```
    # Flow: User asks question
    # -> Triage routes to Specialist A
    # -> Specialist A hands off to Specialist B
    # -> Specialist B asks clarifying question
    # -> User provides more info
    # -> Routes back to Specialist B (who is currently
handling the conversation)
```

> **Note**
>
> This feature routes to whichever agent most recently responded, whether that's
>
> the coordinator or a specialist. The conversation continues with that agent until
>
> they either hand off to another agent or the termination condition is met.

participant_factories	Register factories that produce agents or executors for the handoff workflow. Each factory is a callable that returns an AgentProtocol or Executor instance. Factories are invoked when building the workflow, allowing for lazy instantiation and state isolation per workflow instance. Example: .. code-block:: python from agent_framework import ChatAgent, HandoffBuilder def create_coordinator() -> ChatAgent: return ... def create_refund_agent() -> ChatAgent: return ... def create_billing_agent() -> ChatAgent: return ... factories = { "coordinator": create_coordinator, "refund": create_refund_agent, "billing": create_billing_agent, } builder = HandoffBuilder().participant_factories(factories)

Use the factory IDs to create handoffs and set the coordinator

builder.add_handoff("coordinator", ["refund", "billing"])
builder.set_coordinator("coordinator")

participants

Register the agents or executors that will participate in the handoff workflow.

Each participant must have a unique identifier (name for agents, id for executors). The workflow will automatically create an alias map so agents can be referenced by their name, display_name, or executor id when routing.

Example:

Python

```
   from agent_framework import HandoffBuilder
   from agent_framework.openai import OpenAIChatClient

   client = OpenAIChatClient()
   coordinator = client.create_agent(instructions="...",
name="coordinator")
   refund = client.create_agent(instructions="...",
name="refund_agent")
   billing = client.create_agent(instructions="...",
name="billing_agent")

   builder = HandoffBuilder().participants([coordinator,
refund, billing])
   # Now you can call .set_coordinator() to designate the
entry point
```

> **Note**
>
> This method resets any previously configured coordinator, so you must call
>
> .set_coordinator(...) again after changing participants.

request_prompt

Set a custom prompt message displayed when requesting user input.

By default, the workflow uses a generic prompt: "Provide your next input for the conversation." Use this method to customize the message shown to users when the workflow needs their response.

Example:

Python

```
   workflow = (
       HandoffBuilder(participants=[triage, refund,
```

```
billing])
        .set_coordinator("triage")
        .request_prompt("How can we help you today?")
        .build()
    )

    # For more context-aware prompts, you can access the
prompt via
    # RequestInfoEvent.data.prompt in your event handling
loop
```

> **Note**
>
> The prompt is static and set once during workflow construction. If you need
>
> dynamic prompts based on conversation state, you'll need to handle that in
>
> your application's event processing logic.

set_coordinator	Designate which agent receives initial user input and orchestrates specialist routing. The coordinator agent is responsible for analyzing user requests and deciding whether to: 1. Handle the request directly and respond to the user, OR 2. Hand off to a specialist agent by including handoff metadata in the response After a specialist responds, the workflow automatically returns control to the user (default) creating a cyclical flow: user -> coordinator -> [specialist] -> user -> ... Configure *with_interaction_mode("autonomous")* to continue with the responding agent without requesting another user turn until a handoff occurs or a termination/turn limit is met. Example: **Python** `# Use factory name with `.participant_factories()` builder = ( HandoffBuilder() .participant_factories({ "coordinator": create_coordinator, "refund": create_refund_agent, "billing": create_billing_agent, })`

```
        .set_coordinator("coordinator")
    )

    # Or pass the agent object directly
    builder = HandoffBuilder().participants([coordinator,
refund, billing]).set_coordinator(coordinator)
```

> **Note**
>
> The coordinator determines routing by invoking a handoff tool call whose
>
> arguments identify the target specialist (for example {"handoff_to": "billing"}).
>
> Decorate the tool with approval_mode="always_require" to ensure the workflow
>
> intercepts the call before execution and can make the transition.

with_checkpointing	Enable workflow state persistence for resumable conversations.
	
	Checkpointing allows the workflow to save its state at key points, enabling you to:
	
	• Resume conversations after application restarts
	• Implement long-running support tickets that span multiple sessions
	• Recover from failures without losing conversation context
	• Audit and replay conversation history
	
	Example (In-Memory):
	
	Python
	
	`from agent_framework import InMemoryCheckpointStorage`
	
	`storage = InMemoryCheckpointStorage()`
	`workflow = (`
	`HandoffBuilder(participants=[triage, refund, billing])`
	`.set_coordinator("triage")`
	`.with_checkpointing(storage)`
	`.build()`
	`)`
	
	`# Run workflow with a session ID for resumption`
	`async for event in workflow.run_stream("Help me", session_id="user_123"):`
	`# Process events...`

```
        pass

    # Later, resume the same conversation
    async for event in workflow.run_stream("I need a
refund", session_id="user_123"):
        # Conversation continues from where it left off
        pass
```

Use Cases:

- Customer support systems with persistent ticket history
- Multi-day conversations that need to survive server restarts
- Compliance requirements for conversation auditing
- A/B testing different agent configurations on same conversation

> ⓘ **Note**
>
> Checkpointing adds overhead for serialization and storage I/O. Use it when
>
> persistence is required, not for simple stateless request-response patterns.

with_interaction_mode	Choose whether the workflow requests user input or runs autonomously after agent replies. In autonomous mode, agents (including specialists) continue iterating on their task until they explicitly invoke a handoff tool or the turn limit is reached. This allows specialists to perform long-running autonomous tasks (e.g., research, coding, analysis) without prematurely returning control to the coordinator or user. Example: **Python** `workflow = (HandoffBuilder(participants=[coordinator, research_agent]) .set_coordinator(coordinator) .add_handoff(coordinator, research_agent) .add_handoff(research_agent, coordinator) .with_interaction_mode("autonomous", autonomous_turn_limit=20) .build()) # Flow: User asks a question # -> Coordinator routes to Research Agent`

```
    # -> Research Agent iterates (researches, analyzes,
refines)
    # -> Research Agent calls handoff_to_coordinator when
done
    # -> Coordinator provides final response
```

with_request_info	Enable request info before participants run in the workflow.
	
	When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.
	
	Example:
	
	Python
	
	``
	    # Pause before all participants
	    workflow = (
	        HandoffBuilder(participants=[coordinator, refund,
	shipping])
	        .set_coordinator("coordinator_agent")
	        .with_request_info()
	        .build()
	    )
	
	    # Pause only before specialist agents (not coordinator)
	    workflow = (
	        HandoffBuilder(participants=[coordinator, refund,
	shipping])
	        .set_coordinator("coordinator_agent")
	        .with_request_info(agents=[refund, shipping])
	        .build()
	    )
	``

with_termination_condition	Set a custom termination condition for the handoff workflow.
	
	The condition can be either synchronous or asynchronous.
	
	Example:
	
	Python
	
	``
	    # Synchronous condition
	    builder.with_termination_condition(
	        lambda conv: len(conv) > 20 or any("goodbye" in
	msg.text.lower() for msg in conv[-2:])
	    )
	``

```
    # Asynchronous condition
    async def check_termination(conv: list[ChatMessage]) ->
bool:
        # Can perform async operations
        return len(conv) > 20


    builder.with_termination_condition(check_termination)
```

__init__

Initialize a HandoffBuilder for creating conversational handoff workflows.

The builder starts in an unconfigured state and requires you to call:

1. *.participants([...])* - Register agents
2. or *.participant_factories({...})* - Register agent/executor factories
3. *.set_coordinator(...)* - Designate which agent receives initial user input
4. *.build()* - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

> **Note**
>
> Participants must have stable names/ids because the workflow maps the
>
> handoff tool arguments to these identifiers. Agent names should match
>
> the strings emitted by the coordinator's handoff tool (e.g., a tool that
>
> outputs {"handoff_to": "billing"} requires an agent named billing).

Python

```
__init__(*, name: str | None = None, participants: Sequence[AgentProtocol |
Executor] | None = None, participant_factories: Mapping[str, Callable[[],
AgentProtocol | Executor]] | None = None, description: str | None = None) -> None
```

Parameters

Expand table

Name	Description
`name` Required*	str | None Optional workflow identifier used in logging and debugging. If not provided, a default name will be generated.
`participants` Required*	Sequence[AgentProtocol | Executor] | None Optional list of agents (AgentProtocol) or executors that will participate in the handoff workflow. You can also call *.participants([...])* later. Each participant must have a unique identifier (name for agents, id for executors).
`participant_factories` Required*	Mapping[str, Callable[[], AgentProtocol | Executor]] | None Optional mapping of factory names to callables that produce agents or executors when invoked. This allows for lazy instantiation and state isolation per workflow instance created by this builder.
`description` Required*	str | None Optional human-readable description explaining the workflow's purpose. Useful for documentation and observability.

Keyword-Only Parameters

Expand table

Name	Description
`name`	Default value: None
`participants`	Default value: None
`participant_factories`	Default value: None
`description`	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

add_handoff

Add handoff routing from a source agent to one or more target agents.

This method enables specialist-to-specialist handoffs by configuring which agents can hand off to which others. Call this method multiple times to build a complete routing graph. By default, only the starting agent can hand off to all other participants; use this method to enable additional routing paths.

> **Note**
>
> Handoff tools are automatically registered for each source agent
>
> If a source agent is configured multiple times via add_handoff, targets are merged

Python

```
add_handoff(source: str | AgentProtocol | Executor, targets: str | AgentProtocol | Executor | Sequence[str | AgentProtocol | Executor], *, tool_name: str | None = None, tool_description: str | None = None) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
source Required*	str | AgentProtocol | Executor The agent that can initiate the handoff. Can be: • Factory name (str): If using participant factories • AgentProtocol instance: The actual agent object • Executor instance: A custom executor wrapping an agent • Cannot mix factory names and instances across source and targets
targets Required*	str | AgentProtocol | Executor | Sequence[str | AgentProtocol | Executor] One or more target agents that the source can hand off to. Can be: • Factory name (str): If using participant factories • AgentProtocol instance: The actual agent object • Executor instance: A custom executor wrapping an agent • Single target: "billing_agent" or agent_instance

Name	Description
	• Multiple targets: ["billing_agent", "support_agent"] or [agent1, agent2] • Cannot mix factory names and instances across source and targets
tool_name Required*	str | None Optional custom name for the handoff tool. Currently not used in the implementation - tools are always auto-generated as "handoff_to_<target>". Reserved for future enhancement.
tool_description Required*	str | None Optional custom description for the handoff tool. Currently not used in the implementation - descriptions are always auto-generated as "Handoff to the <target> agent.". Reserved for future enhancement.

Keyword-Only Parameters

Expand table

Name	Description
tool_name	Default value: None
tool_description	Default value: None

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

Expand table

Type	Description
ValueError	If source or targets are not in the participants list, or if participants(...) hasn't been called yet. 2) If source or targets are factory names (str) but participant_factories(...) hasn't been called yet, or if they are not in the participant_factories list.
TypeError	If mixing factory names (str) and AgentProtocol/Executor instances

Examples

Single target (using factory name):

```python
builder.add_handoff("triage_agent", "billing_agent")
```

Multiple targets (using factory names):

```python
builder.add_handoff("triage_agent", ["billing_agent", "support_agent", "escalation_agent"])
```

Multiple targets (using agent instances):

```python
builder.add_handoff(triage, [billing, support, escalation])
```

Chain multiple configurations:

```python
workflow = (
    HandoffBuilder(participants=[triage, replacement, delivery, billing])
    .set_coordinator(triage)
    .add_handoff(triage, [replacement, delivery, billing])
    .add_handoff(replacement, [delivery, billing])
    .add_handoff(delivery, billing)
    .build()
)
```

build

Construct the final Workflow instance from the configured builder.

This method validates the configuration and assembles all internal components:

- Input normalization executor
- Starting agent executor
- Handoff coordinator

- Specialist agent executors
- User input gateway
- Request/response handling

Example (Minimal):

Python

```
   workflow = (
       HandoffBuilder(participants=[coordinator, refund,
billing]).set_coordinator("coordinator").build()
   )

   # Run the workflow
   async for event in workflow.run_stream("I need help"):
       # Handle events...
       pass
```

Example (Full Configuration):

Python

```
   from agent_framework import InMemoryCheckpointStorage

   storage = InMemoryCheckpointStorage()
   workflow = (
       HandoffBuilder(
           name="support_workflow",
           participants=[coordinator, refund, billing],
           description="Customer support with specialist routing",
       )
       .set_coordinator("coordinator")
       .with_termination_condition(lambda conv: len(conv) > 20)
       .request_prompt("How can we help?")
       .with_checkpointing(storage)
       .build()
   )
```

> ⓘ Note
>
> After calling build(), the builder instance should not be reused. Create a
>
> new builder if you need to construct another workflow with different configuration.

Python

```
build() -> Workflow
```

Returns

Expand table

Type	Description
Workflow	A fully configured Workflow ready to execute via *.run()* or *.run_stream()*.

Exceptions

Expand table

Type	Description
ValueError	If participants or coordinator were not configured, or if required configuration is invalid.

enable_return_to_previous

Enable direct return to the current agent after user input, bypassing the coordinator.

When enabled, after a specialist responds without requesting another handoff, user input routes directly back to that same specialist instead of always routing back to the coordinator agent for re-evaluation.

This is useful when a specialist needs multiple turns with the user to gather information or resolve an issue, avoiding unnecessary coordinator involvement while maintaining context.

Flow Comparison:

Default (disabled): User -> Coordinator -> Specialist -> User -> Coordinator -> Specialist -> ...

With return_to_previous (enabled): User -> Coordinator -> Specialist -> User -> Specialist -> ...

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[triage, technical_support, billing])
```

```
        .set_coordinator("triage")
        .add_handoff(triage, [technical_support, billing])
        .enable_return_to_previous()  # Enable direct return routing
        .build()
    )

    # Flow: User asks question
    # -> Triage routes to Technical Support
    # -> Technical Support asks clarifying question
    # -> User provides more info
    # -> Routes back to Technical Support (not Triage)
    # -> Technical Support continues helping
```

Multi-tier handoff example:

Python

```
    workflow = (
        HandoffBuilder(participants=[triage, specialist_a, specialist_b])
        .set_coordinator("triage")
        .add_handoff(triage, [specialist_a, specialist_b])
        .add_handoff(specialist_a, specialist_b)
        .enable_return_to_previous()
        .build()
    )

    # Flow: User asks question
    # -> Triage routes to Specialist A
    # -> Specialist A hands off to Specialist B
    # -> Specialist B asks clarifying question
    # -> User provides more info
    # -> Routes back to Specialist B (who is currently handling the conversation)
```

> **Note**
>
> This feature routes to whichever agent most recently responded, whether that's
>
> the coordinator or a specialist. The conversation continues with that agent until
>
> they either hand off to another agent or the termination condition is met.

Python

```
enable_return_to_previous(enabled: bool = True) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
`enabled`	bool Whether to enable return-to-previous routing. Default is True. Default value: True

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

participant_factories

Register factories that produce agents or executors for the handoff workflow.

Each factory is a callable that returns an AgentProtocol or Executor instance. Factories are invoked when building the workflow, allowing for lazy instantiation and state isolation per workflow instance.

Example: .. code-block:: python

from agent_framework import ChatAgent, HandoffBuilder

def create_coordinator() -> ChatAgent: return ...

def create_refund_agent() -> ChatAgent: return ...

def create_billing_agent() -> ChatAgent: return ...

factories = { "coordinator": create_coordinator, "refund": create_refund_agent, "billing": create_billing_agent,

}

builder = HandoffBuilder().participant_factories(factories)

Use the factory IDs to create handoffs and set the coordinator

builder.add_handoff("coordinator", ["refund", "billing"]) builder.set_coordinator("coordinator")

Python

```
participant_factories(participant_factories: Mapping[str, Callable[[],
AgentProtocol | Executor]]) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
participant_factories Required*	Mapping[str, Callable[[], AgentProtocol | Executor]] Mapping of factory names to callables that return AgentProtocol or Executor instances. Each produced participant must have a unique identifier (name for agents, id for executors).

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

Expand table

Type	Description
ValueError	If participant_factories is empty or *.participants(...)* or *.participant_factories(...)* has already been called.

participants

Register the agents or executors that will participate in the handoff workflow.

Each participant must have a unique identifier (name for agents, id for executors). The workflow will automatically create an alias map so agents can be referenced by their name, display_name, or executor id when routing.

Example:

Python

```
from agent_framework import HandoffBuilder
from agent_framework.openai import OpenAIChatClient

client = OpenAIChatClient()
coordinator = client.create_agent(instructions="...", name="coordinator")
refund = client.create_agent(instructions="...", name="refund_agent")
billing = client.create_agent(instructions="...", name="billing_agent")

builder = HandoffBuilder().participants([coordinator, refund, billing])
# Now you can call .set_coordinator() to designate the entry point
```

> **Note**
>
> This method resets any previously configured coordinator, so you must call
>
> .set_coordinator(...) again after changing participants.

Python

```
participants(participants: Sequence[AgentProtocol | Executor]) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
participants Required*	Sequence[AgentProtocol | Executor] Sequence of AgentProtocol or Executor instances. Each must have a unique identifier. For agents, the name attribute is used as the primary identifier and must match handoff target strings.

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

Expand table

Type	Description
ValueError	If participants is empty, contains duplicates, or *.participants(...)* or *.participant_factories(...)* has already been called.
TypeError	If participants are not AgentProtocol or Executor instances.

request_prompt

Set a custom prompt message displayed when requesting user input.

By default, the workflow uses a generic prompt: "Provide your next input for the conversation." Use this method to customize the message shown to users when the workflow needs their response.

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[triage, refund, billing])
    .set_coordinator("triage")
    .request_prompt("How can we help you today?")
    .build()
)

# For more context-aware prompts, you can access the prompt via
# RequestInfoEvent.data.prompt in your event handling loop
```

> **Note**
>
> The prompt is static and set once during workflow construction. If you need dynamic prompts based on conversation state, you'll need to handle that in your application's event processing logic.

Python

```
request_prompt(prompt: str | None) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
prompt Required*	str | None Custom prompt text to display, or None to use the default prompt.

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

set_coordinator

Designate which agent receives initial user input and orchestrates specialist routing.

The coordinator agent is responsible for analyzing user requests and deciding whether to:

1. Handle the request directly and respond to the user, OR
2. Hand off to a specialist agent by including handoff metadata in the response

After a specialist responds, the workflow automatically returns control to the user (default) creating a cyclical flow: user -> coordinator -> [specialist] -> user -> ... Configure *with_interaction_mode("autonomous")* to continue with the responding agent without requesting another user turn until a handoff occurs or a termination/turn limit is met.

Example:

Python

```
# Use factory name with `.participant_factories()`
builder = (
    HandoffBuilder()
    .participant_factories({
        "coordinator": create_coordinator,
        "refund": create_refund_agent,
        "billing": create_billing_agent,
    })
    .set_coordinator("coordinator")
)

# Or pass the agent object directly
```

```
   builder = HandoffBuilder().participants([coordinator, refund,
billing]).set_coordinator(coordinator)
```

> **Note**
>
> The coordinator determines routing by invoking a handoff tool call whose arguments identify the target specialist (for example {"handoff_to": "billing"}). Decorate the tool with approval_mode="always_require" to ensure the workflow intercepts the call before execution and can make the transition.

Python

```
set_coordinator(agent: str | AgentProtocol | Executor) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
agent Required*	str | AgentProtocol | Executor The agent to use as the coordinator. Can be: • Factory name (str): If using participant factories • AgentProtocol instance: The actual agent object • Executor instance: A custom executor wrapping an agent

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

Exceptions

Expand table

Type	Description
ValueError	If *agent* is an AgentProtocol or Executor instance but *.participants(...)* hasn't been called yet, or if it is not in the participants list. 2) If *agent* is a factory name (str) but *.participant_factories(...)* hasn't been called yet, or if it is not in the participant_factories list.
TypeError	If *agent* is not a str, AgentProtocol, or Executor instance.

with_checkpointing

Enable workflow state persistence for resumable conversations.

Checkpointing allows the workflow to save its state at key points, enabling you to:

- Resume conversations after application restarts
- Implement long-running support tickets that span multiple sessions
- Recover from failures without losing conversation context
- Audit and replay conversation history

Example (In-Memory):

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(participants=[triage, refund, billing])
    .set_coordinator("triage")
    .with_checkpointing(storage)
    .build()
)

# Run workflow with a session ID for resumption
async for event in workflow.run_stream("Help me", session_id="user_123"):
    # Process events...
    pass

# Later, resume the same conversation
async for event in workflow.run_stream("I need a refund",
session_id="user_123"):
    # Conversation continues from where it left off
    pass
```

Use Cases:

- Customer support systems with persistent ticket history

- Multi-day conversations that need to survive server restarts
- Compliance requirements for conversation auditing
- A/B testing different agent configurations on same conversation

> **Note**
>
> Checkpointing adds overhead for serialization and storage I/O. Use it when persistence is required, not for simple stateless request-response patterns.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
`checkpoint_storage` Required*	CheckpointStorage Storage backend implementing CheckpointStorage interface. Common implementations: InMemoryCheckpointStorage (testing), database-backed storage (production).

Returns

Expand table

Type	Description
HandoffBuilder	Self for method chaining.

with_interaction_mode

Choose whether the workflow requests user input or runs autonomously after agent replies.

In autonomous mode, agents (including specialists) continue iterating on their task until they explicitly invoke a handoff tool or the turn limit is reached. This allows specialists to perform long-running autonomous tasks (e.g., research, coding, analysis) without prematurely returning control to the coordinator or user.

Example:

Python

```
workflow = (
    HandoffBuilder(participants=[coordinator, research_agent])
    .set_coordinator(coordinator)
    .add_handoff(coordinator, research_agent)
    .add_handoff(research_agent, coordinator)
    .with_interaction_mode("autonomous", autonomous_turn_limit=20)
    .build()
)

# Flow: User asks a question
# -> Coordinator routes to Research Agent
# -> Research Agent iterates (researches, analyzes, refines)
# -> Research Agent calls handoff_to_coordinator when done
# -> Coordinator provides final response
```

Python

```
with_interaction_mode(interaction_mode: Literal['human_in_loop', 'autonomous'] = 'human_in_loop', *, autonomous_turn_limit: int | None = None) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
interaction_mode	Literal['human_in_loop', 'autonomous'] "*human_in_loop*" (default) requests user input after each agent response that does not trigger a handoff. "*autonomous*" lets agents continue working until they invoke a handoff tool or the turn limit is reached. Default value: "human_in_loop"
autonomous_turn_limit Required*	int | None

Keyword-Only Parameters

Expand table

Name	Description
autonomous_turn_limit	Maximum number of agent responses before the workflow yields when in autonomous mode. Only applicable when interaction_mode is

Name	Description
	"autonomous". Default is 50. Set to *None* to disable the limit (use with caution). Ignored with a warning if provided when interaction_mode is *"human_in_loop"*. Default value: None

Returns

Expand table

Type	Description
HandoffBuilder	Self for chaining.

with_request_info

Enable request info before participants run in the workflow.

When enabled, the workflow pauses before each participant runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the participant responds. The caller provides input via the standard response_handler/request_info pattern.

Example:

Python

```
# Pause before all participants
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
    .set_coordinator("coordinator_agent")
    .with_request_info()
    .build()
)

# Pause only before specialist agents (not coordinator)
workflow = (
    HandoffBuilder(participants=[coordinator, refund, shipping])
    .set_coordinator("coordinator_agent")
    .with_request_info(agents=[refund, shipping])
    .build()
)
```

Python

```
with_request_info(*, agents: Sequence[str | AgentProtocol | Executor] | None =
None) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
agents Required*	Sequence[str	AgentProtocol	Executor]	None Optional filter - only pause before these specific agents/executors. Accepts agent names (str), agent instances, or executor instances. If None (default), pauses before every participant.

Keyword-Only Parameters

Expand table

Name	Description
agents	Default value: None

Returns

Expand table

Type	Description
<xref:self>	The builder instance for fluent chaining.

with_termination_condition

Set a custom termination condition for the handoff workflow.

The condition can be either synchronous or asynchronous.

Example:

Python

```
   # Synchronous condition
   builder.with_termination_condition(
       lambda conv: len(conv) > 20 or any("goodbye" in msg.text.lower() for msg in
conv[-2:])
```

```
)


# Asynchronous condition
async def check_termination(conv: list[ChatMessage]) -> bool:
    # Can perform async operations
    return len(conv) > 20


builder.with_termination_condition(check_termination)
```

Python

```
with_termination_condition(condition: Callable[[list[ChatMessage]], bool | Awaitable[bool]]) -> HandoffBuilder
```

Parameters

Expand table

Name	Description
condition Required*	Callable[[list[ChatMessage]], bool | Awaitable[bool]] Function that receives the full conversation and returns True (or awaitable True) if the workflow should terminate (not request further user input).

Returns

Expand table

Type	Description
HandoffBuilder	Self for chaining.

HandoffUserInputRequest Class

Request message emitted when the workflow needs fresh user input.

Note: The conversation field is intentionally excluded from checkpoint serialization to prevent duplication. The conversation is preserved in the coordinator's state and will be reconstructed on restore. See issue #2667.

Constructor

Python

```
HandoffUserInputRequest()
```

Parameters

Expand table

Name	Description
conversation Required*	
awaiting_agent_id Required*	
prompt Required*	
source_executor_id Required*	

Methods

Expand table

__init__	
__new__	
from_dict	Deserialize from dict, initializing conversation as empty. The conversation will be reconstructed from the coordinator's state on restore.

to_dict	Serialize to dict, excluding conversation to prevent checkpoint duplication. The conversation is already preserved in the workflow coordinator's state. Including it here would cause duplicate messages when restoring from checkpoint.

__init__

Python

```
__init__(conversation: list[ChatMessage], awaiting_agent_id: str, prompt: str, source_executor_id: str) -> None
```

Parameters

Expand table

Name	Description
conversation Required*	list[ChatMessage]
awaiting_agent_id Required*	str
prompt Required*	str
source_executor_id Required*	str

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Deserialize from dict, initializing conversation as empty.

The conversation will be reconstructed from the coordinator's state on restore.

Python
`from_dict(data: dict[str, Any]) -> HandoffUserInputRequest`

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
HandoffUserInputRequest	

to_dict

Serialize to dict, excluding conversation to prevent checkpoint duplication.

The conversation is already preserved in the workflow coordinator's state. Including it here would cause duplicate messages when restoring from checkpoint.

Python
`to_dict() -> dict[str, Any]`

Returns

Expand table

Type	Description
dict[str, Any]	

Attributes

awaiting_agent_id

Python

```
awaiting_agent_id: str
```

conversation

Python

```
conversation: list[ChatMessage]
```

prompt

Python

```
prompt: str
```

source_executor_id

Python

```
source_executor_id: str
```

HostedCodeInterpreterTool Class

Represents a hosted tool that can be specified to an AI service to enable it to execute generated code.

This tool does not implement code interpretation itself. It serves as a marker to inform a service that it is allowed to execute generated code if the service is capable of doing so.

Initialize the HostedCodeInterpreterTool.

Constructor

Python

```
HostedCodeInterpreterTool()
```

Keyword-Only Parameters

Expand table

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should mostly be HostedFileContent or HostedVectorStoreContent. Can also be DataContent, depending on the service used. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com ", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None
description	A description of the tool. Default value: None
additional_properties	Additional properties associated with the tool. Default value: None
****kwargs**	Additional keyword arguments to pass to the base class.

Examples

Python

```python
from agent_framework import HostedCodeInterpreterTool

# Create a code interpreter tool
code_tool = HostedCodeInterpreterTool()

# With file inputs
code_tool_with_files = HostedCodeInterpreterTool(inputs=[{"file_id": "file-123"},
{"file_id": "file-456"}])
```

Methods

Expand table

__init__	Initialize the HostedCodeInterpreterTool.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the HostedCodeInterpreterTool.

Python

```
__init__(*, inputs: Contents | dict[str, Any] | str | list[Contents | dict[str, Any] | str] | None = None, description: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
inputs Required*	<xref:Contents> | dict[str, Any] | str | list[<xref:Contents> | dict[str, Any] | str] | None
description Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should mostly be HostedFileContent or HostedVectorStoreContent. Can also be DataContent, depending on the service used. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com ", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None
description	A description of the tool. Default value: None

Name	Description
additional_properties	Additional properties associated with the tool. Default value: None
****kwargs**	Additional keyword arguments to pass to the base class.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	

Name	Description
	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

HostedFileContent Class

Represents a hosted file content.

Initializes a HostedFileContent instance.

Constructor

Python

```
HostedFileContent()
```

Parameters

Expand table

Name	Description
file_id Required*	The identifier of the hosted file.

Keyword-Only Parameters

Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```
from agent_framework import HostedFileContent

# Create hosted file content
```

```
file_content = HostedFileContent(file_id="file-abc123")
print(file_content.file_id)  # "file-abc123"
```

Methods

Expand table

__init__	Initializes a HostedFileContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a HostedFileContent instance.

Python

```
__init__(file_id: str, *, additional_properties: dict[str, Any] | None = None,
raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
file_id Required*	str The identifier of the hosted file.
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}`

Name	Description
	Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

file_id

The identifier of the hosted file.

type

The type of content, which is always "hosted_file" for this class.

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

HostedFileSearchTool Class

Represents a file search tool that can be specified to an AI service to enable it to perform file searches.

Initialize a FileSearchTool.

Constructor

Python

```
HostedFileSearchTool()
```

Keyword-Only Parameters

Expand table

Name	Description
`inputs`	A list of contents that the tool can accept as input. Defaults to None. This should be one or more HostedVectorStoreContents. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com ", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None
`max_results`	The maximum number of results to return from the file search. If None, max limit is applied. Default value: None
`description`	A description of the tool. Default value: None
`additional_properties`	Additional properties associated with the tool. Default value: None
`**kwargs`	Additional keyword arguments to pass to the base class.

Examples

Python

```python
from agent_framework import HostedFileSearchTool

# Create a basic file search tool
file_search = HostedFileSearchTool()

# With vector store inputs and max results
file_search_with_inputs = HostedFileSearchTool(
    inputs=[{"vector_store_id": "vs_123"}],
    max_results=10,
    description="Search files in vector store",
)
```

Methods

Expand table

__init__	Initialize a FileSearchTool.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize a FileSearchTool.

Python

```
__init__(*, inputs: Contents | dict[str, Any] | str | list[Contents | dict[str, Any] | str] | None = None, max_results: int | None = None, description: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any)
```

Parameters

Expand table

Name	Description
inputs Required*	<xref:Contents> | dict[str, Any] | str | list[<xref:Contents> | dict[str, Any] | str] | None
max_results Required*	int | None
description Required*	str | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
inputs	A list of contents that the tool can accept as input. Defaults to None. This should be one or more HostedVectorStoreContents. When supplying a list, it can contain: • Contents instances • dicts with properties for Contents (e.g., {"uri": "http://example.com", "media_type": "text/html"}) • strings (which will be converted to UriContent with media_type "text/plain"). If None, defaults to an empty list. Default value: None

Name	Description
max_results	The maximum number of results to return from the file search. If None, max limit is applied. Default value: None
description	A description of the tool. Default value: None
additional_properties	Additional properties associated with the tool. Default value: None
****kwargs**	Additional keyword arguments to pass to the base class.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```python
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}
```

```
# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None

Name	Description
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

HostedMCPSpecificApproval Class

Represents the specific mode for a hosted tool.

When using this mode, the user must specify which tools always or never require approval. This is represented as a dictionary with two optional keys:

Constructor

Python

```
HostedMCPSpecificApproval()
```

Methods

Expand table

__init__	
__new__	
clear	
copy	
fromkeys	Create a new dictionary with keys from iterable and values set to value.
get	Return the value for key if key is in the dictionary, else default.
items	
keys	
pop	If the key is not found, return the default if given; otherwise, raise a KeyError.
popitem	Remove and return a (key, value) pair as a 2-tuple. Pairs are returned in LIFO (last-in, first-out) order. Raises KeyError if the dict is empty.
setdefault	Insert key with a value of default if key is not in the dictionary. Return the value for key if key is in the dictionary, else default.
update	If E is present and has a .keys() method, then does: for k in E: D[k] = E[k] If E is present and lacks a .keys() method, then does: for k, v in E: D[k] = v In either case, this is followed by: for k in F: D[k] = F[k]

values

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

clear

Python

```
clear() ->
```

copy

Python

```
copy() -> a shallow copy of D
```

fromkeys

Create a new dictionary with keys from iterable and values set to value.

Python

```
fromkeys(value=None, /)
```

Positional-Only Parameters

Expand table

Name	Description
iterable Required*	

Name	Description
value	Default value: None

get

Return the value for key if key is in the dictionary, else default.

Python

```
get(key, default=None, /)
```

Positional-Only Parameters

Expand table

Name	Description
key Required*	
default	Default value: None

items

Python

```
items() -> a set-like object providing a view on D's items
```

keys

Python

```
keys() -> a set-like object providing a view on D's keys
```

pop

If the key is not found, return the default if given; otherwise, raise a KeyError.

Python

```
pop(k, [d]) ->
```

popitem

Remove and return a (key, value) pair as a 2-tuple.

Pairs are returned in LIFO (last-in, first-out) order. Raises KeyError if the dict is empty.

Python

```
popitem()
```

setdefault

Insert key with a value of default if key is not in the dictionary.

Return the value for key if key is in the dictionary, else default.

Python

```
setdefault(key, default=None, /)
```

Positional-Only Parameters

Expand table

Name	Description
key Required*	
default	Default value: None

update

If E is present and has a .keys() method, then does: for k in E: D[k] = E[k] If E is present and lacks a .keys() method, then does: for k, v in E: D[k] = v In either case, this is followed by: for k in F: D[k] = F[k]

Python

```
update([E], **F) ->
```

values

Python

```
values() -> an object providing a view on D's values
```

Attributes

always_require_approval

A sequence of tool names that always require approval.

Python

```
always_require_approval: Collection[str] | None
```

never_require_approval

A sequence of tool names that never require approval.

Python

```
never_require_approval: Collection[str] | None
```

HostedMCPTool Class

Represents a MCP tool that is managed and executed by the service.

Create a hosted MCP tool.

Constructor

Python

```
HostedMCPTool()
```

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the tool.
description	A description of the tool. Default value: None
url	The URL of the tool.
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
headers	Headers to include in requests to the tool. Default value: None
additional_properties	Additional properties to include in the tool definition. Default value: None
****kwargs**	Additional keyword arguments to pass to the base class.

Examples

Python

```python
from agent_framework import HostedMCPTool

# Create a basic MCP tool
mcp_tool = HostedMCPTool(
    name="my_mcp_tool",
    url="https://example.com/mcp",
)

# With approval mode and allowed tools
mcp_tool_with_approval = HostedMCPTool(
    name="my_mcp_tool",
    description="My MCP tool",
    url="https://example.com/mcp",
    approval_mode="always_require",
    allowed_tools=["tool1", "tool2"],
    headers={"Authorization": "Bearer token"},
)

# With specific approval mode
mcp_tool_specific = HostedMCPTool(
    name="my_mcp_tool",
    url="https://example.com/mcp",
    approval_mode={
        "always_require_approval": ["dangerous_tool"],
        "never_require_approval": ["safe_tool"],
    },
)
```

Methods

Expand table

__init__	Create a hosted MCP tool.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string.

	This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Create a hosted MCP tool.

Python

```
__init__(*, name: str, description: str | None = None, url: AnyUrl | str,
approval_mode: Literal['always_require', 'never_require'] |
HostedMCPSpecificApproval | None = None, allowed_tools: Collection[str] | None =
None, headers: dict[str, str] | None = None, additional_properties: dict[str, Any]
| None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
name Required*	str
description Required*	str | None
url Required*	<xref:pydantic.networks.AnyUrl> | str
approval_mode Required*	Literal['always_require', 'never_require'] | agent_framework._tools.HostedMCPSpecificApproval | None

Name	Description
allowed_tools Required*	Collection[str] | None
headers Required*	dict[str, str] | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
name	The name of the tool.
description	A description of the tool. Default value: None
url	The URL of the tool.
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
headers	Headers to include in requests to the tool. Default value: None
additional_properties	Additional properties to include in the tool definition. Default value: None
****kwargs**	Additional keyword arguments to pass to the base class.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```
   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	

Name	Description
	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

HostedVectorStoreContent Class

Represents a hosted vector store content.

Initializes a HostedVectorStoreContent instance.

Constructor

Python

```
HostedVectorStoreContent()
```

Parameters

Expand table

Name	Description
vector_store_id Required*	The identifier of the hosted vector store.

Keyword-Only Parameters

Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```
from agent_framework import HostedVectorStoreContent

# Create hosted vector store content
```

```
vs_content = HostedVectorStoreContent(vector_store_id="vs-xyz789")
print(vs_content.vector_store_id)  # "vs-xyz789"
```

Methods

Expand table

__init__	Initializes a HostedVectorStoreContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a HostedVectorStoreContent instance.

Python

```
__init__(vector_store_id: str, *, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
vector_store_id Required*	str The identifier of the hosted vector store.
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}`

Name	Description
	Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

vector_store_id

The identifier of the hosted vector store.

type

The type of content, which is always "hosted_vector_store" for this class.

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

annotations

Python

```
annotations: list[CitationAnnotation] | None
```

HostedWebSearchTool Class

Represents a web search tool that can be specified to an AI service to enable it to perform web searches.

Initialize a HostedWebSearchTool.

Constructor

Python

```
HostedWebSearchTool()
```

Parameters

Expand table

Name	Description
description	Default value: None
additional_properties	Default value: None

Keyword-Only Parameters

Expand table

Name	Description
description	A description of the tool.
additional_properties	Additional properties associated with the tool (e.g., {"user_location": {"city": "Seattle", "country": "US"}}).
****kwargs**	Additional keyword arguments to pass to the base class. if additional_properties is not provided, any kwargs will be added to additional_properties.

Examples

Python

```python
from agent_framework import HostedWebSearchTool

# Create a basic web search tool
search_tool = HostedWebSearchTool()

# With location context
search_tool_with_location = HostedWebSearchTool(
    description="Search the web for information",
    additional_properties={"user_location": {"city": "Seattle", "country": "US"}},
)
```

Methods

Expand table

__init__	Initialize a HostedWebSearchTool.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize a HostedWebSearchTool.

```
Python
__init__(description: str | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any)
```

Parameters

Expand table

Name	Description
description	str | None Default value: None
additional_properties	dict[str, Any] | None Default value: None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
description	A description of the tool.
additional_properties	Additional properties associated with the tool (e.g., {"user_location": {"city": "Seattle", "country": "US"}}).
****kwargs**	Additional keyword arguments to pass to the base class. if additional_properties is not provided, any kwargs will be added to additional_properties.

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}`

Name	Description
	Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

InMemoryCheckpointStorage Class

In-memory checkpoint storage for testing and development.

Initialize the memory storage.

Constructor

```
Python
InMemoryCheckpointStorage()
```

Methods

Expand table

__init__	Initialize the memory storage.
__new__	
delete_checkpoint	Delete a checkpoint by ID.
list_checkpoint_ids	List checkpoint IDs. If workflow_id is provided, filter by that workflow.
list_checkpoints	List checkpoint objects. If workflow_id is provided, filter by that workflow.
load_checkpoint	Load a checkpoint by ID.
save_checkpoint	Save a checkpoint and return its ID.

__init__

Initialize the memory storage.

```
Python
__init__() -> None
```

Returns

Expand table

Type	Description
None	

__new__

Python
`__new__(**kwargs)`

delete_checkpoint

Delete a checkpoint by ID.

Python
`async delete_checkpoint(checkpoint_id: str) -> bool`

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
bool	

list_checkpoint_ids

List checkpoint IDs. If workflow_id is provided, filter by that workflow.

Python
`async list_checkpoint_ids(workflow_id: str | None = None) -> list[str]`

Parameters

Expand table

Name	Description
`workflow_id`	str | None Default value: None

Returns

Expand table

Type	Description
list[str]	

list_checkpoints

List checkpoint objects. If workflow_id is provided, filter by that workflow.

Python

```
async list_checkpoints(workflow_id: str | None = None) -> list[WorkflowCheckpoint]
```

Parameters

Expand table

Name	Description
`workflow_id`	str | None Default value: None

Returns

Expand table

Type	Description
list[WorkflowCheckpoint]	

load_checkpoint

Load a checkpoint by ID.

```
Python
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
WorkflowCheckpoint | None	

save_checkpoint

Save a checkpoint and return its ID.

```
Python
async save_checkpoint(checkpoint: WorkflowCheckpoint) -> str
```

Parameters

Expand table

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

Expand table

Type	Description
str	

InProcRunnerContext Class

In-process execution context for local execution and optional checkpointing.

Initialize the in-process execution context.

Constructor

Python

```
InProcRunnerContext()
```

Parameters

Expand table

Name	Description
checkpoint_storage	Optional storage to enable checkpointing. Default value: None

Methods

Expand table

__init__	Initialize the in-process execution context.
__new__	
add_event	Add an event to the context immediately. Events are enqueued so runners can stream them in real time instead of waiting for superstep boundaries.
add_request_info_event	Add a RequestInfoEvent to the context and track it for correlation.
apply_checkpoint	Apply a checkpoint to the current context, mutating its state.
clear_runtime_checkpoint_storage	Clear runtime checkpoint storage override. This is called automatically by workflow execution methods after a run completes, ensuring runtime storage doesn't leak across runs.
create_checkpoint	

drain_events	Drain all currently queued events without blocking for new ones.
drain_messages	
get_pending_request_info_events	Get the mapping of request IDs to their corresponding RequestInfoEvent.
has_checkpointing	
has_events	
has_messages	
is_streaming	Check if the workflow is in streaming mode.
load_checkpoint	
next_event	Wait for and return the next event. Used by the runner to interleave event emission with ongoing iteration work.
reset_for_new_run	Reset the context for a new workflow run. This clears messages, events, and resets streaming flag. Runtime checkpoint storage is NOT cleared here as it's managed at the workflow level.
send_message	
send_request_info_response	Send a response correlated to a pending request.
set_runtime_checkpoint_storage	Set runtime checkpoint storage to override build-time configuration.
set_streaming	Set whether agents should stream incremental updates.
set_workflow_id	

__init__

Initialize the in-process execution context.

Python

```
__init__(checkpoint_storage: CheckpointStorage | None = None)
```

Parameters

Expand table

Name	Description
checkpoint_storage	CheckpointStorage | None Optional storage to enable checkpointing. Default value: None

__new__

```
Python
__new__(**kwargs)
```

add_event

Add an event to the context immediately.

Events are enqueued so runners can stream them in real time instead of waiting for superstep boundaries.

```
Python
async add_event(event: WorkflowEvent) -> None
```

Parameters

Expand table

Name	Description
event Required*	WorkflowEvent

Returns

Expand table

Type	Description
None	

add_request_info_event

Add a RequestInfoEvent to the context and track it for correlation.

Python

```
async add_request_info_event(event: RequestInfoEvent) -> None
```

Parameters

Expand table

Name	Description
event Required*	RequestInfoEvent The RequestInfoEvent to be added.

Returns

Expand table

Type	Description
None	

apply_checkpoint

Apply a checkpoint to the current context, mutating its state.

Python

```
async apply_checkpoint(checkpoint: WorkflowCheckpoint) -> None
```

Parameters

Expand table

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

Expand table

Type	Description
None	

clear_runtime_checkpoint_storage

Clear runtime checkpoint storage override.

This is called automatically by workflow execution methods after a run completes, ensuring runtime storage doesn't leak across runs.

Python

```
clear_runtime_checkpoint_storage() -> None
```

Returns

Expand table

Type	Description
None	

create_checkpoint

Python

```
async create_checkpoint(shared_state: SharedState, iteration_count: int, metadata:
dict[str, Any] | None = None) -> str
```

Parameters

Expand table

Name	Description
shared_state Required*	SharedState
iteration_count Required*	int
metadata	dict[str, Any] | None Default value: None

Returns

Expand table

Type	Description
str	

drain_events

Drain all currently queued events without blocking for new ones.

```
async drain_events() -> list[WorkflowEvent]
```

Returns

Expand table

Type	Description
list[WorkflowEvent]	

drain_messages

```
async drain_messages() -> dict[str, list[Message]]
```

Returns

Expand table

Type	Description
dict[str, list[Message]]	

get_pending_request_info_events

Get the mapping of request IDs to their corresponding RequestInfoEvent.

```
Python
```

```
async get_pending_request_info_events() -> dict[str, RequestInfoEvent]
```

Returns

Expand table

Type	Description
dict[str, RequestInfoEvent]	A dictionary mapping request IDs to their corresponding RequestInfoEvent.

has_checkpointing

Python

```
has_checkpointing() -> bool
```

Returns

Expand table

Type	Description
bool	

has_events

Python

```
async has_events() -> bool
```

Returns

Expand table

Type	Description
bool	

has_messages

Python

```
async has_messages() -> bool
```

Returns

Expand table

Type	Description
bool	

is_streaming

Check if the workflow is in streaming mode.

Python

```
is_streaming() -> bool
```

Returns

Expand table

Type	Description
bool	True if streaming mode is enabled, False otherwise.

load_checkpoint

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str

Returns

Expand table

Type	Description
WorkflowCheckpoint | None	

next_event

Wait for and return the next event.

Used by the runner to interleave event emission with ongoing iteration work.

Python

```
async next_event() -> WorkflowEvent
```

Returns

Expand table

Type	Description
WorkflowEvent	

reset_for_new_run

Reset the context for a new workflow run.

This clears messages, events, and resets streaming flag. Runtime checkpoint storage is NOT cleared here as it's managed at the workflow level.

Python

```
reset_for_new_run() -> None
```

Returns

Expand table

Type	Description
None	

send_message

Python

```
async send_message(message: Message) -> None
```

Parameters

Expand table

Name	Description
message Required*	Message

Returns

Expand table

Type	Description
None	

send_request_info_response

Send a response correlated to a pending request.

Python

```
async send_request_info_response(request_id: str, response: Any) -> None
```

Parameters

Expand table

Name	Description
request_id Required*	str The ID of the original request.
response Required*	Any The response data to be sent.

Returns

Expand table

Type	Description
None	

set_runtime_checkpoint_storage

Set runtime checkpoint storage to override build-time configuration.

Python

```
set_runtime_checkpoint_storage(storage: CheckpointStorage) -> None
```

Parameters

Expand table

Name	Description
storage Required*	CheckpointStorage The checkpoint storage to use for this run.

Returns

Expand table

Type	Description
None	

set_streaming

Set whether agents should stream incremental updates.

Python

```
set_streaming(streaming: bool) -> None
```

Parameters

Expand table

Name	Description
`streaming` Required*	bool True for streaming mode (run_stream), False for non-streaming (run).

Returns

Expand table

Type	Description
None	

set_workflow_id

Python

```
set_workflow_id(workflow_id: str) -> None
```

Parameters

Expand table

Name	Description
`workflow_id` Required*	str

Returns

Expand table

Type	Description
None	

MCPStdioTool Class

MCP tool for connecting to stdio-based MCP servers.

This class connects to MCP servers that communicate via standard input/output, typically used for local processes.

Initialize the MCP stdio tool.

> **Note**
>
> The arguments are used to create a StdioServerParameters object,
>
> which is then used to create a stdio client. See mcp.client.stdio.stdio_client
>
> and mcp.client.stdio.stdio_server_parameters for more details.

Constructor

Python

```
MCPStdioTool()
```

Parameters

Expand table

Name	Description
name Required*	The name of the tool.
command Required*	The command to run the MCP server.

Keyword-Only Parameters

Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
args	The arguments to pass to the command. Default value: None
env	The environment variables to set for the command. Default value: None
encoding	The encoding to use for the command output. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the stdio client.

Examples

Python

```python
from agent_framework import MCPStdioTool, ChatAgent

# Create an MCP stdio tool
mcp_tool = MCPStdioTool(
    name="filesystem",
    command="npx",
    args=["-y", "@modelcontextprotocol/server-filesystem", "/tmp"],
    description="File system operations",
)

# Use with a chat agent
async with mcp_tool:
    agent = ChatAgent(chat_client=client, name="assistant", tools=mcp_tool)
    response = await agent.run("List files in the directory")
```

Methods

Expand table

__init__	Initialize the MCP stdio tool. **Note** The arguments are used to create a StdioServerParameters object, which is then used to create a stdio client. See mcp.client.stdio.stdio_client and mcp.client.stdio.stdio_server_parameters for more details.
__new__	
call_tool	Call a tool with the given arguments.
close	Disconnect from the MCP server. Closes the connection and cleans up resources.
connect	Connect to the MCP server. Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.
get_mcp_client	Get an MCP stdio client.
get_prompt	Call a prompt with the given arguments.
load_prompts	Load prompts from the MCP server.

	Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.
load_tools	Load tools from the MCP server. Retrieves available tools from the connected MCP server and converts them into AIFunction instances.
logging_callback	Callback function for logging. This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params. **ⓘ Note** Subclass MCPTool and override this function if you want to adapt the behavior.
message_handler	Handle messages from the MCP server. By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received. **ⓘ Note** If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call super().message_handler(message).
sampling_callback	Callback function for sampling. This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses. **ⓘ Note** This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time, so overriding it is the best way to customize this behavior.

__init__

Initialize the MCP stdio tool.

> **Note**
>
> The arguments are used to create a StdioServerParameters object,
>
> which is then used to create a stdio client. See mcp.client.stdio.stdio_client
>
> and mcp.client.stdio.stdio_server_parameters for more details.

Python

```
__init__(name: str, command: str, *, load_tools: bool = True, load_prompts: bool = True, request_timeout: int | None = None, session: ClientSession | None = None, description: str | None = None, approval_mode: Literal['always_require', 'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools: Collection[str] | None = None, args: list[str] | None = None, env: dict[str, str] | None = None, encoding: str | None = None, chat_client: ChatClientProtocol | None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
name Required*	str The name of the tool.
command Required*	str The command to run the MCP server.
load_tools Required*	bool
load_prompts Required*	bool
request_timeout Required*	int | None
session Required*	<xref:mcp.client.session.ClientSession> | None

Name	Description
description Required*	str | None
approval_mode Required*	Literal['always_require', 'never_require'] | agent_framework._tools.HostedMCPSpecificApproval | None
allowed_tools Required*	Collection[str] | None
args Required*	list[str] | None
env Required*	dict[str, str] | None
encoding Required*	str | None
chat_client Required*	ChatClientProtocol | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None

Name	Description
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
args	The arguments to pass to the command. Default value: None
env	The environment variables to set for the command. Default value: None
encoding	The encoding to use for the command output. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the stdio client.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

call_tool

Call a tool with the given arguments.

Python

```
async call_tool(tool_name: str, **kwargs: Any) -> list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent]
```

Parameters

Expand table

Name	Description
tool_name Required*	str The name of the tool to call.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Arguments to pass to the tool.

Returns

Expand table

Type	Description
list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent]	A list of content items returned by the tool.

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, tools are not loaded, or the tool call fails.

close

Disconnect from the MCP server.

Closes the connection and cleans up resources.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

connect

Connect to the MCP server.

Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.

Python

```
async connect() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolException	If connection or session initialization fails.

get_mcp_client

Get an MCP stdio client.

Python

```
get_mcp_client() -> _AsyncGeneratorContextManager[Any, None]
```

Returns

Expand table

Type	Description
<xref:contextlib._AsyncGeneratorContextManager>[Any, None]	An async context manager for the stdio client transport.

get_prompt

Call a prompt with the given arguments.

Python

```
async get_prompt(prompt_name: str, **kwargs: Any) -> list[ChatMessage]
```

Parameters

Expand table

Name	Description
prompt_name Required*	str The name of the prompt to retrieve.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Arguments to pass to the prompt.

Returns

Expand table

Type	Description
list[ChatMessage]	A list of chat messages returned by the prompt.

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, prompts are not loaded, or the prompt call fails.

load_prompts

Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_prompts() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_tools() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

> ⓘ Note

Subclass MCPTool and override this function if you want to adapt the behavior.

Python

```
async logging_callback(params: LoggingMessageNotificationParams) -> None
```

Parameters

Expand table

Name	Description
params Required*	<xref:mcp.types.LoggingMessageNotificationParams> The logging message notification parameters from the MCP server.

Returns

Expand table

Type	Description
None	

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

> **Note**
>
> If you want to extend this behavior, you can subclass MCPTool and override
>
> this function. If you want to keep the default behavior, make sure to call
>
> super().message_handler(message).

Python

```
async message_handler(message: RequestResponder[ServerRequest, ClientResult] |
ServerNotification | Exception) -> None
```

Parameters

Expand table

Name	Description
message Required*	<xref:mcp.shared.session.RequestResponder>[<xref:mcp.types.ServerRequest>, <xref:mcp.types.ClientResult>] | <xref:mcp.types.ServerNotification> | Exception The message from the MCP server (request responder, notification, or exception).

Returns

Expand table

Type	Description
None	

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

> **Note**
>
> This is a simple version of this function. It can be overridden to allow
>
> more complex sampling. It gets added to the session at initialization time,
>
> so overriding it is the best way to customize this behavior.

Python

```
async sampling_callback(context: RequestContext[ClientSession, Any, Any], params:
CreateMessageRequestParams) -> CreateMessageResult | ErrorData
```

Parameters

Expand table

Name	Description
`context` Required*	<xref:mcp.shared.context.RequestContext>[<xref:mcp.client.session.ClientSession>, Any, Any] The request context from the MCP server.
`params` Required*	<xref:mcp.types.CreateMessageRequestParams> The message creation request parameters.

Returns

Expand table

Type	Description
<xref:mcp.types.CreateMessageResult> | <xref:mcp.types.ErrorData>	Either a CreateMessageResult with the generated message or ErrorData if generation fails.

Attributes

functions

Get the list of functions that are allowed.

MCPStreamableHTTPTool Class

MCP tool for connecting to HTTP-based MCP servers.

This class connects to MCP servers that communicate via streamable HTTP/SSE.

Initialize the MCP streamable HTTP tool.

> **Note**
>
> The arguments are used to create a streamable HTTP client.
>
> See mcp.client.streamable_http.streamablehttp_client for more details.
>
> Any extra arguments passed to the constructor will be passed to the
>
> streamable HTTP client constructor.

Constructor

Python

```
MCPStreamableHTTPTool()
```

Parameters

Expand table

Name	Description
name Required*	The name of the tool.
url Required*	The URL of the MCP server.

Keyword-Only Parameters

Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
headers	The headers to send with the request. Default value: None
timeout	The timeout for the request. Default value: None
sse_read_timeout	The timeout for reading from the SSE stream. Default value: None
terminate_on_close	Close the transport when the MCP client is terminated. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the SSE client.

Examples

Python

```python
from agent_framework import MCPStreamableHTTPTool, ChatAgent

# Create an MCP HTTP tool
mcp_tool = MCPStreamableHTTPTool(
    name="web-api",
    url="https://api.example.com/mcp",
    headers={"Authorization": "Bearer token"},
    description="Web API operations",
)

# Use with a chat agent
async with mcp_tool:
    agent = ChatAgent(chat_client=client, name="assistant", tools=mcp_tool)
    response = await agent.run("Fetch data from the API")
```

Methods

Expand table

__init__	Initialize the MCP streamable HTTP tool. **Note** The arguments are used to create a streamable HTTP client. See mcp.client.streamable_http.streamablehttp_client for more details. Any extra arguments passed to the constructor will be passed to the streamable HTTP client constructor.
__new__	
call_tool	Call a tool with the given arguments.
close	Disconnect from the MCP server. Closes the connection and cleans up resources.
connect	Connect to the MCP server. Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.
get_mcp_client	Get an MCP streamable HTTP client.

get_prompt	Call a prompt with the given arguments.
load_prompts	Load prompts from the MCP server. Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.
load_tools	Load tools from the MCP server. Retrieves available tools from the connected MCP server and converts them into AIFunction instances.
logging_callback	Callback function for logging. This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params. **ⓘ Note** Subclass MCPTool and override this function if you want to adapt the behavior.
message_handler	Handle messages from the MCP server. By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received. **ⓘ Note** If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call super().message_handler(message).
sampling_callback	Callback function for sampling. This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses. **ⓘ Note** This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time,

so overriding it is the best way to customize this behavior.

__init__

Initialize the MCP streamable HTTP tool.

> **Note**
>
> The arguments are used to create a streamable HTTP client.
>
> See mcp.client.streamable_http.streamablehttp_client for more details.
>
> Any extra arguments passed to the constructor will be passed to the
>
> streamable HTTP client constructor.

Python

```
__init__(name: str, url: str, *, load_tools: bool = True, load_prompts: bool =
True, request_timeout: int | None = None, session: ClientSession | None = None,
description: str | None = None, approval_mode: Literal['always_require',
'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools:
Collection[str] | None = None, headers: dict[str, Any] | None = None, timeout:
float | None = None, sse_read_timeout: float | None = None, terminate_on_close:
bool | None = None, chat_client: ChatClientProtocol | None = None,
additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
name Required*	str The name of the tool.
url Required*	str The URL of the MCP server.
load_tools Required*	bool
load_prompts Required*	bool

Name	Description
request_timeout Required*	int | None
session Required*	<xref:mcp.client.session.ClientSession> | None
description Required*	str | None
approval_mode Required*	Literal['always_require', 'never_require'] | agent_framework._tools.HostedMCPSpecificApproval | None
allowed_tools Required*	Collection[str] | None
headers Required*	dict[str, Any] | None
timeout Required*	float | None
sse_read_timeout Required*	float | None
terminate_on_close Required*	bool | None
chat_client Required*	ChatClientProtocol | None
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True

Name	Description
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
headers	The headers to send with the request. Default value: None
timeout	The timeout for the request. Default value: None
sse_read_timeout	The timeout for reading from the SSE stream. Default value: None
terminate_on_close	Close the transport when the MCP client is terminated. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the SSE client.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

call_tool

Call a tool with the given arguments.

```
Python
async call_tool(tool_name: str, **kwargs: Any) -> list[TextContent | DataContent |
TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent |
ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent |
FunctionApprovalRequestContent | FunctionApprovalResponseContent]
```

Parameters

Expand table

Name	Description
tool_name Required*	str The name of the tool to call.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Arguments to pass to the tool.

Returns

Expand table

Type	Description
list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent]	A list of content items returned by the tool.

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, tools are not loaded, or the tool call fails.

close

Disconnect from the MCP server.

Closes the connection and cleans up resources.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

connect

Connect to the MCP server.

Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.

Python

```
async connect() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolException	If connection or session initialization fails.

get_mcp_client

Get an MCP streamable HTTP client.

Python

```
get_mcp_client() -> _AsyncGeneratorContextManager[Any, None]
```

Returns

Expand table

Type	Description
<xref:contextlib._AsyncGeneratorContextManager>[Any, None]	An async context manager for the streamable HTTP client transport.

get_prompt

Call a prompt with the given arguments.

Python

```
async get_prompt(prompt_name: str, **kwargs: Any) -> list[ChatMessage]
```

Parameters

Expand table

Name	Description
prompt_name Required*	str The name of the prompt to retrieve.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Arguments to pass to the prompt.

Returns

Expand table

Type	Description
list[ChatMessage]	A list of chat messages returned by the prompt.

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, prompts are not loaded, or the prompt call fails.

load_prompts

Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_prompts() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

Python

```
async load_tools() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

> ⓘ Note
>
> Subclass MCPTool and override this function if you want to adapt the behavior.

Python

```
async logging_callback(params: LoggingMessageNotificationParams) -> None
```

Parameters

Expand table

Name	Description
params Required*	<xref:mcp.types.LoggingMessageNotificationParams> The logging message notification parameters from the MCP server.

Returns

Expand table

Type	Description
None	

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

> ⓘ Note
>
> If you want to extend this behavior, you can subclass MCPTool and override
>
> this function. If you want to keep the default behavior, make sure to call

super().message_handler(message).

Python

```
async message_handler(message: RequestResponder[ServerRequest, ClientResult] |
ServerNotification | Exception) -> None
```

Parameters

Expand table

Name	Description
message Required*	<xref:mcp.shared.session.RequestResponder>[<xref:mcp.types.ServerRequest>, <xref:mcp.types.ClientResult>] | <xref:mcp.types.ServerNotification> | Exception The message from the MCP server (request responder, notification, or exception).

Returns

Expand table

Type	Description
None	

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

> ⓘ Note
>
> This is a simple version of this function. It can be overridden to allow
>
> more complex sampling. It gets added to the session at initialization time,
>
> so overriding it is the best way to customize this behavior.

Python

```
async sampling_callback(context: RequestContext[ClientSession, Any, Any], params:
CreateMessageRequestParams) -> CreateMessageResult | ErrorData
```

Parameters

Expand table

Name	Description
context Required*	<xref:mcp.shared.context.RequestContext>[<xref:mcp.client.session.ClientSession>, Any, Any] The request context from the MCP server.
params Required*	<xref:mcp.types.CreateMessageRequestParams> The message creation request parameters.

Returns

Expand table

Type	Description
<xref:mcp.types.CreateMessageResult> | <xref:mcp.types.ErrorData>	Either a CreateMessageResult with the generated message or ErrorData if generation fails.

Attributes

functions

Get the list of functions that are allowed.

MCPWebsocketTool Class

MCP tool for connecting to WebSocket-based MCP servers.

This class connects to MCP servers that communicate via WebSocket.

Initialize the MCP WebSocket tool.

> **Note**
>
> The arguments are used to create a WebSocket client.
>
> See mcp.client.websocket.websocket_client for more details.
>
> Any extra arguments passed to the constructor will be passed to the
>
> WebSocket client constructor.

Constructor

Python

```
MCPWebsocketTool()
```

Parameters

Expand table

Name	Description
`name` Required*	The name of the tool.
`url` Required*	The URL of the MCP server.

Keyword-Only Parameters

Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
chat_client	The chat client to use for sampling. Default value: None
kwargs	Any extra arguments to pass to the WebSocket client.

Examples

Python

```
   from agent_framework import MCPWebsocketTool, ChatAgent

   # Create an MCP WebSocket tool
   mcp_tool = MCPWebsocketTool(
       name="realtime-service", url="wss://service.example.com/mcp",
description="Real-time service operations"
   )
```

```
    # Use with a chat agent
    async with mcp_tool:
        agent = ChatAgent(chat_client=client, name="assistant", tools=mcp_tool)
        response = await agent.run("Connect to the real-time service")
```

Methods

Expand table

__init__	Initialize the MCP WebSocket tool. **Note** The arguments are used to create a WebSocket client. See mcp.client.websocket.websocket_client for more details. Any extra arguments passed to the constructor will be passed to the WebSocket client constructor.
__new__	
call_tool	Call a tool with the given arguments.
close	Disconnect from the MCP server. Closes the connection and cleans up resources.
connect	Connect to the MCP server. Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.
get_mcp_client	Get an MCP WebSocket client.
get_prompt	Call a prompt with the given arguments.
load_prompts	Load prompts from the MCP server. Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.
load_tools	Load tools from the MCP server. Retrieves available tools from the connected MCP server and converts them into AIFunction instances.
logging_callback	Callback function for logging.

	This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params. **Note** Subclass MCPTool and override this function if you want to adapt the behavior.
message_handler	Handle messages from the MCP server. By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received. **Note** If you want to extend this behavior, you can subclass MCPTool and override this function. If you want to keep the default behavior, make sure to call super().message_handler(message).
sampling_callback	Callback function for sampling. This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses. **Note** This is a simple version of this function. It can be overridden to allow more complex sampling. It gets added to the session at initialization time, so overriding it is the best way to customize this behavior.

__init__

Initialize the MCP WebSocket tool.

Note

The arguments are used to create a WebSocket client.

See mcp.client.websocket.websocket_client for more details.

Any extra arguments passed to the constructor will be passed to the

WebSocket client constructor.

Python

```
__init__(name: str, url: str, *, load_tools: bool = True, load_prompts: bool =
True, request_timeout: int | None = None, session: ClientSession | None = None,
description: str | None = None, approval_mode: Literal['always_require',
'never_require'] | HostedMCPSpecificApproval | None = None, allowed_tools:
Collection[str] | None = None, chat_client: ChatClientProtocol | None = None,
additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
name Required*	str The name of the tool.
url Required*	str The URL of the MCP server.
load_tools Required*	bool
load_prompts Required*	bool
request_timeout Required*	int | None
session Required*	<xref:mcp.client.session.ClientSession> | None
description Required*	str | None
approval_mode Required*	Literal['always_require', 'never_require'] | agent_framework._tools.HostedMCPSpecificApproval | None
allowed_tools Required*	Collection[str] | None
chat_client	ChatClientProtocol | None

Name	Description
Required*	
additional_properties Required*	dict[str, Any] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
load_tools	Whether to load tools from the MCP server. Default value: True
load_prompts	Whether to load prompts from the MCP server. Default value: True
request_timeout	The default timeout in seconds for all requests. Default value: None
session	The session to use for the MCP connection. Default value: None
description	The description of the tool. Default value: None
approval_mode	The approval mode for the tool. This can be: • "always_require": The tool always requires approval before use. • "never_require": The tool never requires approval before use. • A dict with keys *always_require_approval* or *never_require_approval*, followed by a sequence of strings with the names of the relevant tools. A tool should not be listed in both, if so, it will require approval. Default value: None
allowed_tools	A list of tools that are allowed to use this tool. Default value: None
additional_properties	Additional properties. Default value: None
chat_client	The chat client to use for sampling. Default value: None

Name	Description
kwargs	Any extra arguments to pass to the WebSocket client.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

call_tool

Call a tool with the given arguments.

Python

```
async call_tool(tool_name: str, **kwargs: Any) -> list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent]
```

Parameters

Expand table

Name	Description
tool_name Required*	str The name of the tool to call.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Arguments to pass to the tool.

Returns

Expand table

Type	Description
list[TextContent | DataContent | TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent | FunctionApprovalResponseContent]	A list of content items returned by the tool.

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, tools are not loaded, or the tool call fails.

close

Disconnect from the MCP server.

Closes the connection and cleans up resources.

Python

```
async close() -> None
```

Returns

Expand table

Type	Description
None	

connect

Connect to the MCP server.

Establishes a connection to the MCP server, initializes the session, and loads tools and prompts if configured to do so.

```
Python
async connect() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolException	If connection or session initialization fails.

get_mcp_client

Get an MCP WebSocket client.

```
Python
get_mcp_client() -> _AsyncGeneratorContextManager[Any, None]
```

Returns

Expand table

Type	Description
<xref:contextlib._AsyncGeneratorContextManager>[Any, None]	An async context manager for the WebSocket client transport.

get_prompt

Call a prompt with the given arguments.

Python

```
async get_prompt(prompt_name: str, **kwargs: Any) -> list[ChatMessage]
```

Parameters

Expand table

Name	Description
prompt_name Required*	str The name of the prompt to retrieve.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Arguments to pass to the prompt.

Returns

Expand table

Type	Description
list[ChatMessage]	A list of chat messages returned by the prompt.

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected, prompts are not loaded, or the prompt call fails.

load_prompts

Load prompts from the MCP server.

Retrieves available prompts from the connected MCP server and converts them into AIFunction instances.

```
Python
async load_prompts() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

load_tools

Load tools from the MCP server.

Retrieves available tools from the connected MCP server and converts them into AIFunction instances.

```
Python
async load_tools() -> None
```

Returns

Expand table

Type	Description
None	

Exceptions

Expand table

Type	Description
ToolExecutionException	If the MCP server is not connected.

logging_callback

Callback function for logging.

This function is called when the MCP Server sends a log message. By default it will log the message to the logger with the level set in the params.

> **Note**
>
> Subclass MCPTool and override this function if you want to adapt the behavior.

Python

```
async logging_callback(params: LoggingMessageNotificationParams) -> None
```

Parameters

Expand table

Name	Description
params Required*	<xref:mcp.types.LoggingMessageNotificationParams> The logging message notification parameters from the MCP server.

Returns

Expand table

Type	Description
None	

message_handler

Handle messages from the MCP server.

By default this function will handle exceptions on the server by logging them, and it will trigger a reload of the tools and prompts when the list changed notification is received.

> ⓘ Note
>
> If you want to extend this behavior, you can subclass MCPTool and override
>
> this function. If you want to keep the default behavior, make sure to call
>
> super().message_handler(message).

Python

```
async message_handler(message: RequestResponder[ServerRequest, ClientResult] | ServerNotification | Exception) -> None
```

Parameters

Expand table

Name	Description
message Required*	<xref:mcp.shared.session.RequestResponder>[<xref:mcp.types.ServerRequest>, <xref:mcp.types.ClientResult>] | <xref:mcp.types.ServerNotification> | Exception The message from the MCP server (request responder, notification, or exception).

Returns

Expand table

Type	Description
None	

sampling_callback

Callback function for sampling.

This function is called when the MCP server needs to get a message completed. It uses the configured chat client to generate responses.

> ⓘ Note

This is a simple version of this function. It can be overridden to allow

more complex sampling. It gets added to the session at initialization time,

so overriding it is the best way to customize this behavior.

Python

```
async sampling_callback(context: RequestContext[ClientSession, Any, Any], params:
CreateMessageRequestParams) -> CreateMessageResult | ErrorData
```

Parameters

Expand table

Name	Description
context Required*	<xref:mcp.shared.context.RequestContext>[<xref:mcp.client.session.ClientSession>, Any, Any] The request context from the MCP server.
params Required*	<xref:mcp.types.CreateMessageRequestParams> The message creation request parameters.

Returns

Expand table

Type	Description
<xref:mcp.types.CreateMessageResult> | <xref:mcp.types.ErrorData>	Either a CreateMessageResult with the generated message or ErrorData if generation fails.

Attributes

functions

Get the list of functions that are allowed.

MagenticBuilder Class

Fluent builder for creating Magentic One multi-agent orchestration workflows.

Magentic One workflows use an LLM-powered manager to coordinate multiple agents through dynamic task planning, progress tracking, and adaptive replanning. The manager creates plans, selects agents, monitors progress, and determines when to replan or complete.

The builder provides a fluent API for configuring participants, the manager, optional plan review, checkpointing, and event callbacks.

Human-in-the-loop Support: Magentic provides specialized HITL mechanisms via:

- *.with_plan_review()* - Review and approve/revise plans before execution
- *.with_human_input_on_stall()* - Intervene when workflow stalls
- Tool approval via *FunctionApprovalRequestContent* - Approve individual tool calls

These emit *MagenticHumanInterventionRequest* events that provide structured decision options (APPROVE, REVISE, CONTINUE, REPLAN, GUIDANCE) appropriate for Magentic's planning-based orchestration.

Usage:

Python

```
from agent_framework import MagenticBuilder, StandardMagenticManager
from azure.ai.projects.aio import AIProjectClient

# Create manager with LLM client
project_client = AIProjectClient.from_connection_string(...)
chat_client = project_client.inference.get_chat_completions_client()

# Build Magentic workflow with agents
workflow = (
    MagenticBuilder()
    .participants(researcher=research_agent, writer=writing_agent,
coder=coding_agent)
    .with_standard_manager(chat_client=chat_client, max_round_count=20,
max_stall_count=3)
    .with_plan_review(enable=True)
    .with_checkpointing(checkpoint_storage)
    .build()
)

# Execute workflow
```

```
    async for message in workflow.run("Research and write article about AI agents"):
        print(message.text)
```

With custom manager:

Python

```
    # Create custom manager subclass
    class MyCustomManager(MagenticManagerBase):
        async def plan(self, context: MagenticContext) -> ChatMessage:
            # Custom planning logic
            ...


    manager = MyCustomManager()
    workflow = MagenticBuilder().participants(agent1=agent1,
agent2=agent2).with_standard_manager(manager).build()
```

Constructor

Python

```
MagenticBuilder()
```

Methods

Expand table

__init__	
__new__	
build	Build a Magentic workflow with the orchestrator and all agent executors.

participants	Add participant agents or executors to the Magentic workflow.
	
	Participants are the agents that will execute tasks under the manager's direction. Each participant should have distinct capabilities that complement the team. The manager will select which participant to invoke based on the current plan and progress state.
	
	Usage:
	
	Python
	
	`workflow = (`
	`MagenticBuilder()`

```
        .participants(
            researcher=research_agent, writer=writing_agent,
coder=coding_agent, reviewer=review_agent
        )
        .with_standard_manager(agent=manager_agent)
        .build()
    )
```

-[Notes]-

- Participant names become part of the manager's context for selection
- Agent descriptions (if available) are extracted and provided to the manager
- Can be called multiple times to add participants incrementally

start_with	Build a Magentic workflow and return a wrapper with convenience methods.
start_with_message	Build a Magentic workflow and return a wrapper with convenience methods for ChatMessage tasks.
start_with_string	Build a Magentic workflow and return a wrapper with convenience methods for string tasks.

with_checkpointing	Enable workflow state persistence using the provided checkpoint storage.
	
	Checkpointing allows workflows to be paused, resumed across process restarts, or recovered after failures. The entire workflow state including conversation history, task ledgers, and progress is persisted at key points.
	
	Usage:
	
	Python
	
	`from agent_framework import InMemoryCheckpointStorage`
	
	`storage = InMemoryCheckpointStorage()`
	`workflow = (`
	`MagenticBuilder()`
	`.participants(agent1=agent1)`
	`.with_standard_manager(agent=manager_agent)`
	`.with_checkpointing(storage)`
	`.build()`
	`)`
	
	`# First run`
	`thread_id = "task-123"`
	`async for msg in workflow.run("task", thread_id=thread_id):`
	`print(msg.text)`
	
	`# Resume from checkpoint`
	`async for msg in workflow.run("continue",`

```
thread_id=thread_id):
        print(msg.text)
```

-[Notes]-

- Checkpoints are created after each significant state transition
- Thread ID must be consistent across runs to resume properly
- Storage implementations may have different persistence guarantees

with_human_input_on_stall	Enable human intervention when the workflow detects a stall. When enabled, instead of automatically replanning when the workflow detects that agents are not making progress or are stuck in a loop, the workflow will pause and emit a MagenticStallInterventionRequest event. A human can then decide to continue, trigger replanning, or provide guidance. This is useful for: • Workflows where automatic replanning may not resolve the issue • Scenarios requiring human judgment about workflow direction • Debugging stuck workflows with human insight • Complex tasks where human guidance can help agents get back on track When stall detection triggers (based on max_stall_count), instead of calling _reset_and_replan automatically, the workflow will: 1. Emit a MagenticHumanInterventionRequest with kind=STALL 2. Wait for human response via send_responses_streaming 3. Take action based on the human's decision (continue, replan, or guidance) Usage: **Python** `workflow = (MagenticBuilder() .participants(agent1=agent1) .with_standard_manager(agent=manager_agent, max_stall_count=3) .with_human_input_on_stall(enable=True) .build()) # During execution, handle human intervention requests async for event in workflow.run_stream("task"): if isinstance(event, RequestInfoEvent): if event.request_type is MagenticHumanInterventionRequest: request = event.data if request.kind == MagenticHumanInterventionKind.STALL:`

```
                    print(f"Workflow stalled:
{request.stall_reason}")
                    reply = MagenticHumanInterventionReply(

decision=MagenticHumanInterventionDecision.GUIDANCE,
                        comments="Focus on completing the
current step first",
                    )
                    responses = {event.request_id: reply}
                    async for ev in
workflow.send_responses_streaming(responses):
                        ...
```

with_plan_review	Enable or disable human-in-the-loop plan review before task execution.
	
	When enabled, the workflow will pause after the manager generates the initial plan and emit a MagenticHumanInterventionRequest event with kind=PLAN_REVIEW. A human reviewer can then approve, request revisions, or reject the plan. The workflow continues only after approval.
	
	This is useful for:
	
	• High-stakes tasks requiring human oversight
	• Validating the manager's understanding of requirements
	• Catching hallucinations or unrealistic plans early
	• Educational scenarios where learners review AI planning
	
	Usage:
	
	Python
	
	`workflow = ( MagenticBuilder() .participants(agent1=agent1) .with_standard_manager(agent=manager_agent) .with_plan_review(enable=True) .build() )`
	
	`# During execution, handle plan review async for event in workflow.run_stream("task"): if isinstance(event, RequestInfoEvent): request = event.data if isinstance(request, MagenticHumanInterventionRequest): if request.kind == MagenticHumanInterventionKind.PLAN_REVIEW: # Review plan and respond reply = MagenticHumanInterventionReply(decision=MagenticHumanInterventionDecision.APPROVE)`

```
                    await
workflow.send_responses({event.request_id: reply})
```

with_standard_manager	Configure the workflow manager for task planning and agent coordination.
	
	The manager is responsible for creating plans, selecting agents, tracking progress, and deciding when to replan or complete. This method supports two usage patterns:
	
	1. **Provide existing manager**: Pass a pre-configured manager instance (custom or standard) for full control over behavior
	2. **Auto-create with agent**: Pass an agent to automatically create a StandardMagenticManager that uses the agent's configured instructions and options (temperature, seed, etc.)
	
	Usage with agent (recommended):
	
	Python
	
	`from agent_framework import ChatAgent, ChatOptions`
	`from agent_framework.openai import OpenAIChatClient`
	
	`# Configure manager agent with specific options and instructions`
	`manager_agent = ChatAgent(`
	`name="Coordinator",`
	`chat_client=OpenAIChatClient(model_id="gpt-4o"),`
	`chat_options=ChatOptions(temperature=0.3, seed=42),`
	`instructions="Be concise and focus on accuracy",`
	`)`
	
	`workflow = (`
	`MagenticBuilder()`
	`.participants(agent1=agent1, agent2=agent2)`
	`.with_standard_manager(`
	`agent=manager_agent,`
	`max_round_count=20,`
	`max_stall_count=3,`
	`)`
	`.build()`
	`)`
	
	Usage with custom manager:
	
	Python
	
	`class MyManager(MagenticManagerBase):`
	`async def plan(self, context: MagenticContext) -> ChatMessage:`
	`# Custom planning logic`
	`return ChatMessage(role=Role.ASSISTANT,`

```
text="...")


   manager = MyManager()
   workflow =
MagenticBuilder().participants(agent1=agent1).with_standard_
manager(manager).build()
```

Usage with prompt customization:

Python

```
   workflow = (
       MagenticBuilder()
       .participants(coder=coder_agent,
reviewer=reviewer_agent)
       .with_standard_manager(
           agent=manager_agent,
           task_ledger_plan_prompt="Create a detailed step-
by-step plan...",
           progress_ledger_prompt="Assess progress and
decide next action...",
           max_stall_count=2,
       )
       .build()
   )
```

-[Notes]-

- StandardMagenticManager uses structured LLM calls for all decisions
- Custom managers can implement alternative selection strategies
- Prompt templates support Jinja2-style variable substitution
- Stall detection helps prevent infinite loops in stuck scenarios
- The agent's instructions are used as system instructions for all manager prompts

__init__

Python

```
__init__() -> None
```

Returns

Expand table

Type	Description
None	

__new__

Python
`__new__(**kwargs)`

build

Build a Magentic workflow with the orchestrator and all agent executors.

Python
`build() -> Workflow`

Returns

Expand table

Type	Description
Workflow	

participants

Add participant agents or executors to the Magentic workflow.

Participants are the agents that will execute tasks under the manager's direction. Each participant should have distinct capabilities that complement the team. The manager will select which participant to invoke based on the current plan and progress state.

Usage:

Python

```
   workflow = (
       MagenticBuilder()
       .participants(
           researcher=research_agent, writer=writing_agent, coder=coding_agent,
reviewer=review_agent
       )
```

```
        .with_standard_manager(agent=manager_agent)
        .build()
    )
```

-[Notes]-

- Participant names become part of the manager's context for selection
- Agent descriptions (if available) are extracted and provided to the manager
- Can be called multiple times to add participants incrementally

Python

```
participants(**participants: AgentProtocol | Executor) -> Self
```

Parameters

Expand table

Name	Description
`**participants` Required*	AgentProtocol | Executor Named agents or executors to add to the workflow. Names should be descriptive of the agent's role (e.g., researcher=research_agent). Accepts BaseAgent instances or custom Executor implementations.

Returns

Expand table

Type	Description
Self	Self for method chaining

start_with

Build a Magentic workflow and return a wrapper with convenience methods.

Python

```
start_with(task: str | ChatMessage) -> MagenticWorkflow
```

Parameters

Expand table

Name	Description
task Required*	str | ChatMessage The task description as a string or ChatMessage.

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._magentic.MagenticWorkflow>	A MagenticWorkflow wrapper that provides convenience methods.

start_with_message

Build a Magentic workflow and return a wrapper with convenience methods for ChatMessage tasks.

Python

```
start_with_message(task: ChatMessage) -> MagenticWorkflow
```

Parameters

Expand table

Name	Description
task Required*	ChatMessage The task as a ChatMessage.

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._magentic.MagenticWorkflow>	A MagenticWorkflow wrapper that provides convenience methods.

start_with_string

Build a Magentic workflow and return a wrapper with convenience methods for string tasks.

Python

```
start_with_string(task: str) -> MagenticWorkflow
```

Parameters

Expand table

Name	Description
task Required*	str The task description as a string.

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._magentic.MagenticWorkflow>	A MagenticWorkflow wrapper that provides convenience methods for starting with strings.

with_checkpointing

Enable workflow state persistence using the provided checkpoint storage.

Checkpointing allows workflows to be paused, resumed across process restarts, or recovered after failures. The entire workflow state including conversation history, task ledgers, and progress is persisted at key points.

Usage:

Python

```
from agent_framework import InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent)
    .with_checkpointing(storage)
    .build()
)
```

```python
# First run
thread_id = "task-123"
async for msg in workflow.run("task", thread_id=thread_id):
    print(msg.text)

# Resume from checkpoint
async for msg in workflow.run("continue", thread_id=thread_id):
    print(msg.text)
```

-[Notes]-

- Checkpoints are created after each significant state transition
- Thread ID must be consistent across runs to resume properly
- Storage implementations may have different persistence guarantees

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> MagenticBuilder
```

Parameters

Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage Storage backend for checkpoints (e.g., InMemoryCheckpointStorage, FileCheckpointStorage, or custom implementations)

Returns

Expand table

Type	Description
MagenticBuilder	Self for method chaining

with_human_input_on_stall

Enable human intervention when the workflow detects a stall.

When enabled, instead of automatically replanning when the workflow detects that agents are not making progress or are stuck in a loop, the workflow will pause and emit a

MagenticStallInterventionRequest event. A human can then decide to continue, trigger replanning, or provide guidance.

This is useful for:

- Workflows where automatic replanning may not resolve the issue
- Scenarios requiring human judgment about workflow direction
- Debugging stuck workflows with human insight
- Complex tasks where human guidance can help agents get back on track

When stall detection triggers (based on max_stall_count), instead of calling _reset_and_replan automatically, the workflow will:

1. Emit a MagenticHumanInterventionRequest with kind=STALL
2. Wait for human response via send_responses_streaming
3. Take action based on the human's decision (continue, replan, or guidance)

Usage:

Python

```
workflow = (
    MagenticBuilder()
    .participants(agent1=agent1)
    .with_standard_manager(agent=manager_agent, max_stall_count=3)
    .with_human_input_on_stall(enable=True)
    .build()
)

# During execution, handle human intervention requests
async for event in workflow.run_stream("task"):
    if isinstance(event, RequestInfoEvent):
        if event.request_type is MagenticHumanInterventionRequest:
            request = event.data
            if request.kind == MagenticHumanInterventionKind.STALL:
                print(f"Workflow stalled: {request.stall_reason}")
                reply = MagenticHumanInterventionReply(
                    decision=MagenticHumanInterventionDecision.GUIDANCE,
                    comments="Focus on completing the current step first",
                )
                responses = {event.request_id: reply}
                async for ev in workflow.send_responses_streaming(responses):
                    ...
```

Python

```
with_human_input_on_stall(enable: bool = True) -> MagenticBuilder
```

Parameters

Expand table

Name	Description
`enable`	bool Whether to enable stall intervention (default True) Default value: True

Returns

Expand table

Type	Description
MagenticBuilder	Self for method chaining

See also

- <xref:agent_framework.MagenticHumanInterventionRequest>: Unified request type
- MagenticHumanInterventionDecision: Decision options
- with_standard_manager: Configure max_stall_count for stall detection

with_plan_review

Enable or disable human-in-the-loop plan review before task execution.

When enabled, the workflow will pause after the manager generates the initial plan and emit a MagenticHumanInterventionRequest event with kind=PLAN_REVIEW. A human reviewer can then approve, request revisions, or reject the plan. The workflow continues only after approval.

This is useful for:

- High-stakes tasks requiring human oversight
- Validating the manager's understanding of requirements
- Catching hallucinations or unrealistic plans early
- Educational scenarios where learners review AI planning

Usage:

Python

```
    workflow = (
        MagenticBuilder()
        .participants(agent1=agent1)
        .with_standard_manager(agent=manager_agent)
        .with_plan_review(enable=True)
        .build()
    )

    # During execution, handle plan review
    async for event in workflow.run_stream("task"):
        if isinstance(event, RequestInfoEvent):
            request = event.data
            if isinstance(request, MagenticHumanInterventionRequest):
                if request.kind == MagenticHumanInterventionKind.PLAN_REVIEW:
                    # Review plan and respond
                    reply =
MagenticHumanInterventionReply(decision=MagenticHumanInterventionDecision.APPROVE)
                    await workflow.send_responses({event.request_id: reply})
```

Python

```
with_plan_review(enable: bool = True) -> MagenticBuilder
```

Parameters

Expand table

Name	Description
enable	bool Whether to require plan review (default True) Default value: True

Returns

Expand table

Type	Description
MagenticBuilder	Self for method chaining

See also

- <xref:agent_framework.MagenticHumanInterventionRequest>: Event emitted for review
- <xref:agent_framework.MagenticHumanInterventionReply>: Response to send back

- MagenticHumanInterventionDecision: APPROVE/REVISE options

with_standard_manager

Configure the workflow manager for task planning and agent coordination.

The manager is responsible for creating plans, selecting agents, tracking progress, and deciding when to replan or complete. This method supports two usage patterns:

1. **Provide existing manager**: Pass a pre-configured manager instance (custom or standard) for full control over behavior
2. **Auto-create with agent**: Pass an agent to automatically create a StandardMagenticManager that uses the agent's configured instructions and options (temperature, seed, etc.)

Usage with agent (recommended):

Python

```
from agent_framework import ChatAgent, ChatOptions
from agent_framework.openai import OpenAIChatClient

# Configure manager agent with specific options and instructions
manager_agent = ChatAgent(
    name="Coordinator",
    chat_client=OpenAIChatClient(model_id="gpt-4o"),
    chat_options=ChatOptions(temperature=0.3, seed=42),
    instructions="Be concise and focus on accuracy",
)

workflow = (
    MagenticBuilder()
    .participants(agent1=agent1, agent2=agent2)
    .with_standard_manager(
        agent=manager_agent,
        max_round_count=20,
        max_stall_count=3,
    )
    .build()
)
```

Usage with custom manager:

Python

```
   class MyManager(MagenticManagerBase):
       async def plan(self, context: MagenticContext) -> ChatMessage:
           # Custom planning logic
           return ChatMessage(role=Role.ASSISTANT, text="...")


   manager = MyManager()
   workflow =
MagenticBuilder().participants(agent1=agent1).with_standard_manager(manager).build
()
```

Usage with prompt customization:

Python

```
   workflow = (
       MagenticBuilder()
       .participants(coder=coder_agent, reviewer=reviewer_agent)
       .with_standard_manager(
           agent=manager_agent,
           task_ledger_plan_prompt="Create a detailed step-by-step plan...",
           progress_ledger_prompt="Assess progress and decide next action...",
           max_stall_count=2,
       )
       .build()
   )
```

-[Notes]-

- StandardMagenticManager uses structured LLM calls for all decisions
- Custom managers can implement alternative selection strategies
- Prompt templates support Jinja2-style variable substitution
- Stall detection helps prevent infinite loops in stuck scenarios
- The agent's instructions are used as system instructions for all manager prompts

Python

```
with_standard_manager(manager: MagenticManagerBase | None = None, *, agent:
AgentProtocol | None = None, task_ledger: _MagenticTaskLedger | None = None,
task_ledger_facts_prompt: str | None = None, task_ledger_plan_prompt: str | None =
None, task_ledger_full_prompt: str | None = None, task_ledger_facts_update_prompt:
str | None = None, task_ledger_plan_update_prompt: str | None = None,
progress_ledger_prompt: str | None = None, final_answer_prompt: str | None = None,
```

```
max_stall_count: int = 3, max_reset_count: int | None = None, max_round_count: int
| None = None) -> Self
```

Parameters

Expand table

Name	Description
manager	MagenticManagerBase | None Pre-configured manager instance (StandardMagenticManager or custom MagenticManagerBase subclass). If provided, all other arguments are ignored. Default value: None
agent Required*	AgentProtocol | None Agent instance for generating plans and decisions. The agent's configured instructions and options (temperature, seed, etc.) will be applied.
task_ledger Required*	<xref:agent_framework._workflows._magentic._MagenticTaskLedger> | None Optional custom task ledger implementation for specialized prompting or structured output requirements
task_ledger_facts_prompt Required*	str | None Custom prompt template for extracting facts from task description
task_ledger_plan_prompt Required*	str | None Custom prompt template for generating initial plan
task_ledger_full_prompt Required*	str | None Custom prompt template for complete task ledger (facts + plan combined)
task_ledger_facts_update_prompt Required*	str | None Custom prompt template for updating facts based on agent progress
task_ledger_plan_update_prompt Required*	str | None Custom prompt template for replanning when needed
progress_ledger_prompt Required*	str | None Custom prompt template for assessing progress and determining next actions
final_answer_prompt Required*	str | None Custom prompt template for synthesizing final response when task is complete

Name	Description
max_stall_count Required*	int Maximum consecutive rounds without progress before triggering replan (default 3). Set to 0 to disable stall detection.
max_reset_count Required*	int | None Maximum number of complete resets allowed before failing. None means unlimited resets.
max_round_count Required*	int | None Maximum total coordination rounds before stopping with partial result. None means unlimited rounds.

Keyword-Only Parameters

Expand table

Name	Description
agent	Default value: None
task_ledger	Default value: None
task_ledger_facts_prompt	Default value: None
task_ledger_plan_prompt	Default value: None
task_ledger_full_prompt	Default value: None
task_ledger_facts_update_prompt	Default value: None
task_ledger_plan_update_prompt	Default value: None
progress_ledger_prompt	Default value: None
final_answer_prompt	Default value: None
max_stall_count	Default value: 3
max_reset_count	Default value: None
max_round_count	Default value: None

Returns

Expand table

Type	Description
Self	Self for method chaining

Exceptions

Expand table

Type	Description
ValueError	If manager is None and agent is not provided.

MagenticContext Class

Context for the Magentic manager.

Constructor

Python

```
MagenticContext()
```

Parameters

Expand table

Name	Description
task Required*	
chat_history	Default value: <factory>
participant_descriptions	Default value: <factory>
round_count	Default value: 0
stall_count	Default value: 0
reset_count	Default value: 0

Methods

Expand table

__init__	
__new__	
clone	
from_dict	
from_json	
reset	Reset the context.

	This will clear the chat history and reset the stall count. This will not reset the task, round count, or participant descriptions.
to_dict	
to_json	

__init__

Python

```
__init__(task: agent_framework._types.ChatMessage, chat_history:
list[agent_framework._types.ChatMessage] = <factory>, participant_descriptions:
dict[str, str] = <factory>, round_count: int = 0, stall_count: int = 0,
reset_count: int = 0) -> None
```

Parameters

Expand table

Name	Description
task Required*	ChatMessage
chat_history	list[ChatMessage] Default value: <factory>
participant_descriptions	dict[str, str] Default value: <factory>
round_count	int Default value: 0
stall_count	int Default value: 0
reset_count	int Default value: 0

Returns

Expand table

Type	Description
None	

__new__

```python
__new__(**kwargs)
```

clone

```python
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

```python
from_dict(data: dict[str, Any]) -> MagenticContext
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
MagenticContext	

from_json

Python
`from_json(raw: str) -> TModel`

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

reset

Reset the context.

This will clear the chat history and reset the stall count. This will not reset the task, round count, or participant descriptions.

```python
reset() -> None
```

Returns

Expand table

Type	Description
None	

to_dict

```python
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

```python
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

chat_history

Python

```
chat_history: list[ChatMessage]
```

participant_descriptions

Python

```
participant_descriptions: dict[str, str]
```

reset_count

Python

```
reset_count: int = 0
```

round_count

Python

```
round_count: int = 0
```

stall_count

Python

```
stall_count: int = 0
```

task

Python

```
task: ChatMessage
```

MagenticHumanInterventionDecision Enum

Decision options for human intervention responses.

Fields

Expand table

APPROVE
CONTINUE
GUIDANCE
REJECT
REPLAN
REVISE

MagenticHumanInterventionKind Enum

The kind of human intervention being requested.

Fields

Expand table

PLAN_REVIEW
STALL
TOOL_APPROVAL

MagenticManagerBase Class

Base class for the Magentic One manager.

Constructor

```
Python
MagenticManagerBase()
```

Keyword-Only Parameters

Expand table

Name	Description
max_stall_count	Default value: 3
max_reset_count	Default value: None
max_round_count	Default value: None

Methods

Expand table

__init__	
__new__	
create_progress_ledger	Create a progress ledger.
on_checkpoint_restore	Restore runtime state from checkpoint data.
on_checkpoint_save	Serialize runtime state for checkpointing.
plan	Create a plan for the task.
prepare_final_answer	Prepare the final answer.
replan	Replan for the task.

__init__

Python

```
__init__(*, max_stall_count: int = 3, max_reset_count: int | None = None,
max_round_count: int | None = None) -> None
```

Parameters

Expand table

Name	Description
max_stall_count Required*	int
max_reset_count Required*	int | None
max_round_count Required*	int | None

Keyword-Only Parameters

Expand table

Name	Description
max_stall_count	Default value: 3
max_reset_count	Default value: None
max_round_count	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

create_progress_ledger

Create a progress ledger.

Python

```
abstract async create_progress_ledger(magentic_context: MagenticContext) ->
_MagenticProgressLedger
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._magentic._MagenticProgressLedger>	

on_checkpoint_restore

Restore runtime state from checkpoint data.

Python

```
on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any]

Returns

Expand table

Type	Description
None	

on_checkpoint_save

Serialize runtime state for checkpointing.

```
Python
on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

plan

Create a plan for the task.

```
Python
abstract async plan(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
ChatMessage	

prepare_final_answer

Prepare the final answer.

Python

```
abstract async prepare_final_answer(magentic_context: MagenticContext) ->
ChatMessage
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
ChatMessage	

replan

Replan for the task.

Python

```
abstract async replan(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

Expand table

Name	Description
`magentic_context` Required*	MagenticContext

Returns

Expand table

Type	Description
ChatMessage	

ManagerDirectiveModel Class

Pydantic model for structured manager directive output.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python

```
pydantic model ManagerDirectiveModel
```

Keyword-Only Parameters

Expand table

Name	Description
next_agent	Default value: None
message	
finish	Default value: False
final_response	Default value: None

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. *self* is explicitly positional-only to allow *self* as a field name.
__new__	

construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_orm	
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump* Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.

model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

construct

```
Python
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool

Name	Description
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_orm

Python

```
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].

Name	Description
`fallback` Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
`serialize_as_any` Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
`indent`	Default value: None
`ensure_ascii`	Default value: False
`include`	Default value: None
`exclude`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`exclude_unset`	Default value: False
`exclude_defaults`	Default value: False
`exclude_none`	Default value: False
`exclude_computed_fields`	Default value: False
`round_trip`	Default value: False
`warnings`	Default value: True
`fallback`	Default value: None
`serialize_as_any`	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • '*any_of*': Use the *anyOf* keyword to combine schemas (the default). • '*primitive_type_array*': Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
mode	Literal['validation', 'serialization'] The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
union_format	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
params Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.
by_alias	bool | None

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
`json_data` Required*	str | bytes | bytearray The JSON data to validate.
`strict` Required*	bool | None Whether to enforce types strictly.
`extra` Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
`context` Required*	Any | None Extra variables to pass to the validator.
`by_alias` Required*	bool | None Whether to use the field's alias when validating against the provided input data.
`by_name` Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
`strict`	Default value: None
`extra`	Default value: None
`context`	Default value: None
`by_alias`	Default value: None
`by_name`	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name	bool | None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>
content_type Required*	str | None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python

```
parse_obj(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj	Any

Name	Description
Required*	

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

```
Python
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
Python
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

final_response

Final answer to the user when finish is true

Python

```
field final_response: str | None = None
```

finish

Whether the task is complete

Python

```
field finish: bool = False
```

message

Instruction for the selected participant

Python

```
field message: str = ''
```

next_agent

Name of the participant who should act next (null when finish is true)

Python

```
field next_agent: str | None = None
```

ManagerSelectionRequest Class

Request sent to manager agent for next speaker selection.

This dataclass packages the full conversation state and task context for the manager agent to analyze and make a speaker selection decision.

Constructor

Python

```
ManagerSelectionRequest()
```

Parameters

Expand table

Name	Description
task Required*	
participants Required*	
conversation Required*	
round_index Required*	
metadata	Default value: None

Methods

Expand table

__init__	
__new__	
to_dict	Convert to dictionary for serialization.

__init__

```
Python
__init__(task: ChatMessage, participants: dict[str, str], conversation:
list[ChatMessage], round_index: int, metadata: dict[str, Any] | None = None) ->
None
```

Parameters

Expand table

Name	Description
task Required*	ChatMessage
participants Required*	dict[str, str]
conversation Required*	list[ChatMessage]
round_index Required*	int
metadata	dict[str, Any] | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

to_dict

Convert to dictionary for serialization.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Attributes

task

Original user task message

Python

```
task: ChatMessage
```

participants

Mapping of participant names to their descriptions

Python

```
participants: dict[str, str]
```

conversation

Full conversation history including all messages

Python

```
conversation: list[ChatMessage]
```

round_index

Number of manager selection rounds completed so far

Python

```
round_index: int
```

metadata

Optional metadata for extensibility

Python

```
metadata: dict[str, Any] | None = None
```

ManagerSelectionResponse Class

Response from manager agent with speaker selection decision.

The manager agent must produce this structure (or compatible dict/JSON) to communicate its decision back to the orchestrator.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python

```
pydantic model ManagerSelectionResponse
```

Keyword-Only Parameters

Expand table

Name	Description
`selected_participant`	Default value: None
`instruction`	Default value: None
`finish`	Default value: False
`final_message`	Default value: None

Methods

Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

	self is explicitly positional-only to allow *self* as a field name.
__new__	
construct	
copy	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use *model_copy* instead. If you need *include* or *exclude*, use: `python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`
dict	
from_dict	Create from dictionary representation.
from_orm	
get_final_message_as_chat_message	Convert final_message string to ChatMessage if present.
json	
model_construct	Creates a new instance of the *Model* class with validated data. Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" *model_copy* Returns a copy of the model. !!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" *model_dump*

	Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" *model_dump_json* Generates a JSON representation of the model using Pydantic's *to_json* method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
model_post_init	Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

__init__

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None Default value: None
values Required*	Any

Returns

Expand table

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

`python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)`

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> | <xref:MappingIntStrAny> | None Optional set or mapping specifying which fields to exclude in the copied model.

Name	Description
update Required*	Dict[str, Any] | None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str,
Any]
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

Expand table

Type	Description
Dict[str, Any]	

from_dict

Create from dictionary representation.

```
Python
static from_dict(data: dict[str, Any]) -> ManagerSelectionResponse
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
ManagerSelectionResponse	

from_orm

```
Python
from_orm(obj: Any) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

get_final_message_as_chat_message

Convert final_message string to ChatMessage if present.

Python

```
get_final_message_as_chat_message() -> ChatMessage | None
```

Returns

Expand table

Type	Description
ChatMessage | None	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None

Name	Description
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] | None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

Expand table

Name	Description
_fields_set	set[str] | None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the [*model_fields_set*][pydantic.BaseModel.model_fields_set] attribute. Otherwise, the field names from the *values* argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

Expand table

Type	Description
Self	A new instance of the *Model* class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

Expand table

Name	Description
update Required*	Mapping[str, Any] | None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to *True* to make a deep copy of the model.

Keyword-Only Parameters

Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Expand table

Name	Description
mode Required*	Literal['json', 'python'] | str The mode in which *to_python* should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None A set of fields to include in the output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None

Name	Description
	A set of fields to exclude from the output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip Required*	bool If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
mode	Default value: 'python'
include	Default value: None

Name	Description
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
dict[str, Any]	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
```

```
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

Expand table

Name	Description
indent Required*	int | None Indentation to use in the JSON output. If None is passed, the output will be compact.
ensure_ascii Required*	bool If *True*, the output is guaranteed to have all incoming non-ASCII characters escaped. If *False* (the default), these characters will be output as-is.
include Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to include in the JSON output.
exclude Required*	set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, <xref:IncEx> | bool] | Mapping[str, <xref:IncEx> | bool] | bool] | None Field(s) to exclude from the JSON output.
context Required*	Any | None Additional context to pass to the serializer.
by_alias Required*	bool | None Whether to serialize using field aliases.
exclude_unset Required*	bool Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	bool Whether to exclude fields that are set to their default value.
exclude_none Required*	bool Whether to exclude fields that have a value of *None*.
exclude_computed_fields Required*	bool Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated *round_trip* parameter instead.
round_trip	bool

Name	Description
Required*	If True, dumped values should be valid as input for non-idempotent types such as Json[T].
warnings Required*	bool | Literal['none', 'warn', 'error'] How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError].
fallback Required*	Callable[[Any], Any] | None A function to call when an unknown value is encountered. If not provided, a [*PydanticSerializationError*] [pydantic_core.PydanticSerializationError] error is raised.
serialize_as_any Required*	bool Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

Expand table

Type	Description
str	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: • *'any_of'*: Use the *anyOf* keyword to combine schemas (the default). • *'primitive_type_array'*: Use the *type* keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (*string*, *boolean*, *null*, *integer* or *number*) or contains constraints/metadata, falls back to *any_of*.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>]

Name	Description
	To override the logic used to generate the JSON schema, as a subclass of *GenerateJsonSchema* with your desired modifications Default value: GenerateJsonSchema
`mode`	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

Expand table

Name	Description
`union_format`	Default value: 'any_of'

Returns

Expand table

Type	Description
dict[str, Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

Expand table

Name	Description
`params` Required*	tuple[type[Any], ...] Tuple of types of the class. Given a generic class *Model* with 2 type variables and a concrete model *Model[str, int]*, the value *(str, int)* would be passed to *params*.

Returns

Expand table

Type	Description
str	String representing the new class where *params* are passed to *cls* as type variables.

Exceptions

Expand table

Type	Description
TypeError	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after ***init*** and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

Expand table

Name	Description
context Required*	

Parameters

Expand table

Name	Description
context Required*	Any

Returns

Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
None) -> bool | None
```

Parameters

Expand table

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to *False*.
raise_errors Required*	bool Whether to raise errors, defaults to *True*.
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> | None The types namespace, defaults to *None*.

Keyword-Only Parameters

Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True

Name	Description
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

Expand table

Type	Description
bool | None	Returns *None* if the schema is already "complete" and rebuilding was not required. If rebuilding *was* required, returns *True* if rebuilding was successful, otherwise *False*.

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool | None Whether to extract data from object attributes.
context Required*	Any | None Additional context to pass to the validator.

Name	Description
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated model instance.

Exceptions

Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" JSON Parsing

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None =
None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any |
None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
json_data Required*	str | bytes | bytearray The JSON data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias Required*	bool | None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

Expand table

Type	Description
ValidationError	If *json_data* is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool | None Whether to enforce types strictly.
extra Required*	Literal['allow', 'ignore', 'forbid'] | None Whether to ignore, allow, or forbid extra data during model validation. See the [*extra* configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any | None Extra variables to pass to the validator.
by_alias	bool | None

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool | None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

Expand table

Name	Description
path Required*	str | <xref:Path>

Name	Description
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

parse_obj

Python
`parse_obj(obj: Any) -> Self`

Parameters

Expand table

Name	Description
obj Required*	Any

Returns

Expand table

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str =
'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

Expand table

Name	Description
b Required*	str | bytes
content_type Required*	str | None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> | None
allow_pickle Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

Expand table

Name	Description
by_alias	bool Default value: True
ref_template	str Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
Dict[str, Any]	

schema_json

```
Python
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**dumps_kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
by_alias Required*	bool
ref_template Required*	str
dumps_kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

Expand table

Type	Description
str	

update_forward_refs

```
Python
update_forward_refs(**localns: Any) -> None
```

Parameters

Expand table

Name	Description
localns Required*	Any

Returns

Expand table

Type	Description
None	

validate

```
Python
validate(value: Any) -> Self
```

Parameters

Expand table

Name	Description
value Required*	Any

Returns

Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

Expand table

Type	Description
	A dictionary of extra fields, or *None* if *config.extra* is not set to "*allow*".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

selected_participant

Name of participant to speak next (None = finish conversation)

Python

```
field selected_participant: str | None = None
```

instruction

Optional instruction to provide to the selected participant

Python

```
field instruction: str | None = None
```

finish

Whether the conversation should be completed

Python

```
field finish: bool = False
```

final_message

Optional final message string when finishing conversation (will be converted to ChatMessage)

Python

```
field final_message: str | None = None
```

Message Class

A class representing a message in the workflow.

Constructor

```python
Message()
```

Parameters

Expand table

Name	Description
data Required*	
source_id Required*	
target_id	Default value: None
type	Default value: MessageType.STANDARD
trace_contexts	Default value: None
source_span_ids	Default value: None
original_request	Default value: None

Methods

Expand table

__init__	
__new__	
from_dict	Create a Message from a dictionary.
to_dict	Convert the Message to a dictionary for serialization.

__init__

Python

```
__init__(data: Any, source_id: str, target_id: str | None = None, type:
MessageType = MessageType.STANDARD, trace_contexts: list[dict[str, str]] | None =
None, source_span_ids: list[str] | None = None, original_request: Any = None) ->
None
```

Parameters

Expand table

Name	Description
data Required*	Any
source_id Required*	str
target_id	str | None Default value: None
type	<xref:agent_framework._workflows._runner_context.MessageType> Default value: MessageType.STANDARD
trace_contexts	list[dict[str, str]] | None Default value: None
source_span_ids	list[str] | None Default value: None
original_request	Any Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create a Message from a dictionary.

Python

```
static from_dict(data: dict[str, Any]) -> Message
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
Message	

to_dict

Convert the Message to a dictionary for serialization.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Attributes

source_span_id

Get the first source span ID for backward compatibility.

trace_context

Get the first trace context for backward compatibility.

data

Python

```
data: Any
```

original_request

Python

```
original_request: Any = None
```

source_id

Python

```
source_id: str
```

source_span_ids

Python

```
source_span_ids: list[str] | None = None
```

target_id

Python

```
target_id: str | None = None
```

trace_contexts

Python

```
trace_contexts: list[dict[str, str]] | None = None
```

type

Python

```
type: MessageType = 'standard'
```

OrchestrationState Class

Unified state container for orchestrator checkpointing.

This dataclass standardizes checkpoint serialization across all three group chat patterns while allowing pattern-specific extensions via metadata.

Common attributes cover shared orchestration concerns (task, conversation, round tracking). Pattern-specific state goes in the metadata dict.

Constructor

```
Python
OrchestrationState()
```

Parameters

Expand table

Name	Description
conversation	Default value: <factory>
round_index	Default value: 0
metadata	Default value: <factory>
task	Default value: None

Methods

Expand table

__init__	
__new__	
from_dict	Deserialize from checkpointed dict.
to_dict	Serialize to dict for checkpointing.

__init__

Python

```
__init__(conversation: list[agent_framework._types.ChatMessage] = <factory>,
round_index: int = 0, metadata: dict[str, typing.Any] = <factory>, task:
agent_framework._types.ChatMessage | None = None) -> None
```

Parameters

Expand table

Name	Description
conversation	list[ChatMessage] Default value: <factory>
round_index	int Default value: 0
metadata	dict[str, Any] Default value: <factory>
task	ChatMessage | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Deserialize from checkpointed dict.

Python

```
from_dict(data: dict[str, Any]) -> OrchestrationState
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any] Checkpoint data with encoded conversation

Returns

Expand table

Type	Description
OrchestrationState	Restored OrchestrationState instance

to_dict

Serialize to dict for checkpointing.

```
Python
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	Dict with encoded conversation and metadata for persistence

Attributes

conversation

Full conversation history (all messages)

```
Python
conversation: list[ChatMessage]
```

round_index

Number of coordination rounds completed (0 if not tracked)

Python

```
round_index: int = 0
```

metadata

Extensible dict for pattern-specific state

Python

```
metadata: dict[str, Any]
```

task

Optional primary task/question being orchestrated

Python

```
task: ChatMessage | None = None
```

RequestInfoEvent Class

Event triggered when a workflow executor requests external information.

Initialize the request info event.

Constructor

Python

```
RequestInfoEvent()
```

Parameters

Expand table

Name	Description
request_id Required*	Unique identifier for the request.
source_executor_id Required*	ID of the executor that made the request.
request_data Required*	The data associated with the request.
response_type Required*	Expected type of the response.

Methods

Expand table

__init__	Initialize the request info event.
__new__	
from_dict	Create a RequestInfoEvent from a dictionary.
to_dict	Convert the request info event to a dictionary for serialization.

__init__

Initialize the request info event.

```
Python
__init__(request_id: str, source_executor_id: str, request_data: Any,
response_type: type[Any])
```

Parameters

Expand table

Name	Description
request_id Required*	str Unique identifier for the request.
source_executor_id Required*	str ID of the executor that made the request.
request_data Required*	Any The data associated with the request.
response_type Required*	type[Any] Expected type of the response.

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create a RequestInfoEvent from a dictionary.

```
Python
static from_dict(data: dict[str, Any]) -> RequestInfoEvent
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
RequestInfoEvent	

to_dict

Convert the request info event to a dictionary for serialization.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

RequestInfoInterceptor Class

Internal executor that pauses workflow for human input before agent runs.

This executor is inserted into the workflow graph by builders when *.with_request_info()* is called. It intercepts AgentExecutorRequest messages BEFORE the agent runs and pauses the workflow via *ctx.request_info()* with an AgentInputRequest.

When a response is received, the response handler injects the input as a user message into the conversation and forwards the request to the agent.

The optional *agent_filter* parameter allows limiting which agents trigger the pause. If the target agent's ID is not in the filter set, the request is forwarded without pausing.

Initialize the request info interceptor executor.

Constructor

Python

```
RequestInfoInterceptor()
```

Parameters

Expand table

Name	Description
executor_id	ID for this executor. If None, generates a unique ID using the format "request_info_interceptor-<uuid4>". Default value: None
agent_filter	Optional set of agent/executor IDs to filter on. If provided, only requests to these agents trigger a pause. If None (default), all requests trigger a pause. Default value: None

Methods

Expand table

__init__	Initialize the request info interceptor executor.

__new__	
can_handle	Check if the executor can handle a given message type.
clone	
execute	Execute the executor with a given message and context parameters. • Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using @handler decorator.
from_dict	
from_json	
handle_input_response	Handle the human input and forward the modified request to the agent. Injects the response as a user message into the conversation and forwards the modified request to the agent. TODO: Consider having each orchestration implement its own response handler for more specialized behavior.
intercept_agent_request	Intercept request before agent runs and pause for human input. Pauses the workflow and emits a RequestInfoEvent with the current conversation for steering. If an agent filter is configured and this agent is not in the filter, the request is forwarded without pausing.
intercept_concurrent_requests	Intercept requests before concurrent agents run. This handler is used by ConcurrentBuilder to get human input before all parallel agents execute.
intercept_conversation	Intercept conversation before agent runs (used by SequentialBuilder). SequentialBuilder passes list[ChatMessage] directly to agents. This handler intercepts that flow and pauses for human input.
is_request_supported	Check if the executor supports request of the given type and handling a response of the given type.
on_checkpoint_restore	Hook called when the workflow is restored from a checkpoint. Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.
on_checkpoint_save	Hook called when the workflow is being saved to a checkpoint. Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

	The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.
to_dict	Serialize executor definition for workflow topology export.
to_json	

__init__

Initialize the request info interceptor executor.

Python

```
__init__(executor_id: str | None = None, agent_filter: set[str] | None = None) -> None
```

Parameters

Expand table

Name	Description
executor_id	str | None ID for this executor. If None, generates a unique ID using the format "request_info_interceptor-<uuid4>". Default value: None
agent_filter	set[str] | None Optional set of agent/executor IDs to filter on. If provided, only requests to these agents trigger a pause. If None (default), all requests trigger a pause. Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

can_handle

Check if the executor can handle a given message type.

```
Python
can_handle(message: Message) -> bool
```

Parameters

Expand table

Name	Description
message Required*	Message The message to check.

Returns

Expand table

Type	Description
bool	True if the executor can handle the message type, False otherwise.

clone

```
Python
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

Expand table

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.

Name	Description
`trace_contexts`	list[dict[str, str]] | None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
`source_span_ids`	list[str] | None Optional source span IDs from multiple sources for linking. Default value: None

Returns

Expand table

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python

```
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

Expand table

Name	Description
`data` Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

handle_input_response

Handle the human input and forward the modified request to the agent.

Injects the response as a user message into the conversation and forwards the modified request to the agent.

TODO: Consider having each orchestration implement its own response handler for more specialized behavior.

Python

```
async handle_input_response(original_request: AgentInputRequest, response: str,
ctx: WorkflowContext[AgentExecutorRequest | list[ChatMessage], Any]) -> None
```

Parameters

Expand table

Name	Description
original_request Required*	AgentInputRequest The AgentInputRequest that triggered the pause

Name	Description
response Required*	str The human input text
ctx Required*	WorkflowContext[AgentExecutorRequest | list[ChatMessage], Any] Workflow context for continuing the workflow

Returns

Expand table

Type	Description
None	

intercept_agent_request

Intercept request before agent runs and pause for human input.

Pauses the workflow and emits a RequestInfoEvent with the current conversation for steering. If an agent filter is configured and this agent is not in the filter, the request is forwarded without pausing.

Python

```
async intercept_agent_request(request: AgentExecutorRequest, ctx:
WorkflowContext[AgentExecutorRequest, Any]) -> None
```

Parameters

Expand table

Name	Description
request Required*	AgentExecutorRequest The request about to be sent to the agent
ctx Required*	WorkflowContext[AgentExecutorRequest, Any] Workflow context for requesting info

Returns

Expand table

Type	Description
None	

intercept_concurrent_requests

Intercept requests before concurrent agents run.

This handler is used by ConcurrentBuilder to get human input before all parallel agents execute.

Python

```
async intercept_concurrent_requests(requests: list[AgentExecutorRequest], ctx: WorkflowContext[list[AgentExecutorRequest], Any]) -> None
```

Parameters

Expand table

Name	Description
requests Required*	list[AgentExecutorRequest] List of requests for all concurrent agents
ctx Required*	WorkflowContext[list[AgentExecutorRequest], Any] Workflow context for requesting info

Returns

Expand table

Type	Description
None	

intercept_conversation

Intercept conversation before agent runs (used by SequentialBuilder).

SequentialBuilder passes list[ChatMessage] directly to agents. This handler intercepts that flow and pauses for human input.

Python

```
async intercept_conversation(messages: list[ChatMessage], ctx:
WorkflowContext[list[ChatMessage], Any]) -> None
```

Parameters

Expand table

Name	Description
messages Required*	list[ChatMessage] The conversation about to be sent to the agent
ctx Required*	WorkflowContext[list[ChatMessage], Any] Workflow context for requesting info

Returns

Expand table

Type	Description
None	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

Expand table

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

Expand table

Type	Description
bool	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Hook called when the workflow is restored from a checkpoint.

Override this method in subclasses to implement custom logic that should run when the workflow is restored from a checkpoint.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any] The state dictionary that was saved during checkpointing.

Returns

Expand table

Type	Description
None	

on_checkpoint_save

Hook called when the workflow is being saved to a checkpoint.

Override this method in subclasses to implement custom logic that should return state to be saved in the checkpoint.

The returned state dictionary will be passed to *on_checkpoint_restore* when the workflow is restored from the checkpoint. The dictionary should only contain JSON-serializable data.

```
Python
async on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	A state dictionary to be saved during checkpointing.

to_dict

Serialize executor definition for workflow topology export.

```
Python
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

```
Python
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

input_types

Get the list of input types that this executor can handle.

Returns

Expand table

Type	Description
	A list of the message types that this executor's handlers can process.

output_types

Get the list of output types that this executor can produce via send_message().

Returns

Expand table

Type	Description
	A list of the output types inferred from the handlers' WorkflowContext[T] annotations.

workflow_output_types

Get the list of workflow output types that this executor can produce via yield_output().

Returns

Expand table

Type	Description
	A list of the workflow output types inferred from handlers' WorkflowContext[T, U] annotations.

Role Class

Describes the intended purpose of a message within a chat interaction.

Properties: SYSTEM: The role that instructs or sets the behavior of the AI system. USER: The role that provides user input for chat interactions. ASSISTANT: The role that provides responses to system-instructed, user-prompted input. TOOL: The role that provides additional information and references in response to tool use requests.

Initialize Role with a value.

Constructor

Python

```
Role()
```

Parameters

Expand table

Name	Description
value Required*	The string representation of the role.

Examples

Python

```
from agent_framework import Role

# Use predefined role constants
system_role = Role.SYSTEM
user_role = Role.USER
assistant_role = Role.ASSISTANT
tool_role = Role.TOOL

# Create custom role
custom_role = Role(value="custom")

# Compare roles
```

```
print(system_role == Role.SYSTEM)  # True
print(system_role.value)  # "system"
```

Methods

Expand table

__init__	Initialize Role with a value.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize Role with a value.

Python

```
__init__(value: str) -> None
```

Parameters

Expand table

Name	Description
value Required*	str The string representation of the role.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	

Name	Description
	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

value

The string representation of the role.

ASSISTANT

Python

```
ASSISTANT: Role = Role(value='assistant')
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

SYSTEM

Python

```
SYSTEM: Role = Role(value='system')
```

TOOL

Python

```
TOOL: Role = Role(value='tool')
```

USER

Python

```
USER: Role = Role(value='user')
```

Runner Class

A class to run a workflow in Pregel supersteps.

Initialize the runner with edges, shared state, and context.

Constructor

Python

```
Runner()
```

Parameters

Expand table

Name	Description
edge_groups Required*	The edge groups of the workflow.
executors Required*	Map of executor IDs to executor instances.
shared_state Required*	The shared state for the workflow.
ctx Required*	The runner context for the workflow.
max_iterations	The maximum number of iterations to run. Default value: 100
workflow_id	The workflow ID for checkpointing. Default value: None

Methods

Expand table

__init__	Initialize the runner with edges, shared state, and context.
__new__	

reset_iteration_count	Reset the iteration count to zero.
restore_from_checkpoint	Restore workflow state from a checkpoint.
run_until_convergence	Run the workflow until no more messages are sent.

__init__

Initialize the runner with edges, shared state, and context.

Python

```
__init__(edge_groups: Sequence[EdgeGroup], executors: dict[str, Executor],
shared_state: SharedState, ctx: RunnerContext, max_iterations: int = 100,
workflow_id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
edge_groups Required*	Sequence[<xref:agent_framework._workflows._edge.EdgeGroup>] The edge groups of the workflow.
executors Required*	dict[str, Executor] Map of executor IDs to executor instances.
shared_state Required*	SharedState The shared state for the workflow.
ctx Required*	RunnerContext The runner context for the workflow.
max_iterations	int The maximum number of iterations to run. Default value: 100
workflow_id	str | None The workflow ID for checkpointing. Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

reset_iteration_count

Reset the iteration count to zero.

```
Python
reset_iteration_count() -> None
```

Returns

Expand table

Type	Description
None	

restore_from_checkpoint

Restore workflow state from a checkpoint.

```
Python
async restore_from_checkpoint(checkpoint_id: str, checkpoint_storage:
CheckpointStorage | None = None) -> bool
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str The ID of the checkpoint to restore from

Name	Description
`checkpoint_storage`	CheckpointStorage | None Optional storage to load checkpoints from when the runner context itself is not configured with checkpointing. Default value: None

Returns

Expand table

Type	Description
bool	True if restoration was successful, False otherwise

run_until_convergence

Run the workflow until no more messages are sent.

Python

```
async run_until_convergence() -> AsyncGenerator[WorkflowEvent, None]
```

Returns

Expand table

Type	Description
AsyncGenerator[WorkflowEvent, None]	

Attributes

context

Get the workflow context.

RunnerContext Class

Protocol for the execution context used by the runner.

A single context that supports messaging, events, and optional checkpointing. If checkpoint storage is not configured, checkpoint methods may raise.

Constructor

Python

```
RunnerContext()
```

Methods

Expand table

__init__	
__new__	
add_event	Add an event to the execution context.
add_request_info_event	Add a RequestInfoEvent to the context and track it for correlation.
apply_checkpoint	Apply a checkpoint to the current context, mutating its state.
clear_runtime_checkpoint_storage	Clear runtime checkpoint storage override.
create_checkpoint	Create a checkpoint of the current workflow state.
drain_events	Drain all events from the context.
drain_messages	Drain all messages from the context.
get_pending_request_info_events	Get the mapping of request IDs to their corresponding RequestInfoEvent.
has_checkpointing	Check if the context supports checkpointing.
has_events	Check if there are any events in the context.
has_messages	Check if there are any messages in the context.
is_streaming	Check if the workflow is in streaming mode.

load_checkpoint	Load a checkpoint without mutating the current context state.
next_event	Wait for and return the next event emitted by the workflow run.
reset_for_new_run	Reset the context for a new workflow run.
send_message	Send a message from the executor to the context.
send_request_info_response	Send a response correlated to a pending request.
set_runtime_checkpoint_storage	Set runtime checkpoint storage to override build-time configuration.
set_streaming	Set whether agents should stream incremental updates.
set_workflow_id	Set the workflow ID for the context.

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

add_event

Add an event to the execution context.

Python

```
async add_event(event: WorkflowEvent) -> None
```

Parameters

Expand table

Name	Description
event Required*	WorkflowEvent The event to be added.

Returns

Expand table

Type	Description
None	

add_request_info_event

Add a RequestInfoEvent to the context and track it for correlation.

Python

```
async add_request_info_event(event: RequestInfoEvent) -> None
```

Parameters

Expand table

Name	Description
event Required*	RequestInfoEvent The RequestInfoEvent to be added.

Returns

Expand table

Type	Description
None	

apply_checkpoint

Apply a checkpoint to the current context, mutating its state.

Python

```
async apply_checkpoint(checkpoint: WorkflowCheckpoint) -> None
```

Parameters

Expand table

Name	Description
checkpoint Required*	WorkflowCheckpoint The checkpoint whose state is to be applied.

Returns

Expand table

Type	Description
None	

clear_runtime_checkpoint_storage

Clear runtime checkpoint storage override.

Python

```
clear_runtime_checkpoint_storage() -> None
```

Returns

Expand table

Type	Description
None	

create_checkpoint

Create a checkpoint of the current workflow state.

Python

```
async create_checkpoint(shared_state: SharedState, iteration_count: int, metadata:
dict[str, Any] | None = None) -> str
```

Parameters

Expand table

Name	Description
shared_state Required*	SharedState The shared state to include in the checkpoint. This is needed to capture the full state of the workflow. The shared state is not managed by the context itself.
iteration_count Required*	int The current iteration count of the workflow.
metadata	dict[str, Any] | None Optional metadata to associate with the checkpoint. Default value: None

Returns

Expand table

Type	Description
str	The ID of the created checkpoint.

drain_events

Drain all events from the context.

Python

```
async drain_events() -> list[WorkflowEvent]
```

Returns

Expand table

Type	Description
list[WorkflowEvent]	A list of events that were added to the context.

drain_messages

Drain all messages from the context.

Python

```
async drain_messages() -> dict[str, list[Message]]
```

Returns

Expand table

Type	Description
dict[str, list[Message]]	A dictionary mapping executor IDs to lists of messages.

get_pending_request_info_events

Get the mapping of request IDs to their corresponding RequestInfoEvent.

```
Python
async get_pending_request_info_events() -> dict[str, RequestInfoEvent]
```

Returns

Expand table

Type	Description
dict[str, RequestInfoEvent]	A dictionary mapping request IDs to their corresponding RequestInfoEvent.

has_checkpointing

Check if the context supports checkpointing.

```
Python
has_checkpointing() -> bool
```

Returns

Expand table

Type	Description
bool	True if checkpointing is supported, False otherwise.

has_events

Check if there are any events in the context.

Python

```
async has_events() -> bool
```

Returns

Expand table

Type	Description
bool	True if there are events, False otherwise.

has_messages

Check if there are any messages in the context.

Python

```
async has_messages() -> bool
```

Returns

Expand table

Type	Description
bool	True if there are messages, False otherwise.

is_streaming

Check if the workflow is in streaming mode.

Python

```
is_streaming() -> bool
```

Returns

Expand table

Type	Description
bool	True if streaming mode is enabled, False otherwise.

load_checkpoint

Load a checkpoint without mutating the current context state.

Python

```
async load_checkpoint(checkpoint_id: str) -> WorkflowCheckpoint | None
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str The ID of the checkpoint to load.

Returns

Expand table

Type	Description
WorkflowCheckpoint | None	The loaded checkpoint, or None if it does not exist.

next_event

Wait for and return the next event emitted by the workflow run.

Python

```
async next_event() -> WorkflowEvent
```

Returns

Expand table

Type	Description
WorkflowEvent	

reset_for_new_run

Reset the context for a new workflow run.

Python

```
reset_for_new_run() -> None
```

Returns

Expand table

Type	Description
None	

send_message

Send a message from the executor to the context.

Python

```
async send_message(message: Message) -> None
```

Parameters

Expand table

Name	Description
message Required*	Message The message to be sent.

Returns

Expand table

Type	Description
None	

send_request_info_response

Send a response correlated to a pending request.

Python

```
async send_request_info_response(request_id: str, response: Any) -> None
```

Parameters

Expand table

Name	Description
request_id Required*	str The ID of the original request.
response Required*	Any The response data to be sent.

Returns

Expand table

Type	Description
None	

set_runtime_checkpoint_storage

Set runtime checkpoint storage to override build-time configuration.

Python

```
set_runtime_checkpoint_storage(storage: CheckpointStorage) -> None
```

Parameters

Expand table

Name	Description
storage Required*	CheckpointStorage The checkpoint storage to use for this run.

Returns

Expand table

Type	Description
None	

set_streaming

Set whether agents should stream incremental updates.

Python

```
set_streaming(streaming: bool) -> None
```

Parameters

Expand table

Name	Description
`streaming` Required*	bool True for streaming mode (run_stream), False for non-streaming (run).

Returns

Expand table

Type	Description
None	

set_workflow_id

Set the workflow ID for the context.

Python

```
set_workflow_id(workflow_id: str) -> None
```

Parameters

Expand table

Name	Description
`workflow_id` Required*	str

Returns

Expand table

Type	Description
None	

SequentialBuilder Class

High-level builder for sequential agent/executor workflows with shared context.

- *participants([...])* accepts a list of AgentProtocol (recommended) or Executor instances
- *register_participants([...])* accepts a list of factories for AgentProtocol (recommended)

  or Executor factories
- Executors must define a handler that consumes list[ChatMessage] and sends out a list[ChatMessage]
- The workflow wires participants in order, passing a list[ChatMessage] down the chain
- Agents append their assistant messages to the conversation
- Custom executors can transform/summarize and return a list[ChatMessage]
- The final output is the conversation produced by the last participant

Usage:

Python

```
   from agent_framework import SequentialBuilder

   # With agent instances
   workflow = SequentialBuilder().participants([agent1, agent2,
summarizer_exec]).build()

   # With agent factories
   workflow = (
       SequentialBuilder().register_participants([create_agent1, create_agent2,
create_summarizer_exec]).build()
   )

   # Enable checkpoint persistence
   workflow = SequentialBuilder().participants([agent1,
agent2]).with_checkpointing(storage).build()

   # Enable request info for mid-workflow feedback (pauses before each agent)
   workflow = SequentialBuilder().participants([agent1,
agent2]).with_request_info().build()

   # Enable request info only for specific agents
   workflow = (
       SequentialBuilder()
       .participants([agent1, agent2, agent3])
       .with_request_info(agents=[agent2])  # Only pause before agent2
```

```
        .build()
    )
```

Constructor

Python

```
SequentialBuilder()
```

Methods

Expand table

__init__	
__new__	

build	Build and validate the sequential workflow.
	
	Wiring pattern:
	
	• _InputToConversation normalizes the initial input into list[ChatMessage]
	• For each participant in order:
	  ◦ If Agent (or AgentExecutor): pass conversation to the agent, then optionally route through a request info interceptor, then convert response to conversation via _ResponseToConversation
	  ◦ Else (custom Executor): pass conversation directly to the executor
	• _EndWithConversation yields the final conversation and the workflow becomes idle

participants	Define the ordered participants for this sequential workflow. Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances. Raises if empty or duplicates are provided for clarity.
register_participants	Register participant factories for this sequential workflow.
with_checkpointing	Enable checkpointing for the built workflow using the provided storage.
with_request_info	Enable request info before agents run in the workflow. When enabled, the workflow pauses before each agent runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the agent responds. The caller provides input via the standard response_handler/request_info pattern. Example:

Python

```
   # Pause before all agents
   workflow = SequentialBuilder().participants([a1,
a2]).with_request_info().build()

   # Pause only before specific agents
   workflow = (
       SequentialBuilder()
       .participants([drafter, reviewer, finalizer])
       .with_request_info(agents=[reviewer])  # Only pause before
reviewer
       .build()
   )
```

__init__

Python

```
__init__() -> None
```

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

build

Build and validate the sequential workflow.

Wiring pattern:

- _InputToConversation normalizes the initial input into list[ChatMessage]
- For each participant in order:

- If Agent (or AgentExecutor): pass conversation to the agent, then optionally route through a request info interceptor, then convert response to conversation via _ResponseToConversation
- Else (custom Executor): pass conversation directly to the executor

- _EndWithConversation yields the final conversation and the workflow becomes idle

Python

```
build() -> Workflow
```

Returns

Expand table

Type	Description
Workflow	

participants

Define the ordered participants for this sequential workflow.

Accepts AgentProtocol instances (auto-wrapped as AgentExecutor) or Executor instances. Raises if empty or duplicates are provided for clarity.

Python

```
participants(participants: Sequence[AgentProtocol | Executor]) ->
SequentialBuilder
```

Parameters

Expand table

Name	Description
participants Required*	Sequence[AgentProtocol | Executor]

Returns

Expand table

Type	Description
SequentialBuilder	

register_participants

Register participant factories for this sequential workflow.

Python

```
register_participants(participant_factories: Sequence[Callable[[], AgentProtocol | Executor]]) -> SequentialBuilder
```

Parameters

Expand table

Name	Description
participant_factories Required*	Sequence[Callable[[], AgentProtocol | Executor]]

Returns

Expand table

Type	Description
SequentialBuilder	

with_checkpointing

Enable checkpointing for the built workflow using the provided storage.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> SequentialBuilder
```

Parameters

Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage

Returns

Expand table

Type	Description
SequentialBuilder	

with_request_info

Enable request info before agents run in the workflow.

When enabled, the workflow pauses before each agent runs, emitting a RequestInfoEvent that allows the caller to review the conversation and optionally inject guidance before the agent responds. The caller provides input via the standard response_handler/request_info pattern.

Example:

Python

```
   # Pause before all agents
   workflow = SequentialBuilder().participants([a1,
a2]).with_request_info().build()

   # Pause only before specific agents
   workflow = (
       SequentialBuilder()
       .participants([drafter, reviewer, finalizer])
       .with_request_info(agents=[reviewer])  # Only pause before reviewer
       .build()
   )
```

Python

```
with_request_info(*, agents: Sequence[str | AgentProtocol | Executor] | None =
None) -> SequentialBuilder
```

Parameters

Expand table

Name	Description
agents Required*	Sequence[str | AgentProtocol | Executor] | None Optional filter - only pause before these specific agents/executors. Accepts agent names (str), agent instances, or executor instances. If None (default), pauses before every agent.

Keyword-Only Parameters

Expand table

Name	Description
agents	Default value: None

Returns

Expand table

Type	Description
<xref:self>	The builder instance for fluent chaining.

SharedState Class

A class to manage shared state in a workflow.

SharedState provides thread-safe access to workflow state data that needs to be shared across executors during workflow execution.

Reserved Keys: The following keys are reserved for internal framework use and should not be modified by user code:

- *_executor_state*: Stores executor state for checkpointing (managed by Runner)

> ⚠ **Warning**
>
> Do not use keys starting with underscore (_) as they may be reserved for
>
> internal framework operations.

Initialize the shared state.

Constructor

Python

```
SharedState()
```

Methods

Expand table

__init__	Initialize the shared state.
__new__	
clear	Clear the entire shared state.
delete	Delete a key from the shared state.
delete_within_hold	Delete a key without acquiring the lock (unsafe - use within hold() context).
export_state	Get a serialized copy of the entire shared state.

get	Get a value from the shared state.
get_within_hold	Get a value without acquiring the lock (unsafe - use within hold() context).
has	Check if a key exists in the shared state.
has_within_hold	Check if a key exists without acquiring the lock (unsafe - use within hold() context).
hold	Context manager to hold the shared state lock for multiple operations. Usage: async with shared_state.hold(): await shared_state.set_within_hold("key", value) value = await shared_state.get_within_hold("key")
import_state	Populate the shared state from a serialized state dictionary. This replaces the entire current state with the provided state.
set	Set a value in the shared state.
set_within_hold	Set a value without acquiring the lock (unsafe - use within hold() context).

__init__

Initialize the shared state.

Python

```
__init__() -> None
```

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

clear

Clear the entire shared state.

Python

```
async clear() -> None
```

Returns

Expand table

Type	Description
None	

delete

Delete a key from the shared state.

Python

```
async delete(key: str) -> None
```

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
None	

delete_within_hold

Delete a key without acquiring the lock (unsafe - use within hold() context).

Python

```
async delete_within_hold(key: str) -> None
```

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
None	

export_state

Get a serialized copy of the entire shared state.

Python

```
async export_state() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

get

Get a value from the shared state.

Python

```
async get(key: str) -> Any
```

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
Any	

get_within_hold

Get a value without acquiring the lock (unsafe - use within hold() context).

Python

```
async get_within_hold(key: str) -> Any
```

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
Any	

has

Check if a key exists in the shared state.

Python
`async has(key: str) -> bool`

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
bool	

has_within_hold

Check if a key exists without acquiring the lock (unsafe - use within hold() context).

Python
`async has_within_hold(key: str) -> bool`

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
bool	

hold

Context manager to hold the shared state lock for multiple operations.

Usage: async with shared_state.hold(): await shared_state.set_within_hold("key", value) value = await shared_state.get_within_hold("key")

Python

```
hold() -> AsyncIterator[SharedState]
```

Returns

Expand table

Type	Description
AsyncIterator[SharedState]	

import_state

Populate the shared state from a serialized state dictionary.

This replaces the entire current state with the provided state.

Python

```
async import_state(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any]

Returns

Expand table

Type	Description
None	

set

Set a value in the shared state.

Python

```
async set(key: str, value: Any) -> None
```

Parameters

Expand table

Name	Description
key Required*	str
value Required*	Any

Returns

Expand table

Type	Description
None	

set_within_hold

Set a value without acquiring the lock (unsafe - use within hold() context).

Python

```
async set_within_hold(key: str, value: Any) -> None
```

Parameters

Expand table

Name	Description
key Required*	str
value Required*	Any

Returns

Expand table

Type	Description
None	

SingleEdgeGroup Class

Convenience wrapper for a solitary edge, keeping the group API uniform.

Create a one-to-one edge group between two executors.

Constructor

Python

```
SingleEdgeGroup()
```

Parameters

Expand table

Name	Description
source_id Required*	
target_id Required*	
condition	Default value: None

Keyword-Only Parameters

Expand table

Name	Description
id	Default value: None

Examples

Python

```
group = SingleEdgeGroup("ingest", "validate")
assert group.edges[0].source_id == "ingest"
```

Methods

Expand table

__init__	Create a one-to-one edge group between two executors.
__new__	
clone	
from_dict	Hydrate the correct *EdgeGroup* subclass from serialised state. The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
from_json	
register	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.
to_dict	Serialise the group metadata and contained edges into primitives. The payload captures each edge through its own *to_dict* call, enabling round-tripping through formats such as JSON without leaking Python objects.
to_json	

__init__

Create a one-to-one edge group between two executors.

Python

```
__init__(source_id: str, target_id: str, condition: Callable[[Any], bool] | None = None, *, id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
source_id Required*	str

Name	Description
target_id Required*	str
condition	Callable[[Any], bool] | None Default value: None
id Required*	str | None

Keyword-Only Parameters

Expand table

Name	Description
id	Default value: None

Returns

Expand table

Type	Description
None	

Examples

Python

```
group = SingleEdgeGroup("ingest", "validate")
assert group.edges[0].source_id == "ingest"
```

__new__

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._edge.EdgeGroup>	

Examples

Python

```
   payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id":
"b"}]}
   group = EdgeGroup.from_dict(payload)
   assert isinstance(group, EdgeGroup)
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python

```
register(subclass: type[EdgeGroup]) -> type[EdgeGroup]
```

Parameters

Expand table

Name	Description
subclass Required*	type[<xref:agent_framework._workflows._edge.EdgeGroup>]

Returns

Expand table

Type	Description
type[<xref:agent_framework._workflows._edge.EdgeGroup>]	

Examples

Python

```
@EdgeGroup.register
class CustomGroup(EdgeGroup):
    pass


assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup
```

to_dict

Serialise the group metadata and contained edges into primitives.

The payload captures each edge through its own *to_dict* call, enabling round-tripping through formats such as JSON without leaking Python objects.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

Python

```
group = EdgeGroup([Edge("read", "write")])
snapshot = group.to_dict()
assert snapshot["edges"][0]["source_id"] == "read"
```

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

type

Python

```
type: str
```

StandardMagenticManager Class

Standard Magentic manager that performs real LLM calls via a ChatAgent.

The manager constructs prompts that mirror the original Magentic One orchestration:

- Facts gathering
- Plan creation
- Progress ledger in JSON
- Facts update and plan update on reset
- Final answer synthesis

Initialize the Standard Magentic Manager.

Constructor

```python
StandardMagenticManager()
```

Parameters

Expand table

Name	Description
agent Required*	An agent instance to use for LLM calls. The agent's configured options (temperature, seed, instructions, etc.) will be applied.
task_ledger	Optional task ledger for managing task state. Default value: None

Keyword-Only Parameters

Expand table

Name	Description
task_ledger_facts_prompt	Optional prompt for the task ledger facts. Default value: None
task_ledger_plan_prompt	Optional prompt for the task ledger plan. Default value: None

Name	Description
task_ledger_full_prompt	Optional prompt for the full task ledger. Default value: None
task_ledger_facts_update_prompt	Optional prompt for updating task ledger facts. Default value: None
task_ledger_plan_update_prompt	Optional prompt for updating task ledger plan. Default value: None
progress_ledger_prompt	Optional prompt for the progress ledger. Default value: None
final_answer_prompt	Optional prompt for the final answer. Default value: None
max_stall_count	Maximum number of stalls allowed. Default value: 3
max_reset_count	Maximum number of resets allowed. Default value: None
max_round_count	Maximum number of rounds allowed. Default value: None
progress_ledger_retry_count	Maximum number of retries for the progress ledger. Default value: None

Methods

Expand table

__init__	Initialize the Standard Magentic Manager.
__new__	
create_progress_ledger	Use the model to produce a JSON progress ledger based on the conversation so far. Adds lightweight retries with backoff for transient parse issues and avoids selecting a non-existent "unknown" agent. If there are no participants, a clear error is raised.
on_checkpoint_restore	
on_checkpoint_save	
plan	Create facts and plan using the model, then render a combined task ledger as a single assistant message.

prepare_final_answer	Ask the model to produce the final answer addressed to the user.
replan	Update facts and plan when stalling or looping has been detected.

__init__

Initialize the Standard Magentic Manager.

Python

```
__init__(agent: AgentProtocol, task_ledger: _MagenticTaskLedger | None = None, *,
task_ledger_facts_prompt: str | None = None, task_ledger_plan_prompt: str | None =
None, task_ledger_full_prompt: str | None = None, task_ledger_facts_update_prompt:
str | None = None, task_ledger_plan_update_prompt: str | None = None,
progress_ledger_prompt: str | None = None, final_answer_prompt: str | None = None,
max_stall_count: int = 3, max_reset_count: int | None = None, max_round_count: int
| None = None, progress_ledger_retry_count: int | None = None) -> None
```

Parameters

Expand table

Name	Description
agent Required*	AgentProtocol An agent instance to use for LLM calls. The agent's configured options (temperature, seed, instructions, etc.) will be applied.
task_ledger	<xref:agent_framework._workflows._magentic._MagenticTaskLedger> | None Optional task ledger for managing task state. Default value: None
task_ledger_facts_prompt Required*	str | None
task_ledger_plan_prompt Required*	str | None
task_ledger_full_prompt Required*	str | None
task_ledger_facts_update_prompt Required*	str | None
task_ledger_plan_update_prompt Required*	str | None
progress_ledger_prompt Required*	str | None

Name	Description
final_answer_prompt Required*	str | None
max_stall_count Required*	int
max_reset_count Required*	int | None
max_round_count Required*	int | None
progress_ledger_retry_count Required*	int | None

Keyword-Only Parameters

Expand table

Name	Description
task_ledger_facts_prompt	Optional prompt for the task ledger facts. Default value: None
task_ledger_plan_prompt	Optional prompt for the task ledger plan. Default value: None
task_ledger_full_prompt	Optional prompt for the full task ledger. Default value: None
task_ledger_facts_update_prompt	Optional prompt for updating task ledger facts. Default value: None
task_ledger_plan_update_prompt	Optional prompt for updating task ledger plan. Default value: None
progress_ledger_prompt	Optional prompt for the progress ledger. Default value: None
final_answer_prompt	Optional prompt for the final answer. Default value: None
max_stall_count	Maximum number of stalls allowed. Default value: 3
max_reset_count	Maximum number of resets allowed. Default value: None
max_round_count	Maximum number of rounds allowed. Default value: None

Name	Description
progress_ledger_retry_count	Maximum number of retries for the progress ledger. Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

create_progress_ledger

Use the model to produce a JSON progress ledger based on the conversation so far.

Adds lightweight retries with backoff for transient parse issues and avoids selecting a non-existent "unknown" agent. If there are no participants, a clear error is raised.

```
Python
async create_progress_ledger(magentic_context: MagenticContext) ->
_MagenticProgressLedger
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._magentic._MagenticProgressLedger>	

on_checkpoint_restore

```
Python
on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any]

Returns

Expand table

Type	Description
None	

on_checkpoint_save

```
Python
on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

plan

Create facts and plan using the model, then render a combined task ledger as a single assistant message.

Python

```
async plan(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
ChatMessage	

prepare_final_answer

Ask the model to produce the final answer addressed to the user.

Python

```
async prepare_final_answer(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
ChatMessage	

replan

Update facts and plan when stalling or looping has been detected.

```
Python
async replan(magentic_context: MagenticContext) -> ChatMessage
```

Parameters

Expand table

Name	Description
magentic_context Required*	MagenticContext

Returns

Expand table

Type	Description
ChatMessage	

Attributes

task_ledger

```
Python
task_ledger: _MagenticTaskLedger | None
```

SubWorkflowRequestMessage Class

Message sent from a sub-workflow to an executor in the parent workflow to request information.

This message wraps a RequestInfoEvent emitted by the executor in the sub-workflow.

Constructor

Python

```
SubWorkflowRequestMessage()
```

Parameters

Expand table

Name	Description
source_event Required*	
executor_id Required*	

Methods

Expand table

__init__	
__new__	
create_response	Validate and wrap response data into a SubWorkflowResponseMessage. Validation ensures the response data type matches the expected type from the original request.

__init__

Python

```
__init__(source_event: RequestInfoEvent, executor_id: str) -> None
```

Parameters

Expand table

Name	Description
source_event Required*	RequestInfoEvent
executor_id Required*	str

Returns

Expand table

Type	Description
None	

__new__

```python
__new__(**kwargs)
```

create_response

Validate and wrap response data into a SubWorkflowResponseMessage.

Validation ensures the response data type matches the expected type from the original request.

```python
create_response(data: Any) -> SubWorkflowResponseMessage
```

Parameters

Expand table

Name	Description
data	Any

Name	Description
Required*	

Returns

Expand table

Type	Description
SubWorkflowResponseMessage	

Attributes

source_event

The original RequestInfoEvent emitted by the sub-workflow executor.

Python

```
source_event: RequestInfoEvent
```

executor_id

The ID of the WorkflowExecutor in the parent workflow that is responsible for this sub-workflow. This can be used to ensure that the response is sent back to the correct sub-workflow instance.

Python

```
executor_id: str
```

SubWorkflowResponseMessage Class

Message sent from a parent workflow to a sub-workflow via WorkflowExecutor to provide requested information.

This message wraps the response data along with the original RequestInfoEvent emitted by the sub-workflow executor.

Constructor

Python

```
SubWorkflowResponseMessage()
```

Parameters

Expand table

Name	Description
data Required*	
source_event Required*	

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(data: Any, source_event: RequestInfoEvent) -> None
```

Parameters

Expand table

Name	Description
data Required*	Any
source_event Required*	RequestInfoEvent

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

Attributes

data

The response data to the original request.

```
Python
data: Any
```

source_event

The original RequestInfoEvent emitted by the sub-workflow executor.

```
Python
source_event: RequestInfoEvent
```

SuperStepCompletedEvent Class

Event triggered when a superstep ends.

Initialize the superstep event.

Constructor

Python

```
SuperStepCompletedEvent()
```

Parameters

Expand table

Name	Description
iteration Required*	The number of the superstep (1-based index).
data	Optional data associated with the superstep event. Default value: None

Methods

Expand table

__init__	Initialize the superstep event.
__new__	

__init__

Initialize the superstep event.

Python

```
__init__(iteration: int, data: Any | None = None)
```

Parameters

Expand table

Name	Description
`iteration` Required*	int The number of the superstep (1-based index).
`data`	Any | None Optional data associated with the superstep event. Default value: None

__new__

Python
`__new__(**kwargs)`

SuperStepStartedEvent Class

Event triggered when a superstep starts.

Initialize the superstep event.

Constructor

Python

```
SuperStepStartedEvent()
```

Parameters

Expand table

Name	Description
iteration Required*	The number of the superstep (1-based index).
data	Optional data associated with the superstep event. Default value: None

Methods

Expand table

__init__	Initialize the superstep event.
__new__	

__init__

Initialize the superstep event.

Python

```
__init__(iteration: int, data: Any | None = None)
```

Parameters

Expand table

Name	Description
`iteration` Required*	int The number of the superstep (1-based index).
`data`	Any | None Optional data associated with the superstep event. Default value: None

__new__

Python
`__new__(**kwargs)`

SwitchCaseEdgeGroup Class

Fan-out variant that mimics a traditional switch/case control flow.

Each case inspects the message payload and decides whether it should handle the message. Exactly one case-or the default branch-returns a target at runtime, preserving single-dispatch semantics.

Configure a switch/case routing structure for a single source executor.

Constructor

Python

```
SwitchCaseEdgeGroup()
```

Parameters

Expand table

Name	Description
source_id Required*	Identifier of the executor producing the message to be routed.
cases Required*	list[<xref:agent_framework._workflows._edge.SwitchCaseEdgeGroupCase | agent_framework._workflows._edge.SwitchCaseEdgeGroupDefault>] Ordered sequence of case descriptors concluding with a *SwitchCaseEdgeGroupDefault*. Ordering matters because the runtime evaluates each branch sequentially until one matches.
id Required*	Optional explicit identifier for the edge group.
Examples Required*	Python `cases = [ SwitchCaseEdgeGroupCase(lambda payload: payload["kind"] == "csv", target_id="process_csv"), SwitchCaseEdgeGroupDefault(target_id="process_default"), ] group = SwitchCaseEdgeGroup("router", cases)`

Name	Description
	``` encoded = group.to_dict() assert encoded["cases"][0]["type"] == "Case" ```

Keyword-Only Parameters

Expand table

Name	Description
id	Default value: None

Methods

Expand table

__init__	Configure a switch/case routing structure for a single source executor.
__new__	
clone	
from_dict	Hydrate the correct *EdgeGroup* subclass from serialised state. The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.
from_json	
register	Register a subclass so deserialisation can recover the right type. Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.
to_dict	Serialise the switch-case group, capturing all case descriptors. Each case is converted using *encode_value* to respect dataclass semantics as well as any nested serialisable structures.
to_json	

__init__

Configure a switch/case routing structure for a single source executor.

Python

```
__init__(source_id: str, cases: Sequence[SwitchCaseEdgeGroupCase |
SwitchCaseEdgeGroupDefault], *, id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
source_id Required*	str Identifier of the executor producing the message to be routed.
cases Required*	Sequence[SwitchCaseEdgeGroupCase | SwitchCaseEdgeGroupDefault] Ordered sequence of case descriptors concluding with a *SwitchCaseEdgeGroupDefault*. Ordering matters because the runtime evaluates each branch sequentially until one matches.
id Required*	str | None Optional explicit identifier for the edge group.
Examples Required*	Python `cases = [ SwitchCaseEdgeGroupCase(lambda payload: payload["kind"] == "csv", target_id="process_csv"), SwitchCaseEdgeGroupDefault(target_id="process_default"), ] group = SwitchCaseEdgeGroup("router", cases) encoded = group.to_dict() assert encoded["cases"][0]["type"] == "Case"`

Keyword-Only Parameters

Expand table

Name	Description
id	Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

clone

```
Python
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Hydrate the correct *EdgeGroup* subclass from serialised state.

The method inspects the *type* field, allocates the corresponding class without executing subclass ***init***, and then manually restores any subtype-specific attributes. This keeps deserialisation deterministic even for complex group types that configure additional runtime callables.

Python

```
from_dict(data: dict[str, Any]) -> EdgeGroup
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._edge.EdgeGroup>	

Examples

Python

```
   payload = {"type": "EdgeGroup", "edges": [{"source_id": "a", "target_id":
"b"}]}
   group = EdgeGroup.from_dict(payload)
   assert isinstance(group, EdgeGroup)
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

register

Register a subclass so deserialisation can recover the right type.

Registration is typically performed via the decorator syntax applied to each concrete edge group. The registry stores classes by their ***name***, which must therefore remain stable across versions when persisted workflows are in circulation.

Python

```
register(subclass: type[EdgeGroup]) -> type[EdgeGroup]
```

Parameters

Expand table

Name	Description
subclass Required*	type[<xref:agent_framework._workflows._edge.EdgeGroup>]

Returns

Expand table

Type	Description
type[<xref:agent_framework._workflows._edge.EdgeGroup>]	

Examples

Python

```
@EdgeGroup.register
class CustomGroup(EdgeGroup):
    pass


assert EdgeGroup._TYPE_REGISTRY["CustomGroup"] is CustomGroup
```

to_dict

Serialise the switch-case group, capturing all case descriptors.

Each case is converted using *encode_value* to respect dataclass semantics as well as any nested serialisable structures.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

Python

```
group = SwitchCaseEdgeGroup(
    "router",
    [
        SwitchCaseEdgeGroupCase(lambda _: True, target_id="handler"),
        SwitchCaseEdgeGroupDefault(target_id="fallback"),
    ],
```

```
)
snapshot = group.to_dict()
assert len(snapshot["cases"]) == 2
```

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

selection_func

Expose the runtime callable used to select active fan-out targets.

When no selection function was supplied the property returns *None*, signalling that all targets must receive the payload.

Examples

Python

```
group = FanOutEdgeGroup("source", ["x", "y"], selection_func=None)
assert group.selection_func is None
```

source_executor_ids

Return the deduplicated list of upstream executor ids.

The property preserves order-of-first-appearance so the caller can rely on deterministic iteration when reconstructing graph topology.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.source_executor_ids == ["read"]
```

target_executor_ids

Return the ordered, deduplicated list of downstream executor ids.

Examples

Python

```
group = EdgeGroup([Edge("read", "write"), Edge("read", "archive")])
assert group.target_executor_ids == ["write", "archive"]
```

target_ids

Return a shallow copy of the configured downstream executor ids.

The list is defensively copied to prevent callers from mutating the internal state while still providing deterministic ordering.

Examples

Python

```
group = FanOutEdgeGroup("node", ["alpha", "beta"])
assert group.target_ids == ["alpha", "beta"]
```

cases

Python

```
cases: list[SwitchCaseEdgeGroupCase | SwitchCaseEdgeGroupDefault]
```

edges

Python

```
edges: list[Edge]
```

id

Python

```
id: str
```

selection_func_name

Python

```
selection_func_name: str | None
```

type

Python

```
type: str
```

SwitchCaseEdgeGroupCase Class

Persistable description of a single conditional branch in a switch-case.

Unlike the runtime *Case* object this serialisable variant stores only the target identifier and a descriptive name for the predicate. When the underlying callable is unavailable during deserialisation we substitute a proxy placeholder that fails loudly, ensuring the missing dependency is immediately visible.

Record the routing metadata for a conditional case branch.

Constructor

Python

```
SwitchCaseEdgeGroupCase()
```

Parameters

Expand table

Name	Description
condition Required*	Optional live predicate. When omitted we fall back to a placeholder that raises at runtime to highlight missing registrations.
target_id Required*	str Identifier of the executor that should handle messages when the predicate succeeds.
condition_name Required*	<xref:<xref:agent_framework.str	None> > Human-friendly label for the predicate used for diagnostics and on-disk persistence.

Examples
Required*
	Python
	`case = SwitchCaseEdgeGroupCase(lambda payload: payload["type"] == "csv", target_id="csv_handler")`
	`assert case.condition_name == "<lambda>"`

Keyword-Only Parameters

Expand table

Name	Description
condition_name	Default value: None

Methods

Expand table

__init__	Record the routing metadata for a conditional case branch.
__new__	
clone	
from_dict	Instantiate a case from its serialised dictionary payload.
from_json	
to_dict	Serialise the case metadata without the executable predicate.
to_json	

__init__

Record the routing metadata for a conditional case branch.

Python

```
__init__(condition: Callable[[Any], bool] | None, target_id: str, *,
condition_name: str | None = None) -> None
```

Parameters

Expand table

Name	Description
condition Required*	Callable[[Any], bool]	None Optional live predicate. When omitted we fall back to a placeholder that raises at runtime to highlight missing registrations.
target_id Required*	str Identifier of the executor that should handle messages when the predicate succeeds.
condition_name	str	None

Name	Description
Required*	Human-friendly label for the predicate used for diagnostics and on-disk persistence.

Examples
Required*
	Python
	`case = SwitchCaseEdgeGroupCase(lambda payload: payload["type"] == "csv", target_id="csv_handler")`
	`assert case.condition_name == "<lambda>"`

Keyword-Only Parameters

Expand table

Name	Description
condition_name	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Instantiate a case from its serialised dictionary payload.

Python

```
from_dict(data: dict[str, Any]) -> SwitchCaseEdgeGroupCase
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
SwitchCaseEdgeGroupCase	

Examples

Python

```
payload = {"target_id": "handler", "condition_name": "is_ready"}
case = SwitchCaseEdgeGroupCase.from_dict(payload)
assert case.target_id == "handler"
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

to_dict

Serialise the case metadata without the executable predicate.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

Python

```
case = SwitchCaseEdgeGroupCase(lambda _: True, target_id="handler")
assert case.to_dict()["target_id"] == "handler"
```

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

condition

Return the predicate associated with this case.

The placeholder installed during deserialisation raises a *RuntimeError* when invoked so that workflow authors are forced to provide the missing callable explicitly.

Examples

Python

```
   case = SwitchCaseEdgeGroupCase(None, target_id="missing",
condition_name="needs_registration")
   guard = case.condition
   try:
       guard({})
   except RuntimeError:
       pass
```

condition_name

Python

```
condition_name: str | None
```

target_id

Python

```
target_id: str
```

type

Python

```
type: str
```

SwitchCaseEdgeGroupDefault Class

Persistable descriptor for the fallback branch of a switch-case group.

The default branch is guaranteed to exist and is invoked when every other case predicate fails to match the payload.

Point the default branch toward the given executor identifier.

Constructor

Python

```
SwitchCaseEdgeGroupDefault()
```

Parameters

Expand table

Name	Description
target_id Required*	

Examples

Python

```
fallback = SwitchCaseEdgeGroupDefault(target_id="dead_letter")
assert fallback.target_id == "dead_letter"
```

Methods

Expand table

__init__	Point the default branch toward the given executor identifier.
__new__	
clone	

from_dict	Recreate the default branch from its persisted form.
from_json	
to_dict	Serialise the default branch metadata for persistence or logging.
to_json	

__init__

Point the default branch toward the given executor identifier.

Python

```
__init__(target_id: str) -> None
```

Parameters

Expand table

Name	Description
target_id Required*	str

Returns

Expand table

Type	Description
None	

Examples

Python

```
fallback = SwitchCaseEdgeGroupDefault(target_id="dead_letter")
assert fallback.target_id == "dead_letter"
```

__new__

Python

```
__new__(**kwargs)
```

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

Recreate the default branch from its persisted form.

Python

```
from_dict(data: dict[str, Any]) -> SwitchCaseEdgeGroupDefault
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
SwitchCaseEdgeGroupDefault	

Examples

Python

```
payload = {"target_id": "dead_letter", "type": "Default"}
fallback = SwitchCaseEdgeGroupDefault.from_dict(payload)
assert fallback.target_id == "dead_letter"
```

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

to_dict

Serialise the default branch metadata for persistence or logging.

Python

```
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Examples

Python

```
fallback = SwitchCaseEdgeGroupDefault("dead_letter")
assert fallback.to_dict()["type"] == "Default"
```

to_json

Python

```
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

target_id

Python

```
target_id: str
```

type

Python

```
type: str
```

TextContent Class

Represents text content in a chat.

Initializes a TextContent instance.

Constructor

Python

```
TextContent()
```

Parameters

Expand table

Name	Description
text Required*	The text content represented by this instance.

Keyword-Only Parameters

Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```
from agent_framework import TextContent
```

```python
# Create basic text content
text = TextContent(text="Hello, world!")
print(text.text)  # "Hello, world!"

# Concatenate text content
text1 = TextContent(text="Hello, ")
text2 = TextContent(text="world!")
combined = text1 + text2
print(combined.text)  # "Hello, world!"
```

Methods

Expand table

__init__	Initializes a TextContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a TextContent instance.

Python

```
__init__(text: str, *, additional_properties: dict[str, Any] | None = None,
raw_representation: Any | None = None, annotations: Sequence[CitationAnnotation |
MutableMapping[str, Any]] | None = None, **kwargs: Any)
```

Parameters

Expand table

Name	Description
text Required*	str The text content represented by this instance.
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
****kwargs**	Any additional keyword arguments.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}`

Name	Description
	• **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}
```

```
   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }
```

```
# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value	str

Name	Description
Required*	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

text

The text content represented by this instance.

type

The type of content, which is always "text" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

TextReasoningContent Class

Represents text reasoning content in a chat.

Remarks: This class and *TextContent* are superficially similar, but distinct.

Initializes a TextReasoningContent instance.

Constructor

Python

```
TextReasoningContent()
```

Parameters

Expand table

Name	Description
text Required*	The text content represented by this instance.

Keyword-Only Parameters

Expand table

Name	Description
protected_data	This property is used to store data from a provider that should be roundtripped back to the provider but that is not intended for human consumption. It is often encrypted or otherwise redacted information that is only intended to be sent back to the provider and not displayed to the user. It's possible for a TextReasoningContent to contain only *protected_data* and have an empty *text* property. This data also may be associated with the corresponding *text*, acting as a validation signature for it. Note that whereas *text* can be provider agnostic, *protected_data* is provider-specific, and is likely to only be understood by the provider that created it. The data is often represented as a more complex object, so it should be serialized to a string before storing so that the whole object is easily serializable without loss. Default value: None

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import TextReasoningContent

# Create reasoning content
reasoning = TextReasoningContent(text="Let me think step by step...")
print(reasoning.text)  # "Let me think step by step..."

# Concatenate reasoning content
reasoning1 = TextReasoningContent(text="First, ")
reasoning2 = TextReasoningContent(text="second, ")
combined = reasoning1 + reasoning2
print(combined.text)  # "First, second, "
```

Methods

Expand table

__init__	Initializes a TextReasoningContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available

	through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a TextReasoningContent instance.

Python

```
__init__(text: str | None, *, protected_data: str | None = None,
additional_properties: dict[str, Any] | None = None, raw_representation: Any |
None = None, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]]
| None = None, **kwargs: Any)
```

Parameters

Expand table

Name	Description
text Required*	str | None The text content represented by this instance.
protected_data Required*	str | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
protected_data	This property is used to store data from a provider that should be roundtripped back to the provider but that is not intended for human consumption. It is often encrypted or otherwise redacted information that is only intended to be sent back to the provider and not displayed to the user. It's possible for a TextReasoningContent to contain only *protected_data* and have an empty *text* property. This data also may be associated with the corresponding *text*, acting as a validation signature for it. Note that whereas *text* can be provider agnostic, *protected_data* is provider-specific, and is likely to only be understood by the provider that created it. The data is often represented as a more complex object, so it should be serialized to a string before storing so that the whole object is easily serializable without loss. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
annotations	Optional annotations associated with the content. Default value: None
****kwargs**	Any additional keyword arguments.

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated
```

```python
    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
    context_data = {
        "type": "agent_run_context",
        "messages": [{"role": "user", "text": "Hello"}],
        "is_streaming": False,
        "metadata": {"session_id": "abc123"},
        # agent and result are excluded from serialization
    }

    # Inject agent and result during middleware processing
    my_agent = BaseAgent(name="test-agent")
    dependencies = {
        "agent_run_context": {
            "agent": my_agent,
            "result": None,  # Will be populated during execution
        }
    }

    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None

Name	Description
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

text

The text content represented by this instance.

type

The type of content, which is always "text_reasoning" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

TextSpanRegion Class

Represents a region of text that has been annotated.

Initialize TextSpanRegion.

Constructor

Python

```
TextSpanRegion()
```

Keyword-Only Parameters

Expand table

Name	Description
start_index	The start index of the text span. Default value: None
end_index	The end index of the text span. Default value: None
****kwargs**	Additional keyword arguments.

Examples

Python

```
from agent_framework import TextSpanRegion

# Create a text span region
region = TextSpanRegion(start_index=0, end_index=10)
print(region.type)  # "text_span"
```

Methods

Expand table

__init__	Initialize TextSpanRegion.

__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize TextSpanRegion.

Python

```
__init__(*, start_index: int | None = None, end_index: int | None = None,
**kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
start_index Required*	int | None

Name	Description
end_index Required*	int | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
start_index	The start index of the text span. Default value: None
end_index	The end index of the text span. Default value: None
****kwargs**	Additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated
```

```python
    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```python
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
    context_data = {
        "type": "agent_run_context",
        "messages": [{"role": "user", "text": "Hello"}],
        "is_streaming": False,
        "metadata": {"session_id": "abc123"},
        # agent and result are excluded from serialization
    }

    # Inject agent and result during middleware processing
    my_agent = BaseAgent(name="test-agent")
    dependencies = {
        "agent_run_context": {
            "agent": my_agent,
            "result": None,  # Will be populated during execution
        }
    }

    # Reconstruct context with agent dependency for middleware chain
    context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
    # Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ToolMode Class

Defines if and how tools are used in a chat request.

Initialize ToolMode.

Constructor

Python

```
ToolMode()
```

Parameters

Expand table

Name	Description
mode	The tool mode - "auto", "required", or "none". Default value: none

Keyword-Only Parameters

Expand table

Name	Description
required_function_name	Optional function name for required mode. Default value: None

Examples

Python

```
from agent_framework import ToolMode

# Use predefined tool modes
auto_mode = ToolMode.AUTO  # Model decides when to use tools
required_mode = ToolMode.REQUIRED_ANY  # Model must use a tool
none_mode = ToolMode.NONE  # No tools allowed

# Require a specific function
specific_mode = ToolMode.REQUIRED(function_name="get_weather")
```

```
print(specific_mode.required_function_name)  # "get_weather"

# Compare modes
print(auto_mode == "auto")  # True
```

Methods

Expand table

REQUIRED	Returns a ToolMode that requires the specified function to be called.
__init__	Initialize ToolMode.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
serialize_model	Serializes the ToolMode to just the mode string.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

REQUIRED

Returns a ToolMode that requires the specified function to be called.

Python

```
REQUIRED(function_name: str | None = None) -> ToolMode
```

Parameters

Expand table

Name	Description
function_name	str | None Default value: None

Returns

Expand table

Type	Description
ToolMode	

__init__

Initialize ToolMode.

Python

```
__init__(mode: Literal['auto', 'required', 'none'] = 'none', *,
required_function_name: str | None = None) -> None
```

Parameters

Expand table

Name	Description
mode	Literal['auto', 'required', 'none'] The tool mode - "auto", "required", or "none". Default value: "none"
required_function_name Required*	str | None

Keyword-Only Parameters

Expand table

Name	Description
required_function_name	Optional function name for required mode. Default value: None

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```
Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated


# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"


# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}
```

```
# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

serialize_model

Serializes the ToolMode to just the mode string.

Python

```
serialize_model() -> str
```

Returns

Expand table

Type	Description
str	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

AUTO

Python

```
AUTO: ToolMode = ToolMode(mode='auto')
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

NONE

Python

```
NONE: ToolMode = ToolMode(mode='none')
```

REQUIRED_ANY

Python

```
REQUIRED_ANY: ToolMode = ToolMode(mode='required')
```

ToolProtocol Class

Represents a generic tool.

This protocol defines the interface that all tools must implement to be compatible with the agent framework. It is implemented by various tool classes such as HostedMCPTool, HostedWebSearchTool, and AIFunction's. A AIFunction is usually created by the *ai_function* decorator.

Since each connector needs to parse tools differently, users can pass a dict to specify a service-specific tool when no abstraction is available.

Constructor

Python

```
ToolProtocol()
```

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(*args, **kwargs)
```

__new__

Python

```
__new__(**kwargs)
```

Attributes

additional_properties

Additional properties associated with the tool.

Python

```
additional_properties: dict[str, Any] | None
```

description

A description of the tool, suitable for use in describing the purpose to a model.

Python

```
description: str
```

name

The name of the tool.

Python

```
name: str
```

TypeCompatibilityError Class

Exception raised when type incompatibility is detected between connected executors.

Constructor

Python

```
TypeCompatibilityError()
```

Parameters

Expand table

Name	Description
source_executor_id Required*	
target_executor_id Required*	
source_types Required*	
target_types Required*	

Methods

Expand table

__init__	
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

Python

```
__init__(source_executor_id: str, target_executor_id: str, source_types:
list[type[Any]], target_types: list[type[Any]])
```

Parameters

Expand table

Name	Description
source_executor_id Required*	str
target_executor_id Required*	str
source_types Required*	list[type[Any]]
target_types Required*	list[type[Any]]

__new__

Python

```
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

Python

```
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

Python

```
with_traceback()
```

Attributes

args

UriContent Class

Represents a URI content.

> **ⓘ Important**
>
> This is used for content that is identified by a URI, such as an image or a file.
>
> For (binary) data URIs, use DataContent instead.

Initializes a UriContent instance.

Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use *DataContent* instead.

Constructor

Python

```
UriContent()
```

Parameters

Expand table

Name	Description
`uri` Required*	The URI of the content.
`media_type` Required*	The media type of the content.

Keyword-Only Parameters

Expand table

Name	Description
`annotations`	Optional annotations associated with the content. Default value: None

Name	Description
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Examples

Python

```python
from agent_framework import UriContent

# Create URI content for an image
image_uri = UriContent(
    uri="https://example.com/image.png",
    media_type="image/png",
)

# Create URI content for a document
doc_uri = UriContent(
    uri="https://example.com/document.pdf",
    media_type="application/pdf",
)

# Check if it's an image
if image_uri.has_top_level_media_type("image"):
    print("This is an image URI")
```

Methods

Expand table

__init__	Initializes a UriContent instance. Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use *DataContent* instead.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports

	three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
has_top_level_media_type	Returns a boolean indicating if the media type has the specified top-level media type.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a UriContent instance.

Remarks: This is used for content that is identified by a URI, such as an image or a file. For (binary) data URIs, use *DataContent* instead.

Python

```
__init__(uri: str, media_type: str, *, annotations: Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None, additional_properties: dict[str, Any] | None = None, raw_representation: Any | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
uri Required*	str The URI of the content.
media_type Required*	str The media type of the content.

Name	Description
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
annotations	Optional annotations associated with the content. Default value: None
additional_properties	Optional additional properties associated with the content. Default value: None
raw_representation	Optional raw representation of the content. Default value: None
****kwargs**	Any additional keyword arguments.

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
`from_dict()`

Positional-Only Parameters

Expand table

Name	Description
`value` Required*	

Parameters

Expand table

Name	Description
`value` Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
`dependencies` Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
`dependencies`	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}`

Name	Description
	• **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }

   # Inject agent and result during middleware processing
```

```
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

has_top_level_media_type

Returns a boolean indicating if the media type has the specified top-level media type.

Python

```
has_top_level_media_type(top_level_media_type: Literal['application', 'audio', 'image', 'text']) -> bool
```

Parameters

Expand table

Name	Description
top_level_media_type Required*	Literal['application', 'audio', 'image', 'text'] The top-level media type to check for, allowed values: "image", "text", "application", "audio".

Returns

Expand table

Type	Description
bool	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

uri

The URI of the content, e.g., 'https://example.com/image.png '.

media_type

The media type of the content, e.g., 'image/png', 'application/json', etc.

type

The type of content, which is always "uri" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

UsageContent Class

Represents usage information associated with a chat request and response.

Initializes a UsageContent instance.

Constructor

Python

```
UsageContent()
```

Parameters

Expand table

Name	Description
details Required*	

Keyword-Only Parameters

Expand table

Name	Description
annotations	Default value: None
additional_properties	Default value: None
raw_representation	Default value: None

Examples

Python

```
from agent_framework import UsageContent, UsageDetails

# Create usage content
usage = UsageContent(
    details=UsageDetails(
        input_token_count=100,
```

```
            output_token_count=50,
            total_token_count=150,
        ),
    )
    print(usage.details.total_token_count)  # 150
```

Methods

Expand table

__init__	Initializes a UsageContent instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the instance to a dictionary. Extracts additional_properties fields to the root level.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes a UsageContent instance.

Python

```
__init__(details: UsageDetails | MutableMapping[str, Any], *, annotations:
Sequence[CitationAnnotation | MutableMapping[str, Any]] | None = None,
additional_properties: dict[str, Any] | None = None, raw_representation: Any |
None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
details Required*	UsageDetails | MutableMapping[str, Any]
annotations Required*	Sequence[CitationAnnotation | MutableMapping[str, Any]] | None
additional_properties Required*	dict[str, Any] | None
raw_representation Required*	Any | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
annotations	Default value: None
additional_properties	Default value: None
raw_representation	Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}`

Name	Description
	• **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}
```

```
   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
   from agent_framework._middleware import AgentRunContext
   from agent_framework import BaseAgent

   # AgentRunContext has INJECTABLE = {"agent", "result"}
   context_data = {
       "type": "agent_run_context",
       "messages": [{"role": "user", "text": "Hello"}],
       "is_streaming": False,
       "metadata": {"session_id": "abc123"},
       # agent and result are excluded from serialization
   }
```

```
# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value	str

Name	Description
Required*	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

details

The usage information, including input and output token counts, and any additional counts.

type

The type of content, which is always "usage" for this class.

annotations

Optional annotations associated with the content.

Python

```
annotations: list[CitationAnnotation] | None
```

additional_properties

Optional additional properties associated with the content.

raw_representation

Optional raw representation of the content.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties',
'raw_representation'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

UsageDetails Class

Provides usage details about a request/response.

Initializes the UsageDetails instance.

Constructor

Python

```
UsageDetails()
```

Parameters

Expand table

Name	Description
input_token_count	The number of tokens in the input. Default value: None
output_token_count	The number of tokens in the output. Default value: None
total_token_count	The total number of tokens used to produce the response. Default value: None

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional token counts, can be set by passing keyword arguments. They can be retrieved through the *additional_counts* property.

Examples

Python

```
from agent_framework import UsageDetails

# Create usage details
```

```
usage = UsageDetails(
    input_token_count=100,
    output_token_count=50,
    total_token_count=150,
)
print(usage.total_token_count)  # 150

# With additional counts
usage = UsageDetails(
    input_token_count=100,
    output_token_count=50,
    total_token_count=150,
    reasoning_tokens=25,
)
print(usage.additional_counts["reasoning_tokens"])  # 25

# Combine usage details
usage1 = UsageDetails(input_token_count=100, output_token_count=50)
usage2 = UsageDetails(input_token_count=200, output_token_count=100)
combined = usage1 + usage2
print(combined.input_token_count)  # 300
```

Methods

Expand table

__init__	Initializes the UsageDetails instance.
__new__	
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
to_dict	Convert the UsageDetails instance to a dictionary.
to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initializes the UsageDetails instance.

Python

```
__init__(input_token_count: int | None = None, output_token_count: int | None = None, total_token_count: int | None = None, **kwargs: int) -> None
```

Parameters

Expand table

Name	Description
input_token_count	int | None The number of tokens in the input. Default value: None
output_token_count	int | None The number of tokens in the output. Default value: None
total_token_count	int | None The total number of tokens used to produce the response. Default value: None
kwargs Required*	int

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional token counts, can be set by passing keyword arguments. They can be retrieved through the *additional_counts* property.

Returns

Expand table

Type	Description
None	

__new__

```Python
__new__(**kwargs)
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

```Python
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```python
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI


# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```
        # client is excluded from serialization
    }

    # Provide the OpenAI client during deserialization
    openai_client = AsyncOpenAI(api_key="your-api-key")
    dependencies = {"open_ai_chat_client": {"client": openai_client}}

    # The chat client is reconstructed with the OpenAI client injected
    chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
    # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
    from agent_framework import AIFunction
    from typing import Annotated


    # Define a function to be wrapped
    async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
        # In real implementation, this would call a weather API
        return f"Current weather in {location}: 72°F and sunny"


    # AIFunction has INJECTABLE = {"func"}
    function_data = {
        "type": "ai_function",
        "name": "get_weather",
        "description": "Get current weather for a location",
        # func is excluded from serialization
    }

    # Inject the actual function implementation during deserialization
    dependencies = {"ai_function": {"func": get_current_weather}}

    # Reconstruct the AIFunction with the callable injected
    weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
    # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
    from agent_framework._middleware import AgentRunContext
    from agent_framework import BaseAgent

    # AgentRunContext has INJECTABLE = {"agent", "result"}
```

```
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python
`from_json()`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

to_dict

Convert the UsageDetails instance to a dictionary.

Python

```
to_dict(*, exclude_none: bool = True, exclude: set[str] | None = None) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude_none Required*	bool
exclude Required*	set[str] | None

Keyword-Only Parameters

Expand table

Name	Description
exclude_none	Whether to exclude None values from the output. Default value: True
exclude	Set of field names to exclude from the output. Default value: None

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the UsageDetails instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

additional_counts

Represents well-known additional counts for usage. This is not an exhaustive list.

Remarks: To make it possible to avoid collisions between similarly-named, but unrelated, additional counts between different AI services, any keys not explicitly defined here should be prefixed with the name of the AI service, e.g., "openai." or "azure.". The separator "." was chosen because it cannot be a legal character in a JSON key.

Over time additional counts may be added to the base class.

input_token_count

The number of tokens in the input.

output_token_count

The number of tokens in the output.

total_token_count

The total number of tokens used to produce the response.

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'_extra_counts'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

ValidationTypeEnum Enum

Enumeration of workflow validation types.

Fields

Expand table

EDGE_DUPLICATION
EXECUTOR_DUPLICATION
GRAPH_CONNECTIVITY
HANDLER_OUTPUT_ANNOTATION
INTERCEPTOR_CONFLICT
TYPE_COMPATIBILITY

Workflow Class

A graph-based execution engine that orchestrates connected executors.

Overview

A workflow executes a directed graph of executors connected via edge groups using a Pregel-like model, running in supersteps until the graph becomes idle. Workflows are created using the WorkflowBuilder class - do not instantiate this class directly.

Execution Model

Executors run in synchronized supersteps where each executor:

- Is invoked when it receives messages from connected edge groups
- Can send messages to downstream executors via ctx.send_message()
- Can yield workflow-level outputs via ctx.yield_output()
- Can emit custom events via ctx.add_event()

Messages between executors are delivered at the end of each superstep and are not visible in the event stream. Only workflow-level events (outputs, custom events) and status events are observable to callers.

Input/Output Types

Workflow types are discovered at runtime by inspecting:

- Input types: From the start executor's input types
- Output types: Union of all executors' workflow output types Access these via the input_types and output_types properties.

Execution Methods

The workflow provides two primary execution APIs, each supporting multiple scenarios:

- **run()**: Execute to completion, returns WorkflowRunResult with all events
- **run_stream()**: Returns async generator yielding events as they occur

Both methods support:

- Initial workflow runs: Provide *message* parameter
- Checkpoint restoration: Provide *checkpoint_id* (and optionally *checkpoint_storage*)
- HIL continuation: Provide *responses* to continue after RequestInfoExecutor requests
- Runtime checkpointing: Provide *checkpoint_storage* to enable/override checkpointing for this run

State Management

Workflow instances contain states and states are preserved across calls to *run* and *run_stream*. To execute multiple independent runs, create separate Workflow instances via WorkflowBuilder.

External Input Requests

Executors within a workflow can request external input using *ctx.request_info()*:

1. Executor calls *ctx.request_info()* to request input
2. Executor implements *response_handler()* to process the response
3. Requests are emitted as RequestInfoEvent instances in the event stream
4. Workflow enters IDLE_WITH_PENDING_REQUESTS state
5. Caller handles requests and provides responses via the *send_responses* or *send_responses_streaming* methods
6. Responses are routed to the requesting executors and response handlers are invoked

Checkpointing

Checkpointing can be configured at build time or runtime:

Build-time (via WorkflowBuilder): workflow = WorkflowBuilder().with_checkpointing(storage).build()

Runtime (via run/run_stream parameters): result = await workflow.run(message, checkpoint_storage=runtime_storage)

When enabled, checkpoints are created at the end of each superstep, capturing:

- Executor states
- Messages in transit
- Shared state Workflows can be paused and resumed across process restarts using checkpoint storage.

Composition

Workflows can be nested using WorkflowExecutor, which wraps a child workflow as an executor. The nested workflow's input/output types become part of the WorkflowExecutor's types. When invoked, the WorkflowExecutor runs the nested workflow to completion and processes its outputs.

Initialize the workflow with a list of edges.

Constructor

Python

```
Workflow()
```

Parameters

Expand table

Name	Description
edge_groups Required*	A list of EdgeGroup instances that define the workflow edges.
executors Required*	A dictionary mapping executor IDs to Executor instances.
start_executor Required*	The starting executor for the workflow.
runner_context Required*	The RunnerContext instance to be used during workflow execution.
max_iterations	The maximum number of iterations the workflow will run for convergence. Default value: 100
name	Optional human-readable name for the workflow. Default value: None
description	Optional description of what the workflow does. Default value: None
kwargs Required*	Additional keyword arguments. Unused in this implementation.

Methods

Expand table

__init__	Initialize the workflow with a list of edges.
__new__	
as_agent	Create a WorkflowAgent that wraps this workflow. The returned agent converts standard agent inputs (strings, ChatMessage, or lists of these) into a list[ChatMessage] that is passed to the workflow's start executor. This conversion happens in WorkflowAgent._normalize_messages() which transforms: • str -> [ChatMessage(role=USER, text=str)] • ChatMessage -> [ChatMessage] • list[str | ChatMessage] -> list[ChatMessage] (with string elements converted) The workflow's start executor must accept list[ChatMessage] as an input type, otherwise initialization will fail with a ValueError.
clone	
from_dict	
from_json	
get_executors_list	Get the list of executors in the workflow.
get_start_executor	Get the starting executor of the workflow.
run	Run the workflow to completion and return all events. Unified non-streaming interface supporting initial runs and checkpoint restoration.
run_stream	Run the workflow and stream events. Unified streaming interface supporting initial runs and checkpoint restoration.
send_responses	Send responses back to the workflow.
send_responses_streaming	Send responses back to the workflow and stream the events generated by the workflow.
to_dict	Serialize the workflow definition into a JSON-ready dictionary.
to_json	Serialize the workflow definition to JSON.

__init__

Initialize the workflow with a list of edges.

Python

```
__init__(edge_groups: list[EdgeGroup], executors: dict[str, Executor],
start_executor: Executor, runner_context: RunnerContext, max_iterations: int =
DEFAULT_MAX_ITERATIONS, name: str | None = None, description: str | None = None,
**kwargs: Any)
```

Parameters

Expand table

Name	Description
edge_groups Required*	list[<xref:agent_framework._workflows._edge.EdgeGroup>] A list of EdgeGroup instances that define the workflow edges.
executors Required*	dict[str, Executor] A dictionary mapping executor IDs to Executor instances.
start_executor Required*	Executor The starting executor for the workflow.
runner_context Required*	RunnerContext The RunnerContext instance to be used during workflow execution.
max_iterations	int The maximum number of iterations the workflow will run for convergence. Default value: DEFAULT_MAX_ITERATIONS
name	str | None Optional human-readable name for the workflow. Default value: None
description	str | None Optional description of what the workflow does. Default value: None
kwargs Required*	Any Additional keyword arguments. Unused in this implementation.

__new__

Python

```
__new__(**kwargs)
```

as_agent

Create a WorkflowAgent that wraps this workflow.

The returned agent converts standard agent inputs (strings, ChatMessage, or lists of these) into a list[ChatMessage] that is passed to the workflow's start executor. This conversion happens in WorkflowAgent._normalize_messages() which transforms:

- str -> [ChatMessage(role=USER, text=str)]
- ChatMessage -> [ChatMessage]
- list[str | ChatMessage] -> list[ChatMessage] (with string elements converted)

The workflow's start executor must accept list[ChatMessage] as an input type, otherwise initialization will fail with a ValueError.

Python

```
as_agent(name: str | None = None) -> WorkflowAgent
```

Parameters

Expand table

Name	Description
name	str | None Optional name for the agent. If None, a default name will be generated. Default value: None

Returns

Expand table

Type	Description
WorkflowAgent	A WorkflowAgent instance that wraps this workflow.

Exceptions

Expand table

Type	Description
ValueError	If the workflow's start executor cannot handle list[ChatMessage] input.

clone

```
Python
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

from_dict

```
Python
from_dict(data: dict[str, Any]) -> TModel
```

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_json

Python

```
from_json(raw: str) -> TModel
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

get_executors_list

Get the list of executors in the workflow.

Python

```
get_executors_list() -> list[Executor]
```

Returns

Expand table

Type	Description
list[Executor]	

get_start_executor

Get the starting executor of the workflow.

Python

```
get_start_executor() -> Executor
```

Returns

Expand table

Type	Description
Executor	The starting executor instance.

run

Run the workflow to completion and return all events.

Unified non-streaming interface supporting initial runs and checkpoint restoration.

Python

```
async run(message: Any | None = None, *, checkpoint_id: str | None = None,
checkpoint_storage: CheckpointStorage | None = None, include_status_events: bool =
False, **kwargs: Any) -> WorkflowRunResult
```

Parameters

Expand table

Name	Description
message	Any | None

Name	Description
	Initial message for the start executor. Required for new workflow runs, should be None when resuming from checkpoint. Default value: None
checkpoint_id Required*	str | None ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. When resuming, checkpoint_storage must be provided (either at build time or runtime) to load the checkpoint.
checkpoint_storage Required*	CheckpointStorage | None Runtime checkpoint storage with two behaviors: • With checkpoint_id: Used to load and restore the specified checkpoint • Without checkpoint_id: Enables checkpointing for this run, overriding build-time configuration
include_status_events Required*	bool Whether to include WorkflowStatusEvent instances in the result list.
****kwargs** Required*	Any Additional keyword arguments to pass through to agent invocations. These are stored in SharedState and accessible in @ai_function tools via the >>**<<kwargs parameter.

Keyword-Only Parameters

Expand table

Name	Description
checkpoint_id	Default value: None
checkpoint_storage	Default value: None
include_status_events	Default value: False

Returns

Expand table

Type	Description
WorkflowRunResu	A WorkflowRunResult instance containing events generated during workflow

Type	Description
It	execution.

Exceptions

Expand table

Type	Description
ValueError	If both message and checkpoint_id are provided, or if neither is provided.
ValueError	If checkpoint_id is provided but no checkpoint storage is available (neither at build time nor runtime).
RuntimeError	If checkpoint restoration fails.

Examples

Initial run:

Python

```
result = await workflow.run("start message")
outputs = result.get_outputs()
```

With custom context for ai_functions:

Python

```
result = await workflow.run(
    "analyze data",
    custom_data={"endpoint": "https://api.example.com"},
    user_token={"user": "alice"},
)
```

Enable checkpointing at runtime:

Python

```
storage = FileCheckpointStorage("./checkpoints")
result = await workflow.run("start", checkpoint_storage=storage)
```

Resume from checkpoint (storage provided at build time):

Python

```
result = await workflow.run(checkpoint_id="cp_123")
```

Resume from checkpoint (storage provided at runtime):

Python

```
storage = FileCheckpointStorage("./checkpoints")
result = await workflow.run(checkpoint_id="cp_123", checkpoint_storage=storage)
```

run_stream

Run the workflow and stream events.

Unified streaming interface supporting initial runs and checkpoint restoration.

Python

```
async run_stream(message: Any | None = None, *, checkpoint_id: str | None = None,
checkpoint_storage: CheckpointStorage | None = None, **kwargs: Any) ->
AsyncIterable[WorkflowEvent]
```

Parameters

Expand table

Name	Description
message	Any | None Initial message for the start executor. Required for new workflow runs, should be None when resuming from checkpoint. Default value: None
checkpoint_id Required*	str | None ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. When resuming, checkpoint_storage must be provided (either at build time or runtime) to load the checkpoint.
checkpoint_storage Required*	CheckpointStorage | None Runtime checkpoint storage with two behaviors: • With checkpoint_id: Used to load and restore the specified checkpoint • Without checkpoint_id: Enables checkpointing for this run, overriding

Name	Description
	build-time configuration
**kwargs Required*	Any Additional keyword arguments to pass through to agent invocations. These are stored in SharedState and accessible in @ai_function tools via the >>** <<kwargs parameter.

Keyword-Only Parameters

Expand table

Name	Description
checkpoint_id	Default value: None
checkpoint_storage	Default value: None

Returns

Expand table

Type	Description
AsyncIterable[WorkflowEvent]	

Exceptions

Expand table

Type	Description
ValueError	If both message and checkpoint_id are provided, or if neither is provided.
ValueError	If checkpoint_id is provided but no checkpoint storage is available (neither at build time nor runtime).
RuntimeError	If checkpoint restoration fails.

Examples

Initial run:

Python

```
async for event in workflow.run_stream("start message"):
    process(event)
```

With custom context for ai_functions:

Python

```
async for event in workflow.run_stream(
    "analyze data",
    custom_data={"endpoint": "https://api.example.com"},
    user_token={"user": "alice"},
):
    process(event)
```

Enable checkpointing at runtime:

Python

```
storage = FileCheckpointStorage("./checkpoints")
async for event in workflow.run_stream("start", checkpoint_storage=storage):
    process(event)
```

Resume from checkpoint (storage provided at build time):

Python

```
async for event in workflow.run_stream(checkpoint_id="cp_123"):
    process(event)
```

Resume from checkpoint (storage provided at runtime):

Python

```
storage = FileCheckpointStorage("./checkpoints")
async for event in workflow.run_stream(checkpoint_id="cp_123",
checkpoint_storage=storage):
    process(event)
```

send_responses

Send responses back to the workflow.

Python

```
async send_responses(responses: dict[str, Any]) -> WorkflowRunResult
```

Parameters

Expand table

Name	Description
responses Required*	dict[str, Any] A dictionary where keys are request IDs and values are the corresponding response data.

Returns

Expand table

Type	Description
WorkflowRunResult	A WorkflowRunResult instance containing a list of events generated during the workflow execution.

send_responses_streaming

Send responses back to the workflow and stream the events generated by the workflow.

Python

```
async send_responses_streaming(responses: dict[str, Any]) ->
AsyncIterable[WorkflowEvent]
```

Parameters

Expand table

Name	Description
responses Required*	dict[str, Any] The responses to be sent back to the workflow, where keys are request IDs and values are the corresponding response data.

Returns

Expand table

Type	Description
AsyncIterable[WorkflowEvent]	

to_dict

Serialize the workflow definition into a JSON-ready dictionary.

Python
`to_dict() -> dict[str, Any]`

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

Serialize the workflow definition to JSON.

Python
`to_json() -> str`

Returns

Expand table

Type	Description
str	

Attributes

graph_signature_hash

input_types

Get the input types of the workflow.

The input types are the list of input types of the start executor.

Returns

Expand table

Type	Description
	A list of input types that the workflow can accept.

output_types

Get the output types of the workflow.

The output types are the list of all workflow output types from executors that have workflow output types.

Returns

Expand table

Type	Description
	A list of output types that the workflow can produce.

WorkflowAgent Class

An *Agent* subclass that wraps a workflow and exposes it as an agent.

Initialize the WorkflowAgent.

Constructor

```
Python
WorkflowAgent()
```

Parameters

Expand table

Name	Description
workflow Required*	The workflow to wrap as an agent.

Keyword-Only Parameters

Expand table

Name	Description
id	Unique identifier for the agent. If None, will be generated. Default value: None
name	Optional name for the agent. Default value: None
description	Optional description of the agent. Default value: None
****kwargs**	Additional keyword arguments passed to BaseAgent.

Methods

Expand table

__init__	Initialize the WorkflowAgent.
__new__	
as_tool	Create an AIFunction tool that wraps this agent.
deserialize_thread	Deserialize a thread from its serialized state.
from_dict	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
from_json	Create an instance from a JSON string. This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.
get_new_thread	Return a new AgentThread instance that is compatible with the agent.
merge_updates	Merge streaming updates into a single AgentRunResponse. Behavior: • Group updates by response_id; within each response_id, group by message_id and keep a dangling bucket for updates without message_id. • Convert each group (per message and dangling) into an intermediate AgentRunResponse via AgentRunResponse.from_agent_run_response_updates, then sort by created_at and merge. • Append messages from updates without any response_id at the end (global dangling), while aggregating metadata.
run	Get a response from the workflow agent (non-streaming). This method collects all streaming updates and merges them into a single response.
run_stream	Stream response updates from the workflow agent.
to_dict	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

to_json	Convert the instance to a JSON string. This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

__init__

Initialize the WorkflowAgent.

Python

```
__init__(workflow: Workflow, *, id: str | None = None, name: str | None = None,
description: str | None = None, **kwargs: Any) -> None
```

Parameters

Expand table

Name	Description
workflow Required*	Workflow The workflow to wrap as an agent.
id Required*	str | None
name Required*	str | None
description Required*	str | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
id	Unique identifier for the agent. If None, will be generated. Default value: None
name	Optional name for the agent. Default value: None

Name	Description
description	Optional description of the agent. Default value: None
****kwargs**	Additional keyword arguments passed to BaseAgent.

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

as_tool

Create an AIFunction tool that wraps this agent.

```
Python
as_tool(*, name: str | None = None, description: str | None = None, arg_name: str = 'task', arg_description: str | None = None, stream_callback: Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None = None) -> AIFunction[BaseModel, str]
```

Parameters

Expand table

Name	Description
name Required*	str | None
description Required*	str | None
arg_name	str

Name	Description
Required*	
arg_description Required*	str | None
stream_callback Required*	Callable[[AgentRunResponseUpdate], None] | Callable[[AgentRunResponseUpdate], Awaitable[None]] | None

Keyword-Only Parameters

Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

Expand table

Type	Description
AIFunction[<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

Expand table

Type	Description
TypeError	If the agent does not implement AgentProtocol.

Type	Description
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
   from agent_framework import ChatAgent

   # Create an agent
   agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

   # Convert the agent to a tool
   research_tool = agent.as_tool()

   # Use the tool with another agent
   coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • **Simple injection**: `{"<type>": {"<parameter>": value}}` • **Dict parameter injection**: `{"<type>": {"<dict-parameter>": {"<key>": value}}}` • **Instance-specific injection**: `{"<type>": {"<field>:<value>": {"<parameter>": value}}}` Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```

```
   # OpenAI chat client requires an AsyncOpenAI client instance
   # The client is marked as INJECTABLE = {"client"} in OpenAIBase

   # Serialized data contains only the model configuration
   client_data = {
       "type": "open_ai_chat_client",
       "model_id": "gpt-4o-mini",
       # client is excluded from serialization
   }

   # Provide the OpenAI client during deserialization
   openai_client = AsyncOpenAI(api_key="your-api-key")
   dependencies = {"open_ai_chat_client": {"client": openai_client}}

   # The chat client is reconstructed with the OpenAI client injected
   chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
   # Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
   from agent_framework import AIFunction
   from typing import Annotated


   # Define a function to be wrapped
   async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
       # In real implementation, this would call a weather API
       return f"Current weather in {location}: 72°F and sunny"


   # AIFunction has INJECTABLE = {"func"}
   function_data = {
       "type": "ai_function",
       "name": "get_weather",
       "description": "Get current weather for a location",
       # func is excluded from serialization
   }

   # Inject the actual function implementation during deserialization
   dependencies = {"ai_function": {"func": get_current_weather}}

   # Reconstruct the AIFunction with the callable injected
   weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
   # The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None,  # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

Parameters

Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

Expand table

Type	Description
<xref:agent_framework._serialization.TClass>	New instance of the class with any specified dependencies injected.

Exceptions

Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

Expand table

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

Expand table

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

merge_updates

Merge streaming updates into a single AgentRunResponse.

Behavior:

- Group updates by response_id; within each response_id, group by message_id and keep a dangling bucket for

  updates without message_id.

- Convert each group (per message and dangling) into an intermediate AgentRunResponse via AgentRunResponse.from_agent_run_response_updates, then sort by created_at and merge.

- Append messages from updates without any response_id at the end (global dangling), while aggregating metadata.

Python

```
static merge_updates(updates: list[AgentRunResponseUpdate], response_id: str) ->
AgentRunResponse
```

Parameters

Expand table

Name	Description
updates Required*	list[AgentRunResponseUpdate] The list of AgentRunResponseUpdate objects to merge.
response_id Required*	str The response identifier to set on the returned AgentRunResponse.

Returns

Expand table

Type	Description
AgentRunResponse	An AgentRunResponse with messages in processing order and aggregated metadata.

run

Get a response from the workflow agent (non-streaming).

This method collects all streaming updates and merges them into a single response.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None =
None, *, thread: AgentThread | None = None, checkpoint_id: str | None = None,
checkpoint_storage: CheckpointStorage | None = None, **kwargs: Any) ->
AgentRunResponse
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the workflow. Required for new runs, should be None when resuming from checkpoint. Default value: None
thread Required*	AgentThread | None
checkpoint_id Required*	str | None
checkpoint_storage Required*	CheckpointStorage | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread. If None, a new thread will be created. Default value: None
checkpoint_id	ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. Default value: None
checkpoint_storage	Runtime checkpoint storage. When provided with checkpoint_id, used to load and restore the checkpoint. When provided without checkpoint_id, enables checkpointing for this run. Default value: None
****kwargs**	

Name	Description
	Additional keyword arguments passed through to underlying workflow and ai_function tools.

Returns

Expand table

Type	Description
AgentRunResponse	The final workflow response as an AgentRunResponse.

run_stream

Stream response updates from the workflow agent.

Python

```
async run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, checkpoint_id: str | None = None, checkpoint_storage: CheckpointStorage | None = None, **kwargs: Any) -> AsyncIterable[AgentRunResponseUpdate]
```

Parameters

Expand table

Name	Description
messages	str | ChatMessage | list[str] | list[ChatMessage] | None The message(s) to send to the workflow. Required for new runs, should be None when resuming from checkpoint. Default value: None
thread Required*	AgentThread | None
checkpoint_id Required*	str | None
checkpoint_storage Required*	CheckpointStorage | None
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
thread	The conversation thread. If None, a new thread will be created. Default value: None
checkpoint_id	ID of checkpoint to restore from. If provided, the workflow resumes from this checkpoint instead of starting fresh. Default value: None
checkpoint_storage	Runtime checkpoint storage. When provided with checkpoint_id, used to load and restore the checkpoint. When provided without checkpoint_id, enables checkpointing for this run. Default value: None
****kwargs**	Additional keyword arguments passed through to underlying workflow and ai_function tools.

Returns

Expand table

Type	Description
AsyncIterable[AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with '_').

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->
dict[str, Any]
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization beyond the default exclusions (`DEFAULT_EXCLUDE` and `INJECTABLE`). Default value: None
exclude_none	Whether to exclude None values from the output. When True, None values are omitted from the dictionary. Defaults to True. Default value: True

Returns

Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs:
Any) -> str
```

Parameters

Expand table

Name	Description
exclude Required*	set[str] | None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
****kwargs**	Additional keyword arguments passed through to `json.dumps()`. Common options include `indent` for pretty-printing and `ensure_ascii` for Unicode handling.

Returns

Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

pending_requests

workflow

DEFAULT_EXCLUDE

```python
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

```python
INJECTABLE: ClassVar[set[str]] = {}
```

REQUEST_INFO_FUNCTION_NAME

```python
REQUEST_INFO_FUNCTION_NAME: ClassVar[str] = 'request_info'
```

additional_properties

```python
additional_properties: dict[str, Any]
```

middleware

```python
middleware: list[Middleware] | None
```

RequestInfoFunctionArgs Class

RequestInfoFunctionArgs(request_id: str, data: Any)

Constructor

Python

```
RequestInfoFunctionArgs()
```

Parameters

Expand table

Name	Description
request_id Required*	
data Required*	

Methods

Expand table

__init__
__new__
from_dict
from_json
to_dict
to_json

__init__

Python

```
__init__(request_id: str, data: Any) -> None
```

Parameters

Expand table

Name	Description
request_id Required*	str
data Required*	Any

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

```
Python
from_dict(payload: dict[str, Any]) -> RequestInfoFunctionArgs
```

Parameters

Expand table

Name	Description
payload Required*	dict[str, Any]

Returns

Expand table

Type	Description
RequestInfoFunctionArgs	

from_json

```
Python
from_json(raw: str) -> RequestInfoFunctionArgs
```

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
RequestInfoFunctionArgs	

to_dict

```
Python
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

```python
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

data

```python
data: Any
```

request_id

```python
request_id: str
```

WorkflowBuilder Class

A builder class for constructing workflows.

This class provides a fluent API for defining workflow graphs by connecting executors with edges and configuring execution parameters. Call build to create an immutable Workflow instance.

Initialize the WorkflowBuilder with an empty list of edges and no starting executor.

Constructor

Python

```
WorkflowBuilder()
```

Parameters

Expand table

Name	Description
max_iterations	Maximum number of iterations for workflow convergence. Default is 100. Default value: 100
name	Optional human-readable name for the workflow. Default value: None
description	Optional description of what the workflow does. Default value: None

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class UpperCaseExecutor(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())
```

```
    class ReverseExecutor(Executor):
        @handler
        async def process(self, text: str, ctx: WorkflowContext[Never, str]) -> None:
            await ctx.yield_output(text[::-1])


    # Build a workflow
    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: UpperCaseExecutor(id="upper"), name="UpperCase")
        .register_executor(lambda: ReverseExecutor(id="reverse"), name="Reverse")
        .add_edge("UpperCase", "Reverse")
        .set_start_executor("UpperCase")
        .build()
    )

    # Run the workflow
    events = await workflow.run("hello")
    print(events.get_outputs())  # ['OLLEH']
```

Methods

Expand table

__init__	Initialize the WorkflowBuilder with an empty list of edges and no starting executor.
__new__	
add_agent	Add an agent to the workflow by wrapping it in an AgentExecutor. This method creates an AgentExecutor that wraps the agent with the given parameters and ensures that subsequent uses of the same agent instance in other builder methods (like add_edge, set_start_executor, etc.) will reuse the same wrapped executor. Note: Agents adapt their behavior based on how the workflow is executed: • run_stream(): Agents emit incremental AgentRunUpdateEvent events as tokens are produced • run(): Agents emit a single AgentRunEvent containing the complete response
add_chain	Add a chain of executors to the workflow. The output of each executor in the chain will be sent to the next executor in the chain. The input types of each executor must be compatible with the output types of the previous executor.

	Cycles in the chain are not allowed, meaning an executor cannot appear more than once in the chain. Note: If executor instances are provided, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.
add_edge	Add a directed edge between two executors. The output types of the source and the input types of the target must be compatible. Messages sent by the source executor will be routed to the target executor. Note: If instances are provided for both source and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.
add_fan_in_edges	Add multiple edges from sources to a single target executor. The edges will be grouped together for synchronized processing, meaning the target executor will only be executed once all source executors have completed. The target executor will receive a list of messages aggregated from all source executors. Thus the input types of the target executor must be compatible with a list of the output types of the source executors. Note: If instances are provided for sources and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.
add_fan_out_edges	Add multiple edges to the workflow where messages from the source will be sent to all targets. The output types of the source and the input types of the targets must be compatible. Messages from the source will be broadcast to all target executors concurrently. Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.
add_multi_selection_edge_group	Add an edge group that represents a multi-selection execution model.

	The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to multiple target executors based on the provided selection function. The selection function should take a message and a list of target executor IDs, and return a list of executor IDs indicating which target executors should receive the message. Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.
add_switch_case_edge_group	Add an edge group that represents a switch-case statement. The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to one of the target executors based on the provided conditions. Think of this as a switch statement where each target executor corresponds to a case. Each condition function will be evaluated in order, and the first one that returns True will determine which target executor receives the message. The default case (if provided) will receive messages that fall through all conditions (i.e., no condition matched). Note: If instances are provided for source and case targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.
build	Build and return the constructed workflow. This method performs validation before building the workflow to ensure: • A starting executor has been set • All edges connect valid executors • The graph is properly connected • Type compatibility between connected executors
register_agent	Register an agent factory function for lazy initialization. This method allows you to register a factory function that creates an agent. The agent will be instantiated and wrapped in an AgentExecutor only when the workflow is built, enabling deferred initialization and potentially reducing startup time.
register_executor	Register an executor factory function for lazy initialization.

	This method allows you to register a factory function that creates an executor. The executor will be instantiated only when the workflow is built, enabling deferred initialization and potentially reducing startup time.
set_max_iterations	Set the maximum number of iterations for the workflow. When a workflow contains cycles, this limit prevents infinite loops by capping the total number of executor invocations. The default is 100 iterations.
set_start_executor	Set the starting executor for the workflow. The start executor is the entry point for the workflow. When the workflow is executed, the initial message will be sent to this executor.
with_checkpointing	Enable checkpointing with the specified storage. Checkpointing allows workflows to save their state periodically, enabling pause/resume functionality and recovery from failures. The checkpoint storage implementation determines where checkpoints are persisted.

__init__

Initialize the WorkflowBuilder with an empty list of edges and no starting executor.

Python

```
__init__(max_iterations: int = DEFAULT_MAX_ITERATIONS, name: str | None = None, description: str | None = None)
```

Parameters

Expand table

Name	Description
max_iterations	int Maximum number of iterations for workflow convergence. Default is 100. Default value: DEFAULT_MAX_ITERATIONS
name	str | None Optional human-readable name for the workflow. Default value: None
description	str | None Optional description of what the workflow does. Default value: None

__new__

Python

```
__new__(**kwargs)
```

add_agent

Add an agent to the workflow by wrapping it in an AgentExecutor.

This method creates an AgentExecutor that wraps the agent with the given parameters and ensures that subsequent uses of the same agent instance in other builder methods (like add_edge, set_start_executor, etc.) will reuse the same wrapped executor.

Note: Agents adapt their behavior based on how the workflow is executed:

- run_stream(): Agents emit incremental AgentRunUpdateEvent events as tokens are produced
- run(): Agents emit a single AgentRunEvent containing the complete response

Python

```
add_agent(agent: AgentProtocol, agent_thread: Any | None = None, output_response: bool = False, id: str | None = None) -> Self
```

Parameters

Expand table

Name	Description
agent Required*	AgentProtocol The agent to add to the workflow.
agent_thread	Any | None The thread to use for running the agent. If None, a new thread will be created. Default value: None
output_response	bool Whether to yield an AgentRunResponse as a workflow output when the agent completes. Default value: False
id	str | None

Name	Description
	A unique identifier for the executor. If None, the agent's name will be used if available. Default value: None

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Exceptions

Expand table

Type	Description
ValueError	If the provided id or agent name conflicts with an existing executor.

Examples

Python

```
from agent_framework import WorkflowBuilder
from agent_framework_anthropic import AnthropicAgent

# Create an agent
agent = AnthropicAgent(name="writer", model="claude-3-5-sonnet-20241022")

# Add the agent to a workflow
workflow = WorkflowBuilder().add_agent(agent,
output_response=True).set_start_executor(agent).build()
```

add_chain

Add a chain of executors to the workflow.

The output of each executor in the chain will be sent to the next executor in the chain. The input types of each executor must be compatible with the output types of the previous executor.

Cycles in the chain are not allowed, meaning an executor cannot appear more than once in the chain.

Note: If executor instances are provided, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_chain(executors: Sequence[Executor | AgentProtocol | str]) -> Self
```

Parameters

Expand table

Name	Description
executors Required*	Sequence[Executor	AgentProtocol	str] A list of executors or registered names of the executor factories to chain together.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class Step1(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())


class Step2(Executor):
    @handler
```

```
        async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
            await ctx.send_message(text[::-1])


    class Step3(Executor):
        @handler
        async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
            await ctx.yield_output(f"Final: {text}")


    # Chain executors in sequence
    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: Step1(id="step1"), name="step1")
        .register_executor(lambda: Step2(id="step2"), name="step2")
        .register_executor(lambda: Step3(id="step3"), name="step3")
        .add_chain(["step1", "step2", "step3"])
        .set_start_executor("step1")
        .build()
    )
```

add_edge

Add a directed edge between two executors.

The output types of the source and the input types of the target must be compatible. Messages sent by the source executor will be routed to the target executor.

Note: If instances are provided for both source and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_edge(source: Executor | AgentProtocol | str, target: Executor | AgentProtocol
| str, condition: Callable[[Any], bool] | None = None) -> Self
```

Parameters

Expand table

Name	Description
source Required*	Executor | AgentProtocol | str

Name	Description
target Required*	Executor | AgentProtocol | str
condition	Callable[[Any], bool] | None Default value: None

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class ProcessorA(Executor):
    @handler
    async def process(self, data: str, ctx: WorkflowContext[int]) -> None:
        await ctx.send_message(len(data))


class ProcessorB(Executor):
    @handler
    async def process(self, count: int, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(f"Processed {count} characters")


# Connect executors with an edge
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: ProcessorA(id="a"), name="ProcessorA")
    .register_executor(lambda: ProcessorB(id="b"), name="ProcessorB")
    .add_edge("ProcessorA", "ProcessorB")
    .set_start_executor("ProcessorA")
    .build()
)


# With a condition
def only_large_numbers(msg: int) -> bool:
```

```
        return msg > 100


    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: ProcessorA(id="a"), name="ProcessorA")
        .register_executor(lambda: ProcessorB(id="b"), name="ProcessorB")
        .add_edge("ProcessorA", "ProcessorB", condition=only_large_numbers)
        .set_start_executor("ProcessorA")
        .build()
    )
```

add_fan_in_edges

Add multiple edges from sources to a single target executor.

The edges will be grouped together for synchronized processing, meaning the target executor will only be executed once all source executors have completed.

The target executor will receive a list of messages aggregated from all source executors. Thus the input types of the target executor must be compatible with a list of the output types of the source executors.

Note: If instances are provided for sources and target, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_fan_in_edges(sources: Sequence[Executor | AgentProtocol | str], target:
Executor | AgentProtocol | str) -> Self
```

Parameters

Expand table

Name	Description
sources Required*	Sequence[Executor | AgentProtocol | str] A list of source executors or registered names of the source factories for the edges.
target Required*	Executor | AgentProtocol | str The target executor or registered name of the target factory for the edges.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class Producer(Executor):
    @handler
    async def produce(self, seed: int, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(f"result_{seed}")


class Aggregator(Executor):
    @handler
    async def aggregate(self, results: list[str], ctx: WorkflowContext[Never,
str]) -> None:
        combined = ", ".join(results)
        await ctx.yield_output(f"Combined: {combined}")


# Collect results from multiple producers
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: Producer(id="prod_1"), name="Producer1")
    .register_executor(lambda: Producer(id="prod_2"), name="Producer2")
    .register_executor(lambda: Aggregator(id="agg"), name="Aggregator")
    .add_fan_in_edges(["Producer1", "Producer2"], "Aggregator")
    .set_start_executor("Producer1")
    .build()
)
```

add_fan_out_edges

Add multiple edges to the workflow where messages from the source will be sent to all targets.

The output types of the source and the input types of the targets must be compatible. Messages from the source will be broadcast to all target executors concurrently.

Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_fan_out_edges(source: Executor | AgentProtocol | str, targets:
Sequence[Executor | AgentProtocol | str]) -> Self
```

Parameters

Expand table

Name	Description
source Required*	Executor | AgentProtocol | str The source executor or registered name of the source factory for the edges.
targets Required*	Sequence[Executor | AgentProtocol | str] A list of target executors or registered names of the target factories for the edges.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class DataSource(Executor):
    @handler
    async def generate(self, count: int, ctx: WorkflowContext[str]) -> None:
        for i in range(count):
            await ctx.send_message(f"data_{i}")


class ValidatorA(Executor):
    @handler
```

```
        async def validate(self, data: str, ctx: WorkflowContext) -> None:
            print(f"ValidatorA: {data}")


    class ValidatorB(Executor):
        @handler
        async def validate(self, data: str, ctx: WorkflowContext) -> None:
            print(f"ValidatorB: {data}")


    # Broadcast to multiple validators
    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: DataSource(id="source"), name="DataSource")
        .register_executor(lambda: ValidatorA(id="val_a"), name="ValidatorA")
        .register_executor(lambda: ValidatorB(id="val_b"), name="ValidatorB")
        .add_fan_out_edges("DataSource", ["ValidatorA", "ValidatorB"])
        .set_start_executor("DataSource")
        .build()
    )
```

add_multi_selection_edge_group

Add an edge group that represents a multi-selection execution model.

The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to multiple target executors based on the provided selection function.

The selection function should take a message and a list of target executor IDs, and return a list of executor IDs indicating which target executors should receive the message.

Note: If instances are provided for source and targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_multi_selection_edge_group(source: Executor | AgentProtocol | str, targets:
Sequence[Executor | AgentProtocol | str], selection_func: Callable[[Any,
list[str]], list[str]]) -> Self
```

Parameters

Expand table

Name	Description
source Required*	Executor | AgentProtocol | str The source executor or registered name of the source factory for the edge group.
targets Required*	Sequence[Executor | AgentProtocol | str] A list of target executors or registered names of the target factories for the edges.
selection_func Required*	Callable[[Any, list[str]], list[str]] A function that selects target executors for messages. Takes (message, list[executor_id]) and returns list[executor_id].

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```python
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler
from dataclasses import dataclass


@dataclass
class Task:
    priority: str
    data: str


class TaskDispatcher(Executor):
    @handler
    async def dispatch(self, text: str, ctx: WorkflowContext[Task]) -> None:
        priority = "high" if len(text) > 10 else "low"
        await ctx.send_message(Task(priority=priority, data=text))


class WorkerA(Executor):
    @handler
    async def process(self, task: Task, ctx: WorkflowContext) -> None:
        print(f"WorkerA processing: {task.data}")


class WorkerB(Executor):
    @handler
```

```
        async def process(self, task: Task, ctx: WorkflowContext) -> None:
            print(f"WorkerB processing: {task.data}")


    # Select workers based on task priority
    def select_workers(task: Task, available: list[str]) -> list[str]:
        if task.priority == "high":
            return available  # Send to all workers
        return [available[0]]  # Send to first worker only


    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: TaskDispatcher(id="dispatcher"),
name="TaskDispatcher")
        .register_executor(lambda: WorkerA(id="worker_a"), name="WorkerA")
        .register_executor(lambda: WorkerB(id="worker_b"), name="WorkerB")
        .add_multi_selection_edge_group(
            "TaskDispatcher",
            ["WorkerA", "WorkerB"],
            selection_func=select_workers,
        )
        .set_start_executor("TaskDispatcher")
        .build()
    )
```

add_switch_case_edge_group

Add an edge group that represents a switch-case statement.

The output types of the source and the input types of the targets must be compatible. Messages from the source executor will be sent to one of the target executors based on the provided conditions.

Think of this as a switch statement where each target executor corresponds to a case. Each condition function will be evaluated in order, and the first one that returns True will determine which target executor receives the message.

The default case (if provided) will receive messages that fall through all conditions (i.e., no condition matched).

Note: If instances are provided for source and case targets, they will be shared across all workflow instances created from the built Workflow. To avoid this, consider registering the executors and agents using *register_executor* and *register_agent* and referencing them by factory name for lazy initialization instead.

Python

```
add_switch_case_edge_group(source: Executor | AgentProtocol | str, cases:
Sequence[Case | Default]) -> Self
```

Parameters

Expand table

Name	Description
source Required*	Executor | AgentProtocol | str The source executor or registered name of the source factory for the edge group.
cases Required*	Sequence[Case | Default] A list of case objects that determine the target executor for each message.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```python
   from agent_framework import Executor, WorkflowBuilder, WorkflowContext,
handler, Case, Default
   from dataclasses import dataclass


   @dataclass
   class Result:
       score: int


   class Evaluator(Executor):
       @handler
       async def evaluate(self, text: str, ctx: WorkflowContext[Result]) -> None:
           await ctx.send_message(Result(score=len(text)))


   class HighScoreHandler(Executor):
       @handler
       async def handle(self, result: Result, ctx: WorkflowContext) -> None:
           print(f"High score: {result.score}")
```

```
    class LowScoreHandler(Executor):
        @handler
        async def handle(self, result: Result, ctx: WorkflowContext) -> None:
            print(f"Low score: {result.score}")


    # Route based on score value
    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: Evaluator(id="eval"), name="Evaluator")
        .register_executor(lambda: HighScoreHandler(id="high"),
name="HighScoreHandler")
        .register_executor(lambda: LowScoreHandler(id="low"),
name="LowScoreHandler")
        .add_switch_case_edge_group(
            "Evaluator",
            [
                Case(condition=lambda r: r.score > 10, target="HighScoreHandler"),
                Default(target="LowScoreHandler"),
            ],
        )
        .set_start_executor("Evaluator")
        .build()
    )
```

build

Build and return the constructed workflow.

This method performs validation before building the workflow to ensure:

- A starting executor has been set
- All edges connect valid executors
- The graph is properly connected
- Type compatibility between connected executors

Python

```
build() -> Workflow
```

Returns

Expand table

Type	Description
Workflow	An immutable Workflow instance ready for execution.

Exceptions

Expand table

Type	Description
ValueError	If starting executor is not set.
WorkflowValidationError	If workflow validation fails (includes EdgeDuplicationError, TypeCompatibilityError, and GraphConnectivityError subclasses).

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class MyExecutor(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(text.upper())


# Build and execute a workflow
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: MyExecutor(id="executor"), name="MyExecutor")
    .set_start_executor("MyExecutor")
    .build()
)

# The workflow is now immutable and ready to run
events = await workflow.run("hello")
print(events.get_outputs())  # ['HELLO']

# Workflows can be reused multiple times
events2 = await workflow.run("world")
print(events2.get_outputs())  # ['WORLD']
```

register_agent

Register an agent factory function for lazy initialization.

This method allows you to register a factory function that creates an agent. The agent will be instantiated and wrapped in an AgentExecutor only when the workflow is built, enabling

deferred initialization and potentially reducing startup time.

Python

```
register_agent(factory_func: Callable[[], AgentProtocol], name: str, agent_thread:
AgentThread | None = None, output_response: bool = False) -> Self
```

Parameters

Expand table

Name	Description
factory_func Required*	Callable[[], AgentProtocol] A callable that returns an AgentProtocol instance when called.
name Required*	str The name of the registered agent factory. This doesn't have to match the agent's internal name. But it must be unique within the workflow.
agent_thread	AgentThread | None The thread to use for running the agent. If None, a new thread will be created when the agent is instantiated. Default value: None
output_response	bool Whether to yield an AgentRunResponse as a workflow output when the agent completes. Default value: False

Returns

Expand table

Type	Description
Self	

Examples

Python

```
from agent_framework import WorkflowBuilder
from agent_framework_anthropic import AnthropicAgent
```

```
    # Build a workflow
    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: ..., name="SomeOtherExecutor")
        .register_agent(
            lambda: AnthropicAgent(name="writer", model="claude-3-5-sonnet-
20241022"),
            name="WriterAgent",
            output_response=True,
        )
        .add_edge("SomeOtherExecutor", "WriterAgent")
        .set_start_executor("SomeOtherExecutor")
        .build()
    )
```

register_executor

Register an executor factory function for lazy initialization.

This method allows you to register a factory function that creates an executor. The executor will be instantiated only when the workflow is built, enabling deferred initialization and potentially reducing startup time.

Python

```
register_executor(factory_func: Callable[[], Executor], name: str | list[str]) ->
Self
```

Parameters

Expand table

Name	Description
factory_func Required*	Callable[[], Executor] A callable that returns an Executor instance when called.
name Required*	str | list[str] The name(s) of the registered executor factory. This doesn't have to match the executor's ID, but it must be unique within the workflow.

Returns

Expand table

Type	Description
Self	

Examples

If multiple names are provided, the same factory function will be registered under each name.

Python

```
from agent_framework import WorkflowBuilder, Executor, WorkflowContext, handler


class LoggerExecutor(Executor):
    @handler
    async def log(self, message: str, ctx: WorkflowContext) -> None:
        print(f"Log: {message}")


# Register the same executor factory under multiple names
workflow = (
    WorkflowBuilder()
    .register_executor(lambda: LoggerExecutor(id="logger"), name=["ExecutorA",
"ExecutorB"])
    .set_start_executor("ExecutorA")
    .add_edge("ExecutorA", "ExecutorB")
    .build()
```

set_max_iterations

Set the maximum number of iterations for the workflow.

When a workflow contains cycles, this limit prevents infinite loops by capping the total number of executor invocations. The default is 100 iterations.

Python

```
set_max_iterations(max_iterations: int) -> Self
```

Parameters

Expand table

Name	Description
max_iterations	int

Name	Description
Required*	The maximum number of iterations the workflow will run for convergence.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class StepA(Executor):
    @handler
    async def process(self, count: int, ctx: WorkflowContext[int]) -> None:
        if count < 10:
            await ctx.send_message(count + 1)


class StepB(Executor):
    @handler
    async def process(self, count: int, ctx: WorkflowContext[int]) -> None:
        await ctx.send_message(count)


# Set a custom iteration limit for workflow with cycles
workflow = (
    WorkflowBuilder()
    .set_max_iterations(500)
    .register_executor(lambda: StepA(id="step_a"), name="StepA")
    .register_executor(lambda: StepB(id="step_b"), name="StepB")
    .add_edge("StepA", "StepB")
    .add_edge("StepB", "StepA")  # Cycle
    .set_start_executor("StepA")
    .build()
)
```

set_start_executor

Set the starting executor for the workflow.

The start executor is the entry point for the workflow. When the workflow is executed, the initial message will be sent to this executor.

Python

```
set_start_executor(executor: Executor | AgentProtocol | str) -> Self
```

Parameters

Expand table

Name	Description
executor Required*	Executor | AgentProtocol | str The starting executor, which can be an Executor instance, AgentProtocol instance, or the name of a registered executor factory.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler


class EntryPoint(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())


class Processor(Executor):
    @handler
    async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
        await ctx.yield_output(text)


workflow = (
```

```
        WorkflowBuilder()
        .register_executor(lambda: EntryPoint(id="entry"), name="EntryPoint")
        .register_executor(lambda: Processor(id="proc"), name="Processor")
        .add_edge("EntryPoint", "Processor")
        .set_start_executor("EntryPoint")
        .build()
    )
```

with_checkpointing

Enable checkpointing with the specified storage.

Checkpointing allows workflows to save their state periodically, enabling pause/resume functionality and recovery from failures. The checkpoint storage implementation determines where checkpoints are persisted.

Python

```
with_checkpointing(checkpoint_storage: CheckpointStorage) -> Self
```

Parameters

Expand table

Name	Description
checkpoint_storage Required*	CheckpointStorage The checkpoint storage implementation to use.

Returns

Expand table

Type	Description
Self	The WorkflowBuilder instance for method chaining.

Examples

Python

```
from typing_extensions import Never
from agent_framework import Executor, WorkflowBuilder, WorkflowContext, handler
from agent_framework import FileCheckpointStorage
```

```python
    class ProcessorA(Executor):
        @handler
        async def process(self, text: str, ctx: WorkflowContext[str]) -> None:
            await ctx.send_message(text.upper())


    class ProcessorB(Executor):
        @handler
        async def process(self, text: str, ctx: WorkflowContext[Never, str]) ->
None:
            await ctx.yield_output(text)


    # Enable checkpointing with file-based storage
    storage = FileCheckpointStorage("./checkpoints")
    workflow = (
        WorkflowBuilder()
        .register_executor(lambda: ProcessorA(id="proc_a"), name="ProcessorA")
        .register_executor(lambda: ProcessorB(id="proc_b"), name="ProcessorB")
        .add_edge("ProcessorA", "ProcessorB")
        .set_start_executor("ProcessorA")
        .with_checkpointing(storage)
        .build()
    )

    # Run with checkpoint saving
    events = await workflow.run("input")
```

WorkflowCheckpoint Class

Represents a complete checkpoint of workflow state.

Checkpoints capture the full execution state of a workflow at a specific point, enabling workflows to be paused and resumed.

> ⓘ Note
>
> The shared_state dict may contain reserved keys managed by the framework.
>
> See SharedState class documentation for details on reserved keys.

Constructor

Python

```
WorkflowCheckpoint()
```

Parameters

Expand table

Name	Description
checkpoint_id	Default value: <factory>
workflow_id	
timestamp	Default value: <factory>
messages	Default value: <factory>
shared_state	Default value: <factory>
pending_request_info_events	Default value: <factory>
iteration_count	Default value: 0
metadata	Default value: <factory>
version	Default value: 1.0

Methods

Expand table

__init__
__new__
from_dict
to_dict

__init__

Python

```
__init__(checkpoint_id: str = <factory>, workflow_id: str = '', timestamp: str =
<factory>, messages: dict[str, list[dict[str, typing.Any]]] = <factory>,
shared_state: dict[str, typing.Any] = <factory>, pending_request_info_events:
dict[str, dict[str, typing.Any]] = <factory>, iteration_count: int = 0, metadata:
dict[str, typing.Any] = <factory>, version: str = '1.0') -> None
```

Parameters

Expand table

Name	Description
checkpoint_id	str Default value: <factory>
workflow_id Required*	str
timestamp	str Default value: <factory>
messages	dict[str, list[dict[str, Any]]] Default value: <factory>
shared_state	dict[str, Any] Default value: <factory>
pending_request_info_events	dict[str, dict[str, Any]] Default value: <factory>
iteration_count	int Default value: 0

Name	Description
metadata	dict[str, Any] Default value: <factory>
version	str Default value: 1.0

Returns

Expand table

Type	Description
None	

__new__

```
Python
__new__(**kwargs)
```

from_dict

```
Python
from_dict(data: Mapping[str, Any]) -> WorkflowCheckpoint
```

Parameters

Expand table

Name	Description
data Required*	Mapping[str, Any]

Returns

Expand table

Type	Description
WorkflowCheckpoint	

to_dict

```python
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

Attributes

checkpoint_id

Unique identifier for this checkpoint

```python
checkpoint_id: str
```

iteration_count

Current iteration number when checkpoint was created

```python
iteration_count: int
```

messages

Messages exchanged between executors

```python
messages: dict[str, list[dict[str, Any]]]
```

metadata

Additional metadata (e.g., superstep info, graph signature)

Python

```
metadata: dict[str, Any]
```

pending_request_info_events

Python

```
pending_request_info_events: dict[str, dict[str, Any]]
```

shared_state

Complete shared state including user data and executor states. Executor states are stored under the reserved key '_executor_state'.

Python

```
shared_state: dict[str, Any]
```

timestamp

ISO 8601 timestamp when checkpoint was created

Python

```
timestamp: str
```

version

Checkpoint format version

Python

```
version: str
```

workflow_id

Identifier of the workflow this checkpoint belongs to

Python

```
workflow_id: str
```

WorkflowCheckpointSummary Class

Human-readable summary of a workflow checkpoint.

Constructor

Python

```
WorkflowCheckpointSummary()
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	
timestamp Required*	
iteration_count Required*	
targets Required*	
executor_ids Required*	
status Required*	
pending_request_info_events Required*	

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(checkpoint_id: str, timestamp: str, iteration_count: int, targets:
list[str], executor_ids: list[str], status: str, pending_request_info_events:
list[RequestInfoEvent]) -> None
```

Parameters

Expand table

Name	Description
checkpoint_id Required*	str
timestamp Required*	str
iteration_count Required*	int
targets Required*	list[str]
executor_ids Required*	list[str]
status Required*	str
pending_request_info_events Required*	list[RequestInfoEvent]

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

Attributes

checkpoint_id

Python

```
checkpoint_id: str
```

executor_ids

Python

```
executor_ids: list[str]
```

iteration_count

Python

```
iteration_count: int
```

pending_request_info_events

Python

```
pending_request_info_events: list[RequestInfoEvent]
```

status

Python

```
status: str
```

targets

Python

```
targets: list[str]
```

timestamp

Python

```
timestamp: str
```

WorkflowContext Class

Execution context that enables executors to interact with workflows and other executors.

Overview

WorkflowContext provides a controlled interface for executors to send messages, yield outputs, manage state, and interact with the broader workflow ecosystem. It enforces type safety through generic parameters while preventing direct access to internal runtime components.

Type Parameters

The context is parameterized to enforce type safety for different operations:

WorkflowContext (no parameters)

For executors that only perform side effects without sending messages or yielding outputs:

Python

```python
async def log_handler(message: str, ctx: WorkflowContext) -> None:
    print(f"Received: {message}")  # Only side effects
```

WorkflowContext[T_Out]

Enables sending messages of type T_Out to other executors:

Python

```python
async def processor(message: str, ctx: WorkflowContext[int]) -> None:
    result = len(message)
    await ctx.send_message(result)  # Send int to downstream executors
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```python
    async def dual_output(message: str, ctx: WorkflowContext[int, str]) -> None:
        await ctx.send_message(42)  # Send int message
        await ctx.yield_output("complete")  # Yield str workflow output
```

Union Types

Multiple types can be specified using union notation:

Python

```python
    async def flexible(message: str, ctx: WorkflowContext[int | str, bool | dict]) ->
None:
        await ctx.send_message("text")  # or send 42
        await ctx.yield_output(True)  # or yield {"status": "done"}
```

Initialize the executor context with the given workflow context.

Constructor

Python

```python
WorkflowContext()
```

Parameters

Expand table

Name	Description
executor Required*	The executor instance that this context belongs to.
source_executor_ids Required*	The IDs of the source executors that sent messages to this executor. This is a list to support fan_in scenarios where multiple sources send aggregated messages to the same executor.
shared_state Required*	The shared state for the workflow.
runner_context Required*	The runner context that provides methods to send messages and events.
trace_contexts	Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None

Name	Description
source_span_ids	Optional source span IDs from multiple sources for linking (not for nesting). Default value: None

Methods

Expand table

__init__	Initialize the executor context with the given workflow context.
__new__	
add_event	Add an event to the workflow context.
get_executor_state	Retrieve previously persisted state for this executor, if any.
get_sent_messages	Get all messages sent via send_message() during this handler execution.
get_shared_state	Get a value from the shared state.
get_source_executor_id	Get the ID of the source executor that sent the message to this executor.
get_yielded_outputs	Get all outputs yielded via yield_output() during this handler execution.
is_streaming	Check if the workflow is running in streaming mode.
request_info	Request information from outside of the workflow. Calling this method will cause the workflow to emit a RequestInfoEvent, carrying the provided request_data and request_type. External systems listening for such events can then process the request and respond accordingly. Executors must have the corresponding response handlers defined using the @response_handler decorator to handle the incoming responses.
send_message	Send a message to the workflow context.
set_executor_state	Store executor state in shared state under a reserved key. Executors call this with a JSON-serializable dict capturing the minimal state needed to resume. It replaces any previously stored state.
set_shared_state	Set a value in the shared state.
yield_output	Set the output of the workflow.

__init__

Initialize the executor context with the given workflow context.

Python

```
__init__(executor: Executor, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None)
```

Parameters

Expand table

Name	Description
executor Required*	Executor The executor instance that this context belongs to.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor. This is a list to support fan_in scenarios where multiple sources send aggregated messages to the same executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] | None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] | None Optional source span IDs from multiple sources for linking (not for nesting). Default value: None

__new__

Python

```
__new__(**kwargs)
```

add_event

Add an event to the workflow context.

Python

```
async add_event(event: WorkflowEvent) -> None
```

Parameters

Expand table

Name	Description
event Required*	WorkflowEvent

Returns

Expand table

Type	Description
None	

get_executor_state

Retrieve previously persisted state for this executor, if any.

Python

```
get_executor_state() -> dict[str, Any] | None
```

Returns

Expand table

Type	Description
dict[str, Any] | None	

get_sent_messages

Get all messages sent via send_message() during this handler execution.

Python

```
get_sent_messages() -> list[Any]
```

Returns

Expand table

Type	Description
list[Any]	A list of messages that were sent to downstream executors.

get_shared_state

Get a value from the shared state.

Python

```
async get_shared_state(key: str) -> Any
```

Parameters

Expand table

Name	Description
key Required*	str

Returns

Expand table

Type	Description
Any	

get_source_executor_id

Get the ID of the source executor that sent the message to this executor.

Python

```
get_source_executor_id() -> str
```

Returns

Expand table

Type	Description
str	

Exceptions

Expand table

Type	Description
RuntimeError	If there are multiple source executors, this method raises an error.

get_yielded_outputs

Get all outputs yielded via yield_output() during this handler execution.

Python

```
get_yielded_outputs() -> list[Any]
```

Returns

Expand table

Type	Description
list[Any]	A list of outputs that were yielded as workflow outputs.

is_streaming

Check if the workflow is running in streaming mode.

Python

```
is_streaming() -> bool
```

Returns

Expand table

Type	Description
bool	True if the workflow was started with run_stream(), False if started with run().

request_info

Request information from outside of the workflow.

Calling this method will cause the workflow to emit a RequestInfoEvent, carrying the provided request_data and request_type. External systems listening for such events can then process the request and respond accordingly.

Executors must have the corresponding response handlers defined using the @response_handler decorator to handle the incoming responses.

Python

```
async request_info(request_data: object, response_type: type) -> None
```

Parameters

Expand table

Name	Description
request_data Required*	object The data associated with the information request.
response_type Required*	type The expected type of the response, used for validation.

Returns

Expand table

Type	Description
None	

send_message

Send a message to the workflow context.

Python

```
async send_message(message: T_Out, target_id: str | None = None) -> None
```

Parameters

Expand table

Name	Description
message Required*	<xref:agent_framework._workflows._workflow_context.T_Out> The message to send. This must conform to the output type(s) declared on this context.
target_id	str | None The ID of the target executor to send the message to. If None, the message will be sent to all target executors. Default value: None

Returns

Expand table

Type	Description
None	

set_executor_state

Store executor state in shared state under a reserved key.

Executors call this with a JSON-serializable dict capturing the minimal state needed to resume. It replaces any previously stored state.

Python

```
set_executor_state(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any]

Returns

Expand table

Type	Description
None	

set_shared_state

Set a value in the shared state.

Python

```
async set_shared_state(key: str, value: Any) -> None
```

Parameters

Expand table

Name	Description
key Required*	str
value Required*	Any

Returns

Expand table

Type	Description
None	

yield_output

Set the output of the workflow.

Python

```
async yield_output(output: T_W_Out) -> None
```

Parameters

Expand table

Name	Description
output Required*	<xref:agent_framework._workflows._workflow_context.T_W_Out> The output to yield. This must conform to the workflow output type(s) declared on this context.

Returns

Expand table

Type	Description
None	

Attributes

shared_state

Get the shared state.

source_executor_ids

Get the IDs of the source executors that sent messages to this executor.

WorkflowErrorDetails Class

Structured error information to surface in error events/results.

Constructor

Python

```
WorkflowErrorDetails()
```

Parameters

Expand table

Name	Description
error_type Required*	
message Required*	
traceback	Default value: None
executor_id	Default value: None
extra	Default value: None

Methods

Expand table

__init__
__new__
from_exception

__init__

Python

```
__init__(error_type: str, message: str, traceback: str | None = None, executor_id:
```

```
str | None = None, extra: dict[str, Any] | None = None) -> None
```

Parameters

Expand table

Name	Description
error_type Required*	str
message Required*	str
traceback	str | None Default value: None
executor_id	str | None Default value: None
extra	dict[str, Any] | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

from_exception

Python

```
from_exception(exc: BaseException, *, executor_id: str | None = None, extra:
dict[str, Any] | None = None) -> WorkflowErrorDetails
```

Parameters

Expand table

Name	Description
exc Required*	BaseException
executor_id Required*	str | None
extra Required*	dict[str, Any] | None

Keyword-Only Parameters

Expand table

Name	Description
executor_id	Default value: None
extra	Default value: None

Returns

Expand table

Type	Description
WorkflowErrorDetails	

Attributes

error_type

Python

```
error_type: str
```

executor_id

Python

```
executor_id: str | None = None
```

extra

Python

```
extra: dict[str, Any] | None = None
```

message

Python

```
message: str
```

traceback

Python

```
traceback: str | None = None
```

WorkflowEvent Class

Base class for workflow events.

Initialize the workflow event with optional data.

Constructor

```
Python
WorkflowEvent()
```

Parameters

Expand table

Name	Description
data	Default value: None

Methods

Expand table

__init__	Initialize the workflow event with optional data.
__new__	

__init__

Initialize the workflow event with optional data.

```
Python
__init__(data: Any | None = None)
```

Parameters

Expand table

Name	Description
data	Any | None Default value: None

__new__

Python
`__new__(**kwargs)`

WorkflowEventSource Enum

Identifies whether a workflow event came from the framework or an executor.

Use *FRAMEWORK* for events emitted by built-in orchestration paths—even when the code that raises them lives in runner-related modules—and *EXECUTOR* for events surfaced by developer-provided executor implementations.

Fields

Expand table

EXECUTOR
FRAMEWORK

WorkflowExecutor Class

An executor that wraps a workflow to enable hierarchical workflow composition.

Overview

WorkflowExecutor makes a workflow behave as a single executor within a parent workflow, enabling nested workflow architectures. It handles the complete lifecycle of sub-workflow execution including event processing, output forwarding, and request/response coordination between parent and child workflows.

Execution Model

When invoked, WorkflowExecutor:

1. Starts the wrapped workflow with the input message
2. Runs the sub-workflow to completion or until it needs external input
3. Processes the sub-workflow's complete event stream after execution
4. Forwards outputs to the parent workflow as messages
5. Handles external requests by routing them to the parent workflow
6. Accumulates responses and resumes sub-workflow execution

Event Stream Processing

WorkflowExecutor processes events after sub-workflow completion:

Output Forwarding

All outputs from the sub-workflow are automatically forwarded to the parent:

When *allow_direct_output* is False (default):

Python

```python
# An executor in the sub-workflow yields outputs
await ctx.yield_output("sub-workflow result")

# WorkflowExecutor forwards to parent via ctx.send_message()
# Parent receives the output as a regular message
```

When *allow_direct_output* is True:

Request/Response Coordination

When sub-workflows need external information:

Python

```
   # An executor in the sub-workflow makes request
   request = MyDataRequest(query="user info")

   # WorkflowExecutor captures RequestInfoEvent and wraps it in a
SubWorkflowRequestMessage
   # then send it to the receiving executor in parent workflow. The executor in parent
workflow
   # can handle the request locally or forward it to an external source.
   # The WorkflowExecutor tracks the pending request, and implements a response
handler.
   # When the response is received, it executes the response handler to accumulate
responses
   # and resume the sub-workflow when all expected responses are received.
   # The response handler expects a SubWorkflowResponseMessage wrapping the response
data.
```

State Management

WorkflowExecutor maintains execution state across request/response cycles:

- Tracks pending requests by request_id
- Accumulates responses until all expected responses are received
- Resumes sub-workflow execution with complete response batch
- Handles concurrent executions and multiple pending requests

Type System Integration

WorkflowExecutor inherits its type signature from the wrapped workflow:

Input Types

Matches the wrapped workflow's start executor input types:

Python

```
   # If sub-workflow accepts str, WorkflowExecutor accepts str
```

```python
workflow_executor = WorkflowExecutor(my_workflow, id="wrapper")
assert workflow_executor.input_types == my_workflow.input_types
```

Output Types

Combines sub-workflow outputs with request coordination types:

Python

```python
# Includes all sub-workflow output types
# Plus SubWorkflowRequestMessage if sub-workflow can make requests
output_types = workflow.output_types + [SubWorkflowRequestMessage]  # if applicable
```

Error Handling

WorkflowExecutor propagates sub-workflow failures:

- Captures WorkflowFailedEvent from sub-workflow
- Converts to WorkflowErrorEvent in parent context
- Provides detailed error information including sub-workflow ID

Concurrent Execution Support

WorkflowExecutor fully supports multiple concurrent sub-workflow executions:

Per-Execution State Isolation

Each sub-workflow invocation creates an isolated ExecutionContext:

Python

```python
# Multiple concurrent invocations are supported
workflow_executor = WorkflowExecutor(my_workflow, id="concurrent_executor")

# Each invocation gets its own execution context
# Execution 1: processes input_1 independently
# Execution 2: processes input_2 independently
# No state interference between executions
```

Request/Response Coordination

Responses are correctly routed to the originating execution:

- Each execution tracks its own pending requests and expected responses
- Request-to-execution mapping ensures responses reach the correct sub-workflow
- Response accumulation is isolated per execution
- Automatic cleanup when execution completes

Memory Management

- Unlimited concurrent executions supported
- Each execution has unique UUID-based identification
- Cleanup of completed execution contexts
- Thread-safe state management for concurrent access

Important Considerations

Shared Workflow Instance: All concurrent executions use the same underlying workflow instance. For proper isolation, ensure that the wrapped workflow and its executors are stateless.

Python

```
# Avoid: Stateful executor with instance variables
class StatefulExecutor(Executor):
    def __init__(self):
        super().__init__(id="stateful")
        self.data = []  # This will be shared across concurrent executions!
```

Integration with Parent Workflows

Parent workflows can intercept sub-workflow requests:

Implementation Notes

- Sub-workflows run to completion before processing their results
- Event processing is atomic - all outputs are forwarded before requests
- Response accumulation ensures sub-workflows receive complete response batches
- Execution state is maintained for proper resumption after external requests
- Concurrent executions are fully isolated and do not interfere with each other

Initialize the WorkflowExecutor.

Constructor

Python

```
WorkflowExecutor()
```

Parameters

Expand table

Name	Description
workflow Required*	The workflow to execute as a sub-workflow.
id Required*	Unique identifier for this executor.
allow_direct_output	Whether to allow direct output from the sub-workflow. By default, outputs from the sub-workflow are sent to other executors in the parent workflow as messages. When this is set to true, the outputs are yielded directly from the WorkflowExecutor to the parent workflow's event stream. Default value: False

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional keyword arguments passed to the parent constructor.

Methods

Expand table

__init__	Initialize the WorkflowExecutor.
__new__	
can_handle	Override can_handle to only accept messages that the wrapped workflow can handle. This prevents the WorkflowExecutor from accepting messages that should go to other executors because the handler *process_workflow* has no type restrictions.
clone	
execute	Execute the executor with a given message and context parameters.

	• Do not call this method directly - it is invoked by the workflow engine. • Do not override this method. Instead, define handlers using @handler decorator.
from_dict	
from_json	
handle_response	Handle response from parent for a forwarded request. This handler accumulates responses and only resumes the sub-workflow when all expected responses have been received for that execution.
is_request_supported	Check if the executor supports request of the given type and handling a response of the given type.
on_checkpoint_restore	Restore the WorkflowExecutor state from a checkpoint snapshot.
on_checkpoint_save	Get the current state of the WorkflowExecutor for checkpointing purposes.
process_workflow	Execute the sub-workflow with raw input data. This handler starts a new sub-workflow execution. When the sub-workflow needs external information, it pauses and sends a request to the parent.
to_dict	
to_json	

__init__

Initialize the WorkflowExecutor.

Python

```
__init__(workflow: Workflow, id: str, allow_direct_output: bool = False, **kwargs: Any)
```

Parameters

Expand table

Name	Description
workflow Required*	Workflow The workflow to execute as a sub-workflow.
id	str

Name	Description
Required*	Unique identifier for this executor.
allow_direct_output	bool Whether to allow direct output from the sub-workflow. By default, outputs from the sub-workflow are sent to other executors in the parent workflow as messages. When this is set to true, the outputs are yielded directly from the WorkflowExecutor to the parent workflow's event stream. Default value: False
kwargs Required*	Any

Keyword-Only Parameters

Expand table

Name	Description
****kwargs**	Additional keyword arguments passed to the parent constructor.

__new__

```
Python
__new__(**kwargs)
```

can_handle

Override can_handle to only accept messages that the wrapped workflow can handle.

This prevents the WorkflowExecutor from accepting messages that should go to other executors because the handler *process_workflow* has no type restrictions.

```
Python
can_handle(message: Message) -> bool
```

Parameters

Expand table

Name	Description
message Required*	Message

Returns

Expand table

Type	Description
bool	

clone

Python

```
clone(*, deep: bool = True) -> Self
```

Parameters

Expand table

Name	Description
deep Required*	bool

Keyword-Only Parameters

Expand table

Name	Description
deep	Default value: True

Returns

Expand table

Type	Description
Self	

execute

Execute the executor with a given message and context parameters.

- Do not call this method directly - it is invoked by the workflow engine.
- Do not override this method. Instead, define handlers using @handler decorator.

Python

```
async execute(message: Any, source_executor_ids: list[str], shared_state:
SharedState, runner_context: RunnerContext, trace_contexts: list[dict[str, str]] |
None = None, source_span_ids: list[str] | None = None) -> None
```

Parameters

Expand table

Name	Description
message Required*	Any The message to be processed by the executor.
source_executor_ids Required*	list[str] The IDs of the source executors that sent messages to this executor.
shared_state Required*	SharedState The shared state for the workflow.
runner_context Required*	RunnerContext The runner context that provides methods to send messages and events.
trace_contexts	list[dict[str, str]] | None Optional trace contexts from multiple sources for OpenTelemetry propagation. Default value: None
source_span_ids	list[str] | None Optional source span IDs from multiple sources for linking. Default value: None

Returns

Expand table

Type	Description
None	An awaitable that resolves to the result of the execution.

from_dict

Python
`from_dict(data: dict[str, Any]) -> TModel`

Parameters

Expand table

Name	Description
data Required*	dict[str, Any]

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

from_json

Python
`from_json(raw: str) -> TModel`

Parameters

Expand table

Name	Description
raw Required*	str

Returns

Expand table

Type	Description
<xref:agent_framework._workflows._model_utils.TModel>	

handle_response

Handle response from parent for a forwarded request.

This handler accumulates responses and only resumes the sub-workflow when all expected responses have been received for that execution.

Python

```
async handle_response(response: SubWorkflowResponseMessage, ctx:
WorkflowContext[Any, Never]) -> None
```

Parameters

Expand table

Name	Description
response Required*	SubWorkflowResponseMessage The response to a previous request.
ctx Required*	WorkflowContext[Any, Never] The workflow context.

Returns

Expand table

Type	Description
None	

is_request_supported

Check if the executor supports request of the given type and handling a response of the given type.

Python

```
is_request_supported(request_type: type[Any], response_type: type[Any]) -> bool
```

Parameters

Expand table

Name	Description
request_type Required*	type[Any] The type of the request message
response_type Required*	type[Any] The type of the expected response message

Returns

Expand table

Type	Description
bool	True if a response handler is registered for the given request and response types, False otherwise

on_checkpoint_restore

Restore the WorkflowExecutor state from a checkpoint snapshot.

Python

```
async on_checkpoint_restore(state: dict[str, Any]) -> None
```

Parameters

Expand table

Name	Description
state Required*	dict[str, Any]

Returns

Expand table

Type	Description
None	

on_checkpoint_save

Get the current state of the WorkflowExecutor for checkpointing purposes.

Python

```
async on_checkpoint_save() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

process_workflow

Execute the sub-workflow with raw input data.

This handler starts a new sub-workflow execution. When the sub-workflow needs external information, it pauses and sends a request to the parent.

Python

```
async process_workflow(input_data: object, ctx: WorkflowContext[Any, Never]) ->
None
```

Parameters

Expand table

Name	Description
input_data Required*	object The input data to send to the sub-workflow.
ctx Required*	WorkflowContext[Any, Never] The workflow context from the parent.

Returns

Expand table

Type	Description
None	

to_dict

```
Python
to_dict() -> dict[str, Any]
```

Returns

Expand table

Type	Description
dict[str, Any]	

to_json

```
Python
to_json() -> str
```

Returns

Expand table

Type	Description
str	

Attributes

input_types

Get the input types based on the underlying workflow's input types plus WorkflowExecutor-specific types.

Returns

Expand table

Type	Description
	A list of input types that the WorkflowExecutor can accept.

output_types

Get the output types based on the underlying workflow's output types.

Returns

Expand table

Type	Description
	A list of output types that the underlying workflow can produce. Includes the SubWorkflowRequestMessage type if any executor in the sub-workflow is request-response capable.

workflow_output_types

Get the list of workflow output types that this executor can produce via yield_output().

Returns

Expand table

Type	Description
	A list of the workflow output types inferred from handlers' WorkflowContext[T, U] annotations.

WorkflowFailedEvent Class

Built-in lifecycle event emitted when a workflow run terminates with an error.

Constructor

Python

```
WorkflowFailedEvent()
```

Parameters

Expand table

Name	Description
details Required*	
data	Default value: None

Methods

Expand table

__init__
__new__

__init__

Python

```
__init__(details: WorkflowErrorDetails, data: Any | None = None)
```

Parameters

Expand table

Name	Description
details Required*	WorkflowErrorDetails
data	Any | None Default value: None

__new__

Python

```
__new__(**kwargs)
```

WorkflowOutputEvent Class

Event triggered when a workflow executor yields output.

Initialize the workflow output event.

Constructor

Python

```
WorkflowOutputEvent()
```

Parameters

Expand table

Name	Description
data Required*	The output yielded by the executor.
source_executor_id Required*	ID of the executor that yielded the output.

Methods

Expand table

__init__	Initialize the workflow output event.
__new__	

__init__

Initialize the workflow output event.

Python

```
__init__(data: Any, source_executor_id: str)
```

Parameters

Expand table

Name	Description
data Required*	Any The output yielded by the executor.
source_executor_id Required*	str ID of the executor that yielded the output.

__new__

Python

```
__new__(**kwargs)
```

WorkflowRunResult Class

Container for events generated during non-streaming workflow execution.

Overview

Represents the complete execution results of a workflow run, containing all events generated from start to idle state. Workflows produce outputs incrementally through ctx.yield_output() calls during execution.

Event Structure

Maintains separation between data-plane and control-plane events:

- Data-plane events: Executor invocations, completions, outputs, and requests (in main list)
- Control-plane events: Status timeline accessible via status_timeline() method

Key Methods

- get_outputs(): Extract all workflow outputs from the execution
- get_request_info_events(): Retrieve external input requests made during execution
- get_final_state(): Get the final workflow state (IDLE, IDLE_WITH_PENDING_REQUESTS, etc.)
- status_timeline(): Access the complete status event history

Constructor

Python

```
WorkflowRunResult()
```

Parameters

Expand table

Name	Description
events Required*	

Name	Description
`status_events`	Default value: None

Methods

Expand table

__init__	
__new__	
append	Append object to the end of the list.
clear	Remove all items from list.
copy	Return a shallow copy of the list.
count	Return number of occurrences of value.
extend	Extend list by appending elements from the iterable.
get_final_state	Return the final run state based on explicit status events. Returns the last WorkflowStatusEvent.state observed. Raises if none were emitted.
get_outputs	Get all outputs from the workflow run result.
get_request_info_events	Get all request info events from the workflow run result.
index	Return first index of value. Raises ValueError if the value is not present.
insert	Insert object before index.
pop	Remove and return item at index (default last). Raises IndexError if list is empty or index is out of range.
remove	Remove first occurrence of value. Raises ValueError if the value is not present.
reverse	Reverse *IN PLACE*.
sort	Sort the list in ascending order and return None. The sort is in-place (i.e. the list itself is modified) and stable (i.e. the order of two equal elements is maintained).

	If a key function is given, apply it once to each list item and sort them, ascending or descending, according to their function values. The reverse flag can be set to sort in descending order.
status_timeline	Return the list of status events emitted during the run (control-plane).

__init__

Python

```
__init__(events: list[WorkflowEvent], status_events: list[WorkflowStatusEvent] | None = None) -> None
```

Parameters

Expand table

Name	Description
events Required*	list[WorkflowEvent]
status_events	list[WorkflowStatusEvent] | None Default value: None

Returns

Expand table

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

append

Append object to the end of the list.

Python

```
append(object, /)
```

Positional-Only Parameters

Expand table

Name	Description
object Required*	

clear

Remove all items from list.

Python

```
clear()
```

copy

Return a shallow copy of the list.

Python

```
copy()
```

count

Return number of occurrences of value.

Python

```
count(value, /)
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	

extend

Extend list by appending elements from the iterable.

```
Python
extend(iterable, /)
```

Positional-Only Parameters

Expand table

Name	Description
iterable Required*	

get_final_state

Return the final run state based on explicit status events.

Returns the last WorkflowStatusEvent.state observed. Raises if none were emitted.

```
Python
get_final_state() -> WorkflowRunState
```

Returns

Expand table

Type	Description
WorkflowRunState	

get_outputs

Get all outputs from the workflow run result.

Python

```
get_outputs() -> list[Any]
```

Returns

Expand table

Type	Description
list[Any]	A list of outputs produced by the workflow during its execution.

get_request_info_events

Get all request info events from the workflow run result.

Python

```
get_request_info_events() -> list[RequestInfoEvent]
```

Returns

Expand table

Type	Description
list[RequestInfoEvent]	A list of RequestInfoEvent instances found in the workflow run result.

index

Return first index of value.

Raises ValueError if the value is not present.

Python

```
index(value, start=0, stop=9223372036854775807, /)
```

Positional-Only Parameters

Expand table

Name	Description
value Required*	
start	Default value: 0
stop	Default value: 9223372036854775807

insert

Insert object before index.

Python
`insert(index, object, /)`

Positional-Only Parameters

Expand table

Name	Description
index Required*	
object Required*	

pop

Remove and return item at index (default last).

Raises IndexError if list is empty or index is out of range.

Python
`pop(index=-1, /)`

Positional-Only Parameters

Expand table

Name	Description
index	Default value: -1

remove

Remove first occurrence of value.

Raises ValueError if the value is not present.

Python
`remove(value, /)`

Positional-Only Parameters

Expand table

Name	Description
value Required*	

reverse

Reverse *IN PLACE*.

Python
`reverse()`

sort

Sort the list in ascending order and return None.

The sort is in-place (i.e. the list itself is modified) and stable (i.e. the order of two equal elements is maintained).

If a key function is given, apply it once to each list item and sort them, ascending or descending, according to their function values.

The reverse flag can be set to sort in descending order.

Python

```
sort(*, key=None, reverse=False)
```

Keyword-Only Parameters

Expand table

Name	Description
key	Default value: None
reverse	Default value: False

status_timeline

Return the list of status events emitted during the run (control-plane).

Python

```
status_timeline() -> list[WorkflowStatusEvent]
```

Returns

Expand table

Type	Description
list[WorkflowStatusEvent]	

WorkflowRunState Enum

Run-level state of a workflow execution.

Semantics:

- STARTED: Run has been initiated and the workflow context has been created. This is an initial state before any meaningful work is performed. In this codebase we emit a dedicated *WorkflowStartedEvent* for telemetry, and typically advance the status directly to *IN_PROGRESS*. Consumers may still rely on *STARTED* for state machines that need an explicit pre-work phase.
- IN_PROGRESS: The workflow is actively executing (e.g., the initial message has been delivered to the start executor or a superstep is running). This status is emitted at the beginning of a run and can be followed by other statuses as the run progresses.
- IN_PROGRESS_PENDING_REQUESTS: Active execution while one or more request-for-information operations are outstanding. New work may still be scheduled while requests are in flight.
- IDLE: The workflow is quiescent with no outstanding requests and no more work to do. This is the normal terminal state for workflows that have finished executing, potentially having produced outputs along the way.
- IDLE_WITH_PENDING_REQUESTS: The workflow is paused awaiting external input (e.g., emitted a *RequestInfoEvent*). This is a non-terminal state; the workflow can resume when responses are supplied.
- FAILED: Terminal state indicating an error surfaced. Accompanied by a *WorkflowFailedEvent* with structured error details.
- CANCELLED: Terminal state indicating the run was cancelled by a caller or orchestrator. Not currently emitted by default runner paths but included for integrators/orchestrators that support cancellation.

Fields

Expand table

CANCELLED
FAILED
IDLE

IDLE_WITH_PENDING_REQUESTS
IN_PROGRESS
IN_PROGRESS_PENDING_REQUESTS
STARTED

WorkflowStartedEvent Class

Built-in lifecycle event emitted when a workflow run begins.

Initialize the workflow event with optional data.

Constructor

```python
WorkflowStartedEvent()
```

Parameters

Expand table

Name	Description
`data`	Default value: None

Methods

Expand table

__init__	Initialize the workflow event with optional data.
__new__	

__init__

Initialize the workflow event with optional data.

```python
__init__(data: Any | None = None)
```

Parameters

Expand table

Name	Description
data	Any | None Default value: None

__new__

Python
`__new__(**kwargs)`

WorkflowStatusEvent Class

Built-in lifecycle event emitted for workflow run state transitions.

Initialize the workflow status event with a new state and optional data.

Constructor

```
Python
WorkflowStatusEvent()
```

Parameters

Expand table

Name	Description
state Required*	The new state of the workflow run.
data	Optional additional data associated with the state change. Default value: None

Methods

Expand table

__init__	Initialize the workflow status event with a new state and optional data.
__new__	

__init__

Initialize the workflow status event with a new state and optional data.

```
Python
__init__(state: WorkflowRunState, data: Any | None = None)
```

Parameters

Expand table

Name	Description
state Required*	WorkflowRunState The new state of the workflow run.
data	Any | None Optional additional data associated with the state change. Default value: None

__new__

Python

```
__new__(**kwargs)
```

WorkflowValidationError Class

Base exception for workflow validation errors.

Constructor

```
Python
WorkflowValidationError()
```

Parameters

Expand table

Name	Description
message Required*	
validation_type Required*	

Methods

Expand table

__init__	
__new__	
add_note	Exception.add_note(note) – add a note to the exception
with_traceback	Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

__init__

```
Python
__init__(message: str, validation_type: ValidationTypeEnum)
```

Parameters

Expand table

Name	Description
message Required*	str
validation_type Required*	ValidationTypeEnum

__new__

```
Python
__new__(**kwargs)
```

add_note

Exception.add_note(note) – add a note to the exception

```
Python
add_note()
```

with_traceback

Exception.with_traceback(tb) – set self.**traceback** to tb and return self.

```
Python
with_traceback()
```

Attributes

args

WorkflowViz Class

A class for visualizing workflows using graphviz and Mermaid.

Initialize the WorkflowViz with a workflow.

Constructor

```
Python
WorkflowViz()
```

Parameters

Expand table

Name	Description
workflow Required*	The workflow to visualize.

Methods

Expand table

__init__	Initialize the WorkflowViz with a workflow.
__new__	
export	Export the workflow visualization to a file or return the file path.
save_pdf	Convenience method to save as PDF.
save_png	Convenience method to save as PNG.
save_svg	Convenience method to save as SVG.
to_digraph	Export the workflow as a DOT format digraph string.
to_mermaid	Export the workflow as a Mermaid flowchart string.

__init__

Initialize the WorkflowViz with a workflow.

```
Python
__init__(workflow: Workflow)
```

Parameters

Expand table

Name	Description
workflow Required*	Workflow The workflow to visualize.

__new__

```
Python
__new__(**kwargs)
```

export

Export the workflow visualization to a file or return the file path.

```
Python
export(format: Literal['svg', 'png', 'pdf', 'dot'] = 'svg', filename: str | None = None, include_internal_executors: bool = False) -> str
```

Parameters

Expand table

Name	Description
format	Literal['svg', 'png', 'pdf', 'dot'] The output format. Supported formats: 'svg', 'png', 'pdf', 'dot'. Default value: "svg"
filename	str | None Optional filename to save the output. If None, creates a temporary file. Default value: None

Name	Description
include_internal_executors	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

Expand table

Type	Description
str	The path to the saved file.

Exceptions

Expand table

Type	Description
ImportError	If graphviz is not installed.
ValueError	If an unsupported format is specified.

save_pdf

Convenience method to save as PDF.

Python

```
save_pdf(filename: str, include_internal_executors: bool = False) -> str
```

Parameters

Expand table

Name	Description
filename Required*	str The filename to save the PDF file.
include_internal_executors	bool Whether to include internal executors in the visualization. Default is False.

Name	Description
	Default value: False

Returns

Expand table

Type	Description
str	The path to the saved PDF file.

save_png

Convenience method to save as PNG.

Python

```
save_png(filename: str, include_internal_executors: bool = False) -> str
```

Parameters

Expand table

Name	Description
filename Required*	str The filename to save the PNG file.
include_internal_executors	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

Expand table

Type	Description
str	The path to the saved PNG file.

save_svg

Convenience method to save as SVG.

Python

```
save_svg(filename: str, include_internal_executors: bool = False) -> str
```

Parameters

Expand table

Name	Description
`filename` Required*	str The filename to save the SVG file.
`include_internal_executors`	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

Expand table

Type	Description
str	The path to the saved SVG file.

to_digraph

Export the workflow as a DOT format digraph string.

Python

```
to_digraph(include_internal_executors: bool = False) -> str
```

Parameters

Expand table

Name	Description
`include_internal_executors`	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

Expand table

Type	Description
str	A string representation of the workflow in DOT format.

to_mermaid

Export the workflow as a Mermaid flowchart string.

Python

```
to_mermaid(include_internal_executors: bool = False) -> str
```

Parameters

Expand table

Name	Description
include_internal_executors	bool Whether to include internal executors in the visualization. Default is False. Default value: False

Returns

Expand table

Type	Description
str	A string representation of the workflow in Mermaid flowchart syntax.